PRADITA University

Course Module · IT40713

Cloud-Native & Microservices Architecture

Alfa Yohannis



Master of Information Technology
Pradita University

Daftar Isi

1 Orientasi dan Bahasa Bersama Cloud-Native **11**

1.1 Pendahuluan 11
1.2 Cara Kerja Mata Kuliah dan Proyek Capstone 12
1.2.1 Empat Mata Kuliah, Satu Cetak Biru 12
1.2.2 Tonggak Capstone dan Artefak IT40713 13
1.3 Bahasa Bersama untuk Mahasiswa Non-Teknis 15
1.3.1 Aplikasi, Server, dan Basis Data 15
1.3.2 Jaringan dan Pusat Data 16
1.3.3 Cloud sebagai Layanan Komputasi yang Disewa 17
1.3.4 Perjalanan Satu Permintaan 18
1.4 Bahasa Bersama untuk Mahasiswa Teknis 19
1.4.1 Nilai Bisnis dan Cara Manajemen Menilainya 19
1.4.2 Struktur Biaya dan Siklus Anggaran 20
1.4.3 Risiko dan Kepatuhan sebagai Batasan Desain 21
1.4.4 Menulis Rekomendasi untuk Direksi 22
1.5 Kualitas Sistem dan Trade-off 23
1.5.1 Mengapa Kualitas Saling Tarik-Menarik 24
1.5.2 Menuliskan Alasan Keputusan 26
1.6 Arti Cloud-Native bagi Organisasi 27
1.6.1 Mitos yang Sering Menyertai Cloud-Native 28
1.6.2 Perjalanan Mata Kuliah 28
1.7 Aktivitas: Membentuk Tim dan Memetakan Pemicu Modernisasi 29
1.8 Ringkasan 31
1.9 Latihan 32

2 Model Layanan, Model Penerapan, dan Lokasi Data **37**

2.1 Pendahuluan 37
2.2 Lima Karakteristik Esensial Cloud 38
2.3 Model Layanan dan Shared Responsibility Model 41
2.4 Model Penerapan: Public, Private, Hybrid, dan Multi-Cloud 45

2.5 Region, Availability Zone, dan Region Cloud di Indonesia 48
2.6 Lokasi Data dan Kedaulatan Data dalam Regulasi Indonesia . . . 51
2.7 Aktivitas: Matriks Kendala Penempatan Workload 55
2.8 Ringkasan 58
2.9 Latihan 58

3 Ekonomi Cloud, FinOps, dan Keberlanjutan 63
3.1 Pendahuluan 63
3.2 CapEx, OpEx, dan Total Cost of Ownership 64
3.2.1 Komponen TCO dan Biaya yang Sering Terlupa 66
3.2.2 Menyusun Perbandingan TCO yang Jujur 66
3.3 Model Harga Cloud 68
3.3.1 Risiko Komitmen Berlebih 69
3.4 FinOps Framework 2026 71
3.4.1 Enam Prinsip 72
3.4.2 Fase Inform, Optimize, dan Operate 72
3.4.3 Persona, Domain, dan Capability 73
3.4.4 Scope dan Kategori Teknologi 73
3.5 Alokasi Biaya dan Unit Economics 75
3.5.1 Tagging sebagai Dasar Alokasi 75
3.5.2 Showback dan Chargeback 76
3.5.3 Spesifikasi FOCUS 77
3.5.4 Unit Economics 78
3.6 Keberlanjutan dan Jejak Karbon 80
3.6.1 Software Carbon Intensity 80
3.6.2 Region, Pemanfaatan, dan Jadwal Kerja 81
3.7 Aktivitas: Model Biaya Awal dan Sasaran Nilai Cloud 82
3.8 Ringkasan 83
3.9 Latihan 84

4 Kerangka Adopsi Cloud dan Strategi Migrasi 7R 89
4.1 Pendahuluan 89
4.2 Tiga Kerangka Adopsi Cloud 90
4.3 Penilaian Kesiapan dengan Enam Perspektif 93
4.4 Inventaris Portofolio Aplikasi dan Dependensi 96
4.5 Strategi Migrasi 7R 100
4.6 Landing Zone dan Cloud Center of Excellence 104
4.7 Aktivitas: Penilaian Kesiapan dan Portofolio 7R 106
4.8 Ringkasan 107
4.9 Latihan 108

5 Prinsip Cloud-Native dan Twelve-Factor App 113

5.1 Pendahuluan . . . 113
5.2 Definisi Cloud-Native menurut CNCF . . . 114
5.2.1 Perubahan dari Versi 1.0 ke Versi 1.1 . . . 115
5.2.2 Enam Sifat Sistem yang Dituju . . . 116
5.2.3 Cloud-Hosted dan Cloud-Native . . . 116
5.3 Twelve-Factor App dan Statusnya Sekarang . . . 117
5.3.1 Alat Penilaian, Bukan Dogma . . . 118
5.3.2 Tiga Kelompok Faktor . . . 118
5.4 Faktor tentang Kode, Konfigurasi, dan Backing Service . . . 118
5.4.1 Codebase dan Dependencies . . . 119
5.4.2 Config: Konfigurasi di Lingkungan . . . 120
5.4.3 Backing Services . . . 121
5.5 Faktor tentang Build, Proses, dan Skala . . . 123
5.5.1 Build, Release, Run . . . 123
5.5.2 Processes, Port Binding, Concurrency, dan Disposability . 123
5.5.3 Penskalaan Vertikal dan Horizontal . . . 124
5.5.4 Mengapa Sesi di Memori Server Menghambat Penskalaan 124
5.6 Faktor tentang Lingkungan, Log, dan Tugas Administratif . . . 126
5.6.1 Dev/Prod Parity . . . 127
5.6.2 Logs sebagai Aliran Kejadian . . . 127
5.6.3 Admin Processes . . . 128
5.7 Immutable Infrastructure dan Konfigurasi Deklaratif . . . 129
5.7.1 Server yang Dirawat dan Server yang Diganti . . . 129
5.7.2 Langkah Manual dan Konfigurasi Deklaratif . . . 130
5.8 Aktivitas: Kartu Skor Kesenjangan Cloud-Native . . . 131
5.9 Ringkasan . . . 134
5.10 Latihan . . . 134

6 Domain-Driven Design dan Batas Layanan 139
6.1 Pendahuluan . . . 139
6.2 Domain, Subdomain, dan Ubiquitous Language . . . 140
6.2.1 Ubiquitous Language . . . 143
6.3 Bounded Context . . . 144
6.3.1 Ukuran Bounded Context yang Wajar . . . 145
6.4 Context Map dan Pola Hubungannya . . . 147
6.4.1 Hubungan Antartim di Balik Pola . . . 149
6.5 EventStorming sebagai Lokakarya Kolaboratif . . . 150
6.5.1 Menjalankan Lokakarya Daring . . . 152
6.6 Dari Bounded Context ke Kandidat Layanan . . . 153
6.7 Aktivitas: Lokakarya EventStorming dan Context Map . . . 157
6.8 Ringkasan . . . 158
6.9 Latihan . . . 159

7 Monolit, Modular Monolith, dan Microservices 165
7.1 Pendahuluan . . . 165
7.2 Tiga Gaya Arsitektur: Monolit, Modular Monolith, dan Microservices . . . 166
7.2.1 Monolit . . . 167
7.2.2 Modular Monolith . . . 167
7.2.3 Microservices . . . 167
7.2.4 Spektrum, Bukan Pilihan Biner . . . 168
7.3 Coupling, Cohesion, dan Biaya Sistem Terdistribusi . . . 170
7.3.1 Coupling dan Cohesion . . . 170
7.3.2 Empat Jenis Coupling . . . 171
7.3.3 Fallacies of Distributed Computing . . . 172
7.4 Hukum Conway dan Struktur Tim . . . 176
7.4.1 Struktur Komunikasi Membentuk Struktur Sistem . . . 176
7.4.2 Manuver Conway Terbalik . . . 176
7.5 Kapan Microservices Tidak Tepat . . . 178
7.5.1 Pelajaran dari Segment . . . 178
7.5.2 Monolith First dan Pendapat Sebaliknya . . . 179
7.5.3 Membaca Kajian Secara Kritis . . . 180
7.6 Architecture Decision Record . . . 181
7.6.1 Fungsi dan Struktur ADR . . . 181
7.6.2 ADR sebagai Alat Komunikasi . . . 182
7.7 Aktivitas: Diagnosis Arsitektur dan ADR Pertama . . . 184
7.8 Ringkasan . . . 186
7.9 Latihan . . . 187

8 Container, Kubernetes, dan Serverless 191
8.1 Pendahuluan . . . 191
8.2 Virtual Machine dan Container . . . 192
8.2.1 Container Image dan Registry . . . 194
8.2.2 Batas Isolasi Container . . . 195
8.3 Orkestrasi Container dan Konsep Kubernetes . . . 196
8.3.1 Desired State dan Rekonsiliasi . . . 197
8.3.2 Pod, Deployment, dan Service . . . 198
8.3.3 Self-Healing dan Autoscaling . . . 198
8.3.4 Adopsi Kubernetes menurut Survei CNCF 2025 . . . 199
8.4 Managed Kubernetes dan PaaS . . . 201
8.5 Serverless dan Function as a Service . . . 204
8.5.1 Skala sampai Nol dan Biaya per Eksekusi . . . 205
8.5.2 Cold Start, Batas Waktu, dan Ketergantungan pada Penyedia . . . 205
8.6 Memilih Runtime dan Mengelola Vendor Lock-in . . . 207

8.6.1 Kriteria Memilih Runtime 207
8.6.2 Matriks Keputusan dari SaaS sampai VM 208
8.6.3 Vendor Lock-in, Portabilitas, dan Biaya Keluar 209
8.7 Aktivitas: Matriks Keputusan Runtime 210
8.8 Ringkasan 212
8.9 Latihan 213

9 Komunikasi Antarlayanan dan Event-Driven Architecture 217
9.1 Pendahuluan 217
9.2 Komunikasi Sinkron dan Asinkron 218
9.3 Antrean Pesan, Publish/Subscribe, dan Event Broker 221
9.4 Pola Event: Notification, State Transfer, dan Event Sourcing . . . 223
9.5 Choreography dan Orchestration 227
9.6 Pengiriman At-Least-Once dan Idempotensi 229
9.7 Service Mesh dan Batas dengan Desain API 231
9.8 Aktivitas: Peta Interaksi Layanan dan Katalog Event 234
9.9 Ringkasan 235
9.10 Latihan 236

10 Data dalam Microservices: Saga dan Outbox 241
10.1 Pendahuluan 241
10.2 Database per Service dan Basis Data Bersama 242
10.2.1 Privat Tidak Berarti Server Sendiri 244
10.2.2 Cara Layanan Lain Memperoleh Data 245
10.3 Transaksi Terdistribusi dan Eventual Consistency 246
10.3.1 Batas Two-Phase Commit 246
10.3.2 Eventual Consistency 247
10.4 Saga dan Transaksi Kompensasi 248
10.4.1 Kompensasi Bukan Rollback 249
10.4.2 Choreography dan Orchestration pada Saga 250
10.4.3 Saga Tidak Punya Isolasi 250
10.5 Pola Transactional Outbox 252
10.5.1 Dua Cara Menjalankan Penerbit 252
10.5.2 Konsekuensi: Event Dapat Datang Dua Kali 253
10.6 CQRS dan Read Model untuk Pelaporan 255
10.7 Kepemilikan Data Pasien: Satu Pasien, Satu Rekam Medis 257
10.7.1 Persetujuan dan Hak Akses sebagai Batasan Desain . . . 258
10.8 Aktivitas: Peta Kepemilikan Data dan Rancangan Saga 259
10.9 Ringkasan 261
10.10 Latihan 262

11 Observability, SRE, dan SLO 267

11.1 Pendahuluan 267
11.2 Monitoring dan Observability 268
11.2.1 Menelusuri Gangguan di Sistem Terdistribusi 269
11.2.2 Observability bagi Manajemen dan Auditor 270
11.3 Log, Metrik, Trace, dan OpenTelemetry 271
11.3.1 OpenTelemetry sebagai Standar Terbuka 273
11.3.2 Biaya Telemetri dan Cara Mengendalikannya 274
11.4 SLI, SLO, dan SLA 275
11.4.1 Memilih SLI dari Perjalanan Pengguna 276
11.4.2 Menghitung Waktu Tidak Tersedia yang Diizinkan 277
11.4.3 Mengapa Target 100 Persen Hampir Selalu Keliru 278
11.5 Error Budget dan Kebijakannya 279
11.5.1 Kebijakan Error Budget 281
11.5.2 Error Budget sebagai Alat Negosiasi 282
11.6 Praktik SRE: Toil, Alerting, Insiden, dan Postmortem 282
11.6.1 Toil dan Batasnya 283
11.6.2 Alerting yang Bermakna 283
11.6.3 Penanganan Insiden 284
11.6.4 Postmortem tanpa Menyalahkan 285
11.6.5 Pelaporan ke Direksi 286
11.7 Aktivitas: SLO Layanan Kritis dan Rancangan Observability . . . 287
11.8 Ringkasan 289
11.9 Latihan 290

12 Resiliensi dan Disaster Recovery 295
12.1 Pendahuluan 295
12.2 Mode Kegagalan dalam Sistem Terdistribusi 296
12.2.1 Kegagalan Parsial dan Latensi Tinggi 297
12.2.2 Kegagalan Berantai dan Titik Kegagalan Tunggal 297
12.2.3 Kegagalan dari Penyedia dan Pihak Ketiga 299
12.3 Pola Resiliensi 300
12.3.1 Timeout 300
12.3.2 Retry dengan Backoff dan Jitter 301
12.3.3 Circuit Breaker 301
12.3.4 Bulkhead 302
12.3.5 Graceful Degradation 303
12.4 Multi-AZ dan Multi-Region 305
12.4.1 Biaya dan Kerumitan Replikasi Data 306
12.4.2 Kendala Lokasi Data di Indonesia 306
12.5 RTO, RPO, dan Strategi Disaster Recovery 307
12.5.1 RTO dan RPO sebagai Keputusan Bisnis 307
12.5.2 Analisis Dampak Bisnis Sederhana 308

12.5.3 Empat Strategi Disaster Recovery 309
12.6 Ransomware dan Cadangan yang Tidak Dapat Diubah 311
12.6.1 Prinsip Cadangan yang Tahan Ransomware 312
12.6.2 Pelajaran dari PDNS 2 Surabaya 313
12.7 Chaos Engineering dan Game Day 314
12.7.1 Chaos Engineering 314
12.7.2 Game Day 315
12.8 Aktivitas: Rencana Resiliensi dan DR 316
12.9 Ringkasan 318
12.10 Latihan 319

13 DevOps, GitOps, dan Platform Engineering 325
13.1 Pendahuluan 325
13.2 DevOps dan Metrik DORA 326
13.2.1 DevOps sebagai Budaya dan Praktik 327
13.2.2 Metrik DORA 327
13.2.3 Rilis yang Jarang Tidak Membuat Lebih Aman 328
13.2.4 Temuan DORA 2025: AI sebagai Penguat 329
13.3 CI/CD dan Strategi Rilis 330
13.3.1 Continuous Integration, Continuous Delivery, dan Continuous Deployment 331
13.3.2 Strategi Rilis: Blue-Green dan Canary 332
13.3.3 Keamanan Rantai Pasok Perangkat Lunak 333
13.4 Infrastructure as Code dan GitOps 335
13.4.1 Infrastructure as Code 335
13.4.2 Empat Prinsip OpenGitOps 336
13.4.3 Jejak Audit dari GitOps 337
13.5 Platform Engineering dan Internal Developer Platform 339
13.5.1 Platform, IDP, dan Platform Engineering 339
13.5.2 Tim Platform Bukan Loket Tiket 341
13.5.3 Platform Engineering Maturity Model 341
13.6 Team Topologies 342
13.6.1 Empat Tipe Tim dan Beban Kognitif 342
13.6.2 Tiga Mode Interaksi 343
13.7 Aktivitas: Model Operasi Delivery dan Peta Jalan Platform Internal 345
13.8 Ringkasan 347
13.9 Latihan 348

14 Peta Jalan Modernisasi dan Tata Kelola Cloud 353
14.1 Pendahuluan 353
14.2 Pola Strangler Fig dan Modernisasi Bertahap 354
14.2.1 Risiko Penggantian Sekaligus 355

14.2.2 Cara Kerja Strangler Fig 355
14.2.3 Anticorruption Layer selama Masa Transisi 357
14.2.4 Memilih Irisan Pertama 357
14.3 Menyusun Gelombang Modernisasi 359
14.3.1 Gelombang, Dependensi, Risiko, dan Nilai 360
14.3.2 Gerbang Keputusan di antara Gelombang 362
14.4 Guardrail dan Policy as Code pada Landing Zone 364
14.4.1 Guardrail sebagai Batas, Bukan Gerbang 364
14.4.2 Policy as Code 365
14.4.3 Katalog Guardrail Medikara Health 366
14.5 FinOps pada Fase Operate 368
14.5.1 Anggaran, Deteksi Anomali, dan Kebijakan Tagging 369
14.5.2 Tinjauan Biaya per Unit dan Executive Strategy Alignment 370
14.6 Kedaulatan Data dan Sovereign Cloud 371
14.6.1 Tiga Dimensi Kedaulatan 371
14.6.2 Pilihan Sovereign Cloud 372
14.6.3 Status Regulasi Indonesia per Oktober 2026 372
14.7 Metrik Peta Jalan dan Pelaporan Kemajuan 374
14.7.1 Mengapa Jumlah Aplikasi yang Dipindahkan Menyesatkan 374
14.7.2 Empat Keluarga Metrik 374
14.7.3 Laporan Ringkas untuk Direksi 375
14.8 Aktivitas: Peta Jalan Modernisasi dan Guardrail Tata Kelola . . . 376
14.9 Ringkasan 378
14.10 Latihan 379

Daftar Pustaka **383**

Bab 1

Orientasi dan Bahasa Bersama Cloud-Native

Tujuan Pembelajaran

- Menjelaskan dengan bahasa sehari-hari istilah dasar sistem informasi: aplikasi, server, basis data, jaringan, pusat data, dan *cloud*.
- Mengaitkan keputusan teknologi dengan nilai bisnis, biaya, risiko, dan regulasi.
- Merumuskan masalah teknologi informasi di organisasi pilihan tim sebagai pemicu modernisasi.

1.1 Pendahuluan

Rapat direksi Medikara Health, jaringan layanan kesehatan fiktif yang dipakai sepanjang modul ini, membahas laporan triwulan pertama 2026. Isinya tidak menggembirakan. Antrean pendaftaran rawat jalan masih panjang, terutama pada Senin pagi. Lima rumah sakit menjalankan Sistem Informasi Manajemen Rumah Sakit (SIMRS) dengan versi yang berbeda-beda, sehingga setiap sambungan baru ke mitra harus dibangun lima kali. Biaya integrasi terus naik. Di akhir rapat, seorang direktur bertanya: "Kalau semua sistem dipindahkan ke *cloud*, apakah masalah ini selesai?"

Pertanyaan itu wajar dan terdengar di banyak organisasi. Jawabannya tidak sesederhana "ya" atau "tidak". Antrean bisa disebabkan alur pendaftaran, bukan oleh server. Lima versi SIMRS tetap lima versi walaupun dipindahkan ke pusat data lain. Biaya integrasi bahkan bisa naik bila setiap sambungan harus dibangun

ulang di tempat baru. Mata kuliah ini melatih cara menjawab pertanyaan semacam itu dengan runtut: berangkat dari masalah organisasi, menimbang pilihan beserta *trade-off*-nya, lalu menuliskan justifikasi yang dapat dipertanggungjawabkan di depan manajemen.

Bab ini menyiapkan tiga bekal. Pertama, cara kerja mata kuliah dan kaitannya dengan proyek capstone. Kedua, bahasa bersama bagi kelas yang berlatar campuran: istilah dasar sistem informasi bagi mahasiswa non-teknis, serta istilah nilai bisnis, biaya, dan risiko bagi mahasiswa teknis. Ketiga, kualitas sistem yang saling tarik-menarik dan arti *cloud-native* bagi organisasi. Rujukan utamanya adalah definisi *cloud computing* dari National Institute of Standards and Technology (NIST) [91], definisi *cloud native* dari Cloud Native Computing Foundation (CNCF) [38], serta pandangan Richards dan Ford bahwa setiap keputusan arsitektur adalah *trade-off* [124].

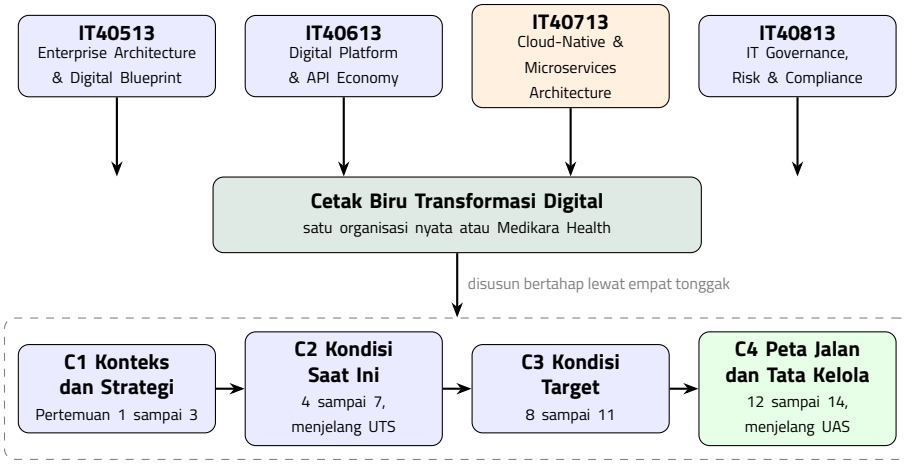
1.2 Cara Kerja Mata Kuliah dan Proyek Capstone

Mata kuliah ini dipelajari secara mandiri lewat modul dan slide. Tatap muka hanya berlangsung pada hari Sabtu, dan waktunya dipakai untuk membahas progres proyek capstone, bukan untuk mengulang isi modul. Artinya, setiap bab dibaca sebelum Sabtu, Aktivitas dikerjakan bersama tim, lalu draf artefaknya dibawa ke kelas untuk ditinjau. Pola ini mirip bimbingan tugas akhir: pertemuan dengan pembimbing dipakai untuk membahas draf yang sudah ada, bukan untuk mendengar ceramah. Dengan 8 tim dan tatap muka 150 menit, setiap tim mendapat sekitar 15 menit. Tim yang datang tanpa draf menghabiskan waktu itu untuk menjelaskan rencana, sehingga umpan baliknya umum dan pekerjaan pekan berikutnya menumpuk.

1.2.1 Empat Mata Kuliah, Satu Cetak Biru

Mata kuliah IT40713 adalah satu dari empat mata kuliah peminatan Digital Enterprise Architecture. Keempatnya menyusun satu proyek capstone bersama, yaitu Cetak Biru Transformasi Digital untuk satu organisasi. Gambar 1.1 memperlihatkan pembagian tugasnya. IT40513 Enterprise Architecture & Digital Blueprint menyusun *capability map*, arsitektur bisnis, dan arsitektur aplikasi. IT40613 Digital Platform & API Economy merancang platform dan Application Programming Interface (API) untuk ekosistem mitra. IT40813 IT Governance, Risk & Compliance menyusun tata kelola, daftar risiko, dan kepatuhan. IT40713 bertanggung jawab atas strategi *cloud*, penilaian portofolio aplikasi, arsitektur target berbasis *cloud-native*, serta rencana operasi dan modernisasinya.

Pembagian ini mirip renovasi sebuah rumah sakit yang melibatkan empat



Gambar 1.1: Empat mata kuliah menyumbang ke satu cetak biru yang disusun lewat empat tonggak capstone

konsultan. Arsitek menggambar denah dan alur ruang. Konsultan akses menata pintu masuk, loket, dan jalur bagi tamu dari luar. Konsultan struktur dan utilitas menghitung fondasi, listrik, dan air. Pengawas perizinan memastikan gedung memenuhi aturan keselamatan. Keempatnya memakai satu set gambar. Bila masing-masing memakai gambar sendiri, hasilnya pintu yang menghadap tembok.

Dua contoh kaitan antarmata kuliah:

1. *Capability map* Medikara Health dari IT40513 berisi 15 kapabilitas tingkat 2. Peta itu menjadi masukan lokakarya batas layanan pada Pertemuan 6.
2. Rencana pemulihan bencana pada Pertemuan 12 menetapkan, misalnya, data rekam medis elektronik boleh hilang paling banyak 15 menit. Angka itu masuk ke daftar risiko dan rencana kelangsungan bisnis di IT40813.

1.2.2 Tonggak Capstone dan Artefak IT40713

Proyek capstone disusun bertahap lewat empat tonggak bersama. Tabel 1.1 merangkum tonggak tersebut beserta artefak yang disumbangkan mata kuliah ini. Tonggak C2 ditinjau menjelang Ujian Tengah Semester (UTS), dan tonggak C4 dipresentasikan menjelang Ujian Akhir Semester (UAS).

Pertanyaan kemudian muncul: artefak sebanyak itu disusun untuk organisasi mana? Tim memilih organisasi nyata, sebaiknya tempat kerja salah satu anggota, atau kasus Medikara Health yang disediakan modul ini. Organisasi nyata memberi pengalaman yang lebih kaya, tetapi menuntut izin dan kehati-hatian.

Dua hal pada organisasi nyata sering tertukar, yaitu izin dan anonimisasi. Pem-

Tabel 1.1: Tonggak capstone dan artefak yang disumbangkan IT40713

Tonggak	Pertemuan	Artefak dari IT40713
C1 Konteks dan Strategi	1 sampai 3	Profil organisasi dan daftar pemicu modernisasi, matriks kendala penempatan *workload*, model biaya awal
C2 Kondisi Saat Ini	4 sampai 7	Penilaian kesiapan *cloud*, portofolio aplikasi, kartu skor kesenjangan *cloud-native*, peta domain, catatan keputusan gaya arsitektur
C3 Kondisi Target	8 sampai 11	Matriks pilihan *runtime*, peta interaksi layanan, peta kepemilikan data, target keandalan layanan kritis
C4 Peta Jalan dan Tata Kelola	12 sampai 14	Rencana pemulihan bencana, model operasi tim, peta jalan modernisasi beserta *guardrail* tata kelolanya

bagiannya begini. Izin menjawab pertanyaan "bolehkah organisasi ini dipelajari dan ditulis?". Izin diberikan secara tertulis oleh pejabat yang berwenang, misalnya kepala divisi teknologi informasi (TI) atau sekretaris perusahaan. Anonimisasi menjawab pertanyaan "apa yang boleh tampil di dokumen?". Nama organisasi, nama orang, angka keuangan yang sensitif, dan data pribadi diganti atau disamarkan. Izin tanpa anonimisasi tetap berisiko membocorkan informasi. Anonimisasi tanpa izin tetap melanggar kepercayaan organisasi.

Keputusan memakai organisasi nyata punya dua konsekuensi. Pertama, tim wajib menyimpan bukti izin tertulis sebelum wawancara pertama. Kedua, data pribadi pasien, pelanggan, atau karyawan tidak boleh dipakai sama sekali, termasuk dalam bentuk tangkapan layar. Tim yang tidak mendapat izin sampai Pertemuan 2 berpindah ke kasus Medikara Health.

Kasus Medikara Health: Profil Singkat

PT Medikara Nusa Sehat, disingkat Medikara Health, adalah organisasi **fiktif**. Seluruh angka dan kejadiannya rekaan dan dipakai bersama oleh keempat mata kuliah peminatan. Medikara Health adalah jaringan layanan kesehatan swasta dengan 5 rumah sakit (Jakarta, Bekasi, Depok, Bogor, dan Bandung), 18 klinik pratama, 1 laboratorium rujukan, dan 12 apotek. Karyawannya sekitar 4.800 orang, dengan sekitar 1,2 juta kunjungan pasien per tahun, atau rata-rata sekitar 3.300 kunjungan per hari.

Setiap rumah sakit menjalankan SIMRS monolitik *on-premise* dengan versi berbeda karena berasal dari akuisisi. Keuangan memakai satu Enterprise Resource Planning (ERP) terpusat. Aplikasi pasien Medikara+ untuk janji temu dan telekonsultasi dibuat oleh vendor. Integrasi ke Badan Penyeleng-

gara Jaminan Sosial (BPJS) Kesehatan dan ke SATUSEHAT, yang berbasis Health Level Seven Fast Healthcare Interoperability Resources (HL7 FHIR), dibangun secara *point-to-point*. Data pasien tersebar, dan banyak laporan manajemen masih disusun dengan *spreadsheet*.
Strategi 2026 sampai 2030 berjudul "Satu Pasien, Satu Rekam Medis". Pemicunya antara lain insiden kebocoran data pasien skala kecil lewat vendor pada 2025, temuan audit internal tentang kontrol akses yang lemah, antrean layanan yang panjang, dan biaya integrasi yang tinggi.

1.3 Bahasa Bersama untuk Mahasiswa Non-Teknis

Kelas ini berisi mahasiswa dari bisnis, manajemen, akuntansi, hukum, kesehatan, dan TI. Diskusi tim sering macet bukan karena perbedaan pendapat, tetapi karena istilah yang dipakai tidak dipahami bersama. Perjalanan permintaan menghubungkan aplikasi pasien, layanan vendor, SIMRS rumah sakit, dan basis data yang dikelolanya. Bagian ini menjelaskan enam istilah dasar dengan bahasa sehari-hari, lalu menelusuri perjalanan satu permintaan dari ponsel pasien sampai ke basis data.

Gambar 1.2 merangkum hubungan komponen dan konsep utama bab ini.

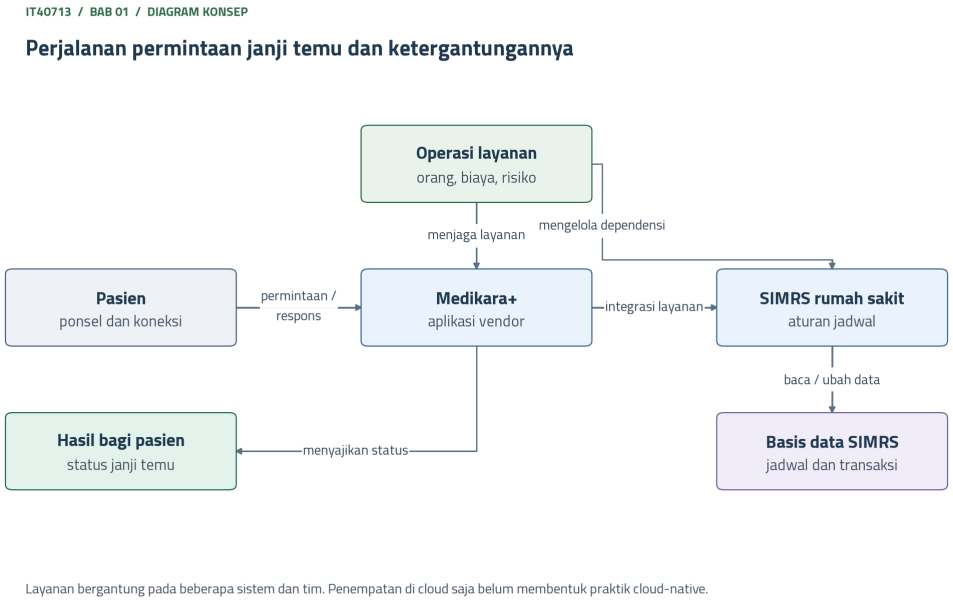
1.3.1 Aplikasi, Server, dan Basis Data

Aplikasi adalah program yang dipakai orang untuk menyelesaikan pekerjaan, misalnya mendaftarkan pasien, membuat janji temu, atau mencatat tagihan. Server adalah komputer yang menjalankan aplikasi bagi banyak pengguna sekaligus. Server menyala terus-menerus dan biasanya ditaruh di ruangan khusus. Basis data adalah tempat data disimpan secara teratur agar bisa dicari, diubah, dan dijaga. Ketiganya bekerja bersama: pengguna menyentuh tampilan aplikasi, aplikasi mengerjakan perintah di server, dan hasilnya dicatat di basis data.

Analoginya adalah restoran. Ruang makan dan daftar menu adalah tampilan aplikasi, tempat pelanggan memesan. Dapur dan para koki adalah server yang mengerjakan pesanan. Gudang bahan dan buku catatan stok adalah basis data. Pelanggan tidak masuk ke dapur dan tidak membuka gudang. Pelanggan cukup memesan, lalu menerima hasilnya.

Dua contoh di Medikara Health:

1. SIMRS rumah sakit Jakarta dipakai sekitar 900 pengguna setiap hari, yaitu dokter, perawat, petugas pendaftaran, dan kasir. Aplikasi itu berjalan di server di ruang server rumah sakit, dan riwayat kunjungan tersimpan di basis



Gambar 1.2: Perjalanan permintaan janji temu dan ketergantungannya

data SIMRS.

2. ERP keuangan adalah satu aplikasi untuk 36 fasilitas: 5 rumah sakit, 18 klinik, 1 laboratorium, dan 12 apotek. Karena terpusat, laporan keuangan bulanan disusun dari satu basis data, tidak perlu menggabungkan 36 berkas terpisah.

1.3.2 Jaringan dan Pusat Data

Jaringan adalah jalur yang menghubungkan perangkat agar dapat bertukar data. Internet adalah jaringan publik yang dipakai bersama oleh semua orang, sedangkan jaringan internal adalah jaringan milik organisasi yang menghubungkan kantor dan fasilitasnya. Pusat data adalah gedung atau ruangan khusus tempat server dijalankan, lengkap dengan listrik cadangan, pendingin, dan pengamanan fisik. Keandalan keduanya menentukan apakah aplikasi bisa dipakai saat dibutuhkan.

Analoginya adalah jalan raya dan gudang logistik. Jaringan adalah jalan yang dilalui kurir. Bila jalan terputus, barang terlambat walaupun gudangnya penuh. Pusat data adalah gudang besar berpendingin, dijaga satpam, dan punya genset sendiri. Gudang yang bagus tidak menolong bila jalan menuju gudang itu putus.

Dua contoh di Medikara Health:

1. Misalkan jaringan internal dari satu klinik pratama di Bogor ke SIMRS rumah sakit Bogor terputus 2 jam. Pendaftaran dicatat di kertas lalu diketik ulang, sekitar 60 pendaftaran tercatat dua kali, dan petugas lembur untuk menyamakannya.
2. Ruang server rumah sakit Depok hanya punya satu unit pendingin. Ketika unit itu rusak pada siang hari, server harus dimatikan dan SIMRS berhenti sekitar 90 menit.

1.3.3 Cloud sebagai Layanan Komputasi yang Disewa

NIST mendefinisikan *cloud computing* sebagai model yang memberi akses lewat jaringan, kapan pun dibutuhkan, ke kumpulan sumber daya komputasi bersama, seperti jaringan, server, penyimpanan, aplikasi, dan layanan. Sumber daya itu dapat disediakan dan dilepas dengan cepat, dengan upaya pengelolaan yang kecil [91]. Definisi yang sama menyebut lima karakteristik esensial: swalayan sesuai permintaan, akses jaringan yang luas, penggabungan sumber daya, elastisitas yang cepat, dan layanan yang terukur. Bab 2 membahasnya lebih rinci. Bagi manajemen, intinya sederhana: *cloud* adalah layanan komputasi yang disewa sesuai pemakaian, kapasitasnya dapat ditambah atau dikurangi dengan cepat, tetapi tanggung jawab atas data dan cara pemakaiannya tidak ikut berpindah.

Analoginya adalah menyewa mobil dibandingkan membeli mobil sendiri. Penyewa tidak perlu membayar harga mobil di muka, tidak mengurus servis, dan dapat memilih mobil besar hanya pada hari libur. Namun, bila mobil dipakai setiap hari untuk jarak jauh, membeli sendiri bisa lebih murah. Penyewa juga tetap bertanggung jawab atas cara mengemudi dan barang yang dibawanya.

Dua contoh di Medikara Health:

1. Misalkan pemakaian Medikara+ pada Senin pagi mencapai 4 kali rata-rata harian. Dengan server sendiri, kapasitas harus dibeli untuk puncak itu, padahal kapasitas tersebut menganggur hampir sepanjang pekan. Di *cloud*, kapasitas tambahan disewa hanya selama beberapa jam puncak.
2. Laporan konsolidasi keuangan dihitung sekali sebulan selama sekitar 6 jam. Server sewaan untuk pekerjaan itu cukup dinyalakan 6 dari 720 jam dalam sebulan.

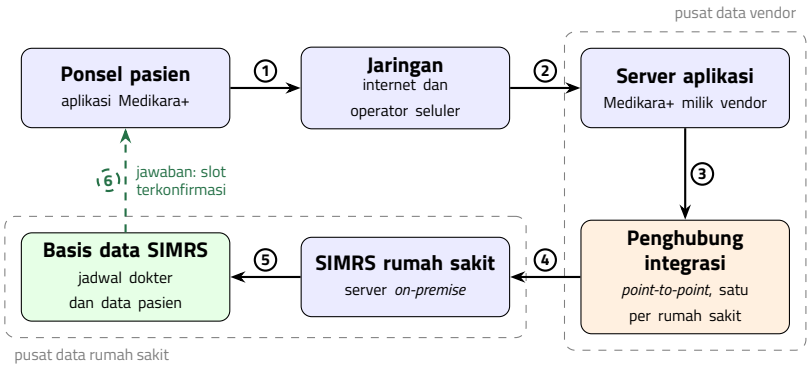
Tabel 1.2 merangkum keenam istilah beserta analoginya. Tabel ini dapat dipakai sebagai kamus kecil saat tim berdiskusi.

Tabel 1.2: Enam istilah dasar sistem informasi dengan bahasa sehari-hari

Istilah	Arti sederhana	Analogi
Aplikasi	Program untuk menyelesaikan pekerjaan	Ruang makan dan menu
Server	Komputer yang menjalankan aplikasi bagi banyak pengguna	Dapur dan koki
Basis data	Tempat data disimpan teratur	Gudang bahan dan buku stok
Jaringan	Jalur yang menghubungkan perangkat	Jalan raya
Pusat data	Ruang server dengan listrik cadangan dan pendingin	Gudang berpendingin dengan genset
Cloud	Layanan komputasi yang disewa sesuai pemakaian	Menyewa mobil

1.3.4 Perjalanan Satu Permintaan

Keenam istilah tadi bertemu dalam satu peristiwa sederhana: pasien memesan janji temu lewat ponsel. Gambar 1.3 memperlihatkan perjalanan permintaan itu dalam enam langkah. Permintaan berangkat dari aplikasi di ponsel, menempuh jaringan, diolah server aplikasi milik vendor, diteruskan lewat penghubung integrasi ke SIMRS rumah sakit, lalu dicatat di basis data. Jawabannya kembali ke ponsel pasien.



Gambar 1.3: Perjalanan satu permintaan janji temu dari Medikara+ sampai ke basis data SIMRS, lalu kembali ke ponsel pasien

Kasus Medikara Health: Janji Temu dari Medikara+

Seorang pasien membuka Medikara+ pada Senin pukul 07.10 dan memilih dokter penyakit dalam di rumah sakit Bandung. Langkah 1 dan 2 membawa permintaan lewat jaringan seluler dan internet ke server aplikasi Medikara+ di pusat data vendor. Server itu tidak menyimpan jadwal dokter, sehingga

pada langkah 3 permintaan diteruskan ke penghubung integrasi khusus rumah sakit Bandung. Pada langkah 4 dan 5, penghubung menerjemahkan permintaan ke format SIMRS Bandung, lalu SIMRS memeriksa slot di basis datanya. Langkah 6 mengirim jawaban "slot terkonfirmasi" kembali ke ponsel.

Misalkan pasien menunggu 25 detik lalu mendapat pesan gagal. Pasien hanya tahu "aplikasinya lambat". Sumber masalahnya bisa ada di enam titik: sinyal ponsel, jaringan, server vendor yang penuh, penghubung Bandung yang belum diperbarui setelah SIMRS Bandung naik versi, SIMRS yang sedang menjalankan proses cadangan, atau basis data yang lambat. Karena lima rumah sakit memakai lima versi SIMRS, Medikara+ membutuhkan lima penghubung berbeda, dan setiap perubahan di satu SIMRS berpotensi merusak satu penghubung.

Pelajaran dari kasus ini: keluhan pengguna adalah gejala, bukan diagnosis. Daftar pemicu modernisasi pada Aktivitas bab ini harus memisahkan keluhan yang dirasakan dari sistem yang diduga menjadi penyebabnya.

1.4 Bahasa Bersama untuk Mahasiswa Teknis

Mahasiswa berlatar TI biasanya fasih menyebut komponen, tetapi sering kesulitan menjelaskan mengapa sebuah perubahan layak dibiayai. Direksi jarang bertanya tentang jenis basis data. Direksi bertanya apakah perubahan itu menambah pendapatan, mengurangi biaya, menurunkan risiko, atau memperbaiki layanan pasien, dan berapa lama modalnya kembali. Bagian ini membahas empat hal yang membentuk bahasa manajemen: nilai bisnis, struktur biaya, risiko dan kepatuhan, serta cara menulis rekomendasi.

1.4.1 Nilai Bisnis dan Cara Manajemen Menilainya

Nilai bisnis adalah manfaat yang dapat diukur bagi organisasi. Bentuknya antara lain pendapatan yang bertambah, biaya yang turun, risiko yang mengecil, pelanggan yang lebih puas, dan kepatuhan yang terjaga. Manajemen menilainya lewat indikator kinerja utama, atau Key Performance Indicator (KPI), misalnya waktu tunggu pendaftaran, jumlah kunjungan, dan biaya per kunjungan. Usulan teknologi yang tidak bisa dikaitkan dengan salah satu KPI biasanya kalah bersaing dengan usulan lain di rapat anggaran.

Analoginya adalah pemilik warung kopi yang ditawari mesin kopi baru. Pemilik itu tidak terkesan oleh angka tekanan uap atau suhu ketel. Pertanyaannya hanya dua: berapa cangkir tambahan yang terjual per hari, dan berapa bulan modalnya

kembali. Penjual yang menjawab dua pertanyaan itu dengan angka lebih mungkin berhasil daripada penjual yang membacakan brosur spesifikasi.

Dua contoh di Medikara Health:

1. Usulan "memperbarui server SIMRS rumah sakit Bekasi" sulit dinilai direksi. Usulan yang sama dapat ditulis ulang menjadi "memangkas waktu tunggu pendaftaran Senin pagi dari rata-rata 70 menit menjadi 30 menit". Kalimat kedua menyebut KPI yang dipantau direksi setiap bulan.
2. Misalkan pada jam sibuk telekonsultasi Medikara+ melayani sekitar 60 sesi per jam dengan tarif rata-rata Rp150.000. Setiap jam aplikasi tidak bisa dipakai pada jam sibuk berarti pendapatan sekitar 60 × Rp150.000 = Rp9 juta yang hilang, belum termasuk pasien yang kecewa dan beralih ke penyedia lain.

Tabel 1.3 memperlihatkan cara menerjemahkan temuan teknis ke dalam bahasa dampak. Kolom tengah berisi pertanyaan yang hampir pasti diajukan direksi saat membaca kolom pertama.

Tabel 1.3: Menerjemahkan temuan teknis ke dalam bahasa dampak bagi direksi

Bahasa teknis	**Pertanyaan direksi**	**Bahasa dampak**
Server SIMRS Bekasi sering kehabisan memori	Apa akibatnya bagi pasien?	Pendaftaran berhenti rata-rata 3 kali sebulan, masing-masing sekitar 40 menit
Integrasi ke lima SIMRS masih *point-to-point*	Berapa biayanya?	Setiap mitra baru membutuhkan lima penghubung, sehingga satu kerja sama baru tertunda sekitar 4 bulan
Akun vendor tidak punya masa berlaku	Seberapa besar risikonya?	Insiden 2025 terjadi lewat jalur yang sama dan wajib dilaporkan dalam 3 kali 24 jam
Laporan manajemen disusun dari *spreadsheet*	Apa yang hilang?	Laporan bulanan baru siap pada tanggal 12, sehingga keputusan selalu terlambat dua pekan

1.4.2 Struktur Biaya dan Siklus Anggaran

Biaya TI dicatat dalam dua kelompok besar. Belanja modal, atau Capital Expenditure (CapEx), adalah pembelian aset yang dibayar di muka dan disusutkan selama beberapa tahun, misalnya server dan perangkat jaringan. Belanja operasional, atau Operational Expenditure (OpEx), adalah pengeluaran rutin yang dibayar selama layanan dipakai, misalnya sewa *cloud*, lisensi bulanan, dan listrik. Pindah ke *cloud* umumnya menggeser sebagian biaya dari CapEx ke OpEx, dan pergeseran itu mengubah cara organisasi merencanakan serta mengendalikan anggaran [135].

Biaya yang dibandingkan juga tidak cukup harga beli. Biaya total kepemilikan, atau Total Cost of Ownership (TCO), mencakup listrik, pendingin, ruang, lisensi, staf yang merawat, dan penggantian perangkat. Bab 3 membahas TCO dan pengelolaan biaya *cloud* secara rinci.

Analoginya adalah mesin fotokopi kantor. Kantor dapat membeli mesin seharga Rp60 juta, membayarnya di muka, dan merawatnya sendiri. Kapasitasnya tetap, entah dipakai 100 lembar atau 10.000 lembar sebulan. Kantor juga dapat menyewa mesin dengan tarif per lembar. Tagihannya berubah setiap bulan mengikuti pemakaian, dan bagian keuangan harus siap dengan tagihan yang naik saat musim laporan.

Siklus anggaran ikut menentukan. Di banyak organisasi, anggaran tahun berikutnya disusun dan disetujui beberapa bulan sebelum tahun berjalan. Anggaran CapEx cocok dengan siklus itu karena angkanya tetap. Tagihan *cloud* yang naik turun setiap bulan perlu kebiasaan baru: perkiraan pemakaian, batas pengeluaran, label biaya per layanan, dan peninjauan tagihan secara rutin.

Dua contoh di Medikara Health:

1. Misalkan penggantian 6 server SIMRS rumah sakit Bandung ditawarkan seharga Rp1,8 miliar. Dengan masa susut 5 tahun, beban penyusutannya Rp360 juta per tahun, belum termasuk listrik, pendingin, dan staf. Penawaran *cloud* untuk beban yang sama misalnya Rp32 juta per bulan, atau Rp384 juta per tahun. Angka mentah ini belum cukup untuk memutuskan, karena kedua sisi belum dihitung sebagai TCO.
2. Tagihan *cloud* Medikara+ bulan Januari Rp28 juta, lalu bulan Februari Rp41 juta karena kampanye pendaftaran vaksinasi. Bagian keuangan bertanya mengapa tagihan naik 46 persen. Tanpa label biaya per layanan, tim TI tidak dapat menjawab dengan cepat.

1.4.3 Risiko dan Kepatuhan sebagai Batasan Desain

Risiko adalah kemungkinan terjadinya peristiwa yang merugikan, dikalikan dengan dampaknya. Kepatuhan adalah keadaan ketika organisasi memenuhi aturan yang wajib diikuti, baik dari undang-undang, regulator, maupun kontrak. Bagi perancang sistem, keduanya bukan lampiran yang ditambahkan menjelang proyek selesai, melainkan batasan desain sejak hari pertama, seperti beban maksimum yang harus ditahan sebuah jembatan.

Di sektor kesehatan Indonesia, dua aturan langsung membentuk batasan itu. Undang-Undang (UU) Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi (PDP) menggolongkan data dan informasi kesehatan sebagai data pribadi yang bersifat spesifik. UU yang sama mewajibkan pengendali data menyampaikan

pemberitahuan tertulis paling lambat 3 kali 24 jam bila terjadi kegagalan pelindungan data pribadi, dan mengatur sanksi administratif berupa denda paling tinggi 2 persen dari pendapatan tahunan [120]. Peraturan Menteri Kesehatan (Permenkes) Nomor 24 Tahun 2022 tentang Rekam Medis mewajibkan fasilitas pelayanan kesehatan menyelenggarakan rekam medis elektronik paling lambat 31 Desember 2023. Permenkes yang sama mengizinkan rekam medis elektronik disimpan di server, di *cloud computing* yang tersertifikasi, atau di media digital lain [83]. Aturan tentang lokasi pengelolaan data dibahas pada Bab 2.

Analoginya adalah aturan keselamatan gedung. Tangga darurat, pintu tahan api, dan jalur evakuasi ditentukan saat denah digambar. Menambahkan tangga darurat setelah gedung berdiri jauh lebih mahal, kadang mustahil tanpa membongkar dinding. Kontrol akses, catatan jejak akses, dan pemisahan data sensitif bekerja dengan cara yang sama.

Kasus Medikara Health: Insiden Vendor dan Temuan Audit

Pada 2025, data sebagian pasien Medikara+ terbuka lewat akun milik vendor aplikasi. Akun itu dibuat untuk keperluan uji, punya hak akses luas, dan masih aktif setelah proyeknya selesai. Skala insidennya kecil, tetapi Medikara Health tetap harus menelusuri data mana yang terbuka dan memberi tahu pasien yang terdampak. Tak lama kemudian, audit internal menemukan kontrol akses yang lemah di beberapa SIMRS: akun karyawan yang sudah keluar masih aktif, dan satu akun dipakai bersama oleh beberapa petugas. Kedua peristiwa itu kini menjadi batasan desain bagi setiap rancangan baru:

1. Setiap akun, termasuk akun vendor, hanya mendapat hak akses yang dibutuhkan dan punya masa berlaku.
2. Setiap akses ke data pasien tercatat dan dapat diaudit.
3. Organisasi harus dapat mengetahui data siapa yang terdampak dalam waktu kurang dari 3 kali 24 jam. Syarat ini menuntut katalog data yang jelas.

Batasan ketiga menunjukkan bahwa regulasi ikut menentukan arsitektur. Tanpa mengetahui letak data pasien, kewajiban pelaporan tidak mungkin dipenuhi tepat waktu.

1.4.4 Menulis Rekomendasi untuk Direksi

Rekomendasi untuk direksi ditulis ringkas, paling panjang satu halaman, dengan jawaban di bagian depan. Prinsip ini dikenal sebagai prinsip piramida: kesimpulan disampaikan lebih dahulu, lalu didukung beberapa alasan, dan setiap alasan didukung data [103]. Isi rekomendasi berbasis dampak, menyebut pilihan lain yang di-

pertimbangkan, menyatakan *trade-off* secara jujur, dan meminta keputusan yang jelas. Rekomendasi yang hanya menjelaskan teknologi tanpa meminta keputusan biasanya berakhir dengan "akan dipelajari lebih lanjut".

Analoginya adalah dokter yang menjelaskan pilihan tindakan kepada keluarga pasien. Dokter menyampaikan diagnosis singkat, dua atau tiga pilihan tindakan, risiko masing-masing, lalu sarannya. Keputusan akhir ada di tangan keluarga. Dokter tidak membacakan buku ajar farmakologi, dan keluarga tidak dibiarkan menebak pilihan mana yang disarankan.

Dua contoh kalimat pembuka:

1. Lemah: "Disarankan migrasi SIMRS ke *microservices* di Kubernetes agar lebih *scalable*." Masalah, dampak, biaya, dan keputusannya tidak ada.
2. Kuat: "Direksi diminta menyetujui uji coba pendaftaran daring terpadu di Bekasi selama 3 bulan, Rp420 juta, dengan sasaran waktu tunggu Senin pagi turun dari 70 menjadi 30 menit." Susunan lengkapnya ada pada Tabel 1.4.

Tabel 1.4: Susunan rekomendasi satu halaman untuk direksi beserta contoh isinya

Bagian	Contoh isi untuk Medikara Health
Keputusan yang diminta	Menyetujui uji coba pendaftaran daring terpadu di rumah sakit Bekasi selama 3 bulan, biaya Rp420 juta
Masalah dan dampak	Waktu tunggu Senin pagi rata-rata 70 menit, sebagian pasien pulang sebelum dilayani
Pilihan yang dipertimbangkan	(a) Menambah petugas loket, (b) uji coba pendaftaran daring terpadu, (c) mengganti seluruh SIMRS
Rekomendasi dan alasan	Pilihan (b): menyentuh penyebab utama, biaya terbatas, hasilnya terukur dalam 3 bulan
Trade-off dan risiko	Lebih lambat daripada mengganti seluruh SIMRS dan belum menyelesaikan perbedaan versi, tetapi risikonya jauh lebih kecil
Langkah berikutnya	Bila sasaran tercapai, perluasan ke rumah sakit Depok dan Bogor diusulkan pada anggaran berikutnya

1.5 Kualitas Sistem dan Trade-off

Kualitas sistem adalah sifat yang menentukan seberapa baik sistem melayani, di luar daftar fiturnya. Dua aplikasi janji temu bisa punya fitur yang sama persis, tetapi yang satu selalu bisa dipakai pada Senin pagi, sedangkan yang lain sering gagal. Mata kuliah ini memakai lima kualitas utama: ketersediaan, skalabilitas, keamanan, biaya, dan kecepatan perubahan. Tabel 1.5 menjelaskan masing-masing beserta contoh ukurannya.

Dua hal pada kualitas sistem sering tertukar, yaitu ketersediaan dan skalabilitas. Pembagiannya begini. Ketersediaan menyangkut apakah sistem hidup dan bisa dipakai. Skalabilitas menyangkut apakah sistem tetap layak dipakai ketika

Tabel 1.5: Lima kualitas sistem yang dipakai sepanjang mata kuliah ini

Kualitas	Pertanyaan yang dijawab	Contoh ukuran	Contoh di Medikara Health
Ketersediaan	Bisakah sistem dipakai saat dibutuhkan?	Persentase waktu sistem dapat dipakai, misalnya 99,9 persen	SIMRS instalasi gawat darurat harus hidup 24 jam
Skalabilitas	Sanggupkah sistem melayani beban yang bertambah?	Jumlah permintaan per menit yang masih dilayani dengan cepat	Lonjakan janji temu pada Senin pagi
Keamanan	Terlindungkah data dan akses?	Jumlah akun tanpa masa berlaku, waktu menutup celah	Akun vendor dan akun karyawan yang sudah keluar
Biaya	Berapa yang dibayar untuk hasil itu?	Biaya infrastruktur per kunjungan pasien	Tagihan *cloud* Medikara+ per bulan
Kecepatan perubahan	Seberapa cepat perubahan sampai ke pengguna?	Waktu dari permintaan perubahan sampai rilis	Fitur baru Medikara+ yang menunggu jadwal rilis vendor

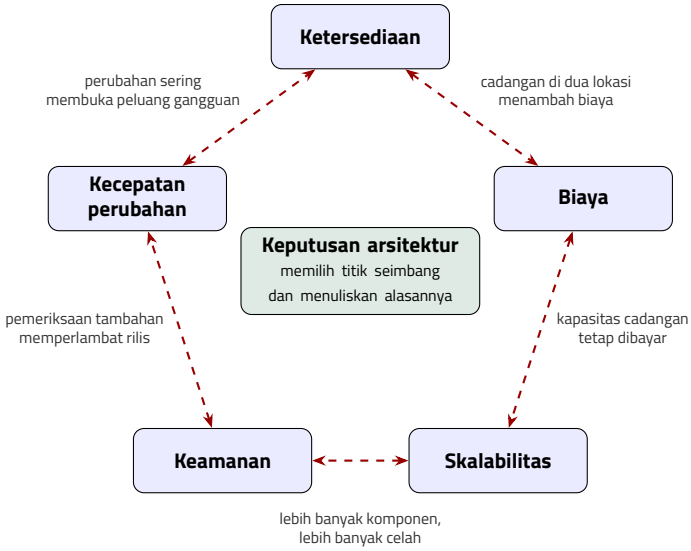
penggunanya bertambah. Sebuah sistem bisa selalu hidup tetapi lambat sekali pada jam ramai, artinya tersedia tetapi tidak skalabel. Sebaliknya, sistem yang mampu melayani lonjakan besar bisa tetap mati total ketika satu-satunya pusat datanya padam.

1.5.1 Mengapa Kualitas Saling Tarik-Menarik

Kelima kualitas tidak dapat dimaksimalkan sekaligus, karena uang, waktu, dan perhatian tim terbatas. Menaikkan satu kualitas hampir selalu membebani kualitas lain. Gambar 1.4 memperlihatkan lima pasangan yang paling sering bertentangan. Richards dan Ford merumuskannya sebagai hukum pertama arsitektur perangkat lunak: setiap keputusan arsitektur adalah *trade-off*. Hukum kedua mereka menyatakan bahwa alasan sebuah keputusan lebih penting daripada cara melaksanakannya [124].

Analoginya adalah memilih ambulans untuk sebuah klinik. Ambulans dengan peralatan lengkap lebih mahal dan lebih berat, sehingga lebih lambat di gang sempit. Ambulans ringan lebih murah dan lincah, tetapi peralatannya terbatas. Tidak ada ambulans yang sekaligus termurah, terlengkap, dan tercepat. Klinik memilih sesuai wilayah dan pasien yang dilayaninya, lalu menerima kekurangannya dengan sadar.

Ketersediaan memberi contoh yang paling mudah dihitung. Setiap angka sembilan tambahan pada target ketersediaan memangkas waktu henti yang diizinkan secara tajam. Tabel 1.6 menghitung waktu henti yang masih diizinkan untuk beberapa target, memakai 8.760 jam per tahun dan 720 jam per bulan. Pengalaman Google dalam *Site Reliability Engineering* menunjukkan bahwa biaya



Gambar 1.4: Lima kualitas sistem yang saling tarik-menarik beserta contoh pertentangannya

menambah keandalan naik jauh lebih cepat daripada manfaatnya, sehingga target perlu disesuaikan dengan kebutuhan pengguna, bukan dibuat setinggi mungkin [27].

Tabel 1.6: Waktu henti yang diizinkan untuk beberapa target ketersediaan

Target	Waktu henti per tahun	Waktu henti per bulan (30 hari)
99%	87,6 jam (sekitar 3,65 hari)	7,2 jam
99,5%	43,8 jam	3,6 jam
99,9%	8,76 jam	43,2 menit
99,95%	4,38 jam	21,6 menit
99,99%	52,6 menit	4,3 menit

Dua contoh *trade-off* di Medikara Health:

1. SIMRS untuk instalasi gawat darurat membutuhkan target tinggi, misalnya 99,95 persen, yang hanya mengizinkan sekitar 4,4 jam henti per tahun. Target itu menuntut server cadangan di lokasi lain dan petugas siaga. Laporan konsolidasi keuangan bulanan cukup memakai target 99 persen, karena keterlambatan beberapa jam tidak membahayakan pasien.
2. Misalkan setiap rilis Medikara+ wajib melewati pemeriksaan keamanan manual oleh tim lain yang hanya bertugas dua kali sebulan. Keamanan membaik, tetapi waktu dari permintaan perubahan sampai rilis naik dari 2 pekan menjadi 5 pekan. Pilihan lain adalah pemeriksaan otomatis pada setiap perubahan, ditambah pemeriksaan manual hanya untuk perubahan

yang menyentuh data pasien.

1.5.2 Menuliskan Alasan Keputusan

Trade-off yang tidak ditulis akan dilupakan. Dua tahun kemudian, tim baru melihat keputusan lama dan menilainya keliru, padahal alasannya masuk akal pada waktu itu. Karena itu, mata kuliah ini membiasakan satu kebiasaan sederhana: setiap keputusan penting dicatat dalam lima baris, yaitu konteks, keputusan, alasan, *trade-off* yang diterima, serta tanggal dan pihak yang memutuskan. Bentuk yang lebih baku, yaitu Architecture Decision Record (ADR), dibahas pada Bab 7. Contoh catatan keputusan untuk Medikara Health:

1. **Konteks:** telekonsultasi Medikara+ kehilangan sekitar Rp9 juta per jam henti pada jam sibuk.
2. **Keputusan:** target ketersediaan 99,9 persen, dijalankan di dua lokasi dalam satu region.
3. **Alasan:** target 99,99 persen menuntut dua region dan biayanya lebih besar daripada kerugian yang dicegah.
4. ***Trade-off* yang diterima:** bila seluruh region terganggu, layanan dapat berhenti beberapa jam.
5. **Tanggal dan pemutus:** Januari 2026, direktur operasional bersama kepala divisi TI.

Pendalaman Teknis (opsional): Model Kualitas ISO/IEC 25010

Lima kualitas di bab ini sengaja dibuat sedikit agar mudah dipakai dalam diskusi dengan manajemen. Model yang lebih lengkap tersedia di standar ISO/IEC 25010:2023 tentang model kualitas produk. Edisi 2023 memuat sembilan karakteristik: *functional suitability*, *performance efficiency*, *compatibility*, *interaction capability*, *reliability*, *security*, *maintainability*, *flexibility*, dan *safety*. Setiap karakteristik dirinci lagi menjadi subkarakteristik yang dapat dispesifikasikan dan diukur [76].

Pemetaan lima kualitas bab ini ke karakteristik terdekat:

- Ketersediaan ke *reliability*, dengan *availability* sebagai subkarakteristiknya.
- Skalabilitas ke *flexibility* (subkarakteristik *scalability*) dan *performance efficiency* (subkarakteristik *capacity*).
- Keamanan ke *security*; kecepatan perubahan ke *maintainability*, walaupun juga ditentukan oleh proses rilis.
- Biaya tidak punya padanan sebagai karakteristik produk. Kerangka

penyedia *cloud* justru menjadikannya pilar tersendiri, misalnya *cost optimization* pada AWS Well-Architected Framework [10].

1.6 Arti Cloud-Native bagi Organisasi

Istilah *cloud-native* sering dipakai seolah-olah sama dengan "sudah di *cloud*". Dua hal ini sering tertukar. Pembagiannya begini. Aplikasi *cloud-hosted* adalah aplikasi lama yang dipindahkan apa adanya ke server sewaan di *cloud*. Cara aplikasi itu dibangun, dirilis, dan ditambah kapasitasnya tidak berubah. Aplikasi *cloud-native* dirancang, bersama cara kerja timnya, untuk memanfaatkan sifat *cloud*: kapasitas yang elastis, otomatisasi, dan kemampuan pulih dari gangguan.

CNCF mendefinisikan praktik *cloud native* sebagai praktik yang memungkinkan organisasi mengembangkan, membangun, dan men-*deploy workload* di lingkungan komputasi publik, privat, maupun hibrida untuk memenuhi kebutuhannya pada skala besar, dengan cara yang terprogram dan dapat diulang. Ciri utamanya adalah sistem yang keterkaitannya longgar dan saling bekerja sama secara aman, tangguh, terkelola, berkelanjutan, dan dapat diamati [38]. Definisi ini dibahas rinci pada Bab 5. Untuk saat ini, satu hal perlu dicatat: definisi itu berbicara tentang praktik dan sifat sistem, bukan tentang merek teknologi tertentu.

Analoginya adalah memindahkan warung ke mal. Warung yang pindah ke mal tanpa mengubah cara kerjanya tetap punya satu kasir dan mencatat stok di buku tulis. Sewanya lebih mahal, sedangkan antreannya tetap panjang. Toko yang dirancang untuk mal memakai mesin kasir yang tersambung ke gudang pusat, menambah kasir saat akhir pekan, dan mengisi ulang stok secara otomatis. Lokasinya sama, hasilnya berbeda karena cara kerjanya ikut berubah.

Dua contoh di Medikara Health:

1. Misalkan SIMRS rumah sakit Bogor dipindahkan apa adanya ke mesin virtual di *cloud*. Ruang server dan pendingin tidak dibutuhkan lagi. Namun, rilis tetap dua kali setahun, kapasitas tetap ditambah secara manual, dan satu kesalahan masih dapat menghentikan seluruh SIMRS. Ini contoh *cloud-hosted*.
2. Layanan janji temu Medikara+ dirancang ulang sebagai layanan tersendiri. Kapasitasnya bertambah otomatis pada Senin pagi, perubahan dirilis setiap pekan lewat alur otomatis, dan gangguan pada satu rumah sakit tidak menghentikan pemesanan di rumah sakit lain. Ini lebih dekat ke *cloud-native*.

Tabel 1.7 merangkum perbedaannya pada lima aspek.

Tabel 1.7: Perbedaan aplikasi *cloud-hosted* dan *cloud-native*

Aspek	***Cloud-hosted***	***Cloud-native***
Bentuk aplikasi	Sama seperti di pusat data lama	Dirancang ulang menjadi bagian yang longgar keterkaitannya
Menambah kapasitas	Manual, menunggu permintaan	Otomatis mengikuti beban
Cara rilis	Jarang, terjadwal, berisiko besar	Sering, kecil, lewat alur otomatis
Saat terjadi gangguan	Satu kesalahan dapat menghentikan semuanya	Gangguan dibatasi pada satu bagian
Yang ikut berubah	Lokasi server	Lokasi, rancangan aplikasi, dan cara kerja tim

1.6.1 Mitos yang Sering Menyertai Cloud-Native

Pertanyaan kemudian muncul: bila *cloud-native* punya banyak keunggulan, mengapa tidak semua organisasi langsung menerapkannya? Jawabannya, keunggulan itu datang bersama biaya dan kerumitan baru. Sejumlah mitos membuat keduanya terlupa. Tabel 1.8 merangkum lima mitos yang paling sering muncul di rapat manajemen.

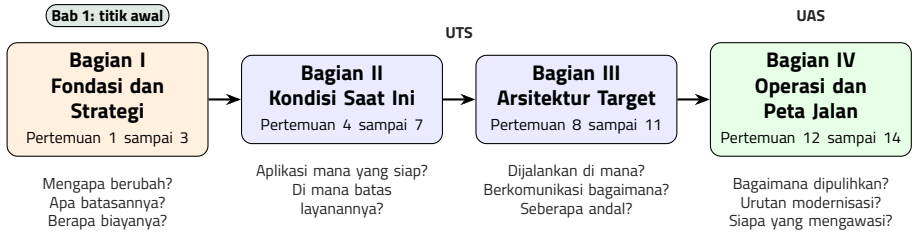
Tabel 1.8: Lima mitos tentang *cloud* dan *cloud-native* beserta kenyataannya

Mitos	**Kenyataan**
Cloud selalu lebih murah	Bergantung pada pola pemakaian dan disiplin pengelolaan. Dalam survei Flexera 2026, responden memperkirakan sekitar 29 persen belanja *cloud* mereka terbuang [55].
Microservices selalu lebih baik	Memecah aplikasi menambah kerumitan operasi. Segment pernah menggabungkan kembali ratusan layanan kecilnya menjadi satu layanan karena beban perawatannya terlalu berat [107].
Pindah ke *cloud* otomatis lebih aman	Penyedia mengamankan infrastrukturnya, tetapi akun, hak akses, dan konfigurasi tetap tanggung jawab pelanggan (Bab 2).
Cloud-native berarti harus memakai Kubernetes	Definisi CNCF menekankan praktik dan sifat sistem, bukan alat tertentu [38].
Sekali pindah, urusan selesai	Modernisasi berlangsung bertahap selama bertahun-tahun dan dikelola sebagai peta jalan (Bab 14).

1.6.2 Perjalanan Mata Kuliah

Setiap mitos di atas dijawab oleh satu atau beberapa pertemuan. Gambar 1.5 memperlihatkan perjalanan mata kuliah dalam empat Bagian. Bagian I menjawab mengapa organisasi perlu berubah, batasan apa yang mengikat, dan berapa biayanya. Bagian II membaca kondisi saat ini. Bagian III merancang arsitektur target. Bagian IV menyusun cara sistem dijalankan, dipulihkan, dan dimodernisasi secara

bertahap. Bab ini adalah titik awal Bagian I.



Gambar 1.5: Perjalanan mata kuliah dalam empat Bagian, dari strategi sampai peta jalan modernisasi

Kasus Medikara Health: Pertanyaan dari "Satu Pasien, Satu Rekam Medis"

Strategi "Satu Pasien, Satu Rekam Medis" terdengar sederhana: setiap pasien punya satu rekam medis yang dapat dibaca di fasilitas mana pun milik Medikara Health, juga oleh mitra yang berhak. Bagi arsitek, strategi itu memunculkan sejumlah pertanyaan:

1. Di mana rekam medis terpadu disimpan, dan aturan apa yang membatasi? (Bab 2)
2. Berapa biaya infrastruktur per kunjungan pasien? (Bab 3)
3. Apakah lima SIMRS diseragamkan, diganti, atau dibungkus? (Bab 4 sampai 7, diputuskan di Bab 14)
4. Bagaimana mitra tersambung tanpa integrasi *point-to-point* baru? (Bab 9)
5. Bagaimana rekam medis dipulihkan bila satu pusat data padam? (Bab 12)

Tidak satu pun pertanyaan itu terjawab dengan "pindahkan semuanya ke *cloud*". Setiap jawaban menuntut pilihan beserta *trade-off*-nya, dan pilihan itu disusun bertahap sepanjang semester.

1.7 Aktivitas: Membentuk Tim dan Memetakan Pemicu Modernisasi

Tujuan. Tim campuran terbentuk, organisasi capstone dipilih, dan masalah TI yang dirasakan di organisasi itu dirumuskan sebagai pemicu modernisasi. Pemicu modernisasi adalah masalah nyata yang membuat organisasi perlu mengubah sistemnya, ditulis lengkap dengan dampak dan sistem yang terlibat. Daftar ini menjadi pijakan seluruh artefak berikutnya, sehingga mutunya menentukan mutu cetak biru.

Pembagian peran. Tim berisi tiga sampai lima orang, dengan sedikitnya satu anggota berlatar teknis dan satu anggota berlatar non-teknis. Pada tim tiga orang, satu anggota boleh memegang dua peran.

- **Koordinator** (anggota mana pun): mengatur jadwal, menyatukan draf, dan mewakili tim saat tatap muka Sabtu.
- **Pemimpin wawancara** (anggota non-teknis): memimpin wawancara singkat atau penelusuran dokumen tentang masalah layanan.
- **Pemeta sistem** (anggota teknis): memetakan aplikasi, server, basis data, dan integrasi yang terlibat dalam setiap masalah.
- **Penjaga izin dan etika** (anggota yang dekat dengan organisasi): mengurus izin tertulis dan memeriksa anonimisasi setiap dokumen.

Langkah kerja.

1. Bentuk tim berisi tiga sampai lima orang dengan latar belakang campuran, lalu tetapkan peran.
2. Pilih organisasi. Bila memakai organisasi nyata, urus izin tertulis dengan mengikuti `sources/chapter01/panduan-izin-dan-anonimisasi.md`. Bila izin tidak didapat sampai Pertemuan 2, pakai kasus Medikara Health.
3. Kumpulkan masalah dari pengguna dan manajemen. Pemimpin wawancara berbicara dengan tiga sampai lima orang, misalnya petugas layanan, kepala unit, dan staf TI, masing-masing sekitar 20 menit. Catat keluhan apa adanya beserta angka yang disebut narasumber. Tim Medikara Health memakai kotak kasus di bab ini sebagai pengganti wawancara.
4. Petakan sistem yang terkait dengan setiap masalah. Pemeta sistem menelusuri perjalanan permintaan seperti pada Gambar 1.3 dan menandai titik yang diduga menjadi penyebab.
5. Tulis profil organisasi satu halaman memakai `sources/chapter01/templat-profil-organisasi.md`, lalu susun lima sampai sepuluh pemicu modernisasi memakai `sources/chapter01/lembar-pemicu-modernisasi.md`. Seluruh anggota menyepakati urutan prioritasnya.

Luaran dan kaitan dengan capstone. Tiga dokumen: profil organisasi satu halaman, daftar lima sampai sepuluh pemicu modernisasi beserta prioritasnya, dan bukti izin tertulis atau pernyataan memakai Medikara Health. Profil dan daftar pemicu adalah artefak pertama tonggak C1 Konteks dan Strategi. Daftar pemicu dipakai lagi pada Pertemuan 2 untuk memetakan *workload*, pada Pertemuan 3 untuk menyusun model biaya awal, serta menjadi masukan analisis kesenjangan di IT40513 dan daftar risiko di IT40813.

Kriteria penilaian. Kejelasan masalah (30 persen): setiap pemicu ditulis se-

bagai masalah yang dirasakan, bukan sebagai solusi. Keterkaitan masalah dan sistem (25 persen): setiap pemicu menyebut sistem yang terlibat dan dugaan penyebabnya. Dampak yang terukur (25 persen): dampak ditulis dengan angka beserta sumbernya. Etika dan anonimisasi (20 persen): bukti izin tersedia dan tidak ada data pribadi di dokumen.

Catatan etika. Wawancara hanya dilakukan dengan narasumber yang bersedia, setelah tujuan dan pemakaian catatannya dijelaskan. Data pribadi pasien, pelanggan, atau karyawan tidak boleh dicatat, difoto, atau disalin, termasuk dalam tangkapan layar.

Pendalaman Teknis (opsional): Menelusuri Konsol Cloud

Anggota berlatar teknis dapat membuat akun *free tier* di satu penyedia *cloud*, misalnya Amazon Web Services (AWS), Google Cloud, atau Microsoft Azure. Tujuannya mengenal konsol dan kalkulator harga, bukan menjalankan sistem.

1. Buat akun dan aktifkan autentikasi dua faktor pada akun utama.
2. Pasang peringatan anggaran dengan batas kecil, misalnya setara Rp50.000, sebelum membuka layanan apa pun.
3. Telusuri daftar layanan dan kelompokkan menjadi komputasi, penyimpanan, basis data, dan jaringan.
4. Buka kalkulator harga resmi, perkirakan biaya bulanan satu mesin virtual kecil dan satu basis data kecil di region Indonesia, lalu catat sebagai bahan Pertemuan 3.

Peringatan: pendaftaran akun umumnya meminta kartu pembayaran. Sumber daya berbayar tidak dijalankan. Sumber daya yang lupa dimatikan tetap ditagih, sehingga setiap uji coba diakhiri dengan memeriksa bahwa tidak ada sumber daya yang masih berjalan.

1.8 Ringkasan

Pertanyaan "apakah pindah ke *cloud* menyelesaikan semuanya?" tidak punya jawaban tunggal. Antrean panjang, SIMRS yang berbeda versi, dan biaya integrasi yang tinggi di Medikara Health punya penyebab yang berbeda-beda: sebagian di proses, sebagian di rancangan aplikasi, dan hanya sebagian di server. Mata kuliah ini, yang dipelajari mandiri dengan tatap muka Sabtu, melatih cara memisahkan penyebab tersebut lewat proyek capstone bersama empat mata kuliah, dari tonggak C1 sampai C4.

Kelas yang berlatar campuran membutuhkan bahasa bersama. Mahasiswa non-teknis perlu memahami aplikasi, server, basis data, jaringan, pusat data, dan

cloud, serta perjalanan satu permintaan yang melewati semuanya, karena keluhan pengguna hanyalah gejala. Mahasiswa teknis perlu fasih berbicara tentang nilai bisnis, CapEx dan OpEx, risiko, serta kepatuhan pada UU PDP dan aturan rekam medis elektronik sebagai batasan desain sejak awal. Rekomendasi untuk direksi ditulis satu halaman, dengan jawaban di depan dan keputusan yang diminta secara jelas.

Kualitas sistem saling tarik-menarik. Ketersediaan, skalabilitas, keamanan, biaya, dan kecepatan perubahan tidak dapat dimaksimalkan sekaligus, sehingga setiap keputusan adalah *trade-off* yang alasannya perlu ditulis. Aplikasi *cloud-hosted* hanya memindahkan lokasi, sedangkan *cloud-native* ikut mengubah rancangan aplikasi dan cara kerja tim.

Pesan utama bab ini: *cloud* adalah alat, bukan tujuan. Modernisasi yang baik berangkat dari masalah organisasi yang dirumuskan dengan jelas dan diukur dengan angka, lalu dijawab dengan pilihan yang *trade-off*-nya disadari dan dituliskan. Daftar pemicu modernisasi yang disusun tim pada Aktivitas bab ini adalah langkah pertama ke arah itu.

1.9 Latihan

Latihan 1: Cloud-Hosted dan Cloud-Native

Latihan ini membuktikan bahwa perbedaan *cloud-hosted* dan *cloud-native* terletak pada cara aplikasi dibangun dan dijalankan, bukan pada lokasi servernya.

1. Pilih satu aplikasi dari tempat kerja atau dari Medikara Health.
2. Tuliskan kondisinya saat ini pada lima aspek: lokasi, cara menambah kapasitas, cara rilis, akibat gangguan, dan pola biaya.
3. Tuliskan bentuknya bila dipindahkan apa adanya ke *cloud*, lalu bila dirancang ulang secara *cloud-native*.
4. Tandai aspek yang berubah dan aspek yang tetap pada kedua skenario.

Tabel 1.9 memuat contoh untuk SIMRS rumah sakit Bogor. Tabel 1.10 adalah lembar isian.

Pertanyaan analisis:

1. Aspek mana yang berubah pada skenario *cloud-hosted*, dan aspek mana yang tetap? Apa artinya bagi masalah yang paling dirasakan pengguna?
2. Bila biaya rancang ulang *cloud-native* misalnya tiga kali biaya pemindahan apa adanya, keuntungan apa yang harus terlihat di tabel agar pilihan itu tetap layak diusulkan?

Tabel 1.9: Latihan 1, contoh terisi: SIMRS rumah sakit Bogor

Aspek	Saat ini	*Cloud-hosted*	*Cloud-native*
Lokasi	Ruang server rumah sakit	Mesin virtual di region Jakarta	Region Jakarta, dua lokasi
Menambah kapasitas	Membeli server, sekitar 3 bulan	Manual, sekitar 1 hari	Otomatis, dalam hitungan menit
Cara rilis	Dua kali setahun	Tetap dua kali setahun	Setiap pekan untuk bagian yang berubah
Akibat gangguan	Seluruh SIMRS berhenti	Seluruh SIMRS berhenti	Hanya bagian yang terganggu berhenti
Pola biaya	CapEx Rp1,2 miliar per 5 tahun	OpEx sekitar Rp25 juta per bulan	OpEx berubah sesuai beban, ditambah biaya rancang ulang

Tabel 1.10: Latihan 1, lembar isian

Aspek	Saat ini	*Cloud-hosted*	*Cloud-native*
Lokasi	…	…	…
Menambah kapasitas	…	…	…
Cara rilis	…	…	…
Akibat gangguan	…	…	…
Pola biaya	…	…	…

Latihan 2: Dua Sisi Satu Masalah Layanan

Latihan ini membuktikan bahwa satu masalah layanan selalu punya sisi bisnis dan sisi teknis, dan keduanya perlu ditulis berdampingan agar dapat diputuskan.

1. Pilih satu masalah layanan di Medikara Health atau di organisasi tim, misalnya antrean pendaftaran yang panjang.
2. Tuliskan sisi bisnisnya: siapa yang terdampak, seberapa sering, dan berapa dampaknya dalam waktu atau rupiah.
3. Tuliskan sisi teknisnya: sistem yang terlibat dan titik yang diduga menjadi penyebab, dengan menelusuri perjalanan permintaan.
4. Hitung dampak tahunan, lalu tulis sumber atau anggapan di balik setiap angka.
5. Tulis satu kalimat rekomendasi yang menyebut keputusan yang diminta.

Tabel 1.11 memuat contoh untuk antrean pendaftaran rawat jalan rumah sakit Bekasi, dengan angka rekaan untuk kasus fiktif ini. Tabel 1.12 adalah lembar isian.

Pertanyaan analisis:

1. Angka mana pada tabel yang paling lemah dasarnya? Bagaimana angka itu

Tabel 1.11: Latihan 2, contoh terisi: antrean pendaftaran rawat jalan rumah sakit Bekasi

Aspek	Isi
Masalah yang dirasakan	Waktu tunggu pendaftaran Senin pagi rata-rata 70 menit
Sisi bisnis: pasien	Sekitar 450 kunjungan rawat jalan per hari; misalkan 4 persen pasien pulang sebelum dilayani pada hari Senin, yaitu 18 pasien
Sisi bisnis: pendapatan	Tarif rata-rata kunjungan Rp350.000; 18 × Rp350.000 = Rp6,3 juta per Senin; 52 × Rp6,3 juta = Rp327,6 juta per tahun
Sisi teknis: sistem	Medikara+, penghubung integrasi Bekasi, SIMRS Bekasi, verifikasi kepesertaan BPJS Kesehatan
Sisi teknis: dugaan penyebab	Pendaftaran daring tidak tersambung langsung ke SIMRS, sehingga data diketik ulang di loket; verifikasi BPJS Kesehatan lambat pada jam sibuk
Kalimat rekomendasi	Menyetujui uji coba pendaftaran daring terpadu di Bekasi selama 3 bulan dengan sasaran waktu tunggu 30 menit

Tabel 1.12: Latihan 2, lembar isian

Aspek	Isi
Masalah yang dirasakan	...
Sisi bisnis: pasien	...
Sisi bisnis: pendapatan	...
Sisi teknis: sistem	...
Sisi teknis: dugaan penyebab	...
Kalimat rekomendasi	...

dapat diperkuat sebelum dibawa ke direksi?

2. Bila dugaan penyebab ternyata ada di alur kerja loket, bukan di sistem, bagaimana kalimat rekomendasinya berubah?

Latihan 3: Biaya dan Ketersediaan yang Tarik-Menarik

Latihan ini membuktikan bahwa target ketersediaan tertinggi belum tentu pilihan terbaik, karena biaya mencegah waktu henti dapat melampaui kerugian yang dicegah.

1. Pilih satu layanan yang kerugian per jam hentinya dapat diperkirakan.
2. Tetapkan tiga pilihan target ketersediaan dan rancangan sederhananya.
3. Hitung waktu henti per tahun untuk setiap target memakai Tabel 1.6, lalu kalikan dengan kerugian per jam.
4. Tambahkan biaya infrastruktur per tahun, lalu bandingkan totalnya.

Tabel 1.13 memuat contoh untuk telekonsultasi Medikara+ dengan anggapan kerugian Rp9 juta per jam henti, sesuai perhitungan pada bagian nilai bisnis. Biaya infrastrukturnya rekaan. Tabel 1.14 adalah lembar isian.

Tabel 1.13: Latihan 3, contoh terisi: telekonsultasi Medikara+ (dalam juta rupiah per tahun)

Pilihan	Target	Henti per tahun	Kerugian	Biaya infrastruktur	Total
A. Satu lokasi	99,5%	43,8 jam	394,2	360	754,2
B. Dua lokasi, satu region	99,9%	8,76 jam	78,8	576	654,8
C. Dua region	99,99%	0,88 jam	7,9	1.020	1.027,9

Tabel 1.14: Latihan 3, lembar isian (dalam juta rupiah per tahun)

Pilihan	Target	Henti per tahun	Kerugian	Biaya infrastruktur	Total
A. …	…	…	…	…	…
B. …	…	…	…	…	…
C. …	…	…	…	…	…

Pertanyaan analisis:

1. Pada contoh, pilihan B memberi total terendah. Pada kerugian per jam berapa pilihan C mulai lebih menguntungkan daripada pilihan B?
2. Perhitungan ini menganggap setiap jam henti terjadi pada jam sibuk. Bagaimana hasilnya berubah bila sebagian besar henti terjadi pada malam hari?
3. Kerugian apa yang tidak tercatat di tabel, misalnya kepercayaan pasien atau kewajiban pelaporan, dan bagaimana kerugian itu memengaruhi pilihan?

Bab 2

Model Layanan, Model Penerapan, dan Lokasi Data

Tujuan Pembelajaran

- Membedakan karakteristik esensial, model layanan, dan model penerapan *cloud* menurut NIST (National Institute of Standards and Technology) SP 800-145.
- Menjelaskan pembagian tanggung jawab antara penyedia dan pelanggan pada IaaS, PaaS, dan SaaS.
- Menganalisis pengaruh aturan lokasi data di Indonesia terhadap pilihan penempatan *workload*.

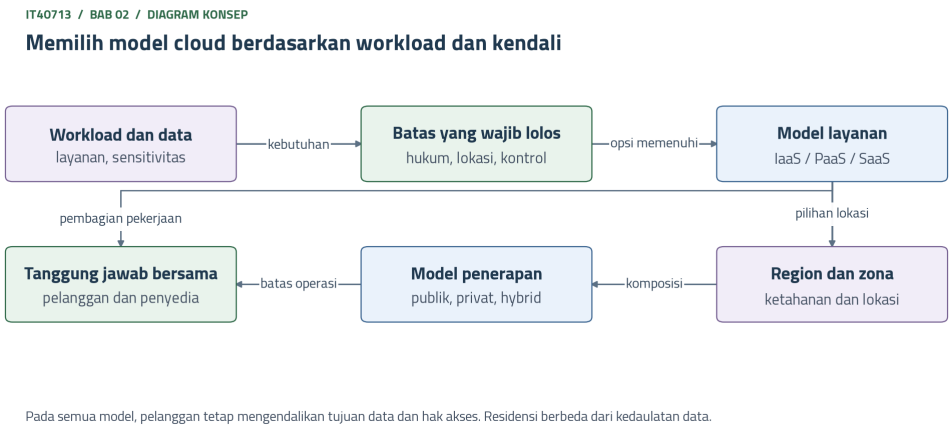
2.1 Pendahuluan

Misalkan dalam satu bulan direksi Medikara Health menerima tiga tawaran. Vendor Sistem Informasi Manajemen Rumah Sakit (SIMRS) untuk rumah sakit Depok dan Bogor menawarkan "SIMRS *cloud*" dengan biaya tetap per bulan. Sebuah integrator menawarkan "*hybrid cloud*" yang menggabungkan ruang server rumah sakit dengan pusat data sewaan. Seorang penyedia *cloud* global menawarkan potongan harga bila Medikara memakai *region* Jakarta selama tiga tahun. Ketiganya memakai kata *cloud*, tetapi yang dijual berbeda dan tanggung jawab yang dibebankan kepada Medikara juga berbeda.

Kebingungan ini bukan soal istilah semata. Setelah insiden kebocoran data pasien lewat vendor pada 2025, direksi ingin tahu tiga hal sebelum menandatangani apa pun: apa yang sebenarnya dibeli, siapa yang bertanggung jawab atas bagian

mana, dan data mana yang boleh ditempatkan di mana. Pertanyaan terakhir paling sulit, karena jawabannya ditentukan oleh beberapa aturan sekaligus. Pilihan model layanan dan penerapan perlu mengikuti kebutuhan workload, syarat lokasi, serta pembagian tanggung jawab yang dapat dijalankan.

Gambar 2.1 merangkum hubungan komponen dan konsep utama bab ini.



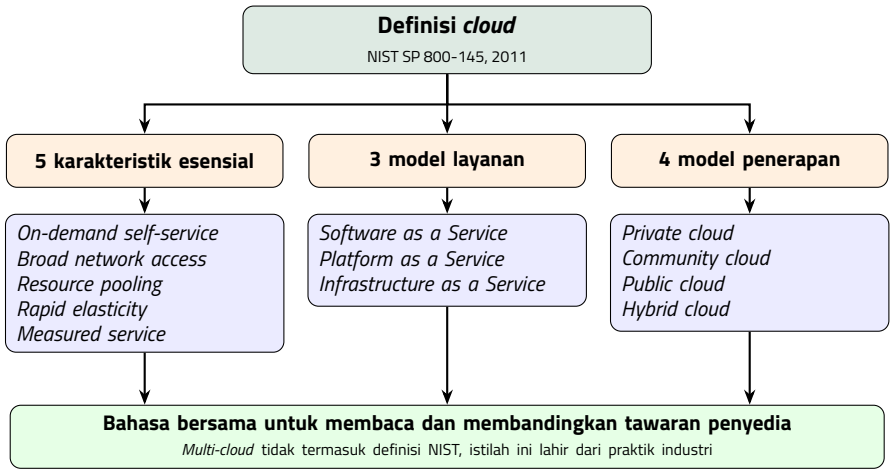
Gambar 2.1: Memilih model cloud berdasarkan workload dan kendali

Bab ini menjawab ketiga pertanyaan tersebut secara berurutan. Bahasa bersamanya diambil dari definisi *cloud* NIST SP 800-145 [91]. Pembagian tanggung jawab dibaca dari *shared responsibility model* yang diterbitkan penyedia [13, 96]. Batasan lokasi data diambil dari Peraturan Pemerintah (PP) 71/2019, PP 28/2024, Peraturan Menteri Kesehatan (Permenkes) 24/2022, dan Undang-Undang (UU) 27/2022 [119, 123, 83, 120]. Luarannya adalah matriks kendala penempatan *workload* untuk tonggak C1. Istilah *workload* di sini berarti satu aplikasi atau sekumpulan proses beserta datanya yang dijalankan sebagai satu kesatuan, misalnya SIMRS satu rumah sakit atau sistem penggajian.

2.2 Lima Karakteristik Esensial Cloud

Pertanyaan pertama: kapan sebuah layanan pantas disebut *cloud*? NIST SP 800-145 menjawabnya dengan satu definisi yang sejak 2011 belum direvisi dan masih berstatus final. *Cloud* adalah model yang memberi akses jaringan yang mudah dan sesuai kebutuhan ke kumpulan sumber daya komputasi bersama, seperti jaringan, server, penyimpanan, aplikasi, dan layanan. Sumber daya itu disediakan dan dilepas dengan cepat, dengan upaya pengelolaan dan interaksi dengan penyedia yang minimal [91]. Definisi ini disusun dari tiga kelompok: lima karakteristik

esensial, tiga model layanan, dan empat model penerapan, seperti terlihat pada Gambar 2.2.



Gambar 2.2: Struktur definisi *cloud* NIST SP 800-145: lima karakteristik, tiga model layanan, dan empat model penerapan

Kelima karakteristik itu menjadi alat uji. Sebuah layanan baru pantas disebut *cloud* bila kelimanya terpenuhi. Tabel 2.1 merangkum arti tiap karakteristik beserta pertanyaan yang dapat diajukan kepada penyedia untuk mengujinya.

Satu karakteristik perlu diberi perhatian khusus. Pada *resource pooling*, NIST mencatat bahwa pelanggan umumnya tidak tahu dan tidak mengendalikan lokasi persis sumber daya yang dipakainya, tetapi mungkin dapat menentukan lokasi pada tingkat yang lebih tinggi, misalnya negara atau pusat data [91]. Kalimat ini menjadi titik awal pembahasan lokasi data pada bagian akhir bab: kendali atas lokasi tidak hilang, tetapi bergeser dari "rak server mana" menjadi "negara dan *region* mana".

Analoginya adalah listrik di rumah. Penghuni menyalakan pendingin ruangan tanpa menelepon perusahaan listrik, dan pemakaian naik saat itu juga. Stop kontak tersedia di setiap ruangan. Pembangkitnya dipakai bersama jutaan pelanggan lain. Tagihan dihitung dari meteran. Bandingkan dengan genset sewaan bulanan: kapasitasnya tetap, biayanya tetap, dan menambah daya berarti menelepon pemilik genset lalu menunggu truk datang. Pusat data yang sekadar diberi label *cloud* sering lebih mirip genset sewaan yang ditempeli logo perusahaan listrik.

Dua contoh kasus nyata:

1. Misalkan kampanye vaksinasi influenza membuat janji temu lewat Medikara+ naik dari rata-rata sekitar 933 menjadi sekitar 1.500 per hari selama satu bulan. Dengan *rapid elasticity*, kapasitas ditambah selama sebulan itu

Tabel 2.1: Lima karakteristik esensial *cloud* menurut NIST SP 800-145 dan pertanyaan untuk mengujinya

Karakteristik	Artinya	Pertanyaan uji
On-demand self-service	Pelanggan menambah atau mengurangi sumber daya sendiri, tanpa menunggu petugas penyedia	Dapatkah server baru disiapkan hari ini tanpa tiket dan tanpa telepon?
Broad network access	Layanan diakses lewat jaringan dengan cara standar dari berbagai perangkat	Dapatkah layanan dipakai dari klinik, rumah, dan ponsel dengan pengamanan yang sama?
Resource pooling	Sumber daya penyedia dipakai bersama banyak pelanggan dan dibagi ulang sesuai permintaan	Apakah kapasitas diambil dari kumpulan bersama, dan sampai tingkat mana lokasinya dapat dipilih?
Rapid elasticity	Kapasitas bertambah dan berkurang dengan cepat, bahkan otomatis, mengikuti beban	Berapa lama kapasitas dapat dilipatgandakan, lalu dikembalikan seperti semula?
Measured service	Pemakaian diukur, dilaporkan, dan menjadi dasar tagihan	Apakah tagihan mengikuti pemakaian nyata, dan laporannya dapat dibaca pelanggan?

lalu dilepas. Tanpa elastisitas, Medikara harus menyiapkan kapasitas untuk 1.500 janji temu per hari sepanjang tahun, padahal kapasitas itu hanya terpakai penuh sekitar 30 hari.

2. Misalkan laboratorium rujukan perlu mengolah ulang 40.000 hasil pemeriksaan untuk audit mutu. Pekerjaan itu memakai 20 server selama 6 jam. Dengan *measured service*, yang dibayar adalah 120 jam-server. Dengan server milik sendiri, 20 server itu dibeli dan dirawat walaupun setelah audit hampir tidak dipakai.

Dua hal pada bagian ini sering tertukar, yaitu virtualisasi dan *cloud*. Pembagiannya begini. Virtualisasi adalah teknik memecah satu server fisik menjadi beberapa mesin virtual. Teknik ini memungkinkan *resource pooling*, tetapi belum menjamin empat karakteristik lainnya. Pusat data yang tervirtualisasi tetapi masih melayani permintaan lewat tiket, berkapasitas tetap, dan menagih biaya tetap adalah pusat data tervirtualisasi, bukan *cloud*. Perbedaan ini penting karena keduanya menghasilkan perbandingan biaya dan risiko yang berbeda.

Kasus Medikara Health: Menguji "SIMRS Cloud" Milik Vendor

Misalkan vendor SIMRS untuk rumah sakit Depok dan Bogor menawarkan pemindahan SIMRS kedua rumah sakit ke pusat data milik vendor dengan nama "SIMRS *Cloud*". Tim teknologi informasi (TI) Medikara memakai pertanyaan pada Tabel 2.1 untuk menguji tawaran itu, lalu mencatat temuannya pada Tabel 2.2.

Hasilnya, hanya satu karakteristik yang terpenuhi penuh dan satu terpenuhi sebagian. Layanan ini lebih tepat disebut *managed hosting*, yaitu server yang disewa dan dikelola pihak lain. Keputusan menyebutnya dengan nama yang tepat punya dua konsekuensi. Pertama, harga tawaran tidak boleh dibandingkan langsung dengan harga *cloud* publik, karena yang dibeli berbeda. Kedua, kontraknya dinilai sebagai alih daya ke vendor, sehingga pelajaran dari insiden kebocoran lewat vendor pada 2025 berlaku penuh: hak audit, kontrol akses petugas vendor, dan lokasi data harus tertulis di kontrak. Penilaian risiko pihak ketiga secara rinci dibahas di IT40813.

Tabel 2.2: Hasil uji lima karakteristik terhadap tawaran "SIMRS *Cloud*" (skenario rekaan)

Karakteristik	Temuan	Terpenuhi?
On-demand self-service	Penambahan server lewat tiket, dikerjakan paling lambat 10 hari kerja	Tidak
Broad network access	Hanya dapat diakses dari jaringan rumah sakit lewat sambungan privat; klinik dan ponsel tidak bisa	Sebagian
Resource pooling	Server fisik dipakai bersama 14 rumah sakit pelanggan vendor	Ya
Rapid elasticity	Kapasitas tetap sesuai kontrak; menambah kapasitas memerlukan adendum	Tidak
Measured service	Biaya tetap per tempat tidur per bulan, tanpa laporan pemakaian	Tidak

2.3 Model Layanan dan Shared Responsibility Model

Setelah tahu sebuah layanan memang *cloud*, pertanyaan berikutnya muncul: bagian mana yang disewa, dan bagian mana yang tetap dikelola sendiri? NIST membedakan tiga model layanan [91]. Pada *Infrastructure as a Service* (IaaS), pelanggan menyewa daya olah, penyimpanan, dan jaringan, lalu memasang dan mengelola sendiri sistem operasi serta aplikasinya. Pada *Platform as a Service* (PaaS), penyedia juga mengelola sistem operasi dan lingkungan menjalankan aplikasi, sehingga pelanggan cukup membawa aplikasi dan mengatur konfigurasinya. Pada *Software as a Service* (SaaS), pelanggan langsung memakai aplikasi milik penyedia

dan hanya mengatur pengaturan yang disediakan untuk pengguna.

Ada satu model yang populer setelah definisi NIST terbit, yaitu *Function as a Service* (FaaS). Pada FaaS, pelanggan hanya menulis potongan kode yang dijalankan ketika sebuah kejadian terjadi, misalnya berkas hasil laboratorium diunggah. Penyedia mengurus seluruh server, dan tagihan dihitung per eksekusi. FaaS dibahas rinci di Bab 8 bersama *container* dan Kubernetes.

Analoginya adalah cara bepergian. Memiliki mobil sendiri berarti membeli, merawat, dan menyetir sendiri, mirip pusat data milik sendiri. Menyewa mobil lepas kunci berarti mobil disediakan, tetapi pengemudi, bensin, dan rute diurus sendiri; itulah IaaS. Menyewa mobil beserta sopirnya berarti cukup menentukan tujuan dan jadwal; itulah PaaS. Naik taksi daring berarti tinggal memakai layanan angkutnya; itulah SaaS. Pada keempat cara itu, penumpang tetap bertanggung jawab atas barang bawaannya dan atas siapa yang diajak naik. Kalimat terakhir ini adalah inti *shared responsibility model*.

Tabel 2.3 memperlihatkan bagian yang disediakan penyedia dan bagian yang tetap dikelola pelanggan, beserta contoh yang biasa ditemui di tempat kerja.

Tabel 2.3: Model layanan, bagian yang disediakan penyedia, dan bagian yang tetap dikelola pelanggan

Model	Disediakan penyedia	Tetap dikelola pelanggan	Contoh sehari-hari
IaaS	Server virtual, penyimpanan, jaringan, gedung	Sistem operasi, aplikasi, data, akun, aturan jaringan	Mesin virtual untuk server laporan, penyimpanan arsip
PaaS	Ditambah sistem operasi, basis data terkelola, lingkungan aplikasi	Kode aplikasi, konfigurasi, data, akun	Basis data terkelola, platform untuk menjalankan aplikasi web
SaaS	Seluruh aplikasi siap pakai	Konfigurasi pengguna, data, akun	Surel dan kolaborasi, aplikasi penggajian
FaaS	Lingkungan yang menjalankan fungsi per kejadian	Kode fungsi, data, hak akses fungsi	Fungsi pengirim pengingat jadwal kontrol pasien

Pembagian kerja itu dirumuskan penyedia sebagai *shared responsibility model*. Amazon Web Services (AWS) meringkasnya sebagai keamanan *of the cloud*, yaitu infrastruktur yang dijaga penyedia, dan keamanan *in the cloud*, yaitu segala sesuatu yang dipasang dan diatur pelanggan [13]. Microsoft menegaskan bahwa pada semua model, data, akun, identitas, dan perangkat pengguna tetap menjadi tanggung jawab pelanggan [96]. Gambar 2.3 menyusun pembagian itu per lapisan, mengikuti dokumen Microsoft tersebut.

Gambar itu memperlihatkan pola yang konsisten. Makin ke kanan, makin

Lapisan	Pusat data sendiri	IaaS	PaaS	SaaS
Data dan informasi	Pelanggan	Pelanggan	Pelanggan	Pelanggan
Akun, identitas, dan perangkat pengguna	Pelanggan	Pelanggan	Pelanggan	Pelanggan
Infrastruktur identitas	Pelanggan	Pelanggan	Bersama	Bersama
Aplikasi	Pelanggan	Pelanggan	Bersama	Penyedia
Kontrol jaringan	Pelanggan	Pelanggan	Bersama	Penyedia
Sistem operasi	Pelanggan	Pelanggan	Penyedia	Penyedia
Host, jaringan, dan gedung fisik	Pelanggan	Penyedia	Penyedia	Penyedia

Pelanggan Bersama Penyedia

Kotak merah: dua lapisan yang selalu dipegang pelanggan, apa pun model layanannya

Gambar 2.3: Pembagian tanggung jawab per lapisan pada pusat data sendiri, IaaS, PaaS, dan SaaS, disederhanakan dari [96]

banyak lapisan yang berpindah ke penyedia. Namun dua lapisan teratas tidak pernah berpindah. Pelanggan yang memakai SaaS sekalipun tetap harus mengatur siapa yang boleh masuk, mencabut akun karyawan yang keluar, dan menentukan data apa yang boleh diunggah.

Dua contoh kasus nyata:

1. Misalkan Medikara memakai layanan surel dan kolaborasi berbentuk SaaS untuk sekitar 4.800 karyawan. Penyedia mengurus server dan pembaruan keamanannya. Misalkan audit internal menemukan 312 akun milik karyawan yang sudah keluar masih aktif. Temuan itu sepenuhnya tanggung jawab Medikara, karena pengelolaan akun berada di lapisan yang selalu dipegang pelanggan.
2. Misalkan basis data Medikara+ berjalan sebagai PaaS. Penyedia sudah menambal sistem operasi dan mesin basis data tepat waktu. Namun petugas vendor membuka akses publik ke basis data uji selama 3 hari agar pengujian lebih mudah. Pengaturan akses jaringan itu ada di sisi pelanggan, sehingga penyedia tidak dapat disalahkan bila data bocor lewat celah tersebut.

Dua hal pada *shared responsibility model* sering tertukar: tanggung jawab teknis dan akuntabilitas. Pembagiannya begini. Tanggung jawab teknis, misalnya menambal sistem operasi, dapat dialihkan ke penyedia lewat pilihan model layanan. Akuntabilitas atas data pasien tidak dapat dialihkan. Menurut UU 27/2022 tentang Pelindungan Data Pribadi (PDP), organisasi yang menentukan tujuan pemrosesan data adalah pengendali data, dan kewajibannya tetap melekat walaupun pem-

rosesannya dikerjakan pihak lain [120]. Ringkasnya, pekerjaan dapat dialihkan, akuntabilitas tidak.

Pendalaman Teknis (opsional)

Garis tanggung jawab sebenarnya ditentukan per layanan, bukan per penyedia. AWS memberi contoh berikut [13]. Amazon EC2 termasuk IaaS, sehingga pelanggan mengelola sistem operasi tamu beserta pembaruan keamanannya, perangkat lunak yang dipasang, dan konfigurasi *firewall* bawaan (*security group*) di tiap mesin. Untuk layanan yang lebih abstrak seperti Amazon S3 dan Amazon DynamoDB, penyedia mengoperasikan infrastruktur, sistem operasi, dan platformnya, sedangkan pelanggan mengelola data, pilihan enkripsi, klasifikasi aset, dan hak akses. Ada pula kendali bersama, misalnya pengelolaan tambalan dan konfigurasi, yang dikerjakan penyedia pada infrastruktur dan oleh pelanggan pada sistem tamu serta aplikasinya.

Akibatnya, satu sistem yang memakai lima layanan dapat memiliki lima garis tanggung jawab yang berbeda. Cara membacanya: buka halaman keamanan tiap layanan, cari bagian yang menjelaskan konfigurasi yang wajib dilakukan pelanggan, lalu catat sebagai daftar kendali yang diperiksa pada setiap perubahan. Daftar ini kelak menjadi bahan *guardrail* tata kelola di Bab 14.

Kasus Medikara Health: Memetakan Surel, ERP, dan Medikara+

Tim memetakan tiga sistem yang sudah berjalan ke model layanannya, lalu mencatat tanggung jawab yang tetap dipegang Medikara. Hasilnya ada pada Tabel 2.4. Detail sistem di bawah ini adalah skenario rekaan.

Pemetaan ini punya dua konsekuensi. Pertama, Medikara+ ternyata melibatkan tiga pihak: Medikara sebagai pemilik layanan, vendor sebagai pembuat dan pengelola aplikasi, dan penyedia *cloud* sebagai pemilik infrastruktur. Kontrak dengan vendor harus mengatur hal yang tidak tercakup kontrak vendor dengan penyedia, terutama akses petugas vendor ke data pasien. Kedua, model layanan ERP (*Enterprise Resource Planning*) keuangan belum diputuskan. Keputusan itu ditunda sampai penilaian portofolio aplikasi dengan pendekatan 7R di Bab 4.

Tabel 2.4: Pemetaan tiga sistem Medikara ke model layanan dan tanggung jawab yang tetap dipegang Medikara (skenario rekaan)

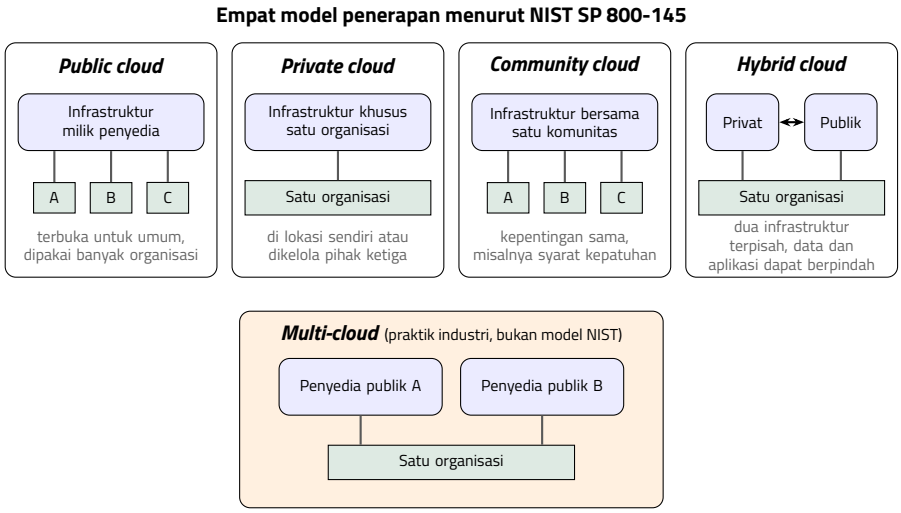
Sistem	Model layanan	Tanggung jawab yang tetap di Medikara
Surel kantor	SaaS, sekitar 4.800 akun	Siklus hidup akun, autentikasi multifaktor, aturan berbagi dokumen ke luar organisasi, masa simpan surel
ERP keuangan	Saat ini di server milik sendiri; kandidat SaaS atau IaaS	Seluruh lapisan selama masih di server sendiri; bila pindah, data, akun, dan pemisahan wewenang persetujuan pembayaran
Medikara+	Bagi Medikara mirip SaaS; vendor menjalankannya di atas IaaS penyedia publik	Data pasien, hak audit dan lokasi data dalam kontrak, persetujuan akses petugas vendor, pemantauan akun administrator

2.4 Model Penerapan: Public, Private, Hybrid, dan Multi-Cloud

Model layanan menjawab apa yang disewa. Pertanyaan berikutnya: infrastruktur itu milik siapa, dan dipakai bersama siapa? NIST membedakan empat model penerapan [91]. *Public cloud* terbuka untuk umum dan berada di fasilitas penyedia. *Private cloud* dipakai khusus oleh satu organisasi, walaupun pemilik dan pengelolanya dapat organisasi itu sendiri, pihak ketiga, atau gabungan keduanya, dan letaknya dapat di dalam atau di luar lokasi organisasi. *Community cloud* dipakai bersama sekelompok organisasi yang punya kepentingan sama, misalnya syarat keamanan atau kepatuhan yang serupa. *Hybrid cloud* adalah gabungan dua atau lebih infrastruktur yang tetap terpisah, tetapi dihubungkan sehingga data dan aplikasi dapat berpindah di antaranya.

Istilah kelima, *multi-cloud*, tidak ada dalam definisi NIST. Istilah ini lahir dari praktik industri dan berarti satu organisasi memakai layanan dari lebih dari satu penyedia *cloud* publik. Gambar 2.4 memperlihatkan kelima pola tersebut.

Analoginya adalah tempat tinggal. Rumah milik sendiri seperti *private cloud*: seluruhnya diatur sendiri, seluruh biayanya juga ditanggung sendiri. Apartemen sewaan seperti *public cloud*: gedungnya milik pengelola, penghuninya banyak, tetapi setiap unit terkunci. Rumah susun milik koperasi pegawai seperti *community cloud*. Keluarga yang punya rumah di Bogor dan menyewa unit apartemen di Jakarta untuk hari kerja menjalankan *hybrid cloud*, karena barang dapat dipindah di antara keduanya. Keluarga yang menyewa dua apartemen dari dua pengelola berbeda menjalankan *multi-cloud*.



Gambar 2.4: Empat model penerapan menurut NIST SP 800-145 dan *multi-cloud* sebagai praktik industri

Dua hal pada bagian ini sering tertukar, yaitu *hybrid cloud* dan *multi-cloud*. Pembagiannya begini. *Hybrid* menggabungkan infrastruktur privat dengan infrastruktur publik. *Multi-cloud* memakai lebih dari satu penyedia publik. Satu organisasi dapat menjalankan keduanya sekaligus. Laporan Flexera 2026, yang disusun dari survei terhadap 753 pengambil keputusan *cloud*, mencatat 73% organisasi menjalankan *hybrid cloud* dan 88% memakai lebih dari satu penyedia [55]. Angka ini perlu dibaca hati-hati. Respondennya adalah organisasi yang sudah memakai *cloud*, dan *multi-cloud* sering terjadi tanpa direncanakan: surel dari satu penyedia, analitik dari penyedia lain, aplikasi SaaS dari penyedia ketiga.

Tabel 2.5 merangkum *trade-off* tiap model dari empat sisi yang biasa ditanyakan manajemen.

Dua contoh kasus nyata:

1. Misalkan sistem informasi laboratorium rujukan berjalan di *private cloud* dengan kapasitas 2.000 hasil pemeriksaan per jam. Saat skrining kesehatan karyawan sebuah perusahaan mitra, permintaan naik menjadi 5.500 hasil per jam selama 3 hari. Dengan pola *hybrid*, pembuatan berkas laporan hasil dipindahkan sementara ke *public cloud*, sedangkan basis data pasien tetap di sisi privat. NIST menyebut pola pemindahan beban seperti ini sebagai *cloud bursting* [91].
2. Misalkan Medikara menjalankan Medikara+ pada penyedia A dan analitik data pada penyedia B. Tim platform yang berisi 6 orang harus menguasai dua konsol, dua cara mengelola identitas, dan dua format tagihan. Setiap

Tabel 2.5: *Trade-off* model penerapan dari sisi biaya, keterampilan, ketergantungan, dan kendali lokasi data

Model	Biaya	Keterampilan tim	Ketergantungan	Kendali lokasi data
Public	Rendah di awal, naik mengikuti pemakaian	Satu platform, banyak bahan belajar	Tinggi pada satu penyedia	Dipilih per *region*
Private	Tinggi di awal, investasi perangkat	Mengelola seluruh lapisan sendiri	Rendah pada penyedia, tinggi pada tim sendiri	Penuh
Community	Dibagi antaranggota	Perlu tata kelola bersama	Pada kesepakatan komunitas	Sesuai kesepakatan
Hybrid	Dua struktur biaya sekaligus	Dua lingkungan dan penghubungnya	Sedang	Data sensitif dapat tetap di sisi privat
Multi-cloud	Biaya transfer data antarpenyedia	Dua konsol, dua model identitas, dua tagihan	Rendah pada satu penyedia	Perlu dicek per penyedia

data yang dikirim dari penyedia A ke penyedia B juga dikenai biaya transfer data keluar, yang dibahas di Bab 3.

Kasus Medikara Health: Model Penerapan untuk SIMRS di Lima Rumah Sakit

Kelima rumah sakit Medikara menjalankan SIMRS monolitik di ruang server masing-masing, dengan versi yang berbeda karena berasal dari akuisisi. Tim menimbang empat opsi pada Tabel 2.6. Kondisi yang disebut di tabel adalah skenario rekaan.

Tim mengusulkan opsi C sebagai model transisi. SIMRS tetap dekat dengan rumah sakit selama versinya belum diseragamkan, sedangkan layanan integrasi ke BPJS (Badan Penyelenggara Jaminan Sosial) Kesehatan dan SATUSEHAT, Medikara+, serta analitik dipindahkan ke *region* Jakarta. Usulan ini punya dua konsekuensi. Pertama, Medikara harus membangun sambungan jaringan yang andal antara lima rumah sakit dan *region* Jakarta. Kedua, opsi C bukan tujuan akhir. Arah selanjutnya ditentukan setelah penilaian 7R di Bab 4 dan diagnosis arsitektur di Bab 7.

Tabel 2.6: Opsi model penerapan untuk SIMRS lima rumah sakit Medikara (skenario rekaan)

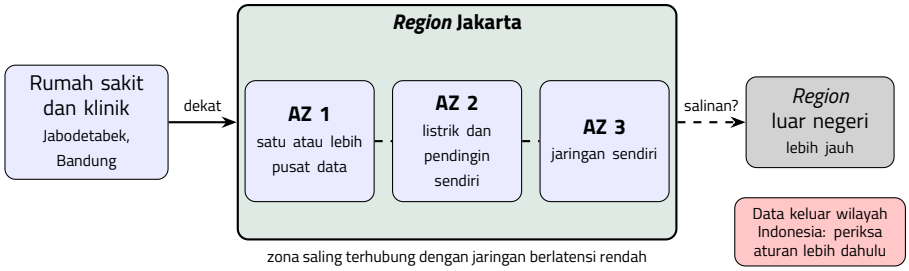
Opsi	Kelebihan	Kelemahan
A. Tetap di lima ruang server	Tanpa migrasi; layanan tetap berjalan saat internet putus	Perangkat berumur 6 sampai 9 tahun; lima tim kecil mengurus hal yang sama
B. *Private cloud* terpusat	Satu tempat untuk diamankan dan diaudit	Lima versi SIMRS tetap berbeda; seluruh rumah sakit bergantung pada satu sambungan
C. *Hybrid*: SIMRS dekat rumah sakit, layanan pendukung di *region* Jakarta	Operasi pelayanan tidak bergantung pada internet; layanan baru dapat tumbuh di *cloud*	Dua lingkungan harus dikelola; perlu penghubung yang aman
D. Seluruhnya di *public cloud*	Elastis, tanpa ruang server	Misalkan sambungan rumah sakit Bandung putus 4 kali setahun dengan total 19 jam; instalasi gawat darurat tidak boleh berhenti selama itu

2.5 Region, Availability Zone, dan Region Cloud di Indonesia

Model penerapan menjawab milik siapa. Pertanyaan berikutnya lebih konkret: di kota mana server itu berada? Penyedia *cloud* publik membagi infrastrukturnya menjadi *region*, yaitu wilayah geografis yang berisi beberapa *availability zone* (AZ). Satu AZ terdiri atas satu atau lebih pusat data dengan pasokan listrik, pendingin, dan jaringan sendiri. AZ dalam satu *region* terhubung dengan jaringan berlatensi rendah, sehingga aplikasi dapat dijalankan di beberapa zona sekaligus [12, 68]. Oracle memakai istilah *availability domain*, dan sebagian *region*-nya hanya memiliki satu *availability domain* yang dibagi lagi menjadi beberapa *fault domain* [118].

Pilihan *region* menentukan tiga hal sekaligus. Pertama, latensi, yaitu waktu tempuh data antara pengguna dan server. Kedua, ketersediaan, karena jumlah zona menentukan seberapa jauh sistem dapat bertahan bila satu pusat data terganggu. Ketiga, lokasi data, karena *region* menentukan di negara mana data berada. Gambar 2.5 memperlihatkan ketiganya.

Analoginya adalah bank dengan beberapa kantor cabang di satu kota. Kota itu adalah *region*. Setiap kantor cabang punya genset dan jalur listrik sendiri, seperti AZ. Bila satu cabang kebanjiran, cabang lain di kota yang sama tetap melayani nasabah. Nasabah juga dapat memilih kota tempat rekeningnya dikelola, dan pilihan itu menentukan aturan kota mana yang berlaku.



Gambar 2.5: *Region* berisi beberapa *availability zone*; pilihan *region* menentukan latensi, ketersediaan, dan lokasi data

Latensi punya batas fisik. Cahaya di dalam serat optik merambat sekitar 200.000 kilometer per detik, sehingga setiap 1.000 kilometer jarak menambah sekitar 10 milidetik waktu pulang-pergi, belum termasuk waktu antre di perangkat jaringan. Jarak tidak dapat dikompensasi dengan membeli server yang lebih cepat.

Dua contoh kasus nyata:

1. Misalkan tim TI Medikara mengukur waktu pulang-pergi dari rumah sakit Bandung: rata-rata 9 milidetik ke *region* Jakarta, 24 milidetik ke *region* Singapura, dan 190 milidetik ke *region* di pantai barat Amerika Serikat. Satu halaman SIMRS yang mengirim 40 permintaan secara berurutan akan menunggu sekitar 0,36 detik bila servernya di Jakarta, tetapi sekitar 7,6 detik bila servernya di Amerika Serikat. Selisih itu terasa langsung oleh petugas pendaftaran.
2. Misalkan sistem antrean klinik dijalankan di satu AZ saja, lalu zona itu terganggu selama 3 jam. Selama 3 jam itu, 18 klinik kembali ke antrean manual. Bila sistem dijalankan di dua AZ, layanan berpindah ke zona lain, tetapi biaya server kira-kira berlipat dua karena ada dua salinan yang selalu siap. Rincian strategi ketersediaan dibahas di Bab 12.

Per Oktober 2026, empat penyedia global mengoperasikan *region* di Indonesia, seperti tercantum pada Tabel 2.7. Beberapa penyedia lain, termasuk penyedia lokal, juga mengoperasikan pusat data di Indonesia; tim yang mempertimbangkannya perlu memeriksa daftar *region* resmi masing-masing penyedia.

Tabel itu menyimpan satu jebakan. Adanya *region* Jakarta tidak berarti seluruh layanan penyedia tersedia di Jakarta. Layanan baru biasanya lebih dahulu hadir di *region* tertua penyedia, lalu menyebar bertahap. Daftar layanan per *region* diterbitkan penyedia dan perlu diperiksa sebelum keputusan diambil [11]. Kenyataan ini punya dua konsekuensi. Pertama, *workload* yang membutuhkan layanan yang belum ada di Jakarta terpaksa memakai *region* luar negeri, dan pada saat itu pertanyaan lokasi data muncul. Kedua, jumlah zona berbeda antarpenyedia. *Re-*

Tabel 2.7: *Region cloud* milik empat penyedia global di Indonesia, per Oktober 2026

Penyedia	Nama dan kode *region*	Mulai beroperasi	Jumlah zona
AWS	Asia Pacific (Jakarta), `ap-southeast-3`	Desember 2021 [3]	3 AZ
Google Cloud	Jakarta, `asia-southeast2`	Juni 2020 [67]	3 zona
Microsoft Azure	Indonesia Central, `indonesiacentral`	Tersedia April 2025, diresmikan Mei 2025 [102]	3 AZ
Oracle Cloud	Indonesia North (Batam), `ap-batam-1`	Mei 2025 [117]	1 *availability domain*

gion dengan satu *availability domain* tidak dapat memberi perlindungan antarzona yang sama dengan *region* tiga zona, sehingga rancangan ketersediaannya harus berbeda.

Pendalaman Teknis (opsional)

Jumlah zona dan katalog layanan dapat dibaca langsung lewat *command line* penyedia. Listing 2.1 memperlihatkan perintah untuk AWS dan Google Cloud. Seluruh perintah hanya membaca katalog dan tidak membuat sumber daya berbayar. *Region* Jakarta pada AWS harus diaktifkan lebih dahulu di pengaturan akun, sehingga perintah pertama memeriksa statusnya. Katalog layanan AWS dibaca dari parameter publik *global infrastructure* di AWS Systems Manager [14].

Listing 2.1: Membaca zona dan katalog layanan *region* Jakarta, tersedia di `sources/chapter02/cek-zona-region.sh`

```
# Status opt-in region Jakarta pada akun AWS.
aws ec2 describe-regions --all-regions --region "$REGION_AWS_PEMBANDING" \
  --region-names "$REGION_AWS_JAKARTA" \
  --query 'Regions[].[RegionName,OptInStatus]' --output text

# Katalog layanan AWS, satu nama per baris.
for REGION in "$REGION_AWS_JAKARTA" "$REGION_AWS_PEMBANDING"; do
  aws ssm get-parameters-by-path --region "$REGION_AWS_PEMBANDING" \
    --path "$JALUR_KATALOG/$REGION/services" \
    --query 'Parameters[].Value' --output text \
    | tr '\t' '\n' | sort > "layanan-aws-$REGION.txt"
done

# Nama zona Google Cloud di region Jakarta.
gcloud compute zones list --filter="region:$REGION_GOOGLE_JAKARTA" \
  --format="value(name)"
```

Dua berkas daftar layanan itu lalu dibandingkan dengan `sources/chapter02/bandingkan_katalog_region.py`, yang mencetak layanan

yang ada di *region* pembanding tetapi belum ada di Jakarta. Untuk Google Cloud dan penyedia lain, daftar layanan per *region* disalin dari halaman lokasi resmi ke berkas teks, satu nama per baris, lalu dibandingkan dengan skrip yang sama. Nama layanan tidak dapat dibandingkan lintas penyedia secara langsung, karena tiap penyedia memberi nama berbeda untuk layanan yang serupa. Perbandingan lintas penyedia dilakukan per kategori kebutuhan, misalnya "basis data relasional terkelola", setelah daftar selisih per penyedia tersusun.

2.6 Lokasi Data dan Kedaulatan Data dalam Regulasi Indonesia

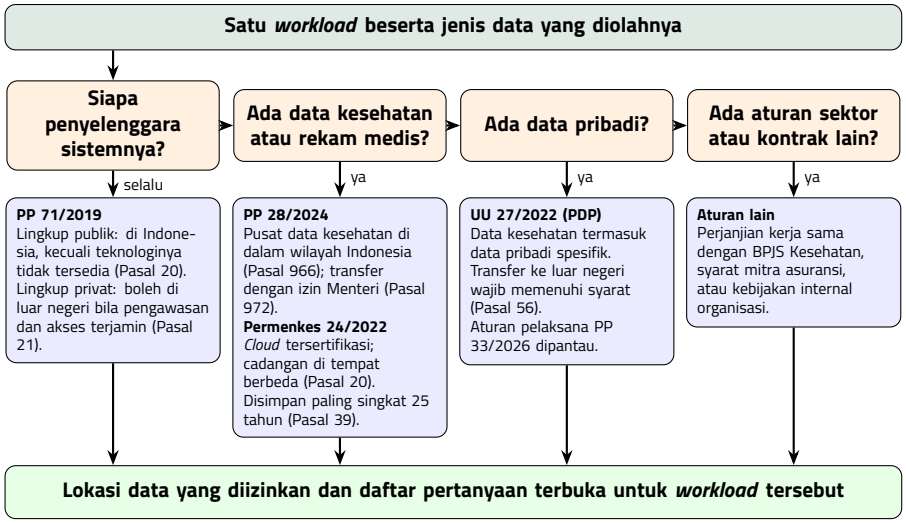
Region Jakarta menjawab di mana server berada. Pertanyaan berikutnya bersifat hukum: data mana yang boleh berada di luar Indonesia, dan dengan syarat apa? Sebelum masuk ke aturannya, tiga istilah perlu dipisahkan, karena ketiganya sering tertukar. Pembagiannya ada pada Tabel 2.8.

Tabel 2.8: Tiga istilah lokasi data yang sering tertukar

Istilah	Pertanyaan yang dijawab	Contoh
Residensi data (*data residency*)	Di negara mana data disimpan dan diolah secara fisik?	Rekam medis disimpan di *region* Jakarta
Kedaulatan data (*data sovereignty*)	Hukum negara mana yang berlaku atas data, dan siapa yang dapat memaksa akses ke data itu?	Data di *region* luar negeri dapat diminta aparat negara tempat data berada
Lokalisasi data (*data localization*)	Apakah ada aturan yang mewajibkan data, atau salinannya, tetap di dalam negeri?	Pasal 20 PP 71/2019 untuk penyelenggara lingkup publik; Pasal 966 PP 28/2024 untuk pusat data kesehatan

Analoginya adalah arsip kertas sebuah perusahaan. Gudang tempat arsip disimpan adalah residensi. Bila gudang itu berada di negara lain, aparat negara itu dapat menyegelnya menurut hukum setempat; itulah kedaulatan. Aturan yang mewajibkan arsip tertentu tetap berada di gudang dalam negeri adalah lokalisasi. Ketiganya berhubungan, tetapi jawabannya dapat berbeda untuk arsip yang sama.

Pertanyaan kemudian muncul: aturan mana yang mengikat Medikara? Jawabannya berlapis. Lapisan pertama berlaku untuk semua sistem elektronik, lapisan berikutnya khusus sektor kesehatan, lalu lapisan data pribadi, dan terakhir kontrak. Gambar 2.6 menyusun lapisan itu sebagai urutan pertanyaan, sekaligus merangkum isi pokok tiap aturan yang dibahas di bawah ini.



Gambar 2.6: Urutan pertanyaan untuk menentukan lokasi data yang diizinkan bagi satu *workload*

PP 71/2019 dan Permenkominfo 5/2020

PP 71/2019 tentang Penyelenggaraan Sistem dan Transaksi Elektronik (PSTE) membedakan Penyelenggara Sistem Elektronik (PSE) lingkup publik dan lingkup privat [119]. PSE lingkup publik adalah instansi penyelenggara negara atau institusi yang ditunjuknya. Menurut Pasal 20, PSE lingkup publik wajib mengelola, memproses, dan menyimpan sistem dan data elektroniknya di wilayah Indonesia, kecuali teknologi penyimpanannya tidak tersedia di dalam negeri menurut kriteria sebuah komite yang dibentuk menteri. PSE lingkup publik yang memakai layanan pihak ketiga juga wajib mengklasifikasikan datanya berdasarkan risiko.

PSE lingkup privat, yaitu orang, badan usaha, dan masyarakat, diatur Pasal 21. PSE lingkup privat boleh mengelola, memproses, dan menyimpan sistem dan data elektroniknya di dalam maupun di luar wilayah Indonesia. Bila memilih luar negeri, PSE tersebut wajib memastikan efektivitas pengawasan oleh kementerian atau lembaga serta penegakan hukum, dan wajib memberikan akses terhadap sistem dan datanya untuk keperluan itu. Untuk sektor keuangan, ketentuannya diatur lebih lanjut oleh otoritas pengatur dan pengawas sektor keuangan.

Peraturan Menteri Komunikasi dan Informatika (Permenkominfo) 5/2020, yang diubah dengan Permenkominfo 10/2021, merinci kewajiban PSE lingkup privat [84]. Dua kewajiban yang relevan untuk bab ini adalah pendaftaran sistem elektronik dan pemberian akses ke sistem serta data elektronik untuk pengawasan dan penegakan hukum. Rencana revisi PP 71/2019 masih dievaluasi Kementerian Komunikasi dan Digital (Komdigi) per Oktober 2026. Karena itu, status kedua aturan perlu dicek ulang di JDIH (Jaringan Dokumentasi dan Informasi Hukum)

setiap kali matriks diperbarui.

Permenkes 24/2022 tentang Rekam Medis

Permenkes 24/2022 mewajibkan fasilitas pelayanan kesehatan menyelenggarakan rekam medis elektronik [83]. Pasal 20 mengatur penyimpanannya. Rekam medis elektronik disimpan pada media penyimpanan berbasis digital, dan penyimpanannya harus menjamin keamanan, keutuhan, kerahasiaan, dan ketersediaan data. Media itu dapat berupa server, komputasi awan (*cloud computing*) yang tersertifikasi sesuai peraturan perundang-undangan, atau media digital lain yang diakui sesuai perkembangan teknologi. Fasilitas pelayanan kesehatan juga wajib memiliki cadangan data yang disimpan di tempat berbeda, dibuat secara berkala, dan dituangkan dalam standar prosedur operasionalnya. Pasal 39 menetapkan masa simpan rekam medis elektronik paling singkat 25 tahun sejak kunjungan terakhir pasien.

Pasal 20 membuka pintu bagi *cloud*, tetapi menyisakan pertanyaan yang tidak dijawab pasal itu sendiri: sertifikasi apa yang dimaksud, dan siapa yang menerbitkannya? Penyedia *cloud* umumnya memegang sertifikasi internasional seperti ISO/IEC 27001, 27017, dan 27018. Apakah sertifikasi itu cukup untuk memenuhi Pasal 20 harus dijawab lewat aturan turunan atau konfirmasi tertulis dari Kementerian Kesehatan, bukan lewat asumsi tim.

PP 28/2024: Aturan Khusus Data Kesehatan

Pasal 21 PP 71/2019 adalah aturan umum. Untuk data kesehatan, PP 28/2024 lebih khusus. Pasal 966 ayat (2) menyatakan pangkalan data kesehatan berupa pusat data yang berada di dalam wilayah Indonesia. Ayat (5) mengizinkan jasa pihak lain bila dilaksanakan di dalam wilayah Indonesia, berdasarkan perjanjian kerja sama, dengan data yang dimiliki dan dikendalikan penuh oleh penyelenggara. Pasal 972 mengatur transfer ke luar wilayah hukum untuk tujuan spesifik dan terbatas, dengan izin Menteri, serta memenuhi syarat transfer data pribadi [123]. Bab ini mengikuti pembacaan IT40813 Bab 3. Salinan cadangan diperlakukan sebagai bagian dari penyimpanan, walaupun Pasal 966 tidak menyebut lokasi cadangan secara tersendiri. Perlakuan itu penafsiran, dan kasus nyata tetap perlu dinilai penasihat hukum.

UU 27/2022 tentang PDP dan PP 33/2026

UU 27/2022 tentang PDP berlaku penuh sejak masa penyesuaian dua tahunnya berakhir pada Oktober 2024 [120]. Dua ketentuan langsung menyentuh lokasi data. Pertama, data dan informasi kesehatan termasuk data pribadi yang bersifat

spesifik, sehingga pemrosesannya menuntut pelindungan lebih ketat. Kedua, Pasal 56 mengatur transfer data pribadi ke luar wilayah hukum Indonesia. Pengendali wajib memastikan negara tujuan memiliki tingkat pelindungan yang setara atau lebih tinggi. Bila tidak, pengendali wajib memastikan pelindungan yang memadai dan bersifat mengikat. Bila keduanya tidak terpenuhi, pengendali wajib mendapat persetujuan subjek data. Aturan pelaksananya, PP 33/2026, menurut ringkasan firma hukum ditetapkan 16 Juli 2026 dan berlaku 16 Januari 2027 [1]. Teks resminya perlu dibaca di JDIH sebelum dikutip. Pembahasan rinci UU PDP, penilaian dampak, dan risiko pihak ketiga ada di IT40813.

Membaca aturan berlapis menuntut satu prinsip: lapisan yang paling ketat menentukan hasil akhir. PP 71/2019 mengizinkan PSE lingkup privat menyimpan data di luar negeri, tetapi izin itu belum menjawab PP 28/2024 untuk data kesehatan, syarat Permenkes untuk rekam medis, dan syarat UU PDP untuk transfer data pribadi. Boleh menurut satu aturan belum berarti boleh menurut seluruh aturan.

Dua contoh kasus nyata:

1. Dengan sekitar 1,2 juta kunjungan per tahun dan masa simpan paling singkat 25 tahun, Medikara akan menyimpan catatan dari sekitar 30 juta kunjungan. Misalkan rata-rata catatan satu kunjungan tanpa citra radiologi berukuran 0,5 megabyte; totalnya sekitar 15 terabyte. Kontrak *cloud* umumnya berjangka 1 sampai 3 tahun, jauh lebih pendek dari 25 tahun. Rencana keluar dari penyedia, termasuk cara memindahkan 15 terabyte itu, harus disiapkan sejak awal.
2. Misalkan aplikasi penggajian berbentuk SaaS untuk 4.800 karyawan menyimpan datanya di *region* Singapura. Data gaji dan rekening adalah data pribadi. Transfernya ke Singapura harus memenuhi Pasal 56 UU PDP, sehingga Medikara perlu menilai tingkat pelindungan di negara tujuan atau menyiapkan perjanjian yang mengikat sebelum kontrak ditandatangani. Hasil pemeriksaan kesehatan berkala karyawan adalah data kesehatan yang bersifat spesifik, sehingga juga tunduk pada PP 28/2024 Pasal 966 dan 972 dan sebaiknya tidak ikut disimpan pada aplikasi itu.

Kasus Medikara Health: Tiga Jenis Data, Tiga Kendala

Medikara Health adalah badan usaha swasta, sehingga termasuk PSE lingkup privat. Tim menerapkan urutan pada Gambar 2.6 untuk tiga jenis data, dan hasilnya tercantum pada Tabel 2.9.
Hasil ini punya dua konsekuensi. Pertama, rekam medis elektronik ditempatkan di Indonesia karena PP 28/2024 Pasal 966, walaupun PP 71/2019 sebagai aturan umum tidak melarang luar negeri. Pertanyaan sertifikasi

tetap perlu dijawab untuk memilih penyedia di Indonesia. Kedua, data karyawan di luar data kesehatan adalah satu-satunya jenis data yang secara wajar dapat berada di luar negeri, asalkan syarat transfer UU PDP terpenuhi. Daftar pertanyaan terbuka pada kolom terakhir menjadi bahan diskusi dengan tim hukum dan masuk ke *risk register* di IT40813.

Tabel 2.9: Kendala lokasi untuk tiga jenis data Medikara Health

Jenis data	Aturan yang mengikat	Usulan lokasi	Pertanyaan terbuka
Rekam medis elektronik	PP 28/2024 Pasal 966 dan 972; PP 71/2019 Pasal 21; Permenkes 24/2022 Pasal 20 dan 39; UU PDP	Indonesia, cadangan di lokasi berbeda di Indonesia	Sertifikasi *cloud* apa yang diakui Kementerian Kesehatan?
Data klaim BPJS Kesehatan	PP 28/2024 Pasal 966; UU PDP (diagnosis dan nomor peserta); perjanjian kerja sama dengan BPJS Kesehatan	Indonesia	Apakah perjanjian kerja sama mengatur lokasi dan keamanan data klaim?
Data karyawan di luar data kesehatan	PP 71/2019 Pasal 21; UU PDP Pasal 56 bila keluar negeri	Indonesia atau luar negeri dengan syarat	Di negara mana vendor SaaS menyimpan data, dan apa dasar transfernya?

2.7 Aktivitas: Matriks Kendala Penempatan Workload

Tujuan

Tim menghasilkan matriks yang menunjukkan model layanan, model penerapan, dan lokasi data yang diizinkan untuk setiap *workload* organisasi. Matriks ini menjadi batasan bagi seluruh keputusan arsitektur berikutnya: *workload* yang datanya wajib di Indonesia tidak boleh muncul di rancangan yang memakai *region* luar negeri.

Pembagian Peran

- Anggota berlatar hukum, kesehatan, atau bisnis menelusuri jenis data tiap *workload* dan aturan yang mengikatnya, dengan `sources/chapter02/ringkasan-regulasi-lokasi-data.md` sebagai titik awal.
- Anggota berlatar teknis menilai model layanan dan model penerapan yang masuk akal, termasuk ketersediaan layanan di *region* Jakarta.

- Seluruh anggota memeriksa konsistensi matriks: tidak ada baris yang usulan lokasinya bertentangan dengan aturan pada baris yang sama.

Langkah Kerja

1. Daftar 8 sampai 12 *workload* organisasi, misalnya SIMRS, ERP keuangan, aplikasi pasien, sistem laboratorium, surel, penggajian, dan pelaporan manajemen. Pakai profil organisasi dari Bab 1.
2. Tentukan jenis data tiap *workload* dan klasifikasinya: tanpa data pribadi, data pribadi umum, data pribadi spesifik, atau rekam medis elektronik.
3. Cocokkan setiap *workload* dengan aturan yang berlaku mengikuti urutan pada Gambar 2.6, lalu tulis pasal yang dirujuk.
4. Usulkan model layanan dan model penerapan, beserta alasan singkat yang menyebut *trade-off*-nya.
5. Tandai lokasi data yang diizinkan dan tulis pertanyaan yang belum terjawab, lengkap dengan nama anggota yang akan menelusurinya.

Tabel 2.10 memperlihatkan contoh lima baris matriks untuk Medikara Health. Kolom lengkapnya, termasuk alasan, pertanyaan terbuka, dan status, ada di templat.

Luaran dan Kaitan dengan Capstone

Luarannya adalah matriks kendala penempatan *workload* untuk tonggak C1 Konteks dan Strategi, ditambah satu halaman catatan yang merangkum tiga pertanyaan terbuka paling penting. Matriks ini dipakai lagi pada Bab 3 untuk memperkirakan biaya per *workload*, dan pada Bab 4 sebagai batasan usulan 7R. Pertanyaan terbuka yang menyangkut risiko diserahkan ke tim IT40813 untuk dicatat di *risk register*.

Kriteria Penilaian

- Kelengkapan: 8 sampai 12 *workload*, setiap baris terisi dan menyebut pasal yang dirujuk.
- Ketepatan: klasifikasi data dan aturan yang dipilih sesuai dengan jenis datanya.
- Konsistensi: usulan lokasi tidak bertentangan dengan aturan pada baris yang sama.
- Kejujuran analisis: pertanyaan yang belum terjawab ditulis terang, bukan diisi dengan asumsi.

Tabel 2.10: Contoh lima baris matriks kendala penempatan *workload* Medikara Health

Workload	Jenis data	Aturan	Model layanan	Model penerapan	Lokasi diizinkan
SIMRS rumah sakit	Rekam medis elektronik	PP 28/2024 Pasal 966; Permenkes 24/2022 Pasal 20, 39; UU PDP	IaaS (transisi)	*Hybrid*	Indonesia
Integrasi klaim BPJS Kesehatan	Diagnosis, tindakan, nomor peserta	PP 28/2024 Pasal 966; UU PDP; perjanjian kerja sama	PaaS	*Public*, *region* Jakarta	Indonesia
Medikara+	Data pasien, jadwal, telekonsultasi	PP 28/2024 Pasal 966; UU PDP; Permenkes 24/2022 untuk catatan konsultasi	SaaS dari vendor	*Public*	Indonesia
Penggajian	Data karyawan, tanpa data kesehatan	UU PDP Pasal 56	SaaS	*Public*	Luar negeri dengan syarat
Situs profil perusahaan	Informasi publik	PP 71/2019 Pasal 21	PaaS	*Public*	Bebas

Berkas Pendukung

Templat matriks tersedia di `sources/chapter02/templat-matriks-penempatan-workload.xlsx`. Lembar Petunjuk menjelaskan arti setiap kolom, sedangkan lembar Matriks memuat contoh Medikara dan baris kosong dengan daftar pilihan. Ringkasan aturan ada di `sources/chapter02/ringkasan-regulasi-lokasi-data.md`, dan lembar isian ketiga latihan ada di `sources/chapter02/lembar-latihan-bab02.md`. Tim yang memakai organisasi nyata wajib memperoleh izin organisasi dan menganonimkan namanya. Data pribadi pasien atau karyawan tidak boleh dimasukkan ke matriks.

Pendalaman Teknis (opsional)

Anggota berlatar teknis membandingkan katalog layanan *region* Jakarta pada dua penyedia dengan perintah dan skrip di kotak pendalaman seksi *region*, lalu mencatat layanan yang dibutuhkan tetapi belum tersedia pada kolom catatan matriks.

2.8 Ringkasan

Kata *cloud* dipakai untuk banyak hal yang berbeda, sehingga organisasi memerlukan alat uji sebelum membandingkan tawaran. NIST SP 800-145 menyediakan alat itu dalam bentuk lima karakteristik esensial: *on-demand self-service*, *broad network access*, *resource pooling*, *rapid elasticity*, dan *measured service*. Layanan yang hanya memenuhi sebagian, seperti "SIMRS *Cloud*" pada kasus Medikara, lebih tepat diperlakukan sebagai *managed hosting* dan dinilai sebagai alih daya ke vendor.

Model layanan menentukan lapisan mana yang disewa. Makin jauh dari IaaS menuju SaaS, makin banyak lapisan yang dikelola penyedia. Namun data, akun, identitas, dan perangkat pengguna tetap dipegang pelanggan pada model apa pun. Tanggung jawab teknis dapat dialihkan lewat pilihan model, sedangkan akuntabilitas atas data pasien tetap melekat pada Medikara sebagai pengendali data.

Model penerapan menentukan milik siapa infrastruktur itu dan dipakai bersama siapa. NIST mengenal *public*, *private*, *community*, dan *hybrid cloud*, sedangkan *multi-cloud* adalah praktik industri. Setiap pilihan membawa *trade-off* pada biaya, keterampilan tim, ketergantungan pada penyedia, dan kendali atas lokasi data. Untuk SIMRS lima rumah sakit, pola *hybrid* dipilih sebagai model transisi, bukan tujuan akhir.

Region dan *availability zone* membuat lokasi menjadi keputusan yang konkret. Empat penyedia global sudah mengoperasikan *region* di Indonesia, tetapi tidak semua layanan tersedia di sana dan jumlah zonanya berbeda. Latensi dibatasi jarak fisik, sehingga *region* yang jauh langsung terasa oleh petugas di loket pendaftaran.

Lokasi data di Indonesia diatur berlapis: PP 71/2019 untuk semua sistem elektronik, PP 28/2024 untuk data kesehatan, Permenkes 24/2022 untuk rekam medis, UU PDP untuk data pribadi dengan PP 33/2026 yang masih dipantau, lalu kontrak dengan pihak lain. Lapisan yang paling ketat menentukan hasil akhir, dan pertanyaan yang belum terjawab harus ditulis terang, bukan ditutup dengan asumsi. Pesan utama bab ini: sebelum memilih penyedia, organisasi harus tahu apa yang dibeli, siapa memegang bagian mana, dan data mana yang boleh berada di mana.

2.9 Latihan

Latihan 1: Klasifikasi Layanan dan Tanggung Jawab yang Tersisa

Latihan ini membuktikan bahwa model layanan apa pun selalu menyisakan tanggung jawab keamanan bagi organisasi.

1. Pilih lima layanan yang dipakai sehari-hari di tempat kerja atau di Medikara Health.
2. Tentukan model layanannya (IaaS, PaaS, SaaS, atau FaaS) dengan memakai Tabel 2.3.
3. Untuk setiap layanan, tulis lapisan yang dikelola penyedia dengan memakai Gambar 2.3.
4. Tulis tanggung jawab keamanan yang tetap dipegang organisasi, sekonkret mungkin.
5. Tandai satu tanggung jawab per layanan yang menurut pengamatan belum dijalankan dengan baik.

Tabel 2.11 berisi contoh terisi untuk Medikara Health, sedangkan Tabel 2.12 adalah lembar isiannya.

Tabel 2.11: Latihan 1: contoh klasifikasi lima layanan Medikara Health

Layanan	Model	Tanggung jawab yang tetap di organisasi	Belum berjalan baik
Surel dan kolaborasi, 4.800 akun	SaaS	Siklus hidup akun, autentikasi multifaktor, aturan berbagi	312 akun mantan karyawan masih aktif
Basis data terkelola Medikara+	PaaS	Akses jaringan, akun basis data, enkripsi dan kuncinya	Akses publik ke basis data uji
Mesin virtual server laporan	IaaS	Tambalan sistem operasi, aturan *firewall*, cadangan	Tambalan tertunda 2 bulan
Aplikasi penggajian	SaaS	Hak akses bagian sumber daya manusia, persetujuan transfer data	Lokasi data belum diketahui
Fungsi pengingat jadwal kontrol	FaaS	Kode fungsi, hak akses fungsi ke data pasien	Hak akses terlalu luas

Tabel 2.12: Latihan 1: lembar isian klasifikasi layanan

Layanan	Model	Tanggung jawab yang tetap di organisasi	Belum berjalan baik
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Pada tabel isian, model layanan mana yang paling banyak menyisakan

tanggung jawab yang belum berjalan baik? Apakah hasil itu sejalan dengan anggapan bahwa SaaS paling aman?

2. Tanggung jawab mana yang muncul di lebih dari satu baris? Apa artinya bagi prioritas perbaikan kontrol akses yang menjadi temuan audit Medikara?

Latihan 2: Cadangan Rekam Medis di Region Luar Negeri

Latihan ini membuktikan bahwa keputusan lokasi data tidak dapat diambil hanya dari selisih harga. Misalkan Medikara menerima dua penawaran untuk cadangan rekam medis elektronik sebesar 15 terabyte: Rp38 juta per bulan di *region* Jakarta dan Rp29 juta per bulan di sebuah *region* luar negeri. Selisihnya Rp9 juta per bulan, atau Rp108 juta per tahun.

1. Hitung selisih biaya per tahun dan bandingkan dengan biaya yang muncul bila cadangan harus dipulihkan, misalnya biaya transfer data keluar.
2. Telusuri setiap lapisan aturan pada Gambar 2.6 dan tulis pertanyaan regulasi yang muncul.
3. Tulis risiko di luar regulasi: kedaulatan data, waktu pemulihan, ketergantungan pada penyedia, dan reputasi setelah insiden 2025.
4. Untuk setiap pertanyaan dan risiko, tentukan dasar rujukannya, status jawabannya, dan pihak yang harus menjawab.
5. Tulis rekomendasi sementara dalam satu paragraf.

Tabel 2.13 berisi contoh daftar pertanyaan dan risiko, sedangkan Tabel 2.14 adalah lembar isiannya.

Pertanyaan analisis:

1. Berapa banyak pertanyaan berstatus "belum terjawab" pada tabel isian? Apakah penghematan Rp108 juta per tahun sebanding dengan biaya untuk menjawab seluruhnya?
2. Pasal 20 Permenkes 24/2022 mewajibkan cadangan di tempat berbeda. Apakah AZ lain di *region* Jakarta, atau *region* lain di Indonesia, sudah memenuhi syarat itu? Jelaskan dengan membaca Tabel 2.7.
3. Pertanyaan mana yang jawabannya dapat membalik rekomendasi sementara?

Latihan 3: Hybrid Cloud dan Multi-Cloud

Latihan ini membuktikan bahwa *hybrid cloud* dan *multi-cloud* menyelesaikan masalah yang berbeda, sehingga perbandingannya harus memakai kriteria yang sama.

Tabel 2.13: Latihan 2: contoh daftar pertanyaan sebelum memindahkan cadangan ke luar negeri

Pertanyaan atau risiko	Dasar rujukan	Status	Penjawab
Apakah cadangan di luar negeri sejalan dengan pusat data di dalam wilayah Indonesia?	PP 28/2024 Pasal 966 ayat (2) dan (5); Pasal 972 bila dibaca sebagai transfer	Kendala utama, perlu tafsir	Tim hukum
Apakah pengawasan dan akses aparat Indonesia tetap terjamin?	PP 71/2019 Pasal 21 ayat 2 dan 3	Belum terjawab	Tim hukum
Apakah layanan penyimpanan di *region* itu termasuk *cloud* tersertifikasi?	Permenkes 24/2022 Pasal 20	Belum terjawab	Kementerian Kesehatan, penyedia
Apakah negara tujuan memiliki pelindungan data setara?	UU PDP Pasal 56	Perlu kajian	Tim hukum
Berapa lama memulihkan 15 terabyte dari luar negeri?	Rencana pemulihan, Bab 12	Perlu uji	Tim TI
Bagaimana persepsi pasien bila diketahui data dikirim ke luar negeri?	Insiden 2025	Risiko tinggi	Direksi

Tabel 2.14: Latihan 2: lembar isian pertanyaan dan risiko

Pertanyaan atau risiko	Dasar rujukan	Status	Penjawab
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…

Misalkan tim platform Medikara berisi 6 orang dan baru berpengalaman pada satu penyedia publik.

1. Tetapkan satu skenario *hybrid* dan satu skenario *multi-cloud* untuk organisasi pilihan tim atau untuk Medikara.
2. Bandingkan keduanya dari sisi biaya, keterampilan tim, dan ketergantungan pada penyedia, ditambah satu kriteria pilihan tim.
3. Beri nilai 1 sampai 5 untuk setiap kriteria, dengan 5 berarti paling menguntungkan bagi organisasi, lalu tulis alasannya.
4. Jumlahkan nilainya, lalu periksa apakah hasilnya berubah bila bobot kriteria keterampilan dilipatduakan.

Tabel 2.15 berisi contoh perbandingan untuk Medikara Health, sedangkan Tabel

2.16 adalah lembar isiannya.

Tabel 2.15: Latihan 3: contoh perbandingan *hybrid cloud* dan *multi-cloud* untuk Medikara Health

Kriteria	***Hybrid*: SIMRS dekat rumah sakit, layanan pendukung di *region* Jakarta**	***Multi-cloud*: Medikara+ di penyedia A, analitik di penyedia B**
Biaya	3: dua struktur biaya, tetapi ruang server lama tetap terpakai	2: biaya transfer data antarpenyedia setiap hari
Keterampilan tim	3: satu penyedia publik ditambah keahlian yang sudah ada	1: 6 orang harus menguasai dua penyedia sekaligus
Ketergantungan pada penyedia	3: bagian inti tetap di tangan sendiri	4: tidak bergantung pada satu penyedia
Kendali lokasi data	4: rekam medis tetap di sisi privat	3: dua kontrak lokasi data harus diperiksa
Jumlah	13	10

Tabel 2.16: Latihan 3: lembar isian perbandingan *hybrid cloud* dan *multi-cloud*

Kriteria	***Hybrid*: ...**	***Multi-cloud*: ...**
Biaya	...	...
Keterampilan tim	...	...
Ketergantungan pada penyedia	...	...
...	...	...
Jumlah	...	...

Pertanyaan analisis:

1. Kriteria mana yang paling menentukan selisih nilai pada tabel isian? Apakah kesimpulannya tetap sama bila bobot keterampilan dilipatduakan?
2. Data Flexera 2026 menyebut 88% organisasi memakai lebih dari satu penyedia. Apakah angka itu menjadi alasan yang cukup bagi Medikara untuk memilih *multi-cloud*? Jelaskan dengan membaca hasil tabel isian.

Bab 3

Ekonomi Cloud, FinOps, dan Keberlanjutan

Tujuan Pembelajaran

- Membandingkan struktur biaya pusat data sendiri dan *cloud* dengan pendekatan TCO.
- Menjelaskan prinsip, fase, dan peran dalam FinOps Framework 2026.
- Merumuskan metrik biaya per unit dan metrik jejak karbon untuk satu layanan.

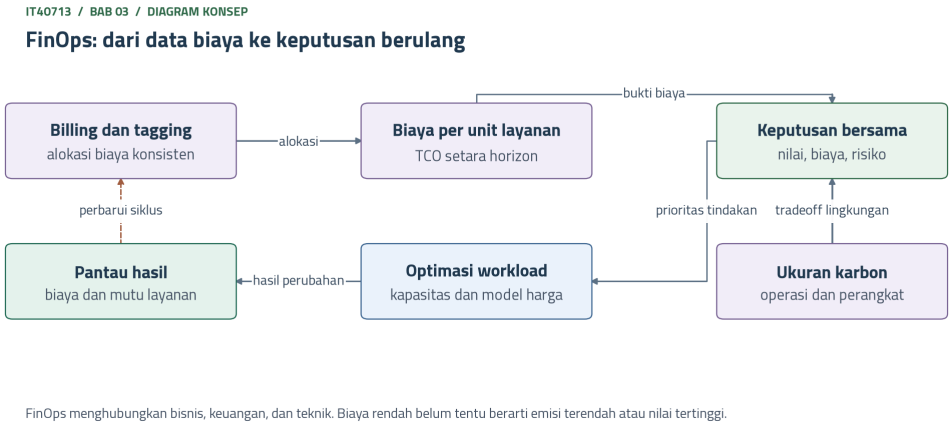
3.1 Pendahuluan

Selama ini Direktorat Keuangan Medikara Health mengenal belanja server sebagai anggaran lima tahunan. Setiap rumah sakit mengajukan peremajaan server, direksi menyetujuinya, lalu nilai pembelian itu disusutkan dalam laporan keuangan selama lima tahun. Polanya mudah diramalkan: angka besar di awal, lalu hampir tidak ada kejutan sampai peremajaan berikutnya. Strategi "Satu Pasien, Satu Rekam Medis" mengubah pola itu karena sebagian sistem akan berpindah ke *cloud*. Tagihan datang setiap bulan, besarnya mengikuti pemakaian, dan angkanya dapat berubah tanpa ada keputusan direksi yang baru. Bagian keuangan khawatir tagihan itu tidak terkendali. Tim teknologi informasi (TI) khawatir dianggap boros tanpa ada data yang bisa dipakai untuk membela diri.

Kekhawatiran itu beralasan. Survei Flexera tahun 2026 mencatat bahwa responden memperkirakan 29 persen belanja Infrastructure as a Service (IaaS) dan Platform as a Service (PaaS) mereka terbuang [55]. Jalan keluarnya bukan

menolak *cloud*, melainkan cara baru mengelola biaya teknologi: keuangan, tim teknik, pemilik layanan, dan pimpinan membaca data yang sama, lalu mengambil keputusan bersama, seperti diperlihatkan Gambar 3.1. Bab ini menyusun cara itu dalam lima langkah: struktur biaya dan Total Cost of Ownership (TCO), model harga *cloud*, FinOps, alokasi biaya dan biaya per unit, serta jejak karbon.

Gambar 3.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 3.1: FinOps: dari data biaya ke keputusan berulang

Rujukan utama bab ini adalah buku *Cloud FinOps* karya Storment dan Fuller [135], FinOps Framework 2026 dari FinOps Foundation [53], spesifikasi FinOps Open Cost and Usage Specification (FOCUS) versi 1.4 [48], serta ISO/IEC 21031:2024 tentang Software Carbon Intensity (SCI) [77]. Hasil bab ini adalah model biaya awal dan sasaran nilai *cloud* untuk tonggak capstone C1.

3.2 CapEx, OpEx, dan Total Cost of Ownership

Capital Expenditure (CapEx) adalah belanja untuk memperoleh aset yang manfaatnya lebih dari satu tahun, misalnya server, perangkat penyimpanan, dan perlengkapan ruang server. Nilainya dicatat sebagai aset di neraca, lalu dibebankan sedikit demi sedikit sebagai penyusutan selama umur manfaatnya. Operational Expenditure (OpEx) adalah belanja untuk menjalankan kegiatan pada periode berjalan, misalnya listrik, gaji, sewa, dan tagihan *cloud*. Nilainya langsung menjadi beban di laporan laba rugi pada bulan terjadinya. *Cloud* memindahkan sebagian besar belanja infrastruktur dari CapEx ke OpEx: organisasi tidak lagi membeli server, tetapi membayar pemakaian.

Analoginya seperti memilih antara membeli mobil operasional dan memakai taksi daring. Mobil yang dibeli menghabiskan uang besar sekali di awal, dicatat sebagai aset, dan tetap disusutkan walaupun lebih sering diparkir. Taksi dibayar per perjalanan, tanpa aset, tanpa biaya parkir dan servis. Namun bila taksi dipakai sepanjang hari setiap hari, total ongkosnya bisa melampaui cicilan mobil. Pilihan yang tepat ditentukan oleh pola pemakaian, bukan oleh mana yang terdengar lebih modern.

Tabel 3.1 merangkum perbedaan keduanya dari sisi yang biasa ditanyakan bagian keuangan.

Tabel 3.1: Perbandingan CapEx pada pusat data sendiri dan OpEx pada *cloud*

Aspek	**CapEx (pusat data sendiri)**	**OpEx (*cloud*)**
Contoh belanja	Server, penyimpanan, perangkat jaringan, ruang server	Tagihan komputasi, penyimpanan, dan transfer data per bulan
Pencatatan	Aset di neraca, disusutkan selama umur manfaat	Beban di laba rugi pada bulan pemakaian
Pola arus kas	Keluar besar di awal, lalu kecil dan stabil	Kecil di awal, naik turun mengikuti pemakaian
Saat keputusan	Sekali, dalam siklus anggaran modal	Terus-menerus, setiap kali sumber daya dibuat atau diubah
Risiko utama	Kapasitas menganggur karena dibeli untuk beban puncak	Tagihan membengkak karena sumber daya lupa dimatikan

Perpindahan dari CapEx ke OpEx punya dua konsekuensi. Pertama, arus kas menjadi lebih ringan di awal, sehingga layanan baru dapat dimulai tanpa menunggu siklus anggaran modal tahun berikutnya. Kedua, biaya menjadi variabel. Anggaran tahunan yang dikunci di awal tahun tidak lagi cukup sebagai alat kendali, karena keputusan yang menambah biaya kini diambil setiap hari oleh tim teknik, bukan sekali setahun oleh direksi.

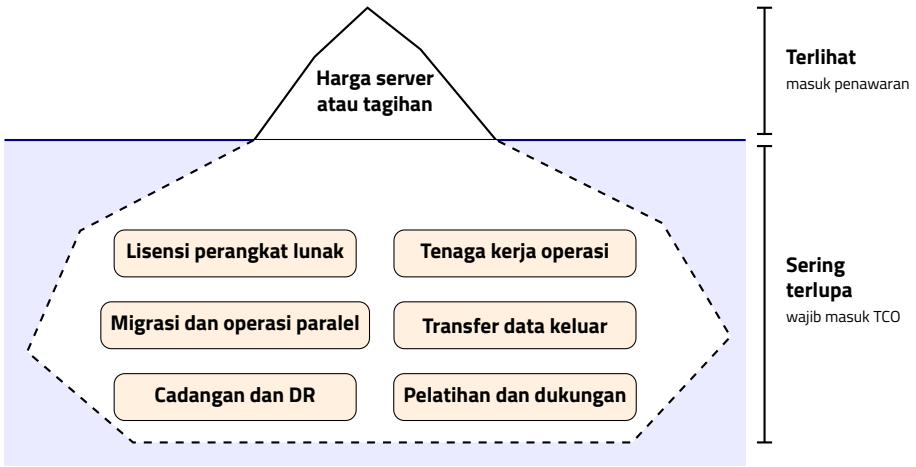
Dua contoh dari Medikara Health:

1. Peremajaan server SIMRS (Sistem Informasi Manajemen Rumah Sakit) di rumah sakit Bekasi bernilai Rp2,4 miliar dan disusutkan selama lima tahun, sehingga laporan laba rugi menanggung Rp480 juta per tahun. Server itu dibeli untuk beban puncak pagi hari, padahal pemanfaatan prosesornya rata-rata hanya sekitar 25 persen.
2. Medikara+ dijalankan vendor di *cloud*. Tagihannya Rp57 juta pada bulan biasa dan naik menjadi Rp79,3 juta pada bulan kampanye vaksinasi. Tidak ada aset yang dibeli, tetapi bagian keuangan harus menjelaskan selisih Rp22,3 juta kepada direksi.

3.2.1 Komponen TCO dan Biaya yang Sering Terlupa

TCO adalah seluruh biaya untuk memiliki dan menjalankan sebuah sistem selama masa tertentu, biasanya tiga sampai lima tahun. TCO tidak berhenti pada harga beli server atau tagihan bulanan *cloud*. Di dalamnya ada lisensi, tenaga kerja, fasilitas, cadangan data, migrasi, dan pelatihan. Perbandingan yang hanya memakai harga server melawan tagihan *cloud* hampir selalu menyesatkan, karena dua angka itu mewakili lingkup biaya yang berbeda.

Gambaran yang mudah diingat adalah gunung es pada Gambar 3.2. Harga server atau tagihan bulanan hanya puncak yang terlihat di atas air. Bagian terbesarnya ada di bawah permukaan, dan bagian itulah yang paling sering hilang dari perhitungan.



Gambar 3.2: TCO sebagai gunung es: harga yang terlihat di penawaran hanya sebagian kecil dari biaya selama masa pakai

Salah satu biaya di bawah permukaan perlu dijelaskan khusus. Biaya transfer data keluar (*egress*) adalah biaya yang dikenakan ketika data keluar dari jaringan penyedia, misalnya dikirim ke pengguna internet, ke pusat data sendiri, atau ke penyedia lain. Data yang masuk umumnya tidak dikenai biaya, sehingga biaya ini baru terasa setelah sistem berjalan, misalnya ketika aplikasi pasien mengirim berkas hasil pemeriksaan berukuran besar. Tabel 3.2 mendaftar komponen TCO beserta alasan masing-masing sering terlupa.

3.2.2 Menyusun Perbandingan TCO yang Jujur

Perbandingan TCO yang jujur memenuhi lima syarat. Pertama, masanya sama, misalnya lima tahun untuk kedua sisi. Kedua, lingkupnya sama: bila sisi *cloud* sudah memasukkan DR, sisi pusat data sendiri juga harus memasukkannya. Ke-

Tabel 3.2: Komponen TCO pada pusat data sendiri dan *cloud*

Komponen	**Pusat data sendiri**	***Cloud***	**Mengapa sering terlupa**
Perangkat keras	Server, penyimpanan, jaringan	Tidak dibeli, masuk harga sumber daya	Jarang terlupa
Fasilitas	Ruang, listrik, pendingin, keamanan fisik	Sudah termasuk harga	Dibayar bagian umum, bukan anggaran TI
Lisensi	Sistem operasi, basis data, aplikasi	Tetap ada, dibawa sendiri atau disewa per jam	Aturan lisensi per inti prosesor bisa berubah saat pindah
Tenaga kerja	Operasi, *patch*, cadangan	Berkurang tetapi tidak nol, butuh keterampilan baru	Gaji tercatat di anggaran sumber daya manusia
Migrasi	Tidak ada	Proyek pindah, pengujian, dua sistem berjalan paralel	Dianggap biaya proyek terpisah
Transfer data keluar	Kecil, di dalam jaringan sendiri	Dibayar per gigabyte yang keluar	Tidak terlihat bila tidak diisi di kalkulator
Cadangan dan Disaster Recovery (DR)	Pita cadangan, lokasi kedua	Penyimpanan dan salinan di lokasi lain	Lingkupnya sering tidak setara di kedua sisi
Keluar dari penyedia	Tidak ada	Biaya memindahkan data bila kelak berganti penyedia	Baru dipikirkan ketika kontrak hampir habis

tiga, tingkat layanannya setara, termasuk ketersediaan dan frekuensi cadangan. Keempat, tenaga kerja ikut dihitung, termasuk pelatihan. Kelima, setiap asumsi ditulis dan diuji kepekaannya, misalnya apa yang terjadi bila pemakaian 30 persen lebih tinggi dari perkiraan.

Dua hal pada TCO sering tertukar. Pembagiannya begini. TCO menjawab pertanyaan "berapa biaya seluruhnya". *Business case* menjawab pertanyaan "apakah manfaatnya sepadan dengan biaya itu", sehingga memasukkan antrean yang lebih pendek, rilis fitur yang lebih cepat, dan risiko yang berkurang. TCO adalah salah satu bahan *business case*, bukan penggantinya. Penyusunan *business case* dan portofolio inisiatif dibahas rinci di IT40513.

Kasus Medikara Health: TCO Kasar SIMRS Rumah Sakit Bekasi

Misalkan tim transformasi menghitung TCO kasar SIMRS rumah sakit Bekasi untuk lima tahun. Tiga opsi dibandingkan. Opsi pertama tetap di pusat data sendiri dengan peremajaan server. Opsi A memindahkan SIMRS apa adanya (*rehost*) dan membayar harga *on-demand*. Opsi B juga *rehost*, tetapi ukuran sumber daya lebih dahulu disesuaikan dengan kebutuhan nyata

(*rightsizing*), lalu beban dasarnya ditutup dengan komitmen pemakaian. Hasilnya ada di Tabel 3.3.
Hasil ini punya dua konsekuensi. Pertama, opsi A justru Rp110 juta lebih mahal daripada bertahan. Server yang di pusat data sendiri hanya terpakai 25 persen tetap dibayar penuh per jam di *cloud*, ditambah biaya migrasi dan transfer data. Kedua, opsi B lebih murah Rp1,15 miliar atau sekitar 19,5 persen. Penghematan itu datang dari *rightsizing* dan komitmen, bukan dari perpindahan ke *cloud* itu sendiri. Dengan sekitar 210.000 kunjungan pasien per tahun di rumah sakit Bekasi, biaya infrastruktur SIMRS per kunjungan turun dari Rp5.619 menjadi Rp4.524. Angka ini menjadi bahan metrik biaya per unit pada seksi 3.5.

Tabel 3.3: TCO kasar SIMRS rumah sakit Bekasi selama lima tahun, dalam juta rupiah (angka rekaan)

Komponen	Pusat data sendiri	Opsi A	Opsi B
Perangkat keras server dan penyimpanan	2.400	0	0
Ruang server, listrik, pendingin	450	0	0
Sumber daya *cloud* (komputasi dan penyimpanan)	0	3.300	2.040
Lisensi sistem operasi dan basis data	750	750	750
Tenaga kerja operasi infrastruktur	1.800	900	900
Cadangan dan DR	300	200	200
Koneksi jaringan dan transfer data keluar	200	350	350
Migrasi dan operasi paralel	0	450	450
Pelatihan	0	60	60
Total lima tahun	**5.900**	**6.010**	**4.750**
Biaya per kunjungan pasien (rupiah)	5.619	5.724	4.524

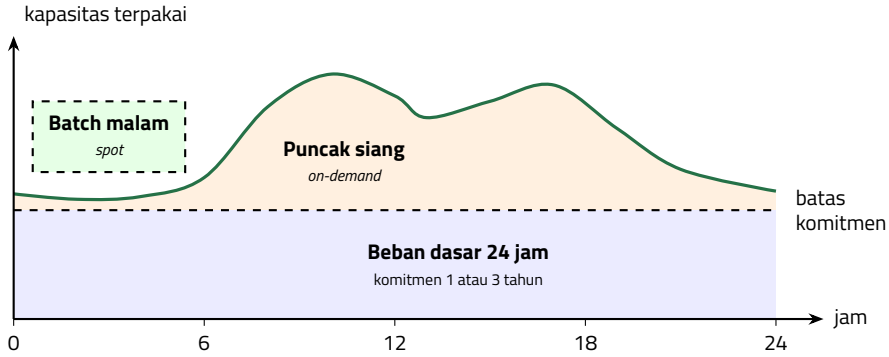
3.3 Model Harga Cloud

Penyedia *cloud* menjual kapasitas yang sama dengan tiga cara bayar. Model *on-demand* dibayar per detik atau per jam pemakaian tanpa komitmen apa pun. Model ini paling fleksibel dan paling mahal per unit. Model komitmen pemakaian meminta organisasi berjanji memakai sejumlah sumber daya, atau membelanjakan sejumlah uang per jam, selama satu atau tiga tahun. Sebagai imbalannya, harga per unit turun. Amazon Web Services (AWS) menyebut potongan Savings Plans dapat mencapai 72 persen dibanding *on-demand* [15], dan penyedia lain menawarkan model serupa dengan nama berbeda. Model ketiga adalah kapasitas *spot*, yaitu kapasitas yang sedang menganggur di pusat data penyedia. Harganya dapat mencapai 90 persen di bawah *on-demand* [16], tetapi penyedia boleh menariknya kembali sewaktu-waktu dengan pemberitahuan dua menit [17]. Angka 72 dan 90 persen adalah batas atas, bukan potongan yang pasti diperoleh setiap organisasi.

Analoginya seperti parkir kendaraan kantor. Parkir harian dibayar per jam:

mahal bila dipakai seharian, tetapi tidak ada kewajiban apa pun. Langganan parkir bulanan jauh lebih murah per hari, tetapi tetap dibayar walaupun kendaraannya tidak datang. Parkir di lahan kosong milik tetangga paling murah, tetapi kendaraan harus segera dipindah begitu pemiliknya membutuhkan lahan itu. Kantor yang cermat memakai ketiganya sekaligus, sesuai sifat kendaraan masing-masing.

Kuncinya adalah membaca pola beban. Gambar 3.3 memperlihatkan kebutuhan kapasitas satu hari. Lantai beban yang selalu ada selama 24 jam cocok ditutup komitmen. Puncak siang yang naik turun lebih aman dibayar *on-demand*. Pekerjaan malam yang boleh terhenti dan diulang cocok memakai *spot*.



Gambar 3.3: Pola beban satu hari menentukan bagian yang ditutup komitmen, *on-demand*, dan *spot*

Tiga contoh dari Medikara Health:

1. Basis data SIMRS dan rekam medis elektronik berjalan 24 jam dengan beban dasar yang stabil. Beban dasar ini calon utama komitmen tiga tahun, setelah ukurannya disesuaikan.
2. Medikara+ melonjak sampai sekitar 60 persen setiap kali program vaksinasi diumumkan, lalu kembali normal setelah kampanye selesai. Lonjakan seperti ini dibayar *on-demand* dan ditangani penskalaan otomatis.
3. Laboratorium rujukan menjalankan pengolahan data hasil pemeriksaan setiap malam selama sekitar empat jam. Pekerjaan itu boleh diulang bila terhenti, sehingga cocok memakai *spot*.

Tabel 3.4 merangkum ketiga model beserta risikonya.

3.3.1 Risiko Komitmen Berlebih

Komitmen dibayar penuh, terpakai atau tidak. Karena itu ada titik impas yang mudah dihitung. Bila potongan komitmen dibanding *on-demand* adalah d, komitmen

Tabel 3.4: Tiga model harga *cloud* dan pola beban yang cocok

Model	Cara bayar	Cocok untuk	Risiko utama
On-demand	Per detik atau per jam, tanpa janji	Beban baru, puncak sementara, uji coba	Paling mahal bila dibiarkan menyala terus
Komitmen pemakaian	Janji satu atau tiga tahun, harga per unit lebih rendah	Beban dasar yang stabil dan sudah dioptimalkan	Membayar kapasitas yang tidak terpakai bila komitmen terlalu besar
Spot	Harga kapasitas menganggur, bisa ditarik kembali	Pekerjaan *batch* yang boleh terhenti dan diulang	Pekerjaan terputus, tidak cocok untuk layanan pasien langsung

baru menguntungkan ketika rata-rata pemanfaatannya melewati $1 - d$:

$$\text{pemanfaatan impas} = 1 - \text{potongan komitmen}.$$

Dengan potongan 40 persen, titik impasnya 60 persen. Komitmen yang rata-rata hanya terpakai 50 persen membuat organisasi membayar lebih mahal daripada tidak berkomitmen sama sekali.

Misalkan Medikara Health membeli komitmen setara 100 unit kapasitas per jam. Harga *on-demand* Rp1.000 per unit-jam dan harga komitmen Rp600 per unit-jam (angka rekaan). Sebulan berisi sekitar 730 jam, sehingga biaya komitmen tetap Rp43,8 juta per bulan. Bila pemakaian rata-rata 70 unit, biaya *on-demand* untuk pemakaian yang sama adalah Rp51,1 juta, sehingga komitmen menghemat Rp7,3 juta. Bila pemakaian rata-rata hanya 50 unit, biaya *on-demand* hanya Rp36,5 juta, sehingga komitmen justru merugikan Rp7,3 juta.

Kesalahan yang paling sering terjadi adalah membeli komitmen sebelum *rightsizing*. Ukuran yang terlalu besar ikut dikunci selama satu atau tiga tahun. Urutan yang aman adalah merapikan ukuran lebih dahulu, berkomitmen hanya pada lantai beban, menambah komitmen secara bertahap, lalu memantau pemanfaatannya setiap bulan.

Pendalaman Teknis (opsional): Kalkulator Harga untuk Tiga Model

Kalkulator harga resmi penyedia dipakai untuk membandingkan ketiga model pada satu *workload*, misalnya *server* aplikasi Medikara+. Langkahnya sebagai berikut.

1. Pilih region Jakarta agar sesuai dengan kendala lokasi data dari Bab 2.
2. Masukkan ukuran mesin virtual hasil *rightsizing*, bukan ukuran server lama.
3. Hitung tiga varian: *on-demand* 730 jam per bulan, komitmen satu tahun tanpa bayar di muka, dan komitmen tiga tahun.

4. Untuk *spot*, catat harga rata-rata dan riwayat interupsi untuk jenis mesin tersebut, lalu nilai apakah pekerjaannya tahan terhadap interupsi.
5. Tambahkan penyimpanan, cadangan, dan transfer data keluar, karena ketiganya sering tertinggal di kalkulator.

Simpan tautan atau ekspor hasil kalkulator sebagai bukti asumsi di `sources/chapter03/templat-tco-sederhana.xlsx`. Kalkulator hanya dipakai untuk memperkirakan biaya; tidak ada sumber daya berbayar yang perlu dijalankan.

3.4 FinOps Framework 2026

FinOps Foundation mendefinisikan FinOps sebagai kerangka operasional dan praktik budaya yang memaksimalkan nilai bisnis teknologi. Caranya dengan mempercepat keputusan berbasis data dan menumbuhkan akuntabilitas keuangan lewat kolaborasi antara tim teknik, keuangan, dan bisnis [53]. Kata kuncinya adalah nilai, bukan penghematan. FinOps tidak bertujuan membuat tagihan sekecil mungkin. Tagihan yang naik karena layanan pasien bertambah adalah kabar baik. Yang dicegah adalah biaya yang tidak menghasilkan nilai dan biaya yang tidak diketahui pemiliknya [135].

Analoginya seperti keluarga besar yang tinggal di beberapa rumah tetapi berbagi satu meteran listrik. Selama tagihan hanya diterima bendahara keluarga, tidak ada yang merasa memiliki pemakaiannya, dan tagihan terus naik. Begitu setiap rumah punya meteran sendiri, masing-masing melihat pemakaiannya, bendahara melihat totalnya, dan rapat keluarga bulanan memutuskan siapa menghemat apa. Listriknya sama, cara mengelolanya yang berubah.

Dua contoh dari Medikara Health:

1. Tanpa FinOps, tagihan *cloud* Juli diterima bagian keuangan sekitar 15 hari setelah bulan berakhir. Tagihan itu naik 17 persen, dan tidak ada yang bisa menjelaskan penyebabnya karena sumber daya tidak diberi label pemilik.
2. Dengan FinOps, setiap tim melihat biaya hariannya sendiri. Lonjakan biaya lingkungan pengembangan Medikara+ terdeteksi pada hari ketiga, bukan pada akhir bulan, sehingga pemborosannya berhenti pada beberapa juta rupiah.

3.4.1 Enam Prinsip

Kerangka ini bertumpu pada enam prinsip yang menjadi pegangan saat ada perbedaan pendapat [53]. Tabel 3.5 memuat keenamnya dengan nama aslinya, maknanya, dan contoh penerapannya.

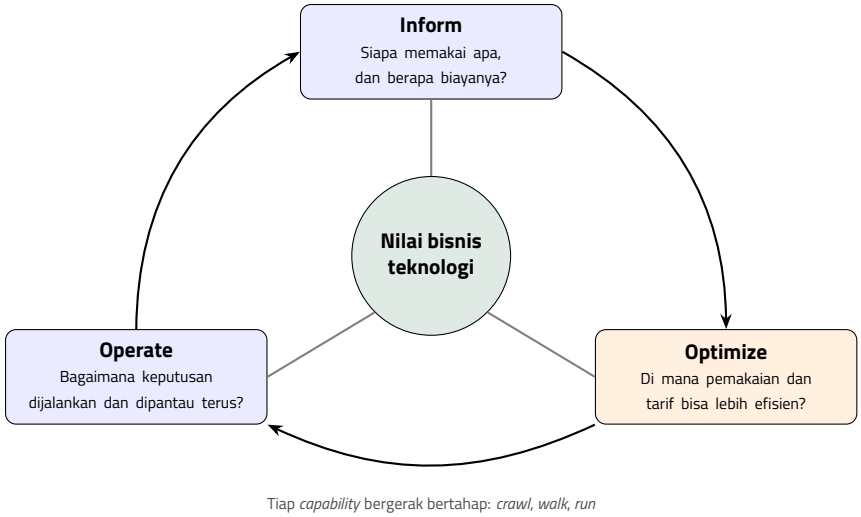
Tabel 3.5: Enam prinsip FinOps dan contoh penerapannya di Medikara Health

Prinsip	Makna	Contoh di Medikara
Teams need to collaborate	Keuangan, teknik, dan bisnis bekerja dengan data yang sama	Rapat biaya bulanan dihadiri keuangan, tim infrastruktur, dan pemilik Medikara+
Business value drives technology decisions	Biaya dinilai terhadap nilai yang dihasilkan	Biaya naik saat janji temu naik dianggap wajar
Everyone takes ownership for their technology usage	Setiap tim bertanggung jawab atas biaya sumber daya yang dibuatnya	Tim pengembang mematikan lingkungan uji di luar jam kerja
FinOps data should be accessible, timely, and accurate	Data biaya tersedia cepat dan dapat dipercaya	*Dashboard* biaya harian, bukan laporan bulanan yang terlambat
FinOps should be enabled centrally	Ada fungsi pusat yang menyediakan data, aturan, dan pembelian komitmen	Analis FinOps di direktorat TI mengelola komitmen untuk seluruh grup
Take advantage of the variable cost model of the cloud	Biaya yang bisa naik turun dimanfaatkan, bukan dilawan	Kapasitas ditambah saat kampanye vaksinasi, lalu dikurangi lagi

3.4.2 Fase Inform, Optimize, dan Operate

Praktik FinOps berjalan dalam tiga fase yang berulang, seperti diperlihatkan Gambar 3.4. Fase Inform membuat biaya terlihat dan dapat dialokasikan: siapa memakai apa, dan berapa biayanya. Fase Optimize mencari peluang efisiensi, baik dari sisi pemakaian (mematikan sumber daya menganggur, *rightsizing*) maupun dari sisi tarif (komitmen dan potongan). Fase Operate menjalankan keputusan itu sebagai kebiasaan: kebijakan, otomasi, anggaran, dan pemantauan terus-menerus. Organisasi tidak menyelesaikan satu fase lalu berhenti. Setiap putaran memperbaiki data, membuka peluang baru, dan memperketat kebiasaan.

Kematangan setiap *capability* dinilai bertahap dengan tiga tingkat: *crawl*, *walk*, dan *run* [53]. Tingkat *crawl* berarti praktiknya baru ada dan masih manual. Tingkat *run* berarti praktiknya sudah otomatis dan menjadi kebiasaan. Tingkat *run* tidak wajib di semua *capability*; kematangan cukup sepadan dengan besar belanja dan risikonya.



Gambar 3.4: Tiga fase FinOps yang berulang dan bermuara pada nilai bisnis teknologi

3.4.3 Persona, Domain, dan Capability

FinOps Framework memakai istilah *persona* untuk peran yang terlibat. Persona inti berjumlah enam: FinOps Practitioner, Engineering, Finance, Product, Procurement, dan Leadership. Persona pendukung datang dari disiplin yang bersinggungan, yaitu pengelolaan aset TI, pengelolaan keuangan TI, pengelolaan layanan TI, keamanan, dan keberlanjutan [53]. Persona bukan jabatan. Satu orang di organisasi kecil bisa memegang dua persona, dan satu persona di organisasi besar bisa diisi satu tim.

Pekerjaan FinOps dikelompokkan ke dalam empat domain. Setiap domain berisi beberapa *capability*, yaitu kemampuan yang perlu dimiliki organisasi. Tabel 3.6 memuat keempat domain dan *capability*-nya sesuai halaman resmi kerangka tersebut. Pembaruan 2026 menambahkan *capability* Executive Strategy Alignment pada domain Manage the FinOps Practice [53]. Sesuai namanya, *capability* ini menekankan keselarasan praktik FinOps dengan strategi dan prioritas pimpinan, sehingga keputusan biaya teknologi diukur terhadap sasaran organisasi, bukan hanya terhadap anggaran TI.

3.4.4 Scope dan Kategori Teknologi

FinOps tidak lagi terbatas pada *public cloud*. Halaman kerangka 2026 menyebut kategori teknologi yang dapat dikelola dengan cara yang sama: Artificial Intelligence (AI), *public cloud*, Software as a Service (SaaS), *data platform*, *private cloud*, lisensi, dan pusat data [53]. Kerangka ini juga memakai istilah *scope*, yaitu

Tabel 3.6: Empat domain FinOps Framework 2026 beserta *capability*-nya

Domain	***Capability***	**Pertanyaan pokok**
Understand Usage & Cost	Data Ingestion; Allocation; Reporting & Analytics; Anomaly Management	Berapa biayanya dan milik siapa?
Quantify Business Value	Planning & Estimating; Forecasting; Budgeting; KPIs & Benchmarking; Unit Economics	Apakah biayanya sepadan dengan nilainya?
Optimize Usage & Cost	Architecting & Workload Placement; Usage Optimization; Rate Optimization; Licensing & SaaS; Sustainability	Di mana bisa lebih efisien?
Manage the FinOps Practice	Executive Strategy Alignment; FinOps Practice Operations; Governance, Policy & Risk; FinOps Education & Enablement; Invoicing & Chargeback; FinOps Assessment; Automation, Tools & Services; Intersecting Disciplines	Bagaimana praktik ini dijalankan dan diatur?

batas penerapan praktik FinOps yang dipilih organisasi, misalnya per produk, per pusat biaya, atau per lingkungan. Pembaruan 2026 merapikan pemisahan antara kategori teknologi dan *scope* tersebut.

Pertanyaan kemudian muncul: mengapa organisasi yang baru mulai perlu memikirkan pusat data dan SaaS? Jawabannya, perbandingan TCO hanya adil bila biaya pusat data sendiri dibaca dengan bahasa yang sama dengan biaya *cloud*. Bila hanya tagihan *cloud* yang dikelola dengan disiplin FinOps, SIMRS di pusat data sendiri akan selalu tampak lebih murah.

Kasus Medikara Health: Siapa Mengisi Persona Inti

Medikara Health belum punya fungsi FinOps. Misalkan direksi membentuk komite biaya teknologi yang bertemu setiap bulan, dan memetakan persona inti ke jabatan yang sudah ada seperti pada Tabel 3.7. Persona FinOps Practitioner diisi satu analis baru di direktorat TI, karena tidak ada jabatan lama yang tugasnya menjembatani data teknik dan data keuangan.

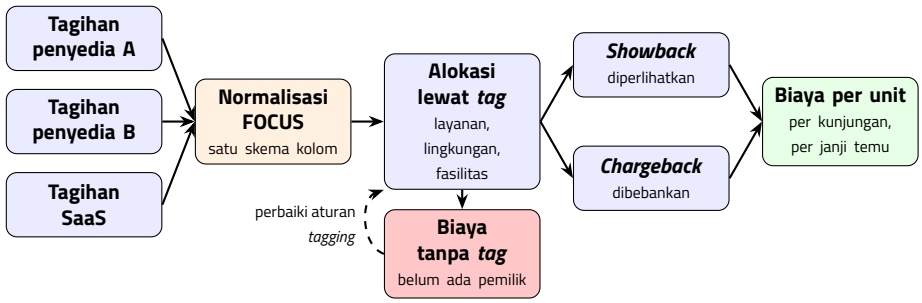
Pemetaan ini punya dua konsekuensi. Pertama, bagian pengadaan ikut sejak awal, karena komitmen pemakaian tiga tahun adalah kontrak yang harus melewati prosedur pengadaan. Kedua, tim keamanan informasi masuk sebagai persona pendukung. Pelajaran dari insiden kebocoran data lewat vendor pada 2025 menunjukkan bahwa keputusan memangkas biaya, misalnya menghapus salinan log, tidak boleh mengorbankan jejak audit.

Tabel 3.7: Pemetaan persona inti FinOps ke jabatan di Medikara Health

Persona	Pengisi di Medikara	Pertanyaan khasnya
FinOps Practitioner	Analis FinOps baru di direktorat TI	Data biaya mana yang belum punya pemilik?
Engineering	Tim infrastruktur pusat dan tim vendor Medikara+	Ukuran sumber daya mana yang berlebihan?
Finance	Bagian akuntansi manajemen dan anggaran	Berapa perkiraan tagihan triwulan depan?
Product	Pemilik produk Medikara+ dan kepala instalasi SIMRS tiap rumah sakit	Berapa biaya per janji temu atau per kunjungan?
Procurement	Bagian pengadaan	Komitmen mana yang aman dikontrakkan?
Leadership	Direktur keuangan, direktur TI, dan direktur utama	Apakah belanja teknologi mendukung strategi 2026 sampai 2030?

3.5 Alokasi Biaya dan Unit Economics

Biaya yang tidak punya pemilik tidak akan pernah dikelola. Seksi ini membahas cara membagi tagihan kepada pemiliknya, cara menyatukan tagihan dari banyak penyedia, dan cara menerjemahkan biaya menjadi angka yang bermakna bagi direksi. Gambar 3.5 memperlihatkan alurnya dari tagihan mentah sampai biaya per unit.



Gambar 3.5: Alur alokasi biaya: tagihan dinormalisasi, dialokasikan lewat *tag*, diperlihatkan atau dibebankan, lalu dibaca sebagai biaya per unit

3.5.1 Tagging sebagai Dasar Alokasi

Tag adalah label berbentuk pasangan kunci dan nilai yang ditempelkan pada setiap sumber daya *cloud*, misalnya `layanan=medikara-plus` atau `lingkungan=produksi`. Label itu ikut tercatat di data tagihan, sehingga biaya dapat dikelompokkan menurut layanan, lingkungan, fasilitas, atau pusat biaya. Aturan *tagging* menetapkan kunci yang wajib, nilai yang diizinkan, dan siapa yang

memeriksa kepatuhannya. Sumber daya tanpa *tag* menghasilkan biaya tanpa pemilik.

Analoginya seperti titipan barang di kantor ekspedisi. Paket yang diberi label nama dan alamat sampai ke penerimanya. Paket tanpa label menumpuk di gudang, memakan tempat, dan tidak ada yang merasa bertanggung jawab. Semakin lama dibiarkan, semakin sulit menelusuri siapa pengirimnya.

Dua contoh dari Medikara Health:

1. Medikara Health menetapkan lima kunci wajib: `layanan`, `lingkungan`, `fasilitas`, `pusat-biaya`, dan `pemilik`. Sasarannya, paling sedikit 95 persen biaya bulanan sudah memiliki *tag* dalam enam bulan.
2. Pada tagihan Agustus 2026, biaya tanpa *tag* sebesar Rp5 juta dari total Rp76,2 juta, atau sekitar 6,6 persen. Sebagian besar berasal dari *snapshot* disk lama tahun 2025 yang tidak pernah dihapus.

Sebagian biaya memang dipakai bersama, misalnya jaringan inti, layanan keamanan, dan platform pemantauan. Biaya bersama seperti ini dibagi dengan salah satu dari tiga cara: proporsional terhadap pemakaian langsung tiap layanan, rata ke setiap layanan, atau dengan persentase tetap yang disepakati [135]. Cara pembagian tidak harus sempurna. Yang penting, caranya disepakati, ditulis, dan dipakai konsisten setiap bulan.

3.5.2 Showback dan Chargeback

Dua hal pada alokasi sering tertukar. Pembagiannya begini. *Showback* memperlihatkan biaya kepada unit pemakai tanpa memindahkannya ke anggaran unit tersebut. *Chargeback* benar-benar membebankan biaya ke anggaran unit pemakai, sehingga muncul di laporan keuangan unit itu. Tabel 3.8 membandingkan keduanya. Penagihan internal semacam ini termasuk *capability* Invoicing & Chargeback dalam FinOps Framework [53].

Tabel 3.8: Perbandingan *showback* dan *chargeback*

Aspek	***Showback***	***Chargeback***
Yang terjadi	Biaya diperlihatkan lewat laporan atau *dashboard*	Biaya dipindahkan ke anggaran unit pemakai
Syarat data	Alokasi cukup akurat untuk dibaca	Alokasi akurat, rinci, dan disepakati, karena menyangkut anggaran
Dampak perilaku	Kesadaran biaya tumbuh tanpa konflik anggaran	Unit terdorong berhemat, tetapi bisa memicu perdebatan angka
Cocok untuk	Tahap awal, saat *tagging* belum rapi	Tahap matang, saat unit punya kendali atas pemakaiannya
Contoh Medikara	Laporan biaya bulanan per rumah sakit	Biaya Medikara+ dibebankan ke anggaran unit layanan digital

Medikara Health sebaiknya memulai dengan *showback*. Alasannya, *tag* belum rapi dan rumah sakit hasil akuisisi masih punya cara anggaran yang berbeda. *Chargeback* yang dipaksakan di atas data yang belum dipercaya hanya akan memindahkan rapat dari "cara berhemat" menjadi "angka siapa yang benar".

3.5.3 Spesifikasi FOCUS

Setiap penyedia *cloud* dan SaaS menerbitkan data tagihan dengan nama kolom, satuan, dan cara mencatat potongan yang berbeda. Organisasi yang memakai dua penyedia harus menerjemahkan keduanya sebelum dapat menjumlahkannya. FOCUS adalah spesifikasi terbuka untuk data tagihan yang menetapkan satu skema kolom bersama, sehingga tagihan dari banyak sumber dapat dibaca dengan cara yang sama [48]. Versi 1.4 terbit pada 2026, dan contoh dalam spesifikasinya mencakup SaaS serta pembelian lewat *marketplace*, bukan hanya *cloud* publik.

Tabel 3.9 memuat kolom FOCUS yang paling sering dipakai untuk alokasi. Empat kolom biaya perlu dibedakan dengan cermat, karena masing-masing menjawab pertanyaan yang berbeda.

Tabel 3.9: Kolom FOCUS yang sering dipakai untuk alokasi biaya

Kolom	Isi dan kegunaannya
`BillingPeriodStart`	Awal periode faktur, dipakai untuk mengelompokkan per bulan
`ChargeCategory`	Jenis baris: pemakaian (`Usage`), pembelian (`Purchase`), pajak, kredit, atau penyesuaian
`ServiceCategory`, `ServiceName`	Golongan layanan (komputasi, penyimpanan, basis data) dan nama layanannya
`RegionId`, `SubAccountId`	Lokasi sumber daya dan akun tempatnya berada; berguna untuk memeriksa kepatuhan lokasi data
`PricingCategory`	Model harga: `Standard`, `Dynamic` (misalnya *spot*), `Committed`, atau `Other`
`ListCost`	Biaya menurut harga katalog, sebelum potongan apa pun
`ContractedCost`	Biaya menurut harga yang sudah dinegosiasikan dalam kontrak
`EffectiveCost`	Biaya setelah seluruh potongan, dengan pembelian komitmen disebar ke pemakaian yang ditutupnya
`BilledCost`	Biaya yang benar-benar tertera di faktur periode itu
`Tags`	Pasangan kunci dan nilai *tag*, dasar alokasi

Perbedaan `BilledCost` dan `EffectiveCost` paling sering menimbulkan salah baca. Misalkan Medikara Health membayar komitmen Rp34 juta per bulan. Di faktur, angka itu muncul sebagai satu baris pembelian yang tidak punya *tag* layanan, sedangkan pemakaian yang ditutup komitmen tampil dengan `BilledCost` nol. Bila alokasi memakai `BilledCost`, Medikara+ tampak hemat dan seluruh Rp34 juta jatuh ke kelompok tanpa pemilik. Bila memakai `EffectiveCost`, biaya komitmen itu tersebar ke layanan yang benar-benar memakainya. Keuangan memakai `BilledCost` untuk mencocokkan faktur, sedangkan alokasi dan biaya

per unit memakai `EffectiveCost`.

3.5.4 Unit Economics

Unit economics adalah cara membaca biaya teknologi sebagai biaya per satuan nilai bisnis, misalnya biaya per kunjungan pasien atau per janji temu. Total tagihan menjawab "berapa yang dibelanjakan". Biaya per unit menjawab "seberapa efisien belanja itu menghasilkan layanan". Pertanyaan kedua inilah yang bermakna bagi direksi, karena layanan yang tumbuh wajar menaikkan total tagihan, tetapi tidak wajar bila menaikkan biaya per unitnya [135].

Analoginya seperti warung makan yang menghitung biaya bahan per porsi. Pemilik warung tidak panik ketika belanja pasar naik pada bulan Ramadan, karena porsi yang terjual juga naik. Yang membuatnya waspada adalah biaya bahan per porsi yang naik, karena itu tanda ada bahan terbuang atau harga pemasok berubah.

Dua contoh dari Medikara Health:

1. Pada Agustus 2026, biaya *cloud* Medikara+ Rp57 juta untuk sekitar 28.000 janji temu, atau Rp2.036 per janji temu.
2. Pada Oktober 2026, bulan kampanye vaksinasi influenza, biayanya naik 39 persen menjadi Rp79,3 juta. Janji temu juga naik menjadi 45.000, sehingga biaya per janji temu justru turun 13 persen menjadi Rp1.762. Tagihan naik, tetapi efisiensinya membaik.

Memilih unit tidak sembarangan. Unit yang baik memenuhi empat syarat. Pertama, dipahami direksi tanpa penjelasan teknis. Kedua, datanya tersedia rutin, paling tidak setiap bulan. Ketiga, ikut bergerak bersama pemicu biaya, sehingga perubahan unit mencerminkan perubahan beban. Keempat, sulit dimanipulasi, misalnya tidak bisa diperbaiki hanya dengan memecah satu transaksi menjadi dua. Tabel 3.10 memuat calon unit untuk beberapa layanan Medikara Health, termasuk integrasi dengan Badan Penyelenggara Jaminan Sosial (BPJS) Kesehatan.

Tabel 3.10: Calon metrik biaya per unit untuk layanan Medikara Health

Layanan	Unit	Sumber data penyebut	Catatan
SIMRS	Kunjungan pasien	Data pendaftaran SIMRS	Mudah dipahami, tetapi rawat inap dan rawat jalan bebannya berbeda
Medikara+	Janji temu terkonfirmasi	Basis data Medikara+	Janji yang dibatalkan sebaiknya tidak dihitung
Laboratorium rujukan	Hasil pemeriksaan terbit	Sistem informasi laboratorium	Mengikuti beban pengolahan dengan baik
Integrasi BPJS Kesehatan	Klaim terkirim	Log integrasi	Klaim yang dikirim ulang perlu dipisahkan

Kasus Medikara Health: Biaya per Kunjungan dan per Janji Temu

Komite biaya teknologi memilih dua metrik untuk dilaporkan ke direksi setiap triwulan. Metrik pertama adalah biaya infrastruktur SIMRS per kunjungan pasien. Untuk rumah sakit Bekasi, nilai awalnya Rp5.619 dan opsi B pada Tabel 3.3 memperkirakan Rp4.524. Metrik kedua adalah biaya *cloud* Medikara+ per janji temu terkonfirmasi, dengan nilai awal Rp2.036. Dari kedua metrik itu dirumuskan sasaran nilai: biaya infrastruktur per kunjungan pasien turun 15 persen dalam dua tahun, menjadi paling tinggi Rp4.776 untuk rumah sakit Bekasi. Sasaran ini sengaja lebih rendah daripada perkiraan opsi B (sekitar 19,5 persen), karena perkiraan TCO masih kasar dan migrasi jarang berjalan persis sesuai rencana. Sasaran itu didampingi syarat: ketersediaan SIMRS tidak boleh turun di bawah tingkat sekarang. Tanpa syarat ini, cara termudah menurunkan biaya per kunjungan adalah memangkas cadangan dan DR.

Pendalaman Teknis (opsional): Mengolah Tagihan FOCUS

Berkas `sources/chapter03/contoh-tagihan-focus.csv` berisi tagihan rekaan Medikara Health bulan Agustus 2026 dengan sebagian kolom FOCUS. Listing 3.1 mengelompokkan `EffectiveCost` menurut *tag* layanan dan lingkungan.

```
def kelompokkan_biaya(jalur_berkas):
  """Menjumlahkan biaya per pasangan (layanan, lingkungan)."""
  total = defaultdict(float)
  with open(jalur_berkas, newline="", encoding="utf-8") as berkas:
    for baris in csv.DictReader(baris_data(berkas)):
      tag = baca_tag(baris["Tags"])
      layanan = tag.get("layanan", TANPA_TAG)
      lingkungan = tag.get("lingkungan", TANPA_TAG)
      total[(layanan, lingkungan)] += float(baris[KOLOM_BIAYA])
  return total
```

Listing 3.1: Pengelompokan biaya per *tag*, tersedia di `sources/chapter03/olah-tagihan-focus.py`

Fungsi `cetak_biaya_per_unit` di berkas yang sama lalu membagi total tiap layanan dengan jumlah unitnya. Skrip dijalankan dengan `python olah-tagihan-focus.py contoh-tagihan-focus.csv`. Hasilnya: Medikara+ Rp57 juta (Rp47 juta produksi dan Rp10 juta pengembangan), atau Rp2.036 per janji temu; laboratorium rujukan Rp150 per hasil pemeriksaan; dan biaya tanpa *tag* Rp5 juta. Lingkungan pengembangan sengaja ikut dihitung dalam biaya per unit, karena keberadaannya hanya untuk melayani layanan tersebut. Pengelompokan yang sama dapat ditulis sebagai *query* `GROUP BY` pada nilai *tag*, bila data tagihan sudah dimuat ke gudang data.

3.6 Keberlanjutan dan Jejak Karbon

Setiap *workload* digital menghasilkan emisi dari dua sumber. Sumber pertama adalah emisi operasional, yaitu listrik yang dipakai server, penyimpanan, jaringan, dan pendingin. Besarnya bergantung pada jumlah listrik yang dipakai dan pada cara listrik itu dibangkitkan di lokasi pusat data. Sumber kedua adalah emisi yang melekat pada perangkat keras (*embodied emissions*), yaitu emisi dari pembuatan, pengiriman, dan pembuangan perangkat.

Kerangka Well-Architected dari AWS dan Google Cloud memasukkan keberlanjutan sebagai salah satu pilarnya [10, 69]. Pilar keberlanjutan AWS membagi tanggung jawab dengan cara yang mirip pembagian tanggung jawab keamanan pada Bab 2. Penyedia bertanggung jawab atas keberlanjutan pusat datanya, misalnya efisiensi gedung dan sumber energinya. Pelanggan bertanggung jawab atas keberlanjutan *workload*-nya sendiri, yaitu seberapa efisien sumber daya dirancang dan dipakai [18]. Pindah ke *cloud* tidak otomatis membuat *workload* hijau; server yang menganggur di *cloud* tetap membuang energi.

Analoginya seperti konsumsi bahan bakar armada ambulans. Total liter per bulan wajar naik bila jumlah perjalanan naik. Ukuran yang menunjukkan efisiensi adalah liter per 100 kilometer, atau liter per pasien yang diantar. Armada yang total bahan bakarnya naik bisa saja makin efisien, dan sebaliknya.

3.6.1 Software Carbon Intensity

Software Carbon Intensity (SCI) adalah ukuran laju emisi karbon perangkat lunak per satuan fungsi. Spesifikasinya dikembangkan Green Software Foundation dan diterbitkan sebagai standar ISO/IEC 21031:2024 [77, 70]. Rumusnya:

$$\mathrm{SCI} = \frac{(E \times I) + M}{R}$$

Huruf E adalah energi yang dipakai perangkat lunak (kWh). Huruf I adalah intensitas karbon listrik di lokasi tersebut (gram setara karbon dioksida per kWh). Huruf M adalah porsi emisi melekat perangkat keras yang dipakai. Huruf R adalah satuan fungsi, misalnya per janji temu atau per kunjungan pasien. Satuan fungsi inilah yang membuat SCI sejalan dengan *unit economics*: biaya per janji temu dan karbon per janji temu dibaca berdampingan. Spesifikasi ini menekankan pengurangan emisi yang nyata, sehingga pembelian kompensasi karbon (*offset*) tidak dipakai untuk menurunkan skor SCI [70].

Dua hal pada jejak karbon sering tertukar. Pembagiannya begini. Total emisi adalah jumlah seluruh emisi dalam satu periode, dan wajar naik ketika layanan tumbuh. SCI adalah intensitas, yaitu emisi per satuan fungsi. Laporan keberlan-

jutan perusahaan biasanya memakai total, sedangkan keputusan arsitektur lebih tepat dinilai dengan intensitas.

3.6.2 Region, Pemanfaatan, dan Jadwal Kerja

Tiga tuas memengaruhi emisi *workload*. Tuas pertama adalah pilihan region, karena intensitas karbon listrik berbeda antarlokasi. Tuas kedua adalah pemanfaatan sumber daya: satu server yang terpakai 60 persen lebih efisien daripada tiga server yang masing-masing terpakai 20 persen. Tuas ketiga adalah jadwal kerja: lingkungan uji dapat dimatikan di luar jam kerja, dan pekerjaan yang tidak mendesak dapat digeser ke waktu atau lokasi dengan listrik yang lebih bersih. Tabel 3.11 memperlihatkan dampak setiap tuas pada biaya dan karbon, beserta batasannya bagi Medikara Health.

Tabel 3.11: Tiga tuas keberlanjutan beserta dampaknya pada biaya dan karbon

Tuas	Dampak biaya	Dampak karbon	Batasan di Medikara
Pilihan region	Bisa naik atau turun, bergantung harga region	Turun bila listrik region lebih bersih	Rekam medis harus tetap di Indonesia (Bab 2), sehingga pilihan region terbatas
Pemanfaatan sumber daya	Turun karena kapasitas menganggur berkurang	Turun karena energi dan perangkat terpakai lebih efisien	*Rightsizing* harus memberi ruang untuk puncak layanan pasien
Jadwal kerja	Turun karena jam menyala berkurang	Turun, terutama untuk lingkungan uji dan pekerjaan *batch*	Layanan pasien 24 jam tidak dapat dijadwalkan ulang

Biaya dan karbon umumnya bergerak searah: mematikan sumber daya menganggur dan *rightsizing* menurunkan keduanya. Namun keduanya tidak selalu sejalan, dan pertentangan itu perlu ditulis eksplisit. Komitmen tiga tahun yang besar dapat mendorong tim membiarkan sumber daya tetap menyala karena "sudah dibayar". Region yang lebih murah belum tentu memakai listrik yang lebih bersih. Strategi DR yang menyiapkan salinan aktif di lokasi kedua menambah biaya sekaligus emisi, tetapi diperlukan demi keselamatan pasien. Keputusannya tetap menimbang ketiga sisi: biaya, karbon, dan risiko layanan.

Kasus Medikara Health: SCI per Janji Temu Medikara+

Misalkan tim memperkirakan SCI Medikara+ untuk satu bulan biasa dengan sekitar 28.000 janji temu. Energi diperkirakan dari data alat jejak karbon penyedia, dan intensitas karbon listrik memakai angka asumsi 0,70 kilogram setara karbon dioksida per kWh. Angka intensitas ini rekaan untuk latihan; nilai yang berlaku diambil dari data penyedia atau faktor emisi resmi. Tabel

3.12 memperlihatkan perhitungan sebelum dan sesudah dua perbaikan: lingkungan pengembangan dimatikan di luar jam kerja dan mesin produksi di-*rightsizing*.
Hasilnya, SCI turun dari sekitar 58,2 gram menjadi 44,8 gram per janji temu, atau sekitar 23 persen. Perbaikan yang sama juga menurunkan biaya lingkungan pengembangan, sehingga satu keputusan menggerakkan dua metrik sekaligus. Region tidak diubah karena rekam medis dan data pasien harus tetap berada di Indonesia.

Tabel 3.12: Perhitungan SCI Medikara+ per janji temu untuk satu bulan (angka rekaan)

Komponen	Sebelum	Sesudah
E: energi per bulan (kWh)	1.900	1.450
I: intensitas karbon listrik (kg per kWh, asumsi)	0,70	0,70
$E \times I$: emisi operasional (kg)	1.330	1.015
M: porsi emisi melekat perangkat keras (kg)	300	240
R: janji temu per bulan	28.000	28.000
SCI (gram per janji temu)	**58,2**	**44,8**

3.7 Aktivitas: Model Biaya Awal dan Sasaran Nilai Cloud

Tujuan. Tim menghasilkan tiga hal untuk satu *workload*: perkiraan TCO kasar, satu metrik biaya per unit, dan satu sasaran nilai *cloud* yang terukur. Ketiganya menjadi bagian tonggak capstone C1 (Konteks dan Strategi).

Pembagian peran.

- Anggota berlatar akuntansi atau keuangan memimpin struktur biaya, asumsi, dan perhitungan biaya per unit.
- Anggota berlatar teknis memperkirakan biaya sumber daya dengan kalkulator harga penyedia dan menilai pola bebannya.
- Anggota lain, misalnya yang berlatar manajemen, hukum, atau kesehatan, memeriksa kewajaran asumsi, kendala regulasi, dan makna metrik bagi direksi.

Langkah kerja.

1. Pilih satu *workload* dari matriks kendala penempatan Bab 2. Utamakan *workload* yang datanya dapat diperoleh dan pemiliknya dapat ditanya.
2. Susun biaya kondisi sekarang selama lima tahun dengan `sources/chapter03/templat-tco-sederhana.xlsx`. Setiap angka diberi asumsi dan sumber di kolom terakhir.

3. Perkirakan satu atau dua opsi *cloud*, misalnya *rehost* apa adanya dan *rehost* dengan *rightsizing* serta komitmen. Gunakan Gambar 3.2 sebagai daftar periksa biaya yang sering terlupa.
4. Tetapkan satu metrik biaya per unit dengan empat syarat unit yang baik, lalu hitung nilai awalnya.
5. Rumuskan sasaran nilai yang memuat angka, arah, dan batas waktu, misalnya "biaya infrastruktur per kunjungan pasien turun 15 persen dalam dua tahun", lengkap dengan satu metrik pendamping. Tulis di `sources/chapter03/lembar-sasaran-nilai.md`.

Luaran. Templat TCO yang terisi, lembar sasaran nilai, dan ringkasan satu halaman yang menjelaskan opsi yang direkomendasikan beserta asumsi terbesarnya. Luaran ini masuk ke C1 sebagai model biaya awal dan sasaran nilai *cloud*, dan akan dipakai lagi saat menilai strategi migrasi 7R pada Bab 4.

Kriteria penilaian. Tabel 3.13 memuat kriteria singkat yang dipakai saat tinjauan Sabtu.

Tabel 3.13: Kriteria penilaian Aktivitas Bab 03

Kriteria	Bobot	Bukti yang dicari
Kelengkapan dan kejujuran TCO	30 persen	Komponen yang sering terlupa ikut dihitung, lingkup kedua sisi setara
Asumsi yang dapat diperiksa	20 persen	Setiap angka punya sumber atau alasan tertulis
Kualitas metrik biaya per unit	25 persen	Unit bermakna bagi direksi dan datanya tersedia rutin
Sasaran nilai yang terukur	25 persen	Ada angka, arah, batas waktu, dan metrik pendamping

Etika. Data organisasi nyata hanya dipakai seizin organisasi dan dianonimkan. Angka biaya cukup dalam bentuk pembulatan. Data pribadi pasien atau karyawan tidak boleh dicantumkan.

Pendalaman teknis (opsional). Anggota berlatar teknis menjalankan `sources/chapter03/olah-tagihan-focus.py` seperti pada kotak pendalaman seksi 3.5, atau pada ekspor tagihan FOCUS organisasi pilihan bila diizinkan.

3.8 Ringkasan

Perpindahan ke *cloud* mengubah belanja infrastruktur dari CapEx yang diputuskan sekali dalam siklus anggaran menjadi OpEx yang bergerak setiap hari mengikuti pemakaian. Perubahan ini meringankan arus kas di awal, tetapi memindahkan keputusan biaya dari ruang direksi ke tangan tim teknik. TCO yang jujur memban-

dingkan kedua sisi dengan masa, lingkup, dan tingkat layanan yang sama, serta memasukkan biaya di bawah permukaan: lisensi, tenaga kerja, migrasi, transfer data keluar, dan pelatihan. Kasus SIMRS rumah sakit Bekasi menunjukkan bahwa memindahkan sistem apa adanya bisa lebih mahal, sedangkan penghematan datang dari *rightsizing* dan komitmen.

Model harga *on-demand*, komitmen pemakaian, dan *spot* dipakai bersamaan sesuai pola beban. Komitmen punya titik impas, yaitu satu dikurangi potongannya, dan membeli komitmen sebelum *rightsizing* adalah kesalahan yang paling mahal. FinOps Framework 2026 menjadikan pengelolaan biaya teknologi sebagai kerja bersama, bukan tugas bagian keuangan saja. Enam prinsipnya menekankan kolaborasi, nilai bisnis, kepemilikan, dan data yang tepat waktu. Tiga fasenya, Inform, Optimize, dan Operate, berulang terus dengan kematangan yang naik bertahap. Persona inti, empat domain, dan *capability*-nya memberi bahasa bersama, termasuk *capability* Executive Strategy Alignment yang menautkan biaya teknologi dengan strategi pimpinan.

Alokasi dimulai dari *tag* yang disiplin, dilanjutkan dengan *showback* sebelum *chargeback*. FOCUS menyatukan tagihan dari banyak penyedia ke satu skema, dan perbedaan `BilledCost` dengan `EffectiveCost` menentukan apakah biaya komitmen jatuh ke pemilik yang benar. Biaya per unit, misalnya per kunjungan pasien atau per janji temu, menerjemahkan tagihan menjadi efisiensi yang dapat dinilai direksi. SCI menerapkan cara pandang yang sama pada karbon, sehingga biaya dan emisi per janji temu dapat dibaca berdampingan.

Pesan utama bab ini sederhana: tagihan *cloud* yang naik belum tentu buruk, dan tagihan yang turun belum tentu baik. Yang menentukan adalah biaya dan emisi per satuan nilai layanan, siapa pemiliknya, dan seberapa cepat organisasi mengetahuinya. Model biaya awal dan sasaran nilai yang disusun pada Aktivitas bab ini menjadi patokan bagi setiap keputusan arsitektur pada bab-bab berikutnya.

3.9 Latihan

Latihan 1: Tagihan Naik, Pengguna Tetap

Latihan ini membuktikan bahwa kenaikan tagihan *cloud* dapat dijelaskan dari data tagihan itu sendiri. Pertanyaannya: mengapa tagihan *cloud* sebuah organisasi bisa naik walaupun jumlah penggunanya tetap? Berikan tiga penyebab yang mungkin.

1. Ambil tagihan dua bulan berurutan untuk satu layanan, dari organisasi pilihan (seizin organisasi) atau dari contoh Tabel 3.14.
2. Pastikan jumlah pengguna atau transaksi kedua bulan relatif sama.
3. Hitung selisih per komponen biaya, lalu urutkan dari yang terbesar.

4. Untuk tiga selisih terbesar, tulis dugaan penyebab dan cara memeriksanya.
5. Isi Tabel 3.15.

Tabel 3.14: Contoh Latihan 1: tagihan Medikara+ Juni dan Juli 2026, sekitar 28.000 janji temu per bulan, dalam juta rupiah

Komponen	Juni	Juli	Selisih	Dugaan penyebab
Komputasi produksi	27,5	27,5	0,0	Tidak berubah
Basis data produksi	12,0	12,0	0,0	Tidak berubah
Lingkungan pengembangan	3,4	10,0	6,6	Dibiarkan menyala 24 jam setelah uji fitur baru
Penyimpanan	2,4	3,2	0,8	Log dan cadangan tanpa batas masa simpan
Transfer data keluar	3,3	4,3	1,0	Fitur unduh hasil pemeriksaan berukuran besar
Total	**48,6**	**57,0**	**8,4**	Naik 17 persen; per janji temu dari Rp1.736 menjadi Rp2.036

Tabel 3.15: Lembar isian Latihan 1

Komponen	Bulan 1	Bulan 2	Selisih	Dugaan penyebab
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
Total	…	…	…	…

Pertanyaan analisis:

1. Pada contoh, berapa persen kenaikan total yang berasal dari lingkungan pengembangan? Apa artinya bagi prioritas perbaikan?
2. Dari tiga penyebab yang ditemukan, mana yang akan terdeteksi lebih cepat bila Medikara Health sudah memakai *dashboard* biaya harian dan *tag* `lingkungan`?

Latihan 2: Metrik Biaya per Unit bagi Direksi

Latihan ini membuktikan bahwa pilihan unit menentukan apakah metrik biaya bermakna bagi direksi. Pilih satu layanan Medikara Health dan usulkan metrik biaya per unit yang bermakna bagi direksi. Jelaskan alasan pemilihannya.

1. Pilih satu layanan, misalnya laboratorium rujukan, Medikara+, atau integrasi BPJS Kesehatan.
2. Tuliskan tiga calon unit, termasuk satu unit teknis sebagai pembanding.
3. Hitung nilai contoh setiap calon dengan biaya bulanan layanan.
4. Nilai setiap calon dengan empat syarat unit yang baik: dipahami direksi, data tersedia rutin, mengikuti pemicu biaya, dan sulit dimanipulasi.

5. Pilih satu unit dan tulis alasannya dalam satu paragraf.

Tabel 3.16: Contoh Latihan 2: calon unit untuk laboratorium rujukan, biaya Agustus 2026 Rp7,8 juta (angka rekaan)

Calon unit	Penyebut	Nilai	Penilaian empat syarat
Per hasil pemeriksaan	52.000 hasil	Rp150	Dipahami, tersedia, mengikuti beban; sulit dimanipulasi. Terpilih
Per pasien laboratorium	21.000 pasien	Rp371	Dipahami, tetapi satu pasien bisa punya satu atau sepuluh pemeriksaan
Per jam komputasi *batch*	1.240 jam	Rp6.290	Tersedia, tetapi teknis dan tidak bermakna bagi direksi

Tabel 3.17: Lembar isian Latihan 2

Calon unit	Penyebut	Nilai	Penilaian empat syarat
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Bila jumlah pemeriksaan per pasien naik karena paket pemeriksaan baru, apa yang terjadi pada biaya per pasien dan biaya per hasil pemeriksaan? Metrik mana yang menyesatkan?
2. Metrik pendamping apa yang perlu dipasang agar biaya per unit tidak diturunkan dengan mengorbankan mutu layanan?

Latihan 3: Kapan Komitmen Menguntungkan

Latihan ini membuktikan bahwa keuntungan komitmen ditentukan oleh pemanfaatannya. Pertanyaannya: kapan model harga berbasis komitmen lebih menguntungkan daripada *on-demand*, dan kapan justru merugikan?

1. Tetapkan harga *on-demand*, potongan komitmen, dan besar komitmen. Contoh memakai Rp1.000 per unit-jam, potongan 40 persen, dan komitmen 100 unit selama 730 jam per bulan.
2. Hitung titik impas pemanfaatan dengan rumus satu dikurangi potongan.
3. Hitung biaya bulanan untuk empat tingkat pemakaian rata-rata, baik dengan *on-demand* maupun dengan komitmen.
4. Tandai tingkat pemakaian yang membuat komitmen merugi.
5. Isi Tabel 3.19 dengan angka dari penyedia atau dari kalkulator harga.

Pertanyaan analisis:

Tabel 3.18: Contoh Latihan 3: biaya bulanan komitmen 100 unit dengan potongan 40 persen, dalam juta rupiah (angka rekaan)

Pemakaian rata-rata	*On-demand*	Komitmen	Selisih	Lebih untung
50 unit (50 persen)	36,5	43,8	−7,3	*On-demand*
60 unit (60 persen)	43,8	43,8	0,0	Impas
70 unit (70 persen)	51,1	43,8	7,3	Komitmen
90 unit (90 persen)	65,7	43,8	21,9	Komitmen

Tabel 3.19: Lembar isian Latihan 3

Pemakaian rata-rata	*On-demand*	Komitmen	Selisih	Lebih untung
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

1. SIMRS versi lama di salah satu rumah sakit direncanakan diganti dalam 14 bulan. Apakah komitmen tiga tahun masuk akal untuk server-nya, walaupun pemanfaatannya 90 persen? Jelaskan dengan membaca tabel.
2. Pemakaian Medikara+ berkisar 45 unit di malam hari dan 90 unit di siang hari. Berapa besar komitmen yang aman, dan bagian mana yang sebaiknya tetap *on-demand*?
3. Bagaimana titik impas berubah bila potongan komitmen hanya 25 persen?

Bab 4

Kerangka Adopsi Cloud dan Strategi Migrasi 7R

Tujuan Pembelajaran

- Membandingkan struktur kerangka adopsi *cloud* dari AWS (Amazon Web Services), Microsoft, dan Google.
- Menilai kesiapan organisasi dengan enam perspektif AWS CAF.
- Memberi usulan strategi 7R yang beralasan untuk setiap aplikasi dalam portofolio.

4.1 Pendahuluan

Medikara Health tumbuh lewat akuisisi. Setiap rumah sakit yang bergabung membawa SIMRS (Sistem Informasi Manajemen Rumah Sakit), server, dan aplikasi pendukungnya sendiri. Misalkan inventaris pertama tim teknologi informasi (TI) mencatat 46 aplikasi. Tidak ada yang tahu persis aplikasi mana yang harus dipindah lebih dulu, mana yang sebaiknya ditinggal, dan mana yang lebih baik dimatikan. Bab 2 mengusulkan model *hybrid* sebagai model transisi dan menunda keputusan tentang ERP (*Enterprise Resource Planning*) keuangan. Bab 3 menghitung biaya per kunjungan pasien. Keduanya menyisakan satu pertanyaan untuk bab ini: aplikasi mana diperlakukan dengan cara apa, dan apakah organisasinya sudah siap?

Pertanyaan kedua itu sering dilewati. Migrasi yang hanya digerakkan divisi TI sering gagal bukan karena teknologinya. Servernya berhasil dipindah, tetapi pengguna tidak dilatih, anggaran *cloud* tidak jelas pemiliknya, dan kontrol akses yang lemah ikut pindah apa adanya. Kerangka adopsi *cloud* disusun untuk mencegah

pola ini. Kerangka itu memaksa organisasi menilai kesiapan orang, proses, dan tata kelola, bukan hanya kesiapan server. Penilaian kesiapan dan pilihan migrasi perlu melibatkan pemilik layanan, keuangan, keamanan, serta tim operasi. Landing zone menjadi prasyarat gelombang migrasi.

Gambar 4.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 4.1: Kesiapan dan fondasi mendahului gelombang migrasi

Bab ini berjalan dalam lima langkah: membandingkan tiga kerangka adopsi, menilai kesiapan dengan enam perspektif, menginventaris portofolio aplikasi beserta dependensinya, memberi usulan strategi migrasi 7R, lalu menyiapkan fondasi berupa *landing zone* dan tim pemandu adopsi. Rujukan utamanya adalah AWS Cloud Adoption Framework [2], Cloud Adoption Framework for Azure [95], Google Cloud Adoption Framework [66], dan panduan strategi migrasi dari AWS Prescriptive Guidance [9].

4.2 Tiga Kerangka Adopsi Cloud

Pertanyaan pertama: mengapa organisasi butuh kerangka? Kesalahan dalam adopsi *cloud* cenderung berulang dari satu organisasi ke organisasi lain. Penyedia *cloud* besar melihat ribuan migrasi, lalu merangkum pelajarannya menjadi kerangka yang dapat dipakai pelanggan.

Kerangka adopsi *cloud*, atau CAF (*Cloud Adoption Framework*), adalah kumpulan panduan, praktik, dan alat bantu untuk merencanakan dan menjalankan adopsi *cloud* di tingkat organisasi. Menurut AWS, kerangkanya dipakai untuk tiga hal: menemukan dan memprioritaskan peluang transformasi, menilai dan meningkatkan

kesiapan *cloud*, serta memperbarui peta jalan transformasi secara bertahap [2]. CAF tidak merancang satu aplikasi. CAF menjawab pertanyaan yang lebih luas: siapa yang perlu dilibatkan, kemampuan apa yang harus dibangun, dan dalam urutan apa.

Analoginya seperti buku panduan dari perusahaan jasa pindahan kantor. Setiap perusahaan pindahan punya daftar periksa sendiri dengan judul bab yang berbeda. Isinya mirip: siapa yang memutuskan, barang apa yang dibawa, kapan kantor baru siap ditempati, siapa yang menjaga keamanan selama pindahan, dan bagaimana karyawan diberi tahu. Kantor yang hanya menyiapkan truk tanpa memberi tahu karyawannya tetap akan kacau pada hari pertama.

Dua contoh kasus nyata yang dicegah oleh kerangka adopsi:

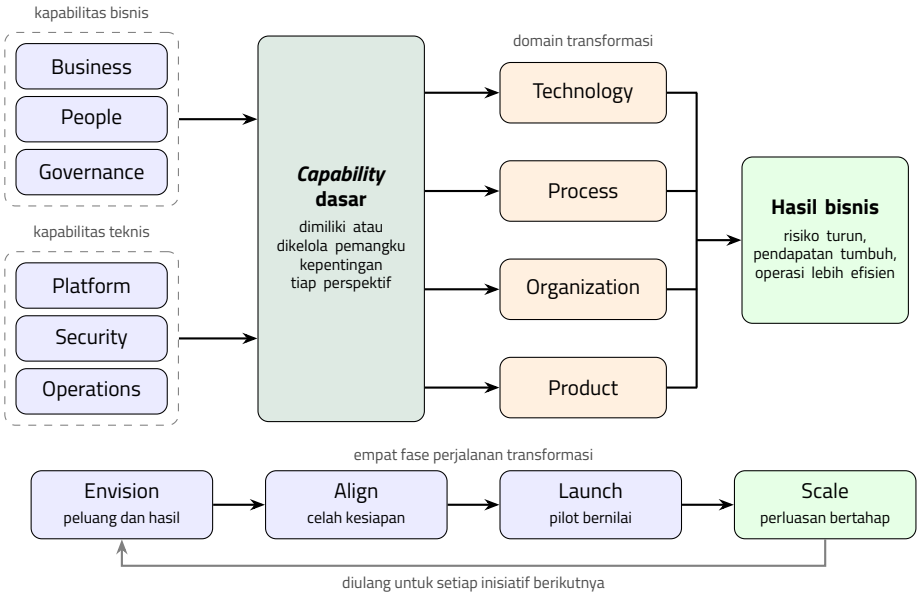
1. Misalkan rumah sakit Bekasi memindahkan aplikasi jadwal kamar operasi ke *cloud* dalam 6 minggu tanpa melibatkan perawat. Servernya berjalan, tetapi 40 perawat kamar operasi tetap memakai papan tulis selama 3 bulan karena tidak pernah dilatih. Celahnya ada pada orang, bukan pada server.
2. Misalkan tim TI rumah sakit Depok membuka akun *cloud* sendiri dengan kartu kredit divisi. Tagihan sekitar Rp18 juta per bulan tidak tercatat di anggaran unit mana pun sampai audit keuangan menemukannya 7 bulan kemudian. Celahnya ada pada tata kelola.

AWS CAF: Enam Perspektif

AWS CAF mengelompokkan kemampuan organisasi ke dalam enam perspektif: Business, People, Governance, Platform, Security, dan Operations [4]. Tiga perspektif pertama berisi kapabilitas bisnis, tiga terakhir berisi kapabilitas teknis. Setiap perspektif berisi sekumpulan *capability*, yaitu kemampuan organisasi memakai proses untuk mengerahkan sumber daya (orang, teknologi, dan aset lain) demi hasil tertentu. *Capability* itu dimiliki atau dikelola oleh pemangku kepentingan yang fungsinya berdekatan. *Capability* kemudian mendukung empat domain transformasi, yaitu Technology, Process, Organization, dan Product, yang bermuara pada hasil bisnis. Perjalanannya dibagi menjadi empat fase yang diulang per inisiatif: Envision, Align, Launch, dan Scale [2]. Gambar 4.2 merangkum susunannya.

Microsoft CAF: Tujuh Fase

Cloud Adoption Framework for Azure menyusun panduannya sebagai urutan fase, bukan sebagai perspektif. Pembaruan Agustus 2026 memuat tujuh fase: Strategy, Plan, Ready, Adopt, Govern, Secure, dan Manage, ditambah panduan skenario untuk kebutuhan kedaulatan data (*sovereignty*) [95]. Secara ringkas, Strategy menetapkan alasan dan hasil bisnis, Plan menyusun rencana adopsi dan kesiapan



Gambar 4.2: Susunan AWS CAF: enam perspektif berisi *capability* dasar yang mendukung empat domain transformasi, dijalankan dalam empat fase berulang

keterampilan, Ready menyiapkan *landing zone*, dan Adopt menjalankan migrasi atau membangun yang baru. Govern, Secure, dan Manage berjalan terus selama *workload* ada di *cloud*. Bentuk fase ini memudahkan penyusunan rencana proyek, karena setiap fase dapat dijadikan paket kerja. Skenario kedaulatan data relevan bagi organisasi Indonesia yang terikat aturan lokasi data, seperti dibahas di Bab 2.

Google Cloud Adoption Framework: Empat Tema dan Tiga Fase

Google menyusun kerangkanya sebagai skala kematangan. Ada empat tema: Learn (keterampilan tim), Lead (dukungan pimpinan dan tim lintas fungsi), Scale (otomasi dan cara operasi yang dapat diperluas), dan Secure (kemampuan melindungi layanan dan data). Setiap tema dinilai pada tiga fase kematangan: *tactical*, *strategic*, dan *transformational*. Seluruhnya dibaca melalui tiga komponen: orang, proses, dan teknologi [66]. Bentuk ini cocok untuk menjawab pertanyaan direksi tentang tingkat kematangan organisasi saat ini, karena jawabannya berupa tingkat yang mudah dibandingkan dari tahun ke tahun.

Tabel 4.1 membandingkan ketiganya.

Ketiga kerangka tidak saling meniadakan. Organisasi yang memakai lebih dari satu penyedia tetap dapat memakai satu kerangka untuk menilai kesiapan, karena yang dinilai adalah organisasinya, bukan layanan penyedianya. Bab ini

Tabel 4.1: Perbandingan tiga kerangka adopsi *cloud*

Aspek	AWS CAF	Microsoft CAF	Google CAF
Unit utama	Enam perspektif berisi *capability*	Tujuh fase	Empat tema
Pembagian waktu	Envision, Align, Launch, Scale	Urutan fase, sebagian berjalan terus	*Tactical*, *strategic*, *transformational*
Pertanyaan yang paling terbantu	Siapa yang memiliki kemampuan apa?	Pekerjaan apa yang dilakukan lebih dulu?	Sudah semampu apa organisasi saat ini?
Kekuatan	Jelas memetakan pemangku kepentingan	Mudah dijadikan paket kerja proyek	Mudah dikomunikasikan sebagai tingkat kematangan
Kelemahan bila dipakai sendirian	Tidak memberi urutan kerja rinci	Mudah terbaca sebagai urusan proyek TI saja	Kurang rinci untuk menilai satu kemampuan

memakai enam perspektif AWS CAF untuk penilaian kesiapan, karena pembagian perspektifnya paling jelas memisahkan pekerjaan anggota non-teknis dan anggota teknis dalam tim capstone.

Dua hal pada kerangka penyedia sering tertukar. Pembagiannya begini. Kerangka adopsi (CAF) menilai kesiapan organisasi dan mengatur perjalanan adopsi. Kerangka arsitektur, misalnya AWS Well-Architected dengan enam pilarnya, menilai mutu rancangan satu *workload* [10]. Pertanyaan "apakah tim keamanan sudah siap" milik CAF. Pertanyaan "apakah SIMRS di *cloud* punya cadangan di dua zona" milik kerangka arsitektur.

4.3 Penilaian Kesiapan dengan Enam Perspektif

Penilaian kesiapan *cloud* adalah potret kemampuan organisasi pada enam perspektif, diambil sebelum migrasi besar dimulai dan diulang secara berkala. AWS menyebut kesiapan *cloud* sebagai kemampuan organisasi memanfaatkan *cloud* secara efektif untuk transformasi digital [4]. Hasil penilaian bukan nilai rapor. Hasilnya adalah daftar celah yang harus ditutup, lengkap dengan pemilik dan tenggatnya.

Analoginya seperti pemeriksaan kesehatan berkala karyawan. Dokter memeriksa beberapa organ secara terpisah: jantung, paru, gula darah, dan kolesterol. Rata-rata semua hasil tidak berarti apa-apa. Karyawan dengan lima hasil normal dan satu kadar gula yang jauh di atas batas tetap perlu ditangani sekarang. Penilaian kesiapan dibaca dengan cara yang sama: yang menentukan adalah perspektif terlemah, bukan rata-ratanya.

Tabel 4.2 memuat fokus, pertanyaan kunci, dan pemangku kepentingan tiap perspektif. Kolom pemangku kepentingan menerjemahkan peran umum yang disebut AWS [4] ke jabatan yang lazim di rumah sakit swasta.

Tabel 4.2: Enam perspektif AWS CAF, pertanyaan kunci, dan pemangku kepentingannya

Perspektif	Pertanyaan kunci	Pemangku kepentingan
Business	Hasil bisnis apa yang dikejar, dengan angka berapa, dan siapa pemilik manfaatnya?	Direktur utama, direktur keuangan, direktur operasional, kepala TI
People	Keterampilan, budaya, dan struktur tim apa yang harus berubah?	Kepala TI, direktur operasional, kepala sumber daya manusia
Governance	Siapa yang memutuskan, bagaimana anggaran dan risiko dikendalikan?	Direktur keuangan, kepala manajemen risiko, kepala TI
Platform	Arsitektur dan lingkungan apa yang dibangun, dan bagaimana disiapkan berulang?	Kepala TI, arsitek, *engineer* infrastruktur
Security	Bagaimana kerahasiaan, keutuhan, dan ketersediaan data serta *workload* dijaga?	Kepala keamanan informasi, kepala kepatuhan, audit internal
Operations	Bagaimana layanan dipantau, dipulihkan, dan diubah tanpa mengganggu pelayanan?	Kepala operasional TI, pengelola layanan TI

Pertanyaan kemudian muncul: bagaimana memberi skor tanpa terjebak formalitas? Penilaian kesiapan mudah berubah menjadi formulir yang diisi sendiri oleh pemilik proses dengan angka yang menyenangkan semua orang. Empat aturan menjaga penilaian tetap jujur.

1. Setiap pertanyaan diberi skor 1 sampai 5: 1 belum ada, 2 ada secara informal dan bergantung pada orang tertentu, 3 terdokumentasi dan dijalankan sebagian, 4 dijalankan konsisten dan diukur, 5 diperbaiki terus berdasarkan data. Setiap skor wajib disertai bukti. Skor tanpa bukti dicatat 1.
2. Penilai bukan pemilik proses yang dinilai. Kepala keamanan informasi tidak menilai perspektif Security sendirian.
3. Keputusan dibaca dari skor terendah dan dari selisihnya terhadap batas minimal, bukan dari rata-rata seluruh perspektif.
4. Setiap celah punya pemilik, rencana penutupan, dan tanggal tinjau ulang, biasanya tiga bulan kemudian.

Dua hal pada penilaian ini sering tertukar. Pembagiannya begini. Audit memeriksa kepatuhan terhadap aturan yang sudah ditetapkan dan menghasilkan temuan. Penilaian kesiapan memeriksa kemampuan yang dibutuhkan untuk perubahan yang akan datang dan menghasilkan rencana. Temuan audit menjadi bukti yang berguna dalam penilaian kesiapan, tetapi penilaian kesiapan tidak boleh diperlakukan seperti audit, karena orang cenderung menyembunyikan kelemahan

di depan auditor.

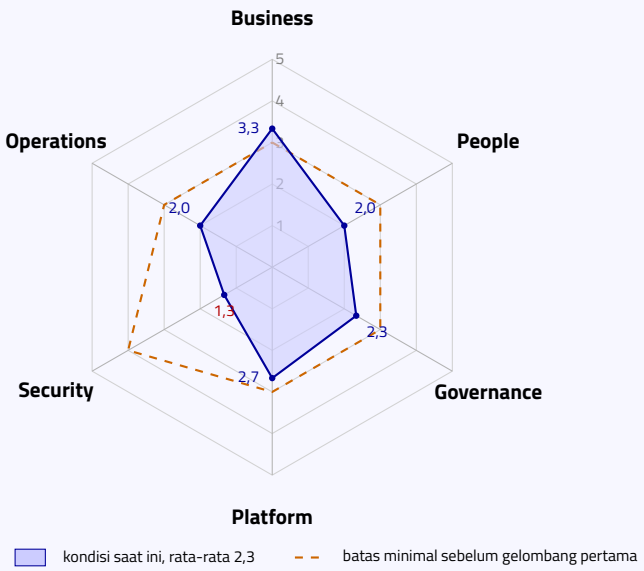
Dua contoh kasus nyata:

1. Misalkan sebuah jaringan klinik menilai dirinya dengan rata-rata 3,4 dan merasa siap. Skor Security-nya hanya 1,7 karena akun administrator dipakai bersama oleh 9 orang. Bila migrasi tetap dimulai, akun bersama itu ikut pindah ke *cloud* dan menjadi pintu masuk yang lebih luas.
2. Misalkan sebuah perusahaan asuransi memberi skor People 4 karena 30 staf TI sudah mengikuti pelatihan *cloud*. Buktinya ternyata daftar hadir, bukan hasil uji. Setelah aturan bukti diterapkan, skornya turun menjadi 2, dan anggaran sertifikasi masuk ke rencana kerja.

Kasus Medikara Health: Kesiapan Tim TI yang Tersebar

Misalkan tim TI Medikara Health terdiri atas 52 orang yang tersebar di lima rumah sakit dan kantor pusat. Setiap tim rumah sakit berdiri sendiri sejak akuisisi, dengan lima prosedur penanganan insiden yang berbeda. Hanya 4 orang yang sudah bersertifikat *cloud*. Audit internal menemukan 212 akun SIMRS milik mantan karyawan yang masih aktif, dan petugas vendor Medikara+ masuk ke sistem memakai akun bersama. Akun vendor itulah jalur insiden kebocoran data pada 2025.

Tim penilai memakai `sources/chapter04/lembar-penilaian-kesiapan-caf.xlsx` dengan tiga pertanyaan per perspektif. Hasilnya ada pada Gambar 4.3. Rata-rata seluruh perspektif 2,3. Business mendapat skor tertinggi (3,3) karena strategi 2026 sampai 2030 sudah memuat target dan biaya per kunjungan dari Bab 3. Security mendapat skor terendah (1,3) karena dua temuan di atas.



Gambar 4.3: Profil kesiapan Medikara Health pada enam perspektif (angka rekaan) dibandingkan dengan batas minimal: skor 3, kecuali Security 4

Batas minimal ditetapkan 3 untuk setiap perspektif dan 4 untuk Security, karena hampir setiap aplikasi Medikara menyimpan data pasien. Hasil ini punya dua konsekuensi. Pertama, gelombang migrasi pertama ditunda sekitar enam bulan untuk menutup tiga celah terbesar: membersihkan akun dan akses vendor (Security, selisih 2,7), menyeragamkan prosedur insiden (Operations, selisih 1,0), dan menyusun rencana pelatihan yang didanai (People, selisih 1,0). Kedua, pekerjaan yang tidak memindahkan data pasien tidak perlu menunggu. Aplikasi yang sudah tidak dipakai dapat langsung dimatikan, dan inventaris portofolio dapat dimulai sekarang.

4.4 Inventaris Portofolio Aplikasi dan Dependensi

Usulan migrasi tidak dapat diberikan untuk aplikasi yang tidak diketahui keberadaannya. Organisasi hasil akuisisi hampir selalu punya aplikasi yang dibeli oleh satu unit, dipasang di satu komputer, dan dilupakan setelah pembuatnya pindah kerja.

Inventaris portofolio aplikasi adalah daftar seluruh aplikasi organisasi beserta atribut yang dibutuhkan untuk mengambil keputusan migrasi. Satu baris mewakili satu aplikasi. Atribut minimalnya ada pada Tabel 4.3. Templatnya tersedia di `sources/chapter04/templat-inventaris-aplikasi.xlsx`.

Tabel 4.3: Atribut minimal inventaris portofolio aplikasi

Atribut	Mengapa dibutuhkan	Contoh: aplikasi apotek
Pemilik bisnis dan teknis	Menentukan siapa yang memutuskan dan siapa yang mengerjakan	Kepala unit apotek; tim TI kantor pusat
Pengguna	Menunjukkan siapa yang terdampak saat peralihan	Sekitar 96 staf di 12 apotek
Kritikalitas	Menentukan urutan gelombang dan jendela peralihan	Sedang
Teknologi	Menentukan apakah aplikasi dapat dipindah apa adanya	Aplikasi desktop, basis data lokal
Kontrak dan lisensi	Menentukan kapan peralihan paling murah	Dukungan vendor berhenti akhir 2026
Dependensi	Menentukan aplikasi yang harus pindah bersama	Membaca stok obat langsung dari basis data SIMRS
Kondisi	Menunjukkan masalah yang sudah dirasakan	Stok baru sinkron tiap malam

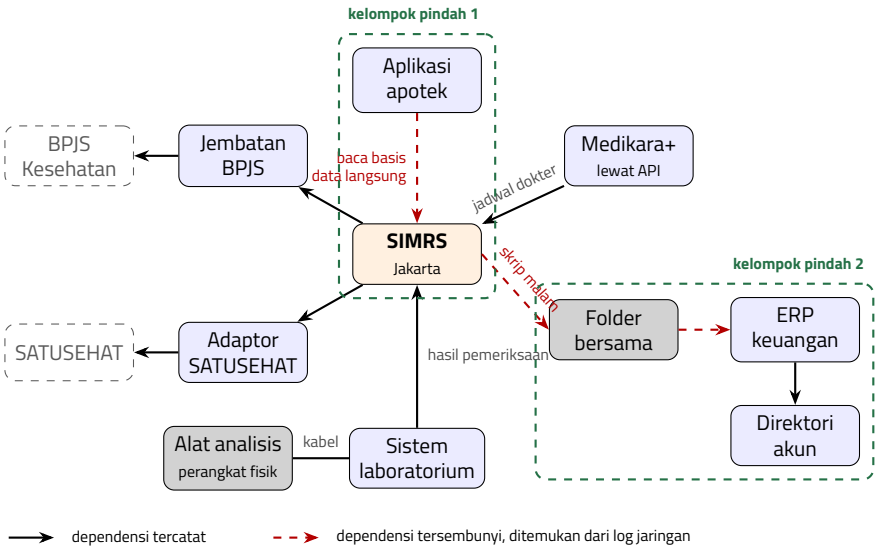
Analoginya seperti pindah rumah. Daftar barang dibuat lebih dulu, lengkap dengan pemilik tiap barang. Bagian yang paling sering terlupa adalah kabel di balik lemari. Lampu meja di kamar anak ternyata menyala dari stopkontak di kamar sebelah. Saat lemari diangkat, kabelnya tertarik dan lampunya padam. Dependensi aplikasi bekerja dengan cara yang sama: yang tidak tercatat baru ketahuan ketika salah satu ujungnya dipindah.

Dependensi tersembunyi bukan masalah kecil. Laporan State of the Cloud 2020 dari Flexera, yang menyurvei 750 responden pada kuartal pertama 2020, menempatkan pemahaman dependensi aplikasi sebagai tantangan teratas dalam migrasi *cloud* [54]. Dependensi dicari dari beberapa sumber sekaligus, karena setiap sumber hanya menangkap sebagian:

1. wawancara dengan pemilik aplikasi dan staf yang menjalankannya setiap hari;
2. berkas konfigurasi, misalnya alamat basis data atau alamat server lain yang ditulis langsung di aplikasi;
3. jadwal tugas otomatis yang berjalan malam hari, misalnya skrip penyalin data;
4. log jaringan atau alat *discovery* yang mencatat koneksi antarserver selama beberapa pekan.

Gambar 4.4 memperlihatkan sebagian peta dependensi Medikara Health di sekitar SIMRS Jakarta. Garis merah adalah dependensi yang tidak disebut dalam wawancara dan baru terlihat di log jaringan. Medikara+ terhubung ke SIMRS lewat Application Programming Interface (API) yang memang dirancang untuk dipanggil

lewat internet, sehingga keduanya boleh berada di lokasi berbeda. Hal yang sama berlaku untuk jembatan ke BPJS (Badan Penyelenggara Jaminan Sosial) Kesehatan dan adaptor SATUSEHAT, yang menurut model *hybrid* Bab 2 dipindah ke *region* Jakarta.



Gambar 4.4: Sebagian peta dependensi Medikara Health (rekaan). Dua dependensi tersembunyi mengubah susunan kelompok pindah

Peta ini menghasilkan kelompok pindah (*move group*), yaitu sekumpulan aplikasi yang harus dipindah dalam gelombang yang sama. Bila dipisah, koneksi di antara mereka melintasi jaringan antara ruang server dan *cloud*, sehingga menjadi lambat atau putus. Dua contoh dari peta tersebut:

1. Aplikasi apotek membaca stok obat langsung dari basis data SIMRS sekitar 30.000 kali per bulan. Selama hubungan ini ada, aplikasi apotek tidak dapat dipindah tanpa SIMRS. Pilihannya dua: memindah keduanya bersama, atau mengganti pembacaan langsung dengan antarmuka resmi sebelum migrasi.
2. Skrip malam di server SIMRS menyalin data tagihan ke folder bersama, lalu ERP keuangan membacanya setiap pagi. Hubungannya hanya 30 kali per bulan, tetapi cukup untuk menyatukan kelompok SIMRS dan kelompok ERP. Keputusan tentang ERP yang ditunda di Bab 2 ternyata terikat pada keputusan tentang SIMRS.

Sistem laboratorium memberi pelajaran yang berbeda. Aplikasinya terhubung dengan kabel langsung ke alat analisis, sehingga terikat pada gedung laboratorium. Alat bantu apa pun akan memasukkannya ke kelompok SIMRS, karena koneksinya ke SIMRS mencapai ribuan kali per bulan. Tim tetap harus memutuskan dengan

pengetahuan yang tidak tercatat di log: alat fisik tidak ikut pindah.

Inventaris dan peta dependensi juga menjadi bahan dua alat yang dibahas rinci di mata kuliah IT40513. Prinsip arsitektur, misalnya "data pasien hanya diakses lewat antarmuka resmi", dipakai untuk menilai setiap baris inventaris. *Gap analysis* membandingkan kondisi saat ini dalam inventaris dengan kondisi target pada tonggak C3. Bab ini cukup memastikan bahwa inventarisnya lengkap dan jujur, karena kedua alat itu bergantung padanya.

Pendalaman Teknis (opsional): Kelompok Pindah dari Log Jaringan

Catatan koneksi dari log jaringan atau alat *discovery* dapat diolah menjadi kelompok pindah. Setiap aplikasi menjadi titik, setiap koneksi yang cukup sering menjadi garis, dan setiap komponen terhubung menjadi satu kelompok pindah. Data contohnya ada di `sources/chapter04/contoh-koneksi-jaringan.csv`, berisi kolom sumber, tujuan, *port*, jumlah koneksi 30 hari, dan `boleh_dipisah`. Kolom terakhir diisi tim setelah wawancara, misalnya untuk antarmuka yang memang dirancang untuk dipanggil lewat internet.

Listing 4.1 memperlihatkan dua fungsi intinya. Fungsi pertama menentukan apakah satu koneksi mengikat. Fungsi kedua mencari komponen terhubung dengan penelusuran melebar.

```
def koneksi_mengikat(koneksi):
  """Menentukan apakah satu koneksi memaksa kedua ujungnya pindah
      bersama.

  Koneksi tidak mengikat bila terlalu jarang, bila menuju pihak luar,
      atau
  bila tim sudah menandainya boleh dipisah, misalnya API yang memang
  dirancang untuk dipanggil lewat internet.
  """
  jumlah = int(koneksi["jumlah_koneksi_30_hari"])
  if jumlah < MINIMUM_KONEKSI_30_HARI:
    return False
  if koneksi["sumber"] in PIHAK_LUAR or koneksi["tujuan"] in PIHAK_LUAR
      :
    return False
  return koneksi["boleh_dipisah"].strip().lower() != "ya"


def cari_kelompok(tetangga):
  """Mencari komponen terhubung dengan penelusuran melebar.

  Setiap komponen adalah satu kelompok pindah. Nama diurutkan agar
      hasil
  setiap kali dijalankan sama dan mudah dibandingkan.
  """
  sudah_dikunjungi = set()
  daftar_kelompok = []
  for aplikasi_awal in sorted(tetangga):
```

```
    if aplikasi_awal in sudah_dikunjungi:
      continue
    kelompok = []
    antrean = deque([aplikasi_awal])
    sudah_dikunjungi.add(aplikasi_awal)
    while antrean:
      aplikasi = antrean.popleft()
      kelompok.append(aplikasi)
      for berikutnya in sorted(tetangga[aplikasi]):
        if berikutnya not in sudah_dikunjungi:
          sudah_dikunjungi.add(berikutnya)
          antrean.append(berikutnya)
    daftar_kelompok.append(sorted(kelompok))
  return daftar_kelompok
```

Listing 4.1: Penentu koneksi yang mengikat dan pencari kelompok pindah, tersedia di `sources/chapter04/kelompok-pindah.py`

Dengan data contoh, skrip menghasilkan satu kelompok berisi tujuh aplikasi: SIMRS Jakarta, aplikasi apotek, sistem laboratorium, folder bersama, ERP keuangan, direktori akun, dan portal dokumen akreditasi. Skrip malam yang hanya berjalan 30 kali per bulan menyatukan semuanya. Bila nilai `boleh_dipisah` pada baris skrip malam diubah menjadi `ya`, yaitu dengan anggapan skrip itu diganti lebih dulu, kelompoknya terpecah menjadi tiga aplikasi di sekitar SIMRS dan empat aplikasi di sekitar ERP. Hasil skrip adalah bahan diskusi, bukan keputusan. Skrip tidak tahu bahwa sistem laboratorium terikat kabel ke alat analisis.

Peringatan: log jaringan organisasi nyata hanya dipakai seizin pemiliknya, dengan nama dan alamat server yang dianonimkan. Alat *discovery* yang memindai jaringan hanya dijalankan pada jaringan sendiri atau yang sudah berizin tertulis.

4.5 Strategi Migrasi 7R

Setelah inventaris tersedia, setiap baris perlu diberi keputusan. Strategi migrasi adalah pendekatan yang dipakai untuk memindahkan satu *workload* ke *cloud*. AWS merangkumnya menjadi tujuh strategi yang dikenal sebagai 7R: Retire, Retain, Rehost, Relocate, Repurchase, Replatform, dan Refactor [9]. Daftar ini berkembang dari lima R yang dipopulerkan Gartner pada 2010. Tabel 4.4 merangkum arti dan kasus pemakaian umum tiap strategi. SaaS (*Software as a Service*) pada tabel adalah aplikasi siap pakai yang dijalankan penyedia dan dibayar sesuai langganan, seperti dibahas di Bab 2.

Analoginya kembali ke pindah rumah. Barang yang tidak terpakai dibuang (Retire). Barang yang belum bisa dibawa dititipkan di rumah lama (Retain). Lemari diangkut apa adanya (Rehost). Isi satu kamar dimasukkan ke kontainer dan

Tabel 4.4: Tujuh strategi migrasi dan kasus pemakaian umumnya, diringkas dari AWS Prescriptive Guidance

Strategi	Yang terjadi pada aplikasi	Kasus pemakaian umum
Retire	Dimatikan atau diarsipkan	Tidak bernilai bisnis; pemakaian prosesor dan memori rata-rata di bawah 5 persen; tanpa koneksi masuk 90 hari
Retain	Tetap di tempat asal untuk sementara	Aturan lokasi data, risiko tinggi, menunggu aplikasi lain pindah, baru diperbarui, menunggu versi SaaS, terikat perangkat khusus
Rehost	Dipindah tanpa perubahan (*lift and shift*)	Memindah banyak server dengan cepat; optimasi dilakukan sesudah berjalan di *cloud*
Relocate	Dipindah di tingkat platform, arsitektur tetap	Platform virtualisasi dipindah ke versi *cloud*-nya; pindah antarakun atau antar-*region*
Repurchase	Diganti produk lain (*drop and shop*)	Lisensi lama ke SaaS; versi baru dari vendor; aplikasi kustom diganti produk
Replatform	Dipindah dengan sebagian optimasi	Basis data ke layanan terkelola; sistem operasi diperbarui; mesin virtual ke kontainer tanpa mengubah kode
Refactor	Arsitektur dirancang ulang memakai fitur *cloud-native*	Monolit menghambat rilis; aplikasi lama tidak dapat dipelihara; tuntutan skala yang kuat

dipindah utuh tanpa dibongkar (Relocate). Kulkas tua dijual lalu dibeli yang baru di kota tujuan (Repurchase). Lemari diangkut sambil diganti engselnya supaya awet (Replatform). Dapur dirancang ulang dari nol mengikuti denah rumah baru (Refactor). Merancang ulang dapur memang paling memuaskan, tetapi juga paling lama dan paling mahal.

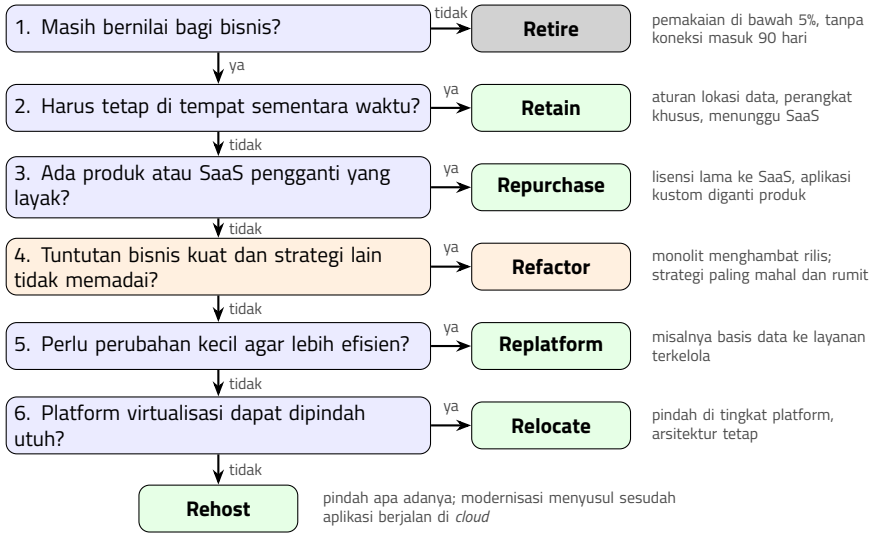
Tiga strategi sering tertukar: Rehost, Relocate, dan Replatform. Pembagiannya begini. Rehost memindah server satu per satu ke mesin virtual di *cloud* tanpa mengubah aplikasi. Relocate memindah sekumpulan server sekaligus di tingkat platform, misalnya dari platform virtualisasi *on-premise* ke versi *cloud* dari platform yang sama, atau memindah sumber daya ke akun lain, tanpa membeli perangkat keras baru dan tanpa mengubah operasi [9]. Replatform mengubah sebagian kecil cara aplikasi dijalankan, misalnya basis datanya dipindah ke layanan basis data terkelola, sementara kode aplikasinya tetap.

Pertanyaan kemudian muncul: mengapa tidak sekalian Refactor? AWS menyatakan bahwa Refactor tidak dianjurkan untuk migrasi berskala besar, karena modernisasi dilakukan selama migrasi. Strategi ini paling rumit dan paling mahal,

dan sulit dikelola untuk banyak aplikasi sekaligus. Anjurannya: aplikasi di-*rehost*, di-*relocate*, atau di-*replatform* lebih dulu, lalu dimodernisasi setelah migrasi selesai. Refactor dipakai hanya bila strategi lain tidak dapat diterima [9]. Prinsip ini sering disingkat "pindah dulu, modernisasi kemudian".

Keputusan ini punya dua konsekuensi. Pertama, ruang server lama berhenti lebih cepat, sehingga masa membayar dua lingkungan sekaligus menjadi pendek. Kedua, masalah rancangan aplikasi ikut pindah. Lima versi SIMRS tetap lima versi walaupun sudah berada di *cloud*. Karena itu, "kemudian" harus punya tanggal dan anggaran, bukan sekadar niat.

Gambar 4.5 menyusun pertanyaan pemilih strategi dalam urutan yang mencerminkan anjuran tersebut. Refactor sengaja diletakkan setelah Retire, Retain, dan Repurchase, dengan syarat ganda: tuntutan bisnisnya kuat dan strategi lain tidak memadai. Panduan lengkapnya ada di `sources/chapter04/panduan-keputusan-7r.md`.



Gambar 4.5: Urutan pertanyaan untuk usulan awal 7R. Jawaban pertama yang cocok menentukan usulan; keterangan di kanan diringkas dari kasus pemakaian AWS

Dua contoh kasus nyata:

1. Misalkan sebuah jaringan klinik menemukan 9 dari 60 aplikasinya tanpa koneksi masuk selama 90 hari. Dengan Retire, 9 server dimatikan sebelum migrasi dimulai, dan pekerjaan migrasi berkurang 15 persen tanpa memindahkan apa pun.
2. Misalkan sebuah rumah sakit memaksakan Refactor pada sistem antrean bersamaan dengan migrasi 25 aplikasi lain. Pengembang yang sama dibu-

tuhkan untuk peralihan aplikasi lain, sehingga proyek molor 11 bulan dan ruang server lama tetap disewa selama itu.

Kasus Medikara Health: Usulan 7R untuk Lima Aplikasi Utama

Tim memakai urutan pertanyaan pada Gambar 4.5 untuk aplikasi utama Medikara Health, ditambah tiga aplikasi lain sebagai pembanding. Hasilnya ada pada Tabel 4.5. Seluruh detail adalah skenario rekaan.

Tabel 4.5: Usulan 7R awal untuk delapan baris inventaris Medikara Health

Aplikasi	Usulan	Alasan satu kalimat
SIMRS, lima versi	Diperdebatkan	Retain lalu Repurchase ke satu versi sejalan dengan model *hybrid* Bab 2, atau Rehost kelima versi ke *region* Jakarta karena perangkatnya berumur 6 sampai 9 tahun
ERP keuangan	Replatform	Basis data yang mendekati batas kapasitas dipindah ke layanan terkelola, sedangkan aplikasinya tetap karena lisensi baru diperpanjang sampai 2028
Medikara+	Relocate	Dipindah dari akun *cloud* milik vendor ke *landing zone* Medikara agar akses vendor dapat dikendalikan; Refactor menyusul sesudahnya
Sistem laboratorium	Retain	Terhubung kabel ke alat analisis yang tidak punya padanan di *cloud*
Aplikasi apotek	Repurchase	Dukungan vendor berhenti akhir 2026 dan produk SaaS apotek sudah memenuhi kebutuhan stok 12 apotek
Klaster virtualisasi Jakarta, 14 aplikasi	Relocate	Dipindah utuh di tingkat platform tanpa mengubah satu pun aplikasinya
Portal dokumen akreditasi	Rehost	Aplikasi web sederhana di perangkat tua, dipindah apa adanya
Aplikasi antrean lama Jakarta	Retire	Sudah digantikan Medikara+ dan tanpa koneksi masuk selama 90 hari; datanya diarsipkan

Usulan untuk seluruh 46 aplikasi tersebar sebagai berikut: Retire 6, Retain 5, Rehost 10, Relocate 15, Repurchase 3, Replatform 2, Refactor 0, dan diperdebatkan 5 (kelima versi SIMRS). Tidak ada Refactor pada gelombang pertama. Medikara+ dijadwalkan untuk Refactor setelah berada di *landing zone* Medikara, dan rancangannya dibahas mulai Bab 8.

SIMRS sengaja ditandai diperdebatkan. Pilihan Retain lalu Repurchase sesuai dengan strategi "Satu Pasien, Satu Rekam Medis", tetapi bergantung pada kesiapan vendor dan pelatihan sekitar 2.900 staf klinis dan administrasi. Pilihan Rehost cepat mengurangi risiko perangkat tua, tetapi memin-

dahkan lima versi apa adanya dan membuat pelayanan bergantung pada sambungan internet setiap rumah sakit. Dependensi tersembunyi ke ERP dan aplikasi apotek ikut menentukan pilihan mana pun. Bab 7 mendiagnosis bentuk arsitekturnya, dan keputusan akhirnya ditetapkan pada peta jalan Bab 14.

4.6 Landing Zone dan Cloud Center of Excellence

Setelah usulan 7R tersedia, pertanyaan berikutnya: ke mana aplikasi itu mendarat, dan siapa yang menjaga aturannya? Tanpa fondasi bersama, setiap tim membuka akun sendiri, memakai aturan jaringan sendiri, dan mengulang kelemahan kontrol akses yang sama di tempat baru.

Landing zone adalah lingkungan *cloud* yang disiapkan sebelum *workload* pertama datang, sehingga setiap *workload* mendarat di atas fondasi yang sama. Microsoft menempatkannya di fase Ready dan menyusunnya dari beberapa area rancangan: identitas dan akses, organisasi sumber daya, jaringan, keamanan, pengelolaan, tata kelola, dan otomasi platform [97, 95]. Isinya pada umumnya enam hal: struktur akun atau langganan, identitas dan akses, jaringan bersama, kebijakan keamanan, log terpusat, serta tag dan anggaran biaya.

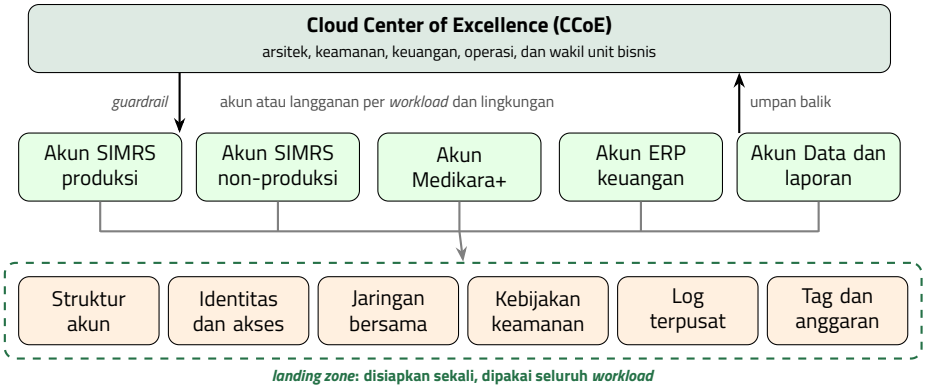
Analoginya seperti kawasan perumahan yang dikembangkan dengan benar. Sebelum rumah pertama dibangun, pengembang sudah memasang jalan, saluran air, listrik, pos satpam, dan aturan tinggi bangunan, sehingga pemilik kavling tinggal membangun rumahnya. Kawasan tanpa fondasi ini berakhir dengan kabel yang semrawut dan pagar yang berbeda di setiap rumah.

Gambar 4.6 memperlihatkan *landing zone* rancangan awal Medikara Health.

Dua contoh kasus nyata:

1. Misalkan sebelum ada *landing zone*, Medikara memiliki 9 akun *cloud* yang dibuka terpisah oleh tim rumah sakit dan vendor. Tiga di antaranya belum memakai autentikasi dua faktor, dan tagihan dua akun tidak dapat dialokasikan ke unit mana pun.
2. Misalkan setelah *landing zone* selesai dalam 8 minggu, akun baru untuk satu *workload* dapat disiapkan dalam 1 hari, lengkap dengan log terpusat dan tag biaya. Sebelumnya, persiapan serupa memakan sekitar 3 minggu dan hasilnya berbeda di setiap rumah sakit.

Landing zone tidak menjaga dirinya sendiri. Organisasi membutuhkan tim yang menetapkan standar, memelihara fondasi, dan membantu tim lain memakainya. Tim ini disebut CCoE (*Cloud Center of Excellence*), yaitu tim lintas fungsi yang



Gambar 4.6: *Landing zone* rancangan awal Medikara Health: fondasi bersama di bawah, akun per *workload* di tengah, dan CCoE yang menjaga standarnya

memandu adopsi *cloud* di seluruh organisasi. Anggotanya bukan hanya orang TI. Tabel 4.6 memuat susunan yang diusulkan untuk Medikara Health.

Tabel 4.6: Usulan susunan CCoE Medikara Health

Peran	Tanggung jawab di CCoE	Asal
Arsitek *cloud*	Rancangan *landing zone* dan pola arsitektur yang boleh dipakai	Tim TI kantor pusat
Wakil keamanan	Kebijakan akses, kebijakan enkripsi, akses vendor	Unit keamanan informasi
Wakil keuangan	Anggaran, tag biaya, laporan biaya per layanan (lihat Bab 3)	Direktorat keuangan
Wakil operasi	Pemantauan, prosedur insiden yang seragam	Tim TI rumah sakit, bergiliran
Wakil unit bisnis	Prioritas, penerimaan pengguna, rencana pelatihan	Direktorat pelayanan medis

CCoE sudah lazim di organisasi besar. Laporan Flexera 2020 mencatat 73 persen perusahaan besar sudah memiliki tim *cloud* pusat atau CCoE [54]. Angka survei yang lebih baru perlu dicek pada laporan Flexera terbaru sebelum dikutip.

Dua hal pada CCoE sering tertukar. Pembagiannya begini. CCoE bukan divisi TI baru yang mengerjakan seluruh migrasi. CCoE menetapkan standar, menyediakan fondasi, dan mengajari tim lain, lalu tim aplikasi yang mengerjakan migrasinya sendiri. Dalam istilah Team Topologies, peran ini dekat dengan gabungan tim platform dan *enabling team* [131]. CCoE yang mengerjakan semuanya sendiri akan menjadi antrean baru. Aturan yang dijaga CCoE, misalnya larangan membuka akun tanpa tag biaya, disebut *guardrail*, dan tata kelolanya dibahas lebih lanjut di Bab 14.

4.7 Aktivitas: Penilaian Kesiapan dan Portofolio 7R

Tujuan. Tim menghasilkan penilaian kesiapan *cloud* enam perspektif dan portofolio aplikasi dengan usulan 7R yang beralasan, untuk organisasi capstone atau untuk Medikara Health.

Pembagian peran. Tim campuran membagi kepemimpinan perspektif sesuai latar belakang. Usulan 7R dibahas oleh seluruh anggota.

- Anggota non-teknis memimpin perspektif Business, People, dan Governance, serta mewawancarai pemilik bisnis aplikasi.
- Anggota teknis memimpin perspektif Platform, Security, dan Operations, serta mencari dependensi dari konfigurasi dan log.
- Satu orang menjadi penjaga inventaris agar templat terisi seragam.
- Moderator diskusi 7R bergiliran per aplikasi dan memastikan setiap usulan punya alasan satu kalimat.

Langkah kerja.

1. Isi `sources/chapter04/lembar-penilaian-kesiapan-caf.xlsx`. Setiap skor disertai bukti dan nama pemilik jawaban. Penilai tidak menilai prosesnya sendiri.
2. Tetapkan batas minimal tiap perspektif, lalu catat tiga celah terbesar beserta rencana penutupnya.
3. Inventaris aplikasi organisasi di `sources/chapter04/templat-inventaris-aplikasi.xlsx`: pemilik, pengguna, kritikalitas, teknologi, kontrak, dependensi, dan kondisinya.
4. Beri usulan 7R awal untuk setiap aplikasi dengan alasan satu kalimat, memakai `sources/chapter04/panduan-keputusan-7r.md`.
5. Tandai aplikasi yang usulannya masih diperdebatkan, lalu tulis dua pilihan yang bersaing dan asumsi yang paling menentukan.

Luaran dan kaitan capstone. Luarannya adalah penilaian kesiapan *cloud* enam perspektif dan portofolio aplikasi dengan usulan 7R. Keduanya masuk ke tonggak C2 Kondisi Saat Ini, yang ditinjau menjelang Ujian Tengah Semester (UTS). Daftar aplikasi yang diperdebatkan dibawa ke Bab 7 sebagai bahan diagnosis arsitektur.

Kriteria penilaian. Tabel 4.7 memuat kriteria singkatnya.

Catatan etika. Data organisasi nyata dipakai hanya seizin organisasi dan dianonimkan. Nama aplikasi dan server boleh disamarkan. Data pribadi pasien atau karyawan tidak boleh masuk ke lembar penilaian maupun inventaris, termasuk dalam tangkapan layar.

Tabel 4.7: Kriteria penilaian Aktivitas Bab 4

Kriteria	Bobot	Indikator
Bukti penilaian kesiapan	25%	Setiap skor punya bukti dan pemilik jawaban
Kelengkapan inventaris	25%	Atribut minimal terisi; dependensi tidak hanya dari ingatan
Alasan usulan 7R	30%	Alasan menyebut fakta dari inventaris, bukan selera
Kejujuran tentang ketidakpastian	10%	Usulan yang diperdebatkan ditandai beserta dua pilihannya
Etika dan anonimisasi	10%	Izin tersedia; tidak ada data pribadi

Pendalaman Teknis (opsional): Diagram Dependensi dari Data Integrasi

Anggota berlatar teknis dapat menggambar diagram dependensi antaraplikasi dari data integrasi, misalnya daftar antarmuka, jadwal tugas, atau catatan koneksi yang sudah dianonimkan. Data disusun dalam format yang sama dengan `sources/chapter04/contoh-koneksi-jaringan.csv`, lalu diolah dengan `sources/chapter04/kelompok-pindah.py`. Hasilnya dipakai untuk menandai kelompok aplikasi yang harus dipindahkan bersamaan, lalu dibandingkan dengan usulan 7R. Kelompok yang anggotanya mendapat usulan berbeda, misalnya satu Retain dan satu Rehost, ditulis sebagai risiko dan dibawa ke diskusi tim.

4.8 Ringkasan

Medikara Health tidak kekurangan server untuk dipindah. Yang kurang adalah gambaran lengkap tentang aplikasi yang dimilikinya dan kesiapan organisasinya untuk berubah. Bab ini menjawab dua kekurangan itu secara berurutan: kesiapan dulu, lalu portofolio.

Tiga penyedia besar menyusun pengalaman migrasi mereka menjadi kerangka adopsi. AWS CAF memakai enam perspektif yang memetakan pemangku kepentingan. Microsoft CAF memakai tujuh fase yang mudah dijadikan paket kerja, ditambah skenario kedaulatan data. Google memakai empat tema dengan tiga fase kematangan. Ketiganya sepakat bahwa adopsi *cloud* adalah urusan orang, proses, dan tata kelola, bukan hanya teknologi.

Penilaian kesiapan enam perspektif dibaca dari perspektif terlemah, bukan dari rata-ratanya. Skor tanpa bukti dicatat 1, dan penilai bukan pemilik proses. Pada kasus Medikara, rata-rata 2,3 menyembunyikan skor Security 1,3 yang lahir dari akun mantan karyawan dan akses vendor. Akibatnya, gelombang migrasi pertama ditunda sampai celah itu ditutup.

Inventaris portofolio memuat pemilik, pengguna, kritikalitas, teknologi, kon-

trak, dependensi, dan kondisi setiap aplikasi. Dependensi tersembunyi, misalnya skrip malam yang menyalin data ke folder bersama, dapat menyatukan dua kelompok pindah yang semula dianggap terpisah. Dari inventaris itulah usulan 7R disusun: Retire, Retain, Rehost, Relocate, Repurchase, Replatform, dan Refactor. Untuk migrasi besar, Refactor dipakai hanya bila strategi lain tidak dapat diterima, dan modernisasi dijadwalkan sesudah aplikasi pindah.

Pesan utama bab ini: migrasi yang baik tidak dimulai dari memindahkan server, tetapi dari mengenali organisasi dan portofolionya dengan jujur. *Landing zone* menyediakan fondasi yang sama untuk setiap *workload*, dan CCoE lintas fungsi menjaga standarnya tanpa mengambil alih pekerjaan tim lain. Penilaian kesiapan dan portofolio 7R yang disusun tim pada Aktivitas bab ini menjadi bagian pertama tonggak C2.

4.9 Latihan

Latihan 1: Mengapa Refactor Tidak Dipakai untuk Sebagian Besar Aplikasi

Latihan ini membuktikan, dengan angka, mengapa panduan AWS menyarankan agar strategi Refactor tidak dipakai untuk sebagian besar aplikasi dalam migrasi berskala besar.

1. Ambil jumlah aplikasi yang akan dipindah dari portofolio tim atau dari Medikara Health.
2. Susun dua skenario: A, pindah dulu lalu modernisasi; B, sebagian aplikasi di-*refactor* selama migrasi.
3. Perkirakan lama migrasi, lama berjalan ganda antara ruang server lama dan *cloud*, biaya ruang server lama selama berjalan ganda, jumlah pengembang yang terikat, dan jumlah peralihan berisiko tinggi.
4. Bandingkan kedua skenario dan tulis kesimpulan satu paragraf.

Tabel 4.8 memuat contoh untuk 30 aplikasi Medikara Health yang akan dipindah, dengan biaya ruang server lama Rp85 juta per bulan. Tabel 4.9 adalah lembar isian.

Pertanyaan analisis:

1. Pada tabel isian, komponen mana yang paling membesar dari skenario A ke B? Apa hubungannya dengan alasan AWS bahwa Refactor sulit dikelola untuk banyak aplikasi?
2. Bila skenario A dipilih, berapa lama dan berapa biaya yang harus disediakan untuk modernisasi sesudahnya agar "kemudian" tidak berubah menjadi

Tabel 4.8: Latihan 1, contoh terisi: 30 aplikasi Medikara Health (angka rekaan)

Komponen	A: pindah dulu	B: Refactor selama migrasi
Aplikasi yang di-*refactor* selama migrasi	0	8
Perkiraan lama migrasi	14 bulan	30 bulan
Bulan berjalan ganda	8 bulan	22 bulan
Biaya ruang server lama selama berjalan ganda	Rp680 juta	Rp1,87 miliar
Pengembang yang terikat proyek migrasi	6 orang	20 orang
Peralihan berisiko tinggi	5	13

Tabel 4.9: Latihan 1, lembar isian

Komponen	A: pindah dulu	B: Refactor selama migrasi
Aplikasi yang di-*refactor* selama migrasi	...	...
Perkiraan lama migrasi	...	...
Bulan berjalan ganda	...	...
Biaya ruang server lama selama berjalan ganda	...	...
Pengembang yang terikat proyek migrasi	...	...
Peralihan berisiko tinggi	...	...

"tidak pernah"?

Latihan 2: Usulan 7R untuk SIMRS

Latihan ini membuktikan bahwa usulan 7R untuk aplikasi yang diperdebatkan harus dapat dipertahankan terhadap alternatifnya dengan kriteria yang disepakati sebelum skor diisi.

1. Baca kembali kondisi SIMRS Medikara Health: lima versi di lima rumah sakit, perangkat berumur 6 sampai 9 tahun, model *hybrid* dari Bab 2, dan dependensi tersembunyi pada Gambar 4.4.
2. Pilih satu usulan 7R dan satu alternatif terkuat.
3. Tetapkan kriteria dan bobotnya lebih dulu, lalu beri skor 1 sampai 5 untuk kedua pilihan.
4. Hitung skor berbobot, lalu tulis asumsi yang paling memengaruhi hasil.

Tabel 4.10 memuat contoh cara membandingkan untuk aplikasi apotek, bukan untuk SIMRS. Tabel 4.11 adalah lembar isian untuk SIMRS.

Pertanyaan analisis:

1. Pada contoh apotek, Rehost unggul pada tiga kriteria tetapi kalah secara

Tabel 4.10: Latihan 2, contoh terisi: aplikasi apotek, Repurchase dibandingkan Rehost

Kriteria	Bobot	Repurchase	Rehost
Kesesuaian dengan "Satu Pasien, Satu Rekam Medis"	30%	5	2
Waktu sampai manfaat terasa	20%	4	5
Biaya lima tahun	20%	4	3
Risiko layanan saat peralihan	20%	3	4
Beban perubahan bagi pengguna	10%	2	5
Skor berbobot	100%	**3,9**	**3,5**

Tabel 4.11: Latihan 2, lembar isian: SIMRS

Kriteria	Bobot	Usulan: ...	Alternatif: ...
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...
Skor berbobot	100%	...	...

total. Kriteria mana yang membuat Repurchase menang, dan berapa bobot kriteria itu agar hasilnya berbalik?

2. Pada lembar SIMRS, asumsi mana yang bila keliru akan membalik hasil? Siapa di Medikara yang dapat memastikan asumsi itu?

Latihan 3: Perspektif yang Diabaikan dalam Migrasi Digerakkan TI

Latihan ini membuktikan bahwa migrasi yang hanya digerakkan divisi TI meninggalkan perspektif tertentu, dan akibatnya dapat diukur.

1. Pilih satu proyek migrasi atau pengadaan sistem yang pernah dijalankan di tempat kerja, atau pakai contoh Medikara Health di bawah.
2. Untuk setiap perspektif, catat apakah pemangku kepentingannya dilibatkan sejak awal.
3. Catat gejala yang muncul dan akibatnya dalam angka, misalnya tiket bantuan, rupiah, atau minggu keterlambatan.
4. Tentukan perspektif yang paling diabaikan dan jelaskan akibatnya.

Tabel 4.12 memuat contoh pemindahan portal dokumen akreditasi Medikara Health yang dikerjakan tim TI sendirian. Tabel 4.13 adalah lembar isian.

Pertanyaan analisis:

1. Pada tabel isian, perspektif mana yang dilibatkan dan mana yang tidak?

Tabel 4.12: Latihan 3, contoh terisi: pemindahan portal dokumen akreditasi (angka rekaan)

Perspektif	Dilibatkan	Gejala	Akibat terukur
Business	Tidak	Tidak ada pemilik manfaat	Setelah 4 bulan, direksi menilai hasil proyek tidak jelas
People	Tidak	Staf tidak diberi tahu alamat baru dan cara masuk	212 tiket bantuan pada pekan pertama dari sekitar 600 pengguna
Governance	Sebagian	Tagihan masuk anggaran TI tanpa tag	Rp6 juta per bulan tidak dapat dialokasikan
Platform	Ya	Server berjalan sesuai rencana	Selesai dalam 3 minggu
Security	Sebagian	Akun administrator bersama ikut dipindah	Temuan audit yang sama muncul lagi
Operations	Ya	Pemantauan dipasang	Ketersediaan 99,8 persen pada bulan pertama

Tabel 4.13: Latihan 3, lembar isian

Perspektif	Dilibatkan	Gejala	Akibat terukur
Business	…	…	…
People	…	…	…
Governance	…	…	…
Platform	…	…	…
Security	…	…	…
Operations	…	…	…

Apakah polanya sama dengan contoh, yaitu teknis dilibatkan dan bisnis ditinggal?

2. Akibat terukur mana yang paling mahal bagi organisasi? Bandingkan dengan biaya melibatkan pemangku kepentingan perspektif itu sejak awal.

Bab 5

Prinsip Cloud-Native dan Twelve-Factor App

Tujuan Pembelajaran

- Menjelaskan definisi *cloud-native* menurut CNCF (Cloud Native Computing Foundation) dan perbedaannya dengan aplikasi yang sekadar *cloud-hosted*.
- Menerjemahkan dua belas faktor Twelve-Factor App ke dalam dampak bisnis yang mudah dipahami.
- Menilai kesenjangan aplikasi yang ada terhadap prinsip *cloud-native*.

5.1 Pendahuluan

Aplikasi pasien Medikara+ sudah berjalan di *cloud* sejak 2023. Vendor pembuatnya menyewa dua mesin virtual di sebuah penyedia *cloud*, lalu memasang Medikara+ di sana. Di atas kertas, masalah kapasitas seharusnya selesai. Kenyataannya lain. Setiap kali program vaksinasi diumumkan, Medikara+ melambat sampai banyak pemesanan gagal. Satu pembaruan kecil, misalnya perubahan teks pengingat jadwal, tetap butuh dua minggu untuk sampai ke ponsel pasien. Direksi bertanya dengan wajar: bukankah aplikasi ini sudah di *cloud*?

Jawabannya, berada di *cloud* belum tentu berarti *cloud-native*. Lokasi server berubah, tetapi cara aplikasi dibangun, dikonfigurasi, dijalankan, dan ditambah kapasitasnya masih sama dengan cara lama. Bab 1 sudah membedakan *cloud-hosted* dan *cloud-native* secara garis besar, dan Bab 4 sudah memberi label strategi migrasi untuk setiap aplikasi. Bab ini menyediakan alat ukurnya: definisi resmi untuk menilai arah, dan daftar periksa untuk menemukan titik yang menghambat.

Hasilnya adalah kartu skor kesenjangan *cloud-native* untuk tonggak capstone C2.

Pembahasan dimulai dari definisi *cloud-native* versi 1.1 yang disetujui Cloud Native Computing Foundation (CNCF) pada 26 Februari 2024 [38]. Setelah itu, dua belas faktor Twelve-Factor App [152] dibahas dalam tiga kelompok, lengkap dengan statusnya sebagai proyek *open source* sejak November 2024 [81] dan perluasannya menjadi lima belas faktor oleh Hoffman [72]. Bab ditutup dengan *immutable infrastructure* dan konfigurasi deklaratif [104, 137], dua gagasan yang menjadi jembatan ke pembahasan *container* pada Bab 8 dan GitOps pada Bab 13.

5.2 Definisi Cloud-Native menurut CNCF

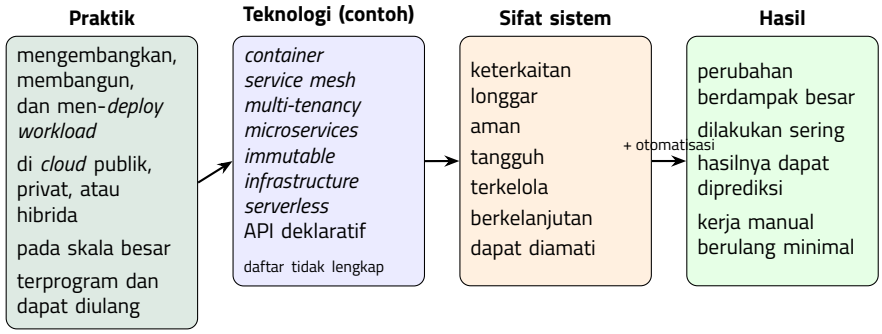
CNCF adalah yayasan di bawah Linux Foundation yang menaungi banyak proyek *open source*, termasuk Kubernetes. Definisi *cloud-native* versi 1.1 disetujui oleh komite teknis (*Technical Oversight Committee*) dan dewan pengurus (*Governing Board*) CNCF pada 26 Februari 2024 [38]. Isinya terdiri atas tiga bagian. Bagian pertama menyatakan bahwa praktik *cloud-native* memungkinkan organisasi mengembangkan, membangun, dan men-*deploy workload* di lingkungan komputasi publik, privat, maupun hibrida, untuk memenuhi kebutuhannya pada skala besar, dengan cara yang terprogram dan dapat diulang. Cirinya adalah sistem yang keterkaitannya longgar dan saling bekerja sama secara aman, tangguh, terkelola, berkelanjutan, dan dapat diamati. Bagian kedua menyebut teknologi yang biasanya dipakai: *container*, *service mesh*, *multi-tenancy*, *microservices*, *immutable infrastructure*, *serverless*, dan API (*Application Programming Interface*) deklaratif. Definisi itu menegaskan bahwa daftar ini tidak lengkap. Bagian ketiga menyatakan hasilnya: bersama otomatisasi yang andal, teknologi tersebut memungkinkan perekayasa melakukan perubahan berdampak besar secara sering dan dapat diprediksi, dengan kerja manual yang minimal dan pembagian tanggung jawab yang jelas [38].

Gambar 5.1 membaca definisi itu dari kiri ke kanan. Praktik menjadi titik awal, teknologi hanya contoh, sifat sistem menjadi sasaran, dan hasilnya adalah kemampuan berubah dengan cepat tanpa takut.

Analoginya adalah standar pelayanan loket di sebuah rumah sakit. Standar yang baik menyebut sifat layanan yang dituju: antrean tertib, data pasien aman, waktu tunggu terukur, dan keluhan dapat ditelusuri. Standar itu tidak menyebut merek mesin antrean atau merek komputer loket. Merek boleh berganti selama sifatnya terpenuhi. Definisi CNCF bekerja dengan cara yang sama. Kubernetes, *container*, dan *serverless* adalah contoh alat, sedangkan yang dinilai adalah praktik dan sifat sistemnya.

Dua contoh cara membaca definisi ini:

1. Sebuah organisasi memakai Kubernetes untuk menjalankan satu aplikasi



Gambar 5.1: Susunan definisi *cloud-native* CNCF versi 1.1: praktik, contoh teknologi, sifat sistem yang dituju, dan hasil yang diharapkan

lama apa adanya. Aplikasi itu tetap dirilis empat kali setahun lewat 60 langkah manual. Alatnya *cloud-native*, tetapi hasilnya belum: perubahan tidak sering dan tidak dapat diprediksi.

2. Misalkan sebuah bank daerah menjalankan sistemnya di pusat data sendiri, tetapi pemasangannya sudah otomatis dan setiap perubahan dapat dirilis dalam 30 menit. Sistem ini lebih dekat ke *cloud-native* daripada contoh pertama, walaupun tidak berada di *cloud* publik, karena definisi CNCF juga mencakup lingkungan privat.

5.2.1 Perubahan dari Versi 1.0 ke Versi 1.1

Definisi versi 1.0 dari 2018 tersimpan di riwayat berkas yang sama [38]. Perbandingan keduanya menunjukkan pergeseran cara pandang, seperti dirangkum Tabel 5.1.

Tabel 5.1: Perbedaan definisi *cloud-native* CNCF versi 1.0 dan versi 1.1

Aspek	**Versi 1.0 (2018)**	**Versi 1.1 (2024)**
Pokok kalimat	Teknologi *cloud-native*	Praktik *cloud-native*
Yang dibangun	Aplikasi yang dapat diskalakan	*Workload*, pada skala besar, secara terprogram dan dapat diulang
Sifat sistem	Keterkaitan longgar, tangguh, terkelola, dapat diamati	Ditambah aman dan berkelanjutan, serta saling bekerja sama
Contoh teknologi	*Container*, *service mesh*, *microservices*, *immutable infrastructure*, API deklaratif	Ditambah *multi-tenancy* dan *serverless*, dengan penegasan bahwa daftarnya tidak lengkap
Hasil	Perubahan berdampak besar, sering, dapat diprediksi, kerja manual minimal	Sama, ditambah pembagian tanggung jawab yang jelas

Pergeseran dari "teknologi" ke "praktik" punya dua konsekuensi. Pertama,

membeli alat tidak otomatis membuat organisasi menjadi *cloud-native*. Cara tim membangun dan mengubah sistem ikut dinilai. Kedua, sifat "aman" dan "berkelanjutan" kini menjadi bagian definisi. Keamanan tidak lagi dianggap urusan tambahan sesudah sistem jadi, dan pemakaian energi serta jejak karbon yang dibahas pada Bab 3 ikut menjadi ukuran mutu.

5.2.2 Enam Sifat Sistem yang Dituju

Sifat sistem dalam definisi CNCF sering terdengar abstrak bagi pembaca non-teknis. Tabel 5.2 menerjemahkannya ke bahasa sehari-hari, lengkap dengan pertanyaan yang dapat diajukan kepada tim TI (teknologi informasi) tentang aplikasi yang sedang dinilai.

Tabel 5.2: Enam sifat sistem dalam definisi CNCF versi 1.1 dan pertanyaan pengujinya

Sifat	Arti sehari-hari	Pertanyaan penguji
Keterkaitan longgar	Satu bagian dapat diubah tanpa membongkar bagian lain	Bila modul janji temu diubah, apakah modul tagihan harus ikut diuji ulang?
Aman	Akses dibatasi dan diperiksa di setiap pertemuan antarbagian	Siapa saja yang dapat membaca kunci akses BPJS Kesehatan?
Tangguh	Gangguan pada satu bagian tidak menjatuhkan semuanya	Bila satu server mati, berapa pasien yang ikut terputus?
Terkelola	Perubahan dan perawatan dilakukan lewat cara yang baku	Dapatkah satu pengaturan diubah tanpa memasang ulang aplikasi?
Berkelanjutan	Sumber daya dipakai sesuai kebutuhan, termasuk energi	Apakah kapasitas turun lagi setelah jam sibuk berlalu?
Dapat diamati	Keadaan sistem dapat diketahui dari luar	Berapa menit yang dibutuhkan untuk mengetahui penyebab gangguan terakhir?

Bagian hasil dalam definisi CNCF menyebut kerja manual yang minimal. Istilah aslinya *toil*. Buku *Site Reliability Engineering* dari Google mendefinisikan *toil* sebagai pekerjaan manual yang berulang, dapat diotomatisasi, bersifat taktis, tidak meninggalkan nilai jangka panjang, dan bertambah seiring layanan tumbuh [27]. Daftar 37 langkah manual yang dijalankan vendor Medikara+ setiap kali memasang versi baru adalah contoh *toil*. Jumlah langkahnya tetap, tetapi frekuensinya akan naik bila organisasi ingin rilis lebih sering.

5.2.3 Cloud-Hosted dan Cloud-Native

Dua hal pada istilah ini sering tertukar. Pembagiannya begini. Aplikasi *cloud-hosted* menumpang di infrastruktur *cloud*, tetapi rancangan dan cara kerjanya tidak berubah. Aplikasi *cloud-native* memenuhi sifat yang dituju definisi CNCF. Ukurannya bukan lokasi, melainkan sifat. Kedua bentuk itu adalah ujung sebuah

rentang, bukan dua kotak terpisah, dan sebagian besar aplikasi organisasi berada di tengahnya.

Dua contoh di Medikara Health:

1. Medikara+ berjalan di dua mesin virtual di *cloud* milik vendor. Kapasitas ditambah dengan mengganti mesin menjadi lebih besar, butuh sekitar 40 menit, dan seluruh pengguna terputus. Pembaruan butuh dua minggu. Aplikasi ini *cloud-hosted*.
2. Enterprise Resource Planning (ERP) keuangan dipakai sekitar 120 staf keuangan pada jam kerja, bebannya stabil, dan perubahannya jarang. Pertanyaan yang tepat untuk aplikasi ini adalah sifat mana yang benar-benar dibutuhkan, misalnya aman dan dapat diamati, bukan seluruh daftar.

5.3 Twelve-Factor App dan Statusnya Sekarang

Definisi CNCF menyebut sifat yang dituju, tetapi tidak menyebut cara memeriksa sebuah aplikasi satu per satu. Pertanyaan kemudian muncul: daftar periksa apa yang dapat dipakai? Jawaban yang paling luas dikenal adalah Twelve-Factor App.

Twelve-Factor App adalah metodologi untuk membangun aplikasi yang dipakai lewat jaringan sebagai layanan, atau *Software as a Service* (SaaS). Metodologi ini ditulis oleh Adam Wiggins, salah satu pendiri platform Heroku, dari pengalaman menjalankan ratusan ribu aplikasi di platform itu. Versi pertamanya terbit pada 2011, dan teks yang dibaca sekarang terakhir diperbarui pada 2017 [152]. Dua belas faktornya bertujuan agar aplikasi mudah disiapkan lewat otomatisasi, mudah dipindahkan antarlingkungan, cocok dipasang di platform *cloud*, punya perbedaan sekecil mungkin antara lingkungan pengembangan dan produksi, serta dapat diskalakan tanpa perubahan besar pada rancangan [152].

Statusnya berubah pada 12 November 2024. Heroku mengumumkan bahwa Twelve-Factor App kini menjadi proyek *open source* yang dikelola komunitas, dengan repositori di `github.com/twelve-factor/twelve-factor` yang memuat dokumen visi, daftar pengelola, dan pedoman kontribusi. Wiggins menyebutnya dokumen yang dikelola komunitas dan dapat berkembang dari waktu ke waktu [81]. Konsekuensinya bagi penilai: bunyi faktor dapat berubah pada tahun-tahun mendatang, sehingga kartu skor sebaiknya mencatat tanggal akses dan versi teks faktor yang dipakai.

Twelve-Factor App juga bukan satu-satunya daftar. Hoffman memperluasnya menjadi lima belas faktor dengan menambahkan API yang dirancang lebih dahulu (*API first*), telemetri, serta autentikasi dan otorisasi [72]. Tambahan terakhir relevan bagi Medikara Health, mengingat temuan audit internal tentang kontrol akses yang lemah.

5.3.1 Alat Penilaian, Bukan Dogma

Dua belas faktor paling berguna bila dipakai sebagai alat penilaian. Setiap faktor memunculkan pertanyaan tentang aplikasi, dan jawabannya menunjukkan kesenjangan beserta dampaknya. Faktor bukan perintah yang wajib dipenuhi seluruhnya. Faktor itu ditulis untuk aplikasi SaaS yang terus berubah dan dipakai banyak orang, dan tidak semua aplikasi organisasi berbentuk seperti itu.

Analoginya adalah pemeriksaan kesehatan berkala. Hasil laboratorium berisi belasan angka: gula darah, kolesterol, fungsi hati, dan seterusnya. Dokter tidak menuntut semua angka sempurna. Dokter membaca angka mana yang menyimpang, menilai seberapa berbahaya penyimpangan itu bagi usia dan pekerjaan pasien, lalu memilih dua atau tiga hal yang perlu ditangani lebih dahulu. Kartu skor *twelve-factor* dibaca dengan cara yang sama.

Dua contoh kasus:

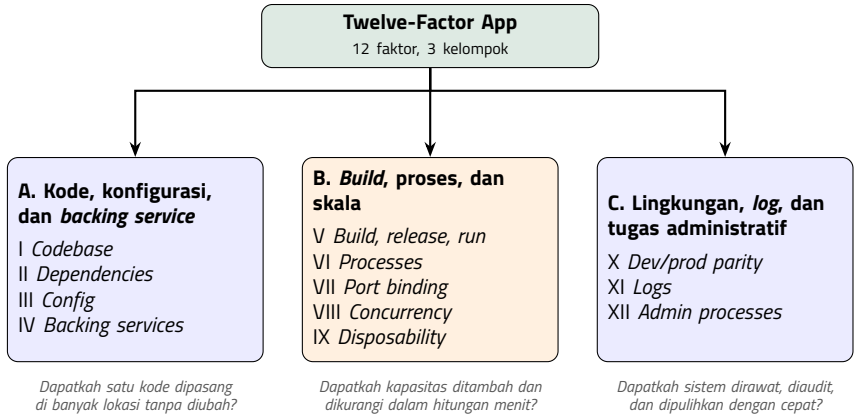
1. Medikara+ dipakai sekitar 310.000 pengguna terdaftar, bebannya naik turun tajam, dan pembaruannya diharapkan terjadi setiap pekan. Hampir semua faktor relevan, sehingga kesenjangan pada faktor mana pun berdampak langsung pada pasien.
2. Aplikasi rekap lembur bulanan di bagian sumber daya manusia dipakai 12 orang dan dijalankan sekali sebulan. Faktor tentang penskalaan hampir tidak relevan. Faktor tentang konfigurasi dan *log* tetap penting, karena aplikasi itu menyimpan data karyawan dan perlu dapat diaudit.

5.3.2 Tiga Kelompok Faktor

Dua belas faktor terlalu banyak untuk diingat sekaligus oleh pembaca non-teknis. Bab ini mengelompokkannya menjadi tiga, dan setiap kelompok diwakili satu pertanyaan bisnis. Gambar 5.2 memperlihatkan pengelompokan itu, dan Tabel 5.3 memuat bunyi singkat setiap faktor beserta terjemahan bebasnya. Pengelompokan ini adalah alat bantu bab ini, bukan bagian dari teks resmi Twelve-Factor App.

5.4 Faktor tentang Kode, Konfigurasi, dan Backing Service

Kelompok A menjawab satu pertanyaan bisnis: dapatkah satu kode dipasang di banyak lokasi tanpa diubah? Bagi Medikara Health yang punya lima rumah sakit, 18 klinik, dan satu laboratorium rujukan, pertanyaan ini menentukan berapa kali satu perubahan harus dikerjakan.



Gambar 5.2: Dua belas faktor dalam tiga kelompok, masing-masing dengan satu pertanyaan bisnis

Tabel 5.3: Dua belas faktor Twelve-Factor App dan terjemahan bebasnya, diolah dari [152]

No.	Faktor	Bunyi singkat (terjemahan bebas)	Kel.
I	*Codebase*	Satu basis kode dalam sistem kendali versi, dipasang di banyak tempat	A
II	*Dependencies*	Ketergantungan dinyatakan dan diisolasi secara eksplisit	A
III	*Config*	Konfigurasi disimpan di lingkungan, bukan di kode	A
IV	*Backing services*	Layanan pendukung diperlakukan sebagai sumber daya yang dapat dipasang dan dilepas	A
V	*Build, release, run*	Tahap *build* dan tahap menjalankan dipisahkan dengan tegas	B
VI	*Processes*	Aplikasi dijalankan sebagai satu atau beberapa proses tanpa *state*	B
VII	*Port binding*	Layanan disediakan lewat *port* milik aplikasi sendiri	B
VIII	*Concurrency*	Kapasitas ditambah dengan menambah proses	B
IX	*Disposability*	Proses cepat menyala dan berhenti dengan rapi	B
X	*Dev/prod parity*	Lingkungan pengembangan, uji, dan produksi dibuat semirip mungkin	C
XI	*Logs*	*Log* diperlakukan sebagai aliran kejadian	C
XII	*Admin processes*	Tugas administratif dijalankan sebagai proses sekali jalan	C

5.4.1 Codebase dan Dependencies

Faktor I menyatakan bahwa satu aplikasi punya tepat satu basis kode (*codebase*) yang dicatat dalam sistem kendali versi, lalu dipasang di banyak tempat. Setiap tempat pemasangan disebut *deploy*: lingkungan uji, lingkungan produksi, atau lokasi cabang. Bila ada beberapa basis kode, yang ada bukan lagi satu aplikasi. Bila beberapa aplikasi berbagi kode yang sama, kode itu seharusnya dipisahkan menjadi pustaka yang dipakai bersama [152]. Faktor II menyatakan bahwa setiap

ketergantungan, misalnya pustaka dan versinya, ditulis secara eksplisit dan diisolasi. Aplikasi tidak boleh bergantung pada perangkat lunak yang kebetulan sudah terpasang di server [152].

Dua faktor ini paling mudah dipahami lewat dampaknya. Bila lima rumah sakit masing-masing menyimpan salinan kode sendiri, satu perbaikan harus disalin lima kali, dan setiap salinan dapat menyimpang sedikit demi sedikit. Bila ketergantungan tidak tertulis, server baru hanya dapat disiapkan oleh orang yang ingat urutan pemasangannya. Pengetahuan itu hilang ketika orangnya pindah kerja.

5.4.2 Config: Konfigurasi di Lingkungan

Faktor III mendefinisikan konfigurasi sebagai segala sesuatu yang berbeda antar-*deploy*. Isinya tiga jenis: alamat dan kredensial basis data serta layanan pendukung lain, kredensial ke layanan luar, dan nilai khas setiap lokasi, misalnya kode fasilitas [152]. Pengaturan internal yang sama di semua lokasi, misalnya urutan menu, bukan konfigurasi dalam arti ini. Faktor III meminta konfigurasi dipisahkan tegas dari kode dan disimpan di lingkungan tempat aplikasi berjalan, biasanya sebagai *environment variable*. Twelve-Factor App memberi uji sederhana: bila basis kode dibuka ke publik hari ini, apakah ada kredensial yang ikut bocor? Bila ya, konfigurasi belum terpisah dari kode [152].

Analoginya adalah prosedur standar yang dipakai di lima cabang. Badan prosedur berlaku sama untuk semua cabang, sedangkan nomor telepon darurat dan nama kepala ruang ditulis di lembar lampiran setiap cabang. Bila nomor telepon ditulis di dalam badan prosedur, setiap pergantian kepala ruang memaksa seluruh prosedur dicetak ulang dan disahkan ulang. Akibatnya lima cabang lama-kelamaan memegang lima versi prosedur yang berbeda.

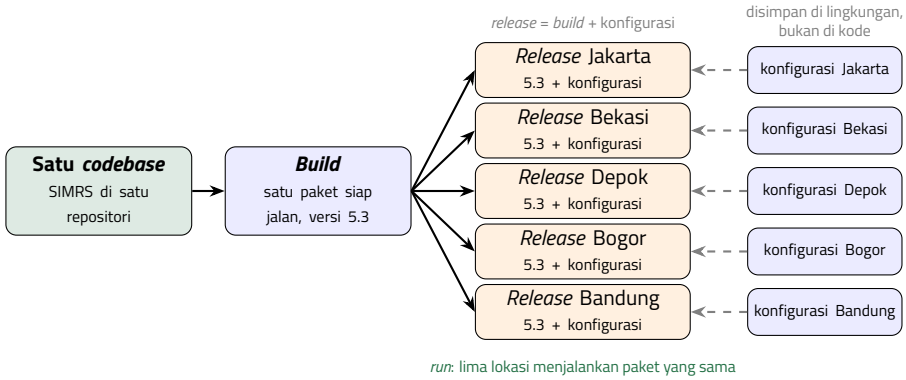
Dua contoh kasus:

1. Alamat layanan integrasi BPJS (Badan Penyelenggara Jaminan Sosial) Kesehatan tertulis di dalam kode SIMRS (Sistem Informasi Manajemen Rumah Sakit) di lima rumah sakit. Ketika alamat itu berubah, setiap rumah sakit harus mengubah kode, membangun ulang, menguji, dan memasang ulang. Dengan dua hari kerja per rumah sakit, totalnya 10 hari kerja atau dua minggu.
2. Kata sandi basis data tertulis di berkas kode yang disalin ke laptop tiga vendor berbeda. Siapa pun yang memegang salinan kode juga memegang kata sandinya. Risiko ini berkaitan langsung dengan insiden kebocoran data lewat vendor pada 2025 dan dengan kewajiban pengamanan dalam Undang-Undang (UU) Pelindungan Data Pribadi (PDP) [120].

5.4.3 Backing Services

Faktor IV menyebut setiap layanan yang dipakai aplikasi lewat jaringan sebagai *backing service*: basis data, antrean pesan, layanan surel, *cache*, dan layanan pihak ketiga. Aplikasi memperlakukan semuanya sebagai sumber daya yang dipasang lewat konfigurasi, tanpa membedakan layanan milik sendiri dan milik pihak ketiga. Akibatnya, satu basis data dapat diganti cukup dengan mengubah alamat dan kredensial di konfigurasi, tanpa mengubah kode [152]. Bagi Medikara Health, BPJS Kesehatan dan SATUSEHAT juga *backing service*. Lingkungan uji SATUSEHAT dan lingkungan produksinya seharusnya dapat ditukar hanya lewat konfigurasi.

Gambar 5.3 merangkai faktor I dan III dengan faktor V yang dibahas pada seksi berikutnya. Satu basis kode dibangun sekali, lalu digabung dengan konfigurasi setiap rumah sakit. Tabel 5.4 menerjemahkan keempat faktor kelompok A ke pelanggaran yang sering ditemui dan dampak bisnisnya.



Gambar 5.3: Satu *codebase*, satu *build*, lima *release*: yang berbeda antarlokasi hanya konfigurasinya

Tabel 5.4: Faktor kelompok A, pelanggaran yang sering ditemui, dan dampak bisnisnya

Faktor	Pelanggaran yang sering ditemui	Dampak bisnis
I *Codebase*	Setiap lokasi menyimpan salinan kode sendiri	Satu perbaikan dikerjakan berkali-kali; versi antarlokasi menyimpang
II *Dependencies*	Pustaka dipasang manual di server tanpa catatan	Server baru hanya dapat disiapkan oleh orang tertentu; pemulihan lambat
III *Config*	Alamat, kata sandi, dan kode fasilitas tertulis di kode	Perubahan kecil butuh rilis penuh; kredensial ikut tersebar bersama kode
IV *Backing services*	Alamat basis data dan layanan luar tertanam di banyak tempat	Penggantian penyedia atau migrasi basis data menjadi proyek besar

Kasus Medikara Health: Konfigurasi SIMRS di Lima Rumah Sakit

SIMRS di lima rumah sakit Medikara Health berasal dari satu keluarga produk, tetapi versinya berbeda, dari versi 3.9 di Bandung sampai versi 5.2 di Jakarta. Setiap rumah sakit menyimpan salinan kode sendiri yang sudah dimodifikasi tim TI setempat. Penelusuran tim TI pusat pada 2026 menemukan 37 nilai konfigurasi yang tertanam di kode dan berbeda antar-rumah sakit. Sembilan di antaranya adalah kata sandi atau kunci akses ke layanan luar.

Misalkan BPJS Kesehatan mengumumkan perubahan alamat dan kunci akses layanan integrasi klaimnya. Dengan kondisi sekarang, setiap rumah sakit mengubah kode, membangun ulang, menguji, dan memasang ulang SIMRS-nya, rata-rata dua hari kerja. Totalnya 10 hari kerja. Selama masa itu, sebagian rumah sakit sudah memakai alamat baru dan sebagian lain belum, sehingga laporan klaim harian sulit dibandingkan.

Bila konfigurasi dipindahkan ke lingkungan, perubahan yang sama cukup dengan mengganti dua nilai di setiap lokasi lalu menyalakan ulang aplikasi. Perkiraan tim TI pusat: satu hari kerja untuk kelima rumah sakit. Langkah ini tidak menyatukan lima versi SIMRS, karena itu urusan faktor I. Namun, langkah ini menghapus sembilan kredensial dari kode dan memangkas waktu perubahan konfigurasi dari dua minggu menjadi satu hari.

Pendalaman Teknis (opsional): Konfigurasi dari Environment Variable

Listing 5.1 memperlihatkan cara aplikasi contoh membaca konfigurasinya. Semua nilai yang berbeda antarlokasi diambil dari *environment variable*. Bila ada nilai wajib yang kosong, aplikasi menolak menyala dan menulis alasannya ke *log*. Konfigurasi yang hilang lebih baik ketahuan saat aplikasi mulai daripada saat pasien sudah memesan.

Listing 5.1: Membaca konfigurasi dari *environment variable*, tersedia di `sources/chapter05/contoh-aplikasi-konfigurasi/app.py`

```
def baca_konfigurasi(lingkungan):
  """Mengambil konfigurasi dari environment variable.

  Mengapa: nilai yang berbeda antarlokasi tidak boleh tertanam di kode.
  Akibat: bila ada variabel wajib yang kosong, aplikasi berhenti dengan
  pesan yang jelas, bukan gagal diam-diam saat melayani pasien.
  """
  kosong = [nama for nama in VARIABEL_WAJIB if not lingkungan.get(nama)
      ]
  if kosong:
    tulis_log("error", "konfigurasi_tidak_lengkap", variabel=kosong)
    sys.exit(KODE_KELUAR_GAGAL)
  return {
      "kode_rumah_sakit": lingkungan["KODE_RUMAH_SAKIT"],
```

```
        "alamat_basis_data": lingkungan["ALAMAT_BASIS_DATA"],
        "kunci_akses_bpjs": lingkungan["KUNCI_AKSES_BPJS"],
        "port": int(lingkungan.get("PORT", PORT_BAWAAN)),
    }
```

Contoh nilai untuk rumah sakit Bogor tersedia di `sources/chapter05/contoh-aplikasi-konfigurasi/contoh-bogor.env`. Nilainya rekaan. Di lingkungan produksi, kunci akses tidak disimpan di berkas teks biasa, melainkan di pengelola rahasia milik platform.

5.5 Faktor tentang Build, Proses, dan Skala

Kelompok B menjawab pertanyaan bisnis kedua: dapatkah kapasitas ditambah dan dikurangi dalam hitungan menit? Lima faktor di kelompok ini menentukan apakah lonjakan pengguna dapat dilayani dengan menambah server, atau hanya dapat ditunggu sampai reda.

5.5.1 Build, Release, Run

Faktor V memisahkan tiga tahap dengan tegas. Tahap *build* mengubah kode menjadi paket siap jalan. Tahap *release* menggabungkan paket itu dengan konfigurasi satu lokasi. Tahap *run* menjalankan *release* tersebut. Setiap *release* punya nomor unik dan tidak boleh diubah setelah dibuat. Perubahan apa pun menghasilkan *release* baru, sehingga kembali ke versi sebelumnya cukup dengan menjalankan *release* lama. Kode tidak boleh diubah langsung di server yang sedang berjalan [152].

Dampak bisnisnya ada dua. Pertama, pertanyaan audit seperti "versi berapa yang berjalan di rumah sakit Bogor pada 3 Maret?" dapat dijawab dari catatan *release*. Kedua, pemulihan dari rilis yang bermasalah turun dari hitungan jam menjadi hitungan menit, karena *release* sebelumnya masih utuh.

5.5.2 Processes, Port Binding, Concurrency, dan Disposability

Faktor VI menyatakan bahwa aplikasi berjalan sebagai satu atau beberapa proses tanpa *state* yang tidak saling berbagi apa pun. Data yang perlu bertahan disimpan di *backing service*, misalnya basis data, dan memori proses hanya dipakai sementara untuk satu permintaan. Twelve-Factor App menyebut *sticky session*, yaitu pengguna yang selalu diarahkan ke server yang sama karena sesinya tersimpan di sana, sebagai pelanggaran. Data sesi seharusnya disimpan di penyimpanan bersama yang punya batas kedaluwarsa [152].

Faktor VII menyatakan bahwa aplikasi menyediakan layanannya sendiri lewat sebuah *port*, tanpa bergantung pada server web yang disuntikkan dari luar. Akibatnya, satu aplikasi dapat menjadi *backing service* bagi aplikasi lain cukup dengan memberikan alamat dan *port*-nya [152]. Faktor VIII menyatakan bahwa kapasitas ditambah dengan menambah proses. Pekerjaan dibagi ke jenis proses, misalnya proses web untuk melayani permintaan dan proses *worker* untuk pekerjaan latar belakang, lalu setiap jenis ditambah jumlahnya sesuai beban [152]. Faktor IX menyatakan bahwa proses dapat dinyalakan dan dimatikan kapan saja. Waktu mulai dibuat sesingkat mungkin, proses berhenti dengan rapi ketika menerima sinyal `SIGTERM` dari platform, dan proses tetap aman bila mati mendadak [152].

Keempat faktor ini saling mengunci. Menambah proses tidak berguna bila proses baru butuh 11 menit untuk menyala, karena lonjakan sudah memuncak lebih dahulu. Menambah proses juga tidak berguna bila sesi pengguna terikat pada proses lama.

5.5.3 Penskalaan Vertikal dan Horizontal

Ada dua cara menambah kapasitas. Penskalaan vertikal (*scale up*) mengganti server dengan server yang lebih besar: prosesor lebih banyak dan memori lebih besar. Penskalaan horizontal (*scale out*) menambah server atau proses yang sama persis, lalu membagi beban di antara semuanya lewat penyeimbang beban (*load balancer*). Faktor VIII Twelve-Factor App mengarah ke cara kedua [152].

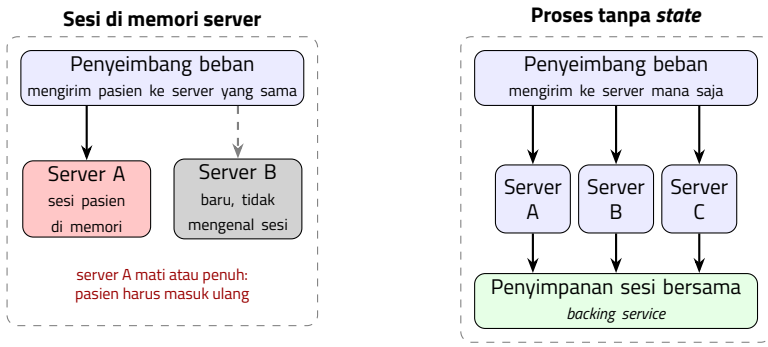
Analoginya adalah warung yang ramai saat jam makan siang. Cara pertama adalah mengganti kasir dengan kasir yang lebih cepat. Ada batasnya, karena secepat apa pun satu orang, ia tetap satu orang, dan selama pergantian berlangsung warung berhenti melayani. Cara kedua adalah membuka dua meja kasir tambahan yang bekerja dengan cara yang sama. Syaratnya satu: catatan pesanan tidak boleh ada di kepala satu kasir, tetapi di buku nota yang dapat dibaca semua kasir. Tabel 5.5 membandingkan kedua cara tersebut.

5.5.4 Mengapa Sesi di Memori Server Menghambat Penskalaan

Sesi adalah catatan sementara tentang pengguna yang sedang aktif: siapa yang sudah masuk, isi formulir yang belum selesai, dan langkah terakhir yang dibuka. Bila sesi disimpan di memori satu server, pengguna itu terikat pada server tersebut, dan penyeimbang beban terpaksa selalu mengirimnya ke sana. Server baru yang ditambahkan tidak mengenal pengguna lama, sehingga hanya dapat melayani pengguna yang baru masuk. Bila server lama mati atau dinyalakan ulang, semua sesi di dalamnya hilang dan penggunanya harus masuk ulang. Gambar 5.4 membandingkan kondisi itu dengan proses tanpa *state*.

Tabel 5.5: Penskalaan vertikal dan horizontal

Aspek	Vertikal (*scale up*)	Horizontal (*scale out*)
Cara	Mengganti server dengan yang lebih besar	Menambah server atau proses yang sama
Batas atas	Ukuran mesin terbesar yang disediakan penyedia	Kemampuan *backing service*, misalnya basis data, untuk melayani semua proses
Waktu menambah	Sering butuh penyalaan ulang, puluhan menit	Menit atau detik, bila proses cepat menyala
Saat satu server mati	Seluruh layanan berhenti	Server lain tetap melayani
Syarat pada aplikasi	Hampir tidak ada	Proses tanpa *state*, cepat menyala, konfigurasi dari lingkungan
Pola biaya	Server besar dibayar terus, termasuk di luar jam sibuk	Mengikuti beban bila jumlah server diturunkan kembali



Gambar 5.4: Sesi di memori server mengikat pengguna pada satu server, sedangkan proses tanpa *state* berbagi satu penyimpanan sesi

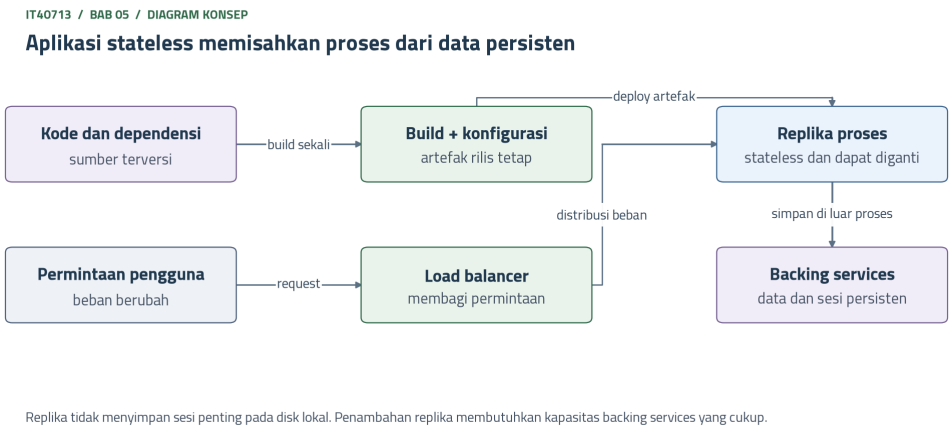
Proses aplikasi dibuat dapat diganti dan ditambah secara identik. Data serta sesi persisten berada pada layanan pendukung di luar proses aplikasi.

Gambar 5.5 merangkum hubungan komponen dan konsep utama bab ini.

Memindahkan sesi ke penyimpanan bersama punya harga: satu layanan tambahan yang harus dirawat dan dijaga ketersediaannya. Kasus berikut memperlihatkan mengapa harga itu layak dibayar oleh Medikara+.

Kasus Medikara Health: Lonjakan Medikara+ saat Program Vaksinasi

Misalkan Medikara Health mengumumkan program vaksinasi influenza bagi karyawan perusahaan mitra pada Senin pukul 09.00 lewat notifikasi Medikara+. Pada hari biasa, Medikara+ melayani sekitar 1.200 pengguna aktif bersamaan. Dalam 20 menit setelah notifikasi, jumlahnya naik menjadi



Gambar 5.5: Aplikasi stateless memisahkan proses dari data persisten

sekitar 9.600, atau delapan kali lipat. Waktu respons naik dari 1,5 detik menjadi 14 detik, dan 23 persen pemesanan gagal pada satu jam pertama. Vendor menempuh penskalaan vertikal: kedua mesin diganti dengan mesin berukuran dua kali lipat, dari 8 menjadi 16 inti prosesor. Prosesnya butuh sekitar 40 menit dan setiap mesin harus dinyalakan ulang. Karena sesi tersimpan di memori, semua pengguna yang sedang memesan terputus dan harus masuk ulang. Mesin baru juga butuh sekitar 11 menit untuk menyala karena memuat seluruh jadwal dokter ke memori lebih dahulu.

Penelusuran dengan kelompok B menunjukkan tiga kesenjangan. Faktor VI: sesi di memori server. Faktor VIII: kapasitas hanya dapat ditambah secara vertikal. Faktor IX: waktu menyala 11 menit, lebih lama dari separuh masa lonjakan. Ketiganya menjelaskan mengapa Medikara+ yang sudah di *cloud* tetap lambat. Elastisitas *cloud* tersedia, tetapi aplikasinya tidak dapat memanfaatkannya.

5.6 Faktor tentang Lingkungan, Log, dan Tugas Administratif

Kelompok C menjawab pertanyaan bisnis ketiga: dapatkah sistem dirawat, diaudit, dan dipulihkan dengan cepat? Tiga faktor di kelompok ini jarang dibicarakan di rapat manajemen, tetapi dampaknya terasa setiap kali terjadi gangguan atau pemeriksaan audit.

5.6.1 Dev/Prod Parity

Faktor X meminta lingkungan pengembangan, lingkungan uji (*staging*), dan lingkungan produksi dibuat semirip mungkin. Twelve-Factor App menyebut tiga kesenjangan yang perlu dipersempit. Kesenjangan waktu: kode baru sampai ke produksi setelah berminggu-minggu. Kesenjangan orang: pengembang menulis kode, sedangkan orang lain yang memasangnya. Kesenjangan alat: pengembang memakai basis data atau sistem operasi yang berbeda dari produksi. Twelve-Factor App juga mengingatkan agar pengembang tidak tergoda memakai *backing service* yang lebih ringan di lingkungan pengembangan [152]. Tabel 5.6 merangkum ketiganya.

Tabel 5.6: Tiga kesenjangan antara lingkungan pengembangan dan produksi, diolah dari [152]

Kesenjangan	Aplikasi tradisional	Aplikasi *twelve-factor*	Dampak bisnis bila dibiarkan
Waktu	Berminggu-minggu sampai berbulan-bulan	Hitungan jam atau menit	Kesalahan baru ketahuan lama setelah kodenya ditulis
Orang	Pengembang menulis, tim operasi memasang	Pengembang ikut memasang dan mengamati	Serah terima menambah antrean dan saling tunggu
Alat	Berbeda antara laptop dan produksi	Semirip mungkin	Kesalahan hanya muncul di produksi dan sulit ditiru

5.6.2 Logs sebagai Aliran Kejadian

Faktor XI memperlakukan *log* sebagai aliran kejadian yang terus mengalir, bukan sebagai berkas. Aplikasi tidak mengurus ke mana *log* dikirim atau bagaimana disimpan. Aplikasi cukup menulis setiap kejadian ke keluaran standar (`stdout`). Lingkungan tempat aplikasi berjalan yang menangkap aliran itu, menggabungkannya dari semua proses, lalu mengirimnya ke tempat penyimpanan dan analisis [152]. Pembahasan *observability* yang lebih luas ada di Bab 11.

Analoginya adalah kamera pengawas di lorong rumah sakit. Kamera tidak menyimpan rekaman di badannya sendiri. Rekaman dikirim ke ruang kendali yang menyimpannya, memberi cap waktu, dan memungkinkan pencarian. Bila setiap kamera menyimpan rekaman sendiri, mencari satu kejadian berarti mendatangi kamera satu per satu, dan rekaman ikut hilang ketika kameranya rusak.

Dua contoh kasus:

1. Sistem laboratorium rujukan menulis *log* ke berkas di tiga server. Saat gangguan, teknisi masuk ke tiga server bergantian dan mencocokkan jam secara manual, rata-rata sekitar 2 jam per penelusuran.

2. Bila ketiga server menulis ke `stdout` dan platform mengumpulkannya, satu pencarian berdasarkan nomor permintaan sudah memperlihatkan seluruh jejak kejadian.

5.6.3 Admin Processes

Faktor XII mengatur tugas administratif sekali jalan, misalnya migrasi struktur basis data, perbaikan data, atau skrip pembersihan. Tugas seperti itu dijalankan di lingkungan yang sama dengan proses biasa, memakai *release*, basis kode, dan konfigurasi yang sama. Kode tugas administratif disimpan bersama kode aplikasi [152]. Dampak bisnisnya terkait langsung dengan kontrol akses. Skrip perbaikan data yang dijalankan seseorang langsung di basis data produksi, dari laptopnya, tanpa catatan, adalah jenis kejadian yang sulit dijelaskan kepada auditor.

Kasus Medikara Health: Pertanyaan Audit tentang Hasil Laboratorium

Audit internal Medikara Health menemukan kontrol akses yang lemah. Salah satu tindak lanjutnya, auditor meminta daftar petugas yang membuka hasil laboratorium sepuluh pasien contoh selama 90 hari terakhir.
Sistem laboratorium rujukan tidak dapat memenuhinya. *Log* hanya disimpan tujuh hari di tiap server, dengan format yang berbeda karena ketiga server dipasang pada tahun yang berbeda. Tim TI butuh tiga hari kerja untuk mengumpulkan dan mencocokkan *log* tujuh hari itu saja. Misalkan pada Maret 2026 juga terjadi gangguan enam jam setelah pembaruan, karena lingkungan uji memakai versi basis data yang lebih baru daripada produksi. Kesalahan itu tidak muncul saat pengujian.
Dua kesenjangan kelompok C terlihat jelas. Faktor XI: *log* berupa berkas lokal yang berumur pendek. Faktor X: lingkungan uji tidak sama dengan produksi. Dampaknya bukan hanya waktu kerja tim TI, tetapi juga kemampuan organisasi membuktikan kepatuhan. Berapa lama *log* akses wajib disimpan adalah keputusan tata kelola dan kepatuhan yang dibahas di IT40813. Mata kuliah ini memastikan *log* dapat dikumpulkan dan dicari begitu angka itu ditetapkan.

Pendalaman Teknis (opsional): Log Terstruktur ke Stdout

Listing 5.2 memperlihatkan fungsi penulis *log* pada aplikasi contoh. Setiap kejadian ditulis sebagai satu baris berformat JavaScript Object Notation (JSON) ke `stdout`, dengan nama kejadian dan rincian sebagai pasangan nama dan nilai. Format ini dapat dibaca manusia dan mesin sekaligus, sehingga platform dapat menyaring *log* berdasarkan rumah sakit, tingkat

keparahan, atau nama kejadian.

Listing 5.2: Menulis *log* terstruktur ke `stdout`, tersedia di `sources/chapter05/contoh-aplikasi-konfigurasi/app.py`

```
def tulis_log(tingkat, kejadian, **rincian):
  """Menulis satu kejadian sebagai satu baris JSON ke stdout.

  Mengapa: aplikasi tidak mengurus berkas log, rotasi, atau penyimpanan.
  Akibat: platform dapat mengumpulkan log semua proses ke satu tempat,
  sehingga satu pertanyaan audit dijawab dengan satu pencarian.
  """
  catatan = {
      "waktu": datetime.now(timezone.utc).isoformat(),
      "tingkat": tingkat,
      "kejadian": kejadian,
  }
  catatan.update(rincian)
  print(json.dumps(catatan), flush=True)
```

Dua hal perlu dijaga. Pertama, `flush=True` memastikan setiap baris langsung keluar, tidak tertahan di penyangga. Kedua, kunci akses dan data pribadi pasien tidak boleh ikut ditulis ke *log*. Aplikasi contoh hanya mencatat kode rumah sakit dan alamat yang diminta, dan juga menangani `SIGTERM` agar berhenti dengan rapi, sesuai faktor IX.

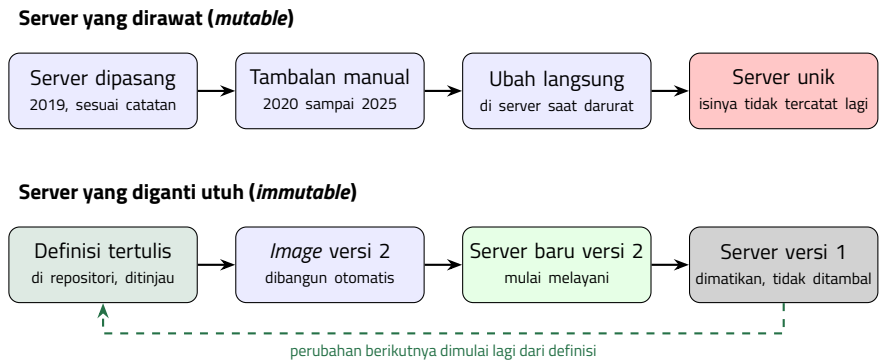
5.7 Immutable Infrastructure dan Konfigurasi Deklaratif

Dua belas faktor berbicara tentang aplikasi. Pertanyaan berikutnya menyangkut server tempat aplikasi berjalan: dirawat atau diganti?

5.7.1 Server yang Dirawat dan Server yang Diganti

Server yang dirawat satu per satu lama-kelamaan menjadi unik. Setiap tambalan keamanan, perubahan pengaturan saat darurat, dan pustaka yang dipasang manual menambah perbedaan kecil yang tidak tercatat. Fowler menyebut server seperti ini *snowflake server*, server kepingan salju yang tidak ada duanya dan sulit dibuat ulang [59]. *Immutable infrastructure* mengambil jalan sebaliknya. Server tidak diubah setelah dipasang. Setiap perubahan dibuat pada definisi tertulis, dari definisi itu dibangun *image* baru, lalu server baru menggantikan server lama secara utuh [104]. Gambar 5.6 membandingkan kedua jalan tersebut.

Analoginya adalah prosedur tertulis di sebuah rumah sakit. Bila setiap ruangan mencoret dan menambah catatan pada salinan prosedurnya sendiri, setelah beberapa tahun tidak ada dua salinan yang sama. Cara yang tertib adalah merevisi dokumen induk, mengesahkannya, lalu menarik semua salinan lama dan memba-



Gambar 5.6: Server yang dirawat menumpuk perubahan manual, sedangkan server *immutable* dibangun ulang dari definisi dan diganti utuh

gikan salinan baru. Tidak ada coretan di salinan, sehingga isi setiap ruangan selalu sama dengan dokumen induk.

Dua contoh kasus:

1. Server SIMRS rumah sakit Depok dipasang pada 2019 dan dirawat manual selama enam tahun. Ketika cakramnya rusak pada 2025, tim TI butuh empat hari untuk membangun server pengganti, karena sebagian pengaturannya hanya diketahui dari ingatan dua orang.
2. Misalkan server yang sama didefinisikan secara tertulis dan *image*-nya dibangun otomatis. Server pengganti dapat disiapkan dalam sekitar 45 menit, dan hasilnya sama dengan server yang rusak.

5.7.2 Langkah Manual dan Konfigurasi Deklaratif

Dua cara menuliskan definisi sering tertukar. Pembagiannya begini. Cara imperatif menuliskan langkah: pasang ini, ubah itu, jalankan tiga salinan. Cara deklaratif menuliskan keadaan akhir yang diinginkan: harus ada tiga salinan aplikasi versi 5.3 dengan konfigurasi rumah sakit Bogor. Sistem lalu mencari sendiri langkah yang diperlukan dan terus membandingkan keadaan nyata dengan keadaan yang diinginkan. Dokumentasi Kubernetes membedakan tiga teknik pengelolaan objek: perintah imperatif, konfigurasi objek imperatif, dan konfigurasi objek deklaratif [137]. Contoh manifest deklaratif untuk aplikasi contoh bab ini tersedia di `sources/chapter05/contoh-aplikasi-konfigurasi/deployment.yaml`.

Analoginya adalah memberi petunjuk jalan. Cara imperatif berbunyi "lurus 200 meter, belok kiri di lampu merah, lalu belok kanan setelah masjid". Bila satu jalan ditutup, petunjuk itu gagal. Cara deklaratif cukup menyebut alamat tujuan di aplikasi peta. Aplikasi peta mencari rute sendiri dan menghitung ulang bila ada

jalan yang ditutup. Tabel 5.7 membandingkan keduanya dari sisi yang dirasakan organisasi.

Tabel 5.7: Langkah manual (imperatif) dan konfigurasi deklaratif

Aspek	Langkah manual (imperatif)	Konfigurasi deklaratif
Yang ditulis	Urutan langkah	Keadaan akhir yang diinginkan
Bila diulang	Hasil dapat berbeda bila ada langkah terlewat	Hasil sama setiap kali
Bila keadaan menyimpang	Tidak ketahuan sampai ada gangguan	Penyimpangan dideteksi dan dikembalikan
Jejak audit	Bergantung pada catatan petugas	Setiap perubahan definisi tercatat dan dapat ditinjau
Kebutuhan awal	Rendah, cukup dokumen langkah	Lebih tinggi: alat, keterampilan, dan disiplin tim

Keputusan beralih ke konfigurasi deklaratif punya dua konsekuensi. Pertama, setiap perubahan server harus lewat definisi tertulis, termasuk perubahan saat darurat. Kebiasaan "masuk ke server lalu ubah langsung" harus dihentikan, dan ini lebih sering menjadi tantangan budaya daripada tantangan teknis. Kedua, definisi itu menjadi aset yang dirawat seperti kode: disimpan di repositori, ditinjau sebelum dipakai, dan diberi nomor versi. Prinsip OpenGitOps menyatakan hal serupa: keadaan yang diinginkan dinyatakan secara deklaratif, disimpan dengan versi yang tidak dapat diubah, ditarik otomatis oleh sistem, dan terus dicocokkan dengan keadaan nyata [114]. *Container* pada Bab 8 adalah bentuk paling umum dari *image* yang tidak diubah, dan GitOps pada Bab 13 adalah cara mengelola definisi deklaratifnya.

5.8 Aktivitas: Kartu Skor Kesenjangan Cloud-Native

Tujuan. Tim menilai dua sampai tiga aplikasi inti terhadap dua belas faktor, lalu menerjemahkan setiap kesenjangan ke dampak bisnis yang dapat dipahami manajemen. Hasilnya bukan nilai lulus atau gagal, melainkan daftar kesenjangan yang paling merugikan beserta buktinya.

Pembagian peran. Anggota berlatar teknis menjelaskan arti setiap faktor kepada tim, menyusun pertanyaan untuk tim TI organisasi, dan mengumpulkan bukti dari dokumentasi, wawancara, atau pengamatan cara rilis. Anggota berlatar non-teknis menuliskan dampak bisnis setiap kesenjangan dengan angka, misalnya hari kerja, jumlah pasien, atau rupiah, lalu memeriksa apakah bahasanya dapat dipahami direksi. Satu anggota ditunjuk sebagai pencatat bukti agar setiap skor dapat ditelusuri sumbernya.

Langkah kerja.

1. Pilih dua sampai tiga aplikasi inti dari portofolio Bab 4. Dahulukan aplikasi yang diberi usulan Replatform atau Refactor, karena kesenjangannya menentukan besar pekerjaan. Aplikasi yang akan dipensiunkan cukup dicatat alasannya.
2. Pelajari arti setiap faktor bersama, memakai Tabel 5.3 dan daftar pertanyaan di `sources/chapter05/pertanyaan-bukti-per-faktor.md`.
3. Kumpulkan bukti untuk setiap faktor: dokumen prosedur rilis, hasil wawancara dengan tim TI atau vendor, atau pengamatan langsung yang tidak memuat data rahasia.
4. Beri skor setiap faktor memakai skala pada Tabel 5.8. Setiap skor wajib disertai satu kalimat bukti.
5. Tulis dampak bisnis setiap faktor yang skornya 0 atau 1, dengan angka dan sumber angkanya.
6. Tandai tiga kesenjangan yang paling merugikan untuk seluruh aplikasi yang dinilai, lalu tulis alasan pemilihannya dalam satu paragraf.

Tabel 5.8: Skala skor kartu *twelve-factor*

Skor	Arti	Contoh bukti pada faktor III *Config*
0	Belum dipenuhi	Kata sandi dan alamat basis data tertulis di kode
1	Sebagian kecil	Sebagian nilai di berkas terpisah yang ikut tersimpan bersama kode
2	Sebagian besar	Konfigurasi di lingkungan, kecuali beberapa nilai lama
3	Dipenuhi	Semua nilai yang berbeda antarlokasi ada di lingkungan atau pengelola rahasia
TR	Tidak relevan	Faktor tidak berlaku untuk bentuk aplikasi ini; alasannya ditulis

Tabel 5.9 memperlihatkan contoh kartu skor untuk tiga aplikasi Medikara Health. Angkanya rekaan untuk kasus fiktif ini, tetapi disusun agar konsisten dengan kasus-kasus di bab ini.

Jumlah skor tidak boleh dibaca sebagai peringkat mutu aplikasi. Medikara+ punya jumlah tertinggi, tetapi satu kesenjangannya, yaitu sesi di memori, menyebabkan kerugian paling terasa bagi pasien. Dari contoh ini, tiga kesenjangan yang paling merugikan adalah faktor VI pada Medikara+ (pemesanan gagal saat lonjakan), faktor III pada SIMRS (dua minggu untuk satu perubahan konfigurasi dan sembilan kredensial di kode), serta faktor XI pada sistem laboratorium (pertanyaan audit tidak dapat dijawab).

Luaran dan kaitan capstone. Luarannya adalah kartu skor kesenjangan *cloud-native* untuk aplikasi inti, diisi di `sources/chapter05/kartu-skor-twelve-factor.xlsx`, ditambah ringkasan satu halaman bagi manajemen yang memuat tiga kesenjangan terpenting beserta dampaknya. Kedua dokumen itu masuk ke

Tabel 5.9: Contoh kartu skor kesenjangan *cloud-native* tiga aplikasi Medikara Health (skor 0 sampai 3)

Faktor	SIMRS Bogor	Medikara+	Sistem lab	Bukti utama
I *Codebase*	0	2	2	Lima salinan kode SIMRS
II *Dependencies*	1	2	1	Pustaka SIMRS dipasang manual
III *Config*	0	1	1	37 nilai tertanam di kode SIMRS
IV *Backing services*	1	2	1	Alamat BPJS Kesehatan di kode
V *Build, release, run*	0	1	1	Rilis Medikara+ lewat 37 langkah manual
VI *Processes*	1	0	2	Sesi Medikara+ di memori server
VII *Port binding*	1	2	1	SIMRS bergantung server web terpasang
VIII *Concurrency*	0	1	1	Medikara+ hanya naik vertikal
IX *Disposability*	0	1	1	Mesin baru Medikara+ menyala 11 menit
X *Dev/prod parity*	0	1	1	Versi basis data uji lab berbeda
XI *Logs*	1	1	0	*Log* lab lokal, tujuh hari
XII *Admin processes*	1	2	1	Skrip perbaikan SIMRS dari laptop
Jumlah (dari 36)	**6**	**16**	**13**	Persentase: 17, 44, dan 36

tonggak C2 Kondisi Saat Ini, yang ditinjau menjelang Ujian Tengah Semester (UTS). Kartu skor ini juga menjadi masukan Bab 7 saat tim mendiagnosis keterkaitan aplikasi inti, dan masukan Bab 14 saat menyusun urutan modernisasi.

Kriteria penilaian. Ketepatan arti faktor (20%): setiap faktor dinilai sesuai bunyinya. Kekuatan bukti (30%): setiap skor dapat ditelusuri sumbernya. Dampak bisnis terukur (30%): dampak ditulis dengan angka beserta sumber atau anggapannya. Prioritas yang beralasan (20%): tiga kesenjangan dipilih dari dampak, bukan dari skor terendah semata.

Catatan etika. Bukti konfigurasi sering memuat kata sandi, alamat internal, atau kunci akses. Nilai seperti itu tidak boleh disalin ke kartu skor atau ditampilkan dalam tangkapan layar. Cukup tulis keberadaannya, misalnya "kata sandi basis data tertulis di berkas kode". Data pribadi pasien atau karyawan tidak boleh dipakai sama sekali.

Pendalaman teknis (opsional). Anggota berlatar teknis menjalankan `sources/chapter05/contoh-aplikasi-konfigurasi/app.py` di laptop sendiri dua kali, sekali dengan konfigurasi Bogor dan sekali dengan konfigurasi rumah sakit lain, tanpa mengubah kode. Lalu proses dihentikan dengan sinyal `SIGTERM` untuk melihat *log* berhenti yang rapi. Langkahnya ada di `sources/chapter05/README.md`. Aplikasi contoh hanya dijalankan di komputer sendiri, tidak diarahkan ke sistem milik organisasi.

5.9 Ringkasan

Medikara+ sudah berada di *cloud*, tetapi tetap lambat saat lonjakan dan tetap butuh dua minggu untuk satu pembaruan. Bab ini menjelaskan mengapa hal itu mungkin terjadi. Definisi *cloud-native* CNCF versi 1.1 tidak berbicara tentang lokasi atau merek teknologi. Definisi itu berbicara tentang praktik dan sifat sistem: keterkaitan longgar, aman, tangguh, terkelola, berkelanjutan, dan dapat diamati, sehingga perubahan berdampak besar dapat dilakukan sering dan dapat diprediksi dengan kerja manual yang minimal.

Twelve-Factor App memberi daftar periksa yang lebih rinci. Ditulis pada 2011 dari pengalaman Heroku, daftar itu kini menjadi proyek *open source* yang dikelola komunitas sejak November 2024. Dua belas faktornya dapat dikelompokkan menjadi tiga pertanyaan bisnis: dapatkah satu kode dipasang di banyak lokasi tanpa diubah, dapatkah kapasitas ditambah dan dikurangi dalam hitungan menit, dan dapatkah sistem dirawat, diaudit, dan dipulihkan dengan cepat.

Setiap faktor punya dampak bisnis yang dapat dihitung. Konfigurasi yang tertanam di kode SIMRS membuat satu perubahan alamat BPJS Kesehatan memakan dua minggu dan menyebarkan sembilan kredensial. Sesi di memori server membuat Medikara+ tidak dapat memanfaatkan penskalaan horizontal saat program vaksinasi. *Log* yang tersimpan sebagai berkas lokal tujuh hari membuat pertanyaan audit tentang akses hasil laboratorium tidak dapat dijawab.

Di luar aplikasi, cara merawat server ikut menentukan. Server yang dirawat manual bertahun-tahun menjadi unik dan sulit dibuat ulang, sedangkan *immutable infrastructure* membangun ulang server dari definisi tertulis. Konfigurasi deklaratif menuliskan keadaan akhir yang diinginkan, bukan langkahnya, dan menjadi dasar *container* pada Bab 8 serta GitOps pada Bab 13.

Pesan utama bab ini: *cloud-native* adalah sifat yang dibuktikan, bukan lokasi yang ditempati. Kartu skor kesenjangan yang disusun tim pada Aktivitas bab ini mengubah pertanyaan "sudah di *cloud* atau belum" menjadi pertanyaan yang lebih berguna bagi direksi: kesenjangan mana yang paling merugikan, berapa kerugiannya, dan apa buktinya.

5.10 Latihan

Latihan 1: Faktor di Balik Lambatnya Medikara+

Latihan ini membuktikan bahwa masalah kinerja yang terlihat sama dapat berasal dari beberapa faktor sekaligus, dan bahwa faktor yang paling relevan dipilih dari bukti, bukan dugaan. Pertanyaan pemicunya: Medikara+ sering lambat setiap kali program vaksinasi diumumkan. Faktor mana dalam Twelve-Factor App yang paling

relevan, dan mengapa?

1. Baca kembali kasus lonjakan Medikara+ di seksi kelompok B.
2. Daftar semua faktor yang mungkin berperan, sedikitnya empat.
3. Tulis gejala setiap faktor pada Medikara+ dan angka yang menjadi buktinya.
4. Beri tingkat relevansi tinggi, sedang, atau rendah, beserta alasannya.
5. Pilih satu faktor yang paling relevan dan tulis satu paragraf pembelaannya.

Tabel 5.10 memuat contoh terisi. Tabel 5.11 adalah lembar isian.

Tabel 5.10: Latihan 1, contoh terisi: faktor di balik lambatnya Medikara+

Faktor	Gejala pada Medikara+	Bukti dan angka	Relevansi
VI *Processes*	Sesi di memori; mesin tambahan tidak membantu pengguna lama	1.200 pengguna lama tetap di dua mesin	Tinggi
VIII *Concurrency*	Kapasitas hanya ditambah secara vertikal	40 menit, semua pengguna terputus	Tinggi
IX *Disposability*	Mesin baru lama menyala	11 menit, lonjakan memuncak dalam 20 menit	Sedang
V *Build, release, run*	Rilis manual, perbaikan cepat sulit dipasang	37 langkah, dua minggu	Rendah

Tabel 5.11: Latihan 1, lembar isian

Faktor	Gejala pada Medikara+	Bukti dan angka	Relevansi
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Bila hanya faktor IX yang diperbaiki sehingga mesin baru menyala dalam satu menit, berapa persen masalah yang kira-kira terselesaikan? Gunakan angka di tabel untuk menalar jawabannya.
2. Pada contoh, faktor VI dan VIII sama-sama bernilai tinggi. Mana yang harus diperbaiki lebih dahulu, dan mengapa urutannya penting?

Latihan 2: Sesi di Memori Server dalam Bahasa Direksi

Latihan ini membuktikan bahwa alasan teknis di balik keputusan arsitektur dapat dijelaskan tanpa istilah teknis, asalkan analogi, akibat, dan angkanya disusun berurutan. Pertanyaan pemicunya: mengapa aplikasi yang menyimpan sesi pengguna di memori server sulit diskalakan?

1. Pilih satu analogi sehari-hari yang menggambarkan sesi di memori server.
2. Tulis akibatnya saat server ditambah, lalu saat satu server mati atau dinyalakan ulang.
3. Sertakan satu angka dari Medikara+ atau dari organisasi tim.
4. Tutup dengan satu kalimat usulan beserta biayanya.

Tabel 5.12 memuat contoh terisi. Tabel 5.13 adalah lembar isian.

Tabel 5.12: Latihan 2, contoh terisi: penjelasan untuk direksi Medikara Health

Bagian	Isi
Analogi	Kasir yang mengingat pesanan pelanggan di kepalanya sendiri, bukan di buku nota bersama
Akibat saat server ditambah	Kasir baru hanya dapat melayani pelanggan baru; pelanggan lama tetap antre di kasir lama
Akibat saat server mati	Semua pesanan di kepala kasir itu hilang; pelanggan harus memesan ulang
Angka	Saat vaksinasi, 9.600 pengguna aktif bersamaan; penggantian mesin 40 menit memutus semua pemesanan yang sedang berjalan
Usulan	Memindahkan sesi ke penyimpanan bersama; biayanya satu layanan tambahan dan pekerjaan ubahan oleh vendor

Tabel 5.13: Latihan 2, lembar isian

Bagian	Isi
Analogi	...
Akibat saat server ditambah	...
Akibat saat server mati	...
Angka	...
Usulan	...

Pertanyaan analisis:

1. Bagian mana di tabel yang paling mungkin dipertanyakan direksi? Data apa yang perlu disiapkan untuk menjawabnya?
2. Penyimpanan sesi bersama juga dapat mati. Apakah usulan pada contoh hanya memindahkan titik lemah? Jelaskan dengan membandingkan akibatnya.

Latihan 3: Aplikasi yang Tidak Perlu Memenuhi Dua Belas Faktor

Latihan ini membuktikan bahwa dua belas faktor adalah alat penilaian, bukan kewajiban yang sama untuk semua aplikasi. Pertanyaan pemicunya: apakah semua aplikasi organisasi perlu memenuhi dua belas faktor? Latihan ini meminta satu contoh aplikasi yang tidak perlu, beserta alasannya.

1. Pilih tiga aplikasi dengan sifat berbeda, misalnya satu yang dipakai pasien, satu alat internal kecil, dan satu yang akan dipensiunkan.
2. Tulis sifat setiap aplikasi: pengguna, frekuensi perubahan, dan umur yang tersisa.
3. Tentukan faktor yang tetap wajib dan faktor yang boleh diabaikan.
4. Tulis keputusan dan alasannya dalam satu kalimat.

Tabel 5.14 memuat contoh terisi untuk Medikara Health dengan angka rekaan. Tabel 5.15 adalah lembar isian.

Tabel 5.14: Latihan 3, contoh terisi: tiga aplikasi Medikara Health

Aplikasi	Sifat	Faktor wajib	Keputusan
Medikara+	310.000 pengguna, beban naik turun, rilis diharapkan tiap pekan	Hampir semua	Kejar seluruh faktor, mulai dari VI dan VIII
Rekap lembur bulanan	12 pengguna, jalan sekali sebulan, jarang berubah	III, XI	Cukup pindahkan kredensial dan rapikan *log*; faktor skala diabaikan
SIMRS Bandung	Versi 3.9, misalkan dijadwalkan diganti pada 2027	III, XI	Jangan investasi besar; amankan kredensial dan *log* sampai diganti

Tabel 5.15: Latihan 3, lembar isian

Aplikasi	Sifat	Faktor wajib	Keputusan
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Pada contoh, faktor III dan XI muncul sebagai faktor wajib di semua baris. Apa yang membuat kedua faktor itu berbeda dari faktor lain?
2. Bila jadwal penggantian SIMRS Bandung mundur dari 2027 ke 2030, faktor mana yang perlu ditambahkan ke daftar wajib? Jelaskan dari sisi biaya dan risiko.

Bab 6

Domain-Driven Design dan Batas Layanan

Tujuan Pembelajaran

- Mengidentifikasi *core*, *supporting*, dan *generic subdomain* dalam sebuah organisasi.
- Memfasilitasi lokakarya EventStorming untuk menemukan *bounded context*.
- Menyusun *context map* yang menunjukkan hubungan antar-*bounded context*.

6.1 Pendahuluan

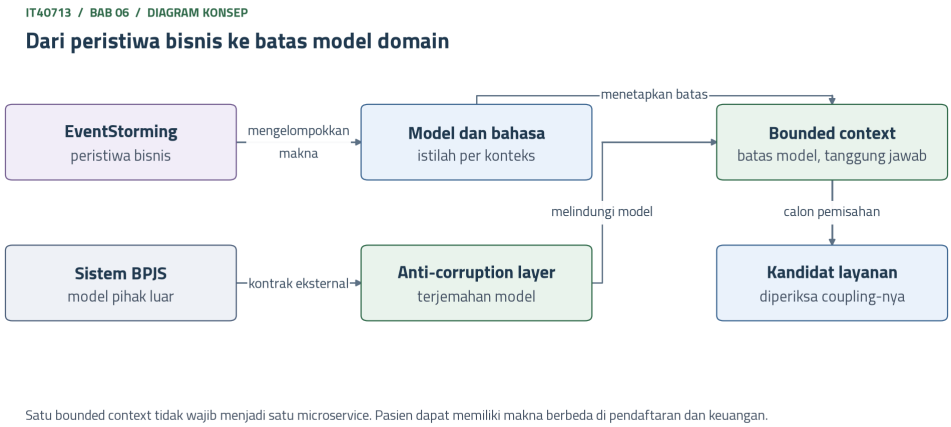
Setelah kartu skor *twelve-factor* pada Bab 5, tim teknologi informasi (TI) Medikara Health sepakat bahwa SIMRS (Sistem Informasi Manajemen Rumah Sakit) perlu dipecah menjadi beberapa layanan. Rapat pertama langsung tersendat pada satu kata. Kepala pendaftaran menyebut pasien sebagai orang yang sudah punya nomor rekam medis dan boleh dilayani. Bagian keuangan menyebut pasien sebagai pihak yang ditagih, lengkap dengan penjaminnya: BPJS Kesehatan, asuransi, perusahaan, atau pribadi. Laboratorium menyebut pasien sebagai pemilik spesimen pada satu permintaan pemeriksaan. Ketiganya benar di tempat masing-masing. Masalahnya, SIMRS menyimpan ketiga makna itu dalam satu tabel yang dipakai bersama, sehingga perubahan kecil untuk satu bagian sering merusak bagian lain.

Memecah sistem tanpa membereskan perbedaan makna ini berbahaya. Bila batas layanan ditarik di tempat yang salah, misalnya satu layanan "pasien" untuk semua bagian, layanan baru tetap saling bergantung seperti modul lama. Bedanya, kini ketergantungan itu berjalan lewat jaringan. Hasilnya sistem baru yang sama

kusutnya dengan sistem lama, ditambah kelambatan dan titik gagal yang lebih banyak. Karena itu, sebelum bertanya "layanan apa yang dibangun", tim perlu bertanya lebih dahulu "bisnis ini terdiri dari bagian apa, dan kata apa yang dipakai di setiap bagian". Cara menjawabnya adalah lokakarya: pakar bisnis dan tim teknis menyusun alur kerja bersama di satu dinding.

Hasil lokakarya dipakai untuk menelusuri peristiwa, bahasa domain, batas model, dan calon pemisahan layanan.

Gambar 6.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 6.1: Dari peristiwa bisnis ke batas model domain

Rujukan utama bab ini adalah buku *Domain-Driven Design* karya Evans [47], *Implementing Domain-Driven Design* karya Vernon [148], *Learning Domain-Driven Design* karya Khononov [85], buku EventStorming karya Brandolini [29], serta lembar ringkas *context mapping* dari komunitas DDD Crew [42]. Bab ini bergerak dalam lima langkah: *subdomain* dan *ubiquitous language*, *bounded context*, *context map*, EventStorming, dan penurunan kandidat layanan. Hasilnya adalah peta domain dan *context map* kondisi saat ini untuk tonggak capstone C2.

6.2 Domain, Subdomain, dan Ubiquitous Language

Domain-Driven Design (DDD) adalah pendekatan merancang perangkat lunak yang berangkat dari pemahaman mendalam tentang bisnis yang dilayaninya. Evans memperkenalkan pendekatan ini pada 2003 dengan satu gagasan pokok: struktur perangkat lunak mengikuti model bisnis yang disepakati bersama oleh pakar bisnis dan tim teknis [47]. Domain adalah wilayah kegiatan yang dilayani sistem,

misalnya layanan kesehatan bagi Medikara Health. Domain yang besar terdiri dari beberapa *subdomain*, yaitu bagian bisnis dengan tujuan, aturan, dan pelakunya sendiri, misalnya pendaftaran, penagihan, atau laboratorium. *Subdomain* ditemukan di bisnis, bukan dirancang oleh tim teknis. Tugas tim adalah mengenalinya.

Subdomain tidak sama penting. Vernon dan Khononov membaginya menjadi tiga jenis [148, 85]. *Core subdomain* adalah bagian yang membuat organisasi berbeda dari pesaingnya, sehingga menjadi alasan pelanggan atau mitra memilihnya. *Supporting subdomain* diperlukan agar bisnis berjalan dan cukup khas untuk organisasi itu, tetapi tidak menjadi pembeda. *Generic subdomain* adalah bagian yang dikerjakan semua organisasi sejenis dengan cara yang hampir sama, sehingga solusi jadi sudah tersedia di pasar.

Analoginya seperti rumah makan padang yang terkenal karena rendangnya. Resep dan cara memasak rendang adalah alasan pelanggan datang. Resep itu dijaga ketat dan dipegang juru masak terbaik. Jadwal belanja bahan dan cara menata etalase disesuaikan dengan menu rumah makan itu, tetapi tidak ada pelanggan yang datang karena jadwal belanjanya. Mesin pembayaran kartu dan aplikasi pembukuan dibeli atau disewa, karena tidak ada untungnya membuat sendiri. Mengerahkan juru masak terbaik untuk merakit mesin kasir adalah pemborosan.

Khononov menambahkan dua pembeda yang mudah diperiksa: kerumitan aturan bisnisnya dan ketersediaan solusi di pasar [85]. Tabel 6.1 merangkum ketiga jenis beserta keputusan yang biasanya menyertainya.

Tabel 6.1: Tiga jenis *subdomain* dan keputusan yang menyertainya

Aspek	***Core***	***Supporting***	***Generic***
Peran bagi bisnis	Pembeda dari pesaing, alasan pasien atau mitra memilih	Menopang *core*, khas organisasi, bukan pembeda	Dibutuhkan semua organisasi sejenis
Kerumitan aturan	Tinggi dan terus dipelajari	Rendah sampai sedang	Bisa tinggi, tetapi sudah dipecahkan pihak lain
Laju perubahan	Sering, mengikuti strategi	Jarang sampai sedang	Mengikuti pembaruan vendor atau aturan umum
Bangun atau beli	Dibangun dan dikuasai sendiri	Dibangun sederhana, atau dibeli lalu disesuaikan	Dibeli atau disewa, misalnya Software as a Service (SaaS)
Penempatan orang	Tim terbaik dan pakar domain paling senior	Tim yang lebih kecil	Tim pengelola kontrak dan integrasi

Pembagian ini punya dua konsekuensi. Pertama, keputusan membangun sendiri atau membeli menjadi beralasan, bukan selera. Membangun sendiri bagian *generic* berarti membayar mahal untuk sesuatu yang tidak membedakan organisasi. Membeli bagian *core* berarti menyerahkan pembeda kepada vendor yang

juga menjualnya ke pesaing. Kedua, kerja DDD yang mahal, misalnya lokakarya panjang dan model yang dirancang hati-hati, diprioritaskan untuk *core*. Bagian *generic* cukup dipilih produknya dan diamankan integrasinya.

Klasifikasi juga bergantung pada strategi. Penagihan bisa *generic* bagi sebuah rumah sakit, tetapi menjadi *core* bagi perusahaan yang menjual perangkat lunak penagihan. Klasifikasi juga dapat bergeser seiring waktu, ketika strategi organisasi berubah.

Dua contoh dari Medikara Health:

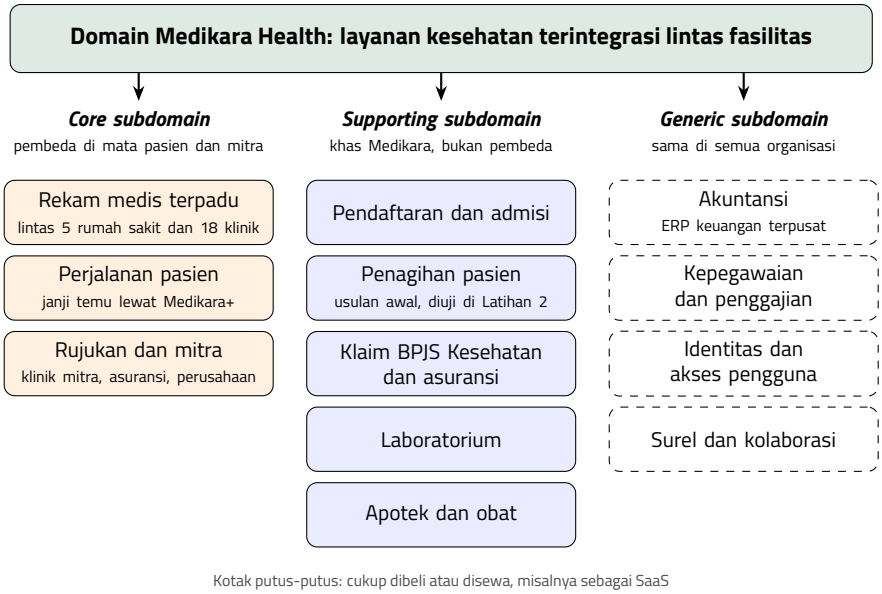
1. Penggajian sekitar 4.800 karyawan mengikuti aturan pajak dan iuran jaminan sosial yang sama untuk semua perusahaan. Bagian ini *generic*. Membangunnya sendiri berarti memelihara perubahan aturan pajak setiap tahun tanpa satu pun manfaat bagi pasien.
2. Rekam medis terpadu lintas 5 rumah sakit dan 18 klinik pratama adalah inti strategi "Satu Pasien, Satu Rekam Medis". Data kunjungan 2025 menunjukkan sekitar 168.000 dari 1,2 juta kunjungan, atau 14 persen, dilakukan pasien yang sebelumnya berobat di fasilitas Medikara lain. Pola rujukan seperti ini khas Medikara, sehingga bagian ini *core*.

Kasus Medikara Health: Peta Subdomain Awal

Tim transformasi menyusun peta *subdomain* awal dari dokumen strategi 2026 sampai 2030 dan wawancara dengan tujuh kepala bagian. Hasilnya ada di Gambar 6.2. Tiga *subdomain* digolongkan *core* karena langsung menopang strategi: rekam medis terpadu, perjalanan pasien dan janji temu, serta rujukan dan ekosistem mitra. Lima digolongkan *supporting*: pendaftaran, penagihan, klaim, laboratorium, dan apotek. Empat digolongkan *generic*: akuntansi, kepegawaian dan penggajian, identitas dan akses pengguna, serta surel.

Perdebatan paling panjang terjadi pada klaim BPJS Kesehatan. Misalkan sekitar 46 persen pendapatan rumah sakit Medikara berasal dari klaim ini, sehingga sebagian peserta ingin menggolongkannya *core*. Kesimpulannya tetap *supporting*. Nilainya besar, tetapi aturannya ditetapkan BPJS Kesehatan dan sama untuk semua rumah sakit, sehingga tidak membuat pasien memilih Medikara. Nilai yang besar tidak sama dengan pembeda.

Peta ini punya dua konsekuensi. Pertama, Medikara+ yang kini dibuat vendor ternyata menopang *subdomain core*. Ketergantungan penuh pada vendor untuk bagian pembeda adalah risiko strategis yang perlu masuk ke peta jalan modernisasi. Kedua, ERP (*Enterprise Resource Planning*) keuangan yang sudah terpusat tidak perlu diusik. Cukup integrasinya yang dirapikan.



Gambar 6.2: Peta *subdomain* awal Medikara Health: tiga *core*, lima *supporting*, dan empat *generic*

6.2.1 Ubiquitous Language

Ubiquitous language adalah kosakata bersama yang dipakai pakar bisnis dan tim teknis secara konsisten di percakapan, dokumen, diagram, dan kode, di dalam satu batas model [47, 57]. Bila petugas pendaftaran menyebut "kunjungan", layar, laporan, dan nama tabel juga memakai "kunjungan", bukan "registrasi" di satu tempat dan "episode" di tempat lain. Bahasa ini tidak disusun sekali lalu dibekukan. Bahasa ini tumbuh setiap kali tim menemukan istilah yang membingungkan, lalu menyepakati maknanya.

Analoginya seperti proyek bangunan. Arsitek, mandor, dan tukang memakai istilah yang sama pada gambar kerja: sloof, kolom, balok, dan ring balok. Bila mandor menyebut "balok" untuk bagian yang di gambar kerja disebut "sloof", tukang bisa mengecor di tempat yang salah. Proyek yang rapi menyepakati istilah sejak awal, lalu memakainya di setiap gambar, rapat, dan laporan harian.

Glosarium adalah wujud tertulis *ubiquitous language*. Tabel 6.2 menunjukkan potongan glosarium Medikara Health untuk alur rawat jalan. Kolom terakhir mencatat istilah lain yang ditemukan di SIMRS hasil akuisisi.

Dua contoh dari Medikara Health:

1. Lima SIMRS memakai tiga istilah berbeda untuk kunjungan. Laporan jumlah kunjungan bulanan yang digabung dari *spreadsheet* lima rumah sakit pernah selisih sekitar 9 persen, karena satu rumah sakit menghitung kunjungan

Tabel 6.2: Potongan glosarium alur rawat jalan Medikara Health

Istilah	Makna yang disepakati	Istilah lain yang ditemukan
Kunjungan	Satu kedatangan pasien ke satu poliklinik pada satu hari; dibuka saat pendaftaran, ditutup setelah tagihan selesai	Registrasi (Jakarta), episode (Bandung), *encounter* (adaptor SATUSEHAT)
Penjamin	Pihak yang menanggung biaya kunjungan: BPJS Kesehatan, asuransi, perusahaan, atau pribadi	Cara bayar, kelompok pasien, debitur
Rujukan masuk	Surat dari fasilitas lain yang menjadi dasar pasien BPJS Kesehatan dilayani di rumah sakit	Surat pengantar, *referral*
Pasien baru	Orang yang belum punya nomor rekam medis di seluruh jaringan Medikara	Pasien baru per rumah sakit (Depok, Bogor)

ulang ke poliklinik yang sama pada hari yang sama sebagai dua registrasi.

2. Istilah "pasien baru" di Depok dan Bogor berarti baru di rumah sakit itu, sedangkan di Jakarta berarti baru di seluruh jaringan. Akibatnya, satu orang bisa punya dua atau tiga nomor rekam medis. Pencocokan awal menemukan sekitar 38.000 nomor ganda akibat perbedaan makna ini saja. Angka itu hanya sebagian dari sekitar 600.000 salinan identitas di tujuh sumber data pada IT40513 Bab 9 (2,1 juta catatan untuk sekitar 1,5 juta orang), yang juga mencakup sebab lain.

Pertanyaan kemudian muncul: apakah seluruh Medikara Health harus memakai satu makna untuk setiap istilah? Jawabannya tidak. Glosarium pada Tabel 6.2 berlaku untuk alur rawat jalan. Bagian lain boleh memakai kata yang sama dengan makna lain, asalkan batasnya jelas. Batas itulah yang dibahas pada seksi berikut.

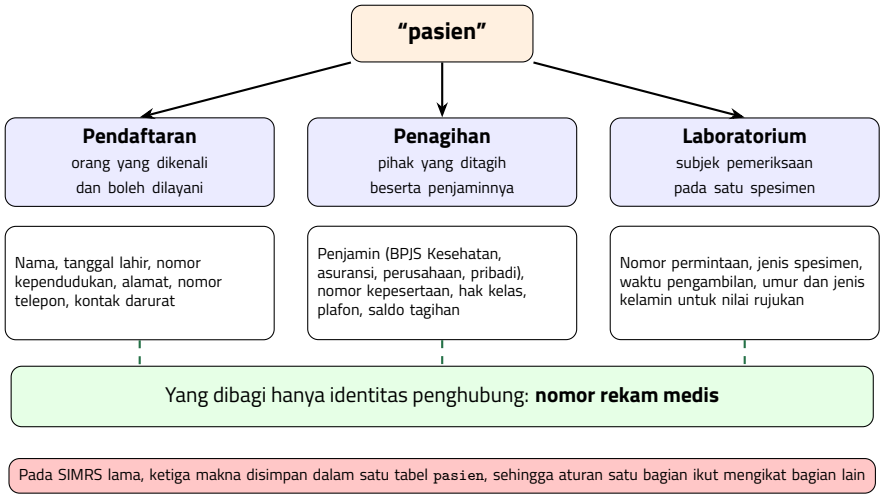
6.3 Bounded Context

Bounded context adalah batas eksplisit tempat satu model dan satu *ubiquitous language* berlaku secara konsisten [47]. Di dalam batas itu, setiap istilah punya satu makna. Di luar batas itu, istilah yang sama boleh bermakna lain. Fowler mencatat bahwa satu model tunggal untuk seluruh organisasi besar hampir tidak mungkin dicapai dan tidak sepadan biayanya. DDD menanganinya dengan membagi model besar menjadi beberapa *bounded context*, lalu menuliskan hubungan antarkonteks secara eksplisit [60]. Batas ini mencakup bahasa, model, data, dan kode. Batas bukan pagar yang menghalangi kerja sama, melainkan penanda tempat penerjemahan harus dilakukan.

Analoginya seperti kata "kartu". Di bank, kartu berarti kartu debit. Di loket rumah sakit, kartu berarti kartu berobat. Di lapangan sepak bola, kartu berarti kartu kuning atau merah. Tidak ada yang bingung, karena setiap tempat punya konteks

yang jelas. Kebingungan baru muncul bila ketiga tempat dipaksa memakai satu formulir "kartu" dengan semua kolom sekaligus: nomor rekening, nomor rekam medis, dan nama pemain yang diganjar.

Gambar 6.3 memperlihatkan cara *bounded context* menangani kata "pasien" di Medikara Health. Setiap konteks hanya menyimpan atribut yang dibutuhkannya. Yang dibagi antarkonteks hanya identitas penghubung, yaitu nomor rekam medis. Pemisahan seperti ini juga dipakai standar HL7 FHIR (*Health Level Seven Fast Healthcare Interoperability Resources*) yang menjadi dasar SATUSEHAT. Data demografis dan administratif pasien ada di *resource* Patient, sedangkan data penjaminan ada di *resource* Coverage yang terpisah [71].



Gambar 6.3: Satu kata "pasien", tiga makna di tiga *bounded context*; yang dibagi hanya identitas penghubung

Dua hal pada DDD sering tertukar: *subdomain* dan *bounded context*. Pembagiannya begini. *Subdomain* berada di ruang masalah: bagian bisnis yang ditemukan, ada atau tidak ada sistemnya. *Bounded context* berada di ruang solusi: batas model yang dirancang tim. Idealnya satu *bounded context* melayani satu *subdomain*, tetapi hubungan keduanya tidak selalu satu banding satu [85]. Tabel 6.3 merangkum perbedaannya.

6.3.1 Ukuran Bounded Context yang Wajar

Tidak ada ukuran baku untuk *bounded context*. Ada dua arah kesalahan. Konteks yang terlalu besar ditandai oleh istilah bermakna ganda di dalamnya, banyak tim yang mengubah model yang sama, dan rapat koordinasi di setiap rilis. Konteks yang terlalu kecil ditandai oleh satu perubahan bisnis yang selalu menyentuh beberapa konteks sekaligus, dan konteks yang tidak bisa menyelesaikan tugasnya

Tabel 6.3: Perbedaan *subdomain* dan *bounded context*

Aspek	***Subdomain***	***Bounded context***
Pertanyaan	Bagian bisnis apa saja yang ada?	Di mana satu model dan satu bahasa berlaku?
Sifat	Ditemukan dari bisnis	Dirancang oleh tim
Penentu	Strategi dan cara kerja organisasi	Arsitek dan tim, bersama pakar domain
Wujud	Peta *subdomain*	Batas model, kode, data, dan tim pemilik
Hubungan	Satu *subdomain* bisa dilayani satu atau beberapa konteks	Satu konteks idealnya melayani satu *subdomain*; pada sistem lama sering melayani banyak

tanpa terus bertanya ke konteks lain. Khononov memberi patokan kepemilikan: satu *bounded context* dimiliki satu tim, walaupun satu tim boleh memiliki beberapa konteks [85]. Patokan bahasa juga membantu. Bila dua kelompok pakar domain butuh "kamus" untuk saling mengerti, di situ ada batas.

Dua contoh dari Medikara Health:

1. Satu konteks "layanan rumah sakit" yang mencakup pendaftaran, pelayanan klinis, laboratorium, apotek, penagihan, dan klaim terlalu besar. Tabel pasien di SIMRS rumah sakit Jakarta berisi 87 kolom dan diubah oleh enam modul. Setiap perubahan kolom harus diuji ulang oleh keenam modul itu.
2. Memecah pendaftaran menjadi tiga konteks, yaitu identitas pasien, antrean loket, dan verifikasi penjamin, yang semuanya dikerjakan tim yang sama, terlalu kecil. Setiap perubahan formulir pendaftaran akan menyentuh tiga konteks dan tiga rilis.

Kasus Medikara Health: Satu Kolom, Tiga Bagian

Misalkan pada Maret 2026 bagian keuangan rumah sakit Depok meminta kolom penjamin pada tabel pasien diwajibkan terisi sebelum pasien dilayani, agar tidak ada tagihan tanpa penanggung. Vendor SIMRS mengubah aturan itu dalam satu hari. Akibatnya terasa di dua tempat lain. Di instalasi gawat darurat, petugas tidak bisa membuat permintaan pemeriksaan laboratorium untuk pasien yang datang tanpa identitas lengkap. Selama tiga hari, sekitar 140 permintaan pemeriksaan tertahan rata-rata 25 menit. Di laboratorium, label spesimen gagal dicetak untuk pasien rujukan dari klinik mitra yang penjaminnya belum diverifikasi.

Aturan keuangan itu benar di konteks penagihan, tetapi salah di konteks pelayanan darurat dan laboratorium. Dengan *bounded context*, aturan "penjamin wajib jelas" hanya berlaku di konteks penagihan, yaitu sebelum tagih-

an ditutup, bukan sebelum pasien dilayani. Laboratorium cukup mengenal pasien sebagai subjek spesimen dengan nomor rekam medis, umur, dan jenis kelamin. Insiden ini menjadi bahan Latihan 1.

6.4 Context Map dan Pola Hubungannya

Context map adalah peta yang menunjukkan *bounded context* dalam satu sistem atau organisasi, beserta hubungan di antaranya [47]. Setiap hubungan punya arah. Konteks hulu (*upstream*) adalah konteks yang keputusannya memengaruhi konteks lain. Konteks hilir (*downstream*) adalah konteks yang terdampak oleh perubahan di hulu, tetapi tidak sebaliknya. DDD Crew membedakan tiga jenis hubungan tim di balik peta itu: saling bergantung, hulu-hilir, dan bebas [42]. Pola hubungan menjelaskan cara kedua pihak menangani arah pengaruh tersebut.

Analoginya seperti irigasi sawah. Pengelola pintu air di hulu menentukan kapan dan berapa banyak air mengalir. Petani di hilir punya beberapa pilihan. Bila pengelola mau mendengar, petani mengajukan jadwal kebutuhan air setiap musim tanam. Bila pengelola tidak bisa diajak bicara, petani menyesuaikan jadwal tanam dengan jadwal air apa adanya. Petani yang cermat membuat kolam penampung dan saringan sendiri, sehingga lumpur dari hulu tidak langsung masuk ke sawah.

Tiga pilihan petani itu sejajar dengan tiga pola yang paling sering dipakai: *customer-supplier*, *conformist*, dan *anticorruption layer*. Tabel 6.4 merangkum lima pola yang dibahas di bab ini [47, 42].

Open host service hampir selalu berpasangan dengan *published language*, yaitu bahasa pertukaran yang terdokumentasi dan dipakai bersama, misalnya HL7 FHIR. DDD Crew juga mencatat pola lain [42]: *partnership* untuk dua tim yang berhasil atau gagal bersama, *separate ways* untuk dua konteks yang sengaja tidak dihubungkan, dan *big ball of mud* untuk bagian sistem yang modelnya bercampur tanpa batas jelas.

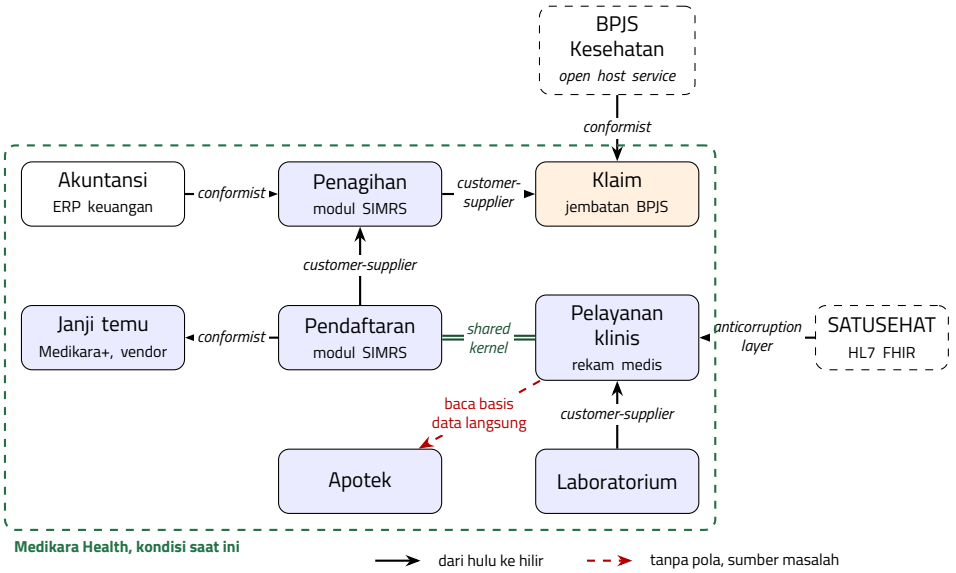
Gambar 6.4 adalah *context map* kondisi saat ini untuk alur rawat jalan Medikara Health. Peta ini jujur tentang keadaan, bukan gambaran yang diinginkan. Klaim ditandai warna lain karena menjadi prioritas perbaikan.

Dua contoh dari peta tersebut:

1. ERP keuangan adalah produk *generic* yang dibeli. Penagihan mengikuti format jurnal ERP apa adanya. Pola *conformist* di sini wajar, karena model akuntansi sudah baku dan jarang berubah. Membangun penerjemah sendiri hanya menambah biaya.
2. Aplikasi apotek di 12 apotek membaca tabel resep di basis data SIMRS seca-

Tabel 6.4: Lima pola hubungan pada *context map*

Pola	Arti singkat	Kapan dipakai	Biaya atau risiko
Customer-supplier	Hilir menyampaikan kebutuhan, hulu merencanakan pemenuhannya	Kedua tim dalam satu organisasi dan bisa bernegosiasi	Butuh forum perencanaan bersama yang rutin
Conformist	Hilir memakai model hulu apa adanya	Hulu tidak mau atau tidak bisa menyesuaikan, dan modelnya cukup baik	Model hilir ikut berubah setiap kali hulu berubah
Anticorruption layer	Hilir membangun lapisan penerjemah agar model hulu tidak masuk ke modelnya	Model hulu berbeda jauh, sering berubah, atau mutunya buruk	Ada kode dan pemeliharaan tambahan
Shared kernel	Dua konteks berbagi sebagian kecil model secara eksplisit	Dua tim yang bekerja berdekatan, bagian bersamanya kecil	Setiap perubahan bagian bersama wajib disepakati kedua tim
Open host service	Hulu menyediakan protokol terbuka dan terdokumentasi untuk banyak pemakai	Hulu melayani banyak konteks hilir	Hulu memelihara kontrak dan versinya



Gambar 6.4: *Context map* kondisi saat ini untuk alur rawat jalan Medikara Health

ra langsung, sesuai temuan pemetaan dependensi pada Bab 4. Hubungan ini tidak mengikuti pola apa pun. Setiap perubahan struktur tabel resep berisiko menghentikan pelayanan obat. Hubungan seperti ini adalah ciri *big ball of mud* dan menjadi calon perbaikan pertama.

6.4.1 Hubungan Antartim di Balik Pola

Conway mengamati bahwa organisasi yang merancang sistem cenderung menghasilkan rancangan yang meniru struktur komunikasinya [40]. Pengamatan ini dikenal sebagai hukum Conway. Akibatnya, *context map* sekaligus peta hubungan antartim dan antarorganisasi. Pola tidak dipilih menurut selera teknis, tetapi menurut kenyataan siapa yang bisa memengaruhi siapa.

Keputusan memilih pola punya dua konsekuensi. Pertama, *customer-supplier* hanya mungkin bila hulu bersedia mendengar. Di dalam Medikara, tim pendaftaran dan tim penagihan berada di bawah direksi yang sama, sehingga kebutuhan penagihan dapat dijadwalkan dalam rencana kerja pendaftaran. Kedua, terhadap pihak luar yang jauh lebih besar, pilihan realistisnya hanya *conformist* atau *anticorruption layer*. Satu rumah sakit hanyalah satu dari banyak fasilitas kesehatan yang terhubung ke BPJS Kesehatan dan SATUSEHAT, sehingga tidak dapat meminta perubahan model.

Kasus Medikara Health: BPJS Kesehatan dan SATUSEHAT

Saat ini modul klaim di kelima SIMRS bersikap *conformist* terhadap BPJS Kesehatan. Kode, nama kolom, dan status dari layanan web BPJS Kesehatan dipakai langsung di layar, tabel, dan laporan. Misalkan dalam satu tahun BPJS Kesehatan mengubah ketentuan teknis layanannya tiga kali. Setiap perubahan dikerjakan di lima versi SIMRS oleh tiga vendor berbeda. Rata-rata dibutuhkan 30 hari kerja sampai seluruh rumah sakit siap, dan selama masa itu sebagian berkas klaim tertunda.

Integrasi ke SATUSEHAT sedikit lebih baik. Adaptor SATUSEHAT yang sudah berjalan menerjemahkan data kunjungan SIMRS ke *resource* FHIR, misalnya Patient dan Encounter, sebelum dikirim. Adaptor ini sudah berfungsi sebagai *anticorruption layer*, walaupun tidak dirancang dengan sengaja. Masih ada aturan klinis yang diselipkan di dalamnya, misalnya penentuan jenis kunjungan.

Usulan untuk kondisi target ada dua. Pertama, satu *anticorruption layer* bersama dipasang di depan konteks klaim, sehingga perubahan dari BPJS Kesehatan cukup dikerjakan di satu tempat. Kedua, adaptor SATUSEHAT dirapikan menjadi *anticorruption layer* resmi, dan aturan klinis dikembalikan ke konteks pelayanan klinis. Biaya tambahannya satu komponen yang harus dipelihara tim integrasi. Perhitungan untung ruginya menjadi bahan Latihan 3.

Pendalaman Teknis (opsional): Struktur Anticorruption Layer

Evans menggambarkan *anticorruption layer* sebagai gabungan tiga bagian [47]. Ketiganya dapat dipetakan ke integrasi klaim Medikara sebagai berikut.

1. *Facade*: antarmuka sederhana ke layanan web BPJS Kesehatan yang menyembunyikan rincian alamat, otentikasi, dan versi.
2. *Adapter*: komponen yang menerima permintaan dalam bahasa konteks klaim, misalnya "ajukan klaim kunjungan ini", lalu memanggil *facade*.
3. *Translator*: fungsi yang menerjemahkan kode dan status BPJS Kesehatan ke istilah konteks klaim, misalnya status "dikembalikan untuk perbaikan".

Pengujian dipusatkan pada *translator*. Setiap perubahan dari hulu cukup ditambahkan sebagai kasus uji baru: contoh respons lama dan baru, lalu hasil terjemahan yang diharapkan. Bila *anticorruption layer* dijalankan sebagai layanan terpisah, ia ikut menambah satu titik gagal, sehingga perlu antrean dan percobaan ulang. Rinciannya dibahas pada Bab 9.

6.5 EventStorming sebagai Lokakarya Kolaboratif

EventStorming adalah format lokakarya yang dikembangkan Alberto Brandolini untuk mempelajari domain bisnis secara bersama-sama [29]. Peserta menulis *domain event*, yaitu kejadian penting dalam bisnis yang ditulis dalam bentuk lampau, misalnya "Pasien terdaftar" atau "Klaim diajukan", di catatan tempel berwarna oranye. Catatan itu ditempel di dinding panjang dan diurutkan dari kiri ke kanan mengikuti waktu. Dinding yang penuh memperlihatkan alur bisnis yang selama ini tersebar di kepala banyak orang. Brandolini membedakan tiga format: Big Picture untuk memahami satu lini bisnis secara utuh, Process Modelling untuk menelaah satu proses, dan Software Design untuk merancang perangkat lunak di dalam satu *bounded context* [29, 43]. Bab ini memakai Big Picture yang dipersempit pada satu alur layanan pasien.

Domain event dipilih sebagai titik awal karena paling mudah disepakati. Pakar bisnis terbiasa bercerita dengan kejadian: "setelah pasien terdaftar, perawat memanggil". Mereka jarang terbiasa bercerita dengan tabel dan kolom. Akibatnya, peserta non-teknis dapat menyumbang sama banyaknya dengan peserta teknis.

Analoginya seperti panitia pernikahan yang menyusun kronologi hari H di papan tulis untuk evaluasi. Setiap seksi menulis kejadian yang ditanganinya: tamu datang, akad dimulai, katering tiba, foto keluarga selesai. Kertas ditempel berurutan, lalu urutannya diperbaiki bersama. Celah langsung terlihat: katering tiba setelah

tamu mulai makan, karena seksi konsumsi menunggu kabar dari seksi acara yang tidak pernah dikirim. Tidak ada satu seksi pun yang tahu seluruh cerita, tetapi bersama mereka tahu.

Selain *domain event*, papan EventStorming memakai beberapa jenis catatan lain dengan warna tetap. Tabel 6.5 mendaftar elemen yang dipakai di bab ini, mengikuti lembar ringkas DDD Crew [43].

Tabel 6.5: Elemen papan EventStorming dan warna catatannya

Elemen	Warna	Bentuk tulisan	Contoh Medikara
Domain event	Oranye	Kata kerja bentuk lampau	Pasien terdaftar
Command	Biru	Kata kerja perintah	Daftarkan pasien
Aktor	Kuning, kecil	Peran orang, bukan nama orang	Petugas loket
Sistem eksternal	Merah muda, lebar	Nama sistem	Layanan web BPJS Kesehatan
Policy	Ungu muda	"Setiap kali …, maka …"	Setiap kali hasil kritis tervalidasi, dokter diberi tahu
Informasi	Hijau	Informasi untuk mengambil keputusan	Daftar antrean poliklinik
Hot spot	Merah muda terang	Pertanyaan, masalah, atau perselisihan	Siapa menutup kunjungan bila pasien pulang sebelum membayar?

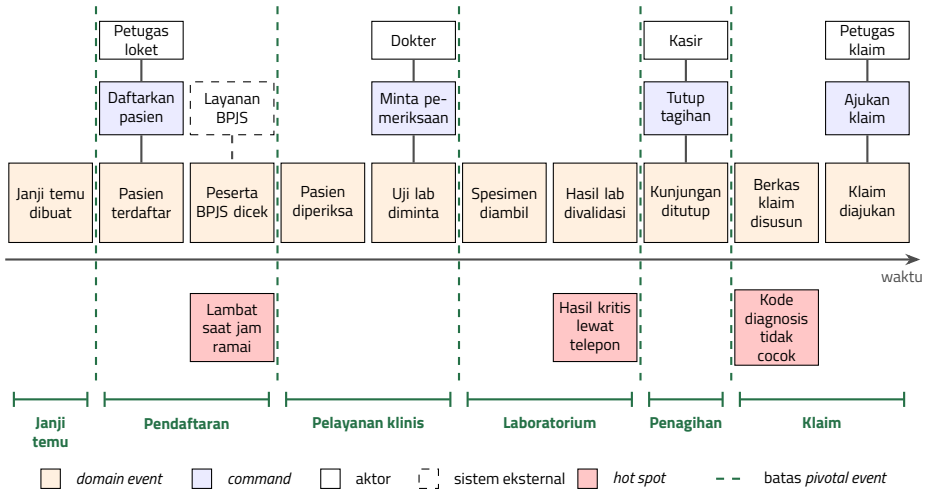
Lokakarya membutuhkan empat peran. Pakar domain adalah orang yang mengerjakan atau mengawasi proses, misalnya kepala pendaftaran, perawat, dokter, dan kasir. Tugasnya bercerita dan mengoreksi. Fasilitator menjaga aturan main, mengajukan pertanyaan, tidak memihak, dan memastikan setiap orang ikut menempel. Pencatat istilah memegang glosarium dan mencatat istilah yang diperdebatkan. Penjaga waktu memastikan setiap langkah selesai tepat waktu, lalu merangkum hasil di akhir sesi.

Urutan kerja lokakarya Big Picture yang dipersempit terdiri dari lima langkah:

1. **Eksplorasi bebas.** Selama sekitar 20 menit, setiap peserta menulis *domain event* yang ia ketahui tanpa diskusi. Jumlah lebih penting daripada kerapian.
2. **Menegakkan urutan waktu.** Peserta menyusun catatan menjadi satu garis waktu, membuang duplikat, dan menandai *pivotal event*, yaitu kejadian penting yang menandai pergantian tahap, misalnya "Kunjungan ditutup".
3. **Menandai *hot spot*.** Alur dibaca dari kiri ke kanan. Setiap perdebatan yang tidak selesai dalam dua menit ditulis sebagai *hot spot*, lalu ditinggalkan.
4. **Menambah pemicu.** *Command*, aktor, sistem eksternal, dan *policy* ditempel di dekat *event* yang dipicunya.

5. **Mengelompokkan menjadi kandidat *bounded context*.** *Event* yang melayani satu tujuan dan memakai satu bahasa dikelompokkan. *Pivotal event*, pergantian aktor, dan pergantian makna istilah menjadi petunjuk batas.

Gambar 6.5 menunjukkan papan ringkas hasil kelima langkah untuk alur rawat jalan. Warna pada gambar memakai palet modul, sedangkan warna catatan fisik mengikuti Tabel 6.5.



Gambar 6.5: Papan EventStorming ringkas dari pendaftaran rawat jalan sampai klaim BPJS Kesehatan, dengan *pivotal event* sebagai petunjuk batas *bounded context*

6.5.1 Menjalankan Lokakarya Daring

Tim capstone jarang bisa berkumpul di depan satu dinding, karena tatap muka hanya pada hari Sabtu. Lokakarya daring dengan papan virtual menjadi pilihan yang wajar. Beberapa penyesuaian membantu:

- Bagi lokakarya menjadi dua sesi masing-masing 120 menit, bukan satu hari penuh. Perhatian peserta di depan layar lebih cepat habis daripada di depan dinding.
- Siapkan templat papan dengan garis waktu kosong dan warna catatan yang dikunci, agar peserta tidak memilih warna sendiri.
- Batasi peserta enam sampai sepuluh orang, dan minta setiap orang mencoba menempel catatan sebelum sesi dimulai.
- Minta kamera menyala. Fasilitator mengatur giliran bicara, dibantu satu pendamping yang memantau ruang obrolan.
- Buka glosarium di sisi papan, dan ekspor papan sebagai gambar di akhir

setiap sesi.

Panduan lengkap, agenda menit per menit, dan daftar periksa ada di `sources/chapter06/panduan-fasilitasi-eventstorming.md`.

Kasus Medikara Health: Dari Pendaftaran Rawat Jalan sampai Klaim BPJS

Misalkan tim menjalankan lokakarya di rumah sakit Bogor dalam dua sesi daring masing-masing 120 menit. Pesertanya sembilan orang: kepala pendaftaran, perawat poliklinik, dokter umum, analis laboratorium, apoteker, kasir, petugas klaim, serta dua anggota tim TI sebagai fasilitator dan pencatat istilah. Sesi pertama menghasilkan 64 *domain event* dan 11 *hot spot*. Setelah duplikat dibuang, tersisa 47 *domain event*. Sesi kedua menambahkan pemicu, lalu mengelompokkan *event* menjadi enam kandidat *bounded context*: janji temu, pendaftaran, pelayanan klinis, laboratorium, penagihan, dan klaim. Apotek dibahas pada sesi tambahan karena apoteker harus kembali bertugas.

Tiga *hot spot* paling ramai tampak di Gambar 6.5. Pertama, verifikasi kepesertaan BPJS Kesehatan lambat pada jam ramai, sehingga antrean loket memanjang. Kedua, hasil laboratorium kritis masih disampaikan lewat telepon karena belum masuk rekam medis tepat waktu. Ketiga, berkas klaim dikembalikan karena kode diagnosis di klaim tidak sama dengan catatan dokter. Misalkan sekitar 7 persen berkas klaim rawat jalan dikembalikan setiap bulan, dan setiap berkas butuh rata-rata dua hari kerja untuk diperbaiki.

Hot spot ketiga juga membuka temuan bahasa. Kata "diagnosis" ternyata punya dua makna: diagnosis klinis yang ditulis dokter, dan kode diagnosis yang disusun petugas klaim agar sesuai aturan BPJS Kesehatan. Keduanya tersimpan di tabel yang sama dan sama-sama boleh diubah. Inilah sumber berkas yang tidak cocok.

6.6 Dari Bounded Context ke Kandidat Layanan

Kandidat layanan adalah usulan unit perangkat lunak yang kelak dapat dibangun, di-*deploy*, dan diubah secara mandiri. Kandidat ini diturunkan dari *bounded context*, tetapi tidak otomatis satu banding satu. Newman menyarankan batas layanan mengikuti domain bisnis, agar satu perubahan bisnis cukup menyentuh satu layanan [106]. Skelton dan Pais menyebut batas seperti ini *fracture plane*, yaitu garis alami tempat sistem dapat dibelah tanpa merusak aliran kerja tim. *Bounded context* adalah salah satunya, bersama kepatuhan regulasi, laju perubahan, isolasi kinerja, dan lokasi tim [131].

Analoginya seperti dapur restoran besar yang dibagi menjadi beberapa stasiun: stasiun panggangan, stasiun sayur, dan stasiun kue. Setiap stasiun punya bahan, alat, koki penanggung jawab, dan ritmenya sendiri. Kue disiapkan sejak subuh, sedangkan panggangan paling sibuk saat makan siang. Namun tidak ada restoran yang membuat stasiun khusus untuk garam. Stasiun yang terlalu kecil membuat koki sibuk saling mengoper piring.

Lima heuristik membantu menurunkan kandidat layanan. Tabel 6.6 memuat pertanyaan pemandu dan contohnya di Medikara Health.

Tabel 6.6: Heuristik menurunkan kandidat layanan dari *bounded context*

Heuristik	Pertanyaan pemandu	Tanda layak dipisah	Contoh Medikara
Kepemilikan data	Siapa satu-satunya yang boleh mengubah data ini?	Data punya satu pemilik jelas; konteks lain cukup membaca lewat Application Programming Interface (API) atau *event*	Hasil laboratorium hanya diubah laboratorium
Laju perubahan	Seberapa sering aturannya berubah, dan karena siapa?	Berubah jauh lebih sering atau karena pihak luar	Aturan klaim berubah mengikuti BPJS Kesehatan
Kebutuhan skala	Apakah bebannya naik turun berbeda dari bagian lain?	Puncak beban berbeda waktu atau besarnya	Janji temu naik sekitar 60 persen saat program vaksinasi
Tim pengelola	Tim mana yang menjaganya dalam jangka panjang?	Ada satu tim yang sanggup memilikinya penuh	Tim kanal digital untuk Medikara+
Kepatuhan dan risiko	Apakah bagian ini memegang data atau aturan dengan tuntutan khusus?	Kontrol akses dan jejak audit berbeda dari bagian lain	Rekam medis elektronik

Hubungan antara *bounded context* dan layanan punya tiga bentuk. Bentuk pertama, satu konteks menjadi satu layanan. Ini bentuk yang paling umum. Bentuk kedua, satu konteks dipecah menjadi beberapa layanan karena kebutuhan skala atau risikonya berbeda, misalnya pengiriman notifikasi janji temu yang bebannya melonjak dipisah dari pengelolaan jadwal. Bentuk ketiga, beberapa konteks tetap berada dalam satu aplikasi yang dipasang bersama, tetapi modulnya dipisah tegas. Bentuk ini disebut *modular monolith* dan dibahas pada Bab 7.

Dua kesalahan sering muncul. Kesalahan pertama memecah menurut entitas, misalnya layanan pasien, layanan dokter, dan layanan obat. Hampir setiap alur bisnis lalu memanggil ketiganya, sehingga layanan saling bergantung erat. Kesalahan kedua memecah menurut lapisan teknis, misalnya layanan tampilan, layanan logika, dan layanan data. Setiap perubahan fitur menyentuh ketiga lapisan dan ketiga tim.

Capability map dari mata kuliah IT40513 menjadi masukan yang berguna di sini. *Capability map* memperlihatkan kemampuan bisnis yang dibutuhkan organisasi, sehingga membantu menebak *subdomain*. Namun satu *capability* tidak otomatis menjadi satu *bounded context*. Penyusunan *capability map* dibahas rinci di IT40513.

Dua contoh dari Medikara Health:

1. Janji temu Medikara+ melonjak sekitar 60 persen setiap kali program vaksinasi diumumkan, sedangkan pendaftaran di loket tidak. Perbedaan kebutuhan skala ini menjadikan janji temu kandidat layanan tersendiri yang dapat diskalakan tanpa ikut menambah kapasitas SIMRS.
2. Pendaftaran dan pelayanan klinis berbagi identitas pasien dengan erat dan dikelola tim yang sama. Memisahkan keduanya sekarang hanya menambah panggilan jaringan. Keduanya tetap satu aplikasi dengan modul yang dipisah tegas, lalu ditinjau ulang setelah satu tahun.

Kasus Medikara Health: Daftar Kandidat Layanan Awal

Tim memakai hasil lokakarya Bogor dan peta *subdomain* untuk menyusun daftar kandidat layanan awal pada Tabel 6.7. Daftar ini masih kandidat. Keterkaitan di antaranya didiagnosis pada Bab 7, dan pilihan *runtime*-nya ditentukan pada Bab 8.

Keputusan ini punya dua konsekuensi. Pertama, klaim dan janji temu menjadi calon awal pemisahan, karena keduanya punya alasan yang paling kuat: aturan dari pihak luar dan lonjakan beban. Urutan akhirnya ditetapkan pada peta jalan Bab 14. Kedua, rekam medis justru dipecah paling akhir. Bagian ini paling bernilai dan paling sensitif, sehingga perubahannya ditunda sampai tim sudah berpengalaman dari pemisahan awal.

Pendalaman Teknis (opsional): Mencocokkan Konteks dengan Tabel Basis Data

Batas yang digambar di papan perlu diuji pada sistem yang ada. Caranya, setiap modul aplikasi dikelompokkan ke satu *bounded context*, lalu akses modul ke tabel basis data dicatat dari dokumentasi, kode, atau catatan *query*. Berkas `sources/chapter06/contoh-akses-tabel-simrs.csv` berisi data rekaan 34 akses dari 8 modul SIMRS rumah sakit Jakarta ke 14 tabel. Listing 6.1 mengumpulkan konteks penulis dan pembaca setiap tabel.

```
def kelompokkan_akses(daftar_akses):
  """Mengumpulkan konteks penulis dan pembaca untuk setiap tabel.

  Himpunan dipakai agar satu konteks yang punya beberapa modul penulis
      tidak
```

Tabel 6.7: Kandidat layanan awal Medikara Health dari alur rawat jalan

Bounded context	**Jenis**	**Keputusan awal**	**Alasan utama**
Janji temu	*Core*	Layanan sendiri	Beban melonjak saat vaksinasi; dimiliki tim kanal digital
Pendaftaran	*Supporting*	Modul dalam aplikasi klinis	Berbagi identitas pasien dengan pelayanan klinis; tim sama
Pelayanan klinis dan rekam medis	*Core*	Aplikasi klinis, dipecah paling akhir	Data paling sensitif; kontrol akses dan jejak audit ketat
Laboratorium	*Supporting*	Layanan sendiri	Satu laboratorium rujukan melayani seluruh jaringan; hasil pemeriksaan punya pemilik jelas
Penagihan	*Supporting*	Modul terpisah	Klasifikasinya diuji pada Latihan 2
Klaim	*Supporting*	Layanan sendiri dengan *anticorruption layer*	Aturan berubah mengikuti BPJS Kesehatan
Akuntansi	*Generic*	Tetap di ERP keuangan	Sudah terpusat dan stabil

```
    terhitung dua kali. Dua modul dalam satu konteks memang boleh berbagi
        tabel.
    """
    penulis = defaultdict(set)
    pembaca = defaultdict(set)
    for akses in daftar_akses:
      tabel = akses["tabel"]
      if akses["akses"] == AKSES_TULIS:
        penulis[tabel].add(akses["konteks"])
      elif akses["akses"] == AKSES_BACA:
        pembaca[tabel].add(akses["konteks"])
    return penulis, pembaca
```

Listing 6.1: Pengelompokan akses tabel per konteks, tersedia di `sources/chapter06/cocokkan-tabel-konteks.py`

Skrip dijalankan dengan `python cocokkan-tabel-konteks.py contoh-akses-tabel-simrs.csv`. Hasilnya: hanya 9 dari 14 tabel yang punya satu konteks penulis. Lima tabel ditulis lebih dari satu konteks, yaitu `item_tagihan` (apotek, laboratorium, penagihan), `kunjungan` (pendaftaran, pelayanan klinis, penagihan), `diagnosis` (pelayanan klinis, klaim), `pasien` (pendaftaran, pelayanan klinis), dan `penjamin_pasien` (pendaftaran, penagihan). Skrip juga mencatat 10 pembacaan lintas konteks yang kelak perlu diganti dengan API atau *event*. Tabel `diagnosis` yang ditulis dua konteks cocok dengan *hot spot* kode diagnosis di lokakarya Bogor. Temuan seperti ini memperkuat batas yang diusulkan, atau justru

menunjukkan bahwa batasnya perlu digeser.

6.7 Aktivitas: Lokakarya EventStorming dan Context Map

Tujuan. Tim menemukan *bounded context* kondisi saat ini dari satu alur layanan pasien, lalu menggambarkan hubungannya dalam *context map*. Keduanya menjadi bagian tonggak capstone C2 (Kondisi Saat Ini).

Pembagian peran.

- Anggota yang memahami proses bisnis, terutama yang berlatar kesehatan atau manajemen, menjadi pakar domain. Bila memungkinkan, tim mengundang satu atau dua narasumber dari organisasi pilihan.
- Anggota berlatar teknis menjadi fasilitator dan pencatat istilah.
- Satu anggota menjaga waktu dan merangkum hasil di akhir setiap sesi.

Langkah kerja.

1. Pilih satu alur layanan pasien, misalnya dari pendaftaran rawat jalan sampai klaim BPJS Kesehatan. Bawa *capability map* dari IT40513 dan peta dependensi dari Bab 4 sebagai bahan awal.
2. Tempel *domain event* secara berurutan di papan virtual, mengikuti agenda sesi 1 di `sources/chapter06/panduan-fasilitasi-eventstorming.md`. Tandai *pivotal event* dan *hot spot*.
3. Tambahkan *command*, aktor, dan sistem eksternal. Catat setiap istilah bermakna ganda di glosarium.
4. Kelompokkan *event* menjadi *bounded context*. Beri setiap konteks nama, jenis *subdomain*, dan tim pemilik saat ini.
5. Gambar *context map* kondisi saat ini dengan `sources/chapter06/templat-context-map.drawio`, termasuk hubungan dengan BPJS Kesehatan dan SATUSEHAT. Setiap panah diberi arah hulu-hilir dan nama pola.

Luaran. Ekspor papan EventStorming, glosarium paling sedikit sepuluh istilah, peta *subdomain*, dan *context map* kondisi saat ini, ditambah ringkasan satu halaman tentang tiga *hot spot* terpenting. Luaran ini masuk ke C2 sebagai peta domain dan *context map* kondisi saat ini, dan dipakai lagi untuk mendiagnosis keterkaitan pada Bab 7.

Kriteria penilaian. Tabel 6.8 memuat kriteria singkat yang dipakai saat tinjauan Sabtu.

Tabel 6.8: Kriteria penilaian Aktivitas Bab 06

Kriteria	Bobot	Bukti yang dicari
Kelengkapan alur dan *event*	25 persen	Alur utuh dari awal sampai akhir; *event* berbentuk lampau; *hot spot* tercatat
Kualitas batas *bounded context*	30 persen	Setiap batas dijelaskan dengan *pivotal event*, bahasa, atau pemilik
Context map	25 persen	Semua hubungan punya arah dan pola, termasuk BPJS Kesehatan dan SATUSEHAT
Glosarium dan kolaborasi	20 persen	Istilah bermakna ganda tercatat; semua peran ikut menyumbang

Etika. Lokakarya dengan organisasi nyata hanya dijalankan seizin organisasi. Nama orang, data pasien, dan data karyawan tidak boleh ditulis di papan. Contoh kejadian cukup ditulis umum, misalnya "Pasien terdaftar", bukan nama pasien yang terdaftar.

Pendalaman teknis (opsional). Anggota berlatar teknis mencocokkan *bounded context* hasil lokakarya dengan modul dan tabel basis data aplikasi yang ada, memakai `sources/chapter06/cocokkan-tabel-konteks.py` seperti pada kotak pendalaman seksi 6.6. Daftar akses diisi dari dokumentasi atau wawancara dengan tim TI, bukan dengan memindai sistem tanpa izin.

6.8 Ringkasan

DDD berangkat dari bisnis, bukan dari teknologi. Domain organisasi dibagi menjadi *subdomain* yang ditemukan dari cara kerja bisnis. *Core subdomain* adalah pembeda dan layak dibangun serta dikuasai sendiri. *Supporting subdomain* diperlukan dan khas, tetapi bukan pembeda. *Generic subdomain* cukup dibeli atau disewa. Klasifikasi ini menentukan tempat investasi dan orang terbaik ditempatkan. Kasus klaim BPJS Kesehatan menunjukkan bahwa nilai yang besar tidak sama dengan pembeda.

Ubiquitous language menyatukan kosakata pakar bisnis dan tim teknis di dalam satu batas. Batas itu adalah *bounded context*: tempat satu model dan satu bahasa berlaku konsisten. Kata "pasien" boleh bermakna orang yang terdaftar di pendaftaran, pihak yang ditagih di penagihan, dan subjek spesimen di laboratorium. Yang dibagi hanya identitas penghubung. Insiden kolom penjamin di Depok memperlihatkan akibatnya bila ketiga makna dijejalkan ke satu tabel.

Context map memetakan hubungan antarkonteks beserta arah hulu dan hilirnya. Pola *customer-supplier*, *conformist*, *anticorruption layer*, *shared kernel*, dan *open host service* dipilih menurut kenyataan hubungan antartim, sejalan dengan hukum

Conway. Terhadap pihak luar yang besar seperti BPJS Kesehatan dan SATUSEHAT, pilihan realistisnya *conformist* atau *anticorruption layer*, dan yang kedua melindungi model sendiri dari perubahan di hulu.

EventStorming adalah cara kolaboratif untuk menemukan semua itu. Pakar domain dan tim teknis menempel *domain event* berurutan, menandai *hot spot*, menambah pemicu, lalu mengelompokkan *event* menjadi kandidat *bounded context*. Dari situ kandidat layanan diturunkan dengan heuristik kepemilikan data, laju perubahan, kebutuhan skala, tim pengelola, serta kepatuhan dan risiko. Satu konteks tidak selalu menjadi satu layanan.

Pesan utama bab ini: batas layanan yang baik ditemukan dari bahasa dan alur bisnis, bukan dari daftar tabel atau struktur menu aplikasi lama. Memecah sistem sebelum batasnya dipahami hanya memindahkan kekusutan ke jaringan. Peta domain dan *context map* kondisi saat ini dari Aktivitas bab ini menjadi dasar diagnosis keterkaitan dan keputusan gaya arsitektur pada Bab 7.

6.9 Latihan

Latihan 1: Satu Kata, Tiga Makna

Latihan ini membuktikan bahwa perbedaan makna satu istilah dapat ditangani dengan batas yang jelas, bukan dengan memaksakan satu definisi. Pertanyaannya: kata "pasien" bisa bermakna berbeda di bagian pendaftaran, keuangan, dan laboratorium. Jelaskan cara *bounded context* menangani perbedaan makna ini.

1. Baca kembali kasus "Satu Kolom, Tiga Bagian" dan Gambar 6.3.
2. Untuk setiap konteks, tulis makna istilah, atribut yang benar-benar dibutuhkan, dan aturan yang hanya berlaku di konteks itu, seperti pada Tabel 6.9.
3. Tentukan identitas penghubung yang dibagi antarkonteks, beserta konteks yang menerbitkannya.
4. Pilih satu istilah bermakna ganda di organisasi pilihan tim, misalnya "pelanggan", "produk", atau "kunjungan", lalu isi Tabel 6.10.
5. Tulis dua paragraf yang menjelaskan cara *bounded context* mencegah insiden serupa.

Pertanyaan analisis:

1. Pada contoh, aturan mana yang memicu insiden Depok ketika diberlakukan di luar konteksnya? Pada titik alur mana aturan itu seharusnya berlaku?
2. Atribut mana yang muncul di lebih dari satu baris tabel hasil tim? Konteks mana yang seharusnya menjadi pemilik atribut itu, dan bagaimana konteks

Tabel 6.9: Contoh Latihan 1: makna "pasien" di tiga konteks Medikara Health

Konteks	Makna "pasien"	Atribut yang dibutuhkan	Aturan yang hanya berlaku di sini
Pendaftaran	Orang yang dikenali dan boleh dilayani	Nama, tanggal lahir, nomor kependudukan, alamat, kontak darurat	Satu orang hanya punya satu nomor rekam medis di seluruh jaringan
Penagihan	Pihak yang ditagih beserta penjaminnya	Penjamin, nomor kepesertaan, hak kelas, plafon, saldo tagihan	Penjamin wajib jelas sebelum tagihan ditutup
Laboratorium	Subjek pemeriksaan pada satu spesimen	Nomor permintaan, jenis spesimen, waktu pengambilan, umur, jenis kelamin	Nilai rujukan dipilih menurut umur dan jenis kelamin
Identitas penghubung	Nomor rekam medis	Diterbitkan konteks pendaftaran	Konteks lain hanya menyimpan dan merujuknya

Tabel 6.10: Lembar isian Latihan 1

Konteks	Makna istilah	Atribut yang dibutuhkan	Aturan yang hanya berlaku di sini
…	…	…	…
…	…	…	…
…	…	…	…
Identitas penghubung	…	…	…

lain memperolehnya?

Latihan 2: Penagihan, Core atau Bukan

Latihan ini membuktikan bahwa klasifikasi *subdomain* ditentukan oleh kriteria yang dapat diperiksa, bukan oleh besarnya nilai uang. Tentukan apakah layanan penagihan di Medikara Health termasuk *core*, *supporting*, atau *generic subdomain*. Jelaskan konsekuensinya terhadap keputusan membangun sendiri atau membeli.

1. Pelajari contoh klasifikasi apotek pada Tabel 6.11.
2. Kumpulkan bahan tentang penagihan: alur tagihan, jenis penjamin, aturan diskon dan paket, serta produk penagihan yang tersedia di pasar.
3. Jawab setiap pertanyaan kriteria untuk penagihan, lalu tentukan arah klasifikasinya.
4. Isi Tabel 6.12, termasuk baris kesimpulan dan keputusan bangun atau beli.
5. Bandingkan hasilnya dengan usulan awal pada Gambar 6.2.

Tabel 6.11: Contoh Latihan 2: klasifikasi *subdomain* apotek Medikara Health

Kriteria	**Pertanyaan**	**Jawaban untuk apotek**	**Arah**
Pembeda	Apakah pasien atau mitra memilih Medikara karena bagian ini?	Jarang; harga dan jenis obat serupa dengan apotek lain	Bukan *core*
Kerumitan	Seberapa rumit aturannya?	Sedang: interaksi obat, stok, kedaluwarsa, resep BPJS Kesehatan	*Supporting* atau *generic*
Solusi pasar	Adakah produk jadi yang sesuai?	Ada, tetapi harus terhubung dengan resep elektronik dari pelayanan klinis	Condong *generic*
Laju perubahan	Seberapa sering berubah, dan karena siapa?	Daftar obat dan harga bulanan; aturan klinis jarang berubah	Rendah
Kekhasan	Apa yang khas Medikara?	12 apotek melayani resep dari 5 rumah sakit dan 18 klinik, termasuk antar obat lewat Medikara+	*Supporting*
Kesimpulan	*Supporting*. Beli produk apotek; bangun sendiri hanya integrasi resep elektronik dan layanan antar obat.		

Tabel 6.12: Lembar isian Latihan 2: klasifikasi *subdomain* penagihan

Kriteria	**Pertanyaan**	**Jawaban untuk penagihan**	**Arah**
Pembeda	Apakah pasien atau mitra memilih Medikara karena bagian ini?	…	…
Kerumitan	Seberapa rumit aturannya?	…	…
Solusi pasar	Adakah produk jadi yang sesuai?	…	…
Laju perubahan	Seberapa sering berubah, dan karena siapa?	…	…
Kekhasan	Apa yang khas Medikara?	…	…
Kesimpulan	…		

Pertanyaan analisis:

1. Pada contoh apotek, kriteria mana yang paling menentukan kesimpulan? Apakah kriteria yang sama juga paling menentukan pada penagihan?
2. Misalkan Medikara kelak menjual paket langganan kesehatan karyawan kepada perusahaan, dengan skema tagihan bulanan per karyawan. Apakah klasifikasi penagihan bergeser? Jelaskan dampaknya pada keputusan membangun atau membeli.

Latihan 3: Melindungi Klaim dengan Anticorruption Layer

Latihan ini membuktikan bahwa manfaat *anticorruption layer* dapat dihitung dari jumlah tempat yang harus diubah setiap kali pihak luar berubah. Pertanyaannya: mengapa integrasi dengan sistem eksternal seperti BPJS Kesehatan sebaiknya dilindungi oleh *anticorruption layer*?

1. Baca kembali kasus "BPJS Kesehatan dan SATUSEHAT" dan contoh pada Tabel 6.13.
2. Pilih satu sistem eksternal lain, misalnya SATUSEHAT atau sistem klaim asuransi swasta.
3. Daftar tiga perubahan yang pernah terjadi atau mungkin terjadi pada sistem itu. Gunakan catatan rilis atau pengumuman resmi bila tersedia.
4. Hitung tempat yang harus diubah dan perkiraan waktunya, tanpa dan dengan *anticorruption layer*.
5. Isi Tabel 6.14, lalu catat biaya tambahan yang muncul bila *anticorruption layer* dibangun.

Tabel 6.13: Contoh Latihan 3: tiga perubahan layanan BPJS Kesehatan dalam setahun, waktu dalam hari kerja (angka rekaan)

Perubahan dari hulu (misalkan)	**Tanpa *anticorruption layer***		**Dengan *anticorruption layer***	
	Tempat	**Waktu**	**Tempat**	**Waktu**
Kode status klaim baru ditambahkan (layar, tabel, laporan di 5 SIMRS)	15	30	1	5
Format tanggal pada respons berubah	5	20	1	3
Versi layanan baru dengan alamat dan cara otentikasi berbeda	5	40	1	10
Total setahun	**25**	**90**	**3**	**18**

Tabel 6.14: Lembar isian Latihan 3

Perubahan dari hulu	**Tanpa *anticorruption layer***		**Dengan *anticorruption layer***	
	Tempat	**Waktu**	**Tempat**	**Waktu**
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
Total	…	…	…	…

Pertanyaan analisis:

1. Pada contoh, berapa kali lebih cepat perubahan selesai dengan *anticorruption layer*? Biaya apa yang tidak tampak di tabel, tetapi tetap harus ditanggung?

2. Pada sistem pilihan tim, perubahan mana yang paling diuntungkan oleh *anticorruption layer*, dan mengapa?
3. Penagihan bersikap *conformist* terhadap ERP keuangan. Mengapa pola itu tetap wajar di sana, padahal terhadap BPJS Kesehatan tidak?

Bab 7

Monolit, Modular Monolith, dan Microservices

Tujuan Pembelajaran

- Membandingkan monolit, *modular monolith*, dan *microservices* dari sisi manfaat dan biayanya.
- Mendiagnosis sumber keterkaitan (*coupling*) pada aplikasi yang ada.
- Menuliskan keputusan gaya arsitektur dalam format ADR.

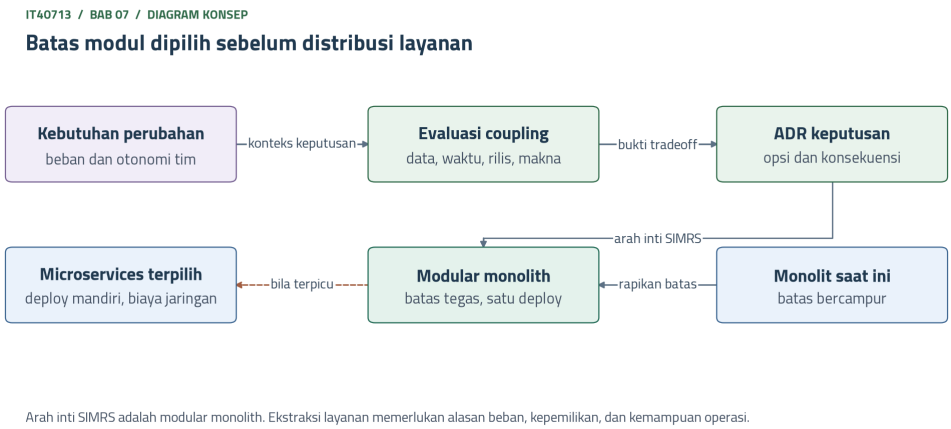
7.1 Pendahuluan

Pada pertengahan 2026 sebuah vendor mempresentasikan "SIMRS berbasis *microservices*" kepada direksi Medikara Health. Janjinya menarik. Lima SIMRS (Sistem Informasi Manajemen Rumah Sakit) yang berbeda versi akan diganti 32 layanan kecil yang dapat dirilis sendiri-sendiri, diskalakan sendiri-sendiri, dan dikerjakan banyak tim sekaligus. Antrean panjang, rilis yang lambat, dan biaya integrasi yang tinggi disebut akan selesai bersamaan. Direktur keuangan mengajukan pertanyaan sederhana: bila *microservices* sebagus itu, mengapa ada perusahaan yang justru kembali ke monolit? Direktur teknologi informasi (TI) belum punya jawaban yang dapat dipertanggungjawabkan.

Pertanyaan itu adalah inti bab ini. *Microservices* adalah satu gaya arsitektur, bukan tujuan. Gaya ini menolong organisasi yang membutuhkan banyak tim bekerja mandiri pada bagian sistem yang berbeda, tetapi menambah beban pada jaringan, data, pengujian, dan operasi. Di antara monolit dan *microservices* ada titik tengah, *modular monolith*, yang sering terlewat dari presentasi vendor. Tiga pilihan

itu perlu dinilai melalui ketergantungan data, waktu, rilis, dan makna. Inti SIMRS diarahkan menjadi modular monolith sebelum ekstraksi layanan yang memang memerlukan distribusi.

Gambar 7.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 7.1: Batas modul dipilih sebelum distribusi layanan

Bab ini disusun dalam lima langkah: ciri tiga gaya arsitektur, keterkaitan dan biaya sistem terdistribusi, hukum Conway tentang hubungan struktur tim dengan struktur sistem, tanda-tanda *microservices* tidak tepat, serta Architecture Decision Record (ADR) sebagai catatan keputusan. Rujukan utamanya adalah definisi *microservices* dari Lewis dan Fowler [87], buku Newman [106], buku Richards dan Ford [124], makalah Conway [40], kajian Jamshidi dkk. [79], dan tulisan Nygard tentang ADR [108]. Hasil bab ini adalah diagnosis arsitektur aplikasi inti dan ADR gaya arsitektur, yang melengkapi tonggak capstone C2 untuk ditinjau menjelang Ujian Tengah Semester (UTS).

7.2 Tiga Gaya Arsitektur: Monolit, Modular Monolith, dan Microservices

Gaya arsitektur adalah cara menata bagian-bagian sistem dan hubungan antarbagiannya. Tiga gaya yang dibahas di sini dibedakan oleh dua pertanyaan. Pertama, berapa unit yang dirilis secara terpisah? Kedua, seberapa tegas batas antarbagian di dalam sistem dijaga? Jawaban atas dua pertanyaan itu menentukan siapa yang dapat bekerja tanpa menunggu siapa, dan berapa biaya yang harus dibayar untuk kemandirian tersebut.

7.2.1 Monolit

Monolit adalah sistem yang seluruh fungsinya dibangun, diuji, dan dirilis sebagai satu unit. Newman memandang monolit sebagai satuan rilis: bila satu bagian berubah, seluruh sistem dirilis ulang [106]. Umumnya semua bagian berjalan dalam satu proses dan berbagi satu basis data. Monolit bukan kata yang buruk. Bagi tim kecil dan domain yang belum stabil, monolit adalah bentuk yang paling sederhana dibangun, diuji, dan dioperasikan: satu tempat menyimpan kode, satu proses rilis, satu tempat mencari kesalahan.

Masalah monolit muncul ketika ukurannya tumbuh tanpa batas internal yang dijaga. Setiap bagian boleh memanggil bagian lain dan membaca tabel mana pun. Lama-kelamaan tidak ada yang berani mengubah satu bagian tanpa menguji semuanya, dan jadwal rilis ditentukan oleh bagian yang paling lambat siap.

Analoginya seperti rumah makan dengan satu dapur. Selama rumah makan masih kecil, satu dapur membuat semuanya cepat: juru masak saling membantu, bahan disimpan di satu lemari, dan pemilik mudah mengawasi. Ketika rumah makan tumbuh menjadi seratus kursi dengan empat jenis masakan, semua juru masak berdesakan di dapur yang sama, memakai kompor yang sama, dan saling menunggu. Dapurnya tidak salah. Ukurannya yang tidak lagi cocok dengan cara kerja itu.

7.2.2 Modular Monolith

Modular monolith tetap dirilis sebagai satu unit, tetapi isinya dibagi menjadi modul dengan batas yang tegas. Setiap modul punya antarmuka publik, dan modul lain hanya boleh memakai modul itu lewat antarmukanya. Data setiap modul dipisahkan, misalnya dalam skema basis data sendiri, sehingga modul lain tidak membaca tabelnya secara langsung. Newman menempatkan *modular monolith* sebagai variasi monolit yang modulnya dapat dikerjakan terpisah walaupun tetap digabungkan saat rilis [106]. Batas modul dijaga dengan aturan tertulis dan alat pemeriksa otomatis, bukan sekadar kesepakatan lisan.

Analoginya seperti rumah sakit dalam satu kompleks bersayap. Ada sayap rawat jalan, sayap laboratorium, dan sayap farmasi. Kompleksnya satu, atapnya satu, genset cadangannya satu. Namun setiap sayap punya pintu, petugas, dan gudangnya sendiri. Petugas laboratorium tidak masuk ke gudang farmasi untuk mengambil reagen. Ia mengajukan permintaan lewat loket farmasi.

7.2.3 Microservices

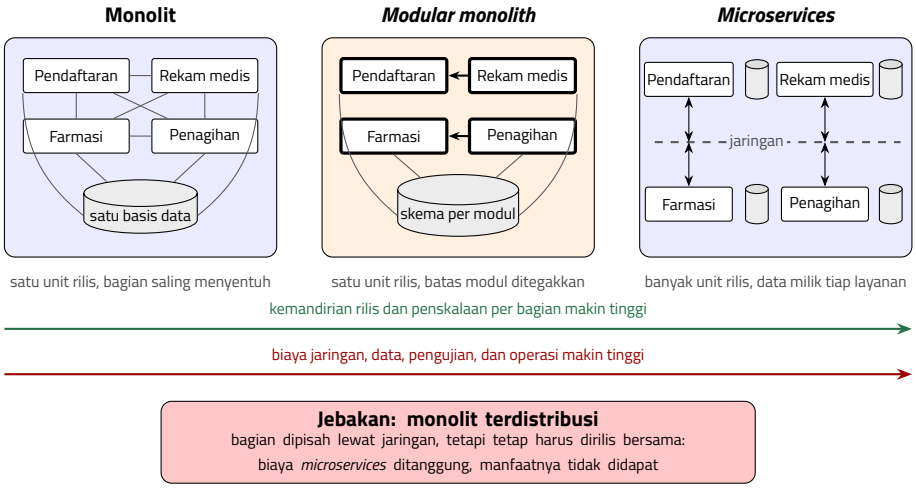
Lewis dan Fowler mendefinisikan *microservices* sebagai pendekatan membangun satu aplikasi sebagai sekumpulan layanan kecil. Setiap layanan berjalan dalam

prosesnya sendiri, berkomunikasi lewat mekanisme ringan seperti API (Application Programming Interface) berbasis web, dibangun mengikuti kapabilitas bisnis, dan dirilis secara mandiri lewat otomasi [87]. Mereka mencatat sembilan ciri umum, antara lain penataan menurut kapabilitas bisnis, pengelolaan data yang tersebar di setiap layanan, otomasi infrastruktur, dan rancangan yang siap menghadapi kegagalan. Newman merumuskannya lebih ringkas: layanan yang dapat dirilis secara mandiri dan dimodelkan di sekitar domain bisnis [106]. Kata kuncinya adalah dirilis secara mandiri. Ukuran kecil bukan tujuan; kemandirian itulah tujuannya.

Analoginya seperti permukiman rumah kecil yang dihubungkan jalan. Setiap rumah dapat direnovasi tanpa mengganggu tetangganya. Namun barang yang dulu cukup dipindah dari satu ruang ke ruang lain kini harus diangkut lewat jalan. Jalan bisa macet, kendaraan bisa mogok, dan kawasan itu membutuhkan petugas yang mengatur lalu lintas.

7.2.4 Spektrum, Bukan Pilihan Biner

Ketiga gaya ini sebaiknya dibaca sebagai titik pada satu spektrum, seperti diperlihatkan Gambar 7.2. Makin ke kanan, kemandirian rilis dan penskalaan per bagian makin tinggi. Biaya jaringan, data, pengujian, dan operasi juga ikut naik. Organisasi dapat berhenti di titik mana pun, dan dapat bergeser bertahap, misalnya memisahkan satu modul menjadi layanan tersendiri ketika data membuktikan perlunya.



Gambar 7.2: Tiga gaya arsitektur pada satu spektrum: manfaat dan biaya naik bersama, dengan monolit terdistribusi sebagai jebakan

Ada satu titik yang harus dihindari. Newman menyebutnya monolit terdistribusi: sistem yang terdiri atas beberapa layanan, tetapi seluruhnya harus dirilis

bersama [106]. Organisasi yang terjebak di sini membayar biaya jaringan seperti *microservices*, tetapi tetap kaku seperti monolit. Tabel 7.1 merangkum perbedaan ketiga gaya dari sisi yang biasa ditanyakan manajemen.

Tabel 7.1: Perbandingan monolit, *modular monolith*, dan *microservices*

Aspek	**Monolit**	***Modular monolith***	***Microservices***
Unit rilis	Satu	Satu	Satu per layanan
Batas antarbagian	Longgar, sering dilanggar	Tegas, dijaga alat pemeriksa	Tegas, dipisah jaringan
Data	Satu basis data bersama	Satu basis data, skema per modul	Basis data per layanan
Komunikasi antarbagian	Pemanggilan di dalam proses	Pemanggilan di dalam proses lewat antarmuka modul	Pesan lewat jaringan
Penskalaan	Seluruh sistem sekaligus	Seluruh sistem sekaligus	Per layanan
Dampak kegagalan satu bagian	Dapat menghentikan seluruh sistem	Dapat menghentikan seluruh sistem	Terbatas, bila dirancang tahan gagal
Syarat tim dan otomasi	Rendah	Sedang: disiplin menjaga batas	Tinggi: rilis otomatis, pemantauan, tim mandiri
Cocok untuk	Tim kecil, domain belum jelas	Beberapa tim, satu jadwal rilis masih dapat diterima	Banyak tim yang perlu rilis dan skala mandiri

Dua contoh kasus nyata:

1. Kode inti Shopify adalah salah satu basis kode Ruby on Rails terbesar, dikerjakan lebih dari seribu pengembang selama lebih dari sepuluh tahun. Tulisan teknik Shopify pada 2019 menjelaskan pilihan untuk mengubahnya menjadi *modular monolith*, bukan memecahnya menjadi *microservices*, dengan alasan setiap panggilan lewat jaringan menambah waktu tunggu dan mengurangi keandalan. Sekitar 6.000 kelas program dipetakan ke komponen seperti pesanan, pengiriman, persediaan, dan penagihan, lalu pelanggaran batas antarkomponen dipantau dengan alat buatan sendiri [150].
2. Segment, penyedia layanan pengolahan data pelanggan, pernah memecah sistem pengirimannya menjadi lebih dari 140 layanan, lalu menyatukannya kembali menjadi satu layanan [107]. Kisah ini dibahas rinci di seksi 7.5.

Kasus Medikara Health: SIMRS Monolitik sebagai Titik Awal

SIMRS di lima rumah sakit Medikara Health berasal dari satu keluarga produk, dengan versi dari 3.9 di Bandung sampai 5.2 di Jakarta. Setiap rumah sakit menyimpan salinan kode sendiri yang sudah dimodifikasi tim

TI setempat. Setiap salinan adalah monolit: sembilan modul, yaitu pendaftaran, rekam medis, rawat inap, farmasi, laboratorium, radiologi, penagihan, klaim BPJS (Badan Penyelenggara Jaminan Sosial) Kesehatan, dan laporan, berjalan dalam satu aplikasi dan berbagi satu basis data.
Misalkan basis data SIMRS Jakarta berisi sekitar 640 tabel, dan modul mana pun boleh membaca tabel mana pun. Rilis dilakukan tiga bulan sekali pada Sabtu malam dengan henti layanan sekitar empat jam. Setiap rilis harus menunggu modul yang paling lambat siap, sehingga perubahan tarif yang diminta bagian keuangan pada awal bulan rata-rata baru berlaku enam sampai sepuluh minggu kemudian.
Diagnosis awal ini punya dua konsekuensi. Pertama, masalah Medikara tidak sesederhana "monolit atau *microservices*". Masalahnya ada dua lapis: ada lima salinan, dan setiap salinan tidak punya batas internal. Kedua, pertanyaan yang lebih tepat untuk direksi adalah "satu sistem atau lima, dan dengan batas atau tanpa batas". Pertanyaan itu dijawab bertahap di seksi-seksi berikutnya, lalu dicatat dalam ADR.

7.3 Coupling, Cohesion, dan Biaya Sistem Terdistribusi

Pertanyaan kemudian muncul: bagaimana mengetahui apakah batas antarbagian sebuah sistem sudah baik? Jawabannya ada pada dua ukuran lama yang tetap dipakai sampai sekarang, yaitu *coupling* dan *cohesion*. Keduanya juga menjelaskan mengapa memecah sistem lewat jaringan tidak otomatis membuatnya lebih mudah diubah.

7.3.1 Coupling dan Cohesion

Coupling adalah derajat ketergantungan antara dua bagian sistem. Makin tinggi *coupling*, makin besar kemungkinan perubahan di satu bagian memaksa perubahan di bagian lain. *Cohesion* adalah derajat keterkaitan isi di dalam satu bagian. Makin tinggi *cohesion*, makin besar kemungkinan hal-hal yang berubah bersama berada di tempat yang sama. Kedua konsep ini diperkenalkan dalam kajian tentang rancangan program terstruktur oleh Stevens, Myers, dan Constantine [134]. Arah yang dicari selalu sama: *coupling* rendah antarbagian dan *cohesion* tinggi di dalam bagian. Bagi *microservices*, ukuran ini menentukan segalanya. Layanan yang terkait erat tidak dapat dirilis secara mandiri, sehingga manfaat utama gaya ini hilang [106].

Analoginya seperti panitia acara pernikahan dengan seksi konsumsi, seksi dekorasi, dan seksi acara. Pada panitia yang baik, seksi konsumsi dapat mengganti menu tanpa rapat dengan seksi dekorasi (*coupling* rendah), dan semua urusan

makanan memang ada di seksi konsumsi (*cohesion* tinggi). Pada panitia yang buruk, pemesanan katering dipegang seksi dekorasi karena kebetulan kenal pemilik katering. Setiap perubahan menu kini membutuhkan dua seksi.

Dua hal pada *coupling* sering tertukar. Pembagiannya begini. *Coupling* yang tampak adalah hubungan yang terlihat di diagram, misalnya modul A memanggil modul B. *Coupling* yang tersembunyi hanya terlihat dari riwayat, misalnya dua modul hampir selalu berubah dalam permintaan perubahan yang sama walaupun di diagram tidak terhubung. Diagnosis yang baik membaca keduanya.

7.3.2 Empat Jenis Coupling

Untuk keperluan diagnosis, bab ini memakai empat jenis *coupling* yang mudah dikenali di lapangan, baik oleh anggota teknis maupun non-teknis. Tabel 7.2 merangkumnya beserta tanda-tandanya.

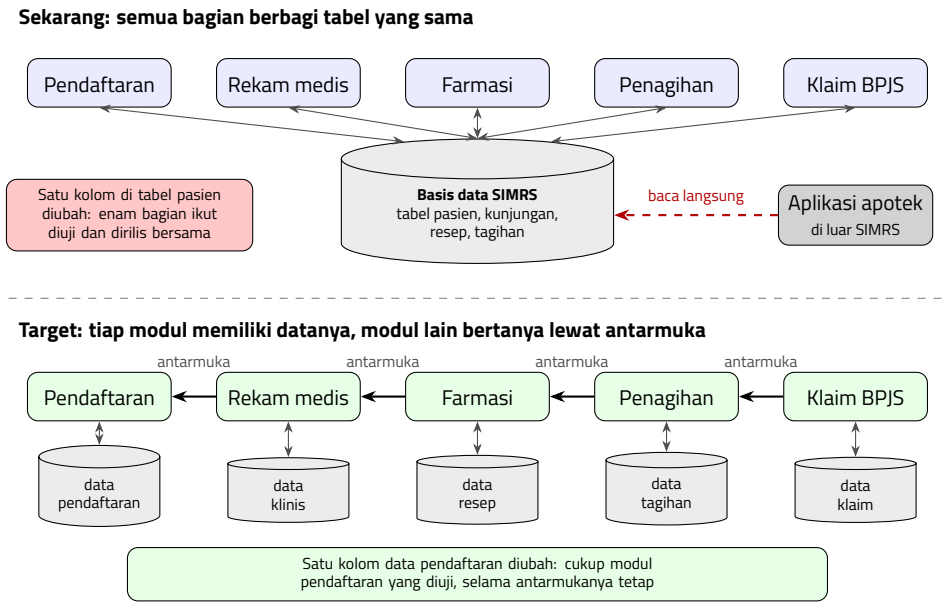
Tabel 7.2: Empat jenis *coupling* dan tandanya di lapangan

Jenis	Arti	Tanda di lapangan	Contoh di Medikara
Data	Dua bagian membaca atau menulis data yang sama	Satu perubahan kolom memaksa banyak bagian diuji ulang	Aplikasi apotek membaca tabel SIMRS secara langsung (Bab 4)
Waktu	Satu bagian hanya dapat bekerja bila bagian lain menyala pada saat yang sama	Gangguan satu sistem ikut menghentikan sistem lain	Pemesanan janji temu di Medikara+ berhenti saat SIMRS dipelihara
Rilis	Dua bagian harus dirilis bersama	Jadwal rilis menunggu tim lain	Perubahan tarif menunggu rilis modul klaim
Makna	Dua bagian bergantung pada arti dan aturan yang sama	Arti satu kode berubah, bagian lain salah tafsir tanpa ada pesan kesalahan	Status “pulang” dipakai rawat inap dan penagihan dengan arti berbeda

Coupling makna paling sulit ditemukan karena tidak menimbulkan *error*. Sistem berjalan normal, tetapi angkanya salah. Jenis ini berkaitan langsung dengan *bounded context* pada Bab 6: satu istilah yang dipakai lintas konteks tanpa kesepakatan arti adalah sumber *coupling* makna. Jenis data paling mudah dilihat dan paling sering ditemui pada monolit. Gambar 7.3 membandingkan kondisi SIMRS sekarang, ketika semua modul berbagi tabel yang sama, dengan kondisi target ketika setiap modul memiliki datanya sendiri.

Dua contoh dari Medikara Health:

1. Misalkan kolom nomor telepon di tabel pasien SIMRS Jakarta diubah dari satu nomor menjadi daftar nomor. Enam bagian membaca tabel itu, termasuk aplikasi apotek di luar SIMRS. Dengan pengujian manual sekitar dua hari kerja



Gambar 7.3: *Coupling* data: berbagi tabel membuat satu perubahan kecil merambat ke banyak bagian, sedangkan kepemilikan data menahannya di satu modul

per bagian, satu perubahan kecil membutuhkan 12 hari kerja pengujian.

2. Medikara+ memesan janji temu dengan memanggil SIMRS secara langsung. Ketika SIMRS Bogor menjalani pemeliharaan malam selama empat jam, pemesanan janji temu untuk rumah sakit Bogor ikut berhenti selama empat jam, walaupun Medikara+ sendiri sehat. Ini contoh *coupling* waktu.

7.3.3 Fallacies of Distributed Computing

Ketika bagian sistem dipisah lewat jaringan, panggilan yang dulu terjadi di dalam satu proses berubah menjadi pesan yang melintasi jaringan. Para insinyur Sun Microsystems pada 1990-an merangkum delapan asumsi keliru yang sering dipegang perancang sistem terdistribusi. Daftar ini dikenal sebagai *fallacies of distributed computing*, dan Richards serta Ford memakainya sebagai titik awal setiap pembahasan gaya arsitektur terdistribusi [124]. Tabel 7.3 menerjemahkan kedelapannya ke akibat yang dirasakan organisasi.

Analoginya seperti memindahkan rapat dari satu ruangan ke surat-menyurat antarkota. Dalam satu ruangan, pertanyaan langsung dijawab. Lewat pos, surat dapat terlambat, hilang, atau dibuka orang di jalan. Ongkos kirimnya tidak nol, dan alamat penerimanya dapat pindah tanpa pemberitahuan.

Pemisahan lewat jaringan membawa empat jenis biaya tambahan. Pertama,

Tabel 7.3: Delapan *fallacies of distributed computing* dan akibatnya bagi organisasi, diolah dari Richards dan Ford [124]

Asumsi keliru	Kenyataan	Akibat bagi organisasi
Jaringan selalu andal	Pesan dapat hilang atau terputus	Setiap panggilan perlu pengulangan dan penanganan gagal
Latensi nol	Setiap panggilan membutuhkan waktu	Permintaan yang melewati banyak layanan terasa lambat bagi pasien
Bandwidth tidak terbatas	Kapasitas jaringan terbatas	Berkas besar seperti citra radiologi membebani jaringan antarlokasi
Jaringan aman	Setiap jalur dapat disadap	Setiap layanan perlu otentikasi dan enkripsi, menambah kerja kepatuhan
Topologi tidak berubah	Alamat dan jalur berpindah	Alamat layanan harus dicari secara dinamis, bukan ditulis di kode
Hanya ada satu administrator	Banyak tim dan vendor terlibat	Perubahan perlu koordinasi lintas tim dan lintas kontrak
Biaya transpor nol	Transfer dan pengolahan pesan berbiaya	Tagihan transfer data dan tenaga pemantauan naik
Jaringan seragam	Perangkat dan protokol beragam	Pengujian harus mencakup banyak kombinasi lingkungan

biaya jaringan: setiap panggilan dapat lambat atau gagal, sehingga perlu batas waktu, pengulangan, dan rancangan tahan gagal yang dibahas di Bab 12. Kedua, biaya data: tabel tidak lagi dapat digabung dalam satu *query*, dan perubahan yang menyentuh beberapa layanan memerlukan pola khusus seperti saga pada Bab 10. Ketiga, biaya pengujian: setiap layanan diuji sendiri, lalu kesepakatan antarlayanan juga harus diuji. Keempat, biaya operasi: setiap layanan butuh jalur rilis, pemantauan, dan pemilik yang siap dihubungi ketika terjadi gangguan, seperti dibahas di Bab 11 dan Bab 13.

Biaya pertama dapat dihitung. Misalkan satu pemesanan janji temu di Medikara+ melewati enam layanan secara berurutan, dan setiap layanan tersedia 99,9 persen dari waktu. Ketersediaan rangkaian itu adalah $0{,}999^6 \approx 0{,}994$, atau sekitar 99,4 persen. Dalam sebulan berisi 30 hari, satu layanan dengan ketersediaan 99,9 persen boleh tidak tersedia sekitar 43 menit. Rangkaian enam layanan boleh tidak tersedia sekitar 4 jam 19 menit, enam kali lebih lama. Angka ini menjelaskan mengapa *microservices* menuntut rancangan tahan gagal sejak awal.

Kasus Medikara Health: Diagnosis Coupling SIMRS Jakarta

Misalkan tim transformasi mengumpulkan dua jenis data dari SIMRS Jakarta. Data pertama adalah 40 permintaan perubahan pada Januari sampai Maret 2025, masing-masing dengan daftar modul yang diubah. Data kedua adalah 63 catatan gangguan sepanjang 2025. Anggota non-teknis mewawancarai bagian peminta untuk mengetahui dampak bisnis setiap modul. Hasilnya ada di Tabel 7.4.

Hasil ini punya tiga konsekuensi. Pertama, 30 dari 40 perubahan (75 persen) menyentuh lebih dari satu modul. Batas modul yang ada sekarang tidak mengikuti cara bisnis berubah. Kedua, modul klaim BPJS dan penagihan berubah bersama 10 kali dari 13 perubahan modul klaim, dengan derajat keterkaitan 67 persen. Keduanya kandidat kuat untuk disatukan dalam satu modul dan dipegang satu tim, bukan dipisah menjadi dua layanan. Ketiga, modul klaim menyumbang 22 dari 63 gangguan (sekitar 35 persen), sehingga modul inilah yang paling mendesak dirapikan. Rincian perhitungannya tersedia di `sources/chapter07/lembar-diagnosis-coupling.xlsx`.

Pendalaman Teknis (opsional): Mengukur Coupling pada Basis Kode

Coupling yang tampak diukur dari dependensi statis di kode. Alat analisis dependensi seperti `jdeps` untuk Java atau `pydeps` untuk Python menghasilkan graf modul yang saling memanggil. Dari graf itu dihitung dua angka untuk setiap modul: berapa modul lain yang bergantung padanya (*afferent coupling*, C_a) dan berapa modul lain yang ia butuhkan (*efferent coupling*, C_e). Martin menggabungkan keduanya menjadi *instability* $I = C_e/(C_a + C_e)$. Nilai mendekati 0 menandakan modul yang banyak diandalkan dan sebaiknya jarang berubah, sedangkan nilai mendekati 1 menandakan modul yang bebas berubah [90]. Misalkan modul penagihan dipakai empat modul dan memakai dua modul. Nilainya $I = 2/6 \approx 0{,}33$, sehingga perubahan antarmukanya harus hati-hati.

Coupling yang tersembunyi diukur dari riwayat perubahan. Listing 7.1 memperlihatkan inti skrip yang menghitung pasangan modul yang sering berubah bersama. Derajat keterkaitan dihitung terhadap rata-rata jumlah perubahan kedua modul, agar modul yang sangat sering berubah tidak membuat semua pasangannya tampak erat. Pada berkas contoh `sources/chapter07/contoh-riwayat-perubahan.csv`, skrip menemukan 30 dari 40 perubahan menyentuh lebih dari satu modul, dan pasangan klaim BPJS dengan penagihan berubah bersama 10 kali dengan derajat 67 persen, sama dengan Tabel 7.4.

```
1 def hitung_keterkaitan(daftar_perubahan):
```

Tabel 7.4: Diagnosis SIMRS Jakarta: perubahan Januari sampai Maret 2025 dan gangguan sepanjang 2025 (angka rekaan)

Modul	Perubahan	Gangguan	Pasangan paling erat (kali berubah bersama)	Dampak bisnis bila terganggu
Pendaftaran	9	9	Rekam medis (5)	Antrean pagi memanjang
Rekam medis	13	6	Pendaftaran (5)	Dokter kembali menulis di kertas
Rawat inap	4	3	Tidak ada yang menonjol	Penempatan pasien lambat
Farmasi	6	5	Tidak ada yang menonjol	Pasien menunggu obat
Laboratorium	5	4	Rekam medis (4)	Hasil terlambat sampai ke dokter
Radiologi	2	2	Tidak ada yang menonjol	Jadwal pemeriksaan bergeser
Penagihan	17	11	Klaim BPJS (10)	Pasien tertahan di kasir
Klaim BPJS	13	22	Penagihan (10)	Klaim tertunda, arus kas terganggu
Laporan	7	1	Klaim BPJS (3)	Rapat direksi tanpa angka terbaru
Jumlah	**76**	**63**	76 sentuhan modul berasal dari 40 permintaan; 30 permintaan (75 persen) menyentuh lebih dari satu modul	

```
    """Menghitung jumlah perubahan per modul dan per pasangan modul.

    Pasangan diurutkan menurut abjad agar (a, b) dan (b, a) dihitung sebagai
    pasangan yang sama.
    """
    per_modul = Counter()
    per_pasangan = Counter()
    for modul_berubah in daftar_perubahan:
        per_modul.update(modul_berubah)
        for pasangan in combinations(sorted(modul_berubah), 2):
            per_pasangan[pasangan] += 1
    return per_modul, per_pasangan


def derajat_keterkaitan(bersama, jumlah_pertama, jumlah_kedua):
    """Persentase perubahan bersama terhadap rata-rata perubahan kedua modul.

    Rata-rata dipakai agar modul yang sangat sering berubah tidak membuat
```

```
    semua pasangannya tampak erat. Hasil di atas 50 persen layak dibahas
        tim.
    """
    rata_rata = (jumlah_pertama + jumlah_kedua) / 2
    return round(100 * bersama / rata_rata)
```

Listing 7.1: Inti penghitung keterkaitan perubahan, tersedia di `sources/chapter07/hitung-coupling.py`

7.4 Hukum Conway dan Struktur Tim

Diagnosis *coupling* menjawab pertanyaan tentang kode. Pertanyaan berikutnya lebih sering dilupakan: siapa yang membuat dan mengubah kode itu, dan bagaimana mereka berkomunikasi? Jawaban atas pertanyaan ini ternyata ikut membentuk arsitektur.

7.4.1 Struktur Komunikasi Membentuk Struktur Sistem

Pada 1968 Melvin Conway menulis bahwa organisasi yang merancang sistem terdorong menghasilkan rancangan yang meniru struktur komunikasi organisasi itu sendiri [40]. Pengamatan ini kemudian dikenal sebagai hukum Conway. Alasannya sederhana. Dua bagian sistem hanya dapat tersambung dengan baik bila orang-orang yang membuatnya berkomunikasi. Tim yang jarang berbicara menghasilkan bagian yang terpisah dengan antarmuka yang kaku. Tim yang selalu bekerja di satu ruangan cenderung menghasilkan bagian yang saling menyentuh. Pengamatan ini juga punya dukungan empiris. MacCormack, Baldwin, dan Rusnak membandingkan pasangan produk perangkat lunak dengan fungsi serupa dan menemukan bahwa produk dari organisasi yang longgar dan tersebar cenderung lebih modular daripada produk dari organisasi yang terikat erat [88].

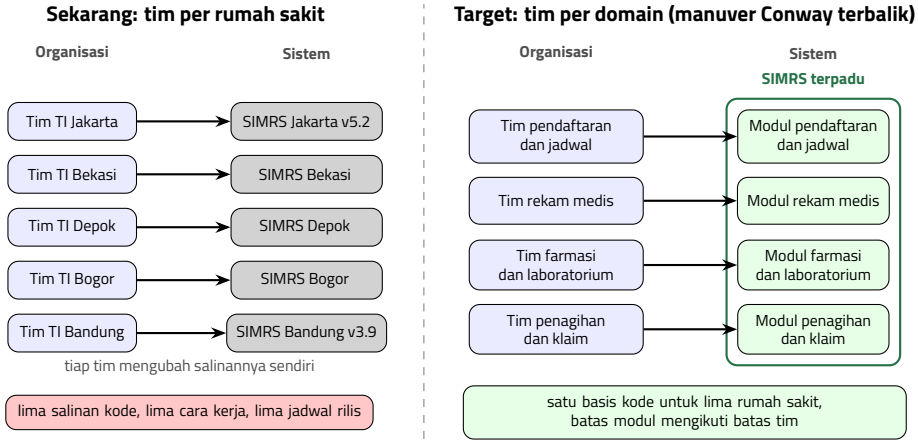
Analoginya seperti satu RW dengan lima RT yang masing-masing mengatur keamanan sendiri. Setiap RT punya portal, jadwal ronda, dan kartu tamu sendiri. Tidak ada yang salah dengan setiap RT. Namun tamu yang melintasi tiga RT ditanyai tiga kali, dan warga yang pindah RT harus belajar aturan baru. Struktur pengurus yang terpisah-pisah membentuk sistem keamanan yang terpisah-pisah.

7.4.2 Manuver Conway Terbalik

Bila struktur tim membentuk struktur sistem, organisasi dapat memakai hukum ini secara sengaja: menata tim lebih dahulu sesuai arsitektur yang diinginkan. Cara ini disebut manuver Conway terbalik (*inverse Conway maneuver*). Skelton dan Pais menyarankan tim yang stabil, kecil, sekitar lima sampai sembilan orang, dengan

tanggung jawab yang dibatasi oleh beban kognitif tim, yaitu jumlah hal yang sanggup dipahami dan dijaga satu tim sekaligus [131]. Akibatnya langsung terasa pada pilihan gaya arsitektur. *Microservices* hanya masuk akal bila setiap layanan punya satu tim pemilik yang sanggup membangun, merilis, dan menjaganya. Bab 13 membahas Team Topologies secara rinci.

Gambar 7.4 memperlihatkan dua keadaan. Di kiri, tim TI per rumah sakit menghasilkan lima salinan SIMRS. Di kanan, tim per domain menghasilkan satu sistem yang modulnya mengikuti batas tim.



Gambar 7.4: Hukum Conway pada SIMRS: tim per rumah sakit menghasilkan lima salinan, tim per domain menghasilkan modul per domain

Dua contoh kasus:

1. Misalkan sebuah perusahaan asuransi membentuk tim per kanal penjualan: agen, situs web, dan aplikasi seluler. Aturan perhitungan premi akhirnya ditulis tiga kali, sekali di setiap kanal. Satu perubahan tarif membutuhkan tiga rilis, dan selama tiga minggu calon nasabah mendapat harga yang berbeda di tiga kanal.
2. Di Segment, lebih dari 140 layanan dijaga oleh tim kecil, dan tiga insinyur penuh waktu menghabiskan sebagian besar waktunya untuk menjaga sistem tetap berjalan [107]. Jumlah layanan jauh melampaui jumlah tim yang dapat memilikinya.

Kasus Medikara Health: Tim TI per Rumah Sakit dan Lima SIMRS

Misalkan tim TI di lima rumah sakit Medikara berjumlah 26 orang: 8 di Jakarta, 5 di Bekasi, 4 di Depok, 4 di Bogor, dan 5 di Bandung. Dari 26 orang itu, 12 orang mengubah kode SIMRS, sedangkan sisanya menangani

perangkat dan layanan bantuan pengguna. Setiap tim melapor kepada direktur rumah sakitnya, dan setiap tim mengubah salinan SIMRS-nya sendiri. Hukum Conway bekerja persis seperti diramalkan: lima struktur komunikasi menghasilkan lima SIMRS.

Misalkan Medikara menerapkan manuver Conway terbalik. Dua belas pengembang ditata menjadi empat tim domain yang mengikuti *bounded context* dari Bab 6: pendaftaran dan jadwal, rekam medis, farmasi dan laboratorium, serta penagihan dan klaim. Setiap tim berisi tiga pengembang dan dua orang yang ditugaskan dari unit pengguna sebagai analis proses dan penguji, sehingga setiap tim berisi lima orang. Empat belas staf lainnya tetap di rumah sakit untuk perangkat dan bantuan pengguna.

Penataan ini punya dua konsekuensi. Pertama, konsekuensi organisasi: direktur rumah sakit tidak lagi punya tim pengembang sendiri, sehingga permintaan perubahan harus melewati tim domain. Perubahan ini perlu dikelola sebagai perubahan organisasi, bukan sekadar perubahan bagan. Kedua, konsekuensi teknis: empat tim yang mengerjakan satu basis kode membutuhkan batas modul yang tegas. Gaya yang paling cocok adalah *modular monolith*. Usulan vendor berisi 32 layanan justru berarti delapan layanan untuk setiap tim berisi tiga pengembang.

7.5 Kapan Microservices Tidak Tepat

Newman, salah satu pendukung utama *microservices*, menegaskan bahwa gaya ini bukan pilihan bawaan. Ia menyebut beberapa keadaan yang membuat *microservices* sebaiknya tidak dipakai: domain yang belum jelas, perusahaan rintisan yang produknya masih mencari bentuk, perangkat lunak yang dipasang dan dikelola sendiri oleh pelanggan, dan tidak adanya alasan kuat [106]. Bagi organisasi yang sudah berjalan, keadaan itu dapat diterjemahkan menjadi lima tanda belum siap pada Tabel 7.5.

7.5.1 Pelajaran dari Segment

Kisah Segment sering dikutip, tetapi jarang dibaca utuh. Noonan menuliskannya pada 2018 [107]. Segment meneruskan data pelanggannya ke lebih dari seratus tujuan, misalnya alat analitik dan alat pemasaran. Awalnya semua tujuan berbagi satu antrean. Satu tujuan yang lambat menahan semua tujuan lain, keadaan yang disebut *head-of-line blocking*. Untuk mengatasinya, tim memberi setiap tujuan layanan dan antrean sendiri.

Keputusan itu menyelesaikan masalah antrean, tetapi menimbulkan masalah baru. Setiap tujuan baru berarti repositori baru, dan jumlah layanan tumbuh

Tabel 7.5: Lima tanda organisasi belum siap memakai *microservices*

Tanda	Pertanyaan pemeriksa	Mengapa penting
Batas domain belum stabil	Sudahkah *bounded context* diuji dengan data perubahan nyata?	Batas yang salah mahal dipindah bila sudah melintasi jaringan dan basis data
Jumlah layanan melebihi kemampuan tim	Berapa layanan yang dipegang setiap tim?	Setiap layanan butuh pemilik yang merilis dan menjaganya
Rilis masih manual	Berapa langkah manual dalam satu rilis?	Banyak layanan berarti banyak rilis; tanpa otomasi, beban naik berlipat
Pemantauan belum terpusat	Dapatkah satu permintaan ditelusuri lintas sistem?	Kesalahan yang tersebar di banyak layanan sulit dicari (Bab 11)
Tidak ada alasan bisnis yang terukur	Masalah apa yang hanya selesai dengan rilis dan skala mandiri?	Tanpa alasan itu, biaya ditanggung tanpa manfaat

melewati 140. Pustaka bersama diperbarui di satu layanan tetapi tidak di layanan lain, sampai setiap basis kode memakai versi yang berbeda. Beban setiap layanan juga sangat berbeda, dari beberapa *event* per hari sampai ribuan per detik. Beban operasi naik setiap kali satu tujuan ditambahkan, dan tiga insinyur penuh waktu menghabiskan sebagian besar waktunya hanya untuk menjaga sistem tetap berjalan.

Tim lalu menyatukan semua tujuan ke dalam satu layanan dan satu repositori, dengan satu versi untuk setiap dependensi. Pengujian seluruh tujuan, yang dulu bisa memakan waktu sampai satu jam, menjadi hitungan milidetik setelah lalu lintas uji direkam dan diputar ulang. Pada tahun ketika arsitekturnya masih *microservices*, tim membuat 32 perbaikan pada pustaka bersama; setahun setelah pindah ke monolit, jumlahnya 46. Noonan juga jujur tentang harganya: kesalahan di satu tujuan kini dapat menghentikan layanan untuk semua tujuan, *cache* di memori kurang efektif karena tersebar di ribuan proses, dan pembaruan satu dependensi dapat merusak beberapa tujuan sekaligus.

Pelajarannya bukan "*microservices* itu buruk". Pelajarannya ada tiga. Pertama, layanan dipecah menurut tujuan pengiriman yang bersifat teknis, bukan menurut domain bisnis atau tim. Kedua, jumlah layanan jauh melampaui jumlah orang yang memilikinya. Ketiga, logika yang sama tersebar di semua layanan lewat pustaka bersama, sehingga *coupling* rilis dan makna tetap ada walaupun kodenya terpisah.

7.5.2 Monolith First dan Pendapat Sebaliknya

Fowler mengamati bahwa hampir semua kisah sukses *microservices* dimulai dari monolit yang tumbuh terlalu besar lalu dipecah. Sebaliknya, hampir semua sistem yang dibangun sebagai *microservices* sejak awal yang ia dengar berakhir dengan

masalah serius. Sarannya dikenal sebagai *monolith first*: mulai dari monolit, pelajari batas domainnya, lalu pecah bila perlu [61]. Tilkov berpendapat sebaliknya. Menurutnya, sistem baru adalah saat terbaik untuk memisahkan bagian, karena memecah monolit belakangan sangat sulit setelah bagian-bagiannya saling terikat lewat kode dan basis data bersama. Ia mengakui pendekatan ini cocok untuk sistem besar dengan domain yang sudah dipahami [147].

Kedua pendapat ini bertemu di satu titik: pemahaman domain menentukan. Bagi Medikara, domain rumah sakit sudah dipahami bertahun-tahun, tetapi batas *bounded context*-nya baru dipetakan di Bab 6 dan belum diuji. *Modular monolith* memberi jalan tengah: batas diuji dulu di dalam satu proses, ketika memindahkan batas masih murah, lalu modul tertentu dipisah menjadi layanan bila data membuktikan perlunya.

7.5.3 Membaca Kajian Secara Kritis

Mahasiswa S2 diharapkan membaca kajian tentang *microservices* secara kritis, termasuk kajian yang dirujuk bab ini. Jamshidi dkk. menulis artikel pengantar edisi tematik IEEE Software tentang perjalanan *microservices* dan tantangan ke depan [79]. Tulisan Noonan adalah kesaksian satu tim di satu perusahaan. Keduanya berguna, tetapi untuk keperluan yang berbeda. Tabel 7.6 memberi lima pertanyaan pembaca yang dapat dipakai untuk kajian apa pun.

Tabel 7.6: Lima pertanyaan pembaca kritis, diterapkan pada dua tulisan

Pertanyaan pembaca	Jamshidi dkk. (2018)	Noonan (2018)
Jenis tulisan apa?	Artikel pengantar edisi tematik di IEEE Software	Tulisan blog teknik perusahaan
Siapa penulisnya dan apa kepentingannya?	Akademisi dan praktisi; Lewis dan Tilkov ikut mempopulerkan *microservices*	Insinyur yang mengalami sendiri dan menjelaskan keputusan timnya
Bukti apa yang dipakai?	Rangkuman perkembangan dan tantangan dari literatur dan praktik	Angka internal satu sistem, tanpa pembanding
Konteks apa yang berlaku?	Umum, lintas organisasi	Satu produk, tim kecil, beban ratusan ribu *event* per detik
Apa yang diperiksa sebelum dipakai?	Apakah tantangan yang disebut relevan bagi organisasi pembaca	Apakah penyebab kegagalannya juga ada di organisasi pembaca

Kasus Medikara Health: Menimbang Usulan 32 Layanan

Tim transformasi memeriksa usulan vendor dengan Tabel 7.5. Misalkan usulan itu memecah SIMRS menjadi 32 layanan dalam 18 bulan, mengikuti menu pada layar SIMRS lama.

Empat dari lima tanda belum siap ditemukan. Batas domain baru dipetakan di Bab 6 dan belum diuji dengan data perubahan. Jumlah orang yang akan memiliki layanan adalah 12 pengembang ditambah 6 orang tim platform, sehingga satu orang memegang hampir dua layanan ($32/18 \approx 1{,}8$). Rilis masih manual: Bab 5 mencatat rilis Medikara+ saja membutuhkan 37 langkah manual. Pemantauan juga masih per sistem, tanpa penelusuran lintas sistem. Satu tanda tidak ditemukan: ada alasan bisnis yang terukur, yaitu lonjakan pemesanan janji temu sampai sekitar 60 persen setiap kali program vaksinasi diumumkan (Bab 3).
Kesimpulannya punya dua konsekuensi. Pertama, *microservices* penuh belum tepat sekarang, dan menu layar bukan dasar yang baik untuk batas layanan. Kedua, alasan bisnis yang ada hanya berlaku untuk satu bagian, yaitu pemesanan janji temu. Bagian itulah calon pertama yang dipisah menjadi layanan tersendiri, setelah otomasi rilis dan pemantauannya siap.

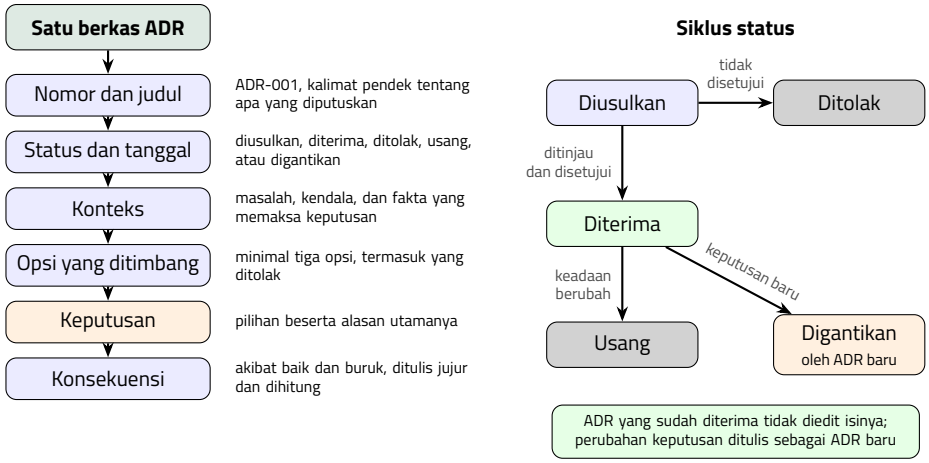
7.6 Architecture Decision Record

Keputusan gaya arsitektur seperti di atas mudah diambil dalam rapat, tetapi mudah juga dilupakan. Dua tahun kemudian anggota tim baru bertanya mengapa SIMRS tidak dipecah menjadi *microservices*, dan perdebatan yang sama diulang dari awal. Pertanyaan kemudian muncul: bagaimana keputusan arsitektur dicatat agar alasannya tetap dapat dibaca?

7.6.1 Fungsi dan Struktur ADR

ADR adalah dokumen pendek yang mencatat satu keputusan arsitektur beserta konteks dan konsekuensinya. Nygard mengusulkan format sederhana: judul, konteks, keputusan, status, dan konsekuensi. Setiap ADR diberi nomor urut dan disimpan bersama dokumen proyek. ADR yang sudah diterima tidak diedit isinya. Bila keputusan berubah, tim menulis ADR baru dan menandai ADR lama sebagai digantikan [108]. Banyak tim menambahkan bagian opsi yang ditimbang, agar pembaca tahu pilihan apa yang ditolak dan mengapa. Gambar 7.5 memperlihatkan susunan satu ADR dan siklus statusnya.

Analoginya seperti notulen rapat RT yang baik. Notulen itu tidak hanya mencatat “iuran keamanan naik menjadi Rp100.000”. Notulen juga mencatat alasannya (dua kali pencurian motor dalam sebulan), pilihan yang ditolak (cukup memasang kamera), dan akibat yang sudah diperkirakan (warga yang menunggak mungkin bertambah). Tiga tahun kemudian, pengurus baru tidak perlu mengulang perdebatan yang sama.



Gambar 7.5: Susunan satu ADR dan siklus statusnya

7.6.2 ADR sebagai Alat Komunikasi

ADR bukan hanya catatan untuk tim teknis. Bagi manajemen, ADR menjelaskan apa yang dibeli dan risiko apa yang diterima, dalam bahasa yang dapat dibaca tanpa penjelasan lisan. Bagi auditor, ADR adalah jejak keputusan: siapa yang memutuskan, kapan, berdasarkan data apa, dan risiko apa yang sudah dipertimbangkan. Jejak ini penting bagi Medikara Health, karena temuan audit internal tentang kontrol akses yang lemah membuat setiap keputusan teknologi berikutnya akan ditanya alasannya.

Mutu ADR sangat beragam. Tabel 7.7 membandingkan ADR yang buruk dan yang baik untuk keputusan yang sama.

Tabel 7.7: Perbandingan ADR yang buruk dan ADR yang baik

Aspek	ADR yang buruk	ADR yang baik
Judul	"Arsitektur SIMRS"	"SIMRS terpadu memakai gaya *modular monolith*"
Konteks	"*Microservices* adalah praktik terbaik industri"	"63 gangguan pada 2025, 22 dari modul klaim; 75 persen perubahan menyentuh lebih dari satu modul"
Opsi	Hanya opsi yang dipilih	Minimal tiga opsi, termasuk yang ditolak beserta alasannya
Konsekuensi	Hanya manfaat	Manfaat dan kerugian, dengan angka bila ada
Bahasa	Penuh istilah tanpa penjelasan	Dapat dibaca direksi dan auditor
Perubahan keputusan	Isi lama ditimpa	ADR baru ditulis, ADR lama ditandai digantikan

Dua contoh ADR:

1. Misalkan vendor Medikara+ menulis ADR satu kalimat: "Sistem memakai *microservices* karena skalabel dan modern." ADR ini tidak menyebut masalah yang diselesaikan, tidak menyebut opsi lain, dan tidak menyebut biaya. Pembaca tidak dapat menilai apakah keputusan itu masih berlaku ketika keadaan berubah.
2. ADR-001 Medikara pada kotak kasus di bawah memuat konteks dengan angka dari diagnosis, empat opsi, keputusan beserta alasannya, tiga konsekuensi baik dan tiga konsekuensi buruk, serta syarat peninjauan ulang yang dapat diukur.

Kasus Medikara Health: ADR-001 Gaya Arsitektur SIMRS Terpadu

Misalkan tim transformasi menyusun ADR pertama berdasarkan diagnosis di seksi sebelumnya, lalu mengajukannya ke komite arsitektur yang beranggotakan direktur TI, direktur pelayanan, dan direktur keuangan. Isinya diringkas di Tabel 7.8. Opsi 3 dipilih bukan karena paling modern, tetapi karena paling cocok dengan kapasitas 18 orang, dengan data perubahan, dan dengan tujuan "Satu Pasien, Satu Rekam Medis". Konsekuensi buruknya ditulis sama jelasnya dengan konsekuensi baiknya, sehingga komite tahu risiko apa yang diterima.

Tabel 7.8: Ringkasan ADR-001 Medikara Health (rekaan)

Bagian	Isi
Judul	ADR-001: SIMRS terpadu memakai gaya *modular monolith*
Status	Diterima, Oktober 2026, oleh komite arsitektur
Konteks	Lima SIMRS versi 3.9 sampai 5.2; di Jakarta 63 gangguan pada 2025 (22 dari modul klaim) dan 75 persen perubahan menyentuh lebih dari satu modul; 12 pengembang dan 6 orang tim platform; vendor mengusulkan 32 layanan dalam 18 bulan
Opsi 1 (ditolak)	Pertahankan lima SIMRS, perbaiki integrasinya: tidak mewujudkan satu rekam medis
Opsi 2 (ditolak)	Satu SIMRS monolit tanpa batas modul: memindahkan kekusutan ke satu tempat
Opsi 3 (dipilih)	Satu SIMRS sebagai *modular monolith*, empat modul domain mengikuti *bounded context* Bab 6; klaim dan penagihan disatukan
Opsi 4 (ditolak)	*Microservices* penuh, 32 layanan: empat dari lima tanda belum siap ditemukan
Konsekuensi baik	Satu basis kode menggantikan lima; satu rilis berlaku untuk lima rumah sakit; batas modul diperiksa otomatis pada setiap rilis
Konsekuensi buruk	Satu unit rilis: kesalahan satu modul dapat menghentikan seluruh SIMRS; penyatuan lima basis data diperkirakan 12 sampai 18 bulan; tim TI rumah sakit ditata ulang
Syarat peninjauan	Modul pendaftaran dan jadwal dipisah menjadi layanan bila lonjakan janji temu melampaui kapasitas SIMRS dua kali dalam enam bulan

7.7 Aktivitas: Diagnosis Arsitektur dan ADR Pertama

Tujuan. Tim mendiagnosis sumber keterkaitan pada SIMRS atau aplikasi inti organisasi pilihan, lalu mencatat keputusan gaya arsitektur target dalam ADR pertama. Pada pertemuan ini tim juga merapikan seluruh artefak tonggak C2 untuk ditinjau menjelang UTS. Tim memakai organisasi nyata dengan izin tertulis dan data yang dianonimkan, atau memakai kasus Medikara Health.

Pembagian peran. Diagnosis membutuhkan dua jenis data: data bisnis dan data teknis. Tabel 7.9 membagi peran agar setiap anggota menyumbang sesuai latar belakangnya.

Tabel 7.9: Pembagian peran dalam Aktivitas Bab 7

Peran	Diisi oleh	Tugas
Pengumpul data bisnis	Anggota non-teknis	Mewawancarai pengguna, mengumpulkan pola permintaan perubahan dan dampak bisnis setiap gangguan
Pemeta modul	Anggota teknis	Memetakan modul, dependensi, dan riwayat perubahan dari tiket atau sistem kendali versi
Penulis ADR	Seluruh anggota, satu orang sebagai penyunting	Menimbang opsi, menulis ADR, dan menjaga agar bahasanya dapat dibaca manajemen
Penghimpun C2	Satu anggota yang ditunjuk	Menggabungkan artefak Bab 4 sampai 7 dan menjalankan daftar periksa C2

Langkah kerja.

1. Identifikasi bagian yang sering berubah bersamaan. Kumpulkan permintaan perubahan setidaknya tiga bulan terakhir, catat modul yang diubah oleh setiap permintaan, lalu hitung pasangan modul yang paling sering berubah bersama.
2. Identifikasi bagian yang paling sering menimbulkan gangguan. Kumpulkan catatan gangguan setidaknya enam bulan terakhir dan kelompokkan menurut modul. Anggota non-teknis menuliskan dampak bisnis setiap modul bila terganggu.
3. Petakan tim pengelola. Untuk setiap modul, catat tim yang mengubahnya, tim yang menjaganya ketika terjadi gangguan, dan jalur pelaporannya. Bandingkan dengan hukum Conway: apakah struktur sistem meniru struktur tim?
4. Tulis ADR gaya arsitektur target dengan minimal tiga opsi. Setiap opsi yang ditolak diberi alasan satu kalimat. Konsekuensi buruk ditulis sejujur konsekuensi baik, dan syarat peninjauan ulang dibuat terukur.
5. Gabungkan artefak Bab 4 sampai 7 menjadi berkas C2: penilaian kesiapan

dan portofolio 7R, kartu skor *twelve-factor*, peta domain dan *context map*, serta diagnosis dan ADR. Periksa konsistensi nama aplikasi dan angka di semua artefak.

Luaran dan kaitan capstone. Luarannya adalah diagnosis arsitektur aplikasi inti (lembar diagnosis yang sudah diisi) dan ADR gaya arsitektur. Keduanya melengkapi tonggak C2 Kondisi Saat Ini, yang ditinjau menjelang UTS. ADR ini juga menjadi titik berangkat tonggak C3 Kondisi Target: pilihan *runtime*, pola komunikasi, dan kepemilikan data di Bab 8 sampai 10 dirancang di dalam batas yang ditetapkan ADR ini.

Kriteria penilaian. Tabel 7.10 memuat kriteria dan bobotnya.

Tabel 7.10: Kriteria penilaian Aktivitas Bab 7

Kriteria	Bobot	Tanda hasil yang baik
Mutu diagnosis	30%	Kesimpulan didukung data perubahan, gangguan, dan tim pengelola; sumber data disebut
Mutu ADR	35%	Minimal tiga opsi, alasan penolakan jelas, konsekuensi buruk ditulis, syarat peninjauan terukur
Kaitan bisnis	20%	Dampak bisnis ditulis dalam bahasa manajemen, misalnya waktu tunggu, klaim tertunda, atau biaya
Kerapian C2	15%	Artefak Bab 4 sampai 7 lengkap dan angkanya konsisten

Berkas pendukung.

- `sources/chapter07/lembar-diagnosis-coupling.xlsx`: lembar contoh Medikara dan lembar isian tim, dengan derajat keterkaitan yang dihitung otomatis.
- `sources/chapter07/templat-adr.md`: templat ADR beserta daftar periksa sebelum diajukan.
- `sources/chapter07/daftar-periksa-c2.md`: daftar periksa penggabungan artefak C2.
- `sources/chapter07/contoh-riwayat-perubahan.csv` dan `sources/chapter07/hitung-coupling.py`: data contoh dan skrip untuk pendalaman teknis.

Etika. Riwayat perubahan dan catatan gangguan dari organisasi nyata dipakai seizin organisasi. Nama orang, data pasien, alamat server, dan kunci akses dihapus sebelum data dibagikan ke anggota tim.

Pendalaman Teknis (opsional): Coupling pada Basis Kode Open Source

Anggota teknis dapat mengulang diagnosis pada satu basis kode *open source* yang sudah disalin ke komputer sendiri. Riwayat *commit* diambil dengan `git log --name-only`, lalu setiap *commit* diubah menjadi satu baris berisi daftar modul yang disentuhnya, dengan kolom yang sama seperti berkas contoh. Skrip `sources/chapter07/hitung-coupling.py` kemudian dijalankan pada berkas itu. Hasilnya dibandingkan dengan graf dependensi dari alat analisis dependensi seperti dijelaskan di kotak pendalaman seksi 7.3. Pasangan yang sering berubah bersama tetapi tidak saling memanggil di graf adalah tanda *coupling* tersembunyi. Kegiatan ini hanya membaca salinan lokal, sehingga tidak membebani server milik pihak lain.

7.8 Ringkasan

Monolit, *modular monolith*, dan *microservices* adalah tiga titik pada satu spektrum, bukan pilihan antara yang lama dan yang modern. Monolit dirilis sebagai satu unit dan paling sederhana dioperasikan. *Modular monolith* tetap satu unit rilis, tetapi menegakkan batas modul dan kepemilikan data. *Microservices* memberi kemandirian rilis dan penskalaan per layanan, dengan biaya jaringan, data, pengujian, dan operasi yang lebih tinggi. Jebakan yang harus dihindari adalah monolit terdistribusi: layanan dipisah lewat jaringan, tetapi tetap harus dirilis bersama.

Mutu batas antarbagian diukur dengan *coupling* dan *cohesion*. Empat jenis *coupling* mudah dikenali di lapangan: data, waktu, rilis, dan makna. *Coupling* yang tersembunyi terlihat dari riwayat perubahan, seperti modul klaim dan penagihan SIMRS Jakarta yang berubah bersama 10 kali dalam tiga bulan. Memecah sistem lewat jaringan tidak menghapus *coupling*. Pemisahan itu justru menambah delapan risiko yang dirangkum dalam *fallacies of distributed computing*, termasuk ketersediaan yang turun dari 99,9 menjadi 99,4 persen ketika enam layanan dirangkai.

Hukum Conway mengingatkan bahwa struktur sistem meniru struktur komunikasi organisasi. Lima tim TI per rumah sakit menghasilkan lima SIMRS. Manuver Conway terbalik menata tim lebih dahulu sesuai arsitektur yang diinginkan, misalnya empat tim domain untuk empat modul. *Microservices* hanya masuk akal bila setiap layanan punya satu tim pemilik yang sanggup merilis dan menjaganya.

Kisah Segment, saran *monolith first*, dan pendapat sebaliknya menunjukkan bahwa *microservices* tidak tepat ketika batas domain belum stabil, layanan melebihi kemampuan tim, rilis dan pemantauan belum otomatis, atau tidak ada alasan bisnis yang terukur. Setiap kajian, baik artikel ilmiah maupun tulisan blog teknik, dibaca dengan pertanyaan tentang jenis tulisan, kepentingan penulis, bukti, dan

konteksnya.

Keputusan gaya arsitektur dicatat dalam ADR: konteks dengan angka, opsi yang ditimbang, keputusan, konsekuensi baik dan buruk, serta syarat peninjauan ulang. Bagi Medikara Health, ADR-001 memilih *modular monolith* untuk SIMRS terpadu dan menyiapkan pemisahan modul janji temu bila data membuktikan perlunya. Pesan utama bab ini: gaya arsitektur dipilih dari diagnosis dan kapasitas organisasi, bukan dari janji vendor, dan alasannya ditulis agar dapat diperiksa kembali.

7.9 Latihan

Latihan 1: Enam Pengembang dan Dua Puluh Microservices

Latihan ini membuktikan bahwa rencana *microservices* dapat diuji dengan pertanyaan yang jawabannya berupa data, bukan selera. Sebuah tim berisi enam pengembang ingin memecah aplikasinya menjadi dua puluh *microservices*. Tugasnya adalah mengajukan tiga pertanyaan yang perlu dijawab sebelum rencana itu disetujui.

1. Baca kembali Tabel 7.5 dan pilih tiga tanda yang paling relevan untuk tim tersebut.
2. Ubah setiap tanda menjadi satu pertanyaan yang dapat dijawab dengan data.
3. Untuk setiap pertanyaan, tentukan data yang harus diperiksa dan cara memperolehnya.
4. Isi temuan contoh dari kasus Medikara atau organisasi pilihan, lalu tulis artinya bagi keputusan.
5. Bandingkan hasil dengan contoh pada Tabel 7.11, lalu isi Tabel 7.12.

Pertanyaan analisis:

1. Dari tiga pertanyaan, mana yang jawabannya paling cepat membatalkan rencana? Jelaskan berdasarkan temuan yang diisi.
2. Bila temuan menunjukkan hanya satu bagian yang membutuhkan skala mandiri, berapa layanan yang masuk akal untuk tahap pertama? Hitung jumlah layanan per orang untuk usulan itu.

Latihan 2: Modular Monolith sebagai Titik Tengah

Latihan ini membuktikan bahwa *modular monolith* sering menjadi titik tengah yang masuk akal bagi organisasi seperti Medikara Health, dengan membandingkan tiga gaya memakai kriteria yang sama.

Tabel 7.11: Latihan 1, contoh terisi: misalkan tim Medikara+ bersama vendornya berisi enam pengembang yang ingin membuat dua puluh layanan (rekaan)

Pertanyaan	Data yang diperiksa	Temuan contoh	Arti bagi keputusan
Masalah bisnis apa yang hanya selesai dengan rilis dan skala mandiri?	Catatan lonjakan dan gangguan	Lonjakan sekitar 60 persen saat vaksinasi hanya terjadi di pemesanan janji temu	Cukup memisahkan satu bagian, bukan dua puluh
Sanggupkah enam orang memiliki dua puluh layanan?	Jumlah orang dan jadwal jaga	$20/6 \approx 3{,}3$ layanan per orang	Satu orang cuti membuat tiga layanan tanpa pemilik
Sudahkah rilis otomatis?	Daftar langkah rilis	Rilis Medikara+ lewat 37 langkah manual (Bab 5)	Dua puluh layanan berarti sampai 740 langkah manual bila semuanya dirilis

Tabel 7.12: Latihan 1, lembar isian: tiga pertanyaan sebelum rencana disetujui

Pertanyaan	Data yang diperiksa	Temuan	Arti bagi keputusan
…	…	…	…
…	…	…	…
…	…	…	…

1. Tentukan lima kriteria penilaian yang penting bagi organisasi, misalnya kapasitas tim, kemandirian kerja per domain, biaya operasi, kemudahan mencapai tujuan strategi, dan risiko migrasi.
2. Beri skor 1 sampai 3 untuk setiap gaya pada setiap kriteria, dengan 3 sebagai yang terbaik.
3. Tulis alasan singkat untuk setiap skor berdasarkan data diagnosis, bukan pendapat umum.
4. Jumlahkan skor, lalu periksa apakah urutan berubah bila satu kriteria dianggap dua kali lebih penting.
5. Bandingkan hasil dengan contoh pada Tabel 7.13, lalu isi Tabel 7.14.

Pertanyaan analisis:

1. Pada contoh, selisih *modular monolith* dan monolit hanya dua poin. Kriteria mana yang menentukan selisih itu, dan apakah kriteria tersebut juga penting di organisasi pilihan tim?
2. Bila kriteria kemandirian kerja diberi bobot dua kali, apakah urutan pada contoh berubah? Tunjukkan perhitungannya.
3. Dalam kondisi apa skor *microservices* pada organisasi pilihan tim dapat naik

Tabel 7.13: Latihan 2, contoh terisi: perbandingan tiga gaya untuk SIMRS terpadu Medikara (skor 1 sampai 3, rekaan)

Kriteria	Monolit	*Modular monolith*	*Micro-services*	Alasan singkat
Kesesuaian dengan 18 orang	3	3	1	32 layanan berarti hampir dua layanan per orang
Kemandirian kerja empat tim domain	1	2	3	Tanpa batas modul, empat tim saling menunggu
Biaya operasi dan pemantauan	3	3	1	Pemantauan lintas layanan belum ada
Kemudahan mencapai satu rekam medis	2	3	2	Satu basis data dengan skema per modul memudahkan penyatuan
Risiko migrasi	2	2	1	*Microservices* memindahkan batas dan data sekaligus
Jumlah	**11**	**13**	**8**	

Tabel 7.14: Latihan 2, lembar isian: perbandingan tiga gaya untuk organisasi pilihan tim

Kriteria	Monolit	*Modular monolith*	*Micro-services*	Alasan singkat
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
Jumlah	…	…	…	

dalam dua tahun ke depan?

Latihan 3: Membaca Kisah Segment

Latihan ini membuktikan kemampuan membaca kesaksian industri secara kritis dan memindahkan pelajarannya ke organisasi lain. Bacalah tulisan Noonan tentang Segment yang kembali ke monolit [107]. Tentukan penyebab utamanya, lalu periksa apakah penyebab itu juga ada di organisasi pilihan tim.

1. Baca tulisan Noonan secara utuh dan catat setiap masalah yang disebut beserta angkanya.
2. Kelompokkan masalah itu menjadi penyebab, misalnya cara memecah layanan, ukuran tim, pustaka bersama, dan pengujian.
3. Untuk setiap penyebab, cari bukti yang setara di organisasi pilihan tim.
4. Tandai setiap penyebab dengan “ada”, “sebagian”, atau “tidak ada”.
5. Bandingkan hasil dengan contoh pada Tabel 7.15, lalu isi Tabel 7.16.

Tabel 7.15: Latihan 3, contoh terisi: penyebab di Segment dibandingkan dengan Medikara Health

Penyebab	Bukti di Segment	Di Medikara?	Bukti di Medikara
Pemecahan menurut hal teknis, bukan domain	Satu layanan per tujuan pengiriman, lebih dari 140 layanan	Sebagian	Usulan vendor mengikuti menu layar SIMRS lama
Layanan melampaui kemampuan tim	Tiga insinyur penuh waktu sibuk menjaga sistem	Ada, bila usulan diterima	32 layanan untuk 18 orang
Versi pustaka bersama menyimpang	Setiap basis kode akhirnya memakai versi berbeda	Ada	Lima SIMRS dengan versi 3.9 sampai 5.2
Pengujian lambat dan rapuh	Pengujian seluruh tujuan sampai satu jam	Ada	Pengujian manual sekitar dua hari kerja per bagian

Tabel 7.16: Latihan 3, lembar isian: penyebab di Segment dibandingkan dengan organisasi pilihan tim

Penyebab	Bukti di Segment	Di organisasi?	Bukti di organisasi
…	…	…	…
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Noonan menyebut tiga kerugian setelah kembali ke monolit. Kerugian mana yang paling berat bagi rumah sakit, mengingat modul klaim menyumbang 22 dari 63 gangguan di Medikara?
2. Penyebab mana pada tabel isian yang dapat diatasi tanpa mengubah gaya arsitektur sama sekali? Jelaskan berdasarkan bukti yang diisi.

Bab 8

Container, Kubernetes, dan Serverless

Tujuan Pembelajaran

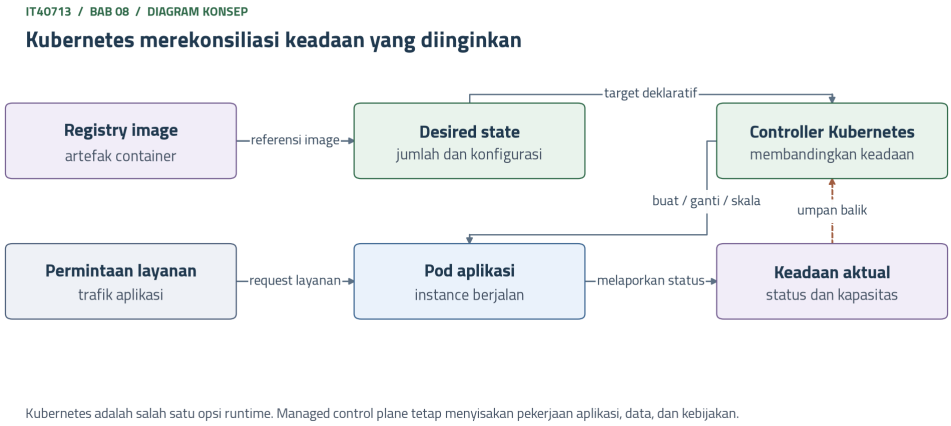
- Menjelaskan perbedaan *virtual machine*, *container*, dan *serverless* dengan analogi dan contoh.
- Menjelaskan cara Kubernetes menjaga sistem tetap sesuai keadaan yang diinginkan.
- Memilih *runtime* untuk setiap kandidat layanan berdasarkan kriteria yang eksplisit.

8.1 Pendahuluan

Misalkan tim teknologi informasi (TI) pusat Medikara Health merilis modul pendaftaran rawat jalan yang baru. Di rumah sakit Jakarta modul itu berjalan mulus. Di rumah sakit Bandung modul yang sama gagal menyimpan tanggal kunjungan. Kodenya tidak berbeda. Yang berbeda adalah lingkungannya: server Bandung, warisan akuisisi, memakai versi sistem operasi dan pustaka yang lain. Tim butuh tiga hari untuk menemukan bahwa satu pustaka pembaca format tanggal berbeda versi. Pada saat yang sama, skrip pengingat jadwal kontrol pasien berjalan di server yang menyala 24 jam sehari, padahal pekerjaannya selesai dalam beberapa menit setiap pagi dan sore.

Dua masalah itu bermuara pada satu pertanyaan: di mana dan dengan cara apa sebuah layanan dijalankan. Tempat dan cara menjalankan itu disebut *runtime*. Bab ini membahas enam pilihan *runtime*: *virtual machine* (VM), *container*, Kuber-

netes yang dikelola sendiri, *managed* Kubernetes, Platform as a Service (PaaS), dan *serverless*. Setiap pilihan memindahkan sebagian pekerjaan dari organisasi ke penyedia, dan setiap pilihan membawa *tradeoff* sendiri antara kendali, biaya, keterampilan, dan ketergantungan. Tidak ada pilihan yang terbaik untuk semua layanan. Pertanyaan yang tepat adalah pilihan mana yang paling cocok untuk layanan tertentu, dengan pola beban tertentu, yang dijalankan oleh tim tertentu. Pada Kubernetes, controller membandingkan keadaan aktual dengan target deklaratif, lalu membuat, mengganti, atau menambah pod sesuai kebutuhan. Gambar 8.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 8.1: Kubernetes merekonsiliasi keadaan yang diinginkan

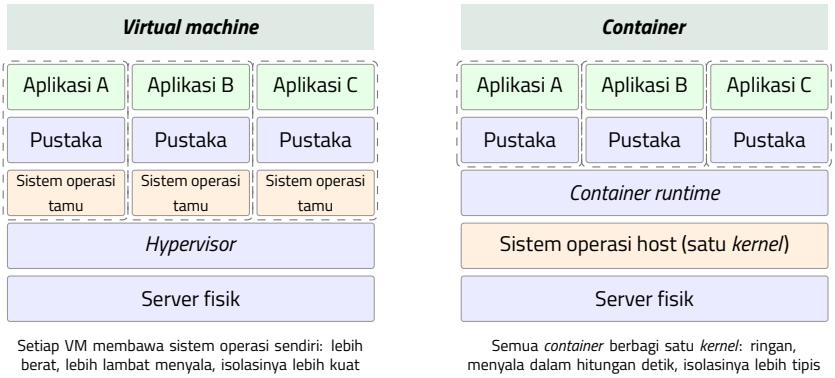
Rujukan utama bab ini adalah panduan keamanan *container* dari National Institute of Standards and Technology (NIST), NIST SP 800-190 [133], dokumentasi resmi Kubernetes [142], tulisan Burns dan rekan tentang asal-usul Kubernetes [31], kajian Berkeley tentang *serverless* [80], dan survei tahunan Cloud Native Computing Foundation (CNCF) untuk tahun 2025 [33]. Bab ini juga memakai matriks kendala penempatan *workload* dari Bab 2 dan model biaya dari Bab 3. Hasilnya adalah matriks keputusan *runtime* per layanan untuk tonggak capstone C3.

8.2 Virtual Machine dan Container

VM adalah komputer tiruan yang dibuat oleh perangkat lunak bernama *hypervisor* di atas server fisik. Setiap VM membawa sistem operasi lengkapnya sendiri, yang disebut sistem operasi tamu, sehingga satu server fisik dapat menampung banyak VM yang saling terpisah seperti komputer yang berbeda. *Container* adalah

paket berisi aplikasi beserta pustaka dan pengaturan yang dibutuhkannya, yang dijalankan sebagai proses terisolasi di atas satu sistem operasi host. *Container* tidak membawa sistem operasi lengkap. Semua *container* pada satu host berbagi *kernel*, yaitu inti sistem operasi, yang sama [133]. Akibatnya *container* jauh lebih kecil dan lebih cepat menyala, sedangkan VM lebih berat tetapi batas isolasinya lebih tebal. Gambar 8.2 memperlihatkan perbedaan susunannya.

Analoginya adalah peti kemas. Sebelum ukuran peti kemas distandarkan, barang dimuat ke kapal satu per satu: karung, peti kayu, dan drum, masing-masing dengan cara angkat sendiri. Setelah ukurannya seragam, isi peti boleh apa saja, tetapi *crane*, kapal, dan truk di pelabuhan mana pun mengangkutnya dengan cara yang sama. *Container* perangkat lunak memakai gagasan yang sama: isinya berbeda-beda, cara menjalankannya seragam. VM lebih mirip gudang utuh yang disewa setiap pemilik barang: lebih lega dan lebih aman, tetapi setiap gudang punya pintu, satpam, dan listrik sendiri.



Gambar 8.2: Susunan lapisan pada *virtual machine* dan *container*: VM menggandakan sistem operasi, *container* berbagi satu *kernel*

Dua contoh dari Medikara Health:

1. Modul pendaftaran rawat jalan pada pendahuluan dikemas sebagai satu *container* bersama pustaka pembaca tanggal versi yang sudah diuji. Kemasan yang sama dijalankan di lima rumah sakit. Tiga hari pencarian penyebab tadi tidak perlu terulang, karena lingkungan aplikasi ikut terbawa di dalam kemasannya.
2. Misalkan rumah sakit Depok menjalankan 12 aplikasi pendukung kecil, masing-masing di VM sendiri. Bila setiap sistem operasi tamu memakai sekitar 2 GB memori, 24 GB memori habis hanya untuk sistem operasi, dan 12 sistem operasi harus diberi *patch* setiap bulan. Bila keduabelas aplikasi itu dikemas sebagai *container* di atas dua host, sistem operasi yang harus dirawat tinggal dua.

8.2.1 Container Image dan Registry

Container image adalah templat baca-saja yang berisi aplikasi, pustaka, dan pengaturan awalnya, disusun berlapis-lapis. *Container* adalah *image* yang sedang dijalankan. Format *image* distandarkan oleh Open Container Initiative [112], sehingga *image* yang dibuat dengan satu alat dapat dijalankan oleh *runtime* lain. *Registry* adalah tempat menyimpan dan membagikan *image*. Cara *registry* menerima dan mengirim *image* juga distandarkan [111]. Setiap *image* diberi *tag*, yaitu nama versi yang mudah dibaca, dan *digest*, yaitu sidik jari yang dihitung dari isi *image* itu sendiri.

Analoginya seperti dokumen induk di lemari arsip kantor pusat. Dokumen induk dikunci dan tidak pernah dicoret. Setiap cabang mengambil salinan dari lemari yang sama, lalu bekerja dengan salinan itu. Bila salinan rusak, cabang cukup mengambil salinan baru. Dokumen induk adalah *image*, salinan yang sedang dikerjakan adalah *container*, dan lemari arsip bersama adalah *registry*.

Tabel 8.1 merangkum istilah yang paling sering muncul dalam percakapan tentang *container*.

Tabel 8.1: Istilah dasar *container* dan contohnya di Medikara Health

Istilah	Arti	Contoh di Medikara Health
Container image	Paket baca-saja berisi aplikasi dan dependensinya	`jadwal-dokter:1.4.2`
Container	*Image* yang sedang berjalan sebagai proses terisolasi	Tiga *container* layanan jadwal dokter di rumah sakit Bandung
Registry	Gudang *image*, publik atau privat	*Registry* privat di *region* Jakarta
Tag	Nama versi yang mudah dibaca dan dapat dipindahkan ke *image* lain	`1.4.2`
Digest	Sidik jari isi *image*; berubah bila satu bit isinya berubah	Dipakai di produksi agar versi yang berjalan pasti sama dengan yang diuji
Dockerfile	Resep tertulis untuk membangun *image*	`sources/chapter08/contoh-container/Dockerfile`

Konsistensi lingkungan ini punya dua konsekuensi. Pertama, *image* yang diuji adalah *image* yang sama dengan yang dijalankan di produksi. Gagasan ini sejalan dengan pemisahan tahap *build*, *release*, dan *run* serta *immutable infrastructure* pada Bab 5. Kedua, *registry* menjadi aset kritis. *Image* yang disusupi akan tersebar ke lima rumah sakit sekaligus, sehingga *registry* memerlukan kontrol akses dan pemindaian kerentanan. *Image* juga tidak boleh memuat data pasien, karena *image* disalin ke banyak tempat.

8.2.2 Batas Isolasi Container

Karena berbagi satu *kernel*, *container* pada satu host tidak terpisah setebal VM. Celah keamanan pada *kernel*, atau satu *container* yang diberi hak berlebihan, dapat berdampak pada semua *container* di host itu. NIST SP 800-190 menyarankan beberapa langkah, antara lain mengelompokkan *container* dengan tujuan, tingkat sensitivitas, dan profil ancaman yang sama pada host yang sama, memakai sistem operasi host yang khusus untuk *container*, dan memindai *image* sebelum dijalankan [133].

Dua hal pada isolasi sering tertukar. Pembagiannya begini. VM memisahkan beban pada tingkat perangkat keras tiruan, dan setiap VM punya *kernel* sendiri. *Container* memisahkan beban pada tingkat proses di dalam satu *kernel*. *Container* bukan VM kecil. Tabel 8.2 membandingkan keduanya.

Tabel 8.2: Perbandingan *virtual machine* dan *container*

Aspek	***Virtual machine***	***Container***
Isi kemasan	Sistem operasi tamu, pustaka, aplikasi	Pustaka dan aplikasi saja
Ukuran	Besar, karena memuat sistem operasi lengkap	Kecil, hanya yang dibutuhkan aplikasi
Waktu menyala	Lebih lama, sistem operasi harus dinyalakan dulu	Cepat, cukup memulai proses
Batas isolasi	*Kernel* sendiri untuk setiap VM	Berbagi satu *kernel* host
Kepadatan per server	Lebih sedikit beban per server	Lebih banyak beban per server
Cocok untuk	Aplikasi lama yang menuntut sistem operasi tertentu; beban yang menuntut isolasi ketat	Layanan baru yang sering diperbarui dan perlu bertambah kapasitas dengan cepat

Dalam praktik keduanya sering dipakai bersamaan: *container* dijalankan di atas VM yang disewa dari penyedia *cloud*. Bagi Medikara Health, saran pengelompokan tadi berarti *container* yang melayani permintaan dari internet, misalnya antarmuka Medikara+, sebaiknya tidak satu host dengan *container* yang mengolah rekam medis elektronik.

Pendalaman Teknis (opsional): Membangun Container Image

Listing 8.1 adalah Dockerfile untuk layanan jadwal dokter tiruan di `sources/chapter08/contoh-container/app.py`. Layanan itu hanya memakai pustaka standar Python, sehingga *image*-nya tidak membawa dependensi pihak ketiga. Setiap instruksi Dockerfile dijelaskan di dokumentasi resminya [44].

Listing 8.1: Dockerfile layanan jadwal dokter, tersedia di `sources/chapter08/contoh-container/Dockerfile`

```
# Membangun image layanan jadwal dokter. Jalankan di folder ini:
#   docker build -t jadwal-dokter:1.0.0 .
# Image dasar versi slim: ukuran kecil, permukaan serangan lebih sempit.
FROM python:3.12-slim

# Aplikasi berjalan sebagai pengguna biasa, bukan root. Bila aplikasi
# dibobol, penyerang tidak langsung berkuasa penuh atas container.
RUN useradd --create-home --uid 10001 layanan
WORKDIR /app
COPY app.py .
USER layanan

# Nilai bawaan. Nilai sebenarnya diisi saat container dijalankan.
ENV PORT=8080
ENV NAMA_FASILITAS="RS Medikara Jakarta"
EXPOSE 8080

# Satu proses per container, keluarannya langsung ke stdout.
CMD ["python", "app.py"]
```

Tiga keputusan dalam berkas itu layak diperhatikan. Pertama, *image* dasar versi `slim` dipilih agar ukuran dan permukaan serangan kecil. Kedua, aplikasi berjalan sebagai pengguna biasa, bukan `root`, sesuai saran NIST SP 800-190. Ketiga, nama fasilitas dan port diisi saat *container* dijalankan, sehingga satu *image* melayani lima rumah sakit. *Image* dibangun dengan `docker build`, diberi nama *registry* dengan `docker tag`, lalu dikirim dengan `docker push`. Untuk latihan, *registry* lokal atau *registry* gratis sudah cukup.

8.3 Orkestrasi Container dan Konsep Kubernetes

Menjalankan tiga *container* di satu server cukup dengan beberapa perintah manual. Misalkan arsitektur target Medikara Health memuat sembilan kandidat layanan, masing-masing dengan tiga replika, di tiga lingkungan (pengembangan, uji, dan produksi). Jumlahnya 81 *container* yang tersebar di belasan server. Server mana yang masih punya ruang? Siapa yang menyalakan ulang *container* yang mati pukul 02.00? Bagaimana versi baru dipasang tanpa menghentikan layanan? Siapa yang menambah kapasitas ketika pasien berebut kuota poli pukul 07.00?

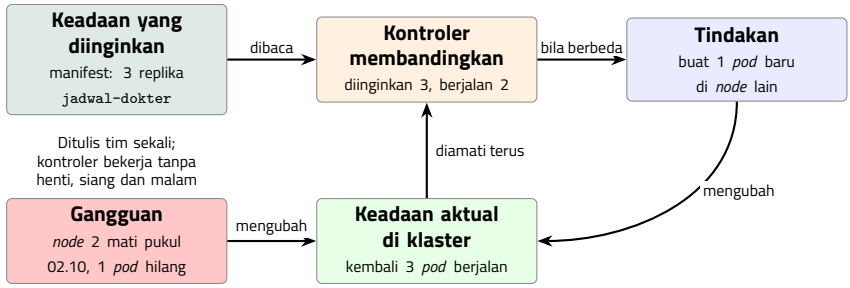
Pertanyaan kemudian muncul: siapa yang mengurus semua itu? Jawabannya adalah orkestrator. Orkestrasi *container* adalah pengelolaan otomatis atas penempatan, penjalanan, pemulihan, penambahan kapasitas, dan pembaruan *container* pada sekumpulan server yang disebut klaster. Kubernetes adalah orkestrator *open source* yang paling banyak dipakai. Kubernetes lahir dari pengalaman Google lebih dari satu dasawarsa menjalankan Borg dan Omega, dua sistem pengelola klaster

internalnya [31]. Dokumentasi resminya menyebut kemampuan inti Kubernetes, antara lain penemuan layanan dan pembagian beban, pembaruan dan pembatalan bertahap, serta pemulihan diri [142].

8.3.1 Desired State dan Rekonsiliasi

Dalam Kubernetes, tim tidak memberi perintah langkah demi langkah seperti "jalankan *container* ini di server 3". Tim menuliskan keadaan yang diinginkan (*desired state*) dalam berkas yang disebut manifest, misalnya "layanan jadwal dokter selalu punya tiga replika versi 1.4.2". Kontroler adalah lingkaran kendali yang terus mengamati keadaan klaster, membandingkannya dengan keadaan yang diinginkan, lalu bertindak untuk mendekatkan keduanya [139]. Proses membandingkan dan membetulkan itu disebut rekonsiliasi. Gambar 8.3 memperlihatkan satu putaran rekonsiliasi.

Analoginya adalah kepala ruang perawat di bangsal rumah sakit. Aturan di papan jaga hanya satu angka: enam perawat setiap sif. Kepala ruang tidak menunggu keluhan pasien. Setiap awal sif ia menghitung perawat yang hadir. Bila satu perawat sakit, ia memanggil pengganti dari daftar cadangan. Bila yang hadir tujuh karena salah jadwal, satu orang diminta pulang. Yang ditulis di papan hanya angka enam; cara memenuhinya diurus kepala ruang.



Gambar 8.3: Satu putaran rekonsiliasi: kontroler membandingkan keadaan yang diinginkan dengan keadaan aktual, lalu bertindak sampai keduanya sama

Desired state punya dua konsekuensi. Pertama, keadaan sistem dapat dibaca dan diaudit dari berkas manifest, bukan dari ingatan petugas. Manifest disimpan di repositori kode bersama riwayat perubahannya, sejalan dengan konfigurasi deklaratif pada Bab 5 dan GitOps pada Bab 13. Kedua, perubahan manual langsung di klaster akan dikembalikan oleh kontroler bila bertentangan dengan manifest. Disiplin perubahan karena itu harus lewat manifest, bukan lewat perintah dadakan di tengah malam.

8.3.2 Pod, Deployment, dan Service

Pod adalah unit terkecil yang dijalankan Kubernetes: satu atau beberapa *container* yang selalu ditempatkan bersama dan berbagi alamat jaringan [143]. *Pod* bersifat sementara; *pod* yang mati diganti dengan *pod* baru, bukan diperbaiki. *Deployment* adalah objek yang menyatakan jumlah replika *pod* dan versi *image* yang diinginkan. Kontroler *deployment* mengganti *pod* yang hilang dan memasang versi baru secara bertahap [140]. *Service* memberi satu alamat tetap dan membagi permintaan ke sekelompok *pod*, karena *pod* datang dan pergi dengan alamat yang berubah-ubah [145]. Klaster terdiri atas *control plane* yang mengambil keputusan dan *node*, yaitu server tempat *pod* berjalan [138].

Tabel 8.3 merangkum konsep itu beserta contohnya pada layanan Medikara Health.

Tabel 8.3: Konsep Kubernetes pada tingkat konsep dan contohnya di Medikara Health

Konsep	Fungsi	Contoh di Medikara Health
Klaster dan *node*	Sekumpulan server yang dikelola sebagai satu kesatuan; *node* adalah satu server di dalamnya	Klaster produksi dengan enam *node* di *region* Jakarta
Control plane	Bagian yang menyimpan keadaan klaster, menjadwalkan *pod*, dan menjalankan kontroler	Dikelola penyedia bila memakai *managed* Kubernetes
Pod	Unit terkecil yang dijalankan, berisi satu atau beberapa *container*	Satu *pod* layanan jadwal dokter
Deployment	Menyatakan jumlah replika dan versi *image*	Tiga replika `jadwal-dokter:1.4.2`
Service	Alamat tetap untuk *pod* yang berganti-ganti	Layanan janji temu memanggil `jadwal-dokter` tanpa tahu *pod* mana yang hidup
Self-healing	Mengganti *container* atau *pod* yang mati atau tidak sehat	*Pod* pengganti muncul sekitar satu menit setelah satu *node* mati
Autoscaling	Menambah atau mengurangi replika menurut beban	Tiga sampai delapan replika layanan janji temu antara pukul 07.00 dan 09.00

8.3.3 Self-Healing dan Autoscaling

Self-healing adalah kemampuan Kubernetes memulihkan dirinya tanpa campur tangan petugas: menyalakan ulang *container* yang gagal, mengganti *pod* di *node* yang mati, dan mematikan *container* yang tidak menjawab pemeriksaan kesehatan [142]. *Autoscaling* adalah penambahan dan pengurangan kapasitas secara otomatis. Pada tingkat *pod*, Horizontal Pod Autoscaler menambah atau mengurangi replika berdasarkan ukuran beban, misalnya rata-rata pemakaian prosesor [141]. Kedua kemampuan ini adalah buah langsung dari rekonsiliasi: *autoscaler* cukup

mengubah angka replika yang diinginkan, lalu kontroler mengerjakan sisanya.

Dua contoh kasus dari Medikara Health:

1. Misalkan pukul 02.10 satu *node* di klaster produksi mati karena gangguan perangkat keras. Dua dari tiga replika layanan jadwal dokter tetap melayani. Kontroler membuat *pod* pengganti di *node* lain dalam sekitar satu menit. Tidak ada petugas yang ditelepon tengah malam; tim membaca laporan kejadian pagi harinya.
2. Permintaan ke layanan janji temu, termasuk pengecekan jadwal dan kuota, naik dari sekitar 10 permintaan per detik pada pukul 05.00 menjadi 50 permintaan per detik antara pukul 07.00 dan 09.00, ketika pasien berebut kuota poli. Yang berakhir menjadi janji temu sekitar 175 per jam (Bab 9). *Autoscaler* diatur untuk menjaga pemakaian prosesor rata-rata 70 persen, dengan batas tiga sampai delapan replika. Replika naik menjadi delapan pada puncak, lalu turun lagi menjadi tiga menjelang siang. Kapasitas tambahan hanya dibayar selama dipakai, asalkan jumlah *node* juga ikut menyusut.

8.3.4 Adopsi Kubernetes menurut Survei CNCF 2025

Survei tahunan CNCF untuk tahun 2025, yang terbit pada Januari 2026, mengumpulkan jawaban 628 responden pada September 2025. Survei itu mencatat bahwa 82 persen pengguna *container* menjalankan Kubernetes di produksi, naik dari 66 persen pada 2023. Survei yang sama mencatat bahwa 66 persen organisasi yang menjalankan sendiri model Artificial Intelligence (AI) generatif memakai Kubernetes untuk mengelola sebagian atau seluruh beban inferensinya [33, 32]. Kubernetes dengan demikian sedang menjadi platform bersama, bukan hanya untuk aplikasi biasa tetapi juga untuk *workload* AI.

Pembacaan tingkat S2 tidak berhenti pada angka utama. Tabel 8.4 menaruh setiap angka bersama penyebutnya, yaitu kelompok yang menjadi dasar persentase itu.

Tabel 8.4: Membaca angka utama survei CNCF 2025 secara kritis

Indikator	Angka	Cara membaca
Jumlah responden dan waktu survei	628, September 2025	Sampel dari komunitas *cloud native*, bukan sensus seluruh organisasi
Pengguna *container* yang menjalankan Kubernetes di produksi	82 persen (2023: 66 persen)	Penyebutnya pengguna *container*, bukan seluruh organisasi
Organisasi yang menjalankan sendiri model AI generatif dan memakai Kubernetes untuk inferensi	66 persen	Penyebutnya hanya organisasi yang menjalankan model sendiri

Hasil pembacaan ini punya dua konsekuensi. Pertama, ekosistem Kubernetes besar: alat, dokumentasi, dan tenaga kerja relatif mudah ditemukan, dan hal ini menurunkan risiko memilih teknologi yang ditinggalkan. Kedua, angka adopsi tidak menjawab apakah Kubernetes tepat untuk organisasi dengan tim kecil. Responden survei adalah komunitas yang sudah dekat dengan *container*, sehingga angka 82 persen tidak berlaku bagi organisasi yang belum memakai *container* sama sekali.

Pendalaman Teknis (opsional): Menjalankan Layanan di Klaster Lokal

Klaster Kubernetes lokal dapat dibuat di laptop dengan kind, yang menjalankan *node* Kubernetes sebagai *container* [86], atau dengan minikube [146]. Listing 8.2 adalah manifest *deployment* untuk layanan jadwal dokter. Baris `replicas: 3` adalah *desired state* yang dijaga kontroler. Pasangannya, `sources/chapter08/contoh-container/service.yaml`, memberi alamat tetap bagi ketiga *pod*.

Listing 8.2: Manifest *deployment* layanan jadwal dokter, tersedia di `sources/chapter08/contoh-container/deployment.yaml`

```yaml
# Deployment contoh Bab 08: tiga replika layanan jadwal dokter.
# Pakai: kubectl apply -f deployment.yaml pada klaster lokal kind/minikube.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jadwal-dokter
  labels:
    app: jadwal-dokter
spec:
  # Keadaan yang diinginkan: selalu tiga pod. Bila satu mati,
  # kontroler membuat penggantinya tanpa perintah manual.
  replicas: 3
  selector:
    matchLabels:
      app: jadwal-dokter
  template:
    metadata:
      labels:
        app: jadwal-dokter
    spec:
      containers:
        - name: jadwal-dokter
          image: jadwal-dokter:1.0.0
          ports:
            - containerPort: 8080
          env:
            - name: NAMA_FASILITAS
              value: "RS Medikara Bandung"
          # Jatah sumber daya: satu pod tidak boleh menghabiskan node.
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
```

```
            limits:
              cpu: "250m"
              memory: "128Mi"
          # Tiga kali /sehat tidak menjawab, container dimulai ulang.
          livenessProbe:
            httpGet:
              path: /sehat
              port: 8080
            periodSeconds: 10
            failureThreshold: 3
```

Listing 8.3 memuat urutan perintah dari membangun *image* sampai menghapus klaster. Perintah `kubectl delete pod` sengaja mematikan satu *pod* agar *self-healing* terlihat: dalam beberapa detik *pod* pengganti muncul dengan nama baru.

Listing 8.3: Menjalankan layanan jadwal dokter di klaster kind, tersedia di `sources/chapter08/README.md`

```
# Jalankan dari folder sources/chapter08/contoh-container.
docker build -t jadwal-dokter:1.0.0 .
kind create cluster --name medikara-lokal
kind load docker-image jadwal-dokter:1.0.0 --name medikara-lokal
kubectl apply -f deployment.yaml -f service.yaml
kubectl get pods -l app=jadwal-dokter
# Hapus satu pod, lalu lihat penggantinya muncul (self-healing).
kubectl delete pod NAMA-POD-DARI-PERINTAH-SEBELUMNYA
kubectl get pods -l app=jadwal-dokter
kubectl port-forward service/jadwal-dokter 8080:80
# Selesai: hapus klaster agar laptop tidak terbebani.
kind delete cluster --name medikara-lokal
```

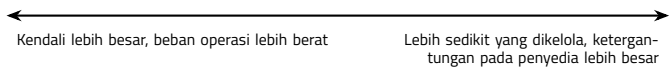
Peringatan: uji beban untuk melihat *autoscaling* hanya dijalankan pada klaster lokal milik sendiri atau sistem yang sudah berizin tertulis, tidak pada sistem milik pihak lain.

8.4 Managed Kubernetes dan PaaS

Kubernetes tidak menghapus pekerjaan operasi; Kubernetes memindahkannya. Klaster harus dijalankan, diamankan, dipantau, dan diperbarui. *Managed* Kubernetes adalah layanan penyedia *cloud* yang menjalankan *control plane* untuk pelanggan: ketersediaannya, cadangan datanya, dan pembaruan versinya. Pelanggan tetap mengurus manifest dan layanannya, *image* beserta kerentanannya, hak akses, jaringan, komponen tambahan, dan umumnya juga pembaruan *node*. Layanan ini berbayar per klaster. Amazon Web Services (AWS), misalnya, menagih 0,10 dolar AS per klaster per jam untuk versi Kubernetes yang masih dalam dukungan standar, dan 0,60 dolar AS per klaster per jam untuk versi yang sudah masuk dukungan diperpanjang [19].

PaaS menyembunyikan lebih banyak lagi. Tim cukup menyerahkan kode atau *image* beserta beberapa pengaturan, lalu platform mengurus server, *runtime*, penambahan kapasitas, dan pembaruan sistem operasi. Banyak PaaS modern dibangun di atas orkestrator, tetapi orkestratornya tidak terlihat oleh pelanggan. Imbalannya, kendali berkurang: pilihan jaringan, versi *runtime*, dan aturan penambahan kapasitas mengikuti apa yang disediakan platform. Gambar 8.4 memetakan pembagian tanggung jawab itu dari VM pada Infrastructure as a Service (IaaS) sampai Software as a Service (SaaS).

Yang dikelola	**VM (IaaS)**	**Kubernetes sendiri**	***Managed* Kubernetes**	**PaaS**	***Serverless***	**SaaS**
Data dan hak akses pengguna	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Pelanggan
Kode aplikasi	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Pelanggan	Penyedia
Penambahan kapasitas	Pelanggan	Pelanggan	Bersama	Bersama	Penyedia	Penyedia
Runtime bahasa dan pustaka	Pelanggan	Pelanggan	Pelanggan	Bersama	Bersama	Penyedia
Orkestrasi (*control plane*)	Pelanggan	Pelanggan	Penyedia	Penyedia	Penyedia	Penyedia
Sistem operasi dan *patch*	Pelanggan	Pelanggan	Bersama	Penyedia	Penyedia	Penyedia
Server fisik dan gedung	Penyedia	Penyedia	Penyedia	Penyedia	Penyedia	Penyedia



Kendali lebih besar, beban operasi lebih berat

Lebih sedikit yang dikelola, ketergantungan pada penyedia lebih besar

Gambar 8.4: Pembagian tanggung jawab pada enam pilihan *runtime*; kolom Kubernetes sendiri diasumsikan berjalan di atas VM sewaan

Analoginya adalah dapur. Kubernetes yang dikelola sendiri sama dengan membangun dan mengoperasikan dapur sendiri: semua bebas diatur, tetapi kompor rusak, gas habis, dan izin usaha menjadi urusan sendiri. *Managed* Kubernetes sama dengan menyewa dapur komersial: gedung, gas, dan perawatan alat diurus pengelola, tetapi koki, menu, dan kebersihan tetap tanggung jawab penyewa. PaaS sama dengan katering yang memasak resep pelanggan: resep diserahkan, katering memasak dan mengantar, tetapi alat masak dan cara penyajian mengikuti kebiasaan katering itu.

Dua hal pada bagian ini sering tertukar. Pembagiannya begini. *Managed* Kubernetes tetap memperlihatkan Kubernetes secara utuh: tim menulis manifest, memilih komponen tambahan, dan mengatur kebijakan jaringan sendiri. PaaS menyembunyikan Kubernetes, walaupun di baliknya mungkin ada Kubernetes. Keterampilan yang dibutuhkan keduanya berbeda jauh.

Dua contoh berangka:

1. Medikara Health butuh tiga klaster: pengembangan, uji, dan produksi. De-

ngan tarif 0,10 dolar AS per jam dan sekitar 730 jam per bulan, biaya *control plane* sebesar 219 dolar AS per bulan, atau sekitar Rp3,6 juta dengan kurs asumsi Rp16.500 per dolar AS. Bila tim terlambat memperbarui versi sehingga ketiganya masuk dukungan diperpanjang, tarifnya menjadi 1.314 dolar AS per bulan, atau sekitar Rp21,7 juta. Keterlambatan pembaruan punya harga yang dapat dihitung.

2. Proyek Kubernetes merilis versi minor sekitar tiga kali setahun, dan setiap versi minor mendapat dukungan *patch* sekitar satu tahun [144]. Agar tetap dalam dukungan, setiap klaster harus diperbarui paling tidak sekali setahun, dan dalam praktik lebih sering. Misalkan setiap pembaruan, termasuk pengujian sembilan layanan, menyita lima hari kerja dua orang. Dengan tiga pembaruan per klaster per tahun di tiga klaster, beban itu mencapai 90 hari kerja per tahun.

Biaya yang paling sering diremehkan adalah biaya keterampilan. Tarif *control plane* mudah dibaca di halaman harga. Gaji orang yang mampu mengoperasikan klaster, pelatihannya, waktu rekrutmennya, dan risiko bila orang itu keluar tidak tercantum di halaman mana pun.

Kasus Medikara Health: Sanggupkah Tim TI Mengelola Kubernetes?

Misalkan arsitektur target memuat sembilan kandidat layanan baru di belakang Medikara+ dan SIMRS (Sistem Informasi Manajemen Rumah Sakit). Angka ini asumsi perencanaan yang memperluas Tabel 6.7. Inti SIMRS tetap *modular monolith* sesuai ADR-001 di Bab 7 dan tidak dihitung. Yang dihitung adalah janji temu dan layanan pendukung di sekitarnya, misalnya jadwal dokter, identitas pasien, notifikasi, pengingat jadwal kontrol, dan integrasi. Dari 52 staf TI, hanya 4 orang yang bersertifikat *cloud* (Bab 4), dan belum ada yang pernah mengoperasikan Kubernetes di produksi. Tim transformasi membandingkan tiga opsi untuk menjalankan sembilan layanan itu selama satu tahun: Kubernetes yang dikelola sendiri di atas VM sewaan, *managed* Kubernetes di *region* Jakarta, dan PaaS di *region* Jakarta. Hasilnya ada di Tabel 8.5.

Hasil ini punya dua konsekuensi. Pertama, biaya tenaga kerja mendominasi. Pada opsi *managed* Kubernetes, tenaga kerja platform menelan Rp780 juta atau 46,9 persen dari total, sedangkan tarif *control plane* hanya Rp43 juta atau 2,6 persen. Diskusi yang hanya membandingkan tarif layanan melewatkan hampir separuh biaya. Kedua, keputusan tidak berhenti pada angka. Merekrut dua *platform engineer* berpengalaman diperkirakan memakan empat sampai enam bulan, dan bila salah satunya keluar, operasi klaster bergantung pada satu orang.

Tim mengusulkan PaaS untuk layanan baru pertama, yang menurut peta jalan Bab 14 mulai berjalan pada gelombang 2, karena lebih murah Rp463 juta atau 27,8 persen daripada *managed* Kubernetes dan dapat dijalankan oleh tim yang ada. *Managed* Kubernetes ditinjau ulang bila jumlah layanan melewati 15 atau bila dua *platform engineer* sudah bergabung. Kubernetes yang dikelola sendiri ditolak.

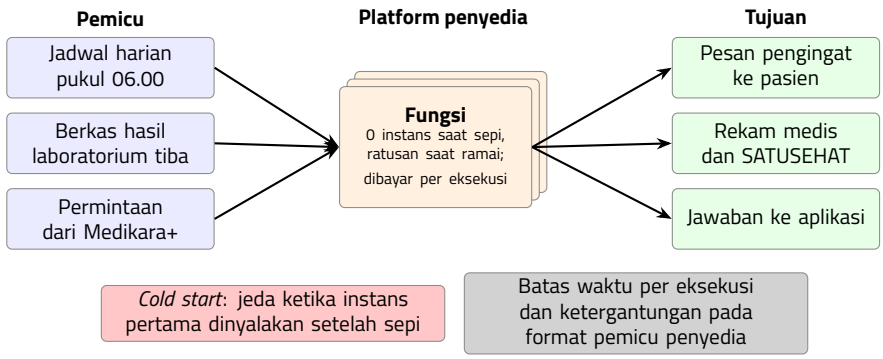
Tabel 8.5: Biaya satu tahun untuk menjalankan sembilan layanan Medikara Health, dalam juta rupiah (angka rekaan kecuali tarif *control plane*)

Komponen	Kubernetes sendiri	*Managed* Kubernetes	PaaS
Control plane (sendiri: 3 klaster dengan VM khusus; *managed*: tarif penyedia)	180	43	0
Kapasitas komputasi untuk sembilan layanan	600	600	720
Tenaga kerja platform (Rp30 juta per orang per bulan, 13 kali setahun)	1.560	780	390
Jumlah orang	4	2	1
Pelatihan dan sertifikasi	120	90	30
Alat pemantauan dan keamanan klaster	150	150	60
Total satu tahun	**2.610**	**1.663**	**1.200**

8.5 Serverless dan Function as a Service

Serverless adalah model komputasi tempat server tetap ada, tetapi sepenuhnya diurus penyedia. Pelanggan tidak menyediakan atau mengatur kapasitas. Kapasitas bertambah otomatis ketika beban naik, menyusut sampai nol ketika tidak ada beban, dan tagihan mengikuti pemakaian nyata [35]. Function as a Service (FaaS) adalah bentuk *serverless* yang paling dikenal. Kode ditulis sebagai fungsi kecil yang dijalankan setiap kali ada pemicu: permintaan dari aplikasi, berkas yang tiba di penyimpanan, jadwal waktu, atau pesan di antrean. Kajian Berkeley merumuskan *serverless* sebagai gabungan FaaS dengan layanan pendukung terkelola, seperti penyimpanan objek dan basis data, yang mereka sebut *Backend as a Service* [80]. Gambar 8.5 memperlihatkan alurnya.

Analoginya adalah lampu sensor gerak di lorong kantor. Lampu menyala sendiri ketika ada orang lewat, padam lagi ketika lorong sepi, dan listrik hanya terpakai selama lampu menyala. Ada jeda sesaat sebelum lampu menyala ketika orang pertama masuk. Untuk lorong, jeda itu tidak masalah. Untuk ruang operasi yang harus terang tanpa jeda sedetik pun, lampu sensor bukan pilihan.



Gambar 8.5: Alur FaaS: pemicu menyalakan instans fungsi sesuai kebutuhan, lalu hasilnya dikirim ke layanan tujuan

8.5.1 Skala sampai Nol dan Biaya per Eksekusi

Skala sampai nol berarti tidak ada instans yang berjalan, dan tidak ada biaya komputasi, ketika tidak ada pemicu. Biaya dihitung per eksekusi. Contohnya AWS Lambda, yang menagih dua komponen: jumlah permintaan, sebesar 0,20 dolar AS per satu juta permintaan, dan durasi eksekusi, sebesar 0,0000166667 dolar AS per GB-detik untuk prosesor x86 di *region* AS Timur. Kuota gratis bulanannya satu juta permintaan dan 400.000 GB-detik [20]. GB-detik adalah memori yang dipesan dikalikan lamanya eksekusi. Fungsi bermemori 0,25 GB yang berjalan 0,5 detik memakai 0,125 GB-detik. Harga di *region* Jakarta dan skema penyedia lain perlu dicek di halaman harga masing-masing.

8.5.2 Cold Start, Batas Waktu, dan Ketergantungan pada Penyedia

Kemudahan ini datang bersama tiga keterbatasan. Pertama, *cold start*. Ketika belum ada instans yang siap, platform harus menyiapkan lingkungan dan memuat kode lebih dahulu, sehingga permintaan pertama setelah masa sepi menunggu lebih lama [80]. Kedua, batas waktu eksekusi. AWS Lambda, misalnya, membatasi satu eksekusi paling lama 900 detik atau 15 menit [21]. Proses yang lebih panjang harus dipecah. Ketiga, ketergantungan pada penyedia. Format pemicu, format *event*, pengaturan izin akses, dan layanan pendukungnya umumnya khas penyedia itu. Fungsi juga tidak menyimpan keadaan di antara eksekusi, sehingga data harus disimpan di layanan lain, sejalan dengan prinsip proses tanpa keadaan pada Bab 5.

Keterbatasan itu menentukan pola beban yang cocok. Tabel 8.6 memetakan enam pola beban yang lazim di Medikara Health.

Tabel 8.6: Kecocokan pola beban dengan *serverless*

Pola beban	Contoh di Medikara Health	Kecocokan	Alasan
Jarang dan tidak menentu	Ekspor data atas permintaan auditor, beberapa kali sebulan	Sangat cocok	Tidak ada biaya saat menganggur
Berkala terjadwal	Pengingat jadwal kontrol pukul 06.00 dan 18.00	Cocok	Berjalan singkat, tidak ada pasien yang menunggu jawaban
Dipicu kedatangan data	Berkas hasil laboratorium setiap 30 menit	Cocok, dengan syarat	Setiap eksekusi harus selesai di bawah batas waktu
Lonjakan tajam yang singkat	Pendaftaran kampanye vaksinasi dibuka	Cocok, dengan catatan	Kapasitas menyusul otomatis; *cold start* dan batas eksekusi serentak perlu diuji
Tinggi dan stabil 24 jam	Antarmuka SIMRS untuk ruang rawat inap	Kurang cocok	Pada beban tetap yang tinggi, bayar per eksekusi lebih mahal daripada kapasitas tetap
Proses panjang	Rekonsiliasi klaim bulanan selama tiga jam	Tidak cocok tanpa dipecah	Melampaui batas waktu eksekusi

Kasus Medikara Health: Pengingat Kontrol dan Hasil Laboratorium

Pengingat jadwal kontrol. Medikara Health melayani sekitar 1,2 juta kunjungan per tahun. Misalkan 40 persen kunjungan berakhir dengan jadwal kontrol. Hasilnya sekitar 480.000 jadwal kontrol per tahun, atau 40.000 per bulan. Setiap jadwal mendapat dua pengingat, tiga hari dan satu hari sebelumnya, sehingga ada 80.000 eksekusi per bulan. Setiap eksekusi berjalan 0,5 detik dengan memori 0,25 GB, sehingga total pemakaiannya 10.000 GB-detik per bulan.

Dengan tarif AWS Lambda di atas dan tanpa kuota gratis, biayanya sekitar 0,18 dolar AS per bulan, atau sekitar Rp3.000. Angka itu masih jauh di dalam kuota gratis. Saat ini skrip yang sama berjalan di VM yang menyala 720 jam per bulan dengan biaya Rp650.000, padahal total waktu kerjanya hanya 80.000 kali 0,5 detik, yaitu sekitar 11,1 jam. Pemanfaatan VM itu sekitar 1,5 persen. Biaya terbesar justru ada di luar komputasi: dengan biaya pesan WhatsApp Rp350 per pengingat, biaya pesannya Rp28 juta per bulan, apa pun *runtime*-nya.

Hasil ini punya dua konsekuensi. Pertama, penghematan komputasinya sekitar Rp647.000 per bulan atau Rp7,8 juta per tahun, angka yang kecil. Argumen yang lebih kuat adalah hilangnya server yang harus diberi *patch*, dan kapasitas yang menyesuaikan sendiri ketika jumlah pengingat naik tiga

kali lipat pada bulan kampanye vaksinasi. Kedua, penghematan terbesar ada di sisi pesan, misalnya mengirim pengingat lewat notifikasi aplikasi Medikara+ bagi pasien yang sudah memasangnya.

Hasil laboratorium. Laboratorium rujukan mengirim berkas hasil setiap 30 menit antara pukul 06.00 dan 22.00, yaitu 32 berkas per hari dengan sekitar 150 hasil per berkas. Misalkan desain awal memakai satu fungsi terjadwal yang memproses semua berkas yang menunggu: mengubahnya ke format HL7 Fast Healthcare Interoperability Resources (FHIR), menyimpannya ke rekam medis elektronik, mengirimnya ke SATUSEHAT, dan memberi tahu dokter. Biasanya hanya ada satu berkas, selesai dalam dua menit. Suatu hari sistem laboratorium mati lima jam, lalu sepuluh berkas tiba bersamaan. Pemrosesannya butuh 20 menit, melampaui batas 15 menit, sehingga eksekusi gagal, diulang, dan gagal lagi.

Desain diperbaiki dengan dua fungsi. Fungsi pertama dipicu setiap kali satu berkas tiba dan hanya memecah berkas itu menjadi 150 pesan di antrean. Fungsi kedua memproses satu hasil per eksekusi, sekitar satu detik. Hasil laboratorium adalah bagian rekam medis, sehingga fungsi, antrean, dan penyimpanannya wajib berada di *region* Jakarta sesuai matriks lokasi data Bab 2. Sebelum memilih, tim memastikan layanan FaaS dan pemicunya tersedia di *region* itu.

8.6 Memilih Runtime dan Mengelola Vendor Lock-in

Enam pilihan *runtime* sudah dibahas. Pertanyaan kemudian muncul: bagaimana memilih di antaranya tanpa terjebak selera pribadi atau tren industri? Jawabannya adalah kriteria yang ditulis dan dibobot sebelum opsi mana pun dinilai. Urutan ini penting. Kriteria yang disusun setelah opsi favorit dipilih cenderung dibuat agar opsi itu menang.

8.6.1 Kriteria Memilih Runtime

Keputusan *runtime* memakai dua jenis kriteria. Kriteria gugur adalah syarat mutlak: opsi yang tidak memenuhinya dicoret tanpa dinilai lebih lanjut. Untuk Medikara Health, kriteria gugur utamanya adalah lokasi data dari matriks kendala penempatan Bab 2. Kriteria berbobot dinilai dengan skor 1 sampai 5, lalu dikalikan bobotnya. Bobot mencerminkan apa yang paling penting bagi organisasi dan disepakati seluruh tim sebelum penilaian.

Analoginya seperti kantor yang memilih kendaraan operasional: motor, mobil, bus jemputan, atau taksi daring. Sebelum membandingkan merek, kantor menulis kebutuhannya: jumlah penumpang, jarak, ketersediaan sopir, dan biaya. Ada juga

syarat mutlak. Mobil berpelat ganjil tidak boleh melewati ruas ganjil-genap pada tanggal genap, senyaman apa pun mobilnya. Untuk rute itu, opsi tersebut gugur sebelum penilaian dimulai.

Tabel 8.7 memuat kriteria yang dipakai dalam Aktivitas bab ini.

Tabel 8.7: Kriteria memilih *runtime* beserta bobot contoh

Kriteria	Pertanyaan kunci	Bobot	Sumber data
Lokasi data	Apakah opsi tersedia di lokasi yang diizinkan untuk data layanan ini?	Gugur	Matriks kendala penempatan Bab 2
Kecocokan pola beban	Kapan dan seberapa sering layanan dipakai? Adakah lonjakan atau masa sepi yang panjang?	25	Catatan permintaan, data kunjungan
Kebutuhan latensi	Adakah orang yang menunggu jawaban? Berapa target waktu tanggapnya?	15	Target layanan, dirinci di Bab 11
Keterampilan tim dan pasar tenaga kerja	Siapa yang mengoperasikan layanan ini pukul 02.00? Mudahkah mencari penggantinya?	25	Daftar keterampilan dan penilaian kesiapan Bab 4
Biaya total tiga tahun	Berapa biaya termasuk tenaga kerja, pelatihan, dan biaya keluar?	20	Model biaya Bab 3
Portabilitas dan risiko *lock-in*	Berapa besar upaya untuk pindah penyedia?	15	Daftar layanan khas penyedia yang dipakai

8.6.2 Matriks Keputusan dari SaaS sampai VM

Matriks keputusan menilai setiap opsi, dari SaaS yang paling sedikit dikelola sampai VM yang paling banyak dikelola, dengan kriteria yang sama. Nilai berbobot dihitung dengan mengalikan skor setiap kriteria dengan bobotnya, menjumlahkan hasilnya, lalu membaginya dengan 100. Hasilnya berada pada skala 1 sampai 5, sama dengan skala skor.

Tabel 8.8 memperlihatkan matriks untuk layanan janji temu, yaitu layanan baru di belakang Medikara+ yang mengatur kuota poli dan pemesanan jadwal. Opsi SaaS gugur karena vendor sistem janji temu yang dinilai menyimpan datanya di luar negeri, sedangkan data janji temu memuat identitas pasien.

Dua contoh pembacaan matriks:

1. Untuk layanan janji temu, PaaS menang dengan nilai 4,10. Keunggulannya datang dari keterampilan tim, bukan dari teknologinya. *Managed* Kubernetes justru unggul di pola beban dan latensi, tetapi skor keterampilan 2 menariknya ke 3,50. Bila kelak ada dua *platform engineer* dan skor keterampilan naik menjadi 4, nilai *managed* Kubernetes menjadi 4,00, hampir menyamai PaaS. Keputusan ini karena itu perlu pemicu tinjau ulang yang tertulis.

Tabel 8.8: Matriks keputusan *runtime* untuk layanan janji temu Medikara Health (skor rekaan)

Kriteria	Bobot	SaaS	PaaS	*Managed* K8s	*Serverless*	VM
Lokasi data (gugur)		Gugur	Lolos	Lolos	Lolos	Lolos
Kecocokan pola beban	25		4	5	4	2
Kebutuhan latensi	15		4	5	3	4
Keterampilan tim	25		5	2	4	4
Biaya total tiga tahun	20		4	2	4	3
Portabilitas dan *lock-in*	15		3	4	2	4
Nilai berbobot	**100**	**-**	**4,10**	**3,50**	**3,55**	**3,30**

2. Untuk layanan pengingat jadwal kontrol, matriks yang sama memberi *serverless* nilai 4,45, jauh di atas VM dengan 3,00. Pola bebannya yang berkala dan singkat mendapat skor 5, dan biayanya juga mendapat skor 5 karena layanan ini hanya bekerja sekitar 11 jam per bulan.

Satu organisasi karena itu wajar memakai beberapa *runtime* sekaligus. Yang perlu dijaga adalah jumlahnya: setiap *runtime* tambahan berarti keterampilan, pemantauan, dan kontrak tambahan.

8.6.3 Vendor Lock-in, Portabilitas, dan Biaya Keluar

Vendor lock-in adalah keadaan ketika pindah dari satu penyedia menjadi mahal atau sulit karena organisasi bergantung pada teknologi, format data, atau kontrak penyedia itu [110]. Portabilitas adalah kebalikannya: kemampuan memindahkan aplikasi dan data ke penyedia lain dengan perubahan kecil. Biaya keluar mencakup biaya transfer data keluar yang dibahas di Bab 3, penulisan ulang kode, pelatihan ulang, dan masa operasi paralel selama perpindahan.

Analoginya seperti membuka toko di pusat perbelanjaan yang menyediakan rak, mesin kasir, dan sistem pembayaran milik pengelola. Toko cepat buka karena semuanya sudah tersedia. Ketika pindah, pemilik toko harus membeli rak dan mesin kasir sendiri dan membiasakan pelanggan dengan cara bayar yang baru. Toko yang sejak awal memakai rak sendiri lebih lama siap, tetapi raknya dapat diangkut.

Tabel 8.9 membandingkan sumber ketergantungan pada *serverless* dan *managed* Kubernetes beserta cara meredamnya.

Dua contoh berangka:

1. Misalkan dalam tiga tahun Medikara Health memiliki 40 fungsi *serverless*, dan 25 di antaranya memakai pemicu khas penyedia. Menulis ulang satu pemicu beserta pengujiannya butuh sekitar tiga hari kerja, sehingga perpindahan memerlukan 75 hari kerja atau sekitar 3,5 bulan kerja satu orang.
2. Sembilan layanan di *managed* Kubernetes memakai 60 berkas manifest. Dari

Tabel 8.9: Sumber *vendor lock-in* pada *serverless* dan *managed* Kubernetes

Sumber	*Serverless*	*Managed* Kubernetes	Cara meredam
Kemasan kode	Paket fungsi mengikuti aturan penyedia	*Container image* berformat standar	Pisahkan logika bisnis dari kode penghubung ke platform
Pemicu dan format *event*	Khas penyedia	Diterima oleh kode sendiri	Buat lapisan penerjemah *event* yang tipis
Izin akses	Memakai sistem izin penyedia	Sebagian memakai sistem izin penyedia	Dokumentasikan dan otomatiskan pemetaan izin
Layanan pendukung	Antrean, basis data, dan penyimpanan terkelola milik penyedia	Penyeimbang beban dan kelas penyimpanan khas penyedia	Pilih layanan berprotokol standar
Data	Format penyimpanan penyedia	Format penyimpanan penyedia	Simpan data klinis dalam format terbuka seperti HL7 FHIR

jumlah itu, 12 berkas atau 20 persen memuat pengaturan khas penyedia, misalnya penyeimbang beban dan kelas penyimpanan. Menyesuaikan satu berkas butuh sekitar setengah hari kerja, sehingga totalnya 6 hari kerja. Ditambah pemindahan *registry* dan pemetaan izin sekitar 10 hari kerja, perpindahan memerlukan sekitar 16 hari kerja.

Biaya transfer data keluar sama besarnya pada kedua contoh, karena besarnya ditentukan jumlah data, bukan *runtime*. *Lock-in* juga tidak selalu buruk. *Serverless* menukar portabilitas dengan kecepatan membangun dan beban operasi yang kecil, dan pertukaran itu sering masuk akal. Yang berbahaya adalah *lock-in* yang tidak disadari. Karena itu setiap keputusan *runtime* memuat rencana keluar: apa yang dipindahkan, berapa perkiraan upayanya, dan data apa yang disimpan dalam format terbuka.

Matriks lokasi data Bab 2 berkaitan langsung dengan risiko ini. Bila data rekam medis wajib berada di Indonesia, pilihan penyedia menyempit menjadi penyedia yang punya *region* di Indonesia dan menyediakan layanan yang dibutuhkan di *region* itu. Makin sedikit penyedia yang memenuhi syarat, makin lemah posisi tawar organisasi, dan makin penting rencana keluar ditulis sejak awal.

8.7 Aktivitas: Matriks Keputusan Runtime

Tujuan

Tim menetapkan *runtime* untuk setiap kandidat layanan dengan kriteria yang dituliskan lebih dahulu. Setiap pilihan disertai alasan yang dapat dipertanggungja-

wabkan di depan manajemen, termasuk opsi yang ditolak dan rencana keluarnya.

Pembagian Peran

- Anggota berlatar teknis menjelaskan konsekuensi operasional setiap opsi: siapa yang memperbarui versi, siapa yang dipanggil ketika layanan mati, dan keterampilan apa yang dibutuhkan.
- Anggota berlatar non-teknis menilai biaya, ketersediaan tenaga kerja di pasar, dan risiko ketergantungan pada penyedia, termasuk isi kontrak dan biaya keluar.
- Seluruh anggota menyepakati bobot kriteria sebelum penilaian dimulai, lalu memeriksa bahwa tidak ada opsi yang lolos padahal melanggar kriteria gugur.

Langkah Kerja

1. Ambil kandidat layanan dari Bab 6 dan Bab 7. Untuk setiap layanan, tulis pola bebannya, kebutuhan latensinya, dan jenis datanya.
2. Tetapkan kriteria dan bobotnya di lembar `Kriteria`. Kriteria pada Tabel 8.7 boleh diubah, asalkan perubahannya disepakati sebelum penilaian dan alasannya ditulis.
3. Nilai opsi SaaS, PaaS, *managed* Kubernetes, *serverless*, dan VM untuk setiap layanan di lembar `Isian`. Terapkan kriteria gugur lokasi data lebih dahulu, memakai matriks kendala penempatan dari Bab 2.
4. Pilih *runtime* untuk setiap layanan, lalu catat alasannya dalam Architecture Decision Record (ADR) singkat di lembar `ADR singkat`: konteks, opsi, keputusan, konsekuensi, rencana keluar, dan pemicu tinjau ulang.

Luaran dan Kaitan dengan Capstone

Luarannya adalah matriks keputusan *runtime* per layanan beserta satu ADR singkat untuk setiap layanan. Matriks ini menjadi bagian tonggak C3 Kondisi Target. Pilihan *runtime* akan dipakai lagi pada Bab 9 ketika pola komunikasi antarlayanan dirancang, pada Bab 11 ketika target keandalan ditetapkan, dan pada Bab 12 ketika strategi pemulihan dipilih.

Kriteria Penilaian

- Kriteria dan bobot ditulis sebelum penilaian, dan perubahannya beralasan (25 persen).

- Kriteria gugur lokasi data diterapkan dengan benar dan merujuk matriks Bab 2 (20 persen).
- Skor setiap opsi disertai alasan yang memakai data, bukan kesan (30 persen).
- ADR memuat opsi yang ditolak, konsekuensi, rencana keluar, dan pemicu tinjau ulang (25 persen).

Berkas Pendukung

- `sources/chapter08/matriks-keputusan-runtime.xlsx`: lembar petunjuk, kriteria, contoh layanan janji temu dan pengingat, lembar isian, templat ADR singkat, dan kalkulator biaya *serverless* untuk Latihan 1.
- `sources/chapter08/contoh-container/`: Dockerfile, aplikasi contoh, dan manifest Kubernetes untuk pendalaman teknis.

Pendalaman Teknis (opsional): Menguji Asumsi Matriks

Anggota teknis menjalankan layanan jadwal dokter di kind (kotak pendalaman Seksi 8.3), mematikan satu *pod*, lalu mencatat waktu sampai penggantinya siap. Catatan itu menjadi bukti bagi skor latensi dan keterampilan.

8.8 Ringkasan

Bab ini berangkat dari dua masalah Medikara Health: aplikasi yang berjalan di satu rumah sakit tetapi gagal di rumah sakit lain, dan server yang menyala sepanjang hari untuk pekerjaan beberapa menit. Keduanya adalah pertanyaan tentang *runtime*. VM memisahkan beban dengan sistem operasi masing-masing. *Container* mengemas aplikasi bersama pustakanya dan berbagi satu *kernel*, sehingga lingkungan yang diuji sama dengan lingkungan produksi. *Image* disimpan di *registry*, yang karena itu menjadi aset kritis, dan *container* tidak boleh diperlakukan sebagai VM kecil karena batas isolasinya lebih tipis.

Ketika jumlah *container* mencapai puluhan atau ratusan, penempatan, pemulihan, penambahan kapasitas, dan pembaruannya perlu diorkestrasi. Kubernetes bekerja dengan *desired state*: tim menulis keadaan yang diinginkan dalam manifest, lalu kontroler terus membandingkan dan membetulkan keadaan aktual. *Pod*, *deployment*, dan *service* adalah bahan dasarnya, sedangkan *self-healing* dan *autoscaling* adalah buah dari rekonsiliasi. Survei CNCF 2025 menunjukkan Kubernetes sudah menjadi platform bersama, termasuk untuk *workload* AI, tetapi angka itu perlu dibaca bersama penyebutnya.

Kubernetes memindahkan pekerjaan operasi, bukan menghapusnya. *Managed*

Kubernetes mengambil alih *control plane*, tetapi pembaruan versi, keamanan, dan manifest tetap tanggung jawab pelanggan. PaaS menyembunyikan orkestrasi dengan imbalan kendali yang lebih kecil. Pada kasus Medikara Health, tenaga kerja platform menelan hampir separuh biaya *managed* Kubernetes, jauh di atas tarif layanannya, sehingga PaaS dipilih untuk layanan baru pertama.

Serverless dan FaaS menjalankan kode hanya ketika ada pemicu, menyusut sampai nol, dan menagih per eksekusi. Model ini cocok untuk beban yang jarang, berkala, atau dipicu kedatangan data, seperti pengingat jadwal kontrol dan hasil laboratorium. Keterbatasannya adalah *cold start*, batas waktu eksekusi, dan ketergantungan pada format serta layanan penyedia. Kasus pengingat juga memperlihatkan bahwa biaya komputasi kadang kecil dibandingkan biaya lain di sekitar layanan itu.

Memilih *runtime* karena itu bukan soal teknologi mana yang paling modern. Kriteria gugur lokasi data dan kriteria berbobot ditulis lebih dahulu, setiap opsi dinilai dengan data, dan *vendor lock-in* dihitung, bukan ditakuti atau diabaikan. Pesan utama bab ini: *runtime* yang tepat adalah yang sanggup dijalankan oleh tim yang ada, dengan biaya total yang tercatat, di lokasi yang diizinkan, dan dengan rencana keluar yang tertulis.

8.9 Latihan

Tiga latihan berikut menguji kemampuan menjelaskan, menilai, dan membandingkan pilihan *runtime* dengan data. Hasil setiap mahasiswa boleh berbeda; yang dinilai adalah penalaran dan perbandingannya.

Latihan 1: Serverless untuk Pengingat Jadwal Kontrol

Latihan ini membuktikan bahwa alasan memilih *serverless* untuk layanan pengingat dapat dijelaskan kepada direktur keuangan dengan angka, termasuk bagian yang tidak menguntungkan.

1. Buka lembar `Biaya Serverless` di `sources/chapter08/matriks-keputusan-runtime.xlsx`.
2. Isi jumlah jadwal kontrol per bulan, jumlah pengingat per jadwal, durasi eksekusi, dan memori fungsi. Pakai angka Medikara Health atau angka organisasi pilihan tim.
3. Periksa tarif terbaru di halaman harga penyedia yang dipilih, lalu isi tarif dan kurs.
4. Bandingkan opsi VM yang menyala terus dengan opsi *serverless*, termasuk biaya pesan, seperti contoh pada Tabel 8.10.

5. Tulis penjelasan satu halaman untuk direktur keuangan berdasarkan Tabel 8.11.

Tabel 8.10: Latihan 1, contoh perbandingan biaya bulanan layanan pengingat Medikara Health

Komponen	VM yang menyala terus	*Serverless*
Jam berjalan per bulan	720	Sekitar 11,1
Biaya komputasi per bulan	Rp650.000	Rp3.014 tanpa kuota gratis; Rp0 di dalam kuota
Pemeliharaan server	*Patch* sistem operasi setiap bulan	Tidak ada
Biaya pesan WhatsApp (80.000 pesan)	Rp28.000.000	Rp28.000.000
Total per bulan	Rp28.650.000	Rp28.003.014
Bila pengingat naik tiga kali lipat	VM perlu diperbesar secara manual	Menyesuaikan sendiri; komputasi sekitar Rp9.000 tanpa kuota gratis

Tabel 8.11: Latihan 1, lembar isian perbandingan biaya layanan pengingat

Komponen	VM yang menyala terus	*Serverless*
Jam berjalan per bulan	…	…
Biaya komputasi per bulan	…	…
Pemeliharaan server	…	…
Biaya pesan	…	…
Total per bulan	…	…
Bila pengingat naik tiga kali lipat	…	…

Pertanyaan analisis:

1. Komponen mana yang paling menentukan total biaya, dan apa artinya bagi argumen yang disampaikan kepada direktur keuangan?
2. Pada jumlah eksekusi per bulan berapa biaya *serverless* tanpa kuota gratis menyamai biaya VM? Tunjukkan perhitungannya.
3. Risiko apa yang tidak tampak di tabel, dan bagaimana risiko itu disampaikan secara jujur?

Latihan 2: Ketika Kubernetes Menjadi Beban

Latihan ini membuktikan bahwa Kubernetes bisa menjadi beban bagi organisasi, dan bahwa kondisinya dapat dikenali dari indikator yang terukur, bukan dari kesan.

1. Baca kembali Tabel 8.5 dan contoh tentang irama pembaruan versi di Seksi 8.4.

2. Rumuskan dua kondisi yang membuat Kubernetes menjadi beban, masing-masing dengan indikator yang dapat diukur.
3. Isi data untuk setiap indikator dari Medikara Health atau organisasi pilihan tim, seperti contoh pada Tabel 8.12.
4. Jelaskan mengapa setiap kondisi menjadi beban, lalu tulis hasilnya di Tabel 8.13.

Tabel 8.12: Latihan 2, contoh dua kondisi Kubernetes menjadi beban di Medikara Health

Kondisi	Indikator	Data Medikara Health	Mengapa menjadi beban
Layanan sedikit, tim platform belum ada	Jumlah layanan; jumlah staf yang pernah mengoperasikan Kubernetes di produksi	Sembilan layanan; 0 dari 52 staf TI	Tenaga kerja platform Rp780 juta per tahun atau 46,9 persen biaya; operasi bergantung pada satu atau dua orang
Tidak sanggup mengikuti irama pembaruan versi	Rilis minor per tahun; masa dukungan; jumlah klaster	Sekitar tiga rilis per tahun; dukungan sekitar satu tahun; tiga klaster	Tarif *control plane* naik dari sekitar Rp3,6 juta menjadi Rp21,7 juta per bulan bila terlambat; celah keamanan tidak lagi ditambal

Tabel 8.13: Latihan 2, lembar isian kondisi Kubernetes menjadi beban

Kondisi	Indikator	Data organisasi	Mengapa menjadi beban
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Pada jumlah layanan dan jumlah *platform engineer* berapa kondisi pertama tidak lagi berlaku? Gunakan angka pada Tabel 8.5 sebagai titik awal.
2. Apakah PaaS bebas dari kedua kondisi itu? Jelaskan dengan membaca Gambar 8.4.

Latihan 3: Vendor Lock-in pada Serverless dan Managed Kubernetes

Latihan ini membuktikan bahwa risiko *vendor lock-in* dapat dibandingkan secara terukur, yaitu dari sumber ketergantungan dan perkiraan upaya pindah penyedia.

1. Daftar layanan khas penyedia yang dipakai atau direncanakan untuk satu layanan, sekali dengan *serverless* dan sekali dengan *managed* Kubernetes.

2. Kelompokkan ketergantungan itu memakai sumber pada Tabel 8.9.
3. Perkirakan upaya pindah penyedia dalam hari kerja untuk setiap opsi, seperti contoh pada Tabel 8.14.
4. Tulis perbandingan dan rencana keluar untuk opsi yang dipilih di Tabel 8.15.

Tabel 8.14: Latihan 3, contoh perbandingan *vendor lock-in* di Medikara Health

Aspek	***Serverless***	***Managed* Kubernetes**
Ukuran yang dipindahkan	40 fungsi	Sembilan layanan, 60 berkas manifest
Bagian khas penyedia	25 fungsi memakai pemicu khas penyedia	12 berkas memuat pengaturan khas penyedia
Upaya per bagian	Sekitar tiga hari kerja per fungsi	Sekitar setengah hari kerja per berkas
Pekerjaan tambahan	Pemetaan izin akses ke penyedia baru	Pemindahan *registry* dan pemetaan izin, sekitar 10 hari kerja
Perkiraan upaya pindah	75 hari kerja	16 hari kerja
Biaya transfer data keluar	Sama, ditentukan jumlah data	Sama, ditentukan jumlah data

Tabel 8.15: Latihan 3, lembar isian perbandingan *vendor lock-in*

Aspek	***Serverless***	***Managed* Kubernetes**
Ukuran yang dipindahkan	…	…
Bagian khas penyedia	…	…
Upaya per bagian	…	…
Pekerjaan tambahan	…	…
Perkiraan upaya pindah	…	…
Biaya transfer data keluar	…	…

Pertanyaan analisis:

1. Mengapa upaya pindah *serverless* jauh lebih besar walaupun jumlah datanya sama?
2. Apakah selisih upaya itu cukup untuk menolak *serverless*? Kaitkan jawaban dengan biaya operasi dan kecepatan membangun.
3. Bagaimana matriks lokasi data Bab 2 mempersempit pilihan penyedia dan mengubah besarnya risiko *lock-in* di organisasi pilihan tim?

Bab 9

Komunikasi Antarlayanan dan Event-Driven Architecture

Tujuan Pembelajaran

- Membedakan komunikasi sinkron dan asinkron beserta dampaknya terhadap ketersediaan layanan.
- Memodelkan proses bisnis sebagai rangkaian *event*.
- Membandingkan koordinasi berbasis *choreography* dan *orchestration*.

9.1 Pendahuluan

Pada satu Selasa pagi di bulan Maret 2026, layanan pengiriman pesan WhatsApp milik vendor notifikasi Medikara+ terganggu selama tiga jam. Gangguan itu seharusnya hanya menunda pengingat jadwal. Kenyataannya, pasien yang menekan tombol "Buat Janji" di Medikara+ melihat pesan gagal. Pada jam sibuk pukul 07.00 sampai 10.00, Medikara+ biasanya menerima sekitar 175 janji temu per jam, sehingga sekitar 520 janji temu gagal dibuat pagi itu. Pusat panggilan kewalahan, dan sebagian pasien memilih berobat ke rumah sakit lain.

Penyebabnya bukan kerusakan di Medikara+ sendiri. Medikara+ memanggil empat sistem secara berurutan dan menunggu jawaban masing-masing, termasuk layanan notifikasi. Bila satu sistem diam, seluruh alur ikut berhenti. Pola yang sama berulang pada integrasi *point-to-point* di Medikara Health: Sistem Informasi Manajemen Rumah Sakit (SIMRS), Enterprise Resource Planning (ERP) keuangan, jembatan ke BPJS Kesehatan, dan jembatan ke SATUSEHAT saling memanggil secara langsung, sehingga gangguan di satu titik merambat ke banyak sistem.

Bab ini membahas cara layanan berkomunikasi agar gangguan tidak merambat: kapan sebuah layanan perlu menunggu jawaban, kapan cukup mengirim kabar, dan bagaimana proses yang melibatkan banyak layanan dikoordinasikan.

Broker menyalurkan fakta bisnis kepada konsumen yang berlangganan. Penyampaian ulang memerlukan identitas event dan pengolahan yang idempoten. Gambar 9.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 9.1: Penyampaian event dan penanganan duplikasi

Rujukan utama bab ini adalah buku *Building Microservices* karya Newman [106], *Microservices Patterns* karya Richardson [125], katalog pola integrasi berbasis pesan dari Hohpe dan Woolf [73], serta tulisan Fowler tentang arti *event-driven* [62]. Dua spesifikasi terbuka dipakai untuk menuliskan *event* secara seragam, yaitu CloudEvents [34] dan AsyncAPI [25]. Hasil bab ini adalah peta interaksi layanan dan katalog *event* untuk tonggak capstone C3.

9.2 Komunikasi Sinkron dan Asinkron

Komunikasi sinkron terjadi ketika layanan pemanggil mengirim permintaan, lalu berhenti menunggu jawaban sebelum melanjutkan pekerjaannya. Pola ini disebut *request/response* dan umumnya berjalan di atas HyperText Transfer Protocol (HTTP), misalnya lewat Application Programming Interface (API). Syaratnya, pemanggil dan yang dipanggil harus hidup pada saat yang sama. Komunikasi asinkron terjadi ketika pengirim menitipkan pesan kepada perantara, lalu langsung melanjutkan pekerjaannya tanpa menunggu. Penerima memproses pesan itu ketika siap. Pengirim dan penerima tidak harus hidup bersamaan, karena perantara menyimpan pesan sampai diambil [106].

Analoginya seperti menelepon dan meninggalkan memo. Staf administrasi yang menelepon gudang untuk menanyakan stok harus menunggu di gagang telepon. Bila gudang tidak mengangkat, pekerjaannya berhenti. Staf yang meninggalkan memo di kotak masuk gudang bisa langsung kembali bekerja, dan gudang membaca memo itu setelah rapatnya selesai. Namun memo tidak cocok untuk pertanyaan yang jawabannya dibutuhkan detik itu juga, misalnya ketika pelanggan sedang menunggu di depan meja.

Tabel 9.1 merangkum perbedaan keduanya dari sisi yang penting bagi pemilik layanan.

Tabel 9.1: Perbandingan komunikasi sinkron dan asinkron

Aspek	**Sinkron (*request/response*)**	**Asinkron (lewat pesan)**
Cara kerja	Pemanggil menunggu jawaban	Pengirim menitipkan pesan lalu melanjutkan pekerjaan
Syarat ketersediaan	Semua pihak dalam rantai harus hidup bersamaan	Cukup pengirim dan perantara yang hidup; penerima boleh menyusul
Waktu tunggu pengguna	Jumlah waktu semua panggilan dalam rantai	Hanya waktu menitipkan pesan
Jawaban	Langsung tersedia, termasuk penolakan	Tidak langsung; hasil dikabarkan kemudian
Wujud kegagalan	Pengguna melihat pesan gagal	Pesan menumpuk dan hasil terlambat
Cocok untuk	Pertanyaan yang jawabannya dibutuhkan saat itu juga	Pekerjaan lanjutan yang boleh terlambat beberapa detik atau menit

Pertanyaan kemudian muncul: seberapa besar pengaruh rantai panggilan sinkron terhadap ketersediaan? Ketika layanan A memanggil B, lalu B memanggil C, alur itu baru berhasil bila ketiganya hidup. Ketersediaan gabungannya dihitung sebagai hasil kali ketersediaan setiap mata rantai, dengan anggapan gangguan masing-masing tidak saling terkait. Waktu tunggunya pun dijumlahkan. Rantai yang panjang selalu lebih rapuh dan lebih lambat daripada mata rantai terlemahnya.

Keputusan memakai rantai sinkron punya dua konsekuensi. Pertama, ketersediaan alur turun setiap kali satu sistem ditambahkan ke rantai, walaupun sistem baru itu cukup andal. Empat sistem dengan ketersediaan 99,5 persen menghasilkan alur dengan ketersediaan sekitar 98 persen, atau sekitar 14 jam gagal per bulan. Kedua, pemilik alur ikut menanggung gangguan sistem yang tidak dimilikinya. Tim Medikara+ tidak dapat memperbaiki layanan notifikasi vendor, tetapi keluhan pasien tetap jatuh ke Medikara+.

Dua contoh dari Medikara Health:

1. Pengecekan slot jadwal dokter harus sinkron. Pasien sedang menatap layar

dan perlu tahu saat itu juga apakah slot Senin pukul 09.30 di poliklinik penyakit dalam masih kosong. Jawaban "nanti dikabari" membuat dua pasien bisa memilih slot yang sama.

2. Pengingat jadwal kontrol sehari sebelum kunjungan cukup asinkron. Sekitar 1.400 pengingat terkirim setiap hari kerja, dan pengingat yang terlambat sepuluh menit tidak merugikan siapa pun. Pasien tidak perlu menunggu pengingat terkirim untuk menyelesaikan janji temunya.

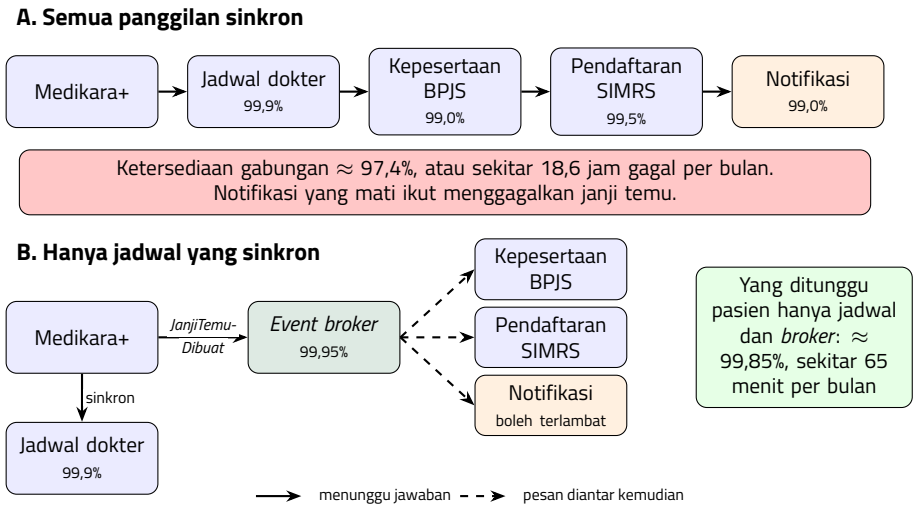
Kasus Medikara Health: Alur Janji Temu yang Memanggil Empat Sistem

Saat ini Medikara+ memproses satu janji temu dengan memanggil empat sistem secara berurutan, seperti diperlihatkan bagian atas Gambar 9.2. Pertama, modul jadwal dokter di SIMRS mengunci slot. Kedua, jembatan BPJS Kesehatan memeriksa kepesertaan pasien. Ketiga, modul pendaftaran SIMRS membuat data kunjungan. Keempat, layanan notifikasi vendor mengirim konfirmasi lewat WhatsApp. Ketersediaan bulanan keempatnya menurut catatan tim operasi adalah 99,9; 99,0; 99,5; dan 99,0 persen.

Hasil kalinya sekitar 97,4 persen. Dalam sebulan berisi 43.200 menit, alur janji temu gagal sekitar 1.114 menit atau 18,6 jam. Waktu tunggunya juga dijumlahkan: 150 milidetik untuk jadwal, 1.200 milidetik untuk kepesertaan, 400 milidetik untuk pendaftaran, dan 650 milidetik untuk notifikasi, sehingga pasien menunggu sekitar 2,4 detik setiap kali menekan tombol.

Misalkan alur itu dirancang ulang seperti bagian bawah gambar. Medikara+ hanya menunggu jadwal dokter, lalu menitipkan *event* `JanjiTemuDibuat` ke *event broker* terkelola dengan ketersediaan 99,95 persen. Kepesertaan, pendaftaran, dan notifikasi bereaksi terhadap *event* itu pada waktunya masing-masing. Ketersediaan yang dirasakan pasien naik menjadi sekitar 99,85 persen atau sekitar 65 menit gagal per bulan, dan waktu tunggunya turun menjadi sekitar 170 milidetik.

Rancangan ulang ini punya harga yang harus diputuskan bagian pelayanan, bukan hanya tim TI. Pasien BPJS Kesehatan kini mendapat janji temu sebelum kepesertaannya terbukti aktif. Bila pemeriksaan kepesertaan gagal, janji temu berubah status menjadi "perlu konfirmasi penjamin" dan pasien diberi tahu lewat notifikasi. Keputusan ini sah secara teknis, tetapi hanya pantas dijalankan bila bagian pelayanan setuju dan alur penanganan status itu sudah disiapkan.



Gambar 9.2: Alur janji temu Medikara+: rantai empat panggilan sinkron (A) dibandingkan dengan rancangan yang hanya menunggu jadwal dokter dan menitipkan sisanya sebagai *event* (B)

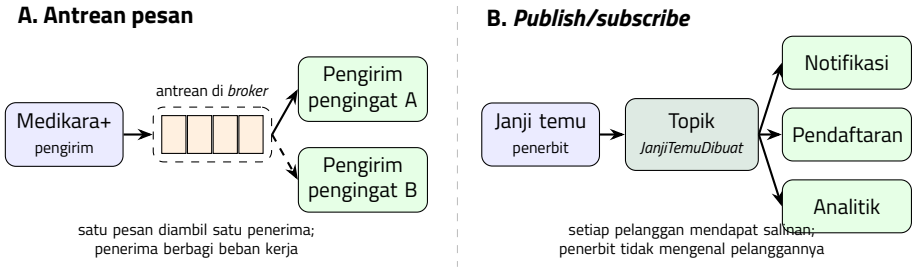
9.3 Antrean Pesan, Publish/Subscribe, dan Event Broker

Komunikasi asinkron membutuhkan perantara. Ada dua bentuk dasar perantara. Antrean pesan (*point-to-point*) menampung pesan secara berurutan, dan setiap pesan diambil oleh tepat satu penerima. Bila ada beberapa penerima yang membaca antrean yang sama, mereka berbagi beban: penerima yang sedang luang mengambil pesan berikutnya. *Publish/subscribe* bekerja dengan topik. Penerbit mengirim pesan ke satu topik, lalu setiap pelanggan topik itu menerima salinannya sendiri. Penerbit tidak perlu tahu siapa saja pelanggannya. Hohpe dan Woolf menamai keduanya *point-to-point channel* dan *publish-subscribe channel* [73]. Perangkat lunak yang menyediakan antrean, topik, atau keduanya disebut *event broker* atau *message broker*. Tugasnya menerima pesan, menyimpannya dengan aman, dan mengantarkannya kepada penerima yang berhak.

Analoginya seperti dua cara kantor melayani orang. Antrean pesan mirip nomor antrean di kantor bank: satu nomor dipanggil oleh satu teller, dan teller yang sedang luang memanggil nomor berikutnya. *Publish/subscribe* mirip papan pengumuman RT: pengurus menempel satu pengumuman, lalu setiap warga yang berkepentingan membacanya. Pengurus RT tidak perlu mendatangi rumah warga satu per satu, dan warga baru cukup ikut membaca papan yang sama. *Event broker* adalah kantor pos yang mengelola kedua cara itu sekaligus.

Gambar 9.3 memperlihatkan kedua bentuk itu berdampingan.

Dua contoh dari Medikara Health:



Gambar 9.3: Antrean pesan membagi pekerjaan di antara penerima (A), sedangkan *publish/subscribe* memberi salinan kepada setiap pelanggan (B)

1. Antrean pesan untuk pengiriman pengingat. Sekitar 1.400 pengingat per hari kerja masuk ke satu antrean dan dikerjakan oleh dua pengirim. Menjelang kampanye vaksinasi, jumlahnya naik menjadi sekitar 4.000 per hari. Tim cukup menambah pengirim ketiga dan keempat tanpa mengubah Medikara+ sama sekali.
2. *Publish/subscribe* untuk *event* janji temu. Notifikasi, pendaftaran SIMRS, dan analitik berlangganan topik `JanjiTemuDibuat`. Selama ini setiap integrasi *point-to-point* baru memakan sekitar enam minggu kerja vendor. Dengan topik bersama, tim analitik yang membutuhkan data janji temu cukup mendaftar sebagai pelanggan, sementara Medikara+ tidak diubah.

Pertanyaan berikutnya: produk apa yang dipakai sebagai *broker*? Pada tingkat konsep, pilihan umumnya terbagi seperti Tabel 9.2. Pilihan produk mengikuti keputusan *runtime* pada Bab 8. *Broker* terkelola mengurangi beban operasi, sedangkan *broker* yang dijalankan sendiri memberi kendali lebih besar dengan biaya tenaga kerja yang lebih tinggi. Satu hal tidak boleh terlupa: *broker* ikut menyimpan isi pesan, sehingga aturan lokasi data dari Bab 2 berlaku juga untuknya.

Pendalaman Teknis (opsional): Menjalankan Broker Lokal

Folder `sources/chapter09/contoh-broker-lokal` berisi `docker-compose.yml` untuk menjalankan RabbitMQ di komputer sendiri dan skrip `kirim_terima_event.py` untuk mengirim serta menerima *event* `JanjiTemuDibuat`. RabbitMQ memakai Advanced Message Queuing Protocol (AMQP) pada port 5672. Langkahnya:

1. Pasang Docker dan Python 3, lalu masuk ke folder `sources/chapter09/contoh-broker-lokal`.
2. Jalankan `docker compose up -d`, kemudian `pip install pika`. Tunggu sekitar 10 detik sampai *broker* siap. Galat `StreamLostError` pada langkah berikutnya berarti *broker* belum siap, sehingga perin-

Tabel 9.2: Gambaran konsep beberapa produk *broker*

Produk	Sifat utama	Cocok untuk
RabbitMQ	*Broker* antrean dan topik; pesan dihapus setelah penerima memberi pengakuan	Pembagian kerja dan pengantaran pesan dengan aturan rute yang luwes
Apache Kafka	Log *event* yang disimpan sesuai masa retensi; pelanggan dapat membaca ulang dari titik mana pun	Aliran *event* bervolume besar, analitik, dan pelanggan baru yang perlu membaca riwayat
Antrean terkelola (Amazon SQS, Azure Service Bus)	Penyedia *cloud* mengurus server, kapasitas, dan pembaruan	Tim kecil yang ingin antrean andal tanpa mengoperasikan *broker*
Publish/subscribe terkelola (Google Cloud Pub/Sub, Amazon SNS, Azure Event Grid)	Topik terkelola dengan banyak pelanggan	Penyebaran *event* antarlayanan dalam satu penyedia *cloud*

tahnya cukup diulang.

3. Jalankan `python kirim_terima_event.py kirim`. Skrip menerbitkan *event* yang sama dua kali ke *exchange* `medikara.janji-temu`.
4. Buka konsol pengelolaan di `http://localhost:15672` dengan pengguna dan sandi `guest`. Antrean `notifikasi.janji-temu-dibuat` berisi dua pesan.
5. Jalankan `python kirim_terima_event.py terima`. Pesan pertama diproses, pesan kedua dilewati. Alasannya dibahas di Seksi 9.6.

Broker ini hanya untuk komputer sendiri. Port 5672 dan 15672 tidak boleh dibuka ke jaringan publik, dan skrip tidak boleh diarahkan ke *broker* milik organisasi lain tanpa izin tertulis.

9.4 Pola Event: Notification, State Transfer, dan Event Sourcing

Event adalah catatan bahwa sesuatu telah terjadi dalam bisnis, misalnya janji temu dibuat atau hasil laboratorium selesai divalidasi. Karena sudah terjadi, *event* ditulis dalam bentuk lampau dan tidak dapat dibatalkan. Kejadian berikutnya hanya dapat mengoreksinya, misalnya `JanjiTemuDibatalkan`. Setiap *event* punya satu pemilik, yaitu layanan yang menerbitkannya.

Dua hal pada pesan sering tertukar: *event* dan perintah. Pembagiannya begini. *Event* seperti `JanjiTemuDibuat` menyatakan fakta, dan penerbitnya tidak peduli siapa yang bereaksi. Perintah seperti `KirimPengingat` adalah permintaan kepada

satu penerima tertentu, dan penerima itu boleh menolak. Fowler mengingatkan bahaya *event* yang menjadi perintah terselubung: penerbit diam-diam berharap penerima tertentu melakukan sesuatu, sehingga keterkaitan yang ingin dihindari muncul kembali [62].

Fowler membedakan tiga pola pemakaian *event* [62]. Pada *event notification*, *event* hanya membawa kabar singkat dan nomor rujukan. Penerima yang membutuhkan rincian memanggil balik penerbit. Pada *event-carried state transfer*, *event* membawa data yang berubah, sehingga penerima dapat menyimpan salinannya dan tidak perlu memanggil balik. Pada *event sourcing*, setiap perubahan status disimpan sebagai rangkaian *event*, dan status saat ini dihitung dengan memutar ulang seluruh rangkaian itu.

Analoginya seperti tiga cara bank memberi kabar. Pesan singkat "ada transaksi di rekening, silakan cek aplikasi" adalah *event notification*: nasabah harus membuka aplikasi untuk tahu rinciannya. Pesan yang langsung memuat nominal, tujuan, dan saldo akhir adalah *event-carried state transfer*: nasabah tidak perlu membuka aplikasi. Buku tabungan adalah *event sourcing*: saldo tidak ditulis sebagai satu angka, melainkan dihitung dari seluruh baris mutasi sejak rekening dibuka.

Tabel 9.3 merangkum kelebihan dan biaya setiap pola.

Tabel 9.3: Tiga pola pemakaian *event* menurut Fowler [62]

Pola	Isi *event*	Kelebihan	Biaya
Event notification	Kabar singkat dan nomor rujukan	Keterkaitan rendah; data sensitif tetap di pemiliknya	Alur logis tidak terlihat di satu tempat; panggilan balik menambah beban penerbit
Event-carried state transfer	Data yang berubah, cukup untuk dipakai penerima	Penerima tetap bekerja walaupun penerbit mati; waktu tunggu pendek	Data tersalin di banyak tempat; salinan bisa sementara tertinggal; data pribadi ikut menyebar
Event sourcing	Seluruh perubahan status sebagai rangkaian *event*	Jejak audit lengkap; kondisi masa lalu dapat direkonstruksi	Rumit; perubahan bentuk *event* dan pemutaran ulang yang melibatkan sistem luar sulit ditangani

Keputusan memilih pola punya dua konsekuensi. Pertama, *event-carried state transfer* membuat data tersebar, sehingga pertanyaan "siapa saja yang menyimpan data pasien ini" makin sulit dijawab. Pertanyaan itu wajib terjawab menurut Undang-Undang Pelindungan Data Pribadi (UU PDP) [120]. Kedua, *event sourcing* adalah keputusan penyimpanan data, bukan sekadar keputusan komunikasi. Pola ini layak dipertimbangkan bila jejak audit lengkap memang dibutuhkan, misalnya

pada riwayat perubahan rekam medis, dan tidak perlu dipakai di semua layanan.

Nama *event* sebaiknya memakai bahasa bisnis yang sama dengan *ubiquitous language* pada Bab 6 [47]. Nama yang baik ditulis dalam bentuk lampau dan dipahami bagian pelayanan, misalnya `HasilLabTersedia`. Nama teknis seperti `RecordUpdated` atau `StatusChanged` memaksa setiap pelanggan menebak artinya. Papan EventStorming yang disusun tim pada Bab 6 adalah sumber terbaik nama *event*, karena kejadian berwarna oranye di papan itu sudah ditulis dalam bahasa bisnis.

Dua contoh dari Medikara Health:

1. Laboratorium rujukan memvalidasi sekitar 2.600 hasil pemeriksaan per hari. Hasil laboratorium adalah data kesehatan yang sensitif, sehingga `HasilLabTersedia` cukup membawa nomor permintaan dan penanda kritis. Dokter yang berhak mengambil isinya lewat API yang memeriksa hak akses.
2. Penagihan menerbitkan sekitar 3.300 tagihan per hari. ERP keuangan membutuhkan nomor tagihan, total, dan penjamin, tetapi tidak membutuhkan data klinis. `TagihanTerbit` membawa data itu secara lengkap, sehingga ERP tidak perlu memanggil balik Penagihan.

Kasus Medikara Health: Tiga Event Pertama

Tim transformasi menetapkan tiga *event* pertama untuk alur janji temu sampai tagihan. Ketiganya diambil dari papan EventStorming Bab 6 dan dirangkum pada Tabel 9.4. Polanya sengaja berbeda-beda, karena kebutuhan pelanggannya juga berbeda.
`JanjiTemuDibuat` memakai *event notification* yang membawa sebagian data. Data itu cukup bagi notifikasi untuk menjadwalkan pengingat, tetapi identitas pasien hanya berupa nomor rujukan. `HasilLabTersedia` sengaja memakai *event notification* murni. Keputusan ini menjawab temuan audit internal tentang kontrol akses yang lemah: isi hasil laboratorium tidak ikut tersebar ke semua pelanggan. `TagihanTerbit` memakai *event-carried state transfer*, karena isinya data keuangan tanpa data klinis dan pelanggannya perlu tetap bekerja saat Penagihan sedang dipelihara.

Pendalaman Teknis (opsional): CloudEvents dan AsyncAPI

CloudEvents adalah spesifikasi netral vendor untuk format data *event*. Versi rilis terakhirnya 1.0.2, dan proyeknya berstatus *graduated* di Cloud Native Computing Foundation (CNCF) sejak Januari 2024 [34]. Spesifikasi ini mewajibkan empat atribut: `id`, `source`, `specversion`, dan `type`. Atribut `time`,

Tabel 9.4: Tiga *event* pertama Medikara Health (angka rekaan)

Event	Pemilik	Isi data	Pelanggan	Per hari
`JanjiTemu-Dibuat`	Janji temu (Medikara+)	Nomor janji temu, rujukan pasien, fasilitas, poliklinik, waktu	Notifikasi, pendaftaran, kepesertaan, analitik	1.400
`HasilLab-Tersedia`	Laboratorium	Nomor permintaan, rujukan pasien, penanda kritis	Pelayanan klinis, notifikasi, penagihan	2.600
`TagihanTerbit`	Penagihan	Nomor tagihan, nomor kunjungan, total, penjamin	Notifikasi, ERP keuangan, klaim	3.300

`subject`, `datacontenttype`, dan `dataschema` bersifat pilihan. Pasangan `source` dan `id` yang sama menandai *event* yang sama, sehingga penerima dapat mengenali kiriman ganda.

AsyncAPI 3.0.0 menggambarkan API berbasis pesan dalam format yang dapat dibaca mesin dan tidak terikat pada protokol tertentu [25]. Dokumennya memuat *channel* sebagai alamat pesan, *operation* dengan aksi `send` atau `receive`, dan *message* beserta isinya. Listing 9.1 memperlihatkan definisi pesan `JanjiTemuDibuat` yang memakai amplop CloudEvents. Berkas lengkapnya ditulis dalam YAML Ain't Markup Language (YAML) dan dapat dijadikan pola katalog *event* dalam format mesin.

```yaml
components:
  messages:
    JanjiTemuDibuat:
      name: JanjiTemuDibuat
      title: Janji temu dibuat
      contentType: application/cloudevents+json
      payload:
        type: object
        required: [specversion, id, source, type, time, data]
        properties:
          specversion:
            type: string
            const: '1.0'
          id:
            type: string
            description: Kunci idempotensi saat dikirim ulang.
          source:
            type: string
            const: /medikara-plus/janji-temu
          type:
            type: string
            const: example.medikara.janjitemu.dibuat.v1
```

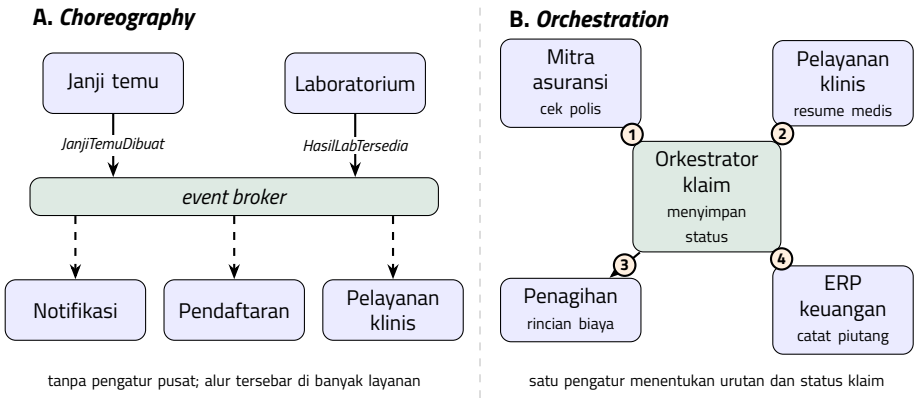
Listing 9.1: Definisi pesan `JanjiTemuDibuat` dalam AsyncAPI, tersedia di `sources/chapter09/asyncapi-janji-temu.yaml`

9.5 Choreography dan Orchestration

Proses bisnis yang melibatkan banyak layanan perlu dikoordinasikan. Ada dua cara. *Choreography* tidak memakai pengatur pusat. Setiap layanan mendengarkan *event* yang relevan, memutuskan sendiri reaksinya, lalu menerbitkan *event* baru bila perlu. Alur proses muncul dari gabungan reaksi itu. *Orchestration* memakai satu komponen pengatur yang disebut orkestrator. Orkestrator mengetahui urutan langkah, memanggil atau mengirim perintah ke setiap layanan, menyimpan status proses, dan menangani batas waktu serta kegagalan [106, 125].

Analoginya seperti kerja bakti dan renovasi rumah. Kerja bakti di lingkungan RT berjalan tanpa mandor: warga melihat selokan sudah dibersihkan, lalu mengangkut sampahnya, lalu warga lain mengecat pagar. Tidak ada yang memerintah, tetapi juga tidak ada yang tahu persis kapan semuanya selesai. Renovasi rumah memakai mandor yang memegang daftar pekerjaan, memanggil tukang listrik setelah tembok berdiri, dan menjadi satu-satunya orang yang bisa ditanya "sudah sampai mana".

Gambar 9.4 memperlihatkan kedua cara itu pada kasus Medikara Health.



Gambar 9.4: *Choreography* untuk reaksi atas janji temu dan hasil laboratorium (A), serta *orchestration* untuk proses klaim asuransi (B)

Tabel 9.5 membandingkan keduanya. Perbedaan terbesarnya ada pada *tradeoff* antara kebebasan setiap layanan dan kemudahan melacak alur.

Pertanyaan kemudian muncul: kapan memilih masing-masing? Tiga pertanyaan membantu. Pertama, apakah urutan langkah penting dan ada batas waktu yang harus dipantau? Bila ya, *orchestration* lebih aman. Kedua, apakah ada orang yang harus bisa menjawab "proses ini sudah sampai mana"? Bila ya, status proses perlu disimpan di satu tempat. Ketiga, apakah reaksi-reaksinya saling lepas dan sering bertambah? Bila ya, *choreography* lebih luwes. Dalam satu organisasi,

Tabel 9.5: Perbandingan *choreography* dan *orchestration*

Aspek	***Choreography***	***Orchestration***
Keterkaitan	Rendah; layanan hanya mengenal *event*	Orkestrator mengenal semua layanan yang dipanggilnya
Melacak alur	Sulit; alur tersebar di banyak layanan	Mudah; status setiap proses ada di orkestrator
Menambah reaksi baru	Mudah; cukup berlangganan *event*	Orkestrator harus diubah
Urutan ketat dan batas waktu	Sulit dijamin	Diatur di satu tempat
Risiko utama	Alur yang tidak dipahami siapa pun	Orkestrator menjadi pusat logika yang gemuk
Cocok untuk	Reaksi lepas antar-*bounded context*	Proses bisnis berurutan dengan status, batas waktu, dan langkah pembatalan

kedua cara biasanya dipakai bersama: *choreography* di antara *bounded context*, dan *orchestration* di dalam satu proses bisnis yang panjang [106].

Dua contoh dari Medikara Health:

1. Reaksi atas `JanjiTemuDibuat` memakai *choreography*. Notifikasi, pendaftaran, dan analitik bereaksi sendiri-sendiri. Ketika bagian pemasaran ingin mengirim survei kepuasan tiga hari setelah kunjungan, layanan survei cukup berlangganan *event* yang sudah ada.
2. Proses klaim asuransi memakai *orchestration*. Prosesnya berurutan, menunggu jawaban mitra sampai berhari-hari, dan petugas klaim harus bisa menjawab pertanyaan rumah sakit tentang posisi setiap klaim. Rinciannya ada pada kotak kasus berikut.

Kasus Medikara Health: Klaim dari Mitra Asuransi sebagai Orchestration

Medikara Health bekerja sama dengan 9 perusahaan asuransi mitra dan mengajukan sekitar 3.600 klaim rawat jalan per bulan. Setiap klaim melewati empat langkah, sesuai bagian kanan Gambar 9.4: memeriksa polis dan manfaat lewat API mitra asuransi, mengambil resume medis dan kode diagnosis dari Pelayanan klinis, mengambil rincian biaya dari Penagihan, lalu mencatat piutang di ERP keuangan setelah keputusan mitra keluar. Keputusan mitra rata-rata keluar dalam 11 hari kerja.
Misalkan proses ini disusun dengan *choreography*. Status satu klaim akan tersebar di lima layanan, dan petugas klaim harus menelusuri lima catatan untuk menjawab satu telepon. Klaim yang macet di mitra juga tidak akan terdeteksi, karena tidak ada komponen yang menunggu jawaban tersebut.

Karena itu tim memilih *orchestration*. Layanan Klaim Asuransi menjadi orkestrator di *bounded context* Klaim. Layanan ini menyimpan status setiap klaim, mengingatkan petugas bila mitra belum menjawab dalam 14 hari kerja, dan menjalankan langkah pembatalan bila salah satu langkah gagal. Rancangan langkah pembatalan itu dibahas pada Bab 10.

9.6 Pengiriman At-Least-Once dan Idempotensi

Setiap *broker* memberi jaminan pengiriman tertentu. Jaminan *at-most-once* (paling banyak sekali) mengirim pesan tanpa menunggu pengakuan dari penerima. Pesan tidak pernah ganda, tetapi bisa hilang. Jaminan *at-least-once* (paling sedikit sekali) mengirim ulang pesan sampai penerima memberi pengakuan (*acknowledgement*). Pesan tidak hilang, tetapi bisa diterima lebih dari sekali. Jaminan *exactly-once* (tepat sekali) menjanjikan pesan diproses tepat satu kali. Janji ini sulit dipenuhi secara menyeluruh di jaringan yang bisa putus kapan saja, sehingga produk yang menawarkannya biasanya membatasinya pada lingkup platform itu sendiri.

Pertanyaan kemudian muncul: mengapa pesan bisa terkirim dua kali? Ada dua penyebab yang umum. Pertama, penerima sudah selesai memproses pesan, tetapi koneksi putus sebelum pengakuannya sampai. *Broker* menganggap pesan belum diproses, lalu mengirimkannya lagi. Kedua, pengirim tidak menerima konfirmasi dari *broker*, lalu mengulang pengirimannya. Dokumentasi RabbitMQ menyatakan bahwa pengakuan menjamin pengiriman *at-least-once*, bahwa penerima harus siap menerima pesan yang pernah dilihatnya, dan bahwa penerima sebaiknya dirancang idempoten [122]. Tabel 9.6 merangkum ketiga jaminan itu.

Tabel 9.6: Tiga jaminan pengiriman pesan

Jaminan	Arti	Risiko	Contoh pemakaian
At-most-once	Dikirim sekali tanpa pengakuan	Pesan bisa hilang	Data telemetri yang segera tergantikan data berikutnya
At-least-once	Dikirim ulang sampai diakui	Pesan bisa ganda	Sebagian besar *event* bisnis, dipasangkan dengan penerima yang idempoten
Exactly-once	Diproses tepat sekali	Berlaku terbatas dalam satu platform; mahal	Pengolahan aliran data di dalam satu platform

Penerima yang idempoten menghasilkan keadaan akhir yang sama, baik pesan diproses sekali maupun berkali-kali. Ada tiga cara umum untuk mencapainya. Pertama, penerima mencatat `id` setiap *event* yang sudah diproses dan melewati `id`

yang sama. Kedua, operasi ditulis sebagai "tetapkan nilai" dan bukan "tambahkan nilai". Ketiga, basis data menolak data ganda lewat kunci unik, misalnya nomor tagihan. Hohpe dan Woolf menyebut penerima seperti ini *idempotent receiver* [73].

Analoginya seperti tombol pemanggil lift. Tombol ditekan sekali atau lima kali, lift tetap datang satu kali. Sebaliknya, kasir yang mencatat setiap struk yang diserahkan tanpa memeriksa nomornya akan mencatat pembayaran yang sama dua kali bila pelanggan menyerahkan fotokopi struknya.

Pertanyaan berikutnya: siapa yang bertanggung jawab atas idempotensi? Jawabannya penerima, karena *broker* tidak tahu arti bisnis sebuah pesan. Pengirim membantu dengan memberi `id` yang tetap sama pada setiap pengiriman ulang. Tanpa `id` yang tetap, penerima tidak bisa membedakan kiriman ulang dari kejadian baru.

Dua contoh dampak bisnis dari Medikara Health:

1. Misalkan layanan pengirim tagihan tidak idempoten, dan 0,1 persen dari sekitar 100.000 *event* `TagihanTerbit` per bulan diterima dua kali. Hasilnya sekitar 100 tagihan ganda per bulan. Dengan rata-rata tagihan Rp385.000, pasien dan perusahaan klien menerima tagihan berlebih sekitar Rp38,5 juta per bulan, belum termasuk waktu petugas untuk mengoreksi dan menjawab keluhan.
2. Pengingat jadwal yang terkirim dua kali hanya mengganggu pasien, dan biaya per pesannya kecil. Karena itu tim memprioritaskan idempotensi pada penerima yang menyentuh uang dan rekam medis, lalu menyusul pada penerima lain.

Pendalaman Teknis (opsional): Penerima yang Idempoten

Skrip pada Seksi 9.3 mengirim *event* yang sama dua kali. Listing 9.2 memperlihatkan cara penerimanya melewati kiriman kedua. `id` *event* dicatat ke tabel SQLite berkunci primer. Penyisipan kedua dengan `id` yang sama ditolak basis data, sehingga pemeriksaan dan pencatatan terjadi dalam satu langkah. Pesan duplikat tetap diakui agar *broker* tidak terus mengirimnya. Pada sistem nyata, pencatatan `id` dan efek bisnisnya disimpan dalam satu transaksi. Pola yang menjaga keduanya tetap sejalan dibahas pada Bab 10.

```
def tandai_terproses(catatan, id_event):
  """Mencatat id event; False berarti id itu sudah pernah dicatat.

  Pemeriksaan dan pencatatan terjadi dalam satu perintah INSERT,
  sehingga dua penerima yang berjalan bersamaan tidak sama-sama
  lolos.
  """
  try:
```

```
    with catatan:
      catatan.execute(
        "INSERT INTO event_terproses (id) VALUES (?)", (id_event,)
      )
    return True
  except sqlite3.IntegrityError:
    return False


def terima_event():
  """Mengambil semua pesan dan memproses setiap event sekali saja.

  Pesan duplikat tetap diakui (ack) agar broker tidak mengirimnya
  terus. Pada sistem nyata, pencatatan id dan efek bisnisnya
  disimpan dalam satu transaksi; contoh ini menyederhanakannya.
  """
  koneksi, kanal = buka_kanal()
  catatan = buka_catatan_terproses()
  while True:
    metode, _properti, isi = kanal.basic_get(queue=NAMA_ANTREAN)
    if metode is None:
      break
    event = json.loads(isi)
    if tandai_terproses(catatan, event["id"]):
      nomor = event["data"]["nomorJanjiTemu"]
      print(f"Proses: pengingat untuk {nomor} dikirim")
    else:
      print(f"Lewati: event {event['id']} sudah pernah diproses")
    kanal.basic_ack(delivery_tag=metode.delivery_tag)
  catatan.close()
  koneksi.close()
```

Listing 9.2: Penerima yang melewati *event* ganda, tersedia di `sources/chapter09/contoh-broker-lokal/kirim_terima_event.py`

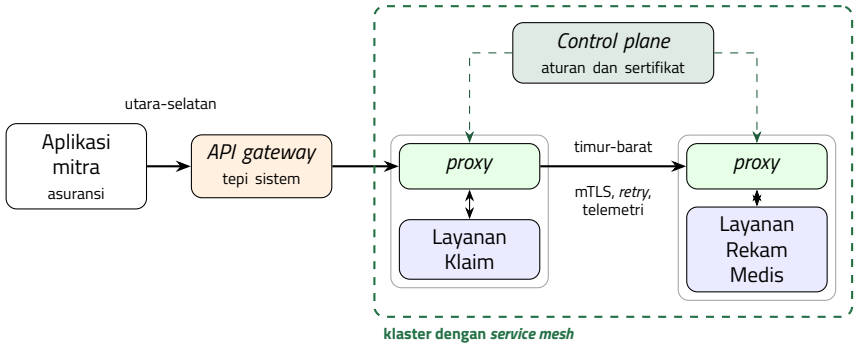
9.7 Service Mesh dan Batas dengan Desain API

Ketika jumlah layanan bertambah, setiap layanan perlu mengurus hal yang sama: mengenkripsi lalu lintas, mengulang panggilan yang gagal (*retry*), membatasi waktu tunggu, dan mencatat telemetri. *Service mesh* memindahkan urusan itu dari kode aplikasi ke lapisan infrastruktur. Dokumentasi Istio mendefinisikannya sebagai lapisan infrastruktur yang memberi aplikasi kemampuan keamanan *zero-trust*, *observability*, dan pengaturan lalu lintas tanpa perubahan kode [78]. Caranya, setiap layanan ditemani *proxy* yang mencegat semua lalu lintas masuk dan keluar. Pola ini dikenal sebagai *sidecar*. *Control plane* membagikan aturan dan sertifikat kepada semua *proxy*. Enkripsi dua arah antarlayanan yang dijalankan *proxy* disebut mutual Transport Layer Security (mTLS).

Analoginya seperti gedung perkantoran yang dikelola bersama. Setiap penyewa tidak perlu menyewa satpam, memasang kamera, dan mencetak kartu

akses sendiri. Pengelola gedung menyediakan satpam di setiap lantai, kartu akses, dan kamera yang terhubung ke satu ruang pantau. Satpam di lobi depan yang memeriksa tamu dari luar gedung adalah urusan lain, dan perannya mirip *API gateway*.

Gambar 9.5 memperlihatkan pembagian itu. *API gateway* mengurus lalu lintas dari luar ke dalam sistem, yang lazim disebut utara-selatan. *Service mesh* mengurus lalu lintas antarlayanan di dalam sistem, yang disebut timur-barat.



Gambar 9.5: *API gateway* di tepi sistem dan *service mesh* untuk lalu lintas antarlayanan, dengan *proxy* di samping setiap layanan

Dua hal pada *service mesh* dan *API gateway* sering tertukar. Pembagiannya begini. *API gateway* adalah pintu depan bagi klien luar: aplikasi pasien, aplikasi mitra, dan sistem pihak ketiga. *Service mesh* adalah jaringan dalam yang mengatur cara layanan saling memanggil. Aturan bisnis tetap berada di kode aplikasi. Tabel 9.7 merinci pembagian urusan di antara ketiganya.

Tabel 9.7: Pembagian urusan antara kode aplikasi, *service mesh*, dan *API gateway*

Urusan	Kode aplikasi	*Service mesh*	*API gateway*
Aturan bisnis, misalnya syarat janji temu	Ya	Tidak	Tidak
Enkripsi dan identitas antarlayanan (mTLS)	Tidak	Ya	Tidak
Retry, batas waktu, pemutus sirkuit antarlayanan	Sebagian	Ya	Tidak
Telemetri lalu lintas antarlayanan	Sebagian	Ya	Tidak
Autentikasi klien luar dan kunci API mitra	Tidak	Tidak	Ya
Pembatasan jumlah permintaan per mitra	Tidak	Tidak	Ya
Idempotensi penerima *event*	Ya	Tidak	Tidak

Pertanyaan berikutnya: kapan *service mesh* tidak diperlukan? Setiap layanan mendapat *proxy* tambahan yang memakai memori dan prosesor, dan *control plane* harus dioperasikan oleh orang yang memahaminya. Beberapa produk kini menawarkan cara tanpa *sidecar*, tetapi beban belajarnya tetap ada. Bila layanan masih

sedikit dan belum ada tim platform, pustaka di dalam aplikasi atau fitur bawaan platform terkelola sering sudah cukup.

Dua contoh dari Medikara Health:

1. Misalkan setelah gelombang 2 peta jalan Bab 14 baru ada enam layanan, dan tim platform yang berisi enam orang masih sibuk dengan fondasi. Enkripsi antarlayanan dipenuhi lewat fitur bawaan platform dan sertifikat dari penyedia. *Service mesh* ditunda.
2. Misalkan sesudah peta jalan selesai jumlah layanan tumbuh menjadi sekitar 35, dimiliki tujuh tim, dan auditor meminta bukti bahwa semua lalu lintas antarlayanan terenkripsi. Pada titik ini, *service mesh* memberi satu tempat untuk menegakkan dan membuktikan aturan itu.

Gaya API juga perlu disinggung karena sering dicampur dengan pola komunikasi. Representational State Transfer (REST) dan GraphQL adalah gaya sinkron untuk meminta atau mengubah data. API berbasis *event* memakai *broker* dan dijelaskan dengan AsyncAPI. *Webhook* adalah panggilan balik ke alamat milik penerima ketika sesuatu terjadi, sehingga cocok untuk mengabari mitra di luar organisasi. Desain kontrak, versi, portofolio API, dan kebijakan *API gateway* dibahas rinci di IT40613. Bab ini hanya menetapkan pola komunikasi di antara layanan milik organisasi sendiri.

Kasus Medikara Health: Integrasi SATUSEHAT sebagai Antarmuka Eksternal

SATUSEHAT milik Kementerian Kesehatan memakai standar Health Level Seven Fast Healthcare Interoperability Resources (HL7 FHIR). Kontraknya ditentukan oleh pemilik platform, sehingga Medikara Health hanya bisa menyesuaikan diri. Pada *context map* Bab 6, hubungan ini dilindungi *anti-corruption layer*.

Di dalam Medikara Health, pengiriman data ke SATUSEHAT dipicu oleh *event* `KunjunganDitutup`. Layanan Gerbang SATUSEHAT berlangganan *event* itu, menerjemahkannya menjadi sumber daya FHIR seperti *Encounter* dan *Observation*, lalu mengirimkannya ke API SATUSEHAT. Bila SATUSEHAT sedang tidak dapat dihubungi, *event* menunggu di antrean dan pelayanan pasien tidak terganggu. *Service mesh* tidak membantu di sini, karena SATUSEHAT berada di luar klaster. Pengelolaan kontrak, versi, dan kredensial API SATUSEHAT menjadi bagian portofolio API yang disusun di IT40613.

9.8 Aktivitas: Peta Interaksi Layanan dan Katalog Event

Tujuan

Tim memodelkan satu alur bisnis sebagai interaksi antarlayanan dan rangkaian *event*, lalu menetapkan pola komunikasi setiap interaksi beserta alasannya. Alur yang disarankan adalah alur janji temu Medikara+: pasien membuat janji, jadwal dokter terkunci, pengingat terkirim, hasil laboratorium tersedia, dan tagihan terbit. Tim yang memakai organisasi nyata memilih alur sejenis dengan izin organisasi dan data yang dianonimkan.

Pembagian Peran

Pembagian peran mengikuti Tabel 9.8. Seluruh anggota bersama-sama memeriksa dampak bila satu layanan mati, karena pemeriksaan itu menyentuh sisi bisnis dan sisi teknis sekaligus.

Tabel 9.8: Pembagian peran dalam Aktivitas Bab 09

Peran	Tanggung jawab
Anggota non-teknis	Menuliskan kejadian bisnis secara berurutan dalam bahasa bisnis, menentukan pihak yang perlu mengetahui setiap kejadian, dan menilai keterlambatan yang masih dapat diterima pasien atau bagian keuangan
Anggota teknis	Menentukan layanan pengirim dan penerima, memilih pola sinkron atau asinkron beserta alasannya, serta memilih pola *event* dan jaminan pengiriman
Seluruh anggota	Memeriksa dampak bila satu layanan mati dan memastikan isi *event* tidak membawa data pribadi yang tidak perlu

Langkah Kerja

1. Pilih alur janji temu atau alur sejenis. Ambil kejadiannya dari papan EventStorming Bab 6 dan, bila ada, dari portofolio API yang disusun di IT40613.
2. Daftar kejadian bisnis secara berurutan dalam bentuk lampau, misalnya `JanjiTemuDibuat` dan `TagihanTerbit`.
3. Tentukan layanan pengirim dan layanan penerima untuk setiap kejadian, lalu isi `sources/chapter09/templat-peta-interaksi.csv`.
4. Pilih sinkron atau asinkron untuk setiap interaksi. Tuliskan alasannya dan akibatnya bila penerima mati selama satu jam pada jam sibuk.
5. Hitung ketersediaan gabungan alur sebelum dan sesudah rancangan ulang, seperti pada kasus janji temu di Seksi 9.2.

6. Susun katalog *event* di `sources/chapter09/templat-katalog-event.csv`: nama, pemilik, pemicu bisnis, pola, isi data, pelanggan, kandungan data pribadi, dan versi.
7. Tandai proses yang memerlukan *orchestration* dan penerima yang wajib idempoten.

Luaran dan Kaitan dengan Capstone

Luarannya adalah peta interaksi layanan berupa diagram satu halaman dan katalog *event* berisi lima sampai sepuluh *event*. Keduanya masuk ke tonggak C3 Kondisi Target sebagai bagian arsitektur target. Katalog *event* menjadi masukan Bab 10, ketika tim menentukan pemilik data dan merancang saga. Pola komunikasi untuk mitra di luar organisasi dicocokkan dengan portofolio API di IT40613.

Kriteria Penilaian

Luaran tim dinilai pada tatap muka Sabtu dengan kriteria pada Tabel 9.9.

Tabel 9.9: Kriteria penilaian Aktivitas Bab 09

Kriteria	Bobot
Kejadian bisnis lengkap, berurutan, dan bernama dalam bahasa bisnis	25%
Pilihan sinkron atau asinkron disertai alasan dan akibat bila penerima mati	30%
Katalog *event* lengkap: pemilik, pola, isi data, pelanggan, dan data pribadi	25%
Perhitungan ketersediaan dan penandaan *orchestration* serta idempotensi	20%

Berkas Pendukung

Berkas pendukung ada di `sources/chapter09/`: `templat-peta-interaksi.csv`, `templat-katalog-event.csv`, contoh katalog dalam format mesin `asyncapi-janji-temu.yaml`, serta folder `contoh-broker-lokal`. Daftar lengkapnya ada di `sources/chapter09/README.md`.

Pendalaman teknis (opsional): anggota teknis menjalankan RabbitMQ secara lokal sesuai Pendalaman Teknis di Seksi 9.3, lalu mengirim dan menerima satu *event* `JanjiTemuDibuat`. Kegiatan ini hanya dijalankan di komputer sendiri.

9.9 Ringkasan

Komunikasi antarlayanan menentukan apakah gangguan di satu sistem tetap kecil atau merambat ke seluruh organisasi. Komunikasi sinkron membuat pemanggil menunggu jawaban, sehingga semua pihak dalam rantai harus hidup bersamaan.

Ketersediaan gabungannya adalah hasil kali ketersediaan setiap mata rantai. Alur janji temu Medikara+ yang memanggil empat sistem berantai hanya tersedia sekitar 97,4 persen. Ketika hanya jadwal dokter yang ditunggu dan sisanya dititipkan sebagai *event*, angka itu naik menjadi sekitar 99,85 persen.

Komunikasi asinkron membutuhkan perantara. Antrean pesan membagi pekerjaan di antara penerima, sedangkan *publish/subscribe* memberi salinan kepada setiap pelanggan tanpa penerbit perlu mengenal mereka. *Event broker* menyediakan keduanya, baik dijalankan sendiri maupun sebagai layanan terkelola. Pilihannya mengikuti keputusan *runtime* dan aturan lokasi data, karena *broker* ikut menyimpan isi pesan.

Event adalah fakta bisnis dalam bentuk lampau, dimiliki satu layanan, dan dinamai dengan bahasa bisnis. Tiga pola pemakaiannya punya biaya berbeda. *Event notification* menjaga data sensitif di pemiliknya. *Event-carried state transfer* membuat penerima mandiri, tetapi menyebarkan data. *Event sourcing* memberi jejak audit lengkap dengan kerumitan yang tinggi. Di Medikara Health, `HasilLabTersedia` sengaja tidak membawa isi hasil laboratorium.

Koordinasi proses dilakukan dengan *choreography* atau *orchestration*. *Choreography* luwes untuk reaksi yang saling lepas, sedangkan *orchestration* tepat untuk proses berurutan yang perlu dilacak, seperti klaim asuransi. Karena sebagian besar *broker* memakai jaminan *at-least-once*, penerima harus idempoten. Tanpa itu, kiriman ulang berubah menjadi tagihan ganda. *Service mesh* mengurus enkripsi, *retry*, dan telemetri antarlayanan di luar kode aplikasi, tetapi baru sepadan ketika jumlah layanan dan tuntutan auditnya cukup besar.

Pesan utama bab ini: pola komunikasi adalah keputusan bisnis yang ditulis dalam bahasa teknis. Setiap kali sebuah layanan memilih menunggu, organisasi menerima risiko bahwa gangguan sistem lain akan dirasakan pasien. Setiap kali layanan memilih mengirim kabar, organisasi menerima bahwa hasilnya datang belakangan. Peta interaksi layanan dan katalog *event* membuat kedua pilihan itu terlihat, terhitung, dan dapat dipertanggungjawabkan.

9.10 Latihan

Latihan 1: Notifikasi Mati Selama Satu Jam

Latihan ini membuktikan bahwa pola komunikasi menentukan seberapa jauh gangguan satu layanan dirasakan pasien. Layanan notifikasi mati selama satu jam pada jam sibuk, ketika Medikara+ menerima sekitar 175 janji temu per jam. Akibatnya dibandingkan antara alur sinkron dan alur berbasis *event*. Tabel 9.10 berisi contoh yang sudah terisi.

1. Tetapkan volume transaksi per jam pada jam sibuk dan lama gangguan.
2. Uraikan akibatnya pada alur sinkron: janji temu yang gagal, pengalaman pasien, dan pekerjaan pemulihan.
3. Uraikan akibatnya pada alur berbasis *event*: janji temu yang tetap dibuat, pesan yang tertunda, dan waktu pemulihan.
4. Ulangi untuk satu layanan lain di organisasi pilihan tim, lalu isi Tabel 9.11.

Tabel 9.10: Latihan 1, contoh: layanan notifikasi mati satu jam pada jam sibuk

Aspek	**Alur sinkron**	**Alur berbasis *event***
Janji temu berhasil	0 dari sekitar 175	Sekitar 175 dari 175
Pengalaman pasien	Pesan gagal; sebagian menelepon pusat panggilan	Janji temu langsung terkonfirmasi di aplikasi
Pesan tertunda	Tidak ada, karena transaksinya batal	Sekitar 175 konfirmasi menunggu di antrean
Setelah layanan pulih	Pasien harus mengulang sendiri; sebagian sudah pindah ke rumah sakit lain	Konfirmasi terkirim dalam beberapa menit setelah pulih
Risiko yang tersisa	Kehilangan pasien dan beban pusat panggilan	Pasien menerima konfirmasi WhatsApp terlambat hingga satu jam

Tabel 9.11: Latihan 1, lembar isian: layanan pilihan tim

Aspek	**Alur sinkron**	**Alur berbasis *event***
Layanan yang mati dan lamanya	…	…
Transaksi berhasil	…	…
Pengalaman pengguna	…	…
Pesan tertunda	…	…
Setelah layanan pulih	…	…
Risiko yang tersisa	…	…

Pertanyaan analisis:

1. Pada kolom alur berbasis *event*, risiko apa yang muncul dan tidak terlihat pada alur sinkron? Siapa yang harus memutuskan bahwa risiko itu dapat diterima?
2. Bila gangguan berlangsung delapan jam, bukan satu jam, baris mana pada tabel tim yang berubah paling besar? Jelaskan dengan angka.

Latihan 2: Idempotensi pada Proses Pembayaran

Latihan ini membuktikan bahwa penerima *event* harus idempoten ketika pesan dapat terkirim lebih dari sekali. Contohnya diambil dari *event* `PembayaranDiterima` yang dikirim gerbang pembayaran ke layanan Penagihan untuk pembayaran pasien umum di Medikara+. Volumenya sekitar 40.000 pembayaran per bulan dengan rata-rata Rp350.000. Tabel 9.12 berisi contoh yang sudah terisi.

1. Tuliskan urutan kejadian ketika pengakuan dari penerima hilang dan *broker* mengirim ulang pesan yang sama.
2. Tuliskan akibatnya pada penerima yang tidak idempoten dan pada penerima yang idempoten.
3. Perkirakan dampak keuangan per bulan bila 0,1 persen pesan terkirim ulang.
4. Ulangi untuk satu penerima lain yang menyentuh uang atau data penting, lalu isi Tabel 9.13.

Tabel 9.12: Latihan 2, contoh: `PembayaranDiterima` terkirim dua kali

Kejadian	Penerima tidak idempoten	Penerima idempoten
Pesan `id` PB-881 diterima pertama kali	Pembayaran Rp350.000 dicatat; tagihan lunas	Pembayaran dicatat; `id` PB-881 disimpan
Koneksi putus sebelum pengakuan sampai	Tidak ada perubahan	Tidak ada perubahan
Broker mengirim ulang PB-881	Pembayaran kedua dicatat; muncul kelebihan bayar Rp350.000	`id` sudah ada; pesan dilewati lalu diakui
Dampak per bulan (0,1 persen dari 40.000)	Sekitar 40 kelebihan bayar palsu, Rp14 juta salah catat	Tidak ada salah catat

Tabel 9.13: Latihan 2, lembar isian: penerima pilihan tim

Kejadian	Penerima tidak idempoten	Penerima idempoten
Pesan diterima pertama kali	…	…
Pengakuan hilang	…	…
Pesan dikirim ulang	…	…
Dampak per bulan	…	…

Pertanyaan analisis:

1. Pada tabel contoh, Rp14 juta per bulan terlihat kecil dibanding total pembayaran. Mengapa angka itu tetap serius bagi bagian keuangan dan bagi kepercayaan pasien?
2. Bila pengirim membuat `id` baru pada setiap pengiriman ulang, baris mana pada tabel yang berubah dan mengapa penerima tidak lagi bisa melindungi

diri?

Latihan 3: Memilih Orchestration atau Choreography

Latihan ini membuktikan bahwa pilihan koordinasi bergantung pada sifat proses, bukan selera tim. Proses klaim asuransi dari mitra dibandingkan dengan proses reaksi atas janji temu, dan hasilnya ada pada Tabel 9.14.

1. Tuliskan langkah setiap proses dan layanan yang terlibat.
2. Nilai setiap proses dengan kriteria pada tabel: urutan, batas waktu, kebutuhan melacak status, dan seberapa sering reaksi baru ditambahkan.
3. Tetapkan pilihan koordinasi beserta alasannya.
4. Ulangi untuk satu proses di organisasi pilihan tim, lalu isi Tabel 9.15.

Tabel 9.14: Latihan 3, contoh: klaim asuransi dan reaksi atas janji temu

Kriteria	Klaim asuransi mitra	Reaksi atas janji temu
Jumlah langkah dan layanan	4 langkah, 5 layanan termasuk mitra	3 reaksi lepas
Urutan langkah	Ketat: polis dicek sebelum berkas dikirim	Bebas
Batas waktu yang dipantau	Ada: 14 hari kerja untuk jawaban mitra	Tidak ada
Perlu melacak status	Ya, sekitar 3.600 klaim per bulan	Tidak
Reaksi baru ditambahkan	Jarang	Sering, misalnya survei kepuasan
Pilihan	***Orchestration***	***Choreography***

Tabel 9.15: Latihan 3, lembar isian: proses pilihan tim

Kriteria	Proses 1: ...	Proses 2: ...
Jumlah langkah dan layanan	...	...
Urutan langkah	...	...
Batas waktu yang dipantau	...	...
Perlu melacak status	...	...
Reaksi baru ditambahkan	...	...
Pilihan	...	...

Pertanyaan analisis:

1. Kriteria mana pada tabel tim yang paling menentukan pilihan? Apakah ada proses yang nilainya terbelah, dan bagaimana tim memutuskannya?
2. Bila orkestrator klaim mati selama satu hari, apa yang terjadi pada sekitar

120 klaim yang sedang berjalan hari itu? Bandingkan dengan akibatnya bila klaim disusun dengan *choreography*.

Bab 10

Data dalam Microservices: Saga dan Outbox

Tujuan Pembelajaran

- Menjelaskan alasan setiap layanan memiliki datanya sendiri dan risiko basis data bersama.
- Merancang saga beserta langkah kompensasinya untuk satu proses lintas layanan.
- Menjelaskan cara pola *transactional outbox* mencegah data dan *event* tidak sinkron.

10.1 Pendahuluan

Seorang pasien Medikara Health yang pernah berobat di rumah sakit Bekasi, lalu dirawat di rumah sakit Depok, hampir pasti punya dua nomor rekam medis. Alamat barunya sudah diperbarui lewat aplikasi Medikara+, tetapi Sistem Informasi Manajemen Rumah Sakit (SIMRS) di Bekasi masih menyimpan alamat lama. Bagian keuangan dan bagian pelayanan menyusun laporan jumlah kunjungan bulan yang sama, dan angkanya tidak pernah persis sama. Strategi "Satu Pasien, Satu Rekam Medis" menuntut keadaan sebaliknya: data pasien yang cocok di seluruh 5 rumah sakit, 18 klinik pratama, laboratorium rujukan, dan 12 apotek.

Bab 6 sampai Bab 9 menentukan batas layanan, tempat menjalankannya, dan cara layanan berkomunikasi. Pemecahan itu belum selesai bila datanya tidak ikut dipikirkan. Layanan baru yang tetap menulis ke basis data SIMRS yang sama hanya memindahkan ketergantungan ke tempat lain. Sebaliknya, layanan yang

menyimpan salinan data sendiri tanpa aturan menambah jumlah versi kebenaran. Tiga pertanyaan kemudian muncul. Siapa pemilik setiap data? Bagaimana proses yang melintasi beberapa layanan tetap utuh tanpa satu transaksi basis data? Bagaimana laporan manajemen tetap tersedia tanpa membebani layanan yang melayani pasien?

Gambar 10.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 10.1: Transaksi lokal, outbox, dan kompensasi saga

Bab ini menjawab ketiga pertanyaan itu secara berurutan. Rujukan utamanya adalah buku *Microservices Patterns* karya Richardson [125] beserta katalog pola daring di microservices.io [126, 128, 127], makalah asli tentang saga dari Garcia-Molina dan Salem [64], buku *Building Microservices* karya Newman [106], artikel Vogels tentang *eventual consistency* [149], dan tulisan Fowler tentang Command Query Responsibility Segregation (CQRS) [58]. Hasil bab ini adalah peta kepemilikan data dan rancangan saga untuk tonggak capstone C3 (Kondisi Target).

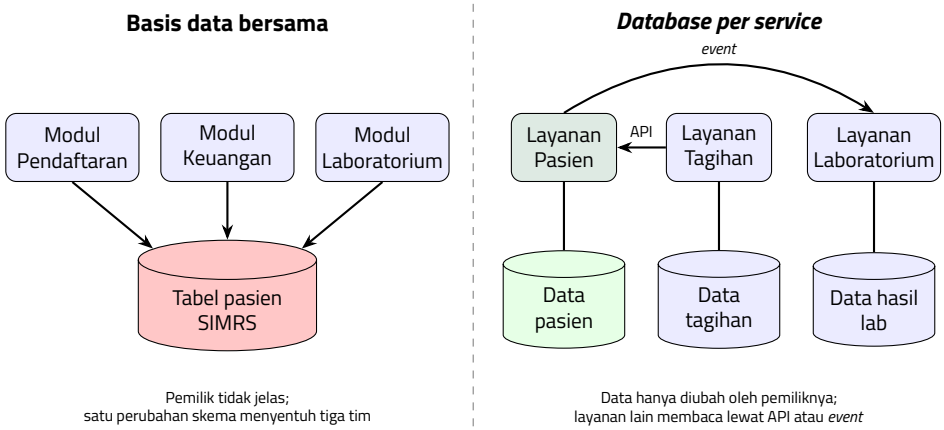
10.2 Database per Service dan Basis Data Bersama

Pola *database per service* menetapkan bahwa data yang disimpan sebuah layanan bersifat privat bagi layanan itu dan hanya dapat diakses lewat Application Programming Interface (API) miliknya [126]. Layanan lain tidak membaca atau menulis tabelnya secara langsung. Bila Layanan Tagihan membutuhkan nama pasien, Layanan Tagihan bertanya kepada Layanan Pasien, bukan membuka tabel pasien. Lawannya adalah basis data bersama: beberapa layanan atau modul membaca dan menulis tabel yang sama. Katalog pola Richardson menggolongkannya sebagai *anti-pattern* [126]. Newman juga menyarankan agar layanan tidak berbagi

basis data, karena layanan lain menjadi bergantung pada rincian penyimpanan yang seharusnya tersembunyi [106].

Analoginya seperti satu buku besar di meja tengah kantor yang boleh ditulisi bagian pendaftaran, keuangan, dan laboratorium. Setiap bagian menambah kolom sesuai kebutuhannya, mencoret isian bagian lain bila dianggap salah, dan tidak ada yang tahu siapa yang terakhir mengubah satu baris. Kantor yang lebih tertib memberi setiap bagian lemari arsip sendiri yang kuncinya dipegang bagian itu. Bagian lain yang memerlukan isi lemari meminta salinan lewat loket, atau menerima tembusan setiap kali ada perubahan penting.

Gambar 10.2 membandingkan kedua susunan itu. Di sisi kiri, tiga modul SIMRS menulis ke satu tabel pasien. Di sisi kanan, setiap layanan memegang datanya sendiri dan layanan lain membaca lewat API atau *event*.



Gambar 10.2: Basis data bersama dibandingkan dengan *database per service*

Pemisahan ini punya dua konsekuensi. Pertama, setiap data punya satu pemilik yang bertanggung jawab atas mutu dan perubahannya. Tim pemilik boleh mengubah struktur tabelnya tanpa rapat dengan tim lain, selama kontrak API dan *event* yang sudah dijanjikan tetap dijaga. Kedua, dua hal yang dulu mudah menjadi sulit: transaksi yang mencakup data beberapa layanan dan *query* yang menggabungkan data beberapa layanan [126]. Seksi 10.3 sampai 10.6 membahas cara menangani kedua kesulitan itu. Tabel 10.1 merangkum perbedaannya dari sisi yang biasa ditanyakan manajemen.

Dua contoh dari Medikara Health:

1. Misalkan vendor ERP (Enterprise Resource Planning) keuangan mengubah arti satu kolom status pembayaran pada basis data SIMRS rumah sakit Depok. Modul laboratorium memakai kolom yang sama untuk menentukan pemeriksaan yang boleh dikerjakan, sehingga selama dua hari 260 permin-

Tabel 10.1: Basis data bersama dan *database per service* dari sisi organisasi

Aspek	Basis data bersama	*Database per service*
Perubahan struktur	Harus disepakati semua tim pengguna; satu perubahan dapat merusak modul lain	Diputuskan tim pemilik; kontrak API dan *event* dijaga
Kepemilikan data	Kabur; tidak jelas siapa yang membetulkan data yang salah	Satu pemilik untuk setiap data
Beban dan kegagalan	*Query* berat satu modul memperlambat semua modul	Beban dan gangguan terkurung di satu layanan
Kontrol akses	Banyak akun punya hak tulis ke tabel sensitif	Akses lewat API yang dicatat dan dibatasi
Rilis	Perubahan besar dirilis serentak	Setiap layanan dirilis mandiri
Harga yang dibayar	Rendah di awal: penggabungan data dan transaksi tunggal mudah	Transaksi dan *query* lintas layanan perlu pola khusus

taan pemeriksaan tertahan sampai kesalahan itu ditemukan.

2. Vendor Medikara+ membaca tabel jadwal dokter SIMRS secara langsung. Setiap kali SIMRS ditingkatkan versinya, Medikara+ harus diuji ulang selama sekitar tiga minggu, karena tidak ada yang dapat memastikan perubahan tabel mana yang memengaruhi aplikasi pasien.

10.2.1 Privat Tidak Berarti Server Sendiri

Dua hal pada *database per service* sering tertukar. Pembagiannya begini. Yang diwajibkan pola ini adalah kepemilikan dan batas akses: tidak ada layanan lain yang menyentuh tabel milik sebuah layanan. Pola ini tidak mewajibkan setiap layanan punya server basis data sendiri, dan tidak mewajibkan setiap layanan memakai teknologi basis data yang berbeda. Richardson menyebut tiga bentuk pada basis data relasional, seperti tercantum pada Tabel 10.2 [126].

Tabel 10.2: Tiga bentuk data privat pada basis data relasional

Bentuk	Cara kerja	Kapan cocok
Tabel privat per layanan	Setiap layanan memiliki sekumpulan tabel yang hanya boleh diakses layanan itu	Awal pemisahan; biaya tambahan paling rendah
Skema privat per layanan	Setiap layanan memiliki skema basis data sendiri dengan akun sendiri	Batas akses perlu ditegakkan oleh basis data, bukan hanya oleh kesepakatan
Server basis data per layanan	Setiap layanan memiliki server basis data sendiri	Layanan dengan beban tinggi atau kebutuhan teknologi khusus

Bagi Medikara Health, bentuk kedua biasanya cukup sebagai langkah pertama. Akun Layanan Tagihan tidak diberi hak apa pun pada skema Layanan Pasien, sehingga aturan kepemilikan tidak bergantung pada disiplin pemrogram semata.

Temuan audit internal tentang kontrol akses yang lemah juga terjawab sebagian, karena hak tulis tidak lagi tersebar ke banyak akun.

10.2.2 Cara Layanan Lain Memperoleh Data

Pertanyaan berikutnya muncul: bila tabel pasien tidak boleh dibuka, bagaimana Layanan Tagihan mencetak nama pasien di tagihan? Ada tiga cara, dengan harga yang berbeda. Tabel 10.3 merangkumnya.

Tabel 10.3: Tiga cara memperoleh data milik layanan lain

Cara	Cara kerja	Kelebihan	Harga
Memanggil API pemilik	Pembaca bertanya setiap kali butuh; beberapa jawaban dapat digabung oleh satu komponen penggabung	Data selalu terbaru	Pembaca ikut terganggu bila pemilik tidak tersedia
Salinan lokal dari *event*	Pemilik menerbitkan *event* perubahan; pembaca menyimpan salinan ringkas yang hanya dibaca	Tetap berjalan walaupun pemilik mati	Salinan bisa tertinggal beberapa saat
Read model khusus	Data beberapa layanan disusun ulang untuk satu kebutuhan baca, misalnya laporan	Cepat untuk laporan dan pencarian lintas layanan	Komponen tambahan dan batas kesegaran

Ketiga cara itu punya satu aturan yang sama: salinan tidak pernah diubah oleh pembacanya. Bila Layanan Tagihan menemukan nama pasien yang salah, Layanan Tagihan melapor kepada Layanan Pasien, bukan membetulkan salinannya sendiri. Aturan ini yang membedakan salinan yang tertib dari data pasien yang tersebar tanpa pemilik. Cara kedua sudah dikenal di Bab 9 sebagai *event-carried state transfer*, dan cara ketiga dibahas di seksi 10.6.

Kasus Medikara Health: Satu Tabel Pasien, Tiga Penulis

Misalkan tim arsitektur memeriksa tabel pasien pada SIMRS rumah sakit Bekasi. Tabel itu punya 64 kolom. Modul pendaftaran menulis 31 kolom identitas dan demografi, modul keuangan menulis 14 kolom penjamin dan status tagihan, dan modul laboratorium menulis 9 kolom data klinis ringkas seperti golongan darah dan alergi. Sisanya, 10 kolom, tidak diketahui lagi siapa penulisnya. Ada 23 akun aplikasi dan pengguna yang punya hak tulis ke tabel itu, termasuk satu akun vendor laboratorium.

Pada tahun lalu, tim keuangan memperlebar kolom nomor kartu penjamin agar dapat menampung format nomor polis asuransi mitra yang baru. Modul pendaftaran tidak siap dengan perubahan itu dan gagal menyimpan data selama tiga jam. Sebanyak 410 pasien didaftarkan dengan formulir kertas, lalu diketik ulang pada malam harinya.
Usulan tim punya tiga bagian. Pertama, Layanan Identitas Pasien menjadi pemilik 31 kolom identitas. Kedua, Layanan Penjaminan menjadi pemilik 14 kolom penjamin. Ketiga, data klinis ringkas pindah ke Layanan Rekam Medis, sedangkan 10 kolom tanpa pemilik ditinjau untuk dihapus. Pemindahan tidak dilakukan sekaligus. Kolom dipindah satu kelompok demi satu kelompok dengan pola *strangler fig* yang dibahas di Bab 14, sambil menutup hak tulis akun lama satu per satu.

10.3 Transaksi Terdistribusi dan Eventual Consistency

Di dalam SIMRS monolitik, pendaftaran pasien rujukan dapat berjalan dalam satu transaksi basis data: catat rujukan, kunci jadwal dokter, catat penjamin, lalu simpan semuanya sekaligus. Transaksi seperti ini punya empat sifat yang disingkat ACID. *Atomicity* berarti semua perubahan tersimpan atau semuanya batal. *Consistency* berarti aturan data tetap terpenuhi. *Isolation* berarti proses lain tidak melihat keadaan setengah jadi. *Durability* berarti perubahan yang sudah tersimpan tidak hilang walaupun server mati. Setelah data dipecah ke beberapa layanan, keempat sifat itu hanya berlaku di dalam satu layanan. Tidak ada lagi satu transaksi yang mencakup rujukan, jadwal, dan penjamin sekaligus.

10.3.1 Batas Two-Phase Commit

Jawaban klasik untuk masalah ini adalah *two-phase commit* (2PC). Satu koordinator bertanya kepada semua peserta apakah mereka siap menyimpan perubahan. Bila semua menjawab siap, koordinator memerintahkan penyimpanan. Bila satu saja menolak, semua peserta membatalkan perubahannya. Analoginya seperti rapat rukun tetangga yang keputusannya baru sah bila seluruh kepala keluarga hadir dan menandatangani. Satu orang yang terlambat membuat seluruh warga menunggu di balai pertemuan, dan selama menunggu tidak ada yang boleh pulang.

Pada *microservices*, 2PC jarang dipakai. Keterbatasannya ada tiga.

1. Semua peserta harus tersedia pada saat yang sama, sehingga ketersediaan transaksi turun [125]. Bila empat layanan masing-masing tersedia 99,5 persen, transaksi yang memerlukan keempatnya hanya dapat berjalan sekitar $0{,}995^4 \approx 98{,}0$ persen dari waktu. Dalam sebulan (720 jam), waktu tidak

tersedia naik dari 3,6 jam menjadi sekitar 14,3 jam.

2. Peserta menahan kunci data selama menunggu keputusan koordinator. Bila koordinator mati setelah semua peserta menyatakan siap, data yang terkunci tidak dapat dipakai proses lain sampai koordinator pulih.
3. Banyak teknologi yang dipakai *microservices* tidak mendukungnya, misalnya sebagian *message broker* dan basis data non-relasional [125]. Layanan eksternal seperti Badan Penyelenggara Jaminan Sosial (BPJS) Kesehatan dan SATUSEHAT juga tidak mungkin ikut serta dalam transaksi milik Medikara Health.

Katalog pola Richardson menyimpulkannya dengan singkat: 2PC bukan pilihan untuk menyimpan perubahan basis data dan menerbitkan pesan sekaligus [127]. Jalan keluarnya adalah menerima bahwa data beberapa layanan tidak selalu cocok pada setiap detik, lalu mengelola selisih itu dengan sengaja.

10.3.2 Eventual Consistency

Eventual consistency adalah jaminan bahwa bila tidak ada perubahan baru, semua pembaca pada akhirnya akan melihat nilai terakhir yang sama. Vogels menyebut rentang waktu antara perubahan dan saat semua pembaca dijamin melihat nilai baru sebagai jendela ketidakkonsistenan (*inconsistency window*) [149]. Selama jendela itu terbuka, dua layanan dapat memberi jawaban berbeda untuk pertanyaan yang sama. Setelah jendela tertutup, keduanya kembali cocok.

Analoginya seperti transfer antarbank. Saldo pengirim langsung berkurang, sedangkan saldo penerima bertambah beberapa menit kemudian. Selama itu aplikasi bank menulis "transaksi sedang diproses". Tidak ada uang yang hilang, dan nasabah tahu apa yang sedang terjadi. Bank tidak berpura-pura bahwa uang sudah sampai, tetapi juga tidak menahan pengirim di depan layar sampai uang diterima.

Pengguna bisnis tidak perlu memahami istilahnya. Yang perlu disepakati bersama mereka adalah tiga pertanyaan untuk setiap data. Berapa lama selisih yang masih dapat diterima? Apa yang dilihat pasien atau petugas selama selisih itu? Apa yang terjadi bila selisih tidak pernah tertutup karena salah satu langkah gagal? Tabel 10.4 memperlihatkan jawabannya untuk lima data Medikara Health.

Tabel itu memperlihatkan bahwa *eventual consistency* bukan aturan untuk semua data. Data yang tidak boleh berselisih sedetik pun, seperti slot jadwal, diletakkan di satu layanan pemilik sehingga transaksi lokalnya tetap ACID. Selisih hanya diterima di antara layanan, bukan di dalam satu layanan.

Dua contoh dampaknya pada pengalaman pasien:

1. Misalkan Medikara+ menampilkan tulisan "pendaftaran berhasil" segera

Tabel 10.4: Toleransi selisih data pada lima data Medikara Health

Data	Selisih yang diterima	Akibat bila terlambat	Pendekatan
Slot jadwal dokter	Tidak ada	Dua pasien di satu slot	Dikunci di dalam Layanan Jadwal sebagai pemilik tunggal
Stok obat apotek	Detik	Obat terjual padahal habis	Stok dipesan di layanan pemilik sebelum dijual
Status kepesertaan pada pendaftaran	Menit	Pasien terlanjur dilayani sebagai peserta	Status "menunggu verifikasi" yang terlihat petugas
Hasil laboratorium di Medikara+	5 menit	Pasien bertanya ke loket	Pemberitahuan dikirim saat hasil tersedia
Laporan kunjungan harian direksi	Beberapa jam	Keputusan memakai angka kemarin	*Read model* yang diperbarui dari *event*

setelah rujukan dicatat, padahal kepesertaan BPJS Kesehatan belum diverifikasi. Dengan sekitar 3.600 rujukan per bulan dan 4 persen di antaranya gagal verifikasi, sekitar 144 pasien per bulan akan menerima kabar berhasil yang kemudian dicabut. Tulisan yang jujur adalah "pendaftaran diterima, menunggu verifikasi kepesertaan".

2. Hasil laboratorium yang sudah divalidasi muncul di Medikara+ paling lambat 5 menit kemudian. Selama 5 menit itu pasien yang bertanya ke loket mendapat jawaban "hasil sedang dikirim ke aplikasi", bukan "hasil belum ada".

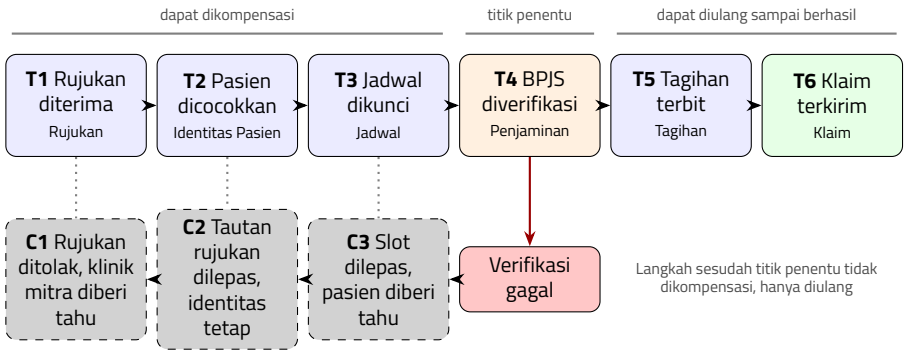
10.4 Saga dan Transaksi Kompensasi

Garcia-Molina dan Salem memperkenalkan saga pada 1987 untuk menangani transaksi panjang (*long-lived transaction*) yang menahan data terlalu lama bila dijalankan sebagai satu transaksi [64]. Gagasannya: transaksi panjang dipecah menjadi rangkaian transaksi kecil $T_1, T_2, \ldots, T_n$ yang masing-masing langsung disimpan. Setiap T_i punya transaksi kompensasi C_i yang membatalkan akibatnya secara bisnis. Sistem menjamin salah satu dari dua hasil: seluruh T_1 sampai T_n selesai, atau T_1 sampai T_j selesai lalu diikuti C_j sampai C_1 secara mundur. Richardson membawa gagasan ini ke *microservices*: saga adalah rangkaian transaksi lokal di beberapa layanan yang dikoordinasikan dengan andal, dan perubahan dari langkah yang sudah selesai dibatalkan oleh transaksi kompensasi bila langkah berikutnya gagal [125, 128].

Analoginya seperti mengurus perjalanan dinas. Tiket pesawat, hotel, dan mobil sewaan dipesan berurutan dari tiga penyedia yang berbeda, dan tidak ada satu tombol untuk membatalkan ketiganya. Bila hotel ternyata penuh, staf memba-

talkan tiket pesawat dan meminta pengembalian dana, mungkin dengan potongan biaya. Pembatalan itu tidak menghapus sejarah: tiket tetap pernah dibeli, lalu dikembalikan, dan keduanya tercatat.

Gambar 10.3 memperlihatkan saga pendaftaran pasien rujukan dari klinik mitra sampai klaim terkirim. Baris atas adalah transaksi lokal, masing-masing milik satu layanan. Baris bawah adalah kompensasi yang berjalan mundur bila verifikasi kepesertaan gagal.



Gambar 10.3: Saga pendaftaran pasien rujukan beserta kompensasinya

10.4.1 Kompensasi Bukan Rollback

Kompensasi berbeda dari *rollback* basis data. *Rollback* menghapus perubahan yang belum disimpan, seolah-olah tidak pernah terjadi. Kompensasi adalah transaksi baru yang membalik akibat bisnis dari transaksi yang sudah disimpan dan mungkin sudah dilihat pihak lain. Akibatnya, kompensasi sering tidak sempurna. Pesan pengingat yang sudah terkirim tidak dapat ditarik kembali, sehingga kompensasinya adalah pesan ralat. Obat yang sudah diserahkan tidak dapat dibatalkan, sehingga kompensasinya adalah proses retur. Kompensasi juga tidak selalu berarti batal. Bila kepesertaan BPJS Kesehatan gagal diverifikasi, pilihan yang lebih baik bagi pasien mungkin menawarkan jalur pembayaran umum atau asuransi lain sebelum jadwal dilepas.

Karena ada langkah yang tidak dapat dibatalkan, Richardson membagi transaksi dalam saga menjadi tiga jenis [125]. Transaksi yang dapat dikompensasi (*compensatable transaction*) berada di depan dan punya kompensasi. Titik penentu (*pivot transaction*) adalah langkah yang menentukan apakah saga akan berlanjut sampai selesai atau berbalik. Transaksi yang dapat diulang (*retriable transaction*) berada sesudah titik penentu dan dijamin akhirnya berhasil bila terus diulang, sehingga tidak perlu kompensasi. Aturan desainnya: langkah yang paling mungkin gagal dan langkah yang tidak dapat dibatalkan diletakkan sedekat mungkin dengan titik penentu.

10.4.2 Choreography dan Orchestration pada Saga

Bab 9 sudah membedakan *choreography* dan *orchestration* untuk alur *event* secara umum. Saga memakai kedua pola yang sama [128]. Pada saga berbasis *choreography*, setiap layanan menyelesaikan transaksinya, menerbitkan *event*, dan layanan berikutnya bereaksi terhadap *event* itu. Pada saga berbasis *orchestration*, satu pengatur saga (*orchestrator*) memberi perintah kepada setiap peserta dan mencatat status saga. Tabel 10.5 membandingkan keduanya.

Tabel 10.5: Saga berbasis *choreography* dan *orchestration*

Aspek	***Choreography***	***Orchestration***
Pengatur alur	Tersebar; setiap layanan tahu *event* apa yang harus ditanggapi	Satu pengatur saga menyimpan urutan langkah
Melacak status satu saga	Sulit; status harus disusun dari banyak *event*	Mudah; status tercatat di pengatur saga
Menambah atau mengubah langkah	Menyentuh beberapa layanan	Terutama mengubah pengatur saga
Risiko	Ketergantungan melingkar yang tidak terlihat	Logika bisnis menumpuk di pengatur saga
Cocok untuk	Saga pendek, dua sampai tiga langkah	Saga panjang, banyak kompensasi, perlu jejak audit

Dua contoh dari Medikara Health:

1. Saga pembatalan janji temu hanya melibatkan dua langkah: slot dilepas, lalu pengingat dibatalkan. *Choreography* cukup, karena alurnya mudah dibaca dari dua *event*.
2. Saga pendaftaran rujukan melibatkan enam layanan dan tiga kompensasi, dan klinik mitra ingin melihat status rujukannya. *Orchestration* lebih tepat, karena status setiap rujukan dapat dibaca di satu tempat dan setiap langkah meninggalkan jejak bagi auditor.

10.4.3 Saga Tidak Punya Isolasi

Saga melepas satu sifat ACID, yaitu isolasi [128]. Setiap transaksi lokal langsung disimpan, sehingga proses lain dapat melihat keadaan setengah jadi. Misalkan laporan kunjungan dihitung ketika jadwal sudah dikunci tetapi kepesertaan belum diverifikasi. Pasien itu ikut terhitung, lalu hilang lagi dari laporan bila verifikasinya gagal.

Richardson menyebut beberapa penangkal untuk masalah ini [125]. Penangkal yang paling mudah dipahami pengguna bisnis adalah kunci semantik (*semantic lock*): data yang sedang diproses saga diberi status yang menandai bahwa prosesnya belum selesai, misalnya `MENUNGGU_VERIFIKASI`. Laporan, petugas loket, dan saga lain membaca status itu dan tahu bahwa data tersebut belum final.

Dua contohnya: laporan kunjungan hanya menghitung rujukan yang statusnya sudah `TERVERIFIKASI`, dan klinik mitra yang ingin membatalkan rujukan berstatus `MENUNGGU_VERIFIKASI` diarahkan menunggu hasil verifikasi atau meminta pembatalan lewat pengatur saga.

Kasus Medikara Health: Saga Pendaftaran Pasien Rujukan

Misalkan klinik mitra mengirim sekitar 3.600 rujukan per bulan ke lima rumah sakit Medikara Health. Sekitar 4 persen, atau 144 rujukan, gagal verifikasi kepesertaan BPJS Kesehatan. Saat ini kegagalan itu ditangani lewat telepon, dan slot dokter spesialis yang sudah dikunci baru dilepas rata-rata dua hari kemudian. Dengan slot 15 menit, sekitar 36 jam praktik dokter spesialis per bulan terkunci tanpa pasien.

Tim merancang saga berbasis *orchestration* dengan pengatur saga di Layanan Rujukan. Langkah dan kompensasinya tercantum pada Tabel 10.6. Anggota tim dari bagian pelayanan menambahkan satu aturan bisnis: sebelum kompensasi C3 berjalan, petugas punya waktu dua jam kerja untuk menawarkan jalur pembayaran umum atau asuransi lain kepada pasien. Bila pasien menolak, kompensasi berjalan otomatis. Dengan aturan ini, slot dilepas paling lambat dua jam kerja setelah verifikasi gagal, bukan dua hari.

Tabel 10.6: Langkah, kompensasi, dan jenis transaksi pada saga pendaftaran rujukan

No	Layanan	Transaksi lokal	Kompensasi	Jenis
T1	Rujukan	Rujukan dicatat dengan status `MENUNGGU_VERIFIKASI`	Status `DITOLAK`; klinik mitra menerima alasannya	Dapat dikompensasi
T2	Identitas Pasien	Pasien dicocokkan dengan identitas induk	Tautan ke rujukan dilepas; identitas tetap disimpan	Dapat dikompensasi
T3	Jadwal	Slot dokter spesialis dikunci	Slot dilepas; pasien dan klinik mitra diberi tahu	Dapat dikompensasi
T4	Penjaminan	Kepesertaan BPJS Kesehatan diverifikasi	Tidak ada; bila gagal, C3, C2, dan C1 berjalan	Titik penentu
T5	Tagihan	Tagihan terbit setelah kunjungan	Tidak ada; koreksi lewat tagihan baru	Dapat diulang
T6	Klaim	Klaim dikirim ke BPJS Kesehatan	Tidak ada; dikirim ulang sampai diterima	Dapat diulang

Ada satu hal yang sering luput pada T2. Identitas pasien yang baru dicocokkan tidak dihapus walaupun rujukannya ditolak. Identitas itu sah dan mungkin dipakai pada kunjungan lain. Yang dibatalkan hanya tautan antara identitas dan rujukan. Contoh kecil ini memperlihatkan mengapa kompensasi harus diperiksa oleh anggota tim yang memahami aturan bisnis, bukan hanya oleh pemrogram.

10.5 Pola Transactional Outbox

Saga bergantung pada *event*. Setiap langkah harus menyimpan perubahan di basis datanya dan menerbitkan *event* ke *event broker* agar langkah berikutnya berjalan. Basis data dan *broker* adalah dua sistem yang berbeda, dan 2PC tidak dapat dipakai untuk menyatukan keduanya. Masalah ini disebut *dual write*: dua penulisan ke dua tempat yang harus sama-sama berhasil, tetapi tidak dilindungi satu transaksi. Tabel 10.7 memperlihatkan akibat setiap urutan penulisan bila proses mati di tengah jalan.

Tabel 10.7: Akibat urutan penulisan bila proses mati di tengah jalan, pada langkah penguncian jadwal

Urutan	Keadaan setelah gagal	Akibat bisnis
Simpan data, lalu terbitkan *event*	Slot terkunci, *event* tidak pernah terbit	Saga macet; pasien tidak pernah diberi tahu jadwalnya
Terbitkan *event*, lalu simpan data	*Event* terbit, penyimpanan slot batal	Pasien menerima kabar jadwal yang sebenarnya tidak terkunci
Transactional outbox	Data dan baris *outbox* tersimpan bersama atau batal bersama	*Event* pasti terbit, tetapi mungkin lebih dari sekali

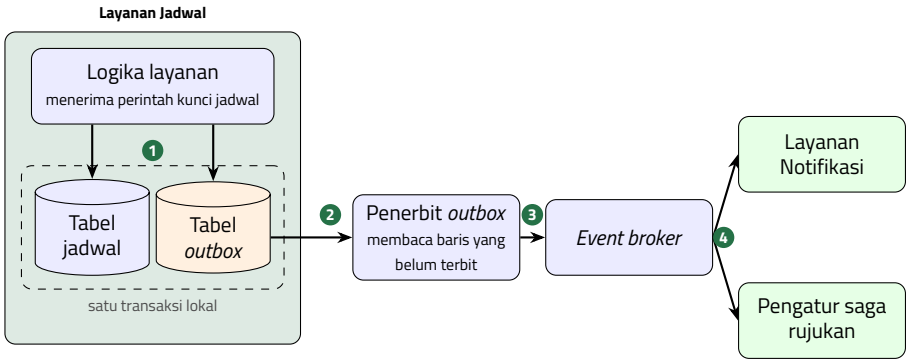
Pola *transactional outbox* menyimpan *event* sebagai baris di tabel *outbox* pada basis data yang sama, di dalam transaksi lokal yang sama dengan perubahan data bisnis [127]. Karena keduanya berada di satu transaksi ACID, keduanya tersimpan bersama atau batal bersama. Sebuah proses terpisah, yang disebut penerbit (*message relay*), membaca baris *outbox* yang belum terbit, meneruskannya ke *broker*, lalu menandainya sudah terbit.

Analoginya seperti baki surat keluar di kantor tata usaha. Staf yang memutuskan sesuatu mencatatnya di buku agenda dan pada saat yang sama menaruh surat tembusan di baki surat keluar. Staf tidak berlari sendiri mengantar surat. Kurir datang berkala, mengambil isi baki, mengantarnya, lalu mencentang buku ekspedisi. Bila kurir sakit hari itu, surat tetap aman di baki dan diantar esok hari. Yang mungkin terjadi hanyalah satu surat terantar dua kali karena kurir lupa mencentang.

Gambar 10.4 memperlihatkan alurnya pada langkah penguncian jadwal.

10.5.1 Dua Cara Menjalankan Penerbit

Penerbit dapat bekerja dengan dua cara [127]. Cara pertama, *polling publisher*, membaca tabel *outbox* secara berkala dengan *query* biasa. Cara kedua, *transaction log tailing*, membaca catatan transaksi basis data (*transaction log*) dan menangkap setiap baris *outbox* yang baru tersimpan. Teknik membaca catatan transaksi ini dikenal sebagai Change Data Capture (CDC). Contoh perangkat sumber terbuka untuk CDC adalah Debezium. Tabel 10.8 membandingkan keduanya.



1 data dan *event* tersimpan bersama atau batal bersama; 2 penerbit membaca dan menandai; 3 *event* diterbitkan; 4 *consumer* idempoten menerima, karena *event* bisa datang dua kali

Gambar 10.4: Alur *transactional outbox* pada Layanan Jadwal

Tabel 10.8: Dua cara menjalankan penerbit *outbox*

Aspek	***Polling publisher***	***Transaction log tailing* (CDC)**
Cara kerja	*Query* berkala ke tabel *outbox*	Membaca catatan transaksi basis data
Kesegaran *event*	Bergantung pada jeda di antara dua *query*, biasanya detik	Hampir seketika
Beban ke basis data	Bertambah seiring frekuensi *query*	Kecil, tetapi butuh hak akses khusus
Keterampilan operasi	Cukup pemahaman *query* biasa	Perlu mengelola perangkat CDC dan konfigurasi basis data
Cocok untuk	Volume kecil sampai menengah, tim yang baru mulai	Volume besar, banyak layanan

10.5.2 Konsekuensi: Event Dapat Datang Dua Kali

Pola ini punya dua konsekuensi. Pertama, penerbit dapat menerbitkan satu *event* lebih dari sekali, misalnya bila penerbit mati setelah meneruskan *event* tetapi sebelum menandainya [127]. Jaminannya adalah *at-least-once*, sehingga setiap *consumer* wajib idempoten seperti dibahas di Bab 9. Cara yang paling umum adalah memberi setiap *event* nomor unik, lalu *consumer* mencatat nomor yang sudah diproses dan menolak nomor yang sama. Kedua, tabel *outbox* dan penerbit menjadi komponen yang harus dirawat: baris lama perlu dibersihkan berkala, dan keterlambatan penerbit perlu dipantau sebagai bagian dari *observability* di Bab 11.

Dua contoh dari Medikara Health:

1. Medikara+ menangani sekitar 28.000 janji temu per bulan. Misalkan tanpa *outbox*, 0,3 persen *event* `JanjiTemuDibuat` hilang ketika layanan dimulai ulang, sehingga sekitar 84 pasien per bulan tidak menerima pengingat dan

sebagian tidak datang. Dengan *outbox*, tidak ada *event* yang hilang. Beberapa pengingat mungkin terkirim dua kali, dan Layanan Notifikasi menolak duplikat berdasarkan nomor *event*.

2. *Event* `TagihanTerbit` diteruskan ke ERP keuangan. Bila ERP tidak idempoten, satu *event* yang terkirim dua kali menjadi dua piutang untuk satu kunjungan. Karena itu, penerimaan *event* di sisi ERP diuji dengan *event* kembar sebelum *outbox* diaktifkan.

Pendalaman Teknis (opsional): Outbox Sederhana dengan PostgreSQL

Folder `sources/chapter10/contoh-outbox-postgresql/` berisi skema dan penerbit *outbox* untuk Layanan Jadwal. Listing 10.1 adalah transaksi lokal yang mengunci slot dan mencatat *event* `JadwalDikunci`. Perintah `UPDATE` dan `INSERT` digabung dalam satu pernyataan, sehingga baris *outbox* hanya tercipta bila slot benar-benar berhasil dikunci.

```
BEGIN;
WITH slot_terkunci AS (
  UPDATE jadwal_dokter
  SET status = 'DIKUNCI', id_rujukan = 'RJK-2026-11-00042'
  WHERE id_slot = 'SLOT-BKS-0915' AND status = 'TERSEDIA'
  RETURNING id_slot, id_dokter, id_rujukan
)
INSERT INTO outbox (jenis_event, id_agregat, isi)
SELECT 'JadwalDikunci', id_slot,
  jsonb_build_object('id_rujukan', id_rujukan, 'id_dokter', id_dokter)
FROM slot_terkunci;
COMMIT;
```

Listing 10.1: Transaksi lokal dengan *outbox*, tersedia di `sources/chapter10/contoh-outbox-postgresql/skema-outbox.sql`

Listing 10.2 adalah inti penerbit dengan cara *polling publisher*. Klausa `FOR UPDATE SKIP LOCKED` pada *query* pengambilan membuat dua penerbit yang berjalan bersamaan tidak mengambil baris yang sama. Agar contoh dapat dijalankan tanpa memasang *broker*, *event* ditulis ke berkas `event-terbit.jsonl` sebagai pengganti *broker*.

```
def proses_satu_kelompok(koneksi):
  """Menerbitkan satu kelompok baris outbox, lalu mengembalikan
      jumlahnya.

  Penandaan baru tersimpan setelah penerbitan. Bila proses mati di
      antara
  keduanya, event yang sama terbit ulang pada putaran berikutnya.
      Karena itu
  consumer wajib idempoten: jaminannya at-least-once, bukan exactly-
      once.
  """
  with koneksi.transaction():
    hasil = koneksi.execute(PERINTAH_AMBIL, (UKURAN_KELOMPOK,))
```

```
    baris_outbox = hasil.fetchall()
    for id_event, jenis, id_agregat, isi, nomor_baris in baris_outbox:
      terbitkan({"id_event": str(id_event), "jenis": jenis,
                 "id_agregat": id_agregat, "isi": isi})
      koneksi.execute(PERINTAH_TANDAI, (nomor_baris,))
  return len(baris_outbox)
```

Listing 10.2: Penerbit *outbox*, tersedia di `sources/chapter10/contoh-outbox-postgresql/penerbit-outbox.py`

Skema dimuat dengan `psql -d medikara -f skema-outbox.sql`, lalu penerbit dijalankan dengan `python penerbit-outbox.py`, hanya pada PostgreSQL milik sendiri. Bila penerbit dihentikan paksa tepat setelah menulis ke berkas lalu dijalankan lagi, *event* yang sama muncul dua kali dengan `id_event` yang sama: itulah *at-least-once*.

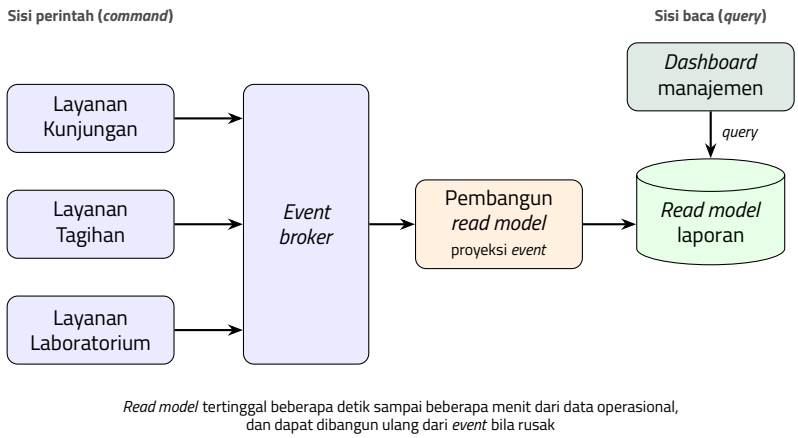
10.6 CQRS dan Read Model untuk Pelaporan

CQRS memisahkan model yang dipakai untuk mengubah data dari model yang dipakai untuk membaca data [58]. Sisi perintah (*command*) menerima perubahan dan menjaga aturan bisnis. Sisi baca (*query*) menyajikan data dalam bentuk yang paling cocok untuk pertanyaan pembacanya. Dalam *microservices*, sisi baca biasanya berupa *read model*: tabel atau basis data terpisah yang dibangun dari *event* yang diterbitkan layanan pemilik. Fowler mencatat bahwa model baca dan model tulis boleh memakai basis data yang terpisah, sehingga basis data sisi baca dapat berfungsi sebagai basis data laporan.

Analoginya seperti papan informasi keberangkatan di bandara. Petugas maskapai mengubah jadwal di sistem operasional mereka. Papan di ruang tunggu hanya menampilkan, dan isinya berubah beberapa detik kemudian. Ribuan penumpang membaca papan itu tanpa mengganggu petugas di meja *check-in*. Bila papan rusak, jadwal penerbangan tidak ikut rusak, dan papan dapat dinyalakan ulang dari data maskapai.

Gambar 10.5 memperlihatkan susunannya. Layanan Kunjungan, Layanan Tagihan, dan Layanan Laboratorium tetap memegang datanya masing-masing. Pembangun *read model* menerima *event* dari ketiganya dan menyusun tabel laporan yang dibaca *dashboard* manajemen.

Read model memberi tiga manfaat. Pertama, *query* laporan yang berat tidak lagi membebani layanan yang melayani pasien. Kedua, laporan yang memerlukan data beberapa layanan dapat disusun tanpa menggabungkan tabel milik layanan lain secara langsung. Ketiga, *read model* dapat dibangun ulang dari *event* bila rusak atau bila bentuk laporannya berubah, selama *event*-nya masih disimpan.



Gambar 10.5: *Read model* laporan yang dibangun dari *event* layanan operasional

Batasannya juga perlu ditulis eksplisit. *Read model* selalu tertinggal dari data operasional, biasanya beberapa detik sampai beberapa menit. Karena itu, *read model* tidak dipakai untuk keputusan yang memerlukan data saat itu juga. Ketersediaan slot jadwal, misalnya, tetap diperiksa di Layanan Jadwal, bukan di *read model*. Fowler juga mengingatkan bahwa bagi sebagian besar sistem CQRS menambah kerumitan yang berisiko, sehingga pola ini dipakai hanya pada bagian sistem yang benar-benar membutuhkannya [58].

Dua contoh dari Medikara Health:

1. Misalkan *dashboard* kunjungan harian direksi saat ini menjalankan *query* langsung ke basis data SIMRS kelima rumah sakit setiap pukul 07.00. Pada jam yang sama loket pendaftaran pagi mulai ramai, dan waktu simpan pendaftaran naik dari 2 detik menjadi 9 detik selama sekitar 20 menit. Dengan *read model*, *query* direksi tidak lagi menyentuh SIMRS.
2. Portal klinik mitra menampilkan status setiap rujukan. Status itu dibaca dari *read model* yang dibangun dari *event* pengatur saga, sehingga sekitar 3.600 rujukan per bulan dapat dipantau mitra tanpa membebani Layanan Rujukan.

Kasus Medikara Health: Laporan Spreadsheet sebagai Calon Read Model

Misalkan bagian perencanaan Medikara Health menyusun 14 laporan manajemen setiap bulan dengan *spreadsheet*. Enam staf bekerja sekitar tiga hari kerja setiap awal bulan untuk menarik ekspor dari lima SIMRS, ERP keuangan, dan sistem laboratorium, lalu menyatukannya secara manual. Jumlah kunjungan pada laporan pelayanan dan laporan keuangan berbeda sampai 3 persen, karena setiap ekspor diambil pada jam yang berbeda dan

memakai definisi kunjungan yang berbeda.
Tim memilih empat laporan pertama untuk dipindah ke *read model*, seperti tercantum pada Tabel 10.9. Definisi "kunjungan" disepakati sekali dan dituangkan ke dalam aturan proyeksi *event*, bukan diulang di setiap *spreadsheet*. Selama dua bulan pertama, laporan lama dan laporan baru disusun berdampingan, lalu selisihnya diperiksa bersama bagian keuangan sebelum *spreadsheet* dihentikan.

Tabel 10.9: Empat laporan pertama yang dipindah ke *read model*

Laporan	*Event* sumber	Kesegaran yang cukup	Pengguna utama
Kunjungan harian per fasilitas	`KunjunganSelesai`	1 jam	Direksi, kepala rumah sakit
Pendapatan per penjamin	`TagihanTerbit`, `PembayaranDiterima`	1 hari kerja	Direktorat Keuangan
Waktu tunggu rujukan	`RujukanDiterima`, `JadwalDikunci`	1 jam	Bagian pelayanan, klinik mitra
Status klaim BPJS Kesehatan	`KlaimTerkirim`, `KlaimDiverifikasi`	1 hari kerja	Bagian klaim

10.7 Kepemilikan Data Pasien: Satu Pasien, Satu Rekam Medis

Strategi "Satu Pasien, Satu Rekam Medis" menuntut satu identitas pasien yang berlaku di seluruh fasilitas. Dalam arsitektur layanan, identitas itu dipegang satu pemilik, misalnya Layanan Identitas Pasien, yang menjalankan indeks pasien induk (*master patient index*, MPI). Layanan ini menyimpan data identitas induk, seperti nama, Nomor Induk Kependudukan (NIK), tanggal lahir, dan nomor rekam medis induk. Layanan lain cukup menyimpan nomor identitas induk dan salinan ringkas yang diperlukan. Bila ditemukan dua catatan untuk orang yang sama, keduanya ditautkan, bukan dihapus, agar jejak riwayatnya tetap utuh. Standar HL7 FHIR (Health Level Seven Fast Healthcare Interoperability Resources) menyediakan elemen `link` pada sumber daya `Patient` untuk menautkan sumber daya `Patient` lain yang menyangkut orang yang sama. Spesifikasi itu juga mencatat bahwa setiap organisasi biasanya memegang catatan `Patient` sendiri, sehingga satu pasien dapat punya beberapa catatan [71].

Analoginya seperti nomor induk mahasiswa. Seorang mahasiswa tercatat di perpustakaan, bagian keuangan, dan laboratorium. Setiap unit menyimpan catatannya sendiri, misalnya pinjaman buku atau tagihan semester, tetapi semuanya

merujuk ke nomor induk yang sama. Biro akademik adalah pemilik data identitas. Perpustakaan tidak mengubah nama mahasiswa di catatannya; bila nama itu salah, perpustakaan meminta biro akademik membetulkannya.

Tabel 10.10 adalah contoh peta kepemilikan data Medikara Health. Setiap entitas punya tepat satu layanan pemilik, sejumlah layanan pembaca, dan cara memperoleh data yang disepakati.

Tabel 10.10: Contoh peta kepemilikan data Medikara Health

Entitas data	Layanan pemilik	Layanan pembaca	Cara memperoleh
Identitas pasien induk	Identitas Pasien	Jadwal, Tagihan, Rekam Medis, Medikara+, Integrasi SATUSEHAT	API pencarian; *event* `PasienDiperbarui`
Persetujuan pasien	Persetujuan	Rekam Medis, Integrasi SATUSEHAT, Medikara+	API yang wajib diperiksa sebelum data dibuka
Rujukan	Rujukan	Jadwal, Penjaminan, Klaim	*Event* `RujukanDiterima`
Jadwal dokter dan janji temu	Jadwal	Medikara+, Notifikasi, laporan	*Event* `JadwalDikunci`, `JanjiTemuDibuat`
Kepesertaan dan penjamin	Penjaminan	Tagihan, Klaim	API saat pendaftaran
Hasil laboratorium	Laboratorium	Rekam Medis, Medikara+, Integrasi SATUSEHAT	*Event* `HasilLabTersedia`
Tagihan dan pembayaran	Tagihan	Klaim, ERP keuangan, laporan	*Event* `TagihanTerbit`
Klaim BPJS Kesehatan	Klaim	ERP keuangan, laporan	*Event* `KlaimTerkirim`

10.7.1 Persetujuan dan Hak Akses sebagai Batasan Desain

Kepemilikan data pasien tidak hanya soal teknis. Undang-Undang Pelindungan Data Pribadi (UU PDP) menggolongkan data dan informasi kesehatan sebagai data pribadi yang bersifat spesifik [120]. Bagi arsitektur layanan, ketentuan ini menjadi tiga batasan desain. Pertama, catatan persetujuan pasien punya pemilik sendiri, yaitu Layanan Persetujuan, dan layanan lain memeriksanya sebelum membuka data. Kedua, *event* membawa data seperlunya. *Event* `HasilLabTersedia` cukup membawa nomor pasien induk dan nomor hasil, bukan seluruh nilai pemeriksaan, sehingga *consumer* yang berhak membaca rinciannya lewat API yang dicatat. Ketiga, hak berlangganan *event* di *broker* dibatasi seketat hak akses ke basis data. Rincian dasar hukum, persetujuan, dan pengendalian akses dibahas di IT40813.

Peta kepemilikan pada tingkat layanan juga perlu sejalan dengan arsitektur data tingkat *enterprise*: domain data, pemilik data dari sisi bisnis (*data owner*), pengurus data (*data steward*), dan glosarium istilah. Peta di bab ini menjawab "layanan mana yang menulis data ini". Arsitektur data *enterprise* menjawab "unit

bisnis mana yang bertanggung jawab atas makna dan mutu data ini". Keduanya dibahas berdampingan, dan bagian *enterprise* dibahas rinci di IT40513.

Kasus Medikara Health: Identitas Induk dan Pertukaran Data dengan SATUSEHAT

Misalkan inventarisasi awal menemukan sekitar 910.000 nomor rekam medis di lima SIMRS dan sistem 18 klinik pratama. Setelah pencocokan berdasarkan NIK, nama, tanggal lahir, dan alamat, nomor itu mewakili sekitar 640.000 orang. Artinya, sekitar 270.000 nomor, atau 29,7 persen, adalah catatan kedua atau ketiga untuk orang yang sama. Catatan ganda itu ditautkan ke identitas induk. Sekitar 12.000 pasangan yang kemiripannya meragukan diperiksa petugas rekam medis, bukan digabung otomatis.
Pengiriman data ke SATUSEHAT ditangani Layanan Integrasi SATUSEHAT. Layanan ini berlangganan *event* dari pemilik data, misalnya `KunjunganSelesai` dan `HasilLabTersedia`, mengambil rincian yang diperlukan lewat API, lalu menyusunnya menjadi sumber daya FHIR seperti `Patient`, `Encounter`, dan `Observation`. Layanan Integrasi SATUSEHAT adalah pembaca, bukan pemilik. Bila SATUSEHAT menolak satu kiriman karena data identitas tidak cocok, perbaikannya dilakukan di Layanan Identitas Pasien, lalu kiriman diulang. Susunan ini menggantikan integrasi *point-to-point* dari setiap SIMRS, sehingga perubahan aturan pengiriman cukup dikerjakan di satu layanan.

10.8 Aktivitas: Peta Kepemilikan Data dan Rancangan Saga

Tujuan. Tim menetapkan pemilik setiap entitas data utama pada proses capstone, lalu merancang satu saga lintas layanan beserta kompensasinya. Keduanya menjadi bagian tonggak capstone C3 (Kondisi Target), melengkapi peta interaksi layanan dan katalog *event* dari Bab 9.

Pembagian peran.

- Anggota non-teknis, misalnya yang berlatar manajemen, keuangan, hukum, atau kesehatan, memastikan aturan bisnis dan langkah kompensasi masuk akal bagi pasien dan bagian keuangan. Mereka juga menulis kalimat status yang dilihat pasien atau mitra selama saga berjalan.
- Anggota teknis memilih pola koordinasi saga, menentukan titik pemakaian *outbox*, dan memastikan setiap *consumer* punya cara menolak *event* kembar.
- Seluruh anggota menguji rancangan dengan skenario gagal dan mencatat keputusan yang diperdebatkan.

Langkah kerja.

1. Daftar entitas data utama pada proses yang dipilih tim, dengan bantuan katalog *event* Bab 9. Pakai lembar "Isian Tim" pada `sources/chapter10/templat-peta-kepemilikan-data.xlsx`.
2. Tetapkan tepat satu layanan pemilik dan sejumlah layanan pembaca untuk setiap entitas, lalu pilih cara memperoleh data dari Tabel 10.3. Tandai entitas yang saat ini ditulis lebih dari satu sistem.
3. Pilih satu proses lintas layanan, misalnya pendaftaran pasien rujukan dari klinik mitra sampai klaim terkirim, atau proses sejenis di organisasi pilihan.
4. Uraikan langkah saga dan kompensasinya dengan `sources/chapter10/templat-rancangan-saga.md`. Tandai titik penentu, langkah yang dapat diulang, dan setiap langkah yang menerbitkan *event* lewat *outbox*.
5. Uji rancangan dengan dua skenario gagal: satu kegagalan sebelum titik penentu dan satu layanan yang tidak tersedia sesudah titik penentu. Catat kompensasi yang berjalan, pihak yang diberi tahu, dan batas waktunya.

Luaran. Peta kepemilikan data yang terisi, satu rancangan saga lengkap dengan dua skenario gagal, dan catatan satu halaman tentang keputusan yang paling diperdebatkan tim beserta *tradeoff*-nya. Luaran ini masuk ke C3 sebagai peta kepemilikan data dan rancangan saga, dan akan dipakai lagi saat menyusun Service Level Objective (SLO) perjalanan pasien pada Bab 11.

Kriteria penilaian. Tabel 10.11 memuat kriteria singkat yang dipakai saat tinjauan Sabtu.

Tabel 10.11: Kriteria penilaian Aktivitas Bab 10

Kriteria	Bobot	Bukti yang dicari
Kejelasan kepemilikan data	25 persen	Setiap entitas punya tepat satu pemilik dan cara baca yang disepakati
Kelengkapan saga	25 persen	Titik penentu, kompensasi, dan langkah yang dapat diulang ditandai
Kewajaran kompensasi bagi bisnis	25 persen	Kompensasi ditulis dalam bahasa bisnis dan diperiksa anggota non-teknis
Ketahanan terhadap kegagalan	25 persen	Dua skenario gagal diuji; *outbox* dan idempotensi dipakai di tempat yang tepat

Etika. Data organisasi nyata hanya dipakai seizin organisasi dan dianonimkan. Peta kepemilikan cukup memuat nama entitas dan atributnya, bukan isi data. Data pribadi pasien atau karyawan tidak boleh dicantumkan.

Pendalaman teknis (opsional). Anggota berlatar teknis menjalankan contoh di `sources/chapter10/contoh-outbox-postgresql/` seperti pada kotak pen-

dalaman seksi 10.5, lalu menambahkan satu *consumer* sederhana yang menolak *event* dengan `id_event` yang sudah pernah diproses. Percobaan hanya dilakukan pada basis data milik sendiri.

10.9 Ringkasan

Memecah sistem menjadi layanan belum selesai sebelum datanya ikut dipecah. Pola *database per service* menetapkan bahwa data setiap layanan bersifat privat dan hanya diakses lewat API-nya. Hasilnya adalah satu pemilik untuk setiap data, perubahan struktur yang tidak lagi memerlukan rapat lintas tim, dan hak tulis yang tidak tersebar ke banyak akun. Privat tidak berarti server sendiri: skema privat sudah cukup sebagai langkah pertama. Layanan lain memperoleh data lewat API, salinan lokal dari *event*, atau *read model*, dan salinan tidak pernah diubah oleh pembacanya.

Harga dari pemisahan itu adalah hilangnya satu transaksi ACID yang mencakup seluruh proses. *Two-phase commit* menuntut semua peserta tersedia bersamaan dan tidak didukung banyak teknologi maupun layanan eksternal seperti BPJS Kesehatan. Jalan keluarnya adalah *eventual consistency*: selisih data antarlayanan diterima selama jendela yang disepakati, diperlihatkan kepada pengguna sebagai status yang jujur, dan ditutup dengan sengaja. Data yang tidak boleh berselisih sedetik pun diletakkan di satu layanan pemilik.

Saga, yang diperkenalkan Garcia-Molina dan Salem pada 1987, menjaga proses lintas layanan tetap utuh lewat rangkaian transaksi lokal dan transaksi kompensasi. Kompensasi bukan *rollback*: kompensasi adalah transaksi bisnis baru yang kadang tidak sempurna dan tidak selalu berarti batal. Pembagian langkah menjadi dapat dikompensasi, titik penentu, dan dapat diulang membantu menempatkan langkah yang tidak dapat dibatalkan di tempat yang aman. Saga pendek cocok dengan *choreography*; saga panjang dengan banyak kompensasi dan kebutuhan audit, seperti pendaftaran rujukan, lebih cocok dengan *orchestration*.

Saga bergantung pada *event* yang pasti terbit. Pola *transactional outbox* menyimpan *event* di tabel *outbox* dalam transaksi yang sama dengan data bisnis, lalu penerbit meneruskannya ke *broker*. Data dan *event* tidak lagi dapat berselisih, dengan harga *event* yang kadang datang dua kali, sehingga *consumer* harus idempoten. Di sisi baca, CQRS dan *read model* menyajikan laporan dari *event* tanpa membebani layanan yang melayani pasien, dengan batas kesegaran yang ditulis eksplisit. Semua pola ini bertumpu pada identitas pasien induk yang dipegang satu pemilik, dengan persetujuan pasien sebagai batasan desain.

Pesan utama bab ini: setiap data punya satu pemilik, dan setiap selisih antarlayanan punya batas waktu, status yang terlihat, serta jalan pulang bila gagal. Peta

kepemilikan data dan rancangan saga yang disusun pada Aktivitas bab ini menjadi dasar bagi SLO perjalanan pasien di Bab 11 dan rencana resiliensi di Bab 12.

10.10 Latihan

Latihan 1: Satu Tabel, Banyak Penulis

Latihan ini membuktikan bahwa basis data bersama mengaburkan kepemilikan data dan memperbesar dampak setiap perubahan. Pertanyaannya: mengapa tim keuangan dan tim layanan pasien sebaiknya tidak menulis langsung ke tabel basis data yang sama?

1. Pilih satu tabel inti yang ditulis lebih dari satu aplikasi atau modul, dari organisasi pilihan (seizin organisasi) atau dari contoh Tabel 10.12.
2. Kelompokkan kolomnya menurut makna bisnis, misalnya identitas, penjamin, dan data klinis.
3. Catat modul atau tim yang menulis setiap kelompok kolom saat ini.
4. Usulkan satu layanan pemilik untuk setiap kelompok dan cara layanan lain memperolehnya.
5. Isi Tabel 10.13.

Tabel 10.12: Contoh Latihan 1: tabel pasien SIMRS rumah sakit Bekasi dengan 64 kolom

Kelompok kolom	Kolom	Penulis saat ini	Pemilik usulan	Cara pembaca memperoleh
Identitas dan demografi	31	Pendaftaran, sebagian juga keuangan	Layanan Identitas Pasien	API pencarian; *event* `PasienDiperbarui`
Penjamin dan status tagihan	14	Keuangan, sebagian juga pendaftaran	Layanan Penjaminan dan Layanan Tagihan	API saat pendaftaran
Data klinis ringkas	9	Laboratorium dan pendaftaran	Layanan Rekam Medis	*Event* `HasilLabTersedia`
Tanpa pemilik jelas	10	Tidak diketahui	Ditinjau untuk dihapus	Tidak ada
Total	**64**	23 akun dengan hak tulis	4 pemilik	1 insiden 3 jam, 410 pasien manual

Pertanyaan analisis:

1. Pada contoh, berapa persen kolom yang ditulis lebih dari satu modul, dan berapa persen yang tidak punya pemilik? Apa artinya bagi risiko perubahan berikutnya?
2. Insiden pelebaran kolom penjamin menghentikan pendaftaran selama tiga

Tabel 10.13: Lembar isian Latihan 1

Kelompok kolom	Kolom	Penulis saat ini	Pemilik usulan	Cara pembaca memperoleh
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
Total	...	...	...	...

jam. Dengan *database per service*, siapa yang akan terdampak oleh perubahan yang sama, dan apa yang harus dijaga tim pemilik agar pembacanya tidak rusak?

Latihan 2: Kompensasi Saat Verifikasi Gagal

Latihan ini membuktikan bahwa kompensasi adalah keputusan bisnis yang urutannya harus dirancang, bukan sekadar pembatalan teknis. Pertanyaannya: pada saga pendaftaran rujukan, verifikasi kepesertaan BPJS Kesehatan gagal setelah jadwal dokter dikunci. Uraikan langkah kompensasinya.

1. Baca saga pada Gambar 10.3 dan Tabel 10.6.
2. Pelajari contoh skenario lain pada Tabel 10.14, yaitu slot dokter yang tidak tersedia (T3 gagal).
3. Tentukan langkah yang sudah selesai ketika verifikasi kepesertaan (T4) gagal.
4. Susun kompensasi secara mundur, termasuk jalur alternatif yang ditawarkan kepada pasien sebelum jadwal dilepas.
5. Tentukan pihak yang diberi tahu dan batas waktu setiap kompensasi, lalu isi Tabel 10.15. Templat lengkapnya ada di `sources/chapter10/templat-rancangan-saga.md`.

Tabel 10.14: Contoh Latihan 2: kompensasi saat slot dokter tidak tersedia (T3 gagal)

Urutan	Kompensasi atau tindakan	Pihak yang diberi tahu	Batas waktu
1	Tawarkan slot di rumah sakit Medikara lain dalam 7 hari	Pasien	2 jam kerja
2	Bila pasien menolak: C2, tautan rujukan dilepas, identitas tetap	Tidak ada	Segera setelah penolakan
3	C1, rujukan berstatus `DITOLAK` dengan alasan "slot tidak tersedia"	Pasien, klinik mitra	1 jam setelah C2
4	Catat rujukan yang gagal untuk laporan waktu tunggu	Bagian pelayanan	Otomatis lewat *read model*

Pertanyaan analisis:

Tabel 10.15: Lembar isian Latihan 2: kompensasi saat verifikasi kepesertaan gagal (T4 gagal)

Urutan	Kompensasi atau tindakan	Pihak yang diberi tahu	Batas waktu
1	…	…	…
2	…	…	…
3	…	…	…
4	…	…	…

1. Dengan 144 kegagalan verifikasi per bulan dan slot 15 menit, berapa jam praktik dokter spesialis yang dapat dipulihkan bila slot dilepas dalam dua jam kerja, bukan dua hari? Asumsi apa yang membuat angka itu terlalu optimistis?
2. Mengapa urutan kompensasi pada isian berbeda dari contoh, padahal keduanya memakai C2 dan C1 yang sama? Langkah mana yang memerlukan keputusan manusia dan mana yang dapat berjalan otomatis?

Latihan 3: Read Model Pengganti Spreadsheet

Latihan ini membuktikan bahwa *read model* dapat menggantikan penyusunan laporan manual tanpa membebani layanan operasional. Pertanyaannya: laporan manajemen Medikara Health masih disusun dengan *spreadsheet*. Jelaskan cara *read model* menggantikan kebiasaan itu tanpa membebani layanan operasional.

1. Pilih tiga laporan rutin dari organisasi pilihan (seizin organisasi) atau dari contoh Tabel 10.16.
2. Catat cara penyusunan saat ini, sistem yang ditarik datanya, dan waktu kerja yang dibutuhkan.
3. Tentukan *event* yang menjadi sumber setiap laporan, dengan bantuan peta kepemilikan data pada Tabel 10.10.
4. Tetapkan kesegaran yang cukup bagi pengguna laporan, lalu bandingkan dengan kesegaran laporan saat ini.
5. Isi Tabel 10.17.

Pertanyaan analisis:

1. Pada contoh, laporan mana yang paling diuntungkan oleh *read model*: yang paling banyak menghemat jam kerja, atau yang paling banyak memperbaiki kesegaran? Jelaskan dengan angka dari tabel.
2. Selisih jumlah kunjungan sampai 3 persen muncul karena definisi kunjungan berbeda. Mengapa *read model* tidak otomatis menghapus selisih itu, dan keputusan apa yang harus diambil lebih dulu bersama bagian keuangan dan

Tabel 10.16: Contoh Latihan 3: tiga laporan bulanan Medikara Health

Laporan	Cara saat ini	*Event* sumber	Kesegaran cukup
Kunjungan per fasilitas	Ekspor 5 SIMRS dan 18 klinik, digabung manual; 2 staf, 1 hari	`KunjunganSelesai`	1 jam
Pendapatan per penjamin	Ekspor ERP dan SIMRS, dicocokkan manual; 2 staf, 1,5 hari	`TagihanTerbit`, `PembayaranDiterima`	1 hari kerja
Waktu tunggu rujukan	Catatan telepon dan SIMRS; 2 staf, 0,5 hari	`RujukanDiterima`, `JadwalDikunci`	1 jam
Total	48 jam kerja per bulan, sepertiga dari 144 jam untuk 14 laporan; selisih antarlaporan sampai 3 persen	5 jenis *event*	Dari bulanan ke jam atau hari

Tabel 10.17: Lembar isian Latihan 3

Laporan	Cara saat ini	*Event* sumber	Kesegaran cukup
…	…	…	…
…	…	…	…
…	…	…	…
Total	…	…	…

pelayanan?

3. Laporan mana yang tetap tidak boleh dibaca dari *read model*, karena keputusan yang diambil darinya memerlukan data saat itu juga?

Bab 11

Observability, SRE, dan SLO

Tujuan Pembelajaran

- Membedakan *monitoring* dan *observability* serta tiga sinyal telemetri: *log*, metrik, dan *trace*.
- Menetapkan SLI, SLO, dan *error budget* untuk perjalanan pengguna yang kritis.
- Menjelaskan praktik SRE (*Site Reliability Engineering*) yang menyeimbangkan kecepatan perubahan dan keandalan.

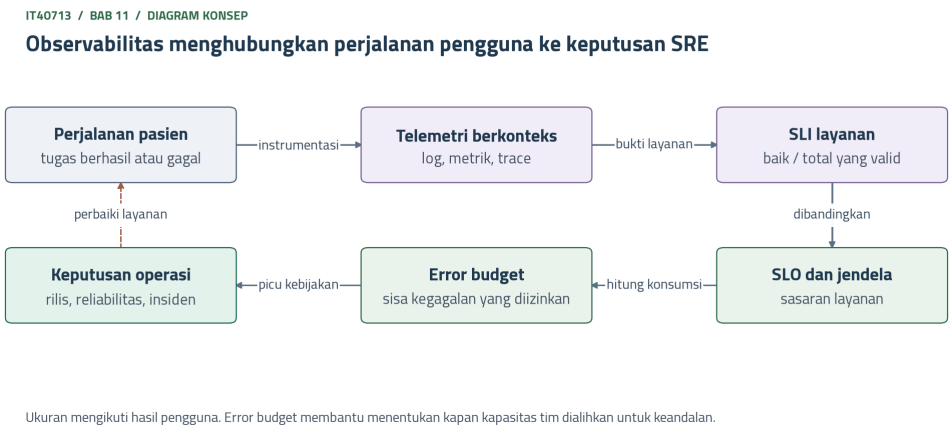
11.1 Pendahuluan

Misalkan tahap pertama modernisasi Medikara Health sudah berjalan. Pemesanan janji temu dari aplikasi Medikara+ kini melewati pintu masuk Application Programming Interface (API) atau *API gateway*, layanan janji temu, layanan jadwal dokter, adaptor ke SIMRS (Sistem Informasi Manajemen Rumah Sakit) tiap rumah sakit, dan layanan notifikasi. Setiap tim memasang dasbor sendiri. Sepanjang September 2026 semua dasbor itu hijau: pemakaian prosesor rata-rata 35 persen, memori 52 persen, dan tidak satu pun server mati. Pada bulan yang sama, pusat layanan pelanggan menerima 1.140 keluhan pasien yang gagal membuat janji temu, terutama untuk poliklinik di rumah sakit Bandung. Sebagian pasien menekan tombol pesan berkali-kali, sehingga petugas pendaftaran menemukan sekitar 2.300 pemesanan ganda yang harus dibatalkan satu per satu. Direksi bertanya dengan wajar: bagaimana sistem yang "sehat" bisa membuat pasien kecewa?

Jawabannya, dasbor itu mengukur kondisi mesin, bukan pengalaman pasien.

Server yang menyala belum tentu melayani dengan benar, dan server yang santai belum tentu cepat. Satu komponen yang lambat cukup untuk menggagalkan perjalanan pasien, walaupun komponen lain tampak sehat. Karena itu, ukuran keandalan perlu berangkat dari pertanyaan pasien: apakah janji temu berhasil dibuat, dan berapa lama harus menunggu? Bab ini membahas cara membangun ukuran semacam itu, cara menetapkan targetnya, dan cara organisasi menyeimbangkan target itu dengan kebutuhan merilis fitur baru.

Gambar 11.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 11.1: Observabilitas menghubungkan perjalanan pengguna ke keputusan SRE

Rujukan utama bab ini adalah buku *Site Reliability Engineering* dari Google [27], pendampingnya *The Site Reliability Workbook* [28], buku *Observability Engineering* karya Majors, Fong-Jones, dan Miranda [89], serta dokumentasi resmi OpenTelemetry [116, 115]. Bab ini bergerak dalam lima langkah: *monitoring* dan *observability*, tiga sinyal telemetri dan OpenTelemetry, SLI (*Service Level Indicator*) dan SLO (*Service Level Objective*), *error budget*, lalu praktik SRE sehari-hari. Hasilnya adalah SLO layanan kritis dan rancangan *observability* untuk tonggak capstone C3.

11.2 Monitoring dan Observability

Monitoring adalah kegiatan mengumpulkan, mengolah, dan menampilkan ukuran yang sudah ditentukan sebelumnya, lalu membunyikan *alert* ketika ukuran itu melewati ambang tertentu [27]. *Monitoring* menjawab pertanyaan yang sudah diketahui sejak awal: apakah server menyala, apakah disk hampir penuh, apakah jumlah kegagalan melonjak. *Observability* adalah kemampuan memahami apa yang sedang terjadi di dalam sistem hanya dari data yang dikeluarkan sistem itu,

termasuk untuk pertanyaan yang belum pernah terpikirkan, tanpa harus menambah kode baru lebih dahulu [89]. *Monitoring* cocok untuk masalah yang sudah dikenal. *Observability* dibutuhkan untuk masalah yang baru pertama kali muncul.

Analoginya seperti pemeriksaan tanda vital di ruang triase. Suhu, tekanan darah, dan nadi diukur cepat untuk setiap pasien. Pemeriksaan ini berguna untuk menangkap masalah yang sudah dikenal, misalnya demam atau tekanan darah tinggi. Ketika pasien datang dengan keluhan yang tidak biasa, dokter tidak cukup melihat tiga angka itu. Dokter menanyakan riwayat, membaca rekam medis, lalu meminta pemeriksaan laboratorium yang sesuai dengan dugaan baru. Tanda vital adalah *monitoring*. Rekam medis yang lengkap dan dapat ditelusuri adalah *observability*.

Dua hal ini sering dianggap bersaing. Pembagiannya begini. *Observability* tidak menggantikan *monitoring*. Dasbor dan *alert* tetap menjaga masalah yang berulang, sedangkan data rinci dipakai ketika dasbor tidak bisa menjelaskan keluhan. Tabel 11.1 merangkum perbedaannya.

Tabel 11.1: Perbedaan *monitoring* dan *observability*

Aspek	***Monitoring***	***Observability***
Pertanyaan	Sudah ditentukan sebelumnya: apakah ukuran X melewati ambang?	Muncul saat itu juga: mengapa pasien Bandung gagal, padahal pasien Bogor tidak?
Bentuk khas	Dasbor tetap dan *alert* berambang	Penelusuran bebas atas telemetri yang rinci, dipotong menurut rumah sakit, versi aplikasi, atau jenis poliklinik
Paling berguna untuk	Masalah yang berulang dan sudah dikenal	Masalah baru di sistem dengan banyak komponen
Contoh di Medikara	Pemakaian prosesor layanan janji temu di atas 80 persen	Menemukan bahwa kegagalan hanya terjadi pada pemesanan ke SIMRS Bandung setelah pukul 07.00
Keterbatasan	Buta terhadap masalah yang belum pernah dibayangkan	Butuh telemetri yang lebih banyak, sehingga biaya dan perlindungan datanya harus dikelola

11.2.1 Menelusuri Gangguan di Sistem Terdistribusi

Pertanyaan kemudian muncul: mengapa kebutuhan ini baru terasa setelah sistem dipecah menjadi banyak layanan? Pada sistem monolitik, satu permintaan diproses di satu program, sehingga catatannya ada di satu tempat. Pada arsitektur *microservices*, satu permintaan melintasi lima atau enam layanan, masing-masing dengan catatan, dasbor, dan tim sendiri. Newman menyebut pergeseran ini sebagai perpindahan dari *monitoring* satu sistem menuju *observability* atas banyak

bagian yang saling bergantung [106]. Setiap tim bisa melaporkan layanannya sehat, sementara perjalanan pasien tetap gagal di sambungan antarlayanan.

Dua contoh dari Medikara Health:

1. Penelusuran 1.140 keluhan pada Pendahuluan butuh 11 hari. Catatan tersebar di lima komponen milik tiga tim internal dan satu vendor. Tim mencocokkan catatan secara manual berdasarkan jam kejadian, padahal jam server Bandung selisih sekitar 2 menit dari server lain. Setelah setiap permintaan diberi nomor penelusuran yang sama di semua layanan, penyebab yang sama ditemukan dalam satu sore.
2. Misalkan adaptor klaim melaporkan status "terkirim" untuk 312 berkas klaim rumah sakit Depok, padahal BPJS Kesehatan menolak berkas itu karena format kode diagnosis. Dasbor adaptor hijau: proses menyala dan antrean kosong. Bagian keuangan baru menemukannya saat tutup buku, 19 hari kemudian, dengan klaim tertunda sekitar Rp1,7 miliar. Dasbor itu tidak bisa menjawab "berapa berkas yang benar-benar diterima BPJS Kesehatan?".

11.2.2 Observability bagi Manajemen dan Auditor

Observability bukan urusan tim teknis saja. Manajemen dan auditor memakainya untuk tiga keperluan. Pertama, bukti tingkat layanan. Direksi dan mitra, misalnya klinik mitra dan perusahaan pelanggan layanan kesehatan karyawan, membutuhkan angka yang dapat diperiksa, bukan sekadar pernyataan "sistem stabil". Kedua, rekonstruksi linimasa insiden. Undang-Undang Pelindungan Data Pribadi (UU PDP) mewajibkan pengendali data menyampaikan pemberitahuan tertulis paling lambat 3 kali 24 jam bila terjadi kegagalan pelindungan data pribadi [120]. Tanpa catatan yang dapat ditelusuri, organisasi tidak dapat menjawab data apa yang terdampak, sejak kapan, dan lewat jalur mana. Ketiga, pemeriksaan kontrol akses. Temuan audit internal Medikara tentang kontrol akses yang lemah hanya bisa ditindaklanjuti bila setiap akses ke rekam medis tercatat beserta pelaku, waktu, dan alasannya.

Ada satu peringatan: telemetri sendiri dapat menjadi jalur kebocoran, seperti insiden lewat vendor pada 2025. Catatan yang memuat nama pasien, nomor induk kependudukan, atau diagnosis dijaga seketat rekam medis. Cara paling aman adalah tidak mencatatnya sama sekali, cukup kode yang tidak dapat dikaitkan langsung dengan orangnya.

11.3 Log, Metrik, Trace, dan OpenTelemetry

Telemetri adalah data yang dikeluarkan sistem tentang dirinya selama berjalan. Dokumentasi OpenTelemetry membedakan tiga sinyal utama [115]. *Log* adalah catatan satu kejadian, biasanya berupa baris teks atau rekaman terstruktur dengan cap waktu. Metrik adalah ukuran berupa angka yang ditangkap saat program berjalan, lalu diringkas per selang waktu, misalnya jumlah permintaan per detik atau persentase permintaan yang gagal. *Trace* adalah jalur satu permintaan saat melewati aplikasi. Satu *trace* tersusun dari beberapa *span*. Setiap *span* mewakili satu langkah kerja, lengkap dengan waktu mulai, durasi, dan induknya. Gagasan *trace* lintas layanan dipopulerkan oleh Dapper, sistem penelusuran internal Google yang makalahnya terbit pada 2010 [129].

Analoginya seperti pengiriman paket. Setiap gudang transit punya buku catatan sendiri tentang paket yang masuk dan keluar: itulah *log*. Kantor pusat punya papan rekap berisi jumlah paket per hari dan persentase yang terlambat: itulah metrik. Pelanggan punya nomor resi yang dipindai di setiap titik transit: itulah *trace*. Bila satu paket terlambat, papan rekap hanya menunjukkan bahwa keterlambatan naik. Buku gudang hanya menceritakan gudang itu sendiri. Nomor resi menunjukkan di gudang mana paket itu tertahan dan berapa lama.

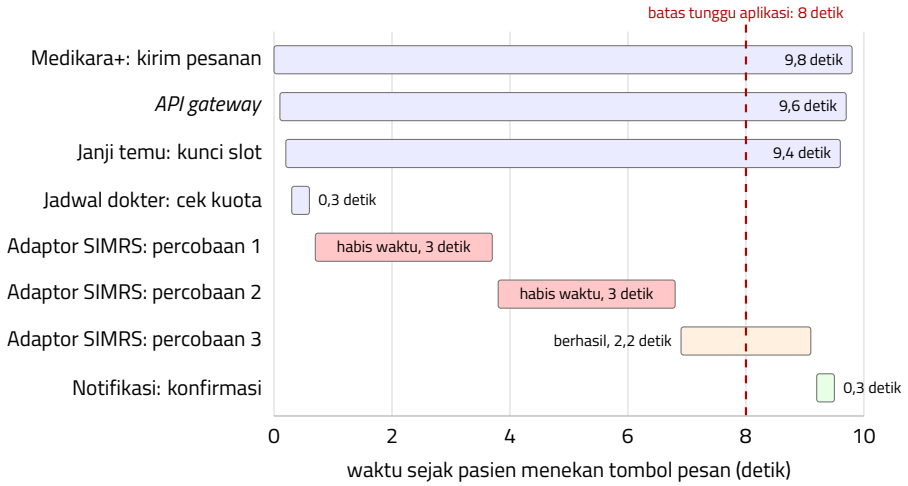
Ketiga sinyal saling melengkapi. Tabel 11.2 merangkum pertanyaan yang dijawab setiap sinyal, contohnya pada pemesanan Medikara+, dan biaya penyimpanannya.

Tabel 11.2: Tiga sinyal telemetri dan kegunaannya

Sinyal	Pertanyaan yang dijawab	Contoh di Medikara+	Biaya simpan
Metrik	Seberapa sering, seberapa banyak, seberapa cepat, secara rata-rata atau per persentil?	Persentase pemesanan gagal per rumah sakit setiap menit	Rendah, karena sudah diringkas
Log	Apa yang terjadi pada satu kejadian tertentu?	"Adaptor SIMRS Bandung: batas waktu 3 detik habis, percobaan ke-2"	Tinggi, tumbuh seiring jumlah kejadian
Trace	Ke mana satu permintaan pergi, dan di langkah mana waktunya habis?	Satu pemesanan yang melintasi enam komponen dalam 9,8 detik	Tinggi, sehingga biasanya diambil sampelnya

Gambar 11.2 memperlihatkan satu *trace* pemesanan dari kasus Pendahuluan. Seluruh pemesanan butuh 9,8 detik. Layanan jadwal dokter hanya butuh 0,3 detik. Waktu terbanyak habis di adaptor SIMRS Bandung, yang menunggu 3 detik dua kali sebelum percobaan ketiga berhasil. Aplikasi di ponsel pasien hanya menunggu 8 detik, lalu menampilkan pesan gagal. Konfirmasi baru terkirim pada detik ke-9,5. Pasien yang melihat pesan gagal menekan tombol pesan lagi, sehingga terjadi pemesanan ganda. Dasbor prosesor tetap hijau, karena menunggu jawaban tidak

memakai prosesor.



Gambar 11.2: Satu *trace* pemesanan janji temu ke rumah sakit Bandung: dua percobaan ke adaptor SIMRS habis waktu, sehingga aplikasi menyerah sebelum konfirmasi tiba

Supaya *span* dari layanan yang berbeda tersambung menjadi satu *trace*, setiap layanan meneruskan nomor penelusuran ke layanan berikutnya. Standar Trace Context dari World Wide Web Consortium mengatur caranya. *Header* `traceparent` membawa versi, nomor *trace*, nomor *span* induk, dan penanda apakah *trace* ini diambil sebagai sampel [82]. Karena formatnya baku, layanan buatan tim yang berbeda, bahkan buatan vendor, tetap dapat menyambung *trace* yang sama.

Dua contoh dari Medikara Health:

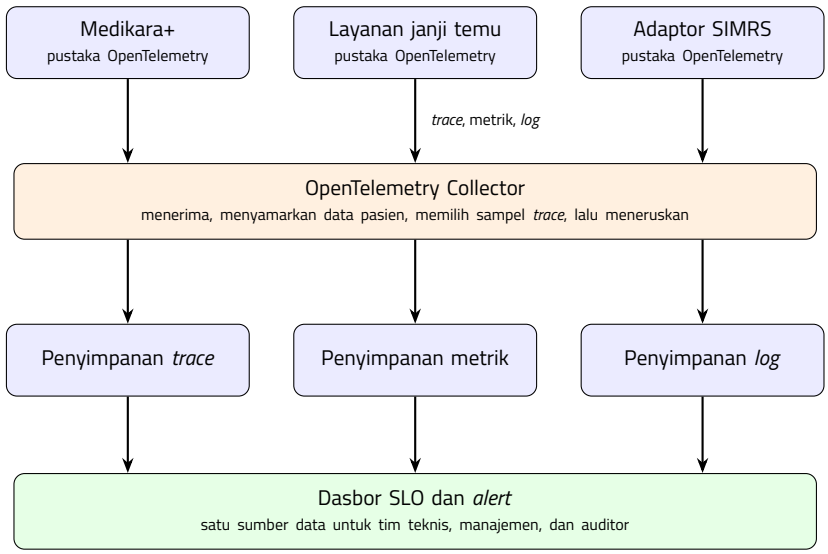
1. *Trace* pada Gambar 11.2 menunjukkan sekitar 86 persen waktu, yaitu 8,4 dari 9,8 detik, habis di adaptor SIMRS Bandung. Misalkan batas waktu adaptor diturunkan menjadi 1 detik dengan satu kali percobaan ulang, dan jadwal dokter Bandung disimpan sementara di layanan jadwal. Waktu pemesanan ke Bandung turun menjadi sekitar 2 detik, jauh di bawah batas tunggu aplikasi.
2. Metrik persentase pemesanan gagal per rumah sakit pada September 2026 menunjukkan Bandung 4,1 persen, sedangkan empat rumah sakit lain di bawah 0,4 persen. Metrik memberi tahu *di mana* masalahnya. *Trace* menjelaskan *mengapa*. *Log* adaptor memberi rincian kesalahan untuk diperbaiki vendor SIMRS Bandung.

11.3.1 OpenTelemetry sebagai Standar Terbuka

OpenTelemetry adalah kerangka dan perangkat sumber terbuka untuk membuat, mengumpulkan, dan mengirim telemetri berupa *trace*, metrik, dan *log*. Proyek ini berada di bawah Cloud Native Computing Foundation (CNCF) dan sengaja dibuat netral terhadap vendor dan alat. OpenTelemetry bukan tempat penyimpanan atau penampil telemetri. Penyimpanan dan dasbor dipilih terpisah, baik produk sumber terbuka maupun komersial [116]. Komponennya meliputi spesifikasi, pustaka untuk berbagai bahasa pemrograman, instrumentasi otomatis untuk kerangka kerja yang populer, dan OpenTelemetry Collector. Collector adalah proses perantara yang menerima telemetri, mengolahnya, lalu meneruskannya ke satu atau beberapa tujuan.

Analoginya seperti colokan listrik standar. Selama peralatan rumah memakai colokan yang sama, pemilik rumah bebas berganti penyedia listrik atau memindahkan peralatan ke rumah lain tanpa memotong kabelnya. Tanpa standar, setiap peralatan terikat pada satu jenis stopkontak, dan pindah rumah berarti membeli ulang peralatan.

Gambar 11.3 memperlihatkan rancangan alur telemetri untuk Medikara. Setiap layanan memakai pustaka OpenTelemetry dan mengirim telemetri ke Collector. Collector menyamarkan data pasien, memilih sampel *trace*, lalu meneruskan setiap sinyal ke penyimpanannya. Dasbor SLO dan *alert* dibangun di atas ketiga penyimpanan itu.



Gambar 11.3: Rancangan alur telemetri Medikara Health dengan OpenTelemetry: layanan cukup diinstrumentasi sekali, sedangkan penyimpanannya dapat diganti

Keputusan memakai standar terbuka punya dua konsekuensi. Pertama, Medikara tidak terkunci pada satu vendor pemantauan. Hal ini penting karena ketergantungan pada vendor sudah menjadi risiko strategis bagi Medikara+ (Bab 6). Bila suatu hari penyimpanan telemetri diganti, kode layanan tidak perlu diubah. Cukup konfigurasi Collector. Kedua, Collector menjadi satu titik kendali untuk kepatuhan. Aturan penyamaran data pasien ditulis sekali di Collector, bukan diulang di setiap layanan dengan risiko terlewat.

11.3.2 Biaya Telemetri dan Cara Mengendalikannya

Telemetri tidak gratis. Volumenya tumbuh seiring jumlah layanan dan jumlah permintaan. *Log* biasanya menjadi bagian terbesar, disusul *trace*. Metrik relatif murah selama labelnya terkendali. Empat pengungkit dipakai untuk mengendalikan biaya. Pertama, tingkat *log*: catatan untuk mencari kesalahan program (*debug*) dimatikan di lingkungan produksi. Kedua, pengambilan sampel *trace*: semua *trace* yang gagal atau lambat disimpan, sedangkan *trace* normal cukup diambil sebagian kecil. Ketiga, masa simpan bertingkat: data terbaru disimpan di tempat yang cepat dicari, data lama dipindah ke arsip murah untuk keperluan audit. Keempat, label metrik dibatasi: label seperti nomor rekam medis membuat jumlah deret data meledak.

Dua contoh dari Medikara Health:

1. Misalkan seluruh layanan baru menghasilkan sekitar 60 *gigabyte* (GB) *log* per hari, dengan biaya masuk dan simpan 30 hari sekitar Rp7.500 per GB. Volumenya sekitar 1.800 GB per bulan, dengan biaya Rp13,5 juta. Setelah *debug* dimatikan di produksi, *trace* normal diambil 10 persen, dan catatan lama dipindah ke arsip, volumenya turun menjadi 22 GB per hari. Biayanya menjadi sekitar Rp4,95 juta per bulan.
2. Satu tim memasang nomor rekam medis sebagai label metrik latensi agar keluhan per pasien mudah dicari. Jumlah deret data naik dari sekitar 300 menjadi lebih dari 400.000. Biaya penyimpanan metrik melonjak, dan nomor rekam medis ikut tersebar ke sistem pemantauan. Label itu dicabut. Penelusuran per pasien dipindah ke *trace* yang sudah disamarkan oleh Collector.

Pendalaman Teknis (opsional): Satu Trace Lintas Dua Layanan

Listing 11.1 memperlihatkan inti contoh instrumentasi dengan pustaka OpenTelemetry untuk Python. Dua layanan, yaitu janji temu dan jadwal dokter, disimulasikan dalam satu proses. Setiap layanan punya `TracerProvider` sendiri dengan nama layanan yang berbeda. Yang dikirim antarlayanan hanya kamus *header*, persis seperti *header* permintaan sungguhan. Fungsi `inject` menulis `traceparent` ke *header* keluar, dan fungsi

`extract` membacanya di layanan penerima.

```
def layanan_jadwal_dokter(header_masuk, kode_poli):
  """Menjawab cek kuota; extract menyambung trace dari traceparent."""
  konteks_induk = extract(header_masuk)
  with TRACER_JADWAL_DOKTER.start_as_current_span(
    "cek kuota poli",
    context=konteks_induk,
    kind=trace.SpanKind.SERVER,
  ) as span:
    span.set_attribute("medikara.kode_poli", kode_poli)
    time.sleep(LAMA_CEK_KUOTA_DETIK)
    return True


def layanan_janji_temu(kode_poli):
  """Memesan slot; inject menulis traceparent ke header keluar."""
  with TRACER_JANJI_TEMU.start_as_current_span(
    "pesan janji temu", kind=trace.SpanKind.SERVER
  ):
    with TRACER_JANJI_TEMU.start_as_current_span(
      "panggil jadwal dokter", kind=trace.SpanKind.CLIENT
    ):
      header_keluar = {}
      inject(header_keluar)
      print("Header yang dikirim:", header_keluar)
      ada_kuota = layanan_jadwal_dokter(header_keluar, kode_poli)
    time.sleep(LAMA_KUNCI_SLOT_DETIK)
    return ada_kuota
```

Listing 11.1: Propagasi *trace* antara dua layanan, tersedia di `sources/chapter11/contoh-opentelemetry/dua_layanan.py`

Skrip `dua_layanan.py` mencetak isi *header* dan tiga *span*. Ketiganya punya `trace_id` yang sama, dan `parent_id` dari "cek kuota poli" sama dengan `span_id` dari "panggil jadwal dokter". Bila baris `inject` dihapus, layanan jadwal dokter memulai `trace_id` baru, sehingga perjalanan pasien terputus menjadi dua *trace* yang tidak saling mengenal.

11.4 SLI, SLO, dan SLA

SLI adalah ukuran kuantitatif yang didefinisikan dengan cermat untuk satu aspek layanan, misalnya latensi, tingkat kegagalan, atau ketersediaan. SLO adalah nilai target untuk SLI itu selama jendela waktu tertentu. SLA (*Service Level Agreement*) adalah perjanjian dengan pengguna atau pelanggan yang memuat konsekuensi bila SLO di dalamnya meleset, misalnya potongan biaya [27]. *The Site Reliability Workbook* menganjurkan SLI ditulis sebagai rasio: jumlah peristiwa yang baik dibagi jumlah seluruh peristiwa [28]. Dengan bentuk ini, setiap SLI bernilai 0 sampai 100 persen, dan semua SLI dapat dibaca dengan cara yang sama. Contohnya: "99,5 persen upaya pemesanan dalam 30 hari terakhir mendapat konfirmasi dalam 5

detik".

Analoginya seperti restoran cepat saji yang menjanjikan pesanan siap dalam 10 menit. Persentase pesanan yang benar-benar siap dalam 10 menit adalah SLI. Target dapur, misalnya 95 persen pesanan setiap minggu siap dalam 10 menit, adalah SLO. Spanduk di depan kasir yang berbunyi "lebih dari 15 menit, minuman gratis" adalah SLA. Dapur menjaga SLO yang lebih ketat daripada SLA, supaya minuman gratis jarang dibagikan.

Dua hal pada istilah ini sering tertukar: SLO dan SLA. Pembagiannya begini. SLO adalah target internal yang ditetapkan organisasi untuk mengambil keputusan teknis dan bisnis. SLA adalah janji kepada pihak luar yang mengandung konsekuensi, sehingga disusun bersama bagian hukum dan bisnis. SLA biasanya lebih longgar daripada SLO, agar organisasi punya ruang memperbaiki diri sebelum denda berlaku. Bila sebuah target tidak punya konsekuensi apa pun ketika meleset, target itu adalah SLO, walaupun di dokumen disebut SLA. Tabel 11.3 merangkum ketiganya.

Tabel 11.3: SLI, SLO, dan SLA dengan contoh dari layanan pemesanan Medikara+

Istilah	Pertanyaan yang dijawab	Penetap	Contoh Medikara
SLI	Seberapa baik layanan saat ini, diukur dari sisi pengguna?	Tim teknis bersama pemilik layanan	Persentase upaya pemesanan yang terkonfirmasi dalam 5 detik
SLO	Seberapa baik layanan seharusnya dalam satu jendela waktu?	Pemilik layanan, disetujui manajemen	99,5 persen dalam 30 hari bergulir
SLA	Apa akibatnya bila janji kepada pihak luar meleset?	Bisnis dan bagian hukum	Misalkan perusahaan mitra mendapat potongan biaya bila janji temu karyawannya di bawah 99 persen dalam sebulan

11.4.1 Memilih SLI dari Perjalanan Pengguna

Perjalanan pengguna kritis adalah urutan langkah yang dilalui pengguna untuk mencapai tujuan yang penting bagi dirinya dan bagi organisasi, misalnya membuat janji temu atau melihat hasil laboratorium. SLI dipilih dari perjalanan ini, bukan dari daftar komponen. Ukurannya diambil sedekat mungkin dengan pengguna. *Workbook* mengelompokkan SLI menurut jenis layanannya [28]. Layanan yang menjawab permintaan diukur dari ketersediaan, latensi, dan kualitas jawabannya. Alur pengolahan data diukur dari kesegaran, kebenaran, dan cakupan datanya. Layanan penyimpanan diukur dari daya tahan datanya. Tabel 11.4 memberi contoh untuk Medikara.

Titik pengukuran juga perlu dipilih. Catatan server mudah diambil, tetapi buta

Tabel 11.4: Jenis SLI menurut jenis layanan, dengan contoh Medikara Health

Jenis layanan	SLI	Contoh: peristiwa baik dibagi seluruh peristiwa
Menjawab permintaan	Ketersediaan	Permintaan pemesanan yang dijawab tanpa kegagalan, dibagi seluruh permintaan pemesanan yang sah
Menjawab permintaan	Latensi	Pemesanan yang terkonfirmasi dalam 5 detik, dibagi seluruh pemesanan yang sah
Alur pengolahan data	Kesegaran	Hasil laboratorium yang tampil di Medikara+ dalam 30 menit setelah divalidasi, dibagi seluruh hasil yang divalidasi
Alur pengolahan data	Kebenaran	Berkas klaim yang diterima BPJS Kesehatan tanpa penolakan format, dibagi seluruh berkas klaim yang dikirim
Penyimpanan	Daya tahan	Dokumen rekam medis yang dapat dibaca kembali utuh, dibagi seluruh dokumen yang disimpan

terhadap permintaan yang tidak pernah sampai ke server. Pengukuran di *API gateway* sudah lebih dekat dengan pasien. Telemetri dari aplikasi di ponsel paling dekat dengan pengalaman pasien, tetapi ikut terpengaruh jaringan seluler pasien. Pemeriksaan sintetis, yaitu permintaan buatan yang dikirim berkala dari luar, menangkap gangguan pada jam sepi. Medikara memadukan dua titik ukur.

Dua contoh dari Medikara Health:

1. Selama September 2026 pemakaian prosesor layanan janji temu rata-rata hanya 35 persen. SLI yang berangkat dari pengalaman pasien menunjukkan hal lain: hanya 98,7 persen upaya pemesanan ke rumah sakit Bandung yang mendapat konfirmasi dalam 8 detik.
2. Halaman hasil laboratorium di Medikara+ dapat dibuka 99,97 persen dari waktu. Namun, hanya 91 persen hasil yang tampil dalam 30 menit setelah divalidasi dokter, karena sinkronisasi dari sistem laboratorium berjalan sekali setiap jam. Pasien tetap datang ke loket untuk menanyakan hasil. SLI ketersediaan halaman tidak menangkap masalah ini, sedangkan SLI kesegaran menangkapnya.

11.4.2 Menghitung Waktu Tidak Tersedia yang Diizinkan

Target SLO dapat dibaca sebagai waktu tidak tersedia yang masih diizinkan. Jendela 30 hari berisi 43.200 menit, sedangkan satu tahun berisi 525.600 menit. Target 99,9 persen berarti 0,1 persen dari waktu itu boleh hilang: 43,2 menit per 30 hari. Tabel 11.5 memperlihatkan perhitungan untuk beberapa target yang umum. Setiap tambahan satu angka sembilan memangkas waktu yang diizinkan menjadi sepersepuluhnya.

Perhitungan berbasis waktu mudah dipahami, tetapi kurang adil untuk layanan

Tabel 11.5: Waktu tidak tersedia yang diizinkan untuk beberapa target ketersediaan

Target	*Error budget*	Per 30 hari	Per tahun (365 hari)
99 persen	1 persen	432 menit (7 jam 12 menit)	87 jam 36 menit (sekitar 3,6 hari)
99,9 persen	0,1 persen	43,2 menit	8 jam 45,6 menit
99,95 persen	0,05 persen	21,6 menit	4 jam 22,8 menit
99,99 persen	0,01 persen	4,32 menit (sekitar 4 menit 19 detik)	52,6 menit
99,999 persen	0,001 persen	sekitar 26 detik	sekitar 5 menit 15 detik

yang bebannya naik turun. Gangguan 10 menit pada pukul 07.00, saat pasien berebut kuota poliklinik, jauh lebih merugikan daripada gangguan 10 menit pada pukul 02.00. Karena itu, SLI berbasis permintaan lebih dianjurkan: yang dihitung adalah jumlah upaya yang gagal, bukan jumlah menit. Misalkan dalam 30 hari ada sekitar 28.000 upaya pemesanan lewat Medikara+ (Bab 3). Target 99,5 persen berarti paling banyak 140 upaya boleh gagal.

11.4.3 Mengapa Target 100 Persen Hampir Selalu Keliru

Pertanyaan kemudian muncul: mengapa tidak menargetkan 100 persen saja, karena pasien jelas ingin layanan yang selalu berhasil? Buku SRE Google menyebut 100 persen sebagai target yang keliru untuk hampir semua layanan [27]. Ada empat alasannya.

Pertama, pasien tidak dapat membedakan. Ponsel, jaringan seluler, dan jaringan rumah pasien jauh kurang andal daripada 99,99 persen. Selisih antara 99,99 dan 100 persen hilang di antara gangguan yang berasal dari luar Medikara. Kedua, biaya naik jauh lebih cepat daripada manfaatnya. Setiap tambahan angka sembilan menuntut cadangan di lebih banyak lokasi, petugas jaga sepanjang waktu, dan pengujian yang lebih mahal, sementara manfaat bagi pasien makin kecil. Ketiga, ketergantungan membatasi. Layanan janji temu bergantung pada adaptor SIMRS yang hanya ditargetkan 99,5 persen. Tanpa rancangan yang tetap berjalan ketika adaptor gagal, misalnya antrean pemesanan, janji temu tidak mungkin lebih andal daripada adaptor itu. Keempat, target 100 persen berarti *error budget* nol. Setiap perubahan membawa risiko, sehingga tidak ada perubahan yang boleh dirilis. Organisasi yang strateginya menuntut layanan baru justru membekukan dirinya sendiri.

Kasus Medikara Health: SLI untuk Tiga Perjalanan Pasien

Tim transformasi memilih tiga perjalanan yang paling sering dikeluhkan dan paling besar dampaknya: membuat janji temu lewat Medikara+, mengambil hasil laboratorium, dan mengirim klaim BPJS Kesehatan. Hasilnya ada di Tabel 11.6. Perwakilan pasien, kepala pendaftaran, kepala laboratorium, dan kepala bagian klaim ikut menentukan arti "layanan yang baik". Tim teknis menerjemahkannya menjadi rasio yang dapat diukur.

Perdebatan paling panjang terjadi pada target janji temu. Direktur pelayanan ingin 99,9 persen. Perhitungan menunjukkan target itu menuntut layanan berjalan di dua zona *cloud* dan petugas jaga 24 jam, dengan biaya kira-kira dua kali lipat. Pasien yang gagal memesan masih dapat mencoba lagi atau menelepon pusat layanan. Kesepakatannya 99,5 persen, ditinjau ulang setelah tiga bulan data terkumpul.

Keputusan ini punya dua konsekuensi. Pertama, SLI janji temu diukur di *API gateway* dan dari telemetri aplikasi, bukan dari server layanan janji temu. Pemesanan yang tidak pernah sampai ke server tetap terhitung gagal. Kedua, SLI klaim memakai jawaban BPJS Kesehatan sebagai bukti. Status "terkirim" di adaptor tidak lagi dianggap berhasil.

Tabel 11.6: SLI dan SLO untuk tiga perjalanan pasien Medikara Health, jendela 30 hari bergulir

Perjalanan	SLI: peristiwa baik dibagi seluruh peristiwa	Titik ukur	SLO	*Error budget*
Membuat janji temu	Upaya pemesanan yang terkonfirmasi dalam 5 detik, dibagi seluruh upaya yang sah	*API gateway* dan aplikasi	99,5%	140 dari sekitar 28.000 upaya
Mengambil hasil laboratorium	Hasil yang tampil di Medikara+ dalam 30 menit setelah divalidasi, dibagi seluruh hasil yang divalidasi	Cap waktu sistem laboratorium dan aplikasi	99,0%	620 dari sekitar 62.000 hasil
Mengirim klaim BPJS Kesehatan	Berkas yang diterima tanpa penolakan format dalam 2 hari kerja setelah pasien pulang, dibagi seluruh berkas	Adaptor klaim dan jawaban BPJS Kesehatan	98,0%	820 dari sekitar 41.000 berkas

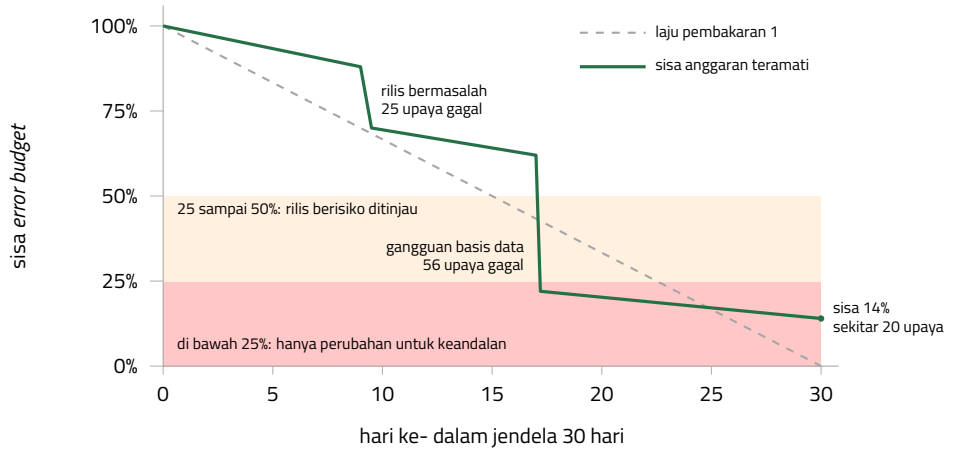
11.5 Error Budget dan Kebijakannya

Error budget adalah selisih antara 100 persen dan target SLO. Selisih itu adalah jumlah kegagalan yang boleh terjadi dalam satu jendela tanpa melanggar SLO [27]. Untuk pemesanan Medikara+, SLO 99,5 persen dengan sekitar 28.000 upaya per

30 hari menghasilkan anggaran 140 upaya gagal. Anggaran ini dipakai oleh insiden, rilis yang bermasalah, dan pemeliharaan yang mengganggu pasien. Selama anggaran masih tersisa, tim boleh merilis perubahan dan mengambil risiko yang terukur. Ketika anggaran menipis atau habis, prioritas berpindah ke keandalan.

Analoginya seperti kuota data seluler bulanan. Pemilik paket 30 GB bebas menonton video selama kuotanya masih ada. Bila pada tanggal 12 kuota tinggal 3 GB, pemilik ponsel mengurangi menonton video dan memakai jaringan kantor. Kuota tidak dimaksudkan untuk disisakan. Kuota yang selalu bersisa banyak di akhir bulan berarti paketnya terlalu besar untuk kebutuhan. Hal yang sama berlaku untuk *error budget*. Anggaran yang tidak pernah terpakai berarti tim terlalu berhati-hati, sehingga perubahan bisa dirilis lebih cepat, atau targetnya terlalu mudah dicapai.

Kecepatan anggaran terpakai disebut laju pembakaran (*burn rate*). Laju pembakaran membandingkan laju kegagalan yang teramati dengan laju yang tepat menghabiskan anggaran di akhir jendela [28]. Laju pembakaran 1 berarti anggaran habis tepat di hari ke-30. Laju pembakaran 2 berarti anggaran habis di hari ke-15. Gambar 11.4 memperlihatkan sisa anggaran pemesanan Medikara+ dalam satu jendela 30 hari.



Gambar 11.4: Sisa *error budget* pemesanan Medikara+ dalam satu jendela 30 hari: dua kejadian memakan 58 persen anggaran, dan kebijakan berlaku setelah sisa di bawah 25 persen

Dua contoh dari Medikara Health, keduanya terlihat pada Gambar 11.4. Misalkan jendelanya November 2026, setelah adaptor SIMRS Bandung diperbaiki.

1. Pada hari ke-9 tim merilis fitur pengingat jadwal yang ternyata mengganggu proses kunci slot. Masalahnya baru disadari beberapa jam kemudian. Dalam setengah hari, 25 upaya gagal, atau 18 persen anggaran. Setelah disadari, rilis dikembalikan ke versi sebelumnya dalam 15 menit.

2. Pada hari ke-17 basis data layanan janji temu penuh dan menolak penulisan selama sekitar 30 menit pada jam sibuk pagi. Hampir semua pemesanan pada selang itu gagal: 56 upaya, atau 40 persen anggaran. Sisa anggaran turun menjadi 22 persen, di bawah ambang 25 persen. Sejak saat itu hanya perubahan yang memperbaiki keandalan yang boleh dirilis. Di akhir jendela anggaran masih tersisa 14 persen, atau sekitar 20 upaya, sehingga SLO terpenuhi.

Pendalaman Teknis (opsional): Menghitung Laju Pembakaran

Laju pembakaran dihitung dengan membagi proporsi kegagalan yang teramati dengan proporsi kegagalan yang diizinkan:

$$\text{laju pembakaran} = \frac{\text{upaya gagal} \,/\, \text{seluruh upaya}}{1 - \text{SLO}}$$

Pada jam biasa, Medikara+ menerima rata-rata sekitar 39 upaya pemesanan, yaitu 28.000 upaya dibagi 720 jam. Misalkan 7,2 persen di antaranya gagal, atau sekitar 3 upaya. Dengan SLO 99,5 persen, proporsi yang diizinkan 0,5 persen, sehingga laju pembakarannya 14,4. Satu jam dengan laju ini memakan 14,4 dibagi 720, atau 2 persen anggaran. Bila laju ini berlanjut, anggaran 30 hari habis dalam 720 dibagi 14,4, atau 50 jam. Skrip `sources/chapter11/hitung_error_budget.py` menghitung angka ini untuk target dan volume yang berbeda.

11.5.1 Kebijakan Error Budget

Kebijakan *error budget* adalah dokumen yang disepakati sebelum ada masalah: apa yang dilakukan ketika anggaran menipis atau habis, dan siapa yang memutuskan. Contoh kebijakan di *Workbook* memakai jendela empat minggu bergulir. Bila anggaran terlampaui, semua perubahan dan rilis dihentikan, kecuali perbaikan berprioritas tertinggi dan perbaikan keamanan, sampai layanan kembali memenuhi SLO. Insiden yang memakan lebih dari 20 persen anggaran dalam empat minggu wajib dianalisis dalam *postmortem* dengan minimal satu tindak lanjut berprioritas tertinggi. Perselisihan dibawa ke pimpinan teknologi tertinggi [28]. Medikara mengadaptasinya menjadi empat zona pada Tabel 11.7.

Dua aturan berlaku di semua zona. Pertama, insiden yang memakan lebih dari 20 persen anggaran wajib ditulis *postmortem*-nya paling lambat lima hari kerja kemudian. Kedua, gangguan di luar kendali tim, misalnya gangguan nasional layanan BPJS Kesehatan, tetap dicatat tetapi tidak menghentikan rilis.

Tabel 11.7: Zona kebijakan *error budget* Medikara Health, diadaptasi dari contoh kebijakan di *Workbook*

Sisa anggaran	Status	Tindakan	Yang memutuskan
Di atas 50 persen	Normal	Rilis seperti biasa, termasuk eksperimen yang terukur	Tim pemilik layanan
25 sampai 50 persen	Waspada	Rilis berisiko besar ditinjau dulu; pengujian dan rilis bertahap diperketat	Pemilik layanan dan pemimpin teknis
Di bawah 25 persen	Fokus keandalan	Hanya perubahan untuk keandalan, keamanan, atau keselamatan pasien	Pemimpin teknis
Habis	Beku	Semua rilis fitur dihentikan sampai SLI kembali di atas SLO; perselisihan dibawa ke direktur teknologi informasi (TI)	Direktur TI

11.5.2 Error Budget sebagai Alat Negosiasi

Tim produk ingin fitur baru cepat dirilis. Tim operasi ingin sistem tidak diganggu. Tanpa ukuran bersama, perdebatan keduanya berakhir pada siapa yang paling keras suaranya. Buku SRE Google memperkenalkan *error budget* untuk menyelesaikan tarik-menarik ini [27]. Keputusan merilis tidak lagi bergantung pada pendapat, tetapi pada sisa anggaran yang dapat dilihat semua pihak. Keputusan ini punya dua konsekuensi. Pertama, tim produk ikut menjaga keandalan, karena rilis yang ceroboh menghabiskan anggaran untuk rilis berikutnya. Kedua, tim operasi tidak boleh menolak rilis selama anggaran masih tersedia, karena keandalan yang melebihi target tidak memberi manfaat tambahan bagi pasien.

Dua contoh dari Medikara Health:

1. Tim produk ingin merilis fitur pembayaran langsung di aplikasi pada hari ke-18 jendela November. Sisa anggaran saat itu tinggal 22 persen. Rilis digeser ke awal jendela Desember, dan waktu hampir dua minggu itu dipakai untuk memasang pemantauan kapasitas basis data. Kedua tim menerima keputusan itu karena angkanya jelas.
2. Layanan hasil laboratorium menyisakan lebih dari 80 persen anggaran selama tiga bulan berturut-turut. Tim menaikkan frekuensi rilis dari sebulan sekali menjadi seminggu sekali, tanpa meminta izin tambahan, karena anggaran membuktikan risikonya masih terkendali.

11.6 Praktik SRE: Toil, Alerting, Insiden, dan Postmortem

SRE adalah disiplin yang lahir di Google. Intinya adalah menerapkan cara kerja rekayasa perangkat lunak pada pekerjaan operasi. Treynor Sloss, pendirinya,

menggambarkan SRE sebagai hasil ketika seorang insinyur perangkat lunak diminta merancang tim operasi [27]. SLO dan *error budget* menjadi kontrak bersama antara tim pengembang dan tim operasi. *Workbook* menempatkan SRE sebagai salah satu cara konkret menjalankan prinsip DevOps [28]. Seksi ini membahas empat praktik SRE sehari-hari: mengendalikan *toil*, membuat *alert* yang bermakna, menangani insiden, dan menulis *postmortem*.

11.6.1 Toil dan Batasnya

Toil adalah pekerjaan operasi yang manual, berulang, dapat diotomasi, bersifat taktis, tidak meninggalkan nilai jangka panjang, dan bertambah seiring pertumbuhan layanan [27]. Tidak semua pekerjaan operasi adalah *toil*. Rapat perencanaan dan merancang sistem baru bukan *toil*. Menyalakan ulang server yang sama setiap pagi adalah *toil*. Google menetapkan batas: pekerjaan operasi, termasuk *toil*, dijaga di bawah 50 persen waktu setiap insinyur SRE. Sisa waktunya dipakai untuk pekerjaan rekayasa yang mengurangi *toil* berikutnya [27].

Analoginya seperti petugas administrasi yang setiap pagi menyalin angka dari lima laporan rumah sakit ke satu lembar kerja. Pekerjaan itu perlu, tetapi tidak membuat laporan besok lebih mudah. Bila jumlah rumah sakit bertambah, waktu menyalinnya ikut bertambah. Satu minggu yang dipakai untuk membuat laporan otomatis membebaskan petugas itu dari pekerjaan yang sama selamanya.

Dua contoh dari Medikara Health:

1. Petugas TI di setiap rumah sakit menyalakan ulang adaptor SIMRS setiap pagi karena pemakaian memorinya terus naik. Satu kali penyalaan ulang butuh sekitar 20 menit. Untuk 5 rumah sakit, totalnya 100 menit per hari, atau sekitar 50 jam per bulan. Pemeriksaan kesehatan otomatis yang menyalakan ulang adaptor saat dibutuhkan (Bab 8) menghapus pekerjaan ini, sambil kebocoran memorinya diperbaiki vendor.
2. Laporan ketersediaan bulanan untuk direksi disusun dari tangkapan layar dasbor oleh tiga orang selama dua hari, atau enam hari kerja per bulan. Laporan SLO yang dibuat otomatis dari penyimpanan metrik menggantikannya.

11.6.2 Alerting yang Bermakna

Alert yang terlalu sering membuat petugas jaga berhenti memperhatikannya. Buku SRE Google memberi tiga syarat untuk *alert* yang memanggil manusia: mendesak, dapat ditindaklanjuti, dan membutuhkan pertimbangan manusia [27]. Buku yang sama menganjurkan *alert* pada gejala yang dirasakan pengguna, bukan pada penyebab di dalam mesin. Untuk dasbor, buku itu menyebut empat sinyal emas (*golden signals*): latensi, lalu lintas, kegagalan, dan saturasi. Pemakaian prosesor

85 persen adalah penyebab yang mungkin. Pasien gagal memesan adalah gejala.

Analoginya seperti alarm asap di dapur. Alarm yang berbunyi setiap kali ada yang menggoreng ikan akhirnya dicabut baterainya oleh penghuni rumah. Saat kebakaran sungguhan terjadi, alarm itu diam. Alarm yang baik berbunyi saat ada bahaya bagi penghuni, bukan setiap ada asap.

Alert berbasis SLO memakai laju pembakaran. *Workbook* menganjurkan *alert* dengan dua jendela: jendela panjang memastikan anggaran yang terpakai cukup berarti, dan jendela pendek, sekitar seperdua belas jendela panjang, memastikan pembakaran masih berlangsung [28]. Tabel 11.8 memuat parameter yang dianjurkan untuk jendela SLO 30 hari. Medikara memakai parameter ini apa adanya.

Tabel 11.8: *Alert* laju pembakaran untuk jendela SLO 30 hari, menurut *Workbook*

Anggaran terpakai	Jendela panjang	Jendela pendek	Laju pembakaran	Pemberitahuan
2 persen	1 jam	5 menit	14,4	Memanggil petugas jaga saat itu juga
5 persen	6 jam	30 menit	6	Memanggil petugas jaga saat itu juga
10 persen	3 hari	6 jam	1	Tiket untuk dikerjakan pada jam kerja

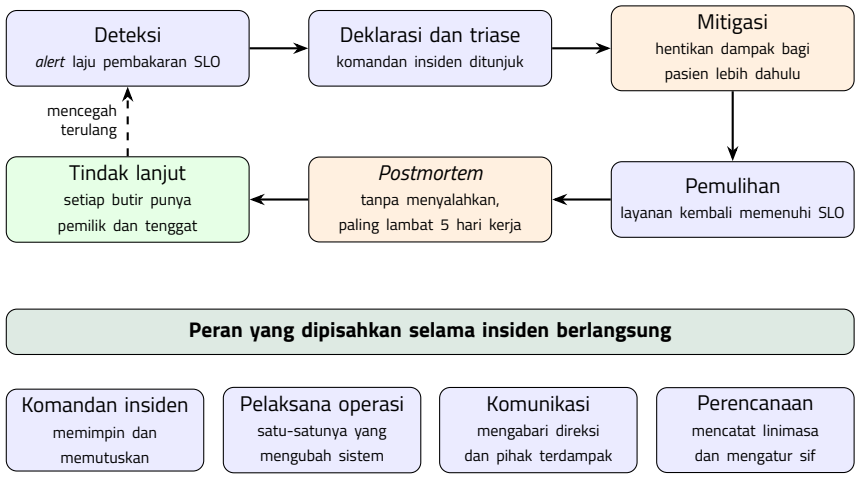
Dua contoh dari Medikara Health:

1. Sistem pemantauan lama mengirim sekitar 300 *alert* per minggu, sebagian besar tentang pemakaian prosesor dan disk. Setelah diganti dengan *alert* laju pembakaran untuk tiga perjalanan pada Tabel 11.6, petugas jaga menerima rata-rata empat panggilan per bulan, dan semuanya membutuhkan tindakan.
2. Pada gangguan basis data hari ke-17 di Gambar 11.4, hampir semua pemesanan gagal, sehingga *alert* laju pembakaran 14,4 berbunyi sekitar lima menit setelah gangguan mulai. Sebelumnya, keluhan pertama dari pusat layanan pelanggan biasanya baru masuk sekitar 25 menit setelah gangguan mulai.

11.6.3 Penanganan Insiden

Insiden adalah kejadian yang mengganggu layanan dan membutuhkan tanggapan terkoordinasi. Buku SRE Google memisahkan empat peran selama insiden berlangsung [27]. Komandan insiden memimpin dan memutuskan. Pelaksana operasi adalah satu-satunya pihak yang mengubah sistem. Penanggung jawab komunikasi mengabari pihak yang terdampak. Penanggung jawab perencanaan mencatat linimasa, menyiapkan pergantian sif, dan mengurus kebutuhan jangka

lebih panjang. Prinsip lainnya: satu tempat komando yang diketahui semua orang, satu dokumen insiden yang terus diperbarui, dan serah terima yang tegas bila komandan berganti. Gambar 11.5 memperlihatkan alur dan perannya. Urutannya penting: dampak bagi pasien dihentikan lebih dahulu, penyebab dicari belakangan.



Gambar 11.5: Alur penanganan insiden dari deteksi sampai tindak lanjut, dengan empat peran yang dipisahkan selama insiden berlangsung

Analoginya seperti kode biru di rumah sakit. Ketika kode biru diumumkan, satu orang memimpin, satu orang melakukan kompresi dada, satu orang menyiapkan obat, dan satu orang mencatat waktu setiap tindakan. Tidak semua orang mengerjakan hal yang sama. Tim yang tidak membagi peran justru saling menghalangi di samping tempat tidur pasien.

11.6.4 Postmortem tanpa Menyalahkan

Postmortem adalah catatan tertulis tentang sebuah insiden: dampaknya, tindakan yang diambil, penyebab utama dan faktor pendukungnya, serta tindak lanjut untuk mencegah insiden serupa. *Postmortem* tanpa menyalahkan berfokus pada sistem dan proses, bukan pada individu. Dasarnya adalah anggapan bahwa setiap orang bertindak dengan niat baik berdasarkan informasi yang dimilikinya saat itu [27]. Bila orang dihukum karena melaporkan kesalahan, kesalahan berikutnya disembunyikan, dan organisasi kehilangan kesempatan belajar.

Analoginya seperti penyelidikan kecelakaan penerbangan. Tujuan penyelidikan adalah mencegah kecelakaan serupa, bukan mencari pihak yang dihukum. Laporannya memuat urutan kejadian, faktor penyebab, dan rekomendasi bagi maskapai, pabrik pesawat, dan pengatur. Karena tujuannya jelas, saksi lebih terbuka menceritakan apa yang terjadi.

Postmortem Medikara memuat tujuh bagian: ringkasan, dampak bagi pasien dan anggaran yang terpakai, linimasa, penyebab utama dan faktor pendukung, hal yang berjalan baik, hal yang berjalan buruk, serta tindak lanjut dengan pemilik dan tenggat. Kasus berikut menerapkannya.

Kasus Medikara Health: Postmortem Gangguan Medikara+ saat Program Vaksinasi

Bab 5 mengisahkan lonjakan Medikara+ setelah notifikasi program vaksinasi influenza: pengguna aktif bersamaan naik dari sekitar 1.200 menjadi 9.600 dalam 20 menit, waktu respons naik dari 1,5 menjadi 14 detik, dan 23 persen pemesanan gagal pada jam pertama. Tim transformasi menulis ulang *postmortem* kejadian itu dengan format baru. Linimasanya ada di Tabel 11.9.

Dampaknya dihitung ulang dengan SLO yang kini berlaku. Misalkan pada jam pertama ada sekitar 6.000 upaya pemesanan. Sebanyak 23 persen gagal, atau sekitar 1.380 upaya. Angka ini hampir sepuluh kali anggaran 30 hari, yang hanya 140 upaya. Satu jam pada saat paling sibuk menghabiskan anggaran sebulan. Inilah alasan SLI berbasis permintaan lebih jujur daripada SLI berbasis waktu: jam yang paling penting bagi pasien diberi bobot paling besar.

Postmortem tidak menyalahkan staf pemasaran yang mengirim notifikasi. Pertanyaannya diganti: mengapa sistem mengizinkan satu notifikasi terkirim ke semua pengguna sekaligus tanpa memberi tahu tim operasi? Lima faktor ditemukan. Kapasitas hanya bisa ditambah secara vertikal. Sesi tersimpan di memori server. Waktu menyala mesin baru 11 menit. Notifikasi massal tidak dikirim bertahap. *Alert* hanya memantau prosesor dan tenggelam di antara puluhan *alert* rutin.

Tindak lanjutnya empat butir, masing-masing dengan pemilik dan tenggat. Notifikasi massal dikirim bertahap per 10 menit, dipegang tim kanal digital. *Alert* laju pembakaran untuk pemesanan dipasang, dipegang tim platform. Sesi dipindah ke penyimpanan bersama agar kapasitas dapat ditambah secara horizontal, dipegang tim layanan janji temu. Kalender kampanye dibagikan ke tim operasi paling lambat tujuh hari sebelumnya, dipegang bagian pemasaran.

11.6.5 Pelaporan ke Direksi

Direksi tidak membutuhkan laju pembakaran, tetapi jawaban atas tiga pertanyaan: apakah perjalanan pasien terpenting berjalan baik, apakah risikonya naik, dan apa yang sedang diperbaiki. Laporan bulanan Medikara memuat persentase keberhasilan setiap perjalanan kritis dibanding targetnya, sisa *error budget* dalam warna

Tabel 11.9: Linimasa *postmortem* gangguan Medikara+ saat program vaksinasi

Pukul	Kejadian	Catatan
09.00	Notifikasi program vaksinasi terkirim ke seluruh pengguna sekaligus	Tim operasi tidak diberi tahu
09.06	*Alert* prosesor berbunyi	Tenggelam di antara sekitar 40 *alert* rutin pagi itu
09.20	Pengguna aktif sekitar 9.600; waktu respons 14 detik	Pemesanan mulai banyak gagal
09.27	Pusat layanan pelanggan melaporkan lonjakan keluhan	Deteksi pertama datang dari manusia
09.35	Vendor memutuskan mengganti mesin dengan ukuran dua kali lipat	Belum ada komandan insiden yang ditunjuk
10.15	Penggantian mesin selesai setelah sekitar 40 menit; semua sesi terputus	Pasien yang sedang memesan harus masuk ulang
10.26	Mesin baru siap setelah memuat jadwal dokter selama sekitar 11 menit	Waktu respons mulai turun

hijau, kuning, atau merah, status tindak lanjut insiden besar, dan tren tiga bulan. Ukuran kinerja pengiriman dari riset DORA (DevOps Research and Assessment) melengkapinya, karena melihat kecepatan dan stabilitas perubahan sekaligus [45]. KPI (*Key Performance Indicator*) dan format laporan direksi dibahas rinci di IT40813.

11.7 Aktivitas: SLO Layanan Kritis dan Rancangan Observability

Tujuan

Tim menetapkan SLI, SLO, dan kebijakan *error budget* untuk dua sampai tiga perjalanan pengguna yang kritis, beserta sinyal telemetri yang dibutuhkan untuk mengukurnya. Tim memakai organisasi nyata yang sudah memberi izin, dengan data yang dianonimkan, atau kasus Medikara Health.

Pembagian Peran

- Anggota non-teknis menentukan arti “layanan yang baik” dari sisi pengguna dan manajemen. Contohnya: berapa lama pasien rela menunggu konfirmasi, dan kapan hasil laboratorium dianggap terlambat.
- Anggota teknis menerjemahkannya menjadi SLI yang terukur, target SLO, titik ukur, dan kebutuhan telemetri.
- Seluruh anggota menyepakati kebijakan *error budget*, termasuk siapa yang

memutuskan ketika anggaran habis.

Langkah Kerja

1. Pilih dua sampai tiga perjalanan pengguna yang kritis. Tulis alasannya: seberapa sering perjalanan itu dipakai, dan apa dampaknya bila gagal.
2. Rumuskan SLI untuk setiap perjalanan sebagai rasio peristiwa baik dibagi seluruh peristiwa. Tentukan titik ukurnya.
3. Tetapkan target SLO dan jendela waktunya, lalu hitung *error budget* dalam menit dan dalam jumlah permintaan. Skrip `sources/chapter11/hitung_error_budget.py` membantu perhitungan ini.
4. Daftar sinyal telemetri yang dibutuhkan: metrik untuk SLI, *trace* untuk menelusuri perjalanan, dan *log* untuk rincian kesalahan. Tandai data pribadi yang tidak boleh ikut tercatat.
5. Tulis kebijakan *error budget* sepanjang satu halaman: zona, tindakan, pihak yang memutuskan, dan syarat *postmortem*.

Luaran dan Kaitan dengan Capstone

Luarannya adalah dokumen SLO layanan kritis dan rancangan *observability*, sebagai bagian tonggak C3 Kondisi Target. Dokumen ini berisi tabel SLI dan SLO yang mengikuti format Tabel 11.6, gambar alur telemetri yang mengikuti Gambar 11.3, dan kebijakan *error budget* satu halaman. Sasaran SLO ini dipakai lagi pada Bab 12 untuk menetapkan target pemulihan dan pada Bab 14 sebagai salah satu ukuran kemajuan peta jalan.

Kriteria Penilaian

- SLI berangkat dari pengalaman pengguna (30 persen): rasio peristiwa baik dibagi seluruh peristiwa, diukur sedekat mungkin dengan pengguna.
- Target dan anggaran beralasan (25 persen): target dikaitkan dengan dampak bisnis dan biaya, dan anggaran dihitung benar dalam menit serta jumlah permintaan.
- Rancangan telemetri (25 persen): ketiga sinyal dipetakan ke kebutuhan, dan data pribadi disamarkan atau tidak dicatat.
- Kebijakan *error budget* (20 persen): zona, tindakan, dan pengambil keputusan jelas serta dapat dijalankan.

Berkas Pendukung

- `sources/chapter11/templat-slo.md`: templat tabel SLI, SLO, titik ukur, dan kebutuhan telemetri.
- `sources/chapter11/templat-kebijakan-error-budget.md`: templat kebijakan *error budget* satu halaman.
- `sources/chapter11/hitung_error_budget.py`: kalkulator *error budget* dan laju pembakaran.
- `sources/chapter11/contoh-opentelemetry/`: contoh *trace* lintas dua layanan untuk pendalaman teknis.

Pendalaman Teknis (Opsional)

Anggota teknis menjalankan contoh di `sources/chapter11/contoh-opentelemetry/` dan menunjukkan satu *trace* lintas dua layanan dengan `trace_id` yang sama. Instrumentasi dan uji beban hanya untuk sistem milik tim sendiri atau yang sudah berizin tertulis. Telemetri dari organisasi nyata tidak boleh memuat data pribadi pasien atau karyawan.

11.8 Ringkasan

Dasbor yang hijau belum menjamin pasien terlayani. *Monitoring* menjawab pertanyaan yang sudah ditentukan sebelumnya, misalnya apakah pemakaian prosesor melewati ambang. *Observability* memungkinkan tim menjawab pertanyaan baru, misalnya mengapa hanya pemesanan ke Bandung yang gagal. Kebutuhan ini makin terasa setelah sistem dipecah menjadi banyak layanan, karena satu permintaan melintasi komponen milik tim dan vendor yang berbeda. Manajemen dan auditor juga membutuhkannya untuk bukti tingkat layanan, rekonstruksi insiden sesuai UU PDP, dan pemeriksaan kontrol akses.

Tiga sinyal telemetri saling melengkapi. Metrik murah dan cocok untuk dasbor serta *alert*. *Log* merekam rincian setiap kejadian. *Trace* menunjukkan perjalanan satu permintaan dan langkah tempat waktunya habis, seperti nomor resi paket. OpenTelemetry menyediakan cara baku dan netral vendor untuk menghasilkan ketiganya, sedangkan Collector menjadi titik kendali untuk menyamarkan data pasien dan menekan biaya penyimpanan.

SLI mengukur layanan dari sisi pengguna sebagai rasio peristiwa baik dibagi seluruh peristiwa. SLO menetapkan targetnya dalam satu jendela waktu. SLA menambahkan konsekuensi bagi pihak luar dan biasanya lebih longgar daripada SLO. Target 100 persen hampir selalu keliru, karena pasien tidak dapat merasakan

bedanya, biayanya melonjak, ketergantungan membatasinya, dan tidak ada ruang lagi untuk perubahan.

Error budget mengubah target menjadi anggaran yang dapat dibelanjakan. Untuk pemesanan Medikara+, SLO 99,5 persen berarti 140 upaya gagal per 30 hari. Laju pembakaran menunjukkan seberapa cepat anggaran itu habis, dan menjadi dasar *alert* yang hanya berbunyi ketika pasien benar-benar terdampak. Kebijakan yang disepakati sebelum ada masalah membuat keputusan merilis atau menahan perubahan berangkat dari angka, bukan dari suara paling keras.

Praktik SRE menjaga keseimbangan itu sehari-hari: *toil* dibatasi dan diotomasi, *alert* dipasang pada gejala, insiden ditangani dengan peran yang terpisah, dan setiap insiden besar dipelajari lewat *postmortem* tanpa menyalahkan. Pesan utama bab ini: keandalan diukur dari pengalaman pasien, ditargetkan secukupnya, dan dibelanjakan dengan sadar untuk membeli kecepatan perubahan.

11.9 Latihan

Ketiga latihan berikut memakai angka Medikara Health dari bab ini. Hasil setiap mahasiswa boleh berbeda. Yang dinilai adalah penalaran dan perbandingannya.

Latihan 1: Waktu Tidak Tersedia dan Biayanya

Latihan ini membuktikan bahwa selisih target yang tampak kecil mengubah waktu toleransi dan biaya secara besar. Langkah kerjanya:

1. Hitung jumlah menit dalam jendela 30 hari.
2. Hitung *error budget* dalam persen dan menit untuk target 99,5 dan 99,9 persen.
3. Hitung upaya pemesanan gagal yang diizinkan dari sekitar 28.000 upaya per 30 hari.
4. Tentukan rancangan minimal tiap target: jumlah zona, jam jaga petugas, dan cara merilis.
5. Perkirakan biaya bulanan tiap rancangan, lalu bandingkan dengan selisih menitnya.

Tabel 11.10 memberi contoh untuk dua target lain. Tabel 11.11 adalah lembar isian.

Pertanyaan analisis:

1. Berapa kali lipat selisih waktu tidak tersedia antara 99,5 dan 99,9 persen? Apakah pasien akan merasakan selisih itu, mengingat pasien masih dapat

Tabel 11.10: Latihan 1, contoh terisi untuk target 99,0 dan 99,95 persen

Target	Anggaran	Menit per 30 hari	Upaya gagal	Rancangan minimal	Biaya per bulan
99,0%	1%	432	280	Satu zona, dua replika, petugas jaga pada jam kerja, rilis kapan saja	Misalkan Rp38 juta
99,95%	0,05%	21,6	14	Dua zona, basis data tereplikasi, petugas jaga 24 jam, rilis bertahap	Misalkan Rp96 juta

Tabel 11.11: Latihan 1, lembar isian untuk target 99,5 dan 99,9 persen

Target	Anggaran	Menit per 30 hari	Upaya gagal	Rancangan minimal	Biaya per bulan
99,5%	…	…	…	…	…
99,9%	…	…	…	…	…

memesan lewat telepon?

2. Berapa biaya tambahan per menit waktu tersedia yang “dibeli” dengan menaikkan target?
3. Dalam kondisi apa selisih biaya itu layak dibayar Medikara Health?

Latihan 2: Mengapa Target 100 Persen Keliru

Latihan ini membuktikan bahwa keandalan sebuah perjalanan dibatasi oleh rantai ketergantungannya, sehingga target 100 persen tidak dapat dicapai dan tidak berguna. Langkah kerjanya:

1. Daftar semua komponen yang dilalui perjalanan, mulai dari jaringan pasien.
2. Catat target ketersediaan setiap komponen.
3. Kalikan target itu berurutan untuk memperoleh ketersediaan gabungan.
4. Bandingkan hasil akhirnya dengan target 100 persen.
5. Hitung *error budget* untuk target 100 persen dan akibatnya bagi rilis fitur baru.

Tabel 11.12 memberi contoh untuk perjalanan mengambil hasil laboratorium, dengan target komponen berupa angka rekaan. Tabel 11.13 adalah lembar isian.

Pertanyaan analisis:

1. Komponen mana yang paling menurunkan angka gabungan, dan apa artinya bagi prioritas?

Tabel 11.12: Latihan 2, contoh terisi untuk perjalanan mengambil hasil laboratorium

Komponen	Target ketersediaan	Gabungan sampai komponen ini
Jaringan seluler pasien	99,0%	99,00%
API gateway	99,95%	98,95%
Layanan hasil laboratorium	99,9%	98,85%
Sistem informasi laboratorium	99,5%	98,36%
Penyimpanan berkas hasil	99,99%	98,35%

Tabel 11.13: Latihan 2, lembar isian untuk perjalanan membuat janji temu

Komponen	Target ketersediaan	Gabungan sampai komponen ini
Jaringan seluler pasien	...	...
API gateway	...	...
Layanan janji temu	...	...
Layanan jadwal dokter	...	...
Adaptor SIMRS	...	...
Layanan notifikasi	...	...

2. Jaringan seluler pasien berada di luar kendali Medikara. Bagaimana kenyataan ini memengaruhi cara memilih target SLO yang masuk akal?
3. Bila direktur pelayanan tetap meminta 100 persen, berapa rilis fitur baru yang masih boleh dilakukan dalam setahun menurut kebijakan pada Tabel 11.7?

Latihan 3: SLI Janji Temu dari Sisi Pasien

Latihan ini membuktikan bahwa SLI yang baik diukur dari pengalaman pasien, bukan dari kondisi server. Langkah kerjanya:

1. Tulis langkah-langkah perjalanan pasien saat membuat janji temu lewat Medikara+.
2. Tentukan peristiwa "baik" dari sisi pasien, misalnya konfirmasi dalam batas waktu tertentu.
3. Daftar tiga kandidat SLI, termasuk minimal satu yang hanya mengukur kondisi server.
4. Nilai tiap kandidat: rumus rasio, titik ukur, kedekatan dengan pasien, dan kelemahannya.
5. Pilih satu kandidat dan tulis rumusnya secara lengkap.

Tabel 11.14 memberi contoh untuk perjalanan mengambil hasil laboratorium. Tabel 11.15 adalah lembar isian untuk perjalanan membuat janji temu.

Tabel 11.14: Latihan 3, contoh terisi untuk perjalanan mengambil hasil laboratorium

Kandidat SLI	Titik ukur	Dekat dengan pasien	Kelemahan
Pemakaian prosesor server hasil laboratorium di bawah 80 persen	Server	Rendah	Bukan rasio peristiwa; server bisa santai saat hasil belum tersinkron
Permintaan halaman hasil yang dijawab tanpa kegagalan, dibagi seluruh permintaan halaman	*API gateway*	Sedang	Halaman terbuka, tetapi hasil terbaru bisa belum ada
Hasil yang tampil dalam 30 menit setelah divalidasi, dibagi seluruh hasil yang divalidasi	Sistem laboratorium dan aplikasi	Tinggi	Butuh jam yang selaras di dua sistem

Tabel 11.15: Latihan 3, lembar isian untuk perjalanan membuat janji temu

Kandidat SLI	Titik ukur	Dekat dengan pasien	Kelemahan
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Pada kasus Pendahuluan, kandidat mana yang akan menangkap 1.140 keluhan pasien Bandung, dan kandidat mana yang tetap hijau?
2. Bila SLI pilihan diukur dari aplikasi di ponsel, bagaimana tim memisahkan kegagalan akibat jaringan pasien dari kegagalan akibat Medikara sendiri?

Bab 12

Resiliensi dan Disaster Recovery

Tujuan Pembelajaran

- Mengidentifikasi mode kegagalan dalam sistem terdistribusi dan pola resiliensi untuk menanganinya.
- Menetapkan RTO dan RPO berdasarkan dampak bisnis.
- Memilih strategi DR yang seimbang antara biaya dan risiko.

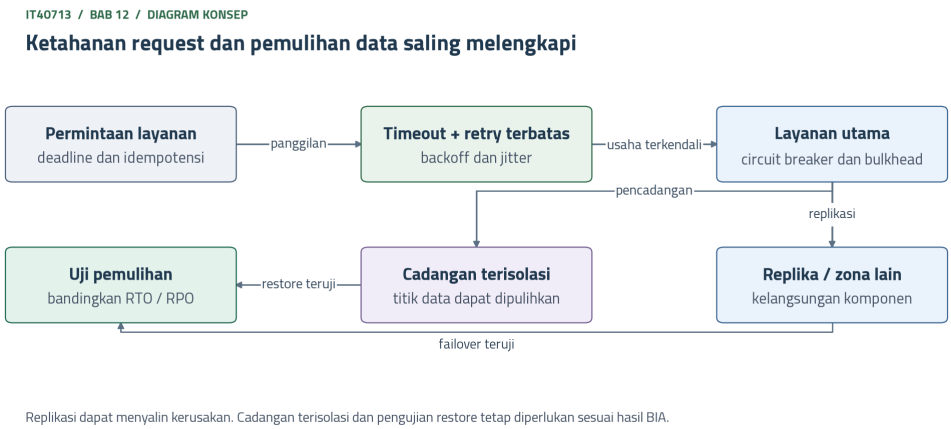
12.1 Pendahuluan

Pada 20 Juni 2024 Pusat Data Nasional Sementara 2 (PDNS 2) di Surabaya lumpuh. Dalam rapat kerja dengan Komisi I Dewan Perwakilan Rakyat (DPR), Menteri Komunikasi dan Informatika menjelaskan bahwa gangguan pertama terdeteksi pada 17 Juni 2024 berupa upaya menonaktifkan fitur keamanan Windows Defender. Aktivitas berbahaya *ransomware* Brain Cipher mulai berjalan pada 20 Juni, dan penyerang meminta tebusan 8 juta dolar Amerika Serikat, sekitar Rp131 miliar [23]. Menurut pengelola pusat data, 282 instansi pengguna (*tenant*) terdampak, dan layanan keimigrasian termasuk tiga layanan pertama yang berangsur pulih [39]. Dalam rapat dengan Komisi I DPR pada 27 Juni 2024, Kepala Badan Siber dan Sandi Negara (BSSN) menyebut hanya 2 persen data di PDNS Surabaya yang memiliki cadangan [153].

Peristiwa itu meninggalkan satu pelajaran yang mudah diucapkan dan sulit dijalankan: cadangan yang tidak lengkap atau tidak pernah diuji sama dengan tidak punya cadangan. Bagi rumah sakit, taruhannya lebih besar dari layanan administrasi. Instalasi Gawat Darurat (IGD) tetap menerima pasien pukul tiga dini hari, apa pun yang terjadi pada pusat data. Bab 1 menyisakan satu pertanyaan

untuk Medikara Health: bagaimana rekam medis dipulihkan bila satu pusat data padam? Bab ini menjawabnya lewat dua konsep. Resiliensi adalah kemampuan sistem tetap melayani, mungkin dengan fungsi terbatas, ketika sebagian komponennya gagal. *Disaster Recovery* (DR) adalah rencana, arsitektur, dan latihan untuk memulihkan layanan setelah kejadian besar yang tidak dapat ditahan oleh resiliensi biasa.

Gambar 12.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 12.1: Ketahanan request dan pemulihan data saling melengkapi

Bab ini disusun dalam enam langkah: mode kegagalan dalam sistem terdistribusi, pola resiliensi, penerapan *multi-AZ* dan *multi-region*, *Recovery Time Objective* (RTO) dan *Recovery Point Objective* (RPO) beserta strategi DR, pertahanan cadangan terhadap *ransomware*, serta *chaos engineering* dan *game day*. Rujukan utamanya adalah *whitepaper* DR dari Amazon Web Services (AWS) [5], pilar keandalan AWS Well-Architected [8], panduan DR Azure Well-Architected [93], buku Nygard tentang perangkat lunak yang siap produksi [109], buku *Site Reliability Engineering* (SRE) Google [27], pedoman perencanaan kontinjensi National Institute of Standards and Technology (NIST) [136], dan kajian Basiri dkk. tentang *chaos engineering* [26]. Hasil bab ini adalah rencana resiliensi dan DR beserta jadwal uji pemulihan, yang menjadi bagian tonggak capstone C4 Peta Jalan dan Tata Kelola.

12.2 Mode Kegagalan dalam Sistem Terdistribusi

Pada monolit, kegagalan biasanya total dan mudah dikenali: aplikasi berhenti, semua orang tahu. Setelah sistem dipecah menjadi beberapa layanan yang berbicara lewat jaringan (Bab 7 dan Bab 9), kegagalan lebih sering bersifat sebagian.

Satu layanan lambat, satu panggilan gagal, satu mitra tidak menjawab, sementara bagian lain masih berjalan. Karena itu pertanyaan pertama dalam merancang resiliensi bukan "bagaimana mencegah semua kegagalan", melainkan "kegagalan seperti apa yang akan terjadi, dan bagaimana sistem bersikap ketika itu terjadi".

12.2.1 Kegagalan Parsial dan Latensi Tinggi

Kegagalan parsial adalah keadaan ketika sebagian komponen gagal sementara komponen lain tetap berjalan. Bentuknya beragam: sebagian permintaan ditolak, satu dari tiga salinan layanan mati, atau satu fungsi tidak dapat dipakai. Bentuk yang paling licin adalah latensi tinggi: layanan tidak mati, tetapi menjawab jauh lebih lambat dari biasanya. Nygard menggolongkan respons yang lambat sebagai salah satu pola penyebab ketidakstabilan, karena respons lambat menahan sumber daya pemanggil lebih lama daripada penolakan yang cepat [109]. Pemanggil tidak segera tahu bahwa ada masalah. Ia menunggu, dan selama menunggu ia memegang koneksi, memori, dan antrean kerja.

Analoginya seperti menelepon kantor lain untuk menanyakan satu data. Telepon yang tidak diangkat segera membuat petugas mencari jalan lain. Telepon yang diangkat lalu dijawab "mohon tunggu sebentar" selama setengah jam justru lebih merugikan. Petugas memegang gagang telepon, tidak dapat melayani tamu lain, dan antrean di depan mejanya terus memanjang.

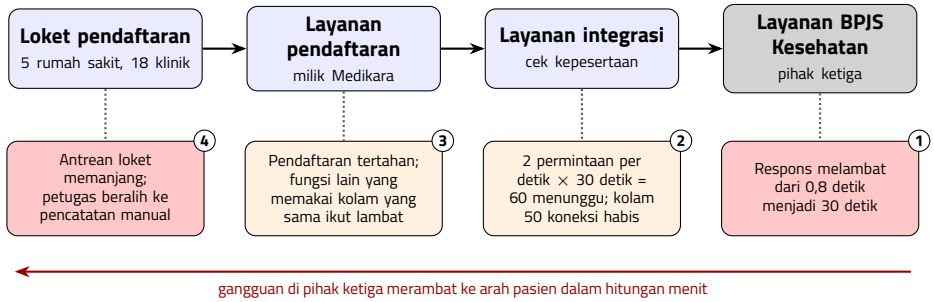
Beban akibat latensi dapat dihitung dengan aturan sederhana: jumlah permintaan yang sedang menunggu kira-kira sama dengan laju permintaan dikali lama menunggu. Dua contoh perhitungannya:

1. Layanan yang menerima 2 permintaan per detik dan menjawab dalam 0,8 detik hanya memegang sekitar 2 permintaan pada satu waktu. Bila jawaban melambat menjadi 30 detik, jumlah itu naik menjadi sekitar 60 permintaan, tiga puluh kali lipat, padahal jumlah pasien tidak bertambah.
2. Bila layanan itu hanya punya kolam 50 koneksi, kolam penuh setelah sekitar 25 detik ($50 \div 2$). Sesudah itu permintaan baru tidak tertahan oleh layanan tujuan, melainkan oleh kolam koneksi milik sendiri.

12.2.2 Kegagalan Berantai dan Titik Kegagalan Tunggal

Kegagalan berantai (*cascading failure*) terjadi ketika kegagalan satu bagian membuat bagian lain ikut gagal, lalu bagian berikutnya, dan seterusnya. Buku SRE Google menjelaskan bahwa kegagalan berantai tumbuh lewat umpan balik positif. Bagian yang kelebihan beban menjadi lambat atau gagal, sehingga beban berpindah ke bagian lain yang kemudian ikut kelebihan beban [27]. Gambar 12.2 memperlihatkan rambatan semacam itu pada alur pendaftaran pasien: gangguan bermula

di layanan pihak ketiga, tetapi yang merasakan akibatnya adalah pasien di loket.



Gambar 12.2: Kegagalan berantai: layanan pihak ketiga yang melambat menghabiskan kolam koneksi, lalu merambat sampai ke antrean loket pendaftaran

Titik kegagalan tunggal (*single point of failure*) adalah komponen yang, bila gagal, menghentikan seluruh layanan. Titik ini sering tersembunyi di tempat yang tidak tergambar pada diagram arsitektur: satu basis data identitas yang dipakai semua aplikasi, satu sertifikat yang kedaluwarsa, satu sambungan jaringan ke kantor pusat, satu akun administrator, atau satu orang yang hafal prosedur pemulihan.

Analoginya seperti supermarket dengan satu kasir yang mesinnya macet. Antrean di kasir merambat ke lorong belanja, troli menghalangi rak, dan pembeli yang hanya ingin mengambil satu barang ikut terjebak. Masalahnya ada di satu titik, tetapi seluruh toko terasa lumpuh.

Dua contoh kasus nyata:

1. Pada 7 Desember 2021 *region* US-EAST-1 milik AWS terganggu. Laporan pascainsiden AWS mencatat bahwa aktivitas otomatis untuk menambah kapasitas satu layanan memicu perilaku tak terduga dari banyak klien di jaringan internal, sehingga terjadi lonjakan koneksi yang membanjiri perangkat jaringan. Keterlambatan menaikkan latensi dan *error*, yang memicu lebih banyak percobaan koneksi dan pengulangan. Kemacetan itu bahkan mengganggu data pemantauan yang dibutuhkan tim operasi untuk menemukan penyebabnya [6].
2. Pada Juli 2024 pembaruan perangkat lunak keamanan dari CrowdStrike membuat banyak komputer Windows di seluruh dunia gagal menyala. Microsoft memperkirakan pembaruan itu mengenai 8,5 juta perangkat Windows, kurang dari satu persen dari seluruh mesin Windows [151]. Persentasenya kecil, tetapi satu vendor yang sama dipakai banyak organisasi sekaligus, sehingga dampaknya terasa di berbagai sektor pada hari yang sama.

12.2.3 Kegagalan dari Penyedia dan Pihak Ketiga

Contoh kedua di atas menunjukkan jenis kegagalan yang paling sulit dikendalikan: kegagalan yang berasal dari penyedia atau pihak ketiga. Medikara Health bergantung pada layanan Badan Penyelenggara Jaminan Sosial (BPJS) Kesehatan, SATUSEHAT, penyedia *cloud*, vendor aplikasi Medikara+, dan penyedia jaringan. Organisasi tidak dapat memperbaiki sistem milik pihak lain. Yang dapat dikendalikan hanya dampaknya di sisi sendiri: berapa lama menunggu, apa yang dilakukan ketika pihak lain tidak menjawab, dan fungsi apa yang tetap berjalan. Penilaian risiko pihak ketiga dari sisi kontrak dan tata kelola dibahas rinci di IT40813.

Tabel 12.1 merangkum enam mode kegagalan beserta contoh di Medikara dan pola penanganan yang dibahas di seksi berikutnya.

Tabel 12.1: Enam mode kegagalan, contoh di Medikara Health, dan pola penanganannya

Mode kegagalan	Ciri yang terlihat	Contoh di Medikara	Pola penanganan
Kegagalan total satu komponen	Komponen berhenti menjawab sama sekali	Server aplikasi farmasi RS Depok mati	Salinan ganda, pindah otomatis
Latensi tinggi	Komponen menjawab, tetapi jauh lebih lambat	Cek kepesertaan melambat dari 0,8 menjadi 30 detik	*Timeout*, *circuit breaker*
Kegagalan parsial	Sebagian permintaan gagal, sebagian berhasil	Satu dari lima kiriman data ke SATUSEHAT ditolak	*Retry* dengan *backoff*
Kegagalan berantai	Gangguan satu bagian merambat ke bagian lain	Antrean loket memanjang karena layanan pihak ketiga lambat	*Bulkhead*, batas *retry*
Titik kegagalan tunggal	Satu komponen menghentikan semuanya	Satu basis data identitas untuk seluruh aplikasi klinis	Redundansi, *multi-AZ*
Kegagalan pihak ketiga	Penyebab di luar kendali organisasi	Gangguan layanan BPJS Kesehatan, pembaruan vendor yang rusak	*Graceful degradation*, rencana cadangan

Kasus Medikara Health: Gangguan BPJS Kesehatan Merambat ke Loket

Misalkan pada Senin pagi, pukul 08.10, layanan kepesertaan BPJS Kesehatan melambat. Jawaban yang biasanya datang dalam 0,8 detik kini butuh sekitar 30 detik. Di sisi Medikara, satu layanan integrasi melayani pengecekan kepesertaan dari pendaftaran, farmasi, dan klaim. Pada jam sibuk pukul 08.00 sampai 10.00 layanan itu menerima sekitar 2 permintaan per detik dan memakai satu kolam berisi 50 koneksi untuk semua tujuan, termasuk pengiriman data ke SATUSEHAT.

Dalam 25 detik kolam koneksi penuh. Aplikasi pendaftaran mengulang

setiap permintaan yang gagal tiga kali, dan petugas loket menekan tombol "ulangi" karena layar tidak bergerak. Pengiriman data ke SATUSEHAT ikut tertahan karena memakai kolam yang sama. Pasien umum yang tidak memakai BPJS Kesehatan juga ikut menunggu, karena layanan pendaftaran menunggu jawaban layanan integrasi sebelum melanjutkan ke langkah berikutnya. Misalkan pada pukul 09.30 waktu tunggu di loket RS Jakarta naik dari rata-rata 15 menit menjadi 70 menit.
Kejadian ini punya dua konsekuensi. Pertama, penyebabnya ada di luar Medikara, tetapi kerugiannya ditanggung Medikara: pasien yang kesal tidak membedakan sistem milik siapa yang lambat. Kedua, pasien umum yang sama sekali tidak berhubungan dengan BPJS Kesehatan ikut menunggu. Itu tanda bahwa sistem tidak punya batas waktu, tidak punya sekat antarketergantungan, dan tidak punya jalur cadangan. Ketiga kekurangan itu dijawab oleh pola pada seksi berikutnya.

12.3 Pola Resiliensi

Nygard menyebut sekumpulan pola yang menjaga sistem tetap stabil ketika bagian lain gagal sebagai pola stabilitas [109]. Bab ini membahas lima di antaranya yang paling sering dibutuhkan: *timeout*, *retry* dengan *backoff*, *circuit breaker*, *bulkhead*, dan *graceful degradation*. Setiap pola menjawab satu mode kegagalan. Setiap pola juga mengubah apa yang dirasakan pengguna, sehingga pemilik bisnis perlu ikut memutuskan perilaku yang dikehendaki.

12.3.1 Timeout

Timeout adalah batas waktu menunggu jawaban. Setelah batas itu lewat, pemanggil berhenti menunggu, melepas sumber dayanya, dan menjalankan rencana lain. Pola ini mengubah latensi tinggi yang tidak terlihat menjadi kegagalan cepat yang dapat ditangani. Batasnya ditetapkan dari data, bukan tebakan. Brooker menjelaskan cara yang dipakai di Amazon: tim menetapkan tingkat *timeout* keliru yang masih dapat diterima, misalnya 0,1 persen, lalu memakai persentil latensi layanan tujuan yang sesuai, dalam contoh itu persentil ke-99,9, sebagai batas waktunya [30].

Analoginya seperti menunggu tamu yang sudah berjanji bertemu. Petugas menunggu 15 menit, lalu mencatat bahwa tamu tidak datang dan beralih ke agenda berikutnya. Tanpa batas itu, satu tamu yang terlambat dapat menghabiskan seluruh hari kerjanya.

Pada kasus BPJS Kesehatan di atas, batas waktu 3 detik membuat jumlah

permintaan yang menunggu paling banyak sekitar $2 \times 3 = 6$, jauh di bawah kapasitas kolam 50 koneksi. Pendaftaran pasien umum tidak lagi ikut tertahan.

12.3.2 Retry dengan Backoff dan Jitter

Retry adalah mengulang permintaan yang gagal, dengan harapan kegagalannya hanya sementara. *Backoff* adalah jeda yang makin panjang di antara pengulangan, misalnya 1 detik, 2 detik, lalu 4 detik. *Jitter* adalah unsur acak yang ditambahkan pada jeda, supaya klien yang gagal bersamaan tidak mengulang bersamaan pula [30]. Brooker menyebut *retry* itu "egois": klien yang mengulang meminta server menghabiskan lebih banyak waktu demi peluang berhasilnya sendiri. Bila kegagalan jarang, itu tidak masalah. Bila kegagalan disebabkan kelebihan beban, pengulangan justru menambah beban [30]. Bila disetel dengan benar, pengguna tidak menyadari apa pun: permintaannya berhasil setelah jeda singkat.

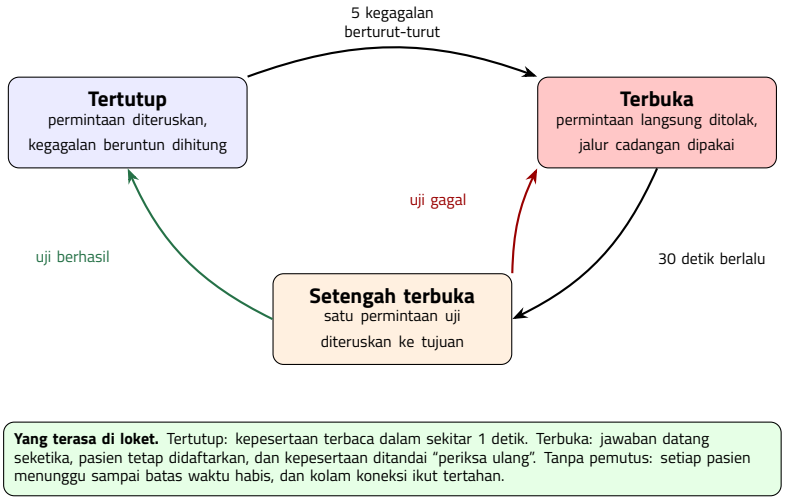
Risiko terbesarnya muncul ketika pengulangan dipasang di banyak lapis. Buku SRE Google memberi contoh: bila tiga lapis masing-masing mengulang tiga kali, atau empat percobaan, satu tindakan pengguna dapat menghasilkan $4^3 = 64$ percobaan ke basis data yang sedang kelebihan beban. Buku itu menyarankan pengulangan dibatasi per permintaan, misalnya lewat anggaran pengulangan sebanyak 60 kali per menit per proses, dan hanya dilakukan di lapis yang memang membutuhkannya [27]. Satu syarat lagi berlaku: hanya operasi yang aman diulang yang boleh diulang. Mengulang pembuatan tagihan tanpa penanda unik dapat menghasilkan tagihan ganda.

Analoginya seperti menelepon ulang kantor yang saluran teleponnya sibuk. Satu orang yang menelepon ulang setelah beberapa menit itu wajar. Seratus orang yang menelepon ulang setiap lima detik membuat saluran tidak pernah lega.

12.3.3 Circuit Breaker

Circuit breaker (pemutus sirkuit) menghentikan panggilan ke layanan yang sudah terbukti gagal, supaya pemanggil tidak terus menunggu dan layanan tujuan diberi waktu untuk pulih. Dokumentasi Azure Architecture Center menjelaskan tiga keadaannya [92]. Pada keadaan tertutup, permintaan diteruskan dan kegagalan terbaru dihitung. Bila jumlah kegagalan melewati ambang, pemutus berpindah ke keadaan terbuka: permintaan langsung ditolak selama jangka waktu tertentu. Setelah jangka itu habis, pemutus masuk ke keadaan setengah terbuka dan mengizinkan sejumlah kecil permintaan uji. Bila uji berhasil, pemutus kembali tertutup. Bila gagal, pemutus kembali terbuka. Keadaan setengah terbuka mencegah layanan yang baru pulih langsung dibanjiri permintaan [92]. Gambar 12.3 memperlihatkan ketiga keadaan itu dengan angka yang dipakai di bab ini.

Analoginya seperti sakelar pemutus arus di rumah. Ketika terjadi hubungan



Gambar 12.3: Tiga keadaan *circuit breaker* dan perilaku yang terasa oleh petugas loket

pendek, sakelar turun dan memutus listrik sebelum kabel terbakar. Penghuni tidak menaikkan sakelar berulang-ulang setiap detik. Ia menunggu, memeriksa, lalu mencoba sekali. Bila sakelar turun lagi, masalahnya belum selesai.

Dua hal pada pola ini sering tertukar. Pembagiannya begini. *Retry* dipakai untuk kegagalan sementara yang diperkirakan segera hilang. *Circuit breaker* dipakai untuk kegagalan yang lebih lama, ketika mengulang hanya membuang sumber daya. Keduanya dapat digabung: pengulangan dijalankan lewat pemutus, dan logika pengulangan berhenti mengulang ketika pemutus menyatakan kegagalan itu tidak sementara [92].

12.3.4 Bulkhead

Bulkhead adalah sekat sumber daya. Kolam koneksi, *thread*, atau salinan layanan dipisah per ketergantungan atau per kelompok pengguna, supaya kegagalan pada satu ketergantungan tidak menghabiskan sumber daya untuk ketergantungan lain. Nama pola ini diambil dari sekat pada lambung kapal. Bila lambung bocor, hanya bagian yang rusak yang terisi air, sehingga kapal tidak tenggelam [98].

Analoginya seperti loket rumah sakit yang dipisah untuk pasien BPJS Kesehatan, pasien umum, dan pengambilan obat. Bila verifikasi di loket BPJS Kesehatan macet, pasien umum tetap dilayani di loketnya sendiri.

Pada kasus di atas, kolam 50 koneksi dapat dibagi menjadi 30 koneksi untuk BPJS Kesehatan, 10 koneksi untuk SATUSEHAT, dan 10 koneksi untuk tujuan lain. Gangguan di BPJS Kesehatan paling banyak menghabiskan 30 koneksi, sementara pengiriman ke SATUSEHAT tetap berjalan.

12.3.5 Graceful Degradation

Graceful degradation adalah kemampuan tetap menjalankan fungsi inti dengan mutu yang diturunkan ketika fungsi pendukung gagal. Sistem tidak memilih antara "jalan penuh" dan "mati total". Ia punya beberapa tingkat layanan yang sudah disepakati sebelumnya. Penentuan tingkat itu adalah keputusan bisnis, bukan keputusan teknis semata, karena menyangkut risiko yang mau diterima.

Analoginya seperti rumah makan saat listrik padam. Lampu darurat menyala, menu dibatasi pada masakan yang tidak membutuhkan oven, dan pembayaran hanya tunai. Tamu tetap dilayani, walaupun pilihannya berkurang.

Dua contoh di Medikara Health:

1. Saat pemutus ke layanan kepesertaan terbuka, pasien tetap didaftarkan dan kepesertaannya ditandai "periksa ulang". Misalkan sekitar 240 pasien terdaftar dengan tanda itu selama satu jam gangguan. Petugas klaim memeriksanya ulang setelah layanan pulih, sehingga pekerjaannya bergeser, bukan hilang.
2. Bila layanan pemeriksaan interaksi obat tidak tersedia, apotek tidak boleh menyerahkan obat seolah-olah pemeriksaan sudah lulus. Tingkat layanan yang diturunkan di sini adalah pemeriksaan manual oleh apoteker. Fungsi yang menyangkut keselamatan pasien tidak boleh dilewati, hanya boleh dikerjakan dengan cara lain.

Setiap pola juga punya risiko bila salah disetel. *Timeout* yang terlalu pendek membatalkan permintaan yang masih akan berhasil. Ambang *circuit breaker* yang terlalu rendah membuat pemutus sering terbuka padahal tujuan sehat. Sekat *bulkhead* yang terlalu kecil membuat antrean di satu fungsi walaupun sumber daya lain menganggur. Karena itu angka setiap pola diambil dari data latensi dan beban, lalu diuji seperti dijelaskan pada seksi 12.7.

Pendalaman Teknis (opsional): Mensimulasikan Circuit Breaker

Di lingkungan produksi, *circuit breaker* biasanya diambil dari pustaka resiliensi yang sudah matang, misalnya Resilience4j untuk Java atau Polly untuk .NET, atau dari *service mesh* seperti dibahas di Bab 9. Untuk memahami perilakunya, skrip `sources/chapter12/simulasi-circuit-breaker.py` meniru cara kerja pustaka tersebut dengan pustaka standar Python saja. Layanan pendaftaran memeriksa kepesertaan satu kali per detik selama 120 detik simulasi, dan layanan tujuan dibuat gangguan pada detik 20 sampai 79. Listing 12.1 memperlihatkan tiga metode inti pemutusnya; metode `pindah` hanya mengganti keadaan dan mencatat riwayatnya.

```python
def boleh_memanggil(self, detik):
    """Menjawab apakah panggilan boleh diteruskan ke layanan tujuan.

    Setelah lama_terbuka habis, satu panggilan uji diizinkan lewat.
    """
    if self.keadaan != TERBUKA:
        return True
    if detik - self.dibuka_pada >= self.lama_terbuka:
        self.pindah(detik, SETENGAH_TERBUKA)
        return True
    return False

def catat_berhasil(self, detik):
    """Satu keberhasilan menutup sirkuit dan mengosongkan hitungan."""
    self.gagal_beruntun = 0
    self.pindah(detik, TERTUTUP)

def catat_gagal(self, detik):
    """Kegagalan menambah hitungan dan dapat membuka sirkuit.

    Pada keadaan setengah terbuka, satu kegagalan langsung membuka
    kembali sirkuit, karena tujuan terbukti belum pulih.
    """
    self.gagal_beruntun += 1
    if (self.keadaan == SETENGAH_TERBUKA
            or self.gagal_beruntun >= self.ambang_gagal):
        self.dibuka_pada = detik
        self.pindah(detik, TERBUKA)
```

Listing 12.1: Tiga metode inti kelas `PemutusSirkuit`, tersedia di `sources/chapter12/simulasi-circuit-breaker.py`

Skrip dijalankan dengan `python simulasi-circuit-breaker.py`. Tanpa pemutus, 60 panggilan gagal setelah menunggu batas waktu 3 detik, dan total waktu tunggu mencapai 228 detik. Dengan pemutus berambang 5 kegagalan beruntun dan jangka terbuka 30 detik, pemutus terbuka pada detik 24, uji pada detik 54 gagal sehingga pemutus terbuka kembali, lalu uji pada detik 84 berhasil dan pemutus tertutup. Hanya 6 panggilan gagal setelah menunggu, 58 permintaan dialihkan ke jalur cadangan, dan total waktu tunggu turun menjadi 62,8 detik. Hasil itu juga menunjukkan harga pemutus: layanan tujuan sudah sehat sejak detik 80, tetapi empat permintaan pada detik 80 sampai 83 tetap dialihkan karena pemutus masih menunggu jangka terbukanya habis. Ubah konstanta `AMBANG_GAGAL_BERUNTUN` dan `LAMA_TERBUKA_DETIK`, lalu amati pertukaran antara waktu tunggu dan jumlah permintaan yang dialihkan.

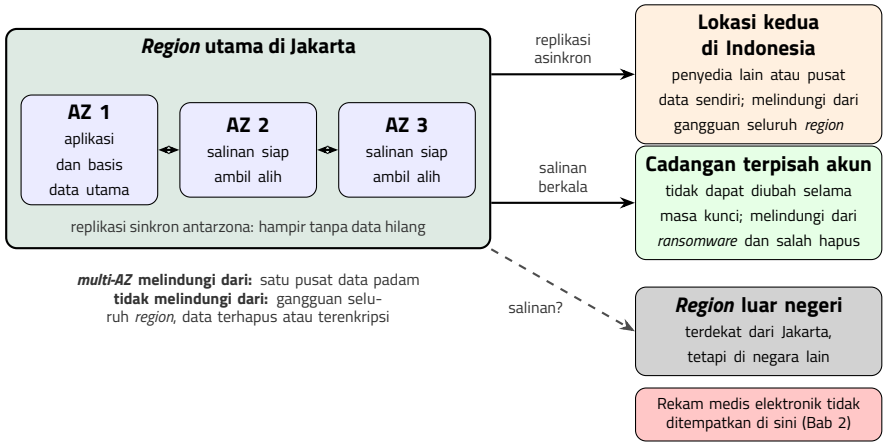
12.4 Multi-AZ dan Multi-Region

Pola resiliensi menangani kegagalan komponen. Pertanyaan berikutnya lebih besar: bagaimana bila yang gagal adalah satu pusat data, atau seluruh *region*? Bab 2 sudah menjelaskan bahwa satu *region* berisi beberapa *availability zone* (AZ), dan setiap AZ punya pasokan listrik, pendingin, dan jaringan sendiri. Susunan itu memberi dua tingkat perlindungan.

Multi-AZ berarti menjalankan salinan aplikasi dan data di dua atau tiga AZ dalam satu *region*. Karena jarak antar-AZ dekat dan jaringannya berlatensi rendah, data dapat direplikasi secara sinkron, sehingga satu pusat data yang padam hampir tidak menghilangkan data. *Multi-region* berarti menyiapkan salinan di *region* lain yang berjauhan. Rancangan ini melindungi dari bencana yang mengenai seluruh *region*, tetapi replikasinya biasanya asinkron karena jaraknya jauh, sehingga data terakhir yang belum terkirim dapat hilang [5]. Pilar keandalan AWS menetapkan hasil yang dikehendaki berupa beban kerja produksi yang tersebar di beberapa AZ atau *region*. Pilar yang sama mencatat dua kekeliruan umum: beban kerja produksi yang hanya berada di satu AZ, dan rancangan *multi-region* yang dipakai padahal *multi-AZ* sudah memenuhi kebutuhan bisnis [7].

Analoginya seperti perlindungan dokumen di sebuah kantor. Genset di ruang bawah tanah melindungi dari listrik padam, tetapi tidak dari banjir yang merendam gedung. Kantor cabang di kota lain melindungi dari banjir. Namun dokumen harus dikirim ke cabang setiap hari, dan selalu ada dokumen hari ini yang belum sampai ketika banjir datang.

Gambar 12.4 menyusun tingkat perlindungan itu untuk Medikara, termasuk lokasi yang tidak boleh dipakai untuk rekam medis.



Gambar 12.4: Cakupan perlindungan *multi-AZ*, lokasi kedua, dan cadangan terpisah, beserta kendala lokasi data untuk rekam medis elektronik

12.4.1 Biaya dan Kerumitan Replikasi Data

Dua hal pada replikasi sering tertukar. Pembagiannya begini. Replikasi sinkron menganggap satu penulisan selesai setelah lokasi utama dan lokasi kedua sama-sama menyimpannya. Tidak ada data yang hilang, tetapi setiap penulisan menunggu perjalanan pulang-pergi ke lokasi kedua, sehingga cara ini hanya wajar bila lokasinya dekat. Replikasi asinkron menganggap penulisan selesai di lokasi utama, lalu salinannya menyusul. Penulisan tetap cepat, tetapi data yang belum terkirim hilang saat bencana. Selisih waktu itulah yang kemudian menjadi RPO.

Selain itu, setiap lokasi tambahan membawa empat jenis biaya. Pertama, infrastruktur ganda: server, penyimpanan, dan lisensi di lokasi kedua. Kedua, biaya transfer data antarlokasi yang umumnya dihitung per gigabyte. Ketiga, biaya menjaga kesamaan: konfigurasi dan versi aplikasi di dua lokasi mudah menyimpang, dan pilar keandalan AWS mencatat penyimpangan data dan konfigurasi sebagai kekeliruan umum [7]. Keempat, biaya latihan: lokasi kedua yang tidak pernah dipakai sungguhan belum terbukti dapat dipakai.

12.4.2 Kendala Lokasi Data di Indonesia

Bab 2 menyimpulkan bahwa rekam medis elektronik Medikara ditempatkan di Indonesia sesuai PP 28/2024 Pasal 966 [123], dan Peraturan Menteri Kesehatan (Permenkes) 24/2022 mewajibkan cadangan data disimpan di tempat berbeda [83]. Per Oktober 2026, setiap penyedia *cloud* global yang tercantum di Bab 2 hanya punya satu *region* di Indonesia. Akibatnya, *multi-region* dengan satu penyedia berarti *region* kedua berada di luar negeri, dan pilihan itu tertutup untuk rekam medis.

Kendala ini punya dua konsekuensi. Pertama, perlindungan dari gangguan seluruh *region* harus dicari dengan cara lain: lokasi kedua milik penyedia lain di Indonesia, pusat data penyedia lokal, atau ruang server milik sendiri di kota lain. Cybersecurity and Infrastructure Security Agency (CISA), badan keamanan siber Amerika Serikat, juga menyarankan organisasi mempertimbangkan cadangan di penyedia lain, untuk berjaga-jaga bila semua akun di satu penyedia terdampak sekaligus [41]. Kedua, rancangan multi-penyedia menambah kerumitan: dua cara kerja, dua kontrak, dan dua jenis keterampilan. Kerumitan itu harus dibandingkan dengan dampak bisnis, bukan diterima atau ditolak begitu saja.

Ukuran data mempertajam kendala itu. Bab 2 memperkirakan rekam medis Medikara mencapai sekitar 15 terabyte. Memindahkan 15 terabyte lewat sambungan 1 gigabit per detik yang terpakai penuh membutuhkan $15 \times 8 \times 10^{12} \div 10^{9} = 120.000$ detik, sekitar 33 jam, jauh di atas target pemulihan dalam hitungan jam. Data sebesar itu harus sudah berada di lokasi kedua sebelum bencana, bukan dikirim setelah bencana.

Tabel 12.2 membandingkan lima rancangan dari sisi yang dilindungi dan yang tidak.

Tabel 12.2: Lima rancangan lokasi dan cakupan perlindungannya

Rancangan	Melindungi dari	Tidak melindungi dari	Catatan bagi Medikara
Satu AZ dengan cadangan harian	Data hilang karena perangkat rusak	AZ padam: layanan berhenti sampai dibangun ulang	Murah, pemulihan dalam hitungan jam atau hari
Multi-AZ	Satu pusat data padam	Gangguan seluruh *region*; data terhapus atau terenkripsi ikut tersalin	Biaya produksi naik; replikasi sinkron layak
Multi-region, satu penyedia	Gangguan seluruh *region*	Akun yang disusupi, karena akun dan kredensial sering sama	*Region* kedua di luar negeri; tidak untuk rekam medis
Lokasi kedua di Indonesia, penyedia lain	Gangguan *region* atau seluruh penyedia	Kerusakan data yang ikut tersalin	Dua cara kerja, dua kontrak, keterampilan ganda
Cadangan terpisah yang tidak dapat diubah	*Ransomware*, salah hapus, akun disusupi	Waktu henti panjang bila hanya ini yang ada	Wajib diuji pulih secara berkala

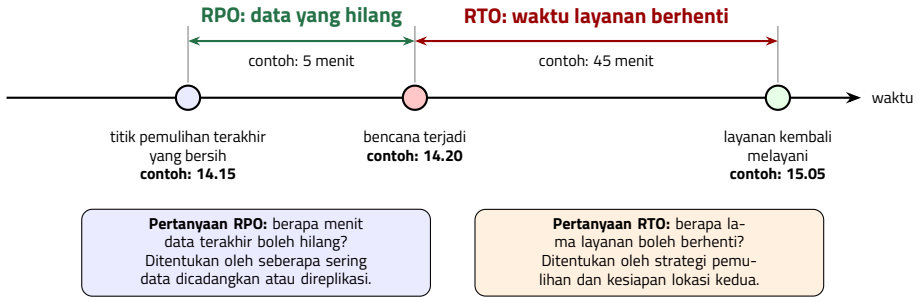
12.5 RTO, RPO, dan Strategi Disaster Recovery

Setelah pilihan lokasi jelas, pertanyaan kemudian muncul: seberapa cepat layanan harus pulih, dan berapa banyak data yang boleh hilang? Dua angka itu menentukan strategi DR dan biayanya. Keduanya adalah keputusan bisnis yang dihitung dari dampak, bukan sifat bawaan teknologi yang kebetulan sedang dipakai.

12.5.1 RTO dan RPO sebagai Keputusan Bisnis

Whitepaper DR AWS mendefinisikan RTO sebagai jeda paling lama yang dapat diterima antara terhentinya layanan dan pulihnya layanan. RPO adalah rentang waktu paling lama yang dapat diterima sejak titik pemulihan data terakhir, sehingga RPO menentukan berapa banyak data yang boleh hilang [5]. Pedoman NIST menambahkan satu batas lagi, yaitu waktu henti maksimum yang dapat ditoleransi (*maximum tolerable downtime*): lama gangguan total yang masih dapat diterima pemilik proses bisnis, dengan memperhitungkan semua dampaknya. RTO harus lebih pendek dari batas itu, karena setelah sistem pulih masih ada waktu untuk memproses ulang pekerjaan yang tertunda. RPO tidak termasuk dalam batas waktu henti, tetapi menggambarkan berapa banyak kehilangan data yang

dapat ditoleransi [136]. Gambar 12.5 meletakkan kedua angka itu pada satu garis waktu.



Gambar 12.5: RPO diukur mundur dari bencana ke titik pemulihan terakhir; RTO diukur maju dari bencana sampai layanan kembali melayani

Analoginya seperti mengetik laporan ketika listrik padam. RPO adalah seberapa sering laporan disimpan: bila terakhir disimpan sepuluh menit lalu, pekerjaan sepuluh menit itu hilang. RTO adalah berapa lama sampai bisa mengetik lagi: mencari laptop lain, membuka berkas, lalu melanjutkan.

Dua hal pada keduanya sering tertukar. Pembagiannya begini. RPO melihat ke belakang dari saat bencana, sedangkan RTO melihat ke depan. Hal kedua yang sering tertukar adalah RTO dan *Service Level Objective* (SLO) ketersediaan dari Bab 11. SLO mengukur ketersediaan sehari-hari dalam satu periode, sedangkan RTO adalah target pemulihan dari satu kejadian besar. SLO 99,9 persen per bulan hanya mengizinkan sekitar 43 menit tidak tersedia ($30 \times 24 \times 60 \times 0{,}001 = 43{,}2$). Satu kejadian besar dengan RTO 4 jam menghabiskan lebih dari lima kali jatah satu bulan. Keduanya harus dibaca bersama.

Whitepaper yang sama memberi satu batas yang penting bagi direksi: bila biaya strategi pemulihan lebih tinggi daripada biaya kegagalan atau kehilangan yang ingin dicegah, strategi itu tidak perlu diterapkan, kecuali ada pendorong lain seperti kewajiban regulasi [5].

12.5.2 Analisis Dampak Bisnis Sederhana

Pedoman NIST menyusun analisis dampak bisnis dalam tiga langkah: menentukan proses bisnis dan tingkat kekritisan pemulihannya, mengenali sumber daya yang dibutuhkan, dan menentukan urutan prioritas pemulihan [136]. Untuk keperluan capstone, langkah itu disederhanakan menjadi lima pertanyaan yang dapat dijawab bersama pemilik bisnis:

1. Apa yang terjadi bila layanan berhenti 1 jam, 4 jam, dan 24 jam?
2. Berapa kerugian finansial per jam, termasuk pendapatan yang tertunda,

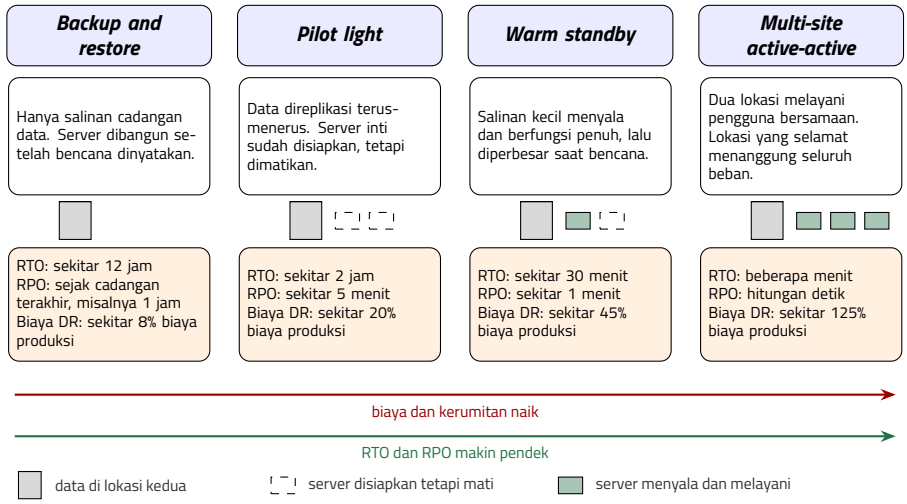
lembur, dan entri ulang?

3. Apakah ada dampak pada keselamatan pasien, dan seberapa besar?
4. Kewajiban regulasi atau kontrak apa yang terlanggar?
5. Berapa waktu henti maksimum yang masih dapat ditoleransi?

Jawaban pertanyaan kelima membatasi RTO. Jawaban tentang pekerjaan yang harus dimasukkan ulang membatasi RPO. Pada tahap ini anggota non-teknis memimpin, karena merekalah yang paling tahu arti satu jam henti bagi IGD atau bagian keuangan.

12.5.3 Empat Strategi Disaster Recovery

Whitepaper DR AWS mengelompokkan strategi DR menjadi empat pendekatan, dari yang murah dan sederhana sampai yang paling rumit [5]. Gambar 12.6 memperlihatkan isi lokasi kedua untuk tiap strategi dan perkiraan angkanya.



Gambar 12.6: Empat strategi DR: makin ke kanan, lokasi kedua makin siap, RTO dan RPO makin pendek, dan biayanya makin tinggi

Backup and restore hanya menyimpan cadangan data di lokasi kedua; infrastruktur dibangun setelah bencana dinyatakan. *Pilot light* mereplikasi data secara terus-menerus dan menyiapkan inti infrastruktur, tetapi server aplikasinya dimatikan sampai dibutuhkan. *Warm standby* menjalankan salinan yang lebih kecil tetapi berfungsi penuh, lalu memperbesarnya saat bencana. *Multi-site active-active* menjalankan beban kerja di beberapa lokasi yang semuanya melayani pengguna. Menurut *whitepaper* yang sama, pendekatan terakhir paling rumit dan paling mahal, tetapi dapat menekan waktu pemulihan sampai mendekati nol. Namun

kerusakan data tetap mungkin membutuhkan cadangan, karena kerusakan itu ikut tersalin ke semua lokasi [5]. Panduan Azure Well-Architected juga meminta rencana DR yang terstruktur, teruji, terdokumentasi, dan selaras dengan target pemulihan [93].

Analoginya seperti usaha katering yang menyiapkan dapur cadangan. Cara pertama menyimpan resep dan bahan beku di gudang lain. Cara kedua menyewa dapur yang kompornya terpasang tetapi tidak dinyalakan, dan bahan segar dikirim ke sana setiap hari. Cara ketiga menyalakan dapur cadangan dengan satu juru masak. Cara keempat menjalankan dua dapur yang sama-sama memasak setiap hari.

Tabel 12.3 merangkum keempatnya. Angka RTO, RPO, dan rasio biaya pada tabel adalah perkiraan kasar yang dipakai bab ini untuk diskusi dan perhitungan latihan, bukan janji penyedia. Angka nyata bergantung pada ukuran data, otomasi, dan latihan tim.

Tabel 12.3: Empat strategi DR dengan perkiraan kasar yang dipakai di bab ini

Strategi	Yang ada di lokasi kedua	RTO kasar	RPO kasar	Biaya DR kasar	Cocok untuk
Backup and restore	Cadangan data saja	Sekitar 12 jam	Sejak cadangan terakhir, misalnya 1 jam	Sekitar 8% biaya produksi	Layanan yang boleh menunggu sehari
Pilot light	Data tereplikasi, server inti siap tetapi mati	Sekitar 2 jam	Sekitar 5 menit	Sekitar 20%	Layanan penting yang boleh berhenti beberapa jam
Warm standby	Salinan kecil yang menyala penuh fungsi	Sekitar 30 menit	Sekitar 1 menit	Sekitar 45%	Layanan klinis yang hanya boleh berhenti sebentar
Multi-site active-active	Salinan penuh yang ikut melayani	Beberapa menit	Hitungan detik	Sekitar 125%	Layanan yang berhenti beberapa menit saja sudah membahayakan

DR adalah bagian dari urusan yang lebih luas, yaitu kelangsungan bisnis. ISO 22301:2019 menetapkan persyaratan sistem manajemen kelangsungan bisnis [75]. Kelangsungan bisnis mencakup orang, gedung, pemasok, dan prosedur manual, bukan hanya sistem teknologi informasi (TI). Contohnya, formulir kertas IGD saat rekam medis elektronik padam adalah bagian dari rencana kelangsungan

bisnis, bukan bagian dari rencana DR. Pembahasan rinci sistem manajemen kelangsungan bisnis ada di IT40813.

Kasus Medikara Health: RTO dan RPO untuk Tiga Layanan

Tim capstone menjalankan analisis dampak bisnis untuk tiga layanan: rekam medis elektronik, Medikara+, dan *Enterprise Resource Planning* (ERP) keuangan. Misalkan pendapatan Medikara sekitar Rp2,2 miliar per hari dari sekitar 3.300 kunjungan per hari (1,2 juta kunjungan dibagi 365 hari) ditambah rawat inap. Bila rekam medis elektronik padam, layanan tetap berjalan dengan formulir kertas, tetapi melambat. Ditambah lembur dan entri ulang, kerugiannya diperkirakan sekitar Rp150 juta per jam. Kewajiban regulasinya mencakup Permenkes 24/2022 dan Undang-Undang (UU) 27/2022 tentang Pelindungan Data Pribadi (PDP) [120]. Hasil lengkapnya tercantum pada Tabel 12.4 dan pada lembar "Contoh Medikara" di `sources/chapter12/templat-analisis-dampak-bisnis.xlsx`.

Rekam medis elektronik mendapat RTO 1 jam karena dampak keselamatan pasien tinggi. Target teknis ini lebih ketat daripada batas RTO sistem SIMRS 120 menit dari analisis dampak bisnis IT40813 Bab 10, supaya ada cadangan waktu. *Pilot light* dengan RTO sekitar 2 jam tidak memenuhi target itu, sehingga tim memilih *warm standby*. Medikara+ cukup memakai *pilot light*, dan ERP keuangan cukup memakai *backup and restore* dengan cadangan log transaksi setiap jam. Total biaya DR ketiganya sekitar Rp724 juta per tahun ($540 + 120 + 64$). Bila ketiganya memakai *multi-site active-active*, biayanya sekitar Rp3,25 miliar per tahun ($1.500 + 750 + 1.000$).

Keputusan ini punya dua konsekuensi. Pertama, selisih sekitar Rp2,5 miliar per tahun tidak membeli perlindungan yang berarti bagi ERP keuangan, yang memang boleh menunggu sampai 24 jam. Kedua, target RTO 1 jam untuk rekam medis harus dibuktikan lewat uji pemulihan. Sebelum diuji, angka itu baru janji di atas kertas.

Satu hal pada tabel itu mudah terlewat. Data pasien di ketiga layanan adalah data pribadi menurut UU PDP, sehingga lokasi kedua dan cadangannya tunduk pada aturan yang sama dengan lokasi utama. Lokasi kedua yang murah tetapi melanggar aturan lokasi data bukan pilihan.

12.6 Ransomware dan Cadangan yang Tidak Dapat Diubah

Strategi DR pada seksi sebelumnya mengandaikan bencana datang dari luar: pusat data padam, jaringan putus, atau *region* terganggu. *Ransomware* berbeda. Penyerang masuk ke dalam, mengenkripsi data, lalu meminta tebusan. CISA mencatat

Tabel 12.4: Hasil analisis dampak bisnis tiga layanan Medikara Health (angka rekaan)

Butir	Rekam medis elektronik	Medikara+	ERP keuangan
Dampak bila berhenti	IGD dan rawat inap kehilangan riwayat alergi, obat, dan hasil laboratorium; formulir kertas dipakai	Pasien tidak dapat membuat janji; telekonsultasi terputus	Penagihan dan pembayaran pemasok tertunda
Kerugian finansial per jam	Sekitar Rp150 juta	Sekitar Rp40 juta	Sekitar Rp15 juta
Dampak keselamatan pasien	Tinggi	Sedang	Rendah
Kewajiban regulasi	Permenkes 24/2022, UU PDP	UU PDP	Pelaporan keuangan dan pajak
Waktu henti maksimum yang dapat ditoleransi	2 jam	8 jam	48 jam
RTO dan RPO	1 jam dan 5 menit	4 jam dan 15 menit	24 jam dan 60 menit
Strategi DR	*Warm standby*	*Pilot light*	*Backup and restore*
Biaya infrastruktur produksi per tahun	Rp1,2 miliar	Rp600 juta	Rp800 juta
Biaya DR per tahun	Sekitar Rp540 juta	Sekitar Rp120 juta	Sekitar Rp64 juta

bahwa banyak varian *ransomware* berusaha menemukan cadangan yang dapat dijangkau, lalu menghapus atau mengenkripsinya [41]. Cadangan yang berada di jaringan yang sama, di akun yang sama, dan dapat dihapus dengan kredensial administrator yang sama bukan pertahanan. Cadangan seperti itu justru menjadi sasaran berikutnya.

Dua hal pada perlindungan data sering tertukar. Pembagiannya begini. Replikasi menyalin keadaan terkini, termasuk kerusakannya: tabel yang terhapus atau berkas yang terenkripsi ikut tersalin dalam hitungan detik. *Whitepaper* DR AWS mencatat bahwa replikasi terus-menerus belum tentu melindungi dari kerusakan data atau serangan berbahaya sebaik cadangan pada titik waktu tertentu [5]. Cadangan menyimpan keadaan masa lalu, sehingga organisasi dapat kembali ke titik sebelum serangan. Replikasi menjaga ketersediaan. Cadangan menjaga kemampuan kembali.

12.6.1 Prinsip Cadangan yang Tahan Ransomware

CISA menyarankan cadangan data kritis yang disimpan terpisah dari jaringan (*offline*), terenkripsi, serta diuji ketersediaan dan keutuhannya secara berkala dalam skenario pemulihan bencana [41]. Beberapa penyedia *cloud* menawarkan penyimpanan yang tidak dapat diubah (*immutable storage*). Selama masa kunci,

data di dalamnya tidak dapat diubah atau dihapus, bahkan oleh administrator. CISA mengingatkan dua hal tentang penyimpanan ini: penyimpanan seperti itu belum tentu memenuhi kriteria kepatuhan tertentu, dan salah konfigurasi dapat menimbulkan biaya besar [41]. Salah satu contohnya adalah masa kunci yang terlalu panjang, sehingga data uji pun tidak dapat dihapus sampai masa kunci berakhir.

Analoginya seperti menyimpan salinan akta tanah. Salinan di laci meja kerja, yang kuncinya tergantung di dinding, mudah diambil siapa pun yang masuk ruangan. Salinan di brankas bank dengan kunci waktu tidak dapat dibuka sebelum waktunya, bahkan oleh pemiliknya sendiri.

Tabel 12.5 merangkum lima prinsip cadangan yang tahan *ransomware* beserta cara memeriksanya. Banyak organisasi juga memakai aturan praktis 3-2-1: tiga salinan data, di dua jenis media, dengan satu salinan di luar lokasi utama.

Tabel 12.5: Lima prinsip cadangan yang tahan *ransomware* dan cara memeriksanya

Prinsip	Artinya	Cara memeriksa
Lengkap	Semua data kritis, konfigurasi, dan kunci enkripsi tercakup	Daftar cadangan dicocokkan dengan daftar layanan kritis; setiap selisih diberi pemilik
Terpisah	Akun, kredensial, dan jaringan berbeda dari produksi; minimal satu salinan di lokasi lain	Administrator produksi dicoba menghapus cadangan, dan hasilnya harus ditolak
Tidak dapat diubah	Selama masa kunci, tidak ada yang dapat mengubah atau menghapus	Masa kunci tercatat; percobaan penghapusan di lingkungan uji ditolak
Terenkripsi	Isi cadangan tidak terbaca bila salinannya dicuri	Kunci enkripsi disimpan terpisah dari cadangan dan ikut dicadangkan
Teruji	Pemulihan dicoba berkala dan waktunya diukur	Catatan uji terakhir memuat waktu pulih nyata dibandingkan dengan RTO

12.6.2 Pelajaran dari PDNS 2 Surabaya

Pendahuluan bab ini mencatat empat fakta PDNS 2: penyerang sudah bergerak sejak 17 Juni, tebusan yang diminta 8 juta dolar Amerika Serikat [23], 282 instansi pengguna terdampak [39], dan hanya 2 persen data yang memiliki cadangan [153]. Fakta itu dapat dibaca sebagai tiga pelajaran arsitektur. Pertama, cakupan cadangan adalah keputusan yang harus dibuat sejak awal dan diperiksa terus, bukan pilihan yang diserahkan pada kebiasaan setiap pengguna. Kedua, keputusan tidak membayar tebusan hanya bermakna bila organisasi sanggup pulih sendiri. Tanpa cadangan yang bersih, tidak membayar berarti kehilangan data. Ketiga, ratusan instansi di satu pusat data adalah konsentrasi risiko. Satu kejadian mengenai semuanya sekaligus. Dari sisi tata kelola, pembagian tanggung jawab antara

pengelola pusat data dan instansi pengguna perlu tertulis jelas, seperti model tanggung jawab bersama di Bab 2. Latihan 2 di akhir bab mengurai pelajaran ini lebih rinci.

Kasus Medikara Health: Audit Cadangan Setelah Membaca Kasus PDNS 2

Misalkan setelah membaca kasus PDNS 2, komite audit meminta tim TI memeriksa cadangan untuk 46 aplikasi dalam inventaris Bab 4. Hasilnya: 31 aplikasi punya jadwal cadangan, 12 aplikasi pernah diuji pulih dalam 12 bulan terakhir, dan hanya 3 aplikasi yang cadangannya berada di akun terpisah. Cadangan SIMRS RS Bandung disimpan di server yang sama dengan basis datanya, dan dapat dihapus dengan akun administrator domain.
Temuan ini punya dua konsekuensi. Pertama, angka "31 dari 46 punya cadangan" terdengar baik, tetapi angka yang lebih jujur adalah "3 dari 46 punya cadangan yang selamat dari *ransomware*". Kedua, perbaikan tidak dimulai dari membeli produk baru, melainkan dari daftar layanan kritis hasil analisis dampak bisnis: rekam medis, Medikara+, dan ERP keuangan lebih dahulu dipindah ke cadangan terpisah yang tidak dapat diubah, lalu diuji pulih.

12.7 Chaos Engineering dan Game Day

Rencana DR yang belum pernah diuji adalah dugaan. Pertanyaan kemudian muncul: bagaimana membuktikan bahwa pola resiliensi dan rencana pemulihan benar-benar bekerja, tanpa menunggu bencana sungguhan? Jawabannya adalah memicu kegagalan dengan sengaja, secara terkendali.

12.7.1 Chaos Engineering

Principles of Chaos Engineering mendefinisikan *chaos engineering* sebagai disiplin bereksperimen pada sebuah sistem untuk membangun keyakinan bahwa sistem itu mampu bertahan dalam kondisi yang bergejolak di lingkungan produksi [121]. Eksperimennya mengikuti empat langkah. Pertama, tetapkan keadaan normal (*steady state*) sebagai keluaran sistem yang terukur. Kedua, susun hipotesis bahwa keadaan normal itu bertahan pada kelompok kontrol maupun kelompok eksperimen. Ketiga, masukkan peristiwa yang mencerminkan kejadian nyata, misalnya server mati, cakram rusak, atau sambungan jaringan terputus. Keempat, cari selisih keadaan normal antara kedua kelompok untuk menguji hipotesis. Prinsip yang sama menuntut cakupan dampak eksperimen (*blast radius*) ditekan sekecil mungkin [121].

Analoginya seperti latihan evakuasi kebakaran di gedung perkantoran. Latihan dijadwalkan, petugas ditunjuk, jalur keluar diperiksa, dan latihan dimulai dari satu lantai sebelum seluruh gedung. Tujuannya menemukan pintu darurat yang terkunci sebelum ada kebakaran sungguhan.

Dua contoh kasus:

1. Netflix menjalankan Chaos Monkey, layanan internal yang memilih secara acak mesin virtual yang menjalankan layanan produksi lalu mematikannya. Chaos Monkey hanya aktif pada jam kerja normal, supaya insinyur dapat segera menangani layanan yang gagal. Netflix juga menjalankan latihan Chaos Kong yang mensimulasikan kegagalan satu *region* penuh, sekali sebulan. Keadaan normalnya diukur dengan jumlah pemutaran video yang dimulai per detik, karena angka itulah yang mencerminkan apakah pengguna dapat menonton [26].
2. Misalkan Medikara memulai dengan eksperimen kecil di lingkungan uji: layanan integrasi dibuat menolak 20 persen panggilan ke tiruan layanan kepesertaan selama 10 menit. Hipotesisnya, waktu pendaftaran tetap di bawah 2 menit karena *timeout* 3 detik dan *circuit breaker* sudah terpasang. Bila hipotesis gagal, temuan itu diperbaiki sebelum eksperimen dipindah ke produksi.

Peringatan etika. Eksperimen kegagalan dan uji beban hanya dijalankan pada sistem milik sendiri atau sistem yang sudah berizin tertulis. Kegagalan layanan BPJS Kesehatan atau SATUSEHAT disimulasikan di sisi Medikara, misalnya dengan tiruan layanan atau dengan memblokir panggilan di layanan integrasi. Sistem milik mitra tidak boleh dibebani atau diganggu.

12.7.2 Game Day

Dua hal sering tertukar di sini. Pembagiannya begini. *Chaos engineering* adalah eksperimen berulang yang sering diotomasi dan berfokus pada perilaku sistem. *Game day* adalah latihan terjadwal yang melibatkan manusia, baik tim teknis maupun tim bisnis, untuk menguji rencana pemulihan dan cara mengambil keputusan. Panduan Azure Well-Architected menyebut latihan semacam ini sebagai *DR drill*: latihan terencana untuk menguji prosedur DR dan memastikan kemampuan pulih dalam kondisi terkendali [93]. *Game day* dapat berupa latihan meja (*tabletop*): peserta duduk bersama, fasilitator membacakan kejadian satu per satu, dan peserta menjawab apa yang akan dilakukan. Tidak ada sistem yang disentuh. Bentuk ini cocok untuk latihan pertama, karena kelemahan prosedur dan komunikasi biasanya sudah terlihat tanpa menyentuh produksi.

Misalkan latihan meja pertama Medikara memakai skenario "*region* utama tidak tersedia selama empat jam". Peserta menemukan bahwa aplikasi rekam

medis di lokasi kedua dapat dinyalakan, tetapi petugas tidak dapat masuk karena layanan identitas hanya ada di *region* utama. Rekam medis baru dapat dipakai pada menit ke-80, sehingga RTO 1 jam tidak terpenuhi. Temuan itu murah karena ditemukan di ruang rapat, bukan di IGD pukul tiga dini hari.

Sebelum eksperimen atau *game day* menyentuh sistem nyata, beberapa syarat harus dipenuhi. Tabel 12.6 merangkumnya.

Tabel 12.6: Syarat aman sebelum eksperimen kegagalan atau *game day* menyentuh sistem nyata

Syarat	Alasan	Contoh di Medikara
Hipotesis dan keadaan normal terukur	Tanpa ukuran, hasil latihan hanya kesan	Misalkan keadaan normal loket: 99 persen pendaftaran selesai di bawah 2 menit, diukur seperti SLI pada Bab 11
Cakupan dampak kecil	Kesalahan latihan tidak boleh menjadi insiden besar	Satu klinik pratama, atau 5 persen lalu lintas Medikara+
Kriteria berhenti dan wewenang menghentikan	Latihan harus dapat dihentikan seketika	Berhenti bila antrean IGD naik di atas 10 pasien; kepala operasi TI berwenang menghentikan
Izin tertulis	Sistem dan pengguna milik pihak yang harus tahu	Persetujuan direktur pelayanan dan pemilik sistem; mitra diberi tahu
Pemantauan berjalan	Dampak harus terlihat saat terjadi	Panel SLO dan antrean loket dapat dilihat seluruh peserta
Cara kembali sudah diuji	Latihan tidak boleh meninggalkan kerusakan	Langkah kembali ke *region* utama sudah dicoba di lingkungan uji

12.8 Aktivitas: Rencana Resiliensi dan DR

Tujuan. Tim menetapkan RTO, RPO, dan strategi DR untuk layanan kritis, memperkirakan biaya tambahannya, memetakan pola resiliensi untuk setiap ketergantungan penting, lalu menyusun satu skenario uji pemulihan beserta jadwalnya. Tim memakai organisasi nyata dengan izin tertulis dan data yang dianonimkan, atau memakai kasus Medikara Health.

Pembagian peran. Rencana DR yang baik membutuhkan angka bisnis dan rancangan teknis dalam satu dokumen. Tabel 12.7 membagi peran agar setiap anggota menyumbang sesuai latar belakangnya.

Langkah kerja.

1. Ambil dua sampai tiga layanan kritis dari Bab 11, beserta SLO-nya. Catat ketergantungan setiap layanan: basis data, layanan identitas, integrasi mitra, dan jaringan.
2. Lakukan analisis dampak bisnis sederhana dengan lima pertanyaan pada

Tabel 12.7: Pembagian peran dalam Aktivitas Bab 12

Peran	Diisi oleh	Tugas
Pemimpin analisis dampak bisnis	Anggota non-teknis	Mewawancarai pemilik bisnis; menghitung kerugian per jam, dampak keselamatan pasien, dan kewajiban regulasi
Perancang resiliensi	Anggota teknis	Memetakan ketergantungan dan memilih pola *timeout*, *retry*, *circuit breaker*, *bulkhead*, dan *graceful degradation*
Perancang DR	Anggota teknis	Memilih strategi DR, lokasi kedua, dan cara penyalinan data; memperkirakan biayanya
Penyusun skenario uji	Seluruh anggota, satu orang sebagai fasilitator	Menyusun urutan kejadian, kriteria berhasil, dan jadwal uji pemulihan

seksi 12.5. Isi lembar "Isian Tim" pada templat analisis dampak bisnis.

3. Tetapkan RTO dan RPO setiap layanan bersama pemilik bisnis. Pastikan RTO lebih pendek dari waktu henti maksimum yang dapat ditoleransi.
4. Pilih strategi DR dan lokasi kedua untuk setiap layanan, dengan memperhatikan kendala lokasi data dari Bab 2. Perkirakan biaya DR per tahun dan bandingkan dengan perkiraan kerugian. Tuliskan pola resiliensi untuk setiap ketergantungan penting.
5. Susun satu skenario uji pemulihan, misalnya *region* utama tidak tersedia selama empat jam. Jalankan sebagai latihan meja, catat temuannya, lalu tetapkan jadwal uji berikutnya untuk satu tahun ke depan.

Luaran dan kaitan capstone. Luarannya adalah rencana resiliensi dan DR untuk layanan kritis, lembar analisis dampak bisnis yang sudah diisi, satu skenario uji beserta catatan hasil latihan meja, dan jadwal uji pemulihan. Keempatnya masuk ke tonggak C4 Peta Jalan dan Tata Kelola. Biaya DR menjadi masukan peta jalan modernisasi di Bab 14, jadwal uji pemulihan menjadi salah satu kontrol tata kelola yang diperiksa secara berkala, dan risiko yang ditemukan masuk ke *risk register* serta rencana kelangsungan bisnis di IT40813.

Kriteria penilaian. Tabel 12.8 memuat kriteria dan bobotnya.

Berkas pendukung. Templat analisis dampak bisnis dengan lembar contoh Medikara, lembar isian tim, dan perhitungan biaya otomatis ada di `sources/chapter12/templat-analisis-dampak-bisnis.xlsx`. Templat rencana beserta daftar periksanya ada di `sources/chapter12/templat-rencana-dr.md`. Contoh skenario latihan meja, templat skenario tim, dan syarat aman ada di `sources/chapter12/skenario-game-day.md`. Skrip untuk pendalaman teknis dan Latihan 3 ada di `sources/chapter12/simulasi-circuit-breaker.py` dan `sources/chapter12/bandingkan-strategi-dr.py`.

Tabel 12.8: Kriteria penilaian Aktivitas Bab 12

Kriteria	Bobot	Tanda hasil yang baik
Mutu analisis dampak bisnis	30%	Kerugian, keselamatan, dan regulasi dihitung dengan dasar yang disebut; pemilik bisnis terlibat
Ketepatan RTO, RPO, dan strategi	30%	Target berasal dari dampak; strategi termurah yang memenuhi target dipilih, dan alasannya ditulis
Pola resiliensi	20%	Setiap ketergantungan penting punya batas waktu, aturan pengulangan, dan perilaku saat gagal yang disepakati
Skenario dan jadwal uji	20%	Skenario realistis, kriteria berhasil terukur, temuan latihan meja dicatat beserta pemiliknya

Etika. Angka kerugian dan catatan gangguan dari organisasi nyata dipakai seizin organisasi. Nama orang, data pasien, alamat server, dan kunci akses dihapus sebelum dibagikan ke anggota tim. Latihan meja tidak menyentuh sistem. Pemindahan nyata hanya dilakukan pada sistem milik sendiri atau yang sudah berizin tertulis, setelah syarat aman pada Tabel 12.6 terpenuhi.

Pendalaman teknis (opsional). Anggota teknis menjalankan `sources/chapter12/simulasi-circuit-breaker.py` seperti dijelaskan di kotak pendalaman seksi 12.3, lalu mengganti ambang dan jangka terbuka sesuai data latensi ketergantungan tim. Hasilnya dipakai untuk mengisi kolom *circuit breaker* pada templat rencana DR.

12.9 Ringkasan

Sistem terdistribusi jarang gagal secara total. Yang lebih sering terjadi adalah kegagalan parsial, latensi tinggi, kegagalan berantai, titik kegagalan tunggal yang tersembunyi, dan kegagalan dari pihak ketiga. Latensi tinggi paling licin, karena jumlah permintaan yang menunggu sama dengan laju dikali lama menunggu. Layanan kepesertaan yang melambat dari 0,8 menjadi 30 detik menaikkan permintaan yang menunggu dari sekitar 2 menjadi 60, cukup untuk menghabiskan kolam 50 koneksi dan memanjangkan antrean loket.

Lima pola resiliensi menjawab mode kegagalan itu. *Timeout* mengubah lambat menjadi gagal cepat. *Retry* dengan *backoff* dan *jitter* menangani gangguan sesaat, tetapi harus dibatasi karena tiga lapis pengulangan dapat melipatgandakan satu tindakan menjadi 64 percobaan. *Circuit breaker* menghentikan panggilan ke layanan yang sudah pasti gagal. *Bulkhead* menyekat sumber daya. *Graceful degradation* menjaga layanan inti tetap berjalan, dengan syarat fungsi keselamatan pasien tidak pernah dilewati.

Multi-AZ melindungi dari padamnya satu pusat data, sedangkan perlindungan

dari gangguan seluruh *region* membutuhkan lokasi kedua. Karena setiap penyedia global hanya punya satu *region* di Indonesia, lokasi kedua untuk rekam medis harus dicari di penyedia lain, pusat data lokal, atau fasilitas sendiri. Data 15 terabyte yang butuh sekitar 33 jam untuk dikirim menunjukkan bahwa data harus sudah berada di lokasi kedua sebelum bencana.

RTO dan RPO adalah keputusan bisnis yang lahir dari analisis dampak, dibatasi oleh waktu henti maksimum yang dapat ditoleransi. Empat strategi DR, dari *backup and restore* sampai *multi-site active-active*, menukar biaya dengan kecepatan pulih. Bagi Medikara, rekam medis memakai *warm standby*, Medikara+ memakai *pilot light*, dan ERP keuangan memakai *backup and restore*, dengan total biaya sekitar Rp724 juta per tahun, jauh di bawah Rp3,25 miliar bila semuanya *active-active*. Cadangan yang lengkap, terpisah, tidak dapat diubah, terenkripsi, dan teruji adalah pertahanan terakhir terhadap *ransomware*, seperti diperlihatkan kasus PDNS 2 yang hanya 2 persen datanya memiliki cadangan.

Chaos engineering dan *game day* mengubah rencana menjadi bukti, dengan cakupan dampak yang kecil, kriteria berhenti yang jelas, dan izin tertulis. Latihan meja sudah cukup untuk menemukan ketergantungan yang terlewat, seperti layanan identitas yang membuat RTO 1 jam meleset menjadi 80 menit. Pesan utama bab ini: resiliensi tidak dibeli sekaligus untuk semua layanan, melainkan ditetapkan per layanan dari dampak bisnisnya, lalu dibuktikan lewat uji pemulihan yang terjadwal.

12.10 Latihan

Latihan 1: RTO dan RPO pada Rekam Medis Elektronik

Latihan ini membuktikan bahwa RTO dan RPO adalah dua ukuran yang berbeda arah, dan bahwa satu rancangan dapat memenuhi target pada satu jenis kejadian tetapi gagal pada kejadian lain. Target rekam medis elektronik Medikara adalah RTO 1 jam dan RPO 5 menit.

1. Tuliskan target RTO dan RPO layanan, lalu jelaskan dengan kalimat sendiri arti masing-masing bagi petugas IGD.
2. Susun tiga skenario kejadian yang berbeda jenis, misalnya satu AZ padam, kesalahan operator, dan *ransomware*.
3. Untuk setiap skenario, catat waktu kejadian, titik pemulihan terakhir yang bersih, dan waktu layanan kembali melayani.
4. Hitung data yang hilang (RPO nyata) dan waktu henti (RTO nyata), lalu bandingkan dengan target.
5. Bandingkan hasil dengan contoh pada Tabel 12.9, lalu isi Tabel 12.10.

Tabel 12.9: Latihan 1, contoh terisi: tiga skenario untuk rekam medis elektronik Medikara dengan target RTO 1 jam dan RPO 5 menit (rekaan)

Skenario dan waktu kejadian	Titik pemulihan bersih terakhir	Pulih pukul	RPO nyata	RTO nyata	Memenuhi?
AZ 1 padam pukul 10.00	10.00, replikasi sinkron ke AZ 2	10.04	0 menit	4 menit	Ya
Operator menghapus tabel pukul 14.20	14.15, pemulihan ke titik waktu	15.05	5 menit	45 menit	Ya, tepat di batas RPO
Ransomware mengenkripsi basis data pukul 02.00; replikasi ikut tersalin	01.00, cadangan per jam yang tidak dapat diubah	11.00	60 menit	9 jam	Tidak, keduanya meleset

Tabel 12.10: Latihan 1, lembar isian: tiga skenario untuk layanan pilihan tim

Skenario dan waktu kejadian	Titik pemulihan bersih terakhir	Pulih pukul	RPO nyata	RTO nyata	Memenuhi?
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…

Pertanyaan analisis:

1. Pada contoh, skenario *ransomware* meleset di RPO dan RTO sekaligus, padahal replikasi sinkron sudah terpasang. Jelaskan penyebabnya berdasarkan isi tabel.
2. Bila cadangan yang tidak dapat diubah dibuat setiap 5 menit, bukan setiap jam, kolom mana pada skenario ketiga yang berubah, dan kolom mana yang tetap? Apa yang masih harus diperbaiki untuk memenuhi RTO?

Latihan 2: Pelajaran Arsitektur dan Tata Kelola dari PDNS 2

Latihan ini membuktikan kemampuan mengubah fakta insiden menjadi keputusan arsitektur dan tata kelola yang dapat diterapkan di organisasi lain. Pada insiden PDNS 2 tahun 2024, hanya sebagian kecil data yang memiliki cadangan. Tugasnya adalah menurunkan pelajaran dari fakta yang dapat dirujuk, bukan dari dugaan.

1. Baca ketiga sumber berita yang dirujuk bab ini [23, 39, 153]. Catat setiap fakta beserta angkanya dan sumbernya.
2. Pisahkan fakta dari pendapat. Pernyataan pejabat dicatat sebagai pernyataan, bukan sebagai hasil audit.
3. Untuk setiap fakta, tulis satu pelajaran arsitektur dan satu pelajaran tata

kelola.

4. Terapkan setiap pelajaran pada Medikara atau organisasi pilihan tim, dalam bentuk tindakan yang dapat diperiksa.
5. Bandingkan hasil dengan contoh pada Tabel 12.11, lalu isi Tabel 12.12.

Tabel 12.11: Latihan 2, contoh terisi: fakta PDNS 2 dan pelajarannya bagi Medikara Health

Fakta dan sumber	Pelajaran arsitektur	Pelajaran tata kelola	Tindakan di Medikara
Hanya 2 persen data memiliki cadangan [153]	Cadangan menjadi bagian rancangan setiap layanan, bukan pilihan pengguna	Cakupan cadangan diperiksa berkala dan dilaporkan ke komite audit	Daftar 46 aplikasi dicocokkan dengan daftar cadangan setiap triwulan
Upaya mematikan fitur keamanan terdeteksi sejak 17 Juni, enkripsi mulai 20 Juni [23]	Perubahan pada kontrol keamanan dipantau sebagai sinyal bahaya	Ada jalur eskalasi yang jelas untuk sinyal keamanan di luar jam kerja	Peringatan "antivirus dimatikan" masuk ke petugas jaga dalam 15 menit
282 instansi pengguna terdampak sekaligus [39]	Konsentrasi pada satu lokasi diimbangi lokasi kedua dan cadangan terpisah	Pembagian tanggung jawab cadangan antara pengelola dan pengguna ditulis di kontrak	Kontrak penyedia memuat siapa yang mencadangkan, menguji, dan memulihkan

Tabel 12.12: Latihan 2, lembar isian: fakta PDNS 2 dan pelajarannya bagi organisasi pilihan tim

Fakta dan sumber	Pelajaran arsitektur	Pelajaran tata kelola	Tindakan di organisasi
…	…	…	…
…	…	…	…
…	…	…	…

Pertanyaan analisis:

1. Dari pelajaran yang diisi, mana yang paling murah diterapkan dan mana yang paling besar pengaruhnya terhadap kemampuan pulih? Apakah keduanya pelajaran yang sama?
2. Penyerang sudah bergerak tiga hari sebelum enkripsi. Bila cadangan hanya disimpan 2 hari, apa akibatnya bagi titik pemulihan yang bersih? Hubungkan dengan masa kunci cadangan.
3. Pelajaran mana yang tidak dapat diselesaikan oleh tim TI sendiri dan membutuhkan keputusan direksi atau perubahan kontrak?

Latihan 3: Mengapa Active-Active Tidak Untuk Semua Layanan

Latihan ini membuktikan bahwa *multi-site active-active* belum tentu tepat untuk semua layanan Medikara Health, dengan menghitung biaya DR dan perkiraan

kerugian tahunan untuk keempat strategi. Total biaya tahunan adalah biaya DR ditambah perkiraan kerugian tahunan. Perkiraan kerugian tahunan adalah frekuensi kejadian besar per tahun dikali RTO strategi dikali kerugian per jam. Misalkan frekuensi kejadian besar, termasuk *ransomware*, kesalahan konfigurasi besar, dan gangguan *region*, adalah 0,5 kali per tahun.

1. Pilih satu layanan dan ambil kerugian per jam, biaya infrastruktur produksi, serta target RTO dan RPO dari Tabel 12.4 atau dari analisis tim.
2. Hitung biaya DR setiap strategi dengan rasio biaya pada Tabel 12.3.
3. Hitung perkiraan kerugian tahunan setiap strategi dengan RTO kasar pada tabel yang sama.
4. Jumlahkan keduanya, lalu tandai strategi yang memenuhi RTO dan RPO target. Skrip `sources/chapter12/bandingkan-strategi-dr.py` dapat dipakai untuk memeriksa hitungan.
5. Bandingkan hasil dengan contoh pada Tabel 12.13, lalu isi Tabel 12.14.

Tabel 12.13: Latihan 3, contoh terisi: rekam medis elektronik Medikara, kerugian Rp150 juta per jam, biaya produksi Rp1,2 miliar per tahun, target RTO 1 jam dan RPO 5 menit (juta rupiah per tahun, rekaan)

Strategi	RTO (jam)	Biaya DR	Perkiraan kerugian	Total	Memenuhi target?
Backup and restore	12	96	900	996	Tidak
Pilot light	2	240	150	390	Tidak
Warm standby	0,5	540	37,5	577,5	Ya
Multi-site active-active	0,1	1.500	7,5	1.507,5	Ya

Tabel 12.14: Latihan 3, lembar isian: empat strategi untuk layanan pilihan tim (juta rupiah per tahun)

Strategi	RTO (jam)	Biaya DR	Perkiraan kerugian	Total	Memenuhi target?
Backup and restore	...	...	...	...	...
Pilot light	...	...	...	...	...
Warm standby	...	...	...	...	...
Multi-site active-active	...	...	...	...	...

Pertanyaan analisis:

1. Pada contoh, total terendah dimiliki *pilot light* (390), tetapi strategi yang memenuhi target adalah *warm standby* (577,5). Siapa yang seharusnya memutuskan apakah selisih Rp187,5 juta per tahun itu layak dibayar, dan atas dasar apa?
2. Ulangi hitungan untuk ERP keuangan dengan angka pada Tabel 12.4. Berapa selisih total antara *multi-site active-active* dan strategi termurah yang

memenuhi target? Apa artinya bagi usulan "semua layanan *active-active*"?

3. Bila frekuensi kejadian besar naik menjadi 2 kali per tahun, apakah urutan strategi untuk rekam medis berubah? Tunjukkan perhitungannya.

Bab 13

DevOps, GitOps, dan Platform Engineering

Tujuan Pembelajaran

- Menjelaskan praktik DevOps dan metrik DORA (DevOps Research and Assessment) untuk mengukur kinerja pengiriman perangkat lunak.
- Menjelaskan alur CI/CD (*Continuous Integration / Continuous Delivery*) dan empat prinsip GitOps.
- Merancang topologi tim dan peran platform internal bagi organisasi.

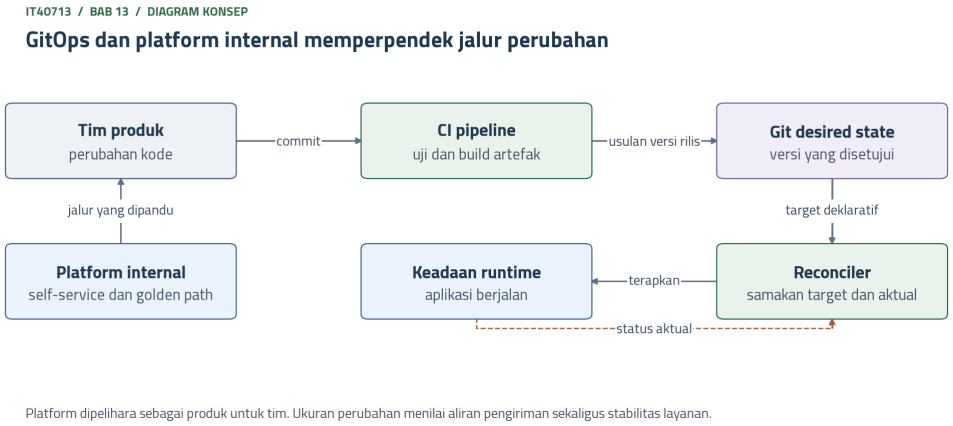
13.1 Pendahuluan

Pada Senin, 15 Desember 2025, bagian pendaftaran RS Medikara Jakarta meminta satu perubahan kecil pada SIMRS (Sistem Informasi Manajemen Rumah Sakit): kolom nomor telepon keluarga yang dapat dihubungi dalam keadaan darurat. Pengembang memperkirakan pemrogramannya selesai dalam dua hari. Kolom itu baru muncul di layar petugas pendaftaran pada Sabtu malam, 21 Maret 2026, atau 96 hari kemudian. Di antara dua tanggal itu, permintaan tersebut melewati tiket layanan bantuan, persetujuan kepala instalasi rekam medis, rapat komite perubahan bulanan, antrean pengembang, uji manual, uji regresi triwulanan, dan jendela rilis tiga bulanan. Waktu kerja yang sesungguhnya sekitar sepuluh hari. Sisanya, 86 hari, habis untuk menunggu.

Kisah ini memperlihatkan masalah yang tidak selesai dengan teknologi baru. Medikara Health dapat memindahkan SIMRS ke *cloud*, menatanya menjadi modul (Bab 7), dan menjalankan layanan barunya di *container* (Bab 8). Bila setiap per-

ubahan tetap menunggu 96 hari, pasien dan petugas tidak merasakan bedanya. Arsitektur *cloud-native* menjanjikan perubahan yang sering dan dapat diprediksi (Bab 5). Janji itu hanya terwujud bila cara organisasi mengirim perubahan ikut berubah: siapa yang menyetujui, apa yang diotomatisasi, bagaimana tim disusun, dan siapa yang menyediakan jalan bagi tim lain.

Gambar 13.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 13.1: GitOps dan platform internal memperpendek jalur perubahan

Bab ini disusun dalam lima langkah: DevOps dan metrik DORA untuk mengukur pengiriman, CI/CD dan strategi rilis, Infrastructure as Code (IaC) dan GitOps, *platform engineering* dan Internal Developer Platform (IDP), serta Team Topologies. Rujukan utamanya adalah laporan DORA 2025 [45], panduan metrik DORA [46], buku Forsgren, Humble, dan Kim [56], buku Humble dan Farley tentang *continuous delivery* [74], prinsip OpenGitOps [114], *Platforms White Paper* dari Cloud Native Computing Foundation (CNCF) [37], dan buku Team Topologies edisi kedua [131]. Hasil bab ini adalah model operasi *delivery* dan peta jalan platform internal untuk tonggak capstone C4 Peta Jalan dan Tata Kelola.

13.2 DevOps dan Metrik DORA

Pertanyaan pertama: bagaimana mengetahui apakah cara sebuah organisasi mengirim perubahan sudah baik? Kesan rapat tidak cukup. Organisasi membutuhkan istilah yang sama dan ukuran yang dapat dihitung dari data.

13.2.1 DevOps sebagai Budaya dan Praktik

DevOps adalah cara kerja yang menyatukan tanggung jawab membangun perangkat lunak (*development*) dan menjalankannya (*operations*). Pada cara kerja lama, kedua pekerjaan itu dipisah. Tim pengembang dinilai dari fitur yang selesai, sedangkan tim operasi dinilai dari sistem yang stabil. Akibatnya kedua tim punya kepentingan yang berlawanan: yang satu ingin sering berubah, yang lain ingin jarang berubah. DevOps bukan nama jabatan dan bukan nama alat. Forsgren, Humble, dan Kim merangkum hasil riset bertahun-tahun menjadi 24 kapabilitas dalam lima kelompok: *continuous delivery*, arsitektur, produk dan proses, manajemen ramping dan pemantauan, serta budaya [56]. Kapabilitas itu bersama-sama mendorong kinerja pengiriman perangkat lunak, yang kemudian berkaitan dengan kinerja organisasi.

Analoginya seperti rumah makan yang dapurnya dipisah tembok dari ruang makan. Juru masak hanya menerima kertas pesanan dan tidak pernah mendengar keluhan tamu. Pelayan mendengar keluhan, tetapi tidak boleh masuk dapur. Ketika sup terlalu asin, keluhan itu baru sampai ke dapur keesokan harinya lewat catatan manajer. DevOps membuka pintu di tembok itu: juru masak ikut melihat piring yang kembali, dan pelayan tahu kapan menu baru siap.

Dua contoh di Medikara Health:

1. Misalkan tim pengembang SIMRS Jakarta dinilai dari jumlah permintaan yang selesai, sedangkan tim infrastruktur pusat dinilai dari jam henti layanan. Sepanjang 2025 tim infrastruktur menolak 9 dari 31 permintaan pemasangan di luar jendela rilis, dan sembilan perbaikan kecil itu menunggu rata-rata 47 hari. Kedua tim benar menurut ukurannya.
2. Vendor Medikara+ menjalankan 37 langkah manual setiap memasang versi baru (Bab 5). Misalkan laporan gangguan sampai ke pengembang vendor lewat surel layanan bantuan, rata-rata dua hari setelah gangguan. Kesalahan yang sama pun terulang.

13.2.2 Metrik DORA

DORA adalah program riset tentang kemampuan organisasi mengirim perangkat lunak. DORA mengukur kinerja pengiriman dengan metrik yang dihitung dari data, bukan dari kesan. Panduan DORA yang terbaru memuat lima metrik dalam dua kelompok [46]. Kelompok pertama, *throughput*, mengukur kecepatan aliran perubahan. Kelompok kedua, *instability*, mengukur seberapa sering perubahan menimbulkan masalah. Empat metrik pertama dikenal sebagai empat kunci (*four keys*) dan sudah dipakai sejak buku Forsgren, Humble, dan Kim [56]. Metrik kelima, tingkat pengerjaan ulang, ditambahkan kemudian. Tabel 13.1 merangkum

kelimanya.

Tabel 13.1: Lima metrik DORA dan cara menghitungnya dari data organisasi, diringkas dari panduan DORA [46]

Metrik	Kelompok	Pertanyaan yang dijawab	Cara menghitung
Lead time perubahan (*change lead time*)	*Throughput*	Berapa lama perubahan yang sudah ditulis menunggu sampai dipakai?	Selisih waktu *commit* dan waktu berjalan di produksi, dirata-rata per perubahan
Frekuensi *deployment* (*deployment frequency*)	*Throughput*	Seberapa sering perubahan sampai ke pengguna?	Jumlah *deployment* ke produksi per bulan, dari catatan rilis
Waktu pemulihan (*failed deployment recovery time*)	*Throughput*	Berapa lama layanan terganggu bila *deployment* gagal?	Waktu dari *deployment* gagal sampai layanan pulih, dari catatan insiden
Tingkat kegagalan perubahan (*change fail rate*)	*Instability*	Berapa bagian *deployment* yang langsung butuh perbaikan?	*Deployment* yang butuh tindakan segera dibagi seluruh *deployment*
Tingkat pengerjaan ulang (*deployment rework rate*)	*Instability*	Berapa bagian *deployment* yang tidak direncanakan karena insiden?	*Deployment* darurat akibat insiden dibagi seluruh *deployment*

Analoginya seperti menilai jasa kurir. Pelanggan peduli pada empat hal: seberapa sering kurir berangkat, berapa lama paket sampai, berapa banyak paket yang rusak, dan berapa lama penggantinya dikirim bila paket rusak. Kurir yang berangkat sebulan sekali dengan truk penuh belum tentu lebih aman daripada kurir yang berangkat setiap hari dengan sepeda motor. Ukurannya harus dibaca bersama, bukan satu per satu.

Dua hal pada *lead time* sering tertukar. Pembagiannya begini. *Lead time* versi DORA dihitung sejak *commit*, yaitu sejak perubahan ditulis dan disimpan di sistem kendali versi. Manajemen dan pengguna merasakan *lead time* permintaan, yaitu sejak permintaan diajukan. Pada kisah kolom nomor telepon keluarga, 52,5 hari dari 96 hari habis sebelum pemrograman dimulai. Bagian itu tidak terlihat dalam metrik DORA. Kedua ukuran dibutuhkan: *lead time* DORA untuk menilai jalur teknis, dan *lead time* permintaan untuk menilai antrean persetujuan dan prioritas. Aktivitas bab ini memetakan keduanya.

13.2.3 Rilis yang Jarang Tidak Membuat Lebih Aman

Banyak organisasi menganggap rilis yang jarang lebih aman: makin jarang sistem disentuh, makin kecil peluang rusak. Riset DORA berulang kali menunjukkan bahwa kecepatan dan stabilitas bukan pilihan yang saling meniadakan [46]. Organisasi

berkinerja tinggi merilis lebih sering, sekaligus lebih jarang gagal dan lebih cepat pulih [56]. Penjelasannya ada pada ukuran rilis. Rilis yang jarang selalu berukuran besar, karena perubahan menumpuk selama menunggu jendela rilis berikutnya.

Analoginya seperti mencuci piring. Rumah tangga yang mencuci piring setiap selesai makan hanya menghadapi beberapa piring. Sisa makanan masih lunak, dan piring yang retak langsung ketahuan. Rumah tangga yang menumpuk piring selama seminggu menghadapi tumpukan besar dengan sisa makanan yang sudah mengeras, dan piring retak baru ketahuan ketika tumpukan dibongkar.

Hitungan sederhana memperlihatkan pengaruh ukuran rilis. Misalkan setiap perubahan punya peluang 1 persen menimbulkan masalah di produksi. Peluang satu rilis berisi n perubahan bebas masalah adalah $0{,}99^n$. Hasilnya ada di Tabel 13.2.

Tabel 13.2: Pengaruh ukuran rilis bila setiap perubahan berpeluang 1 persen menimbulkan masalah (asumsi pengajaran)

Perubahan per rilis	**Peluang rilis bermasalah**	**Perubahan yang dicurigai bila gagal**	**Perubahan baik yang ikut dibatalkan bila *rollback***
60 (rilis triwulanan)	$1 - 0{,}99^{60} \approx 45{,}3$ persen	60	59
8 (rilis bulanan)	$1 - 0{,}99^{8} \approx 7{,}7$ persen	8	7
1 (rilis per perubahan)	1 persen	1	0

Tabel itu punya tiga konsekuensi. Pertama, rilis besar hampir separuhnya bermasalah walaupun setiap perubahan dibuat dengan hati-hati. Kedua, tim harus mencari satu penyebab di antara 60 perubahan, sehingga pemulihan lama. Ketiga, membatalkan rilis besar ikut membatalkan 59 perubahan yang baik, sehingga tim memilih memperbaiki di tempat pada malam hari.

13.2.4 Temuan DORA 2025: AI sebagai Penguat

Laporan DORA 2025, *State of AI-assisted Software Development*, disusun dari survei terhadap hampir 5.000 profesional teknologi. Sekitar 90 persen responden memakai kecerdasan artifisial (*Artificial Intelligence*, AI) dalam pekerjaannya [45]. Temuan utamanya: peran utama AI adalah penguat (*amplifier*). AI memperbesar kekuatan dan kelemahan yang sudah ada di organisasi. Hasil terbesar dari investasi AI datang dari perhatian pada sistem organisasi yang mendasarinya, bukan pada alatnya. Laporan itu juga memperkenalkan DORA AI Capabilities Model berisi tujuh kapabilitas dasar yang memperkuat dampak positif AI, serta tujuh profil tim untuk membaca kinerja pengiriman [45].

Analoginya seperti memberi kurir mobil yang lebih cepat, padahal jalurnya macet di gerbang tol. Mobil yang lebih cepat tidak memperpendek antrean di gerbang. Kurir hanya sampai lebih cepat di ujung antrean.

Dua contoh hitungan di Medikara Health:

1. Misalkan asisten pemrograman AI memangkas pemrograman kolom nomor telepon keluarga dari dua hari menjadi satu hari. *Lead time* permintaan hanya turun dari 96 menjadi 95 hari, karena hambatannya ada di persetujuan dan antrean.
2. Misalkan dengan AI pengembang menghasilkan 30 persen lebih banyak perubahan, sementara jendela rilis tetap triwulanan. Isi rilis naik dari 60 menjadi 78 perubahan, dan peluang rilis bermasalah naik menjadi $1 - 0{,}99^{78} \approx 54{,}3$ persen. Kelemahan yang sudah ada justru diperkuat.

Kasus Medikara Health: Metrik DORA SIMRS Jakarta dan Medikara+

Tim transformasi menghitung metrik DORA dari catatan rilis dan insiden 2025 (data contoh di `sources/chapter13/contoh-riwayat-deployment.csv`, dihitung dengan `sources/chapter13/hitung-metrik-dora.py`). Hasilnya ada di Tabel 13.3. SIMRS Jakarta dirilis tiga bulan sekali pada Sabtu malam dengan henti layanan sekitar empat jam (Bab 7). Medikara+ dirilis vendor sebulan sekali lewat 37 langkah manual (Bab 5).

Hasil ini punya dua konsekuensi. Pertama, data Medikara sendiri tidak mendukung anggapan rilis jarang lebih aman. SIMRS, yang paling jarang dirilis, justru paling sering gagal (43 persen), dengan pemulihan median 9 jam karena membatalkan rilis berisi sekitar 60 perubahan menuntut pemulihan basis data. Kedua, Medikara+ lebih baik, tetapi masih bergantung pada jadwal vendor dan langkah manual. Sasaran awal untuk C4: *deployment* SIMRS bulanan lalu mingguan, tingkat kegagalan di bawah 15 persen, dan pemulihan di bawah satu jam.

13.3 CI/CD dan Strategi Rilis

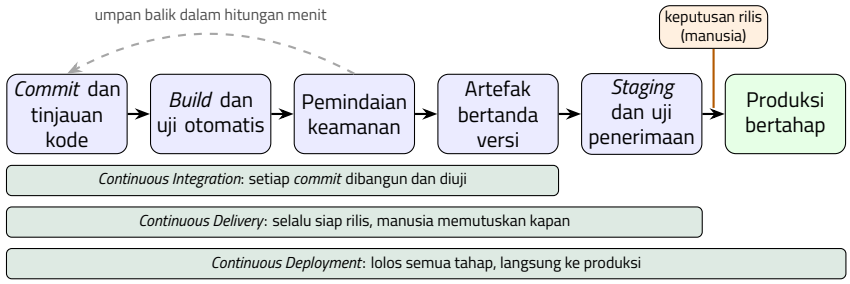
Pertanyaan berikutnya: bagaimana memperkecil ukuran rilis tanpa menambah beban kerja manual? Rilis mingguan dengan 37 langkah manual berarti 52 kali menjalankan 37 langkah dalam setahun. Jawabannya adalah otomasi jalur dari *commit* sampai produksi, yang dikenal sebagai CI/CD.

Tabel 13.3: Metrik DORA SIMRS Jakarta dan Medikara+ tahun 2025 (rekaan)

Metrik	SIMRS RS Jakarta	Medikara+
Deployment per tahun	7 (4 terjadwal, 3 darurat)	14 (12 terjadwal, 2 darurat)
Perubahan per rilis terjadwal	60,5	7,8
Lead time perubahan rata-rata	55 hari (sekitar 8 minggu)	29 hari (sekitar 4 minggu)
Tingkat kegagalan perubahan	3 dari 7 (43 persen)	2 dari 14 (14 persen)
Waktu pemulihan median	9 jam	5 jam
Tingkat pengerjaan ulang	3 dari 7 (43 persen)	2 dari 14 (14 persen)

13.3.1 Continuous Integration, Continuous Delivery, dan Continuous Deployment

Continuous integration adalah praktik menggabungkan perubahan setiap pengembang ke cabang utama sesering mungkin, sedikitnya sekali sehari. Setiap penggabungan memicu *build* dan uji otomatis. Bila hasilnya gagal, perbaikan didahulukan di atas pekerjaan lain. *Continuous delivery* melangkah lebih jauh: setiap perubahan yang lolos seluruh tahap selalu dalam keadaan siap dirilis, sehingga rilis ke produksi menjadi keputusan bisnis yang dijalankan dengan satu tombol. Humble dan Farley menyebut rangkaian tahap otomatis itu *deployment pipeline*, yaitu proses otomatis yang membawa setiap perubahan dari *commit* sampai rilis [74]. *Continuous deployment* menghapus tombol itu: setiap perubahan yang lolos langsung berjalan di produksi. Gambar 13.2 memperlihatkan tahap-tahap *pipeline* dan cakupan ketiga istilah.



Gambar 13.2: Tahap *pipeline* CI/CD dan cakupan *continuous integration*, *continuous delivery*, serta *continuous deployment*

Analoginya seperti dapur katering. Juru masak mencicipi setiap masakan begitu matang, sehingga rasa yang salah ketahuan saat itu juga (*continuous integration*). Masakan yang lolos dicicipi langsung dikemas dan diberi label, siap dikirim kapan pun pemesan meminta (*continuous delivery*). Pada katering tertentu, kotak yang sudah siap langsung dikirim tanpa menunggu telepon pemesan (*continuous deplo-*

yment).

Dua hal pada huruf CD sering tertukar. Pembagiannya begini. *Continuous delivery* mempertahankan keputusan manusia sebelum produksi, sedangkan *continuous deployment* tidak. Bagi rumah sakit, keduanya dapat dipakai bersamaan. Perubahan tampilan Medikara+ berisiko rendah dan cocok dengan *continuous deployment*. Perubahan pada modul resep atau rekam medis cocok dengan *continuous delivery*, karena rilisnya mungkin perlu ditandatangani perwakilan klinis. Yang dibuang bukan persetujuannya, melainkan pekerjaan manual dan antrean di sekitar persetujuan itu.

Dua contoh di Medikara Health:

1. Misalkan 37 langkah manual rilis Medikara+ menyita dua orang vendor selama empat jam, atau 8 jam-orang per rilis. Rilis mingguan secara manual berarti $52 \times 8 = 416$ jam-orang setahun. *Pipeline* menjalankan langkah yang sama sekitar 22 menit, kecuali satu persetujuan, sehingga rilis mingguan menjadi masuk akal.
2. Misalkan uji regresi SIMRS melibatkan enam penguji selama lima hari per rilis. Bila 70 persen skenario diotomatisasi, uji otomatis berjalan sekitar 40 menit pada setiap *commit*, dan uji manual cukup satu setengah hari untuk skenario klinis berisiko.

13.3.2 Strategi Rilis: Blue-Green dan Canary

Rilis SIMRS saat ini menghentikan layanan sekitar empat jam. Strategi rilis modern memasang versi baru tanpa henti layanan dan menyediakan jalan pulang yang cepat. Dua yang paling banyak dipakai adalah *blue-green* dan *canary*, yang keduanya dibahas Humble dan Farley [74]. Pada *blue-green*, organisasi menjalankan dua lingkungan produksi yang identik. Versi lama melayani pengguna di satu lingkungan, versi baru dipasang dan diuji di lingkungan lain, lalu lalu lintas dipindahkan sekaligus. Bila ada masalah, lalu lintas dipindahkan kembali dalam hitungan menit. Pada *canary*, versi baru lebih dahulu melayani sebagian kecil pengguna. Bila ukuran keandalannya baik, porsinya diperbesar bertahap sampai seluruh pengguna. Tabel 13.4 membandingkan keduanya dengan cara lama dan cara bergilir (*rolling*).

Analoginya ada di dunia pertunjukan dan kuliner. *Blue-green* seperti gedung pertunjukan dengan dua panggung. Panggung kedua ditata selama pertunjukan di panggung pertama berlangsung, lalu penonton cukup diputar kursinya. *Canary* seperti restoran waralaba yang mencoba menu baru di satu cabang selama dua minggu sebelum menjualnya di semua cabang.

Dua contoh di Medikara Health:

1. Pemesanan Medikara+ menerima sekitar 28.000 upaya per 30 hari (Bab 11),

Tabel 13.4: Perbandingan strategi rilis

Strategi	Cara kerja	Pembatalan	Biaya tambahan	Cocok untuk
Serentak dengan henti layanan	Sistem dihentikan, versi baru dipasang, sistem dinyalakan	Lambat, sering perlu pemulihan basis data	Rendah	Sistem kecil dengan jendela henti yang dapat diterima
Bergilir (*rolling*)	Server diganti versi baru satu per satu	Sedang, diganti ulang satu per satu	Rendah	Layanan tanpa perubahan skema data yang besar
Blue-green	Dua lingkungan identik, lalu lintas dipindah sekaligus	Cepat, lalu lintas dipindah kembali	Tinggi: dua lingkungan selama peralihan	Perubahan yang harus dapat dibatalkan dalam hitungan menit
Canary	Sebagian kecil pengguna lebih dahulu, dipantau, lalu diperluas	Cepat, dampak terbatas	Sedang: pembagian lalu lintas dan pemantauan	Layanan dengan banyak pengguna dan ukuran keandalan yang jelas

atau sekitar 933 per hari. Dengan *canary* 5 persen, sekitar 47 pemesanan per hari melewati versi baru. Misalkan versi baru menggagalkan 10 persen pemesanan: dalam satu hari hanya sekitar 5 pemesanan gagal, bukan 93. *Error budget* dari Bab 11 menjadi ukuran yang memutuskan apakah porsi *canary* boleh diperbesar.

2. *Blue-green* untuk SIMRS menghapus henti layanan empat jam, tetapi selama peralihan kedua versi memakai basis data yang sama. Setiap perubahan skema harus cocok dengan kedua versi, misalnya kolom baru ditambahkan dahulu dan kolom lama dihapus pada rilis berikutnya. Tanpa disiplin ini, pembatalan cepat hanya ada di atas kertas.

13.3.3 Keamanan Rantai Pasok Perangkat Lunak

Pipeline yang mampu memasang perubahan ke produksi juga menjadi sasaran serangan. Bila penyerang dapat mengubah *pipeline*, menyisipkan pustaka palsu, atau mengganti *image*, perubahan berbahaya ikut terpasang dengan cara yang terlihat sah. Kerangka Supply-chain Levels for Software Artifacts (SLSA) menyusun standar dan kontrol untuk mencegah pengubahan tanpa izin dan menjaga keutuhan paket perangkat lunak, bertahap lewat beberapa tingkat [132]. Tiga kontrol awal yang relevan bagi Medikara adalah: hak mengubah *pipeline* dibatasi dan rahasia tidak disimpan di kode; setiap *image* di produksi dapat dibuktikan berasal dari *pipeline* resmi; dan pustaka pihak ketiga dipindai sebelum dipakai. Insiden

kebocoran data pasien lewat vendor pada 2025 mengingatkan bahwa rantai pasok tidak berhenti di batas organisasi. Topik ini dibahas rinci di mata kuliah IT40813.

Pendalaman Teknis (opsional): Pipeline CI dengan GitHub Actions

GitHub Actions menjalankan *workflow*, yaitu proses otomatis yang dapat dikonfigurasi dan berisi satu atau beberapa *job*. *Workflow* dipicu oleh *event* di repositori, misalnya *pull request*, dan ditulis sebagai berkas YAML di folder `.github/workflows` [65]. Listing 13.1 membangun layanan jadwal dokter dari Bab 8. Urutannya mengikuti Gambar 13.2: periksa sintaks, bangun *image* bertanda *hash commit*, jalankan uji asap (*smoke test*) terhadap alamat `/sehat`, lalu kirim *image* ke *registry* hanya dari cabang `main`.

Listing 13.1: Workflow CI layanan jadwal dokter, tersedia di `sources/chapter13/contoh-pipeline-gitops/ci-jadwal-dokter.yml`

```yaml
# Kegunaan: contoh pipeline CI Bab 13 untuk layanan jadwal dokter Bab 08.
# Pakai: taruh di .github/workflows/ pada repositori milik sendiri yang
# berisi app.py dan Dockerfile dari sources/chapter08/contoh-container.
name: ci-jadwal-dokter

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

env:
  # Ganti dengan nama organisasi sendiri, huruf kecil semua.
  IMAGE: ghcr.io/contoh-medikara/jadwal-dokter

jobs:
  uji-dan-bangun:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      # Kesalahan sintaks ditolak dalam hitungan detik, sebelum build.
      - name: Periksa sintaks
        run: python3 -m py_compile app.py
      # Tag memakai hash commit agar setiap image dapat ditelusuri.
      - name: Bangun image
        run: docker build -t "$IMAGE:${{ github.sha }}" .
      # Uji asap: container harus menjawab /sehat sebelum boleh dikirim.
      - name: Uji asap
        run: |
          docker run -d -p 8080:8080 --name uji "$IMAGE:${{ github.sha }}"
          curl --fail --retry 5 --retry-connrefused \
            http://localhost:8080/sehat
      # Image hanya dikirim dari cabang main, tidak dari pull request.
```

```
      - name: Kirim image
        if: github.event_name == 'push'
        run: |
          echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io \
            -u "${{ github.actor }}" --password-stdin
          docker push "$IMAGE:${{ github.sha }}"
```

Tag *image* memakai *hash commit*, bukan `latest`, sehingga setiap *image* dapat ditelusuri ke satu *commit*. *Pull request* hanya diuji dan tidak pernah mengirim *image*. Contoh ini dijalankan di akun milik sendiri.

13.4 Infrastructure as Code dan GitOps

CI/CD mengotomatisasi jalur perubahan aplikasi. Pertanyaan kemudian muncul: bagaimana dengan server, jaringan, basis data, dan konfigurasi tempat aplikasi itu berjalan? Bila bagian itu tetap diubah tangan lewat tiket, antrean hanya berpindah tempat.

13.4.1 Infrastructure as Code

Infrastructure as Code (IaC) adalah praktik mendefinisikan infrastruktur dalam berkas teks yang disimpan di sistem kendali versi, ditinjau, diuji, dan diterapkan secara otomatis, sama seperti kode aplikasi [105]. Berkas itu umumnya bersifat deklaratif: isinya keadaan yang diinginkan, misalnya "tiga server dengan ukuran ini di *region* Jakarta", bukan urutan perintah untuk membuatnya. Alat IaC membandingkan keadaan yang diinginkan dengan keadaan nyata, lalu melakukan perubahan yang diperlukan. Hasilnya, lingkungan dapat dibuat ulang dengan hasil yang sama, dan setiap perubahan tercatat beserta pembuat dan peninjaunya.

Analoginya seperti resep tertulis di dapur rumah makan berjaringan. Tanpa resep, rasa masakan bergantung pada juru masak yang bertugas, dan rasa di cabang Bandung berbeda dengan rasa di cabang Bekasi. Dengan resep tertulis, setiap cabang memasak dengan takaran yang sama. Bila resep diubah, perubahan itu ditulis, diberi tanggal, dan disetujui kepala dapur. Juru masak yang diam-diam menambah garam di luar resep adalah sumber penyimpangan (*drift*) yang membuat rasa tidak lagi dapat diprediksi.

Dua contoh di Medikara Health:

1. Misalkan membuat lingkungan uji SIMRS baru butuh 11 hari kerja dan empat tiket ke tim jaringan, server, basis data, dan keamanan. Dengan templat IaC yang sudah ditinjau, lingkungan yang sama siap dalam sekitar 35 menit dengan hasil identik setiap kali.

2. Misalkan audit internal menemukan 41 perubahan konfigurasi produksi tanpa tiket selama enam bulan, sebagian berupa aturan *firewall*. Dengan IaC, aturan itu hanya berubah lewat berkas yang ditinjau, sehingga pertanyaan "siapa yang mengubah" selalu terjawab.

13.4.2 Empat Prinsip OpenGitOps

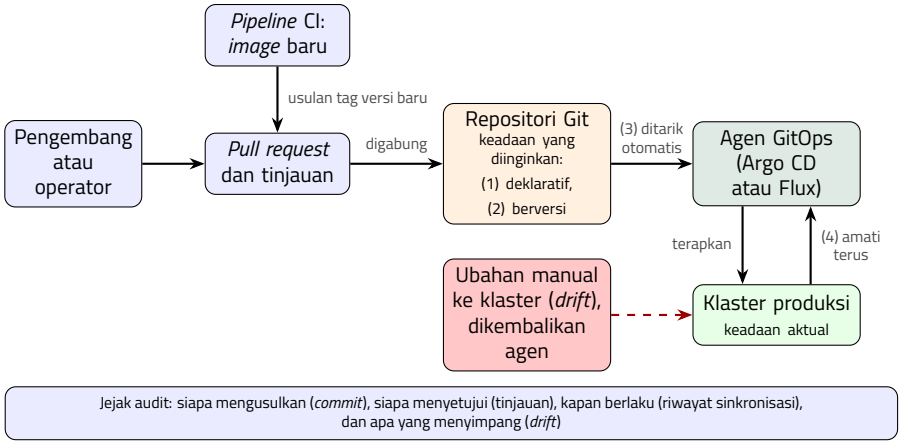
GitOps adalah cara mengoperasikan sistem dengan menjadikan repositori Git sumber kebenaran tentang keadaan yang diinginkan, lalu menyerahkan penerapannya kepada agen perangkat lunak. Kelompok kerja OpenGitOps di bawah CNCF merumuskan empat prinsip dalam versi 1.0.0 [114]. Tabel 13.5 menerjemahkan keempatnya beserta artinya bagi auditor.

Tabel 13.5: Empat prinsip OpenGitOps v1.0.0 [114] dan buktinya bagi auditor

Prinsip	Bunyi prinsip (terjemahan)	Arti praktis	Bukti bagi auditor
Deklaratif (*declarative*)	Keadaan yang diinginkan dari sistem dinyatakan secara deklaratif	Konfigurasi ditulis sebagai keadaan akhir, bukan urutan perintah	Isi berkas dapat dibaca sebagai pernyataan kontrol
Berversi dan tidak berubah (*versioned and immutable*)	Keadaan yang diinginkan disimpan dengan cara yang menjamin tidak berubah, berversi, dan riwayatnya lengkap	Setiap perubahan menjadi versi baru; versi lama tidak ditimpa	Riwayat lengkap: siapa, kapan, apa, dan siapa yang menyetujui
Ditarik otomatis (*pulled automatically*)	Agen perangkat lunak menarik deklarasi keadaan yang diinginkan dari sumbernya secara otomatis	Agen di dalam klaster mengambil perubahan; manusia tidak memasang langsung	Hak ubah langsung ke produksi dapat dicabut
Direkonsiliasi terus (*continuously reconciled*)	Agen terus mengamati keadaan aktual dan berusaha menerapkan keadaan yang diinginkan	Perubahan manual di luar Git terdeteksi dan dikembalikan	Laporan penyimpangan dan riwayat sinkronisasi

Gambar 13.3 memperlihatkan siklusnya. Usulan perubahan masuk lewat *pull request*, ditinjau, lalu digabungkan ke repositori. Agen GitOps seperti Argo CD atau Flux menarik isi repositori, menerapkannya ke klaster, lalu terus membandingkan keadaan aktual dengan isi repositori.

Analoginya seperti pendingin ruangan dengan termostat. Penghuni menetapkan suhu yang diinginkan, misalnya 24 derajat. Sensor terus membandingkan suhu ruangan dengan angka itu. Bila seseorang membuka jendela dan ruangan



Gambar 13.3: Siklus GitOps: perubahan diusulkan lewat Git, ditarik agen, dan direkonsiliasi terus dengan keadaan aktual klaster

memanas, mesin bekerja lebih keras sampai suhu kembali ke 24 derajat. Penghuni tidak perlu menyalakan dan mematikan mesin dengan tangan. Bila suhu yang diinginkan perlu diubah, yang diubah adalah angka di termostat, bukan kipas mesinnya.

Dua hal pada GitOps sering tertukar. Pembagiannya begini. *Pipeline* CI/CD biasa bekerja dengan cara dorong (*push*): *pipeline* di luar klaster memasang perubahan ke produksi, sehingga *pipeline* itu harus memegang kunci akses produksi. GitOps bekerja dengan cara tarik (*pull*): agen di dalam klaster yang mengambil perubahan, sehingga kunci akses produksi tidak perlu keluar dari klaster. Cara dorong juga tidak memperhatikan apa yang terjadi setelah pemasangan. Cara tarik terus merekonsiliasi, sehingga perubahan manual tidak bertahan.

13.4.3 Jejak Audit dari GitOps

Bagi auditor, nilai terbesar GitOps adalah jejak audit yang terbentuk sebagai hasil sampingan kerja sehari-hari. Setiap perubahan produksi meninggalkan empat bukti. Pertama, *commit* mencatat siapa mengusulkan, kapan, dan apa isinya. Kedua, tinjauan *pull request* mencatat siapa yang menyetujui dan apa komentarnya. Ketiga, riwayat sinkronisasi agen mencatat kapan versi itu berlaku di produksi. Keempat, laporan penyimpangan mencatat perubahan yang tidak melalui Git. Argo CD, misalnya, menandai aplikasi yang keadaan aktualnya menyimpang dari Git sebagai `OutOfSync` dan menyediakan jejak audit untuk *event* aplikasi dan panggilan API (Application Programming Interface) [24].

Jejak itu punya batas. Repositori Git menjadi kunci baru, sehingga hak tulis dan aturan peninjauan di repositori harus dijaga seketat hak akses produksi. Rahasia

seperti kata sandi basis data tidak boleh disimpan polos di Git. Akses darurat (*break-glass*) tetap dibutuhkan untuk insiden, tetapi setiap pemakaiannya wajib tercatat dan ditinjau.

Kasus Medikara Health: GitOps sebagai Jawaban Temuan Audit Kontrol Akses

Audit internal Medikara menemukan kontrol akses yang lemah. Misalkan 17 akun dari vendor, tim TI rumah sakit, dan tim pusat memegang hak administrator langsung di server produksi, dan dari 41 perubahan konfigurasi tanpa tiket selama enam bulan, 9 tidak dapat ditelusuri pembuatnya. Auditor meminta bukti bahwa setiap perubahan produksi diusulkan, disetujui, dan diterapkan oleh orang yang berbeda.

Tim mengusulkan GitOps untuk layanan baru di belakang Medikara+ lebih dahulu, lalu SIMRS terpadu. Hak ubah langsung dicabut dari 17 akun, kecuali 2 akun darurat yang setiap pemakaiannya memicu peringatan dan wajib ditinjau dalam 24 jam. Setiap perubahan masuk lewat *pull request* dengan dua peninjau, salah satunya dari tim lain.

Usulan ini punya dua konsekuensi. Pertama, auditor mengambil sampel langsung dari riwayat repositori, dan pertanyaan "siapa mengubah apa" terjawab dalam menit, bukan hari. Kedua, ada biaya: vendor dan tim TI rumah sakit harus belajar bekerja lewat Git, dan perubahan darurat menjadi 15 menit lewat *pull request* yang dipercepat, dibanding 2 menit lewat akses langsung. Prosedur darurat harus ditulis sebelum insiden.

Pendalaman Teknis (opsional): Menyinkronkan Manifest dengan Argo CD

Argo CD adalah alat *continuous delivery* deklaratif berbasis GitOps untuk Kubernetes. Argo CD bekerja sebagai kontroler di dalam klaster yang terus membandingkan keadaan aktual aplikasi dengan keadaan target di Git, dan dapat menerapkan perubahan secara otomatis [24]. Listing 13.2 mendefinisikan satu *Application* untuk layanan jadwal dokter. Repositori konfigurasinya berisi *manifest* `deployment.yaml` dan `service.yaml` dari Bab 8.

Listing 13.2: Application Argo CD layanan jadwal dokter, tersedia di `sources/chapter13/contoh-pipeline-gitops/application-jadwal-dokter.yaml`

```
# Kegunaan: contoh Application Argo CD Bab 13 untuk layanan jadwal dokter.
# Pakai: kubectl apply -n argocd -f application-jadwal-dokter.yaml pada
# klaster latihan sendiri (kind/minikube) yang sudah terpasang Argo CD.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: jadwal-dokter
```

```
  namespace: argocd
spec:
  project: default
  # Prinsip 1 dan 2: keadaan yang diinginkan tersimpan di Git berversi.
  source:
    repoURL: https://github.com/contoh-medikara/konfigurasi-layanan.git
    targetRevision: main
    path: jadwal-dokter/produksi
  destination:
    server: https://kubernetes.default.svc
    namespace: jadwal-dokter
  # Prinsip 3 dan 4: agen menarik perubahan dan terus merekonsiliasi.
  syncPolicy:
    automated:
      # Sumber daya yang dihapus dari Git ikut dihapus dari klaster.
      prune: true
      # Perubahan manual di klaster (drift) dikembalikan ke isi Git.
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Untuk melihat prinsip keempat, ubah jumlah replika dengan `kubectl scale`, lalu amati Argo CD mengembalikannya ke angka di Git. Flux mencapai hasil yang sama dengan `GitRepository` dan `Kustomization`. Percobaan dijalankan di klaster latihan sendiri, bukan di klaster milik organisasi lain.

13.5 Platform Engineering dan Internal Developer Platform

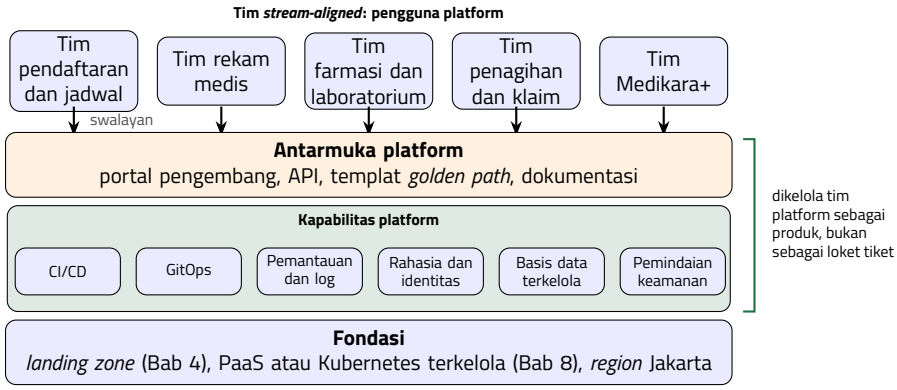
Bila setiap tim harus membangun sendiri *pipeline*, templat IaC, agen GitOps, pemantauan, dan pengelolaan rahasia, beban setiap tim berlipat dan hasilnya berbeda-beda. Pertanyaan berikutnya: siapa yang menyediakan semua itu, dan dengan cara apa?

13.5.1 Platform, IDP, dan Platform Engineering

CNCF mendefinisikan platform untuk komputasi *cloud-native* sebagai kumpulan kapabilitas terpadu yang didefinisikan dan disajikan sesuai kebutuhan penggunanya [37]. Penggunanya adalah tim pengembang di dalam organisasi, sehingga platform ini sering disebut Internal Developer Platform (IDP). *Platform engineering* adalah disiplin membangun dan mengelola platform itu. *White paper* CNCF menyebut tujuh sifat platform yang baik: diperlakukan sebagai produk, pengalaman pengguna yang baik, dokumentasi dan pengenalan yang memadai, swalayan, beban kognitif pengguna yang berkurang, kapabilitas yang opsional dan dapat dirangkai, serta aman sejak awal (*secure by default*). Tim platform tidak harus men-

jalankan sendiri server dan jaringan; tanggung jawab utamanya adalah antarmuka dan pengalaman pengguna.

Tiga gagasan paling sering muncul dalam percakapan manajemen. Pertama, *golden path*, yaitu paket templat proyek awal beserta dokumentasinya, sehingga tim baru langsung berjalan di jalur yang sudah teruji [37]. Kedua, *self-service*: tim pengguna mengambil sendiri kapabilitas yang dibutuhkan tanpa menunggu tiket. Ketiga, *platform as a product*: tim platform memperlakukan tim pengembang sebagai pelanggan, mendengar kebutuhannya, dan mengukur keberhasilan dari pemakaian. Gambar 13.4 memperlihatkan lapisan platform internal yang diusulkan untuk Medikara.



Gambar 13.4: Lapisan platform internal Medikara: tim *stream-aligned* memakai antarmuka swalayan di atas kapabilitas dan fondasi yang dikelola tim platform

Analoginya seperti dapur bersama yang disewakan kepada usaha kuliner kecil. Pengelola menyediakan kompor, gudang pendingin, izin higienis, dan kemasan standar. Penyewa tinggal memasak menu khasnya. Penyewa boleh membawa alat sendiri, tetapi jalur standar lebih murah dan sudah lolos pemeriksaan dinas kesehatan. Pengelola yang baik menanyai penyewa apa yang kurang, bukan menunggu surat permohonan.

Dua contoh di Medikara Health:

1. Misalkan tim farmasi dan laboratorium membutuhkan layanan stok obat. Tanpa platform, tim mengajukan lima tiket untuk repositori, *pipeline*, *namespace*, basis data, dan akses *log*, lalu menunggu 14 hari. Dengan *golden path*, layanan pertama berjalan di *staging* dalam satu hari, lengkap dengan *pipeline*, pemantauan, dan Service Level Objective (SLO) dasar dari Bab 11.
2. Misalkan tim infrastruktur pusat menerima 12 permintaan lingkungan uji per bulan, dengan tunggu 9 hari dan kerja 6 jam per permintaan. Dengan lingkungan uji swalayan berbasis IaC, tunggu turun menjadi sekitar 30 menit,

dan 72 jam kerja per bulan dipakai untuk memperbaiki templat bersama.

13.5.2 Tim Platform Bukan Loket Tiket

Dua hal pada tim platform sering tertukar. Pembagiannya begini. Divisi infrastruktur tradisional melayani tiket satu per satu. Tim platform membangun kapabilitas yang dapat dipakai berulang tanpa tiket. Mengganti nama divisi infrastruktur menjadi "tim platform" tanpa mengubah cara kerjanya hanya menghasilkan loket tiket dengan nama baru. Tabel 13.6 memperlihatkan perbedaannya.

Tabel 13.6: Perbedaan divisi infrastruktur yang melayani tiket dengan tim platform

Aspek	Divisi infrastruktur berbasis tiket	Tim platform
Cara melayani	Mengerjakan permintaan satu per satu	Menyediakan kapabilitas swalayan yang dipakai berulang
Satuan kerja	Tiket	Produk internal dengan peta jalan
Ukuran keberhasilan	Jumlah tiket selesai dan target waktu penyelesaian	Adopsi, kepuasan pengguna, waktu sampai layanan pertama berjalan, metrik DORA tim pengguna
Hubungan dengan tim lain	Penjaga gerbang yang menyetujui atau menolak	Penyedia layanan yang mendengar kebutuhan pengguna
Akibat bila permintaan naik	Antrean memanjang, perlu tambah orang	Biaya per pemakaian turun
Pendanaan	Pusat biaya	Didanai menurut nilai yang dihasilkan

Ada dua jebakan di sisi lain. Jebakan pertama adalah platform yang dibangun lama sebelum ada pengguna, sehingga yang jadi tidak cocok dengan kebutuhan. Skelton dan Pais menyarankan memulai dari platform tertipis yang masih berguna (*thinnest viable platform*), yang pada awalnya bisa sesederhana halaman dokumentasi berisi standar dan templat [131]. Jebakan kedua adalah memaksakan platform lewat perintah. Platform yang baik dipakai karena memudahkan, bukan karena diwajibkan.

13.5.3 Platform Engineering Maturity Model

Untuk menilai posisi organisasi, CNCF menerbitkan *Platform Engineering Maturity Model*. Model ini memuat lima aspek, yaitu investasi, adopsi, antarmuka, operasi, dan pengukuran, dengan empat tingkat kematangan pada setiap aspek [36]. Tabel 13.7 meringkasnya.

Misalkan penilaian awal menempatkan Medikara di tingkat 1 pada kelima aspek: empat orang bersertifikat *cloud* (Bab 4) mengurus platform sambil mengerjakan tugas lain, dan semua permintaan masuk lewat tiket. Sasaran yang wajar

Tabel 13.7: Ringkasan *Platform Engineering Maturity Model* CNCF [36]

Aspek	**1. *Provisional***	**2. *Operationalized***	**3. *Scalable***	**4. *Optimizing***
Investasi	Sukarela atau sementara	Tim khusus, pusat biaya	Didanai sebagai produk	Ekosistem yang diperluas para spesialis
Adopsi	Tidak menentu	Didorong insentif dan perintah	Ditarik oleh nilai	Pengguna ikut berkontribusi
Antarmuka	Proses khusus, banyak campur tangan manual	Alat standar dan *golden path* terdokumentasi	Swalayan "sekali klik"	Menyatu dengan alat kerja tim
Operasi	Dikerjakan bila diminta	Tercatat dan punya pemilik	Diatur terpusat dengan proses baku	Layanan terkelola dengan siklus hidup otomatis
Pengukuran	Seadanya	Pengumpulan data yang konsisten	Wawasan dengan ukuran keberhasilan	Gabungan ukuran kuantitatif dan kualitatif

untuk dua tahun pertama adalah tingkat 2 pada semua aspek dan tingkat 3 pada antarmuka, bukan melompat ke tingkat 4.

13.6 Team Topologies

Platform hanya satu bagian dari jawaban. Pertanyaan yang lebih besar: bagaimana menata seluruh tim agar perubahan mengalir cepat tanpa saling menunggu? Bab 7 sudah memperkenalkan hukum Conway: struktur sistem meniru struktur komunikasi organisasi. Team Topologies memberi kosakata untuk merancang struktur itu dengan sengaja.

13.6.1 Empat Tipe Tim dan Beban Kognitif

Skelton dan Pais membedakan empat tipe tim [131]. Tim *stream-aligned* mengirim perubahan dari awal sampai akhir untuk satu aliran nilai, misalnya satu produk, satu perjalanan pengguna, atau satu domain bisnis. Tipe ini adalah mayoritas. Tiga tipe lainnya ada untuk mengurangi beban tim *stream-aligned*. Tim platform menyediakan layanan internal swalayan. Tim *enabling* membantu tim lain menguasai kemampuan baru, lalu mundur. Tim *complicated-subsystem* menjaga bagian yang membutuhkan keahlian langka. Ukuran yang dipakai untuk menilai pembagian ini adalah beban kognitif tim, yaitu jumlah hal yang harus dipahami dan dijaga satu tim sekaligus. Tim yang menanggung terlalu banyak hal akan melambat, seberapa pun pintar anggotanya. Tabel 13.8 merangkum keempat tipe.

Tabel 13.8: Empat tipe tim menurut Team Topologies [131], dengan analogi rumah sakit dan contoh di TI Medikara

Tipe tim	Tugas	Analogi di rumah sakit	Contoh di TI Medikara
Stream-aligned	Mengirim perubahan untuk satu aliran nilai dari awal sampai akhir	Tim bangsal yang merawat pasien sejak masuk sampai pulang	Tim rekam medis
Platform	Menyediakan layanan internal swalayan agar tim lain tidak memikul beban teknis yang sama	Instalasi sterilisasi pusat yang menyediakan alat steril siap pakai	Tim platform berisi 6 orang
Enabling	Membantu tim lain menguasai kemampuan baru untuk sementara	Tim pencegahan dan pengendalian infeksi yang melatih bangsal	Tim keamanan aplikasi dan keandalan berisi 2 orang
Complicated-subsystem	Menjaga bagian yang membutuhkan keahlian langka	Unit fisika medis untuk radioterapi	Tim integrasi FHIR untuk SATUSEHAT dan BPJS

Analoginya ada di rumah sakit itu sendiri. Perawat bangsal tidak mencuci dan menyeterilkan alat bedahnya sendiri. Bila itu dilakukan, waktu merawat pasien habis untuk pekerjaan penunjang. Bangsal memakai layanan instalasi sterilisasi pusat, dilatih sesekali oleh tim pengendalian infeksi, dan merujuk pasien radioterapi ke unit yang punya fisikawan medis. Pembagian itu menjaga beban kerja bangsal tetap pada inti tugasnya.

13.6.2 Tiga Mode Interaksi

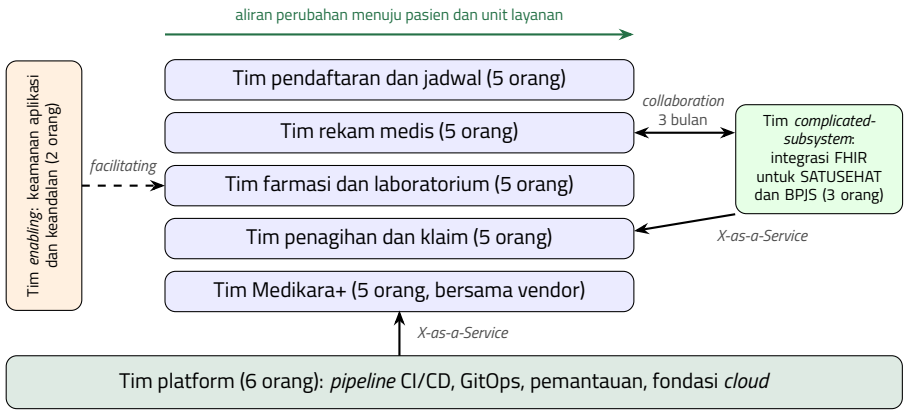
Team Topologies juga membatasi cara tim berinteraksi menjadi tiga mode [131]:

- *Collaboration*: dua tim bekerja berdampingan untuk menemukan sesuatu yang baru. Mode ini mahal dan sebaiknya berjangka waktu.
- *X-as-a-Service*: satu tim memakai layanan tim lain lewat antarmuka yang jelas, dengan koordinasi seminimal mungkin.
- *Facilitating*: satu tim membantu dan melatih tim lain sampai tim itu mandiri.

Mode interaksi sebaiknya berubah seiring waktu. Kolaborasi yang tidak pernah berakhir adalah tanda bahwa batas tanggung jawab belum jelas. Gambar 13.5 memperlihatkan topologi tim target Medikara beserta mode interaksinya.

Dua contoh di Medikara Health:

1. Misalkan tim integrasi FHIR (Fast Healthcare Interoperability Resources) dan tim rekam medis berkolaborasi tiga bulan untuk memetakan rekam medis ke format FHIR yang diminta SATUSEHAT. Setelah stabil, interaksinya menjadi



Gambar 13.5: Topologi tim target TI Medikara: lima tim *stream-aligned* didukung tim platform, tim *enabling*, dan tim *complicated-subsystem*

X-as-a-Service: tim rekam medis cukup memanggil API integrasi.

2. Misalkan tim *enabling* mendampingi setiap tim *stream-aligned* enam minggu secara bergiliran untuk memasang pemindaian pustaka dan SLO dasar. Lima tim membutuhkan 30 minggu, lalu setiap tim menjalankannya sendiri.

Kasus Medikara Health: Dari Tim TI per Rumah Sakit ke Tim Stream-Aligned dan Tim Platform

Medikara punya 52 staf TI (Bab 4). Misalkan 26 orang bekerja di lima rumah sakit, yaitu 12 pengembang SIMRS dan 14 staf perangkat dan bantuan pengguna (Bab 7), dan 26 orang di kantor pusat untuk ERP (Enterprise Resource Planning), infrastruktur, jaringan, dan layanan bantuan. Bab 7 sudah mengusulkan empat tim domain berisi tiga pengembang dan dua perwakilan unit pengguna. Bab ini melengkapinya menjadi topologi utuh pada Gambar 13.5.

Dari kantor pusat, 15 orang pindah ke tim baru: 6 orang menjadi tim platform, 4 orang membentuk tim Medikara+ bersama satu pemilik produk dari bagian layanan pasien, 3 orang menjadi tim integrasi FHIR, dan 2 orang menjadi tim *enabling*. Sebanyak 11 orang tetap menangani ERP dan layanan bantuan pusat. Tim Medikara+ bekerja bersama vendor, tetapi kepemilikan produk kini ada di tangan Medikara.

Penataan ini punya tiga konsekuensi. Pertama, beban kognitif turun. Sebelumnya dua sampai tiga pengembang di setiap rumah sakit menjaga sembilan modul SIMRS beserta server dan cadangannya. Kini setiap tim domain menjaga dua sampai tiga modul, dan urusan *pipeline*, klaster, serta pemantauan diambil alih platform. Kedua, tim *enabling* (2 orang) dan tim

integrasi (3 orang) berada di bawah ukuran tim yang dianjurkan, sehingga dokumentasi dan rotasi pengetahuan menjadi syarat. Ketiga, tanpa *self-service*, enam orang tim platform yang melayani lima tim akan kembali menjadi loket tiket.

13.7 Aktivitas: Model Operasi Delivery dan Peta Jalan Platform Internal

Tujuan. Tim memetakan alur perubahan saat ini, merancang alur CI/CD dan GitOps target, serta menyusun topologi tim dan kapabilitas platform internal yang diprioritaskan. Tim memakai organisasi nyata dengan izin tertulis dan data yang dianonimkan, atau memakai kasus Medikara Health.

Pembagian peran. Alur perubahan memuat dua jenis hambatan: hambatan organisasi seperti persetujuan dan antrean, serta hambatan teknis seperti uji manual dan rilis manual. Tabel 13.9 membagi peran agar setiap anggota menyumbang sesuai latar belakangnya.

Tabel 13.9: Pembagian peran dalam Aktivitas Bab 13

Peran	Diisi oleh	Tugas
Pemeta waktu tunggu	Anggota non-teknis	Menelusuri satu perubahan dari permintaan sampai produksi, mewawancarai pemberi persetujuan, mencatat waktu tunggu setiap langkah
Perancang alur target	Anggota teknis	Merancang tahap CI/CD dan GitOps target, menghitung metrik DORA dari catatan rilis
Perancang topologi	Seluruh anggota	Menyusun tim *stream-aligned*, platform, dan *enabling*, lalu memeriksa beban kognitif setiap tim
Penyusun peta jalan	Satu anggota yang ditunjuk	Mengurutkan kapabilitas platform dan menjaga konsistensi angka dengan artefak C4 lain

Langkah kerja.

1. Petakan alur perubahan saat ini. Pilih satu perubahan kecil yang baru selesai, lalu telusuri dari permintaan pertama sampai berjalan di produksi. Catat setiap langkah, pelakunya, waktu proses, dan waktu tunggu sesudahnya.
2. Tandai antrean dan langkah manual. Kelompokkan setiap langkah menjadi kerja, persetujuan, antrean, uji manual, atau rilis manual. Hitung efisiensi alur, yaitu waktu proses dibagi *lead time* permintaan. Lengkapi dengan metrik DORA dari catatan rilis satu tahun.

3. Rancang alur target. Tentukan tahap yang diotomatisasi lewat CI/CD, perubahan yang dikelola lewat GitOps, dan persetujuan yang dipertahankan. Setiap persetujuan yang dipertahankan diberi alasan risiko tertulis.
4. Susun topologi tim target. Tetapkan tim *stream-aligned* beserta aliran nilainya, tim platform, tim *enabling*, dan bila perlu tim *complicated-subsystem*. Tulis mode interaksi antartim dan jumlah modul atau layanan yang dipegang setiap tim.
5. Pilih tiga sampai lima kapabilitas platform internal dan urutannya. Setiap kapabilitas dikaitkan dengan satu hambatan dari langkah 2, diberi pengguna pertama, ukuran keberhasilan, dan triwulan pelaksanaan.

Tabel 13.10 adalah contoh hasil langkah 1 dan 2 untuk kisah kolom nomor telepon keluarga di Pendahuluan.

Tabel 13.10: Contoh pemetaan alur perubahan: kolom nomor telepon keluarga di SIMRS RS Jakarta (rekaan, satuan hari kalender)

No	Langkah	Waktu proses	Waktu tunggu sesudahnya	Jenis langkah
1	Tiket diajukan dan diklasifikasi layanan bantuan	0,5	4	Kerja manual
2	Persetujuan kepala instalasi rekam medis	0,5	6	Persetujuan
3	Persetujuan komite perubahan bulanan	0,5	21	Persetujuan
4	Antre menunggu pengembang	0	20	Antrean
5	Pemrograman dan tinjauan kode	2	3	Kerja
6	Uji fungsional manual	1	28	Uji manual
7	Uji regresi seluruh SIMRS oleh enam penguji	5	4	Uji manual
8	Rilis Sabtu malam dengan henti layanan 4 jam	0,5	0	Rilis manual
	Jumlah (*lead time* permintaan 96 hari, efisiensi alur 10,4 persen)	**10**	**86**	

Waktu tunggu terbesar ada di jendela uji regresi (28 hari), rapat komite (21 hari), dan antrean pengembang (20 hari). Obatnya berbeda-beda: uji otomatis, persetujuan berbasis risiko yang membebaskan perubahan formulir dari komite, dan daftar kerja mingguan tim domain. Contoh alur target di berkas templat menurunkan *lead time* permintaan dari 96 menjadi sekitar 10 hari.

Luaran dan kaitan capstone. Luarannya adalah model operasi *delivery* dan peta jalan platform internal, yaitu berkas templat yang sudah diisi: pemetaan alur saat ini, metrik DORA, alur target, topologi tim target, dan tiga sampai lima kapabilitas platform beserta urutannya. Keduanya menjadi bagian tonggak C4 Peta Jalan dan Tata Kelola. Metrik DORA dan sasaran *lead time* dari Aktivitas ini

dipakai lagi pada Bab 14 sebagai ukuran kemajuan peta jalan modernisasi.

Kriteria penilaian. Tabel 13.11 memuat kriteria dan bobotnya.

Tabel 13.11: Kriteria penilaian Aktivitas Bab 13

Kriteria	Bobot	Tanda hasil yang baik
Mutu pemetaan alur	30%	Satu perubahan nyata ditelusuri dengan tanggal; waktu proses dan waktu tunggu dipisah; sumber data disebut
Mutu alur target	25%	Tahap otomatis jelas; setiap persetujuan yang dipertahankan punya alasan risiko; jejak audit tergambar
Mutu topologi tim	25%	Tipe tim dan mode interaksi tepat; beban kognitif diperiksa; risiko tim kecil ditulis
Peta jalan platform	20%	Setiap kapabilitas terkait hambatan nyata, punya pengguna pertama dan ukuran keberhasilan

Berkas pendukung.

- `sources/chapter13/templat-model-operasi-delivery.md`: templat luaran dengan baris contoh Medikara untuk pemetaan alur, metrik DORA, alur target, topologi tim, dan peta jalan platform.
- `sources/chapter13/contoh-riwayat-deployment.csv` dan `sources/chapter13/hitung-metrik-dora.py`: data contoh dan skrip penghitung metrik DORA.
- `sources/chapter13/contoh-pipeline-gitops/`: contoh *workflow* GitHub Actions dan *Application* Argo CD. Pendalaman teknis opsional merangkai keduanya di akun dan klaster latihan sendiri, lalu mengukur waktu dari *commit* sampai layanan menjawab; langkahnya ada di `sources/chapter13/README.md`.

Etika. Tiket, catatan rilis, dan bagan organisasi dari organisasi nyata dipakai seizin organisasi. Nama orang, data pasien, alamat server, dan kunci akses dihapus sebelum dibagikan ke anggota tim. Pemetaan waktu tunggu menilai alur, bukan menilai orang. Hasilnya tidak dipakai untuk menyalahkan pemberi persetujuan tertentu.

13.8 Ringkasan

Arsitektur *cloud-native* tidak memberi manfaat bila cara mengirim perubahan tetap lambat. Kolom nomor telepon keluarga di SIMRS Jakarta membutuhkan 96 hari, padahal waktu kerjanya hanya 10 hari. DevOps menyatukan tanggung jawab membangun dan menjalankan perangkat lunak, dan DORA memberinya ukuran. Riset DORA menunjukkan bahwa kecepatan dan stabilitas tidak saling meniadak-

an. Rilis yang jarang berukuran besar, dan rilis besar lebih sering gagal serta lebih lama dipulihkan, seperti SIMRS Jakarta dengan tingkat kegagalan 43 persen.

Laporan DORA 2025 menambahkan peringatan: AI adalah penguat kekuatan maupun kelemahan yang sudah ada. Asisten pemrograman hanya memangkas satu hari dari 96 hari, dan bila jendela rilis tetap triwulanan, kode yang lebih banyak justru memperbesar ukuran rilis.

CI/CD mengotomatisasi jalur dari *commit* sampai produksi, dengan atau tanpa keputusan manusia sebelum rilis. *Blue-green* dan *canary* menghapus henti layanan dan menyediakan jalan pulang yang cepat. IaC menulis infrastruktur sebagai kode yang ditinjau. GitOps menambahkan agen yang menarik keadaan yang diinginkan dari Git dan terus merekonsiliasi, sehingga jejak audit siapa mengubah apa terbentuk dari kerja sehari-hari.

Platform engineering menyediakan kapabilitas itu sebagai produk internal yang swalayan. Tim platform diukur dari adopsi dan kemudahan tim pengguna, bukan dari jumlah tiket selesai. Team Topologies menata tim menjadi empat tipe dan tiga mode interaksi dengan beban kognitif sebagai ukuran. Bagi Medikara, 12 pengembang rumah sakit dan 15 staf pusat ditata menjadi lima tim *stream-aligned*, satu tim platform, satu tim *enabling*, dan satu tim *complicated-subsystem*.

Pesan utama bab ini: kecepatan dan keamanan pengiriman perubahan ditentukan oleh alur kerja, otomasi, dan susunan tim, bukan oleh seberapa jarang organisasi berani merilis.

13.9 Latihan

Latihan 1: Rilis Jarang dan Rasa Aman Semu

Latihan ini membuktikan bahwa anggapan "rilis yang jarang lebih aman" dapat diuji dengan data rilis organisasi sendiri dan dikritisi dengan temuan DORA. Organisasi yang merilis setiap tiga bulan sering memegang anggapan ini.

1. Kumpulkan catatan *deployment* produksi satu tahun untuk satu atau dua aplikasi, atau pakai `sources/chapter13/contoh-riwayat-deployment.csv`.
2. Hitung metrik DORA dengan `sources/chapter13/hitung-metrik-dora.py` atau secara manual: jumlah *deployment*, *lead time* rata-rata, tingkat kegagalan perubahan, dan waktu pemulihan median.
3. Hitung rata-rata perubahan per rilis terjadwal, lalu hitung peluang rilis bermasalah dengan rumus $1 - (1 - p)^n$ dan $p = 1$ persen.
4. Bandingkan hasil dengan contoh pada Tabel 13.12, lalu isi Tabel 13.13.

5. Tulis satu paragraf yang mengkritisi anggapan rilis jarang lebih aman, dengan merujuk temuan DORA dan angka yang diisi.

Tabel 13.12: Latihan 1, contoh terisi: SIMRS RS Jakarta dan Medikara+ tahun 2025 (rekaan)

Ukuran	SIMRS Jakarta	Medikara+	Bacaan
Deployment per tahun	7	14	SIMRS dirilis separuh lebih jarang
Perubahan per rilis terjadwal	60,5	7,8	Rilis SIMRS hampir delapan kali lebih besar
Peluang rilis bermasalah ($p = 1$ persen)	45,3 persen	7,7 persen	Ukuran rilis saja sudah meramalkan perbedaan besar
Tingkat kegagalan nyata	43 persen	14 persen	Data nyata searah dengan hitungan
Waktu pemulihan median	9 jam	5 jam	Rilis besar lebih lama dipulihkan

Tabel 13.13: Latihan 1, lembar isian: metrik DORA dan ukuran rilis organisasi pilihan

Ukuran	Aplikasi 1	Aplikasi 2	Bacaan
Deployment per tahun	...	...	...
Perubahan per rilis terjadwal	...	...	...
Peluang rilis bermasalah	...	...	...
Tingkat kegagalan nyata	...	...	...
Waktu pemulihan median	...	...	...

Pertanyaan analisis:

1. Bila aplikasi yang paling jarang dirilis juga punya tingkat kegagalan tertinggi, apa penjelasan yang paling masuk akal menurut data yang diisi? Apakah ada penjelasan lain, misalnya mutu vendor atau umur sistem, yang perlu diperiksa?
2. Bila SIMRS Jakarta dirilis bulanan dengan jumlah perubahan setahun yang sama, berapa perubahan per rilis dan berapa peluang rilis bermasalahnya? Langkah manual apa yang harus diotomatisasi lebih dahulu agar rilis bulanan tidak menambah beban tim?

Latihan 2: Tim Platform dan Divisi Infrastruktur yang Melayani Tiket

Latihan ini membuktikan bahwa perbedaan tim platform dengan divisi infrastruktur berbasis tiket dapat dilihat dari data permintaan, bukan dari nama unitnya.

1. Kumpulkan tiket permintaan ke tim infrastruktur selama tiga bulan terakhir,

lalu kelompokkan menjadi lima jenis permintaan yang paling sering.

2. Untuk setiap jenis, catat jumlah per bulan, waktu tunggu rata-rata, dan waktu kerja tim infrastruktur per permintaan.
3. Hitung total hari tunggu per bulan (jumlah dikali waktu tunggu) dan total jam kerja per bulan (jumlah dikali waktu kerja).
4. Usulkan bentuk swalayan untuk setiap jenis beserta sasaran waktu tunggunya, dengan contoh pada Tabel 13.14, lalu isi Tabel 13.15.
5. Nilai posisi organisasi pada aspek antarmuka dan operasi di Tabel 13.7, sebelum dan sesudah usulan.

Tabel 13.14: Latihan 2, contoh terisi: lima jenis permintaan ke tim infrastruktur pusat Medikara per bulan (rekaan)

Jenis permintaan	Jumlah	Tunggu (hari)	Kerja (jam)	Bentuk swalayan	Sasaran tunggu
Lingkungan uji baru	12	9	6	Templat IaC lewat portal	30 menit
Repositori dan *pipeline* layanan baru	3	14	16	*Golden path* layanan baru	2 jam
Akses *log* produksi	25	3	1	*Dashboard* pemantauan berbasis peran	Langsung
Penambahan kapasitas	6	5	3	Penskalaan otomatis dalam batas kuota	Otomatis
Sertifikat dan nama domain	4	4	2	Penerbitan otomatis	10 menit
Jumlah	**50**	**271 hari tunggu**	**171 jam**		

Tabel 13.15: Latihan 2, lembar isian: permintaan ke tim infrastruktur organisasi pilihan

Jenis permintaan	Jumlah	Tunggu (hari)	Kerja (jam)	Bentuk swalayan	Sasaran tunggu
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
Jumlah	…	…	…		

Pertanyaan analisis:

1. Urutkan jenis permintaan menurut total hari tunggu, lalu menurut total jam kerja. Bila kedua urutan berbeda, mana yang dipakai untuk menentukan kapabilitas platform pertama, dan mengapa?
2. Pada contoh Medikara, 171 jam kerja per bulan setara dengan sekitar satu

orang penuh waktu. Bila kapabilitas swalayan dibangun, pekerjaan apa yang sebaiknya mengisi waktu itu agar tim tidak kembali menjadi loket tiket?

Latihan 3: GitOps dan Pertanyaan Auditor

Latihan ini membuktikan cara GitOps membantu auditor menelusuri siapa mengubah apa di lingkungan produksi, sekaligus menunjukkan celah yang tetap harus ditutup dengan kontrol lain. Misalkan auditor bertanya tentang perubahan batas jumlah koneksi basis data layanan jadwal dokter di produksi pada 14 Maret 2026.

1. Tuliskan pertanyaan auditor tentang perubahan itu: siapa yang mengubah, siapa yang menyetujui, kapan berlaku, dan apakah yang berjalan sama dengan yang disetujui.
2. Untuk setiap pertanyaan, tuliskan sumber bukti tanpa GitOps di organisasi pilihan.
3. Tuliskan sumber bukti dengan GitOps, merujuk Tabel 13.5 dan Gambar 13.3.
4. Tuliskan celah yang tetap tersisa beserta kontrol tambahannya, dengan contoh pada Tabel 13.16, lalu isi Tabel 13.17.

Tabel 13.16: Latihan 3, contoh terisi: bukti untuk pertanyaan auditor di Medikara (rekaan)

Pertanyaan auditor	Bukti tanpa GitOps	Bukti dengan GitOps	Celah dan kontrol tambahan
Siapa yang mengubah?	Log server bila diaktifkan, sering memakai akun bersama "admin"	Penulis *commit* dengan akun Git pribadi yang terautentikasi	Akun Git bersama dilarang; *commit* ditandatangani
Siapa yang menyetujui?	Surel atau tiket yang tidak terhubung dengan perubahan teknis	Peninjau *pull request* yang tercatat	Aturan dua peninjau dipaksakan di repositori, bukan sekadar kesepakatan
Kapan berlaku di produksi?	Ingatan petugas dan catatan rilis	Riwayat sinkronisasi agen dengan nomor revisi	Log agen disimpan sesuai kebijakan retensi
Apakah yang berjalan sama dengan yang disetujui?	Sulit dibuktikan	Status sinkron agen; *drift* dikembalikan dan dicatat	Pemakaian akun darurat dicatat dan ditinjau dalam 24 jam

Pertanyaan analisis:

1. Dari empat pertanyaan auditor, mana yang paling sulit dijawab di organisasi pilihan saat ini? Berapa lama waktu yang dibutuhkan untuk menjawabnya tanpa GitOps?
2. Celah mana yang tidak dapat ditutup oleh GitOps sendiri dan membutuhkan

Tabel 13.17: Latihan 3, lembar isian: bukti untuk pertanyaan auditor di organisasi pilihan

Pertanyaan auditor	**Bukti tanpa GitOps**	**Bukti dengan GitOps**	**Celah dan kontrol tambahan**
Siapa yang mengubah?	…	…	…
Siapa yang menyetujui?	…	…	…
Kapan berlaku di produksi?	…	…	…
Apakah yang berjalan sama dengan yang disetujui?	…	…	…

kontrol organisasi, misalnya pengelolaan akun atau prosedur darurat? Siapa yang sebaiknya memiliki kontrol itu?

Bab 14

Peta Jalan Modernisasi dan Tata Kelola Cloud

Tujuan Pembelajaran

- Menyusun urutan modernisasi bertahap dengan pola *strangler fig*.
- Merancang *guardrail* tata kelola *cloud* untuk biaya, keamanan, dan lokasi data.
- Mempertahankan peta jalan modernisasi di depan pemangku kepentingan dengan metrik yang terukur.

14.1 Pendahuluan

Pada akhir Oktober 2026 direksi Medikara Health mengajukan permintaan yang terdengar sederhana kepada tim transformasi: satu dokumen yang menjawab tiga pertanyaan. Apa yang dimodernisasi lebih dulu? Berapa biayanya? Bagaimana risikonya dikendalikan? Selama satu semester tim sudah menghasilkan banyak artefak: matriks kendala lokasi data, model biaya, portofolio 46 aplikasi dengan usulan 7R, peta domain, Architecture Decision Record (ADR), matriks keputusan *runtime*, katalog *event*, peta kepemilikan data, Service Level Objective (SLO), rencana pemulihan bencana, dan model operasi *delivery*. Direktur keuangan menyampaikan keberatannya dengan terus terang: tumpukan artefak belum tentu sebuah rencana.

Bab ini adalah penutup mata kuliah. Tugasnya menyatukan artefak itu menjadi peta jalan yang dapat dipertanggungjawabkan. Ada tiga bahan pokok. Pertama, urutan modernisasi bertahap yang tidak menghentikan pelayanan, dengan pola

strangler fig dan susunan gelombang. Kedua, *guardrail* tata kelola *cloud* yang menjaga biaya, keamanan, dan lokasi data tanpa menjadikan setiap perubahan antrean persetujuan. Ketiga, metrik kemajuan yang bermakna bagi direksi, bukan sekadar hitungan aplikasi yang sudah dipindahkan.

Gambar 14.1 merangkum hubungan komponen dan konsep utama bab ini.



Gambar 14.1: Strangler memindahkan irisan layanan secara bertahap

Bab ini disusun dalam enam langkah: pola *strangler fig*, penyusunan gelombang, *guardrail* dan *policy as code*, FinOps pada fase Operate, kedaulatan data, serta metrik kemajuan. Rujukan utamanya adalah tulisan Fowler tentang *strangler fig* [63], pola *strangler fig* dan *anticorruption layer* dari Microsoft [101, 99] yang berakar pada Evans [47], panduan tata kelola dan kedaulatan dalam Microsoft Cloud Adoption Framework (CAF) [95, 94, 100], rujukan kendali Amazon Web Services (AWS) Control Tower [22], dokumentasi Open Policy Agent (OPA) [113], FinOps Framework [53], dan metrik DORA (DevOps Research and Assessment) [46]. Hasil bab ini adalah peta jalan modernisasi tiga gelombang dan *guardrail* tata kelola *cloud*, bagian terakhir tonggak capstone C4 yang dipresentasikan menjelang Ujian Akhir Semester (UAS).

14.2 Pola Strangler Fig dan Modernisasi Bertahap

Pertanyaan pertama direksi hampir selalu sama: mengapa tidak diganti sekaligus saja? Vendor yang dibahas di Bab 7 menawarkan 32 layanan baru dalam 18 bulan untuk menggantikan lima SIMRS (Sistem Informasi Manajemen Rumah Sakit). Penggantian sekaligus menjanjikan satu proyek, satu tanggal peralihan, dan satu kali repot. Jawaban atas pertanyaan itu menentukan bentuk seluruh peta jalan.

14.2.1 Risiko Penggantian Sekaligus

Penggantian sekaligus (*big bang*) berarti sistem lama dimatikan dan sistem baru dinyalakan pada satu waktu peralihan. Cara ini punya tiga kelemahan. Pertama, seluruh risiko terkumpul pada satu malam peralihan. Bila ada yang salah, semua rumah sakit terdampak bersamaan. Kedua, nilai bisnis baru terasa setelah seluruh pekerjaan selesai, sehingga berbulan-bulan pertama hanya berisi biaya. Ketiga, kembali ke sistem lama sulit setelah data baru mulai ditulis di sistem baru, karena data itu harus disalin balik atau diketik ulang. Tabel 14.1 membandingkan penggantian sekaligus dengan penggantian bertahap dari sisi yang biasa ditanyakan direksi.

Tabel 14.1: Penggantian sekaligus dibandingkan dengan penggantian bertahap

Aspek	Penggantian sekaligus	Penggantian bertahap
Titik risiko	Satu malam peralihan untuk seluruh fungsi dan lokasi	Banyak langkah kecil, masing-masing untuk satu fungsi atau satu lokasi
Nilai bisnis pertama	Setelah seluruh pekerjaan selesai	Setelah irisan pertama berjalan
Kembali ke sistem lama	Sulit setelah data baru ditulis	Mudah selama rute lama masih ada
Biaya masa transisi	Kecil, karena tidak ada lapisan sementara	Ada biaya fasad, lapisan penerjemah, dan dua sistem yang berjalan bersama
Pembelajaran tim	Baru terjadi saat peralihan	Terjadi di setiap langkah dan dipakai untuk langkah berikutnya
Cocok untuk	Sistem kecil yang mudah diganti, atau sistem lama yang harus segera dimatikan	Sistem besar yang harus tetap melayani selama penggantian

14.2.2 Cara Kerja Strangler Fig

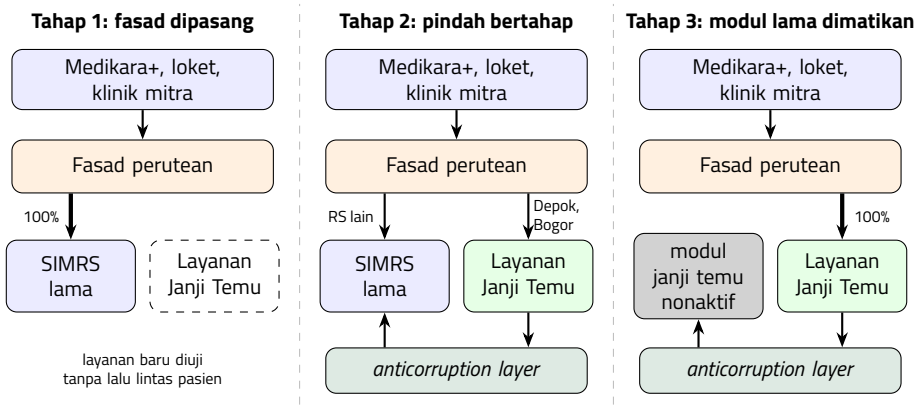
Strangler fig adalah pola penggantian bertahap: sistem baru dibangun di sekitar sistem lama, perilaku sistem lama dipindahkan sedikit demi sedikit, sampai sistem lama dapat dimatikan. Nama pola ini diambil dari pohon ara pencekik yang tumbuh membelit pohon inangnya. Fowler menekankan tiga hal [63]. Pertama, sistem lama jarang punya batas komponen yang rapi, sehingga tim harus menemukan atau membuat celah pemisah (*seam*) tempat satu bagian dapat diganti tersendiri. Kedua, kode sementara yang membuat sistem lama dan baru dapat hidup berdampingan, yang disebut arsitektur transisi, adalah biaya yang sengaja dibayar karena menurunkan risiko dan mempercepat nilai. Ketiga, struktur tim dan budaya kerja ikut berubah. Tanpa itu, sistem baru akan membusuk seperti sistem lama.

Microsoft menjabarkan polanya dalam empat langkah [101]. Sebuah fasad (*proxy*) dipasang di antara klien dan sistem, lalu awalnya meneruskan hampir

semua permintaan ke sistem lama. Fasad kemudian mengalihkan permintaan ke sistem baru sedikit demi sedikit. Setelah semua fungsi pindah dan tidak ada lagi yang bergantung pada sistem lama, sistem lama dimatikan. Terakhir, fasad dilepas dan klien langsung memanggil sistem baru. Fasad tidak boleh menjadi titik gagal tunggal atau penyebab lambat. Pola ini juga tidak cocok bila permintaan ke sistem lama tidak dapat dicegat, bila sistemnya kecil dan mudah diganti, atau bila sistem lama harus segera dimatikan.

Analoginya seperti renovasi gedung rumah sakit yang tidak boleh tutup. Pengelola tidak merobohkan gedung lalu membangun ulang selama dua tahun. Satu sayap ditutup, pasien diarahkan lewat pintu dan papan petunjuk sementara ke sayap yang sudah jadi, lalu sayap berikutnya menyusul. Papan petunjuk itulah fasad. Ketika semua sayap selesai, papan sementara dicabut.

Gambar 14.2 memperlihatkan tiga tahap pola ini untuk fungsi janji temu Medikara Health.



Gambar 14.2: Tiga tahap *strangler fig* untuk fungsi janji temu: fasad dipasang, permintaan dipindah per rumah sakit, lalu modul janji temu SIMRS dinonaktifkan

Dua contoh kasus:

1. Misalkan sebuah jaringan apotek dengan 120 gerai mengganti sistem kasirnya sekaligus pada satu malam. Bila sistem baru gagal pada pagi hari, 120 gerai berhenti bersamaan. Dengan penggantian bertahap yang dimulai dari 5 gerai, kegagalan yang sama hanya menghentikan sekitar 4 persen gerai, dan 115 gerai lain tetap memakai sistem lama.
2. Misalkan sebuah perusahaan asuransi memindahkan fungsi cek status klaim lebih dulu, karena 70 persen panggilan ke pusat layanan menanyakan status klaim. Fungsi ini hanya membaca data, sehingga mudah dikembalikan ke sistem lama bila ada masalah. Nilai pertamanya terasa dalam enam minggu, jauh sebelum fungsi pengajuan klaim yang lebih rumit dipindahkan.

14.2.3 Anticorruption Layer selama Masa Transisi

Selama masa transisi, sistem baru masih perlu memanggil fungsi yang belum pindah, dan bagian sistem lama masih perlu memanggil fungsi yang sudah pindah. *Anticorruption layer* adalah lapisan penerjemah yang ditaruh di antara dua sistem yang maknanya berbeda. Pola ini pertama kali diuraikan Evans [47]. Tujuannya menjaga rancangan sistem baru agar tidak ikut tercemar model data dan istilah sistem lama. Microsoft mencatat beberapa harga yang harus dibayar [99]: lapisan ini menambah waktu tunggu, menambah satu komponen yang harus dirawat dan diskalakan, dan perlu diputuskan apakah bersifat permanen atau dibongkar setelah migrasi selesai. Karena kedua sistem dapat punya tingkat kepercayaan berbeda, data yang melintas juga diperiksa di lapisan ini.

Analoginya seperti juru bahasa dalam rapat dengan mitra dari negara lain. Kedua pihak tetap berbicara dalam bahasanya sendiri, dan juru bahasa menerjemahkan maksudnya. Tanpa juru bahasa, salah satu pihak terpaksa belajar bahasa pihak lain, termasuk istilah-istilah yang sudah usang.

Dua hal pada masa transisi sering tertukar. Pembagiannya begini. Fasad berada di depan dan mengarahkan permintaan dari klien ke sistem lama atau sistem baru; fasad tidak mengubah isi permintaan. *Anticorruption layer* berada di belakang, di antara sistem baru dan sistem lama, dan menerjemahkan isi permintaan dari satu model ke model lain. Misalkan SIMRS Bandung versi 3.9 memakai nomor rekam medis lokal delapan digit, sedangkan SIMRS Jakarta versi 5.2 memakai sepuluh digit dengan awalan kode rumah sakit. Layanan Identitas Pasien dari Bab 10 memakai satu nomor induk pasien. Lapisan penerjemah memetakan ketiganya, sehingga Layanan Janji Temu yang baru tidak perlu mengenal lima format nomor lama.

14.2.4 Memilih Irisan Pertama

Irisan pertama menentukan kepercayaan direksi terhadap seluruh peta jalan. Irisan yang terlalu besar berisiko gagal. Irisan yang terlalu kecil tidak menunjukkan nilai apa pun. Lima kriteria membantu memilihnya: nilai bisnis yang terukur, batas yang jelas dan sedikit dependensi (mengikuti *bounded context* Bab 6), permintaan yang dapat dicegat di satu titik, kemudahan dikembalikan ke sistem lama, dan risiko klinis yang rendah. Tabel 14.2 menilai lima kandidat Medikara Health dengan kriteria itu.

Janji temu unggul di semua kriteria. Identitas pasien bernilai paling strategis, tetapi sulit dibalik setelah data digabung, sehingga lebih aman menjadi irisan kedua setelah tim terlatih.

Tabel 14.2: Penilaian kandidat irisan pertama Medikara Health (penilaian rekaan)

Kandidat	Nilai bisnis	Batas dan dependensi	Dapat dicegat	Dapat dibalik	Risiko klinis
Janji temu	Tinggi: lonjakan sekitar 60 persen saat vaksinasi (Bab 7)	Jelas	Ya, lewat Medikara+	Ya, ubah rute	Rendah
Identitas pasien	Tinggi: dasar "Satu Pasien, Satu Rekam Medis"	Jelas, tetapi banyak pemakai	Sebagian	Sulit setelah data digabung	Sedang
Klaim BPJS (Badan Penyelenggara Jaminan Sosial) Kesehatan	Tinggi: 22 dari 63 gangguan SIMRS Jakarta (Bab 7)	Terkait penagihan	Ya	Sedang	Rendah
Resep dan farmasi	Sedang	Terkait rekam medis dan stok	Tidak, dipanggil dari dalam SIMRS	Sulit	Tinggi
Hasil laboratorium	Sedang	Terkait alat analisis (Retain, Bab 4)	Sebagian	Sedang	Tinggi

Kasus Medikara Health: Memindahkan Janji Temu tanpa Menghentikan Rumah Sakit

ADR-001 di Bab 7 menetapkan syarat peninjauan: modul pendaftaran dan jadwal dipisah menjadi layanan bila lonjakan janji temu melampaui kapasitas SIMRS dua kali dalam enam bulan. Misalkan syarat itu terpenuhi pada 2026 setelah dua program vaksinasi. Medikara+ menangani sekitar 28.000 janji temu per bulan (Bab 3). Saat ini setiap janji temu memanggil modul jadwal dokter dan modul pendaftaran di SIMRS setiap rumah sakit (Bab 9).

Tim transformasi menyusun pemindahan dalam tujuh langkah. Pertama, fasad dipasang di depan panggilan Medikara+ ke modul jadwal. Kedua, selama empat minggu layanan baru menerima salinan setiap permintaan tanpa melayani pasien, lalu hasilnya dibandingkan dengan hasil SIMRS. Misalkan 99,2 persen hasil sama; selisih 0,8 persen berasal dari jadwal cuti dokter Depok yang dicatat manual dan tidak terbaca layanan baru. Langkah ketiga sampai ketujuh memindahkan rute per rumah sakit, dari yang paling sedikit janji temunya: Depok (sekitar 3.100 per bulan), Bogor (3.800), Bekasi

(5.600), Bandung (6.100), dan Jakarta (9.400). Jumlahnya 28.000.
Setiap rumah sakit dipindah paling cepat empat minggu setelah rumah sakit sebelumnya, dengan dua syarat: tingkat keberhasilan membuat janji temu memenuhi SLO 99,5 persen dari Bab 11, dan rute balik ke SIMRS sudah diuji dalam 10 menit. Setelah Jakarta stabil, modul jadwal SIMRS dibuat hanya dapat dibaca, lalu dinonaktifkan untuk janji temu. Data kunjungan tetap dibuat di modul pendaftaran SIMRS lewat *anticorruption layer* sampai SIMRS terpadu berjalan.
Rencana ini punya dua konsekuensi. Pertama, pada langkah pertama kegagalan paling banyak menyentuh sekitar 103 janji temu per hari di Depok ($3.100/30$), bukan seluruh 28.000 janji temu sebulan. Kedua, selama sekitar tujuh bulan Medikara membayar dua jalur sekaligus: fasad, lapisan penerjemah, dan modul lama yang masih dirawat. Biaya itu dicatat sebagai biaya transisi, bukan disembunyikan dalam biaya operasi.

Pendalaman Teknis (opsional): Mode Bayangan dan Pemindahan Data

Fasad umumnya diwujudkan dengan *API gateway*, yaitu gerbang Application Programming Interface (API) yang mendukung aturan rute per *header* atau per jalur, misalnya rute berdasarkan kode rumah sakit. Mode bayangan dijalankan dengan pencerminan lalu lintas (*traffic mirroring*): salinan permintaan dikirim ke layanan baru, tetapi jawabannya dibuang setelah dibandingkan dan dicatat.
Data adalah bagian tersulit. Contoh Microsoft untuk basis data monolit memakai tiga fase [101]. Fase pertama, layanan baru masih membaca dan menulis tabel domainnya di basis data lama. Fase kedua, tabel domain disalin ke basis data baru dengan proses *extract, transform, load*, lalu perubahan berikutnya disinkronkan dengan *change data capture*, dan konsistensi kedua basis data diperiksa. Fase ketiga, basis data baru menjadi sumber kebenaran, lalu tabel lama dihapus. Kembali ke basis data lama masih mudah selama fase kedua. Setelah tabel lama dihapus, kembali berarti memulihkan objek lama dan memutar ulang perubahan, sehingga penghapusan tabel lama sengaja ditaruh sebagai langkah terakhir.

14.3 Menyusun Gelombang Modernisasi

Satu irisan belum menjadi peta jalan. Pertanyaan berikutnya: bagaimana 46 aplikasi diurutkan, dan apa yang harus siap sebelum setiap kelompok dimulai? Bahan jawabannya sudah dikumpulkan sepanjang semester. Tabel 14.3 merangkum keputusan dari Bab 2 sampai Bab 13 yang dibawa ke peta jalan.

Tabel 14.3: Keputusan dari Bab 2 sampai Bab 13 yang menjadi bahan peta jalan Medikara Health

Bab	Artefak	Keputusan yang dibawa
2	Matriks kendala penempatan	Rekam medis dan data pasien ditempatkan di *region* Indonesia
3	Model biaya dan sasaran nilai	Biaya infrastruktur SIMRS per kunjungan rumah sakit (RS) Bekasi turun 15 persen dalam dua tahun, dari Rp5.619 menjadi paling tinggi Rp4.776; biaya Medikara+ Rp2.036 per janji temu
4	Portofolio 7R dan *landing zone*	46 aplikasi: Retire 6, Retain 5, Rehost 10, Relocate 15 (termasuk Medikara+), Repurchase 3, Replatform 2, SIMRS diperdebatkan 5; *landing zone* 8 minggu; Cloud Center of Excellence (CCoE)
5	Kartu skor *cloud-native*	Rilis Medikara+ masih 37 langkah manual
6	Peta domain	*Bounded context* menjadi batas modul dan kandidat layanan
7	ADR-001	SIMRS terpadu bergaya *modular monolith*; janji temu calon pertama yang dipisah
8	Matriks *runtime*	Platform as a Service (PaaS) untuk layanan baru pertama; *managed* Kubernetes ditinjau bila layanan melewati 15
9	Katalog *event*	Alur janji temu berbasis *event*
10	Peta kepemilikan data	Layanan Identitas Pasien memiliki 31 kolom identitas; pemindahan bertahap dengan *strangler fig*
11	SLO	Janji temu 99,5 persen, hasil laboratorium 99,0 persen, klaim BPJS Kesehatan 98,0 persen
12	Rencana Disaster Recovery (DR)	Rekam medis dengan Recovery Time Objective (RTO) 1 jam dan Recovery Point Objective (RPO) 5 menit; biaya DR tiga layanan sekitar Rp724 juta per tahun
13	Model operasi *delivery*	52 staf teknologi informasi (TI); tim platform 6 orang; sasaran SIMRS rilis bulanan lalu mingguan

14.3.1 Gelombang, Dependensi, Risiko, dan Nilai

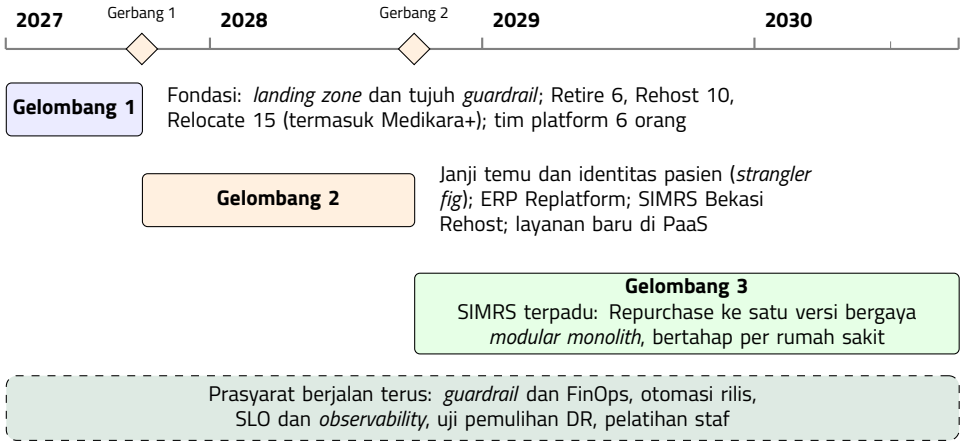
Gelombang modernisasi adalah kelompok aplikasi yang dipindah atau dimodernisasi dalam satu periode, karena saling bergantung atau karena membutuhkan prasyarat yang sama. Tiga kriteria menentukan isi dan urutannya. Dependensi menentukan apa yang harus pindah bersama; aplikasi yang saling memanggil setiap menit tidak boleh dipisah ke gelombang berbeda tanpa rencana jaringan (peta dependensi Bab 4). Risiko menentukan apa yang sebaiknya ditunda sampai tim lebih terlatih, baik risiko klinis, regulasi, maupun teknis. Nilai menentukan apa yang didahulukan agar manfaat terasa lebih awal (sasaran nilai Bab 3). Usulan 7R dari Bab 4 [9] memberi jenis pekerjaan, sedangkan matriks *runtime* Bab 8 memberi tempat berjalannya. Setiap gelombang juga punya prasyarat platform dan tim: tidak ada aplikasi yang mendarat sebelum *landing zone*, *guardrail*, dan pemantauannya siap.

Analoginya seperti pindahan kantor bertahap per lantai. Bagian keuangan dan akuntansi yang saling bertukar berkas setiap jam pindah pada akhir pekan yang

sama. Ruang server dan jaringan di gedung baru disiapkan lebih dulu, sebelum lantai pertama pindah. Bagian yang paling sibuk menjelang tutup buku tidak dipindah pada bulan Desember.

Gambar 14.3 memperlihatkan tiga gelombang Medikara Health dalam 42 bulan, dan Tabel 14.4 merinci isinya. Teknik penyusunan peta jalan dan perencanaan migrasi tingkat *enterprise*, termasuk portofolio inisiatif lintas unit, dibahas rinci di IT40513. Bab ini memusatkan perhatian pada bagian teknis peta jalan.

Peta jalan IT40513 Bab 12 menjadi induk, sehingga tanggal gelombang mengikuti paket kerjanya. Gelombang 1 berjalan bersama P4 *landing zone*, April sampai September 2027, setelah P1 kontrol akses selesai Maret 2027. Pembangunan 8 minggu pada Bab 4 berada di dalam rentang P4, sedangkan pemindahan 31 aplikasi berada di luar P4 dan hanya diatur pada bab ini. Gelombang 2 memuat sisi teknis P3 identitas pasien yang selesai Maret 2028 dan P7 janji temu lintas fasilitas yang mulai April 2028. P5 konsolidasi SIMRS juga mulai April 2028, dan enam bulan pertamanya berisi persiapan yang berimpit dengan gelombang 2. Gelombang 3 memuat peralihan rumah sakitnya: Depok Desember 2028, Bekasi dan Bogor Desember 2029, lalu Jakarta dan Bandung September 2030.



Gambar 14.3: Tiga gelombang modernisasi Medikara Health, April 2027 sampai September 2030, dengan gerbang keputusan di antara gelombang

Dua butir portofolio Bab 4 tidak masuk tabel. Aplikasi apotek tidak menunggu gelombang 1: Repurchase dikerjakan pada November dan Desember 2026, karena dukungan vendor berhenti akhir 2026. Lima aplikasi Retain, termasuk sistem laboratorium yang terhubung kabel ke alat analisis, tetap di tempat asal dan ditinjau ulang pada gerbang kedua.

Tabel 14.4: Isi tiga gelombang modernisasi Medikara Health (rencana rekaan)

Butir	Gelombang 1: April sampai September 2027	Gelombang 2: Oktober 2027 sampai September 2028	Gelombang 3: Oktober 2028 sampai September 2030
Isi dan strategi 7R	Retire 6, Rehost 10, Relocate 15 (14 aplikasi klaster Jakarta dan Medikara+)	Refactor Medikara+ lewat Layanan Janji Temu, dan Layanan Identitas Pasien (keduanya *strangler fig*); Replatform 2, termasuk Enterprise Resource Planning (ERP); Repurchase 2; SIMRS Bekasi Rehost	SIMRS terpadu: Retain lalu Repurchase ke satu versi bergaya *modular monolith*, bertahap per rumah sakit
Prasyarat platform	*Landing zone*, tujuh *guardrail*, tag wajib, akun vendor bersama dicabut	*Pipeline* Continuous Integration / Continuous Delivery (CI/CD) dan GitOps, PaaS, *event broker*, SLO dan *observability*	Lokasi kedua untuk *warm standby* rekam medis, uji pemulihan lulus
Prasyarat tim	Tim platform 6 orang terbentuk; staf bersertifikat *cloud* naik dari 4 menjadi 10	Tim domain janji temu dan tim Medikara+ mandiri	Empat tim domain SIMRS; staf klinis dan administrasi dilatih
Sasaran SLO	Ketersediaan tidak turun dari tingkat sekarang	Janji temu 99,5 persen	Hasil laboratorium 99,0 persen; klaim 98,0 persen
Sasaran biaya	Tagihan dalam anggaran; biaya tak teralokasi di bawah 5 persen	Rp2.036 per janji temu tetap terjaga	RS Bekasi paling tinggi Rp4.776 per kunjungan
Risiko utama	Dependensi tersembunyi; sambungan internet rumah sakit	Data ganda selama transisi; kapasitas tim	Pelatihan sekitar 2.900 staf; penyatuan lima basis data 12 sampai 18 bulan

14.3.2 Gerbang Keputusan di antara Gelombang

Peta jalan 42 bulan tidak dapat dikunci sejak hari pertama. Gerbang keputusan adalah titik tinjauan di akhir setiap gelombang, tempat direksi memutuskan lanjut, menunda, atau mengubah isi gelombang berikutnya berdasarkan bukti. Gerbang 1 pada September 2027 menuntut tiga bukti: seluruh akun *cloud* berada di bawah *landing zone*, biaya tak teralokasi di bawah 5 persen, dan rilis Medikara+ sudah otomatis. Bukti itu tiba sebulan sebelum titik keputusan K2 pada IT40513, yang mensyaratkan *landing zone* selesai. Gerbang 2 pada September 2028 menuntut janji temu kelima rumah sakit berjalan di layanan baru dengan SLO terpenuhi tiga bulan berturut-turut, serta uji pemulihan rekam medis yang memenuhi RTO 1 jam

(Bab 12). Di dalam gelombang 3, keputusan lanjut mengikuti titik keputusan K3 pada Maret 2029 dan K4 pada Oktober 2029.

Gerbang mengubah sifat peta jalan. Peta jalan tidak lagi berupa janji tanggal, tetapi rangkaian keputusan yang diambil saat data tersedia. Investasi gelombang berikutnya juga hanya dicairkan setelah gerbang dilewati, sehingga kerugian terburuk terbatas pada satu gelombang.

Dua contoh kasus:

1. Misalkan peta dependensi Bab 4 memperlihatkan bahwa portal dokumen akreditasi (Rehost) membaca berkas dari server berbagi yang sama dengan SIMRS Jakarta setiap malam. Memindahkan portal itu ke *cloud* pada gelombang 1 berarti 4 gigabyte berkas melintasi sambungan internet setiap malam. Tim memilih memindahkan server berbagi itu lebih dulu, sehingga dua aplikasi bergeser dari bulan ketiga ke bulan kelima gelombang 1.
2. Bab 4 mencatat hanya 4 dari 52 staf TI yang bersertifikat *cloud*. Misalkan setiap aplikasi Relocate dan Rehost membutuhkan satu orang bersertifikat sebagai penanggung jawab teknis, dan satu orang hanya sanggup memegang tiga aplikasi dalam satu gelombang. Untuk 25 aplikasi dibutuhkan sekitar 9 orang. Pelatihan dan sertifikasi 6 orang tambahan harus selesai sebelum bulan kedua, atau gelombang 1 diperpanjang.

Kasus Medikara Health: Memutuskan Lima SIMRS yang Diperdebatkan

Bab 4 menandai lima SIMRS sebagai diperdebatkan: Retain lalu Repurchase ke satu versi, atau Rehost kelima versi karena perangkatnya berumur 6 sampai 9 tahun. Bab 7 kemudian memilih bentuk akhirnya, yaitu satu SIMRS terpadu bergaya *modular monolith*. SIMRS di kelima rumah sakit berasal dari satu keluarga produk, sehingga jalur Repurchase berarti beralih ke satu versi terbaru yang disusun ulang menjadi modul-modul berbatas tegas.

Peta jalan memakai keduanya secara terarah. Jalur utama adalah Retain selama gelombang 1 dan 2, sambil janji temu dan identitas pasien dipindah dengan *strangler fig*, lalu Repurchase ke SIMRS terpadu pada gelombang 3. Satu pengecualian dibuat untuk RS Bekasi. Misalkan perangkat keras SIMRS Bekasi yang paling tua, sembilan tahun. Versi perangkat lunak tertua berada di Depok menurut IT40513, tetapi keputusan ini ditentukan oleh umur perangkat. Bab 3 sudah menghitung bahwa Rehost dengan penyesuaian ukuran menurunkan biaya per kunjungan dari Rp5.619 menjadi sekitar Rp4.524. SIMRS Bekasi di-Rehost pada gelombang 2, lalu ikut beralih ke SIMRS terpadu pada gelombang 3.

Keputusan ini punya dua konsekuensi. Pertama, Bekasi dipindah dua

kali. Biaya migrasi tambahan itu dibandingkan dengan risiko perangkat berumur sembilan tahun selama dua tahun lagi dan dengan sasaran nilai Bab 3 yang jatuh tempo akhir 2028. Kedua, empat rumah sakit lain tetap bergantung pada perangkat lama sampai gilirannya tiba, paling lambat September 2030, sehingga kontrak pemeliharaan perangkat diperpanjang dan cadangan terpisah yang tidak dapat diubah (Bab 12) dipasang lebih dulu di sana.

14.4 Guardrail dan Policy as Code pada Landing Zone

Bab 4 menyebut aturan yang dijaga CCoE sebagai *guardrail*. Pertanyaan kemudian muncul: bagaimana menjaga 46 aplikasi dan beberapa tim tetap patuh tanpa membuat setiap perubahan menunggu persetujuan manual? Sebelum *landing zone* ada, Medikara punya 9 akun *cloud* yang dibuka terpisah, tiga di antaranya tanpa autentikasi dua faktor (Bab 4). Pemeriksaan manual atas setiap perubahan memperlambat tim. Tanpa pemeriksaan sama sekali, kelemahan yang sama terulang.

14.4.1 Guardrail sebagai Batas, Bukan Gerbang

Microsoft CAF mendefinisikan tata kelola *cloud* sebagai cara organisasi mengendalikan pemakaian layanan *cloud* dengan memasang *guardrail*, yaitu sekumpulan kebijakan, prosedur, dan alat yang menetapkan kegiatan yang boleh dan tidak boleh [94]. Cakupannya meliputi keamanan, kepatuhan regulasi, operasi, biaya, data, penyediaan sumber daya, dan AI (Artificial Intelligence). Prosesnya punya lima langkah: membentuk tim tata kelola, menilai risiko, menuliskan kebijakan, menegakkan kebijakan, dan memantau kepatuhan. Langkah pertama dilakukan sekali, sedangkan empat langkah berikutnya berulang terus. Di dalam CAF, tata kelola dan keamanan masing-masing punya fase sendiri, yaitu Govern dan Secure [95].

AWS Control Tower memakai istilah kendali (*control*) dan menyatakan istilah itu sama artinya dengan *guardrail* [22]. Kendali adalah aturan tingkat tinggi yang ditulis dalam bahasa biasa untuk tata kelola yang terus berjalan. Kendali dipasang pada satu unit organisasi, sehingga berlaku untuk setiap akun di dalamnya. Ada tiga jenis perilaku: preventif, detektif, dan proaktif.

Analoginya seperti pagar pengaman di jalan pegunungan. Pagar tidak memperlambat mobil dan tidak menanyai setiap pengemudi. Pagar hanya mencegah mobil jatuh ke jurang. Palang pintu yang memeriksa setiap mobil memang lebih ketat, tetapi antreannya mengular sampai ke kaki bukit.

Dua hal pada tata kelola sering tertukar. Pembagiannya begini. *Guardrail* adalah batas yang diperiksa otomatis; tim bebas bergerak di dalam batas tanpa meminta izin. Gerbang persetujuan adalah titik tunggu yang menuntut seseorang memeriksa dan menyetujui. Gerbang tetap diperlukan untuk keputusan yang jarang dan berdampak besar, misalnya membuka *region* baru. Untuk perubahan harian, gerbang persetujuan menjadi antrean. Tabel 14.5 menjelaskan tiga jenis kendali dengan contoh Medikara.

Tabel 14.5: Tiga jenis kendali *guardrail* dan contohnya di Medikara Health

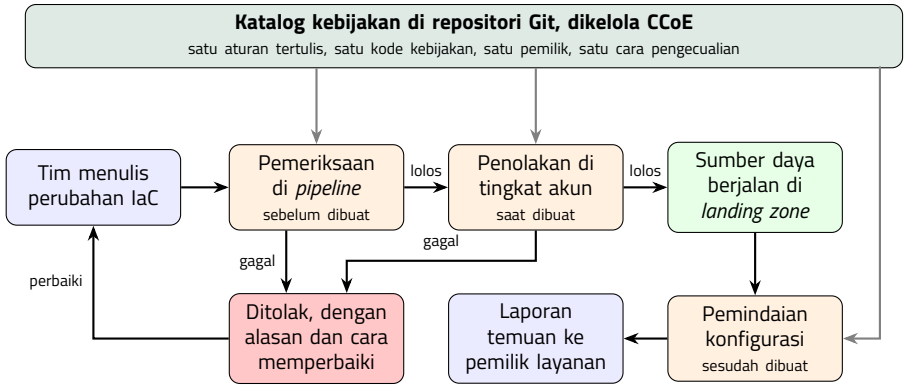
Jenis	Kapan bekerja	Cara kerja umum	Contoh Medikara
Proaktif	Sebelum sumber daya dibuat	Rancangan perubahan diperiksa di *pipeline*; yang melanggar tidak pernah dibuat	Rencana basis data tanpa tag kelas data ditolak saat *pull request*
Preventif	Saat sumber daya hendak dibuat	Permintaan ke penyedia ditolak di tingkat akun atau unit organisasi	Pembuatan sumber daya di luar *region* Indonesia ditolak untuk akun rekam medis
Detektif	Sesudah sumber daya berjalan	Konfigurasi dipindai berkala; temuan dilaporkan ke pemilik	Penyimpanan objek yang diubah menjadi publik ditandai dalam satu jam

14.4.2 Policy as Code

Policy as code adalah cara menuliskan aturan tata kelola sebagai kode yang dapat dibaca mesin. Kode kebijakan disimpan di repositori, ditinjau seperti kode aplikasi, diuji, lalu dijalankan otomatis di setiap pemeriksaan. OPA adalah mesin kebijakan serbaguna bersumber terbuka yang dipakai untuk keperluan ini [113]. Kebijakannya ditulis dalam bahasa deklaratif Rego. Perangkat lunak yang membutuhkan keputusan mengirim data terstruktur, umumnya JSON, ke OPA. OPA menilai data itu terhadap kebijakan, lalu mengembalikan keputusan. Pemisahan ini membuat aplikasi tidak perlu menanam logika kebijakan sendiri. Contoh pemakaiannya meliputi pemeriksaan sebelum objek diterima Kubernetes, *pipeline* CI/CD, *API gateway*, dan pemeriksaan rencana Terraform. OPA berstatus proyek *graduated* di CNCF (Cloud Native Computing Foundation).

Gambar 14.4 memperlihatkan tempat *policy as code* dalam alur perubahan. Tim menulis perubahan sebagai IaC (Infrastructure as Code). Satu katalog kebijakan di repositori Git dibaca oleh ketiga jenis pemeriksaan, sehingga aturan yang sama tidak ditulis tiga kali dengan tiga cara.

Pendekatan ini punya dua konsekuensi. Pertama, pesan penolakan menjadi bagian dari rancangan. Penolakan yang hanya berbunyi "ditolak" mendorong tim mencari jalan pintas. Penolakan yang menyebut aturan yang dilanggar dan



Gambar 14.4: *Policy as code* dalam alur perubahan: pemeriksaan proaktif di *pipeline*, penolakan preventif di tingkat akun, dan pemindaian detektif sesudah sumber daya berjalan

cara memperbaikinya mengajari tim. Kedua, setiap *guardrail* membutuhkan jalur pengecualian yang resmi: siapa yang menyetujui, untuk sumber daya apa, dan sampai kapan. *Guardrail* tanpa jalur pengecualian mendorong tim membuka akun di luar *landing zone*, dan masalah sembilan akun terpisah muncul kembali.

14.4.3 Katalog Guardrail Medikara Health

Tabel 14.6 memuat tujuh *guardrail* awal Medikara Health untuk biaya, keamanan, dan lokasi data. Setiap baris ditulis dalam bahasa biasa lebih dulu, karena direksi dan auditor harus dapat membacanya. Terjemahannya ke kode kebijakan dikerjakan anggota teknis CCoE.

Dua contoh kasus:

1. Bab 10 mencatat 23 akun dengan hak tulis ke tabel pasien SIMRS Bekasi, termasuk satu akun vendor laboratorium. Kebiasaan yang sama mudah terbawa ke *cloud*. Misalkan pemindaian detektif pada bulan pertama menemukan 7 akun bersama di *landing zone*. Ketujuhnya diganti identitas perorangan dalam 30 hari, dan temuan audit kontrol akses dapat ditutup dengan bukti laporan pemindaian.
2. Misalkan dalam tiga bulan pertama pemeriksaan proaktif menolak 41 dari 620 rancangan perubahan, atau sekitar 6,6 persen. Sebanyak 33 penolakan karena tag tidak lengkap dan diperbaiki dalam hari yang sama. Dua penolakan karena *region* di luar Indonesia, dan keduanya berasal dari templat lama yang disalin dari internet. Tanpa pemeriksaan otomatis, 620 rancangan itu membutuhkan peninjauan manual, misalkan 20 menit per rancangan atau sekitar 207 jam kerja.

Tabel 14.6: Tujuh *guardrail* awal Medikara Health (rancangan rekaan)

No.	Rumusan dalam bahasa biasa	Jenis	Risiko yang dicegah	Pemilik
G1	Data rekam medis dan data pribadi spesifik hanya disimpan di *region* yang berlokasi di Indonesia, termasuk cadangannya	Proaktif dan preventif	Pelanggaran Peraturan Menteri Kesehatan (Permenkes) 24/2022 dan Undang-Undang (UU) tentang Pelindungan Data Pribadi (PDP)	Wakil keamanan
G2	Penyimpanan data pasien dienkripsi dengan kunci yang dikelola Medikara	Proaktif dan detektif	Data terbaca pihak yang tidak berhak	Wakil keamanan
G3	Setiap sumber daya punya tag pemilik, layanan, lingkungan, pusat biaya, dan kelas data	Proaktif	Tagihan tidak dapat dialokasikan; kelas data tidak diketahui	Wakil keuangan
G4	Tidak ada penyimpanan data pasien yang dapat diakses publik	Preventif dan detektif	Kebocoran data seperti insiden 2025	Wakil keamanan
G5	Akses manusia memakai identitas perorangan dan autentikasi dua faktor; tidak ada akun bersama vendor	Preventif dan detektif	Temuan audit kontrol akses yang lemah	Wakil keamanan
G6	Log audit dikirim ke akun log terpusat dan tidak dapat dihapus selama 1 tahun	Preventif	Jejak insiden hilang	Wakil operasi
G7	Setiap akun punya anggaran bulanan dengan ambang peringatan 50, 80, dan 100 persen	Proaktif dan detektif	Tagihan melonjak tanpa diketahui	Wakil keuangan

Pendalaman Teknis (opsional): Kebijakan Region dengan Open Policy Agent

Listing 14.1 adalah kebijakan Rego untuk G1. Kebijakan menerima daftar sumber daya dalam format JSON yang disederhanakan, lalu menghasilkan himpunan pesan penolakan. Sumber daya berkelas data rekam medis atau data pribadi spesifik ditolak bila *region*-nya di luar empat *region* Indonesia yang tercatat di Bab 2. Sumber daya tanpa tag kelas data juga ditolak, karena tanpa tag itu kebijakan tidak dapat memutuskan apa pun. Sintaks yang dipakai adalah Rego v1, yaitu sintaks bawaan OPA 1.x.

```
1 package medikara.guardrail.region
2
3 # Kode region yang berlokasi di Indonesia (Bab 2). Bila penyedia
```

```
# membuka region baru, hanya daftar ini yang diubah lewat review.
region_indonesia := {
  "ap-southeast-3",
  "asia-southeast2",
  "indonesiacentral",
  "ap-batam-1"
}

# Kelas data yang wajib tinggal di Indonesia (Permenkes 24/2022 dan
# UU PDP). Kelas lain diatur guardrail terpisah.
kelas_wajib_lokal := {"rekam-medis", "data-pribadi-spesifik"}

# Pesan dikumpulkan sebagai himpunan agar pipeline menampilkan semua
# pelanggaran sekaligus, bukan berhenti di pelanggaran pertama.
tolak contains pesan if {
  some sumber in input.sumber_daya
  sumber.tag["kelas-data"] in kelas_wajib_lokal
  not sumber.region in region_indonesia
  pesan := sprintf("%s: kelas data %s tidak boleh di region %s",
    [sumber.nama, sumber.tag["kelas-data"], sumber.region])
}

# Tanpa tag kelas data, aturan di atas tidak dapat memutuskan,
# sehingga sumber daya ditolak dan pemiliknya diminta melengkapi.
tolak contains pesan if {
  some sumber in input.sumber_daya
  not sumber.tag["kelas-data"]
  pesan := sprintf("%s: tag kelas-data wajib diisi", [sumber.nama])
}
```

Listing 14.1: Kebijakan *region* untuk G1, tersedia di `sources/chapter14/contoh-opa/guardrail-region.rego`

Kebijakan dijalankan dengan perintah `opa eval` terhadap berkas contoh `sources/chapter14/contoh-opa/rencana-sumber-daya.json`, seperti tertulis di `sources/chapter14/README.md`. Dari empat sumber daya contoh, dua ditolak: cadangan rekam medis di *region* Singapura dan penyimpanan log tanpa tag kelas data. Situs informasi publik di Singapura lolos, karena kelas datanya publik. Pada pemakaian nyata, masukannya diambil dari rencana Terraform yang diubah ke JSON, sehingga jalur atribut di kebijakan perlu disesuaikan dengan format penyedia yang dipakai. Kebijakan diuji pada akun uji milik tim sendiri sebelum dipasang di *pipeline* bersama.

14.5 FinOps pada Fase Operate

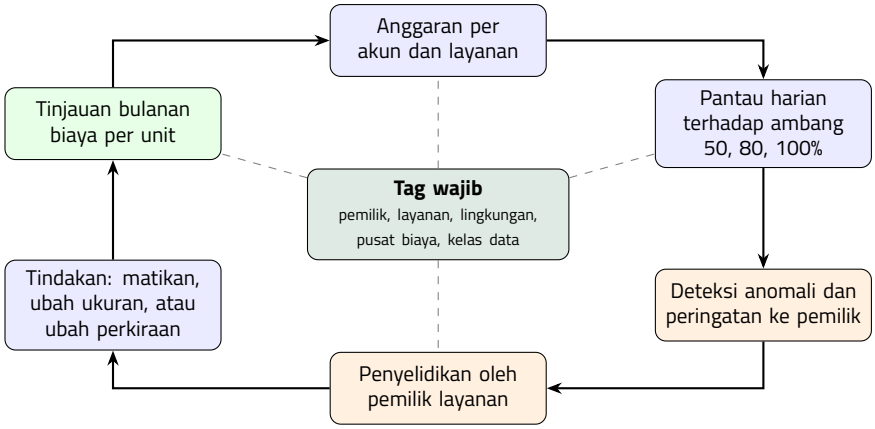
Guardrail menjaga aturan. Pertanyaan berikutnya: siapa yang menjaga tagihan setelah aplikasi pindah? Bab 3 membahas FinOps Framework dan tiga fasenya, yaitu Inform, Optimize, dan Operate [53]. Fase Operate adalah tempat kebiasaan harian dibentuk: anggaran dipantau, lonjakan diselidiki, dan biaya per unit ditinjau setiap bulan. Pada fase inilah peta jalan membuktikan klaim penghematannya.

14.5.1 Anggaran, Deteksi Anomali, dan Kebijakan Tagging

Tiga *capability* FinOps menjadi tulang punggung fase ini. Budgeting adalah proses berjalan untuk menetapkan batas belanja teknologi, memantaunya, dan mengelolanya terhadap tujuan bisnis [51]. Organisasi menetapkan ambang selisih yang dapat diterima antara anggaran dan realisasi. Bila ambang terlewati, jalur eskalasinya bergerak dari tim teknis ke pemilik anggaran, lalu ke bagian keuangan, lalu ke pimpinan. Anomaly Management adalah kemampuan mendeteksi, memberi peringatan, menyelidiki, dan menyelesaikan kejadian biaya yang tidak terduga [50]. Anomali dikenali dari selisih terhadap pola belanja sebelumnya, sehingga pemakaian baru yang memang direncanakan pun dapat memicu peringatan. Allocation adalah cara membebankan biaya ke tim, proyek, atau unit yang bertanggung jawab, memakai akun, tag, label, dan metadata lain, termasuk pembagian biaya bersama [49]. Salah satu ukurannya adalah porsi biaya yang tidak dapat dialokasikan.

Analoginya seperti keuangan rumah tangga yang memakai amplop per pos belanja dan notifikasi bank. Amplop dapur, amplop sekolah, dan amplop listrik adalah anggaran per pemilik. Notifikasi bank untuk transaksi yang tidak biasa adalah deteksi anomali. Catatan di setiap struk belanja tentang untuk amplop mana adalah tag. Tanpa catatan itu, akhir bulan hanya menyisakan angka total tanpa penjelasan.

Gambar 14.5 memperlihatkan enam langkah fase Operate di Medikara Health. Tag wajib (G3) berada di tengah, karena setiap langkah membutuhkan jawaban atas pertanyaan “biaya ini milik siapa?”.



Gambar 14.5: Siklus fase Operate di Medikara Health: anggaran, pemantauan, deteksi anomali, penyelidikan, tindakan, dan tinjauan biaya per unit, dengan tag wajib sebagai pusat data

Tabel 14.7 memuat anggaran *cloud* bulanan Medikara pada akhir gelombang 1 beserta penerima peringatannya.

Tabel 14.7: Anggaran *cloud* bulanan Medikara Health pada akhir gelombang 1, dalam juta rupiah (angka rekaan)

Akun	Anggaran	Pemilik anggaran
Medikara+ produksi	60	Pemilik produk Medikara+
Klaster Relocate, 14 aplikasi	110	Kepala infrastruktur pusat
Aplikasi Rehost, 10 aplikasi	45	Kepala infrastruktur pusat
Seluruh lingkungan non-produksi	35	Kepala tim platform
Fondasi *landing zone*: jaringan, log, keamanan	40	Kepala tim platform
Jumlah	**290**	
Ambang peringatan: 50 persen ke pemilik anggaran; 80 persen ke pemilik dan wakil keuangan CCoE; 100 persen ke direktur TI, dan sumber daya baru di akun itu menunggu persetujuan.		

Dua contoh kasus:

1. Misalkan pada Agustus 2027 basis data uji beban di akun non-produksi lupa dimatikan. Belanja harian akun itu naik dari rata-rata Rp1,1 juta menjadi Rp5,7 juta. Deteksi anomali memberi peringatan pada hari pertama, sedangkan ambang 80 persen anggaran (Rp28 juta) baru terlewati pada hari kelima. Empat hari lebih cepat berarti sekitar Rp18 juta tidak terbuang ($4\times$ Rp4,6 juta).
2. Misalkan pada Januari 2027, 23 persen tagihan *cloud* tidak dapat dialokasikan ke pemilik mana pun, sebagian besar dari akun lama vendor. Setelah G3 ditegakkan secara proaktif dan akun lama dipindah ke *landing zone*, porsi itu turun menjadi 4 persen pada September 2027. Sisanya adalah biaya bersama seperti jaringan dan log, yang dibagi proporsional menurut pemakaian.

14.5.2 Tinjauan Biaya per Unit dan Executive Strategy Alignment

Anggaran menjawab "apakah belanja sesuai rencana". Biaya per unit menjawab "apakah belanja itu efisien". Setiap bulan, komite biaya teknologi dari Bab 3 meninjau dua metrik: biaya infrastruktur SIMRS per kunjungan dan biaya Medikara+ per janji temu. Tagihan yang naik tidak selalu buruk. Bila janji temu naik 30 persen dan tagihan naik 10 persen, biaya per janji temu justru turun. Pembacaan ini memakai *unit economics* yang dibahas Storment dan Fuller [135].

FinOps Framework 2026 memuat *capability* Executive Strategy Alignment. *Capability* ini mengaitkan belanja, pemakaian, dan adopsi teknologi dengan strategi bisnis, sehingga pimpinan dapat membandingkan pilihan, mengelola *tradeoff*, dan mengawasi investasi demi nilai [52]. Cakupannya meliputi keselarasan dengan prioritas pimpinan, strategi investasi multitahun, prioritas produk, dan dukungan keputusan strategis. Bagi Medikara, setiap gerbang keputusan adalah keputusan

investasi multitahun. Laporan triwulan komite biaya teknologi kepada direksi menyandingkan biaya gelombang dengan sasaran nilai, bukan hanya total tagihan.

14.6 Kedaulatan Data dan Sovereign Cloud

G1 memastikan rekam medis disimpan di Indonesia. Bab 2 sudah membedakan residensi, kedaulatan, dan lokalisasi data. Pertanyaan kemudian muncul: bila datanya sudah di Indonesia, apakah urusan kedaulatan sudah selesai? Jawabannya belum, karena lokasi hanya satu dari beberapa bentuk kendali.

14.6.1 Tiga Dimensi Kedaulatan

Panduan kedaulatan dalam Microsoft CAF mendefinisikan kedaulatan digital sebagai kemampuan organisasi mengendalikan data, teknologi, dan tata kelola operasinya sesuai kebutuhan hukum, regulasi, dan bisnis [100]. Kendali itu dicapai lewat gabungan kendali teknis, operasional, dan kontraktual. Kedaulatan adalah bagian dari kewajiban kepatuhan yang lebih luas, dengan fokus pada kendali, yurisdiksi, residensi data, akses administratif, dan tata kelola operasi. Panduan itu juga menegaskan bahwa kedaulatan tidak dicapai dengan isolasi semata. Kendali diterapkan seimbang dengan kebutuhan setiap *workload*, tanpa menciptakan titik gagal tunggal atau ketergantungan baru. Tabel 14.8 menguraikan tiga dimensinya.

Tabel 14.8: Tiga dimensi kedaulatan dan penerapannya di Medikara Health

Dimensi	Pertanyaan	Contoh kendali	Penerapan di Medikara
Kedaulatan data	Di mana data disimpan, siapa yang dapat membacanya, dan hukum mana yang berlaku?	*Region* lokal, enkripsi dengan kunci milik pelanggan, klasifikasi data	G1 dan G2; kelas data pada tag
Kedaulatan operasional	Siapa yang mengoperasikan dan dapat mengakses sistem, dari mana, dan dengan pengawasan apa?	Pencatatan dan persetujuan akses operator, petugas operasi di dalam negeri	G5 dan G6; akses vendor dan operator penyedia dicatat dan ditinjau bulanan
Kedaulatan teknologi	Apakah organisasi dapat pindah penyedia atau tetap berjalan bila penyedia bermasalah?	Format terbuka, rencana keluar, uji ekspor data	Rencana keluar untuk sekitar 15 terabyte rekam medis (Bab 2)

Analoginya seperti menyewa brankas di bank. Lokasi brankas di kota sendiri adalah kedaulatan data. Aturan pegawai bank mana yang boleh masuk ruang

brankas, dan buku tamu yang mencatatnya, adalah kedaulatan operasional. Kemampuan memindahkan isi brankas ke bank lain tanpa izin bank lama adalah kedaulatan teknologi. Brankas di kota sendiri belum aman bila kuncinya dipegang orang yang tidak diketahui.

14.6.2 Pilihan Sovereign Cloud

Penyedia menawarkan beberapa tingkat kendali. Microsoft, misalnya, membedakan tiga model: Sovereign Public Cloud, yaitu kendali kedaulatan di atas *region* publik; Sovereign Private Cloud, yaitu platform penyedia yang berjalan di perangkat dan fasilitas milik pelanggan; serta National Partner Cloud, yang dioperasikan mitra lokal di bawah tata kelola nasional dan hanya tersedia di sedikit negara seperti Jerman dan Prancis [100]. Penyedia lain memakai nama dan susunan berbeda, sehingga perbandingan dilakukan per kendali yang dibutuhkan, bukan per nama produk.

Panduan yang sama menyebut kedaulatan sebagai *tradeoff* antara risiko, keleluasaan, dan biaya. Makin ketat kendalinya, makin tinggi biayanya, karena isolasi dan tanggung jawab operasi bertambah. Model publik dengan kendali kedaulatan memakai harga standar ditambah biaya layanan pelengkap seperti kunci enkripsi milik pelanggan. Model privat menambah biaya perangkat dan operasi sendiri, dan waktu siapnya lebih lama [100]. Prinsipnya: setiap *workload* hanya mendapat kendali yang memang dibutuhkannya.

14.6.3 Status Regulasi Indonesia per Oktober 2026

Tabel 14.9 merangkum aturan yang membentuk *guardrail* lokasi data Medikara, beserta statusnya per Oktober 2026. Bab 2 membahas isi aturan ini lebih rinci. Kepatuhan regulasi, penilaian dampak pelindungan data, dan risiko pihak ketiga dibahas rinci di IT40813.

Satu rancangan masih bergerak. Pada 20 Oktober 2026, Direktur Jenderal Infrastruktur Digital Komdigi menyatakan pemerintah sedang menyiapkan rancangan Perpres tentang pusat data dan kabel bawah laut, dengan tujuan mendorong pembangunan pusat data ke wilayah timur Indonesia, tidak hanya Batam dan Jawa [130]. Berita itu tidak menyebut jadwal penetapan. Karena itu, peta jalan mencantumkan pemantauan regulasi sebagai butir tetap di setiap gerbang keputusan.

Dua contoh kasus:

1. Misalkan tim pengembang ingin memakai layanan AI untuk merangkum catatan dokter, dan layanan itu baru tersedia di *region* Singapura. G1 menolak permintaan untuk data rekam medis asli. Pengecualian hanya diberikan

Tabel 14.9: Status regulasi yang membentuk *guardrail* lokasi data, per Oktober 2026

Aturan	Status	Kaitan dengan *guardrail*
Peraturan Pemerintah (PP) 71/2019 tentang Penyelenggaraan Sistem dan Transaksi Elektronik (PSTE) [119]	Berlaku; rencana revisi masih dievaluasi Kementerian Komunikasi dan Digital (Komdigi)	Pasal 21 membolehkan Penyelenggara Sistem Elektronik (PSE) lingkup privat menyimpan data di luar negeri dengan syarat akses pengawasan, sebagai aturan umum
PP 28/2024 [123]	Berlaku	Pasal 966: pusat data kesehatan di dalam wilayah Indonesia; Pasal 972: transfer untuk tujuan spesifik dengan izin Menteri. Dasar utama G1
Permenkes 24/2022 tentang Rekam Medis [83]	Berlaku	Pasal 20: media penyimpanan dapat berupa *cloud* tersertifikasi; cadangan wajib di tempat berbeda
UU 27/2022 tentang PDP [120]	Berlaku penuh sejak Oktober 2024	Data kesehatan bersifat spesifik; Pasal 56 mengatur transfer ke luar negeri
PP 33/2026, aturan pelaksana UU PDP [1]	Ditetapkan 16 Juli 2026; berlaku 16 Januari 2027 menurut ringkasan firma hukum	Teks resmi dibaca di Jaringan Dokumentasi dan Informasi Hukum (JDIH) sebelum *guardrail* difinalkan
Rancangan Peraturan Presiden (Perpres) tentang pusat data dan kabel bawah laut [130]	Disiapkan Komdigi; sudah melalui konsultasi publik; belum ditetapkan	Isi tentang lokasi data belum dapat dipastikan; dipantau di setiap gerbang keputusan

untuk 2.000 catatan yang sudah dianonimkan, selama enam bulan uji coba, dengan persetujuan wakil keamanan dan petugas pelindungan data.

2. Bab 2 memperkirakan rekam medis Medikara mencapai sekitar 15 terabyte, sedangkan Bab 12 menghitung bahwa memindahkan 15 terabyte lewat sambungan 1 gigabit per detik memakan sekitar 33 jam. Kedaulatan teknologi tidak cukup ditulis di kontrak. Rencana keluar diuji setiap tahun dengan mengekspor 1 terabyte sampel ke lokasi lain dan membukanya dengan perangkat lunak di luar penyedia utama.

Kasus Medikara Health: Guardrail Lokasi Data untuk Rekam Medis Elektronik

Tim transformasi menurunkan G1 menjadi empat aturan yang dapat diperiksa. Pertama, sumber daya bertag kelas data `rekam-medis` hanya boleh dibuat di empat *region* Indonesia yang tercatat di Bab 2. Kedua, cadangan dan lokasi kedua untuk *warm standby* rekam medis (Bab 12) juga wajib berada di Indonesia. Karena setiap penyedia global hanya punya satu *region*

di Indonesia, lokasi kedua dipilih dari tiga pilihan Bab 12: penyedia lain di Indonesia, pusat data penyedia lokal, atau ruang server milik sendiri di kota lain. Ketiga, kunci enkripsi rekam medis dikelola Medikara, bukan kunci bawaan penyedia. Keempat, akses operator penyedia ke sistem rekam medis dicatat dan ditinjau bulanan oleh wakil keamanan.
Aturan ini punya dua konsekuensi. Pertama, layanan yang belum tersedia di *region* Indonesia tidak dapat dipakai untuk rekam medis, sehingga daftar layanan per *region* menjadi bahan rancangan, bukan catatan kaki. Kedua, biaya kunci milik sendiri dan lokasi kedua di Indonesia masuk ke anggaran gelombang 2 dan 3. Biaya itu adalah harga kedaulatan yang disadari, bukan pemborosan yang harus dipangkas.

14.7 Metrik Peta Jalan dan Pelaporan Kemajuan

Direksi akan menanyakan satu hal setiap triwulan: apakah peta jalan berjalan? Jawaban yang paling mudah disiapkan adalah jumlah aplikasi yang sudah dipindahkan. Jawaban itu juga yang paling menyesatkan.

14.7.1 Mengapa Jumlah Aplikasi yang Dipindahkan Menyesatkan

Pada akhir gelombang 1, Medikara dapat melaporkan 31 dari 46 aplikasi, atau sekitar 67 persen, sudah selesai. Angka itu benar, tetapi menyesatkan karena empat alasan. Pertama, ke-31 aplikasi itu adalah Retire, Rehost, dan Relocate, yaitu pekerjaan yang paling mudah. SIMRS, inti pelayanan, belum tersentuh. Kedua, aplikasi tidak setara: portal dokumen akreditasi dan SIMRS Jakarta dihitung sama-sama satu. Ketiga, angka itu tidak berkata apa pun tentang hasil: antrean pasien, kecepatan rilis, keandalan, dan biaya bisa saja tidak berubah atau malah memburuk. Keempat, metrik ini mendorong tim memilih pekerjaan yang mudah dihitung, bukan yang bernilai. Bab 12 memberi contoh serupa: "31 dari 46 aplikasi punya cadangan" terdengar baik, padahal hanya 3 yang cadangannya selamat dari *ransomware*.

Analoginya seperti program diet yang dilaporkan dengan jumlah kunjungan ke pusat kebugaran. Dua puluh kunjungan sebulan terdengar mengesankan, tetapi dokter menanyakan tekanan darah, kadar gula, dan berat badan. Kunjungan adalah kegiatan; tekanan darah adalah hasil.

14.7.2 Empat Keluarga Metrik

Empat keluarga metrik mengukur hasil, bukan kegiatan. Keluarga pertama adalah metrik DORA untuk kinerja pengiriman perangkat lunak (Bab 13). DORA kini

mengelompokkannya menjadi *throughput*, yaitu *lead time* perubahan, frekuensi *deployment*, dan waktu pemulihan *deployment* gagal, serta ketidakstabilan, yaitu tingkat kegagalan perubahan dan tingkat pengerjaan ulang [46]. Keluarga kedua adalah SLO untuk perjalanan pasien (Bab 11), yang mengukur keandalan dari sudut pandang pengguna [27]. Keluarga ketiga adalah biaya per unit (Bab 3), yang mengukur efisiensi. Keluarga keempat adalah jejak karbon dengan Software Carbon Intensity (SCI), yang dinyatakan per unit fungsi seperti per janji temu [77].

Tabel 14.10 adalah kartu metrik yang diusulkan untuk direksi Medikara. Setiap baris punya nilai awal dari bab sebelumnya dan sasaran per gerbang keputusan.

Tabel 14.10: Kartu metrik peta jalan Medikara Health (sasaran rekaan)

Metrik	Nilai awal	Gerbang 1, September 2027	Gerbang 2, September 2028
DORA: *lead time* perubahan Medikara+	29 hari (Bab 13)	14 hari	Di bawah 7 hari
DORA: tingkat kegagalan perubahan SIMRS Jakarta	43 persen (Bab 13)	30 persen	Di bawah 15 persen
SLO: janji temu terkonfirmasi dalam 5 detik	Sekitar 97,4 persen (Bab 9)	98,5 persen	99,5 persen (Bab 11)
Biaya infrastruktur SIMRS per kunjungan, RS Bekasi	Rp5.619 (Bab 3)	Rp5.619	Paling tinggi Rp4.776 pada akhir 2028
Biaya Medikara+ per janji temu	Rp2.036 (Bab 3)	Paling tinggi Rp2.240 selama transisi	Paling tinggi Rp2.036
SCI Medikara+ per janji temu	58,2 gram (Bab 3)	52 gram	Paling tinggi 44,8 gram

Beberapa sasaran sengaja tidak membaik pada gerbang 1. Biaya per janji temu dibiarkan naik sampai Rp2.240 karena selama transisi Medikara membayar dua jalur. Biaya SIMRS Bekasi belum turun karena Rehost baru dikerjakan pada gelombang 2. Sasaran yang jujur tentang masa sulit lebih mudah dipertahankan di depan direksi daripada sasaran yang meleset tanpa penjelasan.

14.7.3 Laporan Ringkas untuk Direksi

Laporan kemajuan untuk direksi cukup satu halaman setiap triwulan, dengan tiga bagian. Bagian pertama adalah status setiap gelombang dalam tiga warna: hijau sesuai rencana, kuning terlambat tetapi dapat dikejar, merah perlu keputusan. Bagian kedua adalah kartu metrik dengan tren tiga triwulan terakhir, bukan hanya angka terbaru. Bagian ketiga adalah tiga risiko teratas dan keputusan yang diminta dari direksi, misalnya menambah anggaran pelatihan atau menunda satu rumah sakit. Penyusunan KPI (Key Performance Indicator) dan pelaporan direksi secara

menyeluruh dibahas rinci di IT40813.

Dua contoh kasus:

1. Misalkan pada triwulan IV 2027 status gelombang 2 kuning karena Bogor tertunda empat minggu. Laporan menyebut penyebabnya: SLO janji temu Depok baru 99,1 persen, di bawah sasaran 99,5 persen, karena antrean pesan saat pagi sibuk. Keputusan yang diminta hanya satu: menyetujui penundaan Bogor, bukan menambah anggaran.
2. Misalkan direktur keuangan menanyakan mengapa tagihan *cloud* naik dari Rp240 juta menjadi Rp290 juta per bulan. Kartu metrik menjawabnya: janji temu naik 18 persen, biaya per janji temu tetap di bawah Rp2.240, dan biaya tak teralokasi hanya 4 persen. Kenaikan tagihan dapat dijelaskan, sehingga pembahasan bergeser dari "mengapa mahal" ke "apakah nilainya sepadan".

14.8 Aktivitas: Peta Jalan Modernisasi dan Guardrail Tata Kelola

Tujuan. Tim menyatukan artefak Bab 1 sampai Bab 13 menjadi peta jalan modernisasi tiga gelombang dan menetapkan *guardrail* tata kelola *cloud*, lalu mempresentasikannya sebagai bagian C4. Tim memakai organisasi nyata dengan izin tertulis dan data yang dianonimkan, atau memakai kasus Medikara Health. Data pribadi pasien atau karyawan tidak dipakai sama sekali.

Pembagian peran. Peta jalan harus masuk akal dari dua sisi: urutan nilai bisnis dan kelayakan teknis. Tabel 14.11 membagi peran agar setiap anggota menyumbang sesuai latar belakangnya.

Tabel 14.11: Pembagian peran dalam Aktivitas Bab 14

Peran	Diisi oleh	Tugas
Penyusun narasi nilai	Anggota non-teknis	Memimpin urutan prioritas, sasaran nilai, dan jawaban atas pertanyaan direksi tentang manfaat
Pemeriksa kelayakan	Anggota teknis	Memeriksa dependensi, prasyarat platform, dan kelayakan setiap gelombang
Perancang *guardrail*	Satu anggota teknis dan satu anggota non-teknis	Merumuskan *guardrail* dalam bahasa biasa, lalu memeriksa cara penegakannya
Penjaga konsistensi	Satu anggota yang ditunjuk	Memastikan angka di peta jalan sama dengan angka di artefak C1 sampai C3
Penyaji	Seluruh anggota	Menyiapkan presentasi dan latihan tanya jawab

Langkah kerja.

1. Kumpulkan artefak C1 sampai C3 dan dua artefak C4 dari Bab 12 dan Bab 13, lalu susun tabel keputusan seperti Tabel 14.3. Tandai keputusan yang saling bertentangan.
2. Susun tiga gelombang di `sources/chapter14/templat-peta-jalan-modernisasi.xlsx`. Setiap gelombang memuat layanan yang dimodernisasi, strategi 7R, prasyarat platform dan tim, sasaran SLO dan biaya, risiko utama, serta kriteria gerbang keputusan.
3. Pilih satu irisan pertama dengan lima kriteria pada Tabel 14.2, lalu tulis urutan pemindahannya dengan pola *strangler fig*.
4. Tetapkan lima sampai tujuh *guardrail* untuk biaya, keamanan, dan lokasi data, memakai `sources/chapter14/katalog-guardrail.md`. Setiap *guardrail* punya rumusan bahasa biasa, jenis kendali, risiko yang dicegah, pemilik, dan jalur pengecualian.
5. Pilih empat metrik kemajuan, satu dari setiap keluarga metrik, lengkap dengan nilai awal dan sasaran per gerbang.
6. Siapkan presentasi C4 menjelang UAS dengan `sources/chapter14/templat-presentasi-c4.md`: 15 menit presentasi dan 10 menit tanya jawab, dengan dosen dan tim lain berperan sebagai direksi.

Luaran dan kaitan dengan capstone. Luarannya adalah peta jalan modernisasi tiga gelombang dan *guardrail* tata kelola *cloud*. Bersama rencana resiliensi dan DR (Bab 12) serta model operasi *delivery* (Bab 13), luaran ini melengkapi tonggak C4 Peta Jalan dan Tata Kelola yang dipresentasikan menjelang UAS. Peta jalan ini juga menjadi bagian teknis dari peta jalan *enterprise* yang disusun di IT40513.

Kriteria penilaian singkat. Tabel 14.12 memuat bobot penilaian.

Tabel 14.12: Kriteria penilaian Aktivitas Bab 14

Kriteria	Bobot
Urutan gelombang masuk akal dari sisi dependensi, risiko, dan nilai	25%
Konsistensi angka dan keputusan dengan artefak C1 sampai C3	20%
Guardrail jelas, dapat ditegakkan, punya pemilik dan jalur pengecualian	20%
Metrik kemajuan mengukur hasil, dengan nilai awal dan sasaran	20%
Presentasi dan jawaban atas pertanyaan direksi	15%

Berkas pendukung. Seluruh berkas ada di `sources/chapter14/`, dengan daftar dan kegunaannya di `sources/chapter14/README.md`.

Pendalaman teknis (opsional). Anggota teknis menulis satu kebijakan *policy as code* dengan Open Policy Agent yang menolak sumber daya di luar *region* yang diizinkan, berangkat dari `sources/chapter14/contoh-opa/`. Kebijakan diuji dengan berkas masukan rekaan dan hanya dijalankan pada akun milik tim sendiri.

Bagian ini tidak dinilai sebagai syarat C4.

14.9 Ringkasan

Bab ini menutup mata kuliah dengan satu pertanyaan direksi: apa yang dimodernisasi lebih dulu, berapa biayanya, dan bagaimana risikonya dikendalikan. Jawabannya bukan satu proyek besar, tetapi rangkaian langkah kecil yang dapat dibalik. Pola *strangler fig* memasang fasad di depan sistem lama, memindahkan fungsi sedikit demi sedikit, lalu mematikan sistem lama setelah tidak ada yang bergantung padanya. *Anticorruption layer* menerjemahkan model data selama masa transisi, sehingga sistem baru tidak mewarisi istilah dan format lama. Irisan pertama dipilih dengan lima kriteria; bagi Medikara, janji temu dipindah per rumah sakit dari Depok sampai Jakarta.

Irisan-irisan itu disusun menjadi gelombang menurut dependensi, risiko, dan nilai, dengan prasyarat platform dan tim di setiap gelombang. Medikara menyusun tiga gelombang dalam 42 bulan: fondasi dan pindah cepat, irisan *cloud-native* pertama, lalu SIMRS terpadu. Gerbang keputusan di antara gelombang mengubah peta jalan dari janji tanggal menjadi rangkaian keputusan berbasis bukti, dan membatasi kerugian terburuk pada satu gelombang.

Guardrail menjaga biaya, keamanan, dan lokasi data tanpa menjadikan setiap perubahan antrean persetujuan. Kendali proaktif, preventif, dan detektif membaca satu katalog kebijakan yang ditulis sebagai kode. FinOps pada fase Operate melengkapinya dengan anggaran berambang, deteksi anomali, tag wajib, dan tinjauan biaya per unit yang dibawa ke direksi lewat Executive Strategy Alignment.

Kedaulatan data lebih luas dari lokasi penyimpanan. Kendali atas data, operasi, dan teknologi diterapkan seimbang dengan kebutuhan setiap *workload*, karena makin ketat kendalinya makin tinggi biayanya. Regulasi Indonesia per Oktober 2026 masih bergerak, termasuk PP 33/2026 yang berlaku Januari 2027 dan rancangan Perpres tentang pusat data, sehingga pemantauan regulasi menjadi butir tetap di setiap gerbang.

Kemajuan peta jalan diukur dengan hasil, bukan kegiatan. Jumlah aplikasi yang dipindahkan mudah dihitung tetapi menyesatkan. Metrik DORA, SLO, biaya per unit, dan jejak karbon menunjukkan apakah modernisasi membuat pelayanan lebih cepat, lebih andal, lebih efisien, dan lebih hemat energi. Pesan utama bab ini, sekaligus mata kuliah ini: modernisasi *cloud-native* adalah rangkaian keputusan yang dapat dibalik, dijaga oleh batas yang jelas, dan dipertanggungjawabkan dengan angka.

14.10 Latihan

Latihan 1: Strangler Fig dibandingkan dengan Penggantian Sekaligus

Latihan ini membuktikan bahwa pola *strangler fig* lebih aman daripada mengganti SIMRS sekaligus dalam satu kali peralihan, dengan menghitung dampak kegagalan, waktu kembali, dan biaya transisi.

1. Pilih satu fungsi SIMRS selain janji temu, misalnya klaim BPJS Kesehatan atau pendaftaran rawat jalan.
2. Perkirakan jumlah transaksi fungsi itu per hari di setiap rumah sakit.
3. Hitung dampak bila peralihan gagal pada hari pertama untuk kedua cara: seluruh rumah sakit sekaligus, atau satu rumah sakit pertama.
4. Perkirakan waktu kembali ke sistem lama, waktu sampai nilai pertama terasa, dan biaya masa transisi untuk kedua cara.
5. Isi Tabel 14.14 dengan mengikuti contoh pada Tabel 14.13.

Tabel 14.13: Latihan 1, contoh terisi: fungsi janji temu Medikara Health (angka rekaan)

Aspek	**Penggantian sekaligus**	***Strangler fig***
Transaksi terdampak bila gagal pada hari pertama	Sekitar 933 janji temu per hari di lima rumah sakit ($28.000/30$)	Sekitar 103 janji temu per hari di Depok
Waktu kembali ke sistem lama	1 sampai 3 hari: pulihkan basis data, ketik ulang data baru	Sekitar 10 menit: ubah aturan rute di fasad
Nilai pertama terasa	Setelah seluruh pekerjaan selesai, misalkan bulan ke-12	Bulan ke-2, setelah Depok pindah
Biaya masa transisi	Kecil, tanpa fasad dan lapisan penerjemah	Misalkan Rp25 juta per bulan selama 7 bulan, sekitar Rp175 juta
Jumlah titik keputusan	1	7: mode bayangan, lima rumah sakit, penonaktifan modul

Tabel 14.14: Latihan 1, lembar isian

Aspek	**Penggantian sekaligus**	***Strangler fig***
Fungsi yang dipilih	…	…
Transaksi terdampak bila gagal pada hari pertama	…	…
Waktu kembali ke sistem lama	…	…
Nilai pertama terasa	…	…
Biaya masa transisi	…	…
Jumlah titik keputusan	…	…

Pertanyaan analisis:

1. Berapa kali lebih banyak transaksi yang terdampak pada penggantian sekaligus dibandingkan langkah pertama *strangler fig* untuk fungsi pilihan? Apakah biaya transisi sepadan dengan selisih itu?
2. Pada fungsi pilihan, adakah kondisi yang membuat *strangler fig* tidak cocok, misalnya permintaan yang tidak dapat dicegat? Bila ada, apa penggantinya?

Latihan 2: Tiga Guardrail untuk Medikara Health

Latihan ini membuktikan bahwa *guardrail* yang baik dapat dibaca direksi, dapat ditegakkan otomatis, dan mencegah risiko yang dapat disebutkan dengan jelas.

1. Baca Tabel 14.6 dan pilih tiga risiko Medikara yang belum tertutup, masing-masing satu untuk biaya, keamanan, dan lokasi data.
2. Rumuskan satu *guardrail* untuk setiap risiko dalam satu kalimat bahasa biasa.
3. Tentukan jenis kendalinya: proaktif, preventif, detektif, atau gabungannya.
4. Tuliskan bukti kepatuhan yang dapat ditunjukkan kepada auditor, pemilik, dan jalur pengecualiannya.
5. Isi Tabel 14.16 dengan mengikuti contoh pada Tabel 14.15.

Tabel 14.15: Latihan 2, contoh terisi: satu *guardrail* biaya untuk lingkungan non-produksi (angka rekaan)

Butir	Isi
Rumusan bahasa biasa	Lingkungan non-produksi dimatikan otomatis pukul 20.00 sampai 07.00 dan sepanjang akhir pekan, kecuali bertag pengecualian yang masih berlaku
Kategori	Biaya
Jenis kendali	Proaktif (tag jadwal wajib saat dibuat) dan detektif (pemindaian harian)
Risiko yang dicegah	Mesin uji menyala 168 jam per minggu padahal dipakai sekitar 55 jam; misalkan pemborosan Rp12 juta per bulan dari anggaran non-produksi Rp35 juta
Bukti kepatuhan	Laporan harian jam nyala per akun; porsi sumber daya bertag jadwal
Pemilik	Kepala tim platform
Jalur pengecualian	Diajukan pemilik layanan, disetujui wakil keuangan CCoE, berlaku paling lama 14 hari

Pertanyaan analisis:

1. *Guardrail* mana yang paling mungkin memicu permintaan pengecualian? Berapa banyak pengecualian per bulan yang menandakan rumusannya terlalu ketat?

Tabel 14.16: Latihan 2, lembar isian

Butir	*Guardrail* biaya	*Guardrail* keamanan	*Guardrail* lokasi data
Rumusan bahasa biasa	…	…	…
Jenis kendali	…	…	…
Risiko yang dicegah	…	…	…
Bukti kepatuhan	…	…	…
Pemilik	…	…	…
Jalur pengecualian	…	…	…

2. Adakah *guardrail* yang hanya dapat ditegakkan secara detektif? Berapa lama celah antara pelanggaran dan temuan, dan apakah celah itu dapat diterima untuk risikonya?

Latihan 3: Empat Metrik dibandingkan dengan Jumlah Aplikasi

Latihan ini membuktikan bahwa jumlah aplikasi yang sudah dipindahkan dapat naik pesat sementara hasil yang dirasakan pasien dan direksi tidak berubah.

1. Baca data triwulan pada Tabel 14.17. Data ini skenario pengandaian yang sengaja meleset dari sasaran pada Tabel 14.10, agar dapat dianalisis.
2. Hitung persentase aplikasi yang sudah dipindahkan pada setiap triwulan dari 46 aplikasi.
3. Bandingkan arah perubahan jumlah aplikasi dengan arah perubahan empat metrik lainnya.
4. Pilih empat metrik untuk laporan direksi, satu dari setiap keluarga, dan tentukan sasarannya.
5. Isi Tabel 14.18.

Tabel 14.17: Latihan 3, contoh terisi: empat triwulan pertama peta jalan Medikara Health, April 2027 sampai Maret 2028 (angka rekaan)

Metrik	Triwulan I	Triwulan II	Triwulan III	Triwulan IV
Aplikasi dipindahkan, kumulatif	12	31	32	34
Lead time perubahan Medikara+ (hari)	29	27	12	6
Janji temu terkonfirmasi dalam 5 detik (%)	97,6	97,1	98,8	99,4
Biaya Medikara+ per janji temu (rupiah)	2.036	2.334	2.240	2.009
SCI Medikara+ per janji temu (gram)	58,2	59,7	52,3	46,4

Pertanyaan analisis:

1. Pada triwulan II jumlah aplikasi naik dari 12 menjadi 31, tetapi tiga metrik

Tabel 14.18: Latihan 3, lembar isian

Keluarga metrik	Metrik yang dipilih	Nilai awal	Sasaran
DORA	…	…	…
SLO	…	…	…
Biaya per unit	…	…	…
Jejak karbon	…	…	…
Alasan tidak memakai jumlah aplikasi	…		

lain memburuk. Apa penjelasan yang paling mungkin, dan bagaimana menyampaikannya kepada direksi?

2. Pada triwulan III dan IV jumlah aplikasi hampir tidak bergerak, tetapi metrik lain membaik pesat. Laporan mana yang lebih jujur menggambarkan kemajuan, dan mengapa?
3. Metrik mana yang paling sulit diukur di organisasi pilihan tim, dan data apa yang perlu dikumpulkan lebih dulu?

Bibliografi

[1] Allen & Gledhill. Indonesia: Data protection regulation 33 introduces new operational requirements. Knowledge Update, ringkasan firma hukum atas PP 33/2026, 2026. URL https://www.allenandgledhill.com/perspectives/publications/knowledge-updates-indonesia/2026/indonesia-data-protection-regulation-33-introduces-new-operational-requirements/. Teks resmi PP 33/2026 perlu dicek di JDIH. Diakses 30 Oktober 2026.

[2] Amazon Web Services. An overview of the AWS cloud adoption framework. AWS Whitepaper, 2021. URL https://docs.aws.amazon.com/whitepapers/latest/overview-aws-cloud-adoption-framework/welcome.html. Diakses 30 Oktober 2026.

[3] Amazon Web Services. Now open: AWS asia pacific (Jakarta) region. AWS News Blog, 2021. URL https://aws.amazon.com/blogs/aws/now-open-aws-asia-pacific-jakarta-region/. Diakses 30 Oktober 2026.

[4] Amazon Web Services. Foundational capabilities. An Overview of the AWS Cloud Adoption Framework, AWS Whitepaper, 2021. URL https://docs.aws.amazon.com/whitepapers/latest/overview-aws-cloud-adoption-framework/foundational-capabilities.html. Diakses 30 Oktober 2026.

[5] Amazon Web Services. Disaster recovery of workloads on AWS: Recovery in the cloud. AWS Whitepaper, terbit 12 Februari 2021, 2021. URL https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-workloads-on-aws.html. Diakses 30 Oktober 2026.

[6] Amazon Web Services. Summary of the AWS service event in the northern virginia (US-EAST-1) region. AWS, 10 Desember 2021, 2021. URL https://aws.amazon.com/message/12721/. Diakses 30 Oktober 2026.

[7] Amazon Web Services. REL10-BP01 deploy the workload to multiple locations. Reliability Pillar, AWS Well-Architected Framework, 2024.

URL https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_fault_isolation_multiaz_region_system.html. Diakses 30 Oktober 2026.

[8] Amazon Web Services. Reliability pillar: AWS well-architected framework. AWS Whitepaper, terbit 6 November 2024, 2024. URL https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html. Diakses 30 Oktober 2026.

[9] Amazon Web Services. About the migration strategies. AWS Prescriptive Guidance: Guide for AWS Large Migrations, 2026. URL https://docs.aws.amazon.com/prescriptive-guidance/latest/large-migration-guide/migration-strategies.html. Diakses 30 Oktober 2026.

[10] Amazon Web Services. AWS well-architected framework: The pillars of the framework. AWS Documentation, 2026. URL https://docs.aws.amazon.com/wellarchitected/latest/framework/the-pillars-of-the-framework.html. Diakses 30 Oktober 2026.

[11] Amazon Web Services. AWS services by region. AWS Global Infrastructure, 2026. URL https://aws.amazon.com/about-aws/global-infrastructure/regional-product-services/. Diakses 30 Oktober 2026.

[12] Amazon Web Services. Regions and availability zones. AWS Global Infrastructure, 2026. URL https://aws.amazon.com/about-aws/global-infrastructure/regions_az/. Diakses 30 Oktober 2026.

[13] Amazon Web Services. Shared responsibility model. Halaman kepatuhan AWS, 2026. URL https://aws.amazon.com/compliance/shared-responsibility-model/. Diakses 30 Oktober 2026.

[14] Amazon Web Services. Calling AWS service, region, endpoint, and availability zone public parameters. AWS Systems Manager User Guide, 2026. URL https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-public-parameters-global-infrastructure.html. Diakses 30 Oktober 2026.

[15] Amazon Web Services. Savings plans. Halaman produk AWS, 2026. URL https://aws.amazon.com/savingsplans/. Diakses 30 Oktober 2026.

[16] Amazon Web Services. Amazon EC2 spot instances. Halaman produk AWS, 2026. URL https://aws.amazon.com/ec2/spot/. Diakses 30 Oktober 2026.

[17] Amazon Web Services. Spot instance interruption notices. Amazon EC2 User Guide, 2026. URL https://docs.aws.amazon.com/AWSEC2/

latest/UserGuide/spot-instance-termination-notices.html. Diakses 30 Oktober 2026.

[18] Amazon Web Services. Sustainability pillar: AWS well-architected framework. AWS Whitepaper, 2026. URL https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html. Diakses 30 Oktober 2026.

[19] Amazon Web Services. Amazon EKS pricing. aws.amazon.com, 2026. URL https://aws.amazon.com/eks/pricing/. Diakses 30 Oktober 2026.

[20] Amazon Web Services. AWS lambda pricing. aws.amazon.com, 2026. URL https://aws.amazon.com/lambda/pricing/. Diakses 30 Oktober 2026.

[21] Amazon Web Services. Lambda quotas: AWS lambda developer guide. docs.aws.amazon.com, 2026. URL https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html. Diakses 30 Oktober 2026.

[22] Amazon Web Services. About controls in AWS control tower. AWS Control Tower Controls Reference Guide, 2026. URL https://docs.aws.amazon.com/controltower/latest/controlreference/controls.html. Diakses 30 Oktober 2026.

[23] ANTARA. Menkominfo jelaskan kronologi serangan siber PDNS 2. ANTARA News Kalimantan Selatan, 27 Juni 2024, 2024. URL https://kalsel.antaranews.com/berita/419220/menkominfo-jelaskan-kronologi-serangan-siber-pdns-2. Diakses 30 Oktober 2026.

[24] Argo Project. Argo CD documentation: Overview. Argo CD Documentation, versi stable, 2026. URL https://argo-cd.readthedocs.io/en/stable/. Diakses 30 Oktober 2026.

[25] AsyncAPI Initiative. AsyncAPI specification 3.0.0. AsyncAPI Initiative, 2023. URL https://www.asyncapi.com/docs/reference/specification/v3.0.0. Diakses 30 Oktober 2026.

[26] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016. doi: 10.1109/MS.2016.60. URL https://doi.org/10.1109/MS.2016.60.

[27] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, Sebastopol, CA, 2016. URL https://sre.google/sre-book/table-of-contents/. Diakses 30 Oktober 2026.

[28] Betsy Beyer, Niall Richard Murphy, David K. Rensin, Kent Kawahara, and

Stephen Thorne, editors. *The Site Reliability Workbook: Practical Ways to Implement SRE*. O'Reilly Media, Sebastopol, CA, 2018. ISBN 9781492029502. URL https://sre.google/workbook/table-of-contents/. Diakses 30 Oktober 2026.

[29] Alberto Brandolini. *Introducing EventStorming: An Act of Deliberate Collective Learning*. Leanpub, 2021. URL https://leanpub.com/introducing_eventstorming. Terbit bertahap, pembaruan terakhir 26 Agustus 2021. Diakses 30 Oktober 2026.

[30] Marc Brooker. Timeouts, retries, and backoff with jitter. Amazon Builders' Library, 2019. URL https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/. Diakses 30 Oktober 2026.

[31] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5): 50–57, 2016. doi: 10.1145/2890784. URL https://doi.org/10.1145/2890784. Diakses 30 Oktober 2026.

[32] Cloud Native Computing Foundation. Kubernetes established as the de facto operating system for AI as production use hits 82% in 2025 CNCF annual cloud native survey. Siaran pers, 20 Januari 2026, 2026. URL https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/. Diakses 30 Oktober 2026.

[33] Cloud Native Computing Foundation. The CNCF annual cloud native survey: The infrastructure of AI's future. Laporan survei, Cloud Native Computing Foundation, 2026. URL https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/. Diakses 30 Oktober 2026.

[34] CloudEvents. CloudEvents specification, version 1.0.2. Cloud Native Computing Foundation, 2022. URL https://cloudevents.io/. Diakses 30 Oktober 2026.

[35] CNCF Serverless Working Group. CNCF serverless whitepaper v1.0. Cloud Native Computing Foundation, 2018. URL https://github.com/cncf/wg-serverless/tree/master/whitepapers/serverless-overview. Diakses 30 Oktober 2026.

[36] CNCF TAG App Delivery Platforms Working Group. Platform engineering maturity model, version 1. Cloud Native Computing Foundation, 2023. URL https://tag-app-delivery.cncf.io/whitepapers/platform-eng-maturity-model/. Diakses 30 Oktober 2026.

[37] CNCF TAG App Delivery Platforms Working Group. CNCF platforms white paper, version 1.0. Cloud Native Computing Foundation, 2023. URL https://tag-app-delivery.cncf.io/whitepapers/platforms/. Diakses 30 Oktober 2026.

[38] CNCF Technical Oversight Committee. CNCF cloud native definition v1.1. Cloud Native Computing Foundation, disetujui 26 Februari 2024, 2024. URL https://github.com/cncf/toc/blob/main/DEFINITION.md. Diakses 30 Oktober 2026.

[39] CNN Indonesia. PDNS 2 diserang hacker, 3 layanan pulih dari total 282 korban. CNN Indonesia, 26 Juni 2024, 2024. URL https://www.cnnindonesia.com/teknologi/20240626015537-192-1114144/pdns-2-diserang-hacker-3-layanan-pulih-dari-total-282-korban. Diakses 30 Oktober 2026.

[40] Melvin E. Conway. How do committees invent? *Datamation*, 14(4):28–31, 1968. URL https://www.melconway.com/Home/Committees_Paper.html. Diakses 30 Oktober 2026.

[41] Cybersecurity and Infrastructure Security Agency. #StopRansomware guide. CISA, versi September 2023, 2023. URL https://www.cisa.gov/stopransomware/ransomware-guide. Diakses 30 Oktober 2026.

[42] DDD Crew. Context mapping. GitHub, ddd-crew/context-mapping, 2026. URL https://github.com/ddd-crew/context-mapping. Diakses 30 Oktober 2026.

[43] DDD Crew. Eventstorming glossary & cheat sheet. GitHub, ddd-crew/eventstorming-glossary-cheat-sheet, 2026. URL https://github.com/ddd-crew/eventstorming-glossary-cheat-sheet. Diakses 30 Oktober 2026.

[44] Docker Inc. Dockerfile reference. Docker Docs, 2026. URL https://docs.docker.com/reference/dockerfile/. Diakses 30 Oktober 2026.

[45] DORA. State of AI-assisted software development 2025. Laporan riset, Google Cloud, 2025. URL https://dora.dev/research/2025/dora-report/. Diakses 30 Oktober 2026.

[46] DORA. DORA's software delivery performance metrics. DORA Guides, diperbarui 5 Januari 2026, 2026. URL https://dora.dev/guides/dora-metrics/. Diakses 30 Oktober 2026.

[47] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston, MA, 2003. ISBN 9780321125217.

[48] FinOps Foundation. FOCUS: FinOps open cost and usage specification,

version 1.4. FinOps Foundation, Linux Foundation, 2026. URL https://focus.finops.org/focus-specification/. Diakses 30 Oktober 2026.

[49] FinOps Foundation. Allocation. FinOps Framework Capability, domain Understand Usage and Cost, 2026. URL https://www.finops.org/framework/capabilities/allocation/. Diakses 30 Oktober 2026.

[50] FinOps Foundation. Anomaly management. FinOps Framework Capability, domain Understand Usage and Cost, 2026. URL https://www.finops.org/framework/capabilities/anomaly-management/. Diakses 30 Oktober 2026.

[51] FinOps Foundation. Budgeting. FinOps Framework Capability, domain Quantify Business Value, 2026. URL https://www.finops.org/framework/capabilities/budgeting/. Diakses 30 Oktober 2026.

[52] FinOps Foundation. Executive strategy alignment. FinOps Framework Capability, domain Manage the FinOps Practice, 2026. URL https://www.finops.org/framework/capabilities/executive-strategy-alignment/. Diakses 30 Oktober 2026.

[53] FinOps Foundation. FinOps framework 2026. FinOps Foundation, Linux Foundation, 2026. URL https://www.finops.org/framework/. Diakses 30 Oktober 2026.

[54] Flexera. Flexera releases 2020 state of the cloud report. Siaran pers Flexera, 28 April 2020, 2020. URL https://www.flexera.com/about-us/press-center/flexera-releases-2020-state-of-the-cloud-report.html. Diakses 30 Oktober 2026.

[55] Flexera. 2026 state of the cloud report. Laporan survei, Flexera Software, 2026. URL https://info.flexera.com/CM-REPORT-State-of-the-Cloud. Survei terhadap 753 pengambil keputusan cloud, dirilis 18 Maret 2026. Ringkasan temuan: siaran pers "Flexera Finds Cloud Value is Rising While AI Waste Grows", https://www.flexera.com/about-us/press-center/flexera-finds-cloud-value-is-rising-while-ai-waste-grows. Diakses 30 Oktober 2026.

[56] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, Portland, OR, 2018. ISBN 9781942788331. URL https://itrevolution.com/product/accelerate/. Diakses 30 Oktober 2026.

[57] Martin Fowler. Ubiquitous language. martinfowler.com, Bliki, 2006. URL https://martinfowler.com/bliki/UbiquitousLanguage.html. Diakses 30 Oktober 2026.

[58] Martin Fowler. CQRS. martinfowler.com, 14 Juli 2011, 2011. URL https://martinfowler.com/bliki/CQRS.html. Diakses 30 Oktober 2026.

[59] Martin Fowler. Snowflake server. martinfowler.com, Bliki, 2012. URL https://martinfowler.com/bliki/SnowflakeServer.html. Diakses 30 Oktober 2026.

[60] Martin Fowler. Bounded context. martinfowler.com, Bliki, 2014. URL https://martinfowler.com/bliki/BoundedContext.html. Diakses 30 Oktober 2026.

[61] Martin Fowler. MonolithFirst. martinfowler.com, 2015. URL https://martinfowler.com/bliki/MonolithFirst.html. Diakses 30 Oktober 2026.

[62] Martin Fowler. What do you mean by "Event-Driven"? martinfowler.com, 2017. URL https://martinfowler.com/articles/201701-event-driven.html. Diakses 30 Oktober 2026.

[63] Martin Fowler. Strangler fig. martinfowler.com, 2024. URL https://martinfowler.com/bliki/StranglerFigApplication.html. Diakses 30 Oktober 2026.

[64] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259. ACM, 1987. doi: 10.1145/38713.38742. URL https://doi.org/10.1145/38713.38742.

[65] GitHub. Understanding GitHub actions. GitHub Docs, 2026. URL https://docs.github.com/en/actions/get-started/understand-github-actions. Diakses 30 Oktober 2026.

[66] Google Cloud. The Google cloud adoption framework. Google Cloud Whitepaper, 2018. URL https://services.google.com/fh/files/misc/google_cloud_adoption_framework_whitepaper.pdf. Diakses 30 Oktober 2026.

[67] Google Cloud. New Google Cloud region in Jakarta now open. Google Cloud Blog, 2020. URL https://cloud.google.com/blog/products/infrastructure/new-google-cloud-region-in-jakarta-now-open. Diakses 30 Oktober 2026.

[68] Google Cloud. Regions and zones. Dokumentasi Compute Engine, 2026. URL https://cloud.google.com/compute/docs/regions-zones. Diakses 30 Oktober 2026.

[69] Google Cloud. Google cloud well-architected framework. Google Cloud Documentation, pembaruan 13 Oktober 2026, 2026. URL https://

docs.cloud.google.com/architecture/framework. Diakses 30 Oktober 2026.

[70] Green Software Foundation. Software carbon intensity (SCI) specification. Green Software Foundation, 2024. URL https://sci.greensoftware.foundation/. Diakses 30 Oktober 2026.

[71] HL7 International. HL7 FHIR release 4 (4.0.1): Resource patient. Spesifikasi resmi HL7, 2019. URL https://hl7.org/fhir/R4/patient.html. Diakses 30 Oktober 2026.

[72] Kevin Hoffman. *Beyond the Twelve-Factor App: Exploring the DNA of Highly Scalable, Resilient Cloud Applications*. O'Reilly Media, Sebastopol, CA, 2016. ISBN 9781491944011.

[73] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, Boston, MA, 2003. ISBN 9780321200686. URL https://www.enterpriseintegrationpatterns.com/. Diakses 30 Oktober 2026.

[74] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. ISBN 9780321601919. URL https://www.informit.com/store/continuous-delivery-reliable-software-releases-through-9780321601919. Diakses 30 Oktober 2026.

[75] International Organization for Standardization. ISO 22301:2019 security and resilience: Business continuity management systems: Requirements. Edisi 2, Oktober 2019, 2019. URL https://www.iso.org/standard/75106.html. Diakses 30 Oktober 2026.

[76] ISO/IEC. ISO/IEC 25010:2023 systems and software engineering: Systems and software quality requirements and evaluation (SQuaRE): Product quality model. International Organization for Standardization, edisi ke-2, November 2023, 2023. URL https://www.iso.org/standard/78176.html. Diakses 30 Oktober 2026.

[77] ISO/IEC. ISO/IEC 21031:2024 information technology: Software carbon intensity (SCI) specification. International Organization for Standardization, 2024. URL https://www.iso.org/standard/86612.html. Diakses 30 Oktober 2026.

[78] Istio. The Istio service mesh. Dokumentasi Istio, 2026. URL https://istio.io/latest/about/service-mesh/. Diakses 30 Oktober 2026.

[79] Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James Lewis, and Stefan Tilkov. Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3):24–35, 2018. doi: 10.1109/MS.2018.2141039. URL

https://doi.org/10.1109/MS.2018.2141039.

[80] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. Cloud programming simplified: A Berkeley view on serverless computing. Technical Report UCB/EECS-2019-3, University of California, Berkeley, 2019. URL https://arxiv.org/abs/1902.03383. Diakses 30 Oktober 2026.

[81] Betty Junod. Heroku open sources the twelve-factor app definition. Heroku Blog, 12 November 2024, 2024. URL https://www.heroku.com/blog/heroku-open-sources-twelve-factor-app-definition. Diakses 30 Oktober 2026.

[82] Sergey Kanzhelev, Morgan McLean, Alois Reitbauer, Bogdan Drutu, Nik Molnar, and Yuri Shkuro. Trace context. World Wide Web Consortium Recommendation, 23 November 2021, 2021. URL https://www.w3.org/TR/trace-context/. Diakses 30 Oktober 2026.

[83] Kementerian Kesehatan Republik Indonesia. Peraturan menteri kesehatan nomor 24 tahun 2022 tentang rekam medis. Berita Negara Republik Indonesia Tahun 2022 Nomor 829, 2022. URL https://peraturan.bpk.go.id/Details/245544. Diakses 30 Oktober 2026.

[84] Kementerian Komunikasi dan Informatika Republik Indonesia. Peraturan menteri komunikasi dan informatika nomor 5 tahun 2020 tentang penyelenggara sistem elektronik lingkup privat. Berita Negara Republik Indonesia Tahun 2020 Nomor 1376, diubah dengan Permenkominfo Nomor 10 Tahun 2021, 2020. URL https://peraturan.bpk.go.id/Details/203049. Diakses 30 Oktober 2026.

[85] Vlad Khononov. *Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy*. O'Reilly Media, Sebastopol, CA, 2021. ISBN 9781098100131.

[86] Kubernetes SIG Testing. kind: Kubernetes in docker. kind.sigs.k8s.io, 2026. URL https://kind.sigs.k8s.io/. Diakses 30 Oktober 2026.

[87] James Lewis and Martin Fowler. Microservices: A definition of this new architectural term. martinfowler.com, 2014. URL https://martinfowler.com/articles/microservices.html. Diakses 30 Oktober 2026.

[88] Alan MacCormack, Carliss Baldwin, and John Rusnak. Exploring the duality between product and organizational architectures: A test of the "mirroring" hypothesis. *Research Policy*, 41(8):1309–1324, 2012. doi: 10.1016/j.respol.2012.04.011. URL https://doi.org/10.1016/j.respol.2012.04.011.

[89] Charity Majors, Liz Fong-Jones, and George Miranda. *Observability Engineering: Achieving Production Excellence*. O'Reilly Media, Sebastopol, CA, 2022. ISBN 9781492076445. URL https://www.oreilly.com/library/view/observability-engineering/9781492076438/. Diakses 30 Oktober 2026.

[90] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, Upper Saddle River, NJ, 2003. ISBN 9780135974445.

[91] Peter Mell and Timothy Grance. The NIST definition of cloud computing. NIST Special Publication 800-145, National Institute of Standards and Technology, 2011. URL https://doi.org/10.6028/NIST.SP.800-145. Diakses 30 Oktober 2026.

[92] Microsoft. Circuit breaker pattern. Azure Architecture Center, Microsoft Learn, versi 5 Februari 2025, 2025. URL https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker. Diakses 30 Oktober 2026.

[93] Microsoft. Architecture strategies for disaster recovery. Microsoft Azure Well-Architected Framework, Microsoft Learn, versi 19 November 2025, 2025. URL https://learn.microsoft.com/en-us/azure/well-architected/reliability/disaster-recovery. Diakses 30 Oktober 2026.

[94] Microsoft. Build a cloud governance team. Cloud Adoption Framework for Azure, Microsoft Learn, 2025. URL https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/govern/build-cloud-governance-team. Diakses 30 Oktober 2026.

[95] Microsoft. Cloud adoption framework for Azure. Microsoft Learn, pembaruan 26 Agustus 2026, 2026. URL https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/overview. Diakses 30 Oktober 2026.

[96] Microsoft. Shared responsibility in the cloud. Microsoft Learn, Azure Security Fundamentals, 2026. URL https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility. Diakses 30 Oktober 2026.

[97] Microsoft. What is an Azure landing zone? Cloud Adoption Framework for Azure, Microsoft Learn, 2026. URL https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/. Diakses 30 Oktober 2026.

[98] Microsoft. Bulkhead pattern. Azure Architecture Center, Microsoft Learn, versi 19 Maret 2026, 2026. URL https://learn.microsoft.com/en-us/

azure/architecture/patterns/bulkhead. Diakses 30 Oktober 2026.

[99] Microsoft. Anti-corruption layer pattern. Azure Architecture Center, Microsoft Learn, 2026. URL https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer. Diakses 30 Oktober 2026.

[100] Microsoft. Digital sovereignty adoption. Cloud Adoption Framework for Azure, Microsoft Learn, 2026. URL https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/sovereignty/executive-strategy. Diakses 30 Oktober 2026.

[101] Microsoft. Strangler fig pattern. Azure Architecture Center, Microsoft Learn, 2026. URL https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig. Diakses 30 Oktober 2026.

[102] Microsoft Indonesia. Microsoft opens its first cloud region in Indonesia to unlock the new AI economy. Microsoft News Center Indonesia, 2025. URL https://news.microsoft.com/id-id/2025/05/27/microsoft-opens-indonesia-central/. Diakses 30 Oktober 2026.

[103] Barbara Minto. *The Pyramid Principle: Logic in Writing and Thinking*. Financial Times Prentice Hall, Harlow, third edition, 2009. ISBN 9780273710516.

[104] Kief Morris. Immutable server. martinfowler.com, Bliki, 2013. URL https://martinfowler.com/bliki/ImmutableServer.html. Diakses 30 Oktober 2026.

[105] Kief Morris. *Infrastructure as Code: Designing and Delivering Dynamic Systems for the Cloud Age*. O'Reilly Media, Sebastopol, CA, third edition, 2025. ISBN 9781098150358. URL https://www.oreilly.com/library/view/-/9781098150341/. Diakses 30 Oktober 2026.

[106] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA, second edition, 2021. ISBN 9781492034025.

[107] Alexandra Noonan. Goodbye microservices: From 100s of problem children to 1 superstar. Segment Engineering Blog, diterbitkan ulang oleh Twilio, 2018. URL https://www.twilio.com/en-us/blog/developers/best-practices/goodbye-microservices. Diakses 30 Oktober 2026.

[108] Michael Nygard. Documenting architecture decisions. Cognitect Blog, 2011. URL https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Diakses 30 Oktober 2026.

[109] Michael Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, Raleigh, NC, 2 edition, 2018. ISBN

9781680502398. URL https://pragprog.com/titles/mnee2/release-it-second-edition/. Diakses 30 Oktober 2026.

[110] Justice Opara-Martins, Reza Sahandi, and Feng Tian. Critical analysis of vendor lock-in and its impact on cloud computing migration: A business perspective. *Journal of Cloud Computing*, 5(4), 2016. doi: 10.1186/s13677-016-0054-z. URL https://doi.org/10.1186/s13677-016-0054-z. Diakses 30 Oktober 2026.

[111] Open Container Initiative. OCI distribution specification. Repositori spesifikasi di GitHub, 2024. URL https://github.com/opencontainers/distribution-spec. Diakses 30 Oktober 2026.

[112] Open Container Initiative. OCI image format specification. Repositori spesifikasi di GitHub, 2024. URL https://github.com/opencontainers/image-spec. Diakses 30 Oktober 2026.

[113] Open Policy Agent. Open policy agent (OPA) documentation. Dokumentasi resmi proyek CNCF, 2026. URL https://www.openpolicyagent.org/docs. Diakses 30 Oktober 2026.

[114] OpenGitOps. GitOps principles v1.0.0. CNCF GitOps Working Group, 2021. URL https://opengitops.dev/. Diakses 30 Oktober 2026.

[115] OpenTelemetry Authors. Signals. OpenTelemetry Documentation, 2026. URL https://opentelemetry.io/docs/concepts/signals/. Diakses 30 Oktober 2026.

[116] OpenTelemetry Authors. What is OpenTelemetry? OpenTelemetry Documentation, 2026. URL https://opentelemetry.io/docs/what-is-opentelemetry/. Diakses 30 Oktober 2026.

[117] Oracle. New region in Batam, Indonesia. Oracle Cloud Infrastructure Release Notes, 2025. URL https://docs.oracle.com/iaas/releasenotes/oci/new-region-batam.htm. Diakses 30 Oktober 2026.

[118] Oracle. Regions and availability domains. Oracle Cloud Infrastructure Documentation, 2026. URL https://docs.oracle.com/en-us/iaas/Content/General/Concepts/regions.htm. Diakses 30 Oktober 2026.

[119] Pemerintah Republik Indonesia. Peraturan pemerintah nomor 71 tahun 2019 tentang penyelenggaraan sistem dan transaksi elektronik. Lembaran Negara Republik Indonesia Tahun 2019 Nomor 185, 2019. URL https://peraturan.bpk.go.id/Details/122030/pp-no-71-tahun-2019. Diakses 30 Oktober 2026.

[120] Pemerintah Republik Indonesia. Undang-undang nomor 27 tahun 2022 tentang pelindungan data pribadi. Lembaran Negara Republik Indonesia

Tahun 2022 Nomor 196, 2022. URL https://peraturan.bpk.go.id/Details/229798/uu-no-27-tahun-2022. Diakses 30 Oktober 2026.

[121] Principles of Chaos Engineering. Principles of chaos engineering. pembaruan terakhir Maret 2019, 2019. URL https://principlesofchaos.org/. Diakses 30 Oktober 2026.

[122] RabbitMQ. Reliability guide. Dokumentasi RabbitMQ, 2026. URL https://www.rabbitmq.com/docs/reliability. Diakses 30 Oktober 2026.

[123] Republik Indonesia. Peraturan pemerintah nomor 28 tahun 2024 tentang peraturan pelaksanaan undang-undang nomor 17 tahun 2023 tentang kesehatan. Lembaran Negara Republik Indonesia Tahun 2024 Nomor 135, 2024. URL https://peraturan.bpk.go.id/Details/294077. Diakses 30 Oktober 2026.

[124] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: A Modern Engineering Approach*. O'Reilly Media, Sebastopol, CA, second edition, 2025. ISBN 9781098175511. URL https://www.oreilly.com/library/view/fundamentals-of-software/9781098175504/. Diakses 30 Oktober 2026.

[125] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island, NY, 2018. ISBN 9781617294549.

[126] Chris Richardson. Pattern: Database per service. microservices.io, 2026. URL https://microservices.io/patterns/data/database-per-service.html. Diakses 30 Oktober 2026.

[127] Chris Richardson. Pattern: Transactional outbox. microservices.io, 2026. URL https://microservices.io/patterns/data/transactional-outbox.html. Diakses 30 Oktober 2026.

[128] Chris Richardson. Pattern: Saga. microservices.io, 2026. URL https://microservices.io/patterns/data/saga.html. Diakses 30 Oktober 2026.

[129] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Laporan teknis, Google, Inc., 2010. URL https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/. Diakses 30 Oktober 2026.

[130] Novita Sari Simamora. Komdigi siapkan perpres data center dan SKKL, dorong pemerataan ke indonesia timur. Bisnis.com, 20 Oktober 2026, 2026. URL https://teknologi.bisnis.com/read/20261020/101/2014210 / komdigi - siapkan - perpres - data - center - dan - skkl -

dorong-pemerataan-ke-indonesia-timur. Diakses 30 Oktober 2026.

[131] Matthew Skelton and Manuel Pais. *Team Topologies: Organizing Business and Technology for Fast Flow of Value*. IT Revolution Press, Portland, OR, second edition, 2025. ISBN 9781966280002.

[132] SLSA. SLSA: Supply-chain levels for software artifacts. Situs resmi SLSA, spesifikasi v1.2, 2026. URL https://slsa.dev/. Diakses 30 Oktober 2026.

[133] Murugiah Souppaya, John Morello, and Karen Scarfone. Application container security guide. NIST Special Publication 800-190, National Institute of Standards and Technology, 2017. URL https://doi.org/10.6028/NIST.SP.800-190. Diakses 30 Oktober 2026.

[134] Wayne P. Stevens, Glenford J. Myers, and Larry L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974. doi: 10.1147/sj.132.0115. URL https://doi.org/10.1147/sj.132.0115.

[135] J. R. Storment and Mike Fuller. *Cloud FinOps: Collaborative, Real-Time Cloud Value Decision Making*. O'Reilly Media, Sebastopol, CA, second edition, 2023. ISBN 9781492098355. URL https://www.oreilly.com/library/view/cloud-finops-2nd/9781492098348/. Diakses 30 Oktober 2026.

[136] Marianne Swanson, Pauline Bowen, Amy Phillips, Dean Gallup, and David Lynes. Contingency planning guide for federal information systems. NIST Special Publication 800-34 Rev. 1, National Institute of Standards and Technology, 2010. URL https://doi.org/10.6028/NIST.SP.800-34r1.

[137] The Kubernetes Authors. Kubernetes object management. Kubernetes Documentation, 2026. URL https://kubernetes.io/docs/concepts/overview/working-with-objects/object-management/. Diakses 30 Oktober 2026.

[138] The Kubernetes Authors. Kubernetes components: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/concepts/overview/components/. Diakses 30 Oktober 2026.

[139] The Kubernetes Authors. Controllers: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/concepts/architecture/controller/. Diakses 30 Oktober 2026.

[140] The Kubernetes Authors. Deployments: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/concepts/workloads/controllers/deployment/. Diakses 30 Oktober 2026.

[141] The Kubernetes Authors. Horizontal pod autoscaling: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/. Diakses 30

Oktober 2026.

[142] The Kubernetes Authors. Overview: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/concepts/overview/. Diakses 30 Oktober 2026.

[143] The Kubernetes Authors. Pods: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/concepts/workloads/pods/. Diakses 30 Oktober 2026.

[144] The Kubernetes Authors. Releases dan kubernetes release cycle. kubernetes.io, 2026. URL https://kubernetes.io/releases/. Diakses 30 Oktober 2026.

[145] The Kubernetes Authors. Service: Kubernetes documentation. kubernetes.io, 2026. URL https://kubernetes.io/docs/concepts/services-networking/service/. Diakses 30 Oktober 2026.

[146] The Kubernetes Authors. minikube documentation. minikube.sigs.k8s.io, 2026. URL https://minikube.sigs.k8s.io/docs/. Diakses 30 Oktober 2026.

[147] Stefan Tilkov. Don't start with a monolith. martinfowler.com, 2015. URL https://martinfowler.com/articles/dont-start-monolith.html. Diakses 30 Oktober 2026.

[148] Vaughn Vernon. *Implementing Domain-Driven Design*. Addison-Wesley, Upper Saddle River, NJ, 2013. ISBN 9780321834577.

[149] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1): 40–44, 2009. doi: 10.1145/1435417.1435432. URL https://doi.org/10.1145/1435417.1435432. Diakses 30 Oktober 2026.

[150] Kirsten Westeinde. Deconstructing the monolith: Designing software that maximizes developer productivity. Shopify Engineering, 2019. URL https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity. Diakses 30 Oktober 2026.

[151] David Weston. Helping our customers through the CrowdStrike outage. The Official Microsoft Blog, 20 Juli 2024, 2024. URL https://blogs.microsoft.com/blog/2024/07/20/helping-our-customers-through-the-crowdstrike-outage/. Diakses 30 Oktober 2026.

[152] Adam Wiggins. The twelve-factor app. Dikelola sebagai proyek open source sejak 12 November 2024, 2011. URL https://12factor.net/. Diakses 30 Oktober 2026.

[153] Emir Yanwardhana. Backup pusat data nasional hanya 2%, begini dampaknya. CNBC Indonesia, 28 Juni 2024, 2024. URL https://www.cnbcindonesia.com/tech/20240628161650-37-550307/backup-pusat-data-nasional-hanya-2-begini-dampaknya. Diakses 30 Oktober 2026.